

LINE: Queueing Analysis Algorithms

Example Manual – C++ Edition

Last revision: October 4, 2026
QORE Lab, Imperial College London

Contents

1	About this manual	1
2	Basic models	2
2.1	Open queueing networks	2
2.1.1	mam_transient_mapmap1	2
2.1.2	oqn_basic	4
2.1.3	oqn_bmmapt	5
2.1.4	oqn_cs_routing	6
2.1.5	oqn_fourqueues	8
2.1.6	oqn_mapt	10
2.1.7	oqn_mapt_service	11
2.1.8	oqn_mmapt	13
2.1.9	oqn_multichain_cs	14
2.1.10	oqn_nhpp	15
2.1.11	oqn_online	17
2.1.12	oqn_trace_driven	18
2.1.13	oqn_vsinks	19
2.2	Closed queueing networks	20
2.2.1	cqn_bas_blocking	21
2.2.2	cqn_bcmp_theorem	22
2.2.3	cqn_lcfs_multiclass	23
2.2.4	cqn_mmpp2_service	24
2.2.5	cqn_multichain_cs	26
2.2.6	cqn_multiserver	27
2.2.7	cqn_multiserver_nc	30
2.2.8	cqn_online	31
2.2.9	cqn_repairmen	32
2.2.10	cqn_repairmen_multi	34
2.2.11	cqn_scheduling_dps	35
2.2.12	cqn_threeclass_hyperl	37
2.2.13	cqn_twoclass_erl	38
2.2.14	cqn_twoclass_hyperl	40
2.2.15	cqn_twoqueues_multi	41
2.3	Mixed networks	43
2.3.1	mqn_basic	43
2.3.2	mqn_multichain_cs	44
2.3.3	mqn_multiserver_fcfs	47
2.3.4	mqn_multiserver_ps	48
2.3.5	mqn_singleserver_fcfs	50

2.3.6	mqn_singleserver_ps	52
2.4	Class switching	54
2.4.1	cs_implicit	54
2.4.2	cs_multi_diamond	55
2.4.3	cs_single_diamond	56
2.4.4	cs_transient_class	58
2.5	Priority models	60
2.5.1	prio_hol_closed	61
2.5.2	prio_hol_open	62
2.5.3	prio_identical	64
2.5.4	prio_psprio	66
2.6	Fork-join networks	67
2.6.1	fj_asymm	67
2.6.2	fj_basic_closed	68
2.6.3	fj_basic_nesting	69
2.6.4	fj_basic_open	70
2.6.5	fj_complex_serial	72
2.6.6	fj_cs_multi_visits	73
2.6.7	fj_cs_postfork	74
2.6.8	fj_cs_prefork	75
2.6.9	fj_deep_nesting	76
2.6.10	fj_delays	78
2.6.11	fj_mixed_openclosed	79
2.6.12	fj_nojoin	80
2.6.13	fj_route_overlap	81
2.6.14	fj_serialfjs_closed	83
2.6.15	fj_serialfjs_open	84
2.6.16	fj_threebranches	85
2.6.17	fj_tiny_closed	86
2.6.18	fj_twoclasses_forked	88
2.6.19	fj_variable_fanout	89
2.7	Layered queueing networks	90
2.7.1	lqn_basic	90
2.7.2	lqn_bpmn	91
2.7.3	lqn_bpmn_trace	93
2.7.4	lqn_flatph	95
2.7.5	lqn_fork_open_arrival	97
2.7.6	lqn_init	98
2.7.7	lqn_jsq	100
2.7.8	lqn_moment3	101
2.7.9	lqn_multi_solvers	102
2.7.10	lqn_ofbiz	103
2.7.11	lqn_open_arrival	106
2.7.12	lqn_paramident	107
2.7.13	lqn_rrobin	108
2.7.14	lqn_serial	109
2.7.15	lqn_server_pools	110
2.7.16	lqn_setup	111
2.7.17	lqn_sockshop	112
2.7.18	lqn_srvnph	115

2.7.19	lqn_transient	116
2.7.20	lqn_twotasks	118
2.7.21	lqn_workflows	119
2.8	Cache models	121
2.8.1	cache_compare_replc	121
2.8.2	cache_itemsize_costcap	122
2.8.3	cache_mmap_rr_env	123
2.8.4	cache_replc_climb	125
2.8.5	cache_replc_fifo	126
2.8.6	cache_replc_hlru	127
2.8.7	cache_replc_lru	128
2.8.8	cache_replc_qlru	129
2.8.9	cache_replc_routing	130
2.8.10	cache_replc_rr	132
2.8.11	cache_rmf_transient	133
2.8.12	cachenet_tandem	134
2.8.13	retrieval_chain	136
2.8.14	retrieval_default	138
2.8.15	retrieval_ps	139
2.8.16	retrieval_routing	142
2.8.17	retrieval_simple	143
2.9	Cluster models	145
2.9.1	cl_basic	145
2.9.2	cl_closed	146
2.9.3	cl_compare	147
2.9.4	cl_multiclass	148
2.9.5	cl_scheduling	149
2.9.6	cl_stations	150
2.9.7	cl_sweep	151
2.9.8	dispatch_closed	152
2.9.9	dispatch_mixed	153
2.9.10	dispatch_open	154
2.10	Stochastic Petri nets	155
2.10.1	spn_basic_closed	155
2.10.2	spn_basic_open	156
2.10.3	spn_closed_fourplaces	157
2.10.4	spn_closed_twoplaces	158
2.10.5	spn_colored_gspn	159
2.10.6	spn_fluid_dae	160
2.10.7	spn_fourmodes	161
2.10.8	spn_inhibiting	162
2.10.9	spn_lpbounds	164
2.10.10	spn_open_sevenplaces	165
2.10.11	spn_pareto_service	166
2.10.12	spn_productform_nc	167
2.10.13	spn_queueing_place	168
2.10.14	spn_twomodes	169
2.10.15	test_spn_nrm_open	170
2.11	Workflow models	171
2.11.1	wf_aph	171

2.11.2	wf_branch	172
2.11.3	wf_complex	172
2.11.4	wf_erlang	173
2.11.5	wf_loop	174
2.11.6	wf_parallel	174
2.11.7	wf_serial	175
3	Advanced models	176
3.1	Load-dependent stations	176
3.1.1	fes_aggregation	176
3.1.2	fes_single_class	178
3.1.3	ld_class_dependence	179
3.1.4	ld_fes_multiclass	180
3.1.5	ld_fes_singleclass	182
3.1.6	ld_global_dependence	183
3.1.7	ld_joint_dependence	184
3.1.8	ld_multiserver_fcfs	186
3.1.9	ld_multiserver_ps	187
3.1.10	ld_multiserver_ps_twoclasses	189
3.1.11	ld_whittle_bandwidth	190
3.2	Finite capacity regions	192
3.2.1	fcr_constraints	192
3.2.2	fcr_lincon	193
3.2.3	fcr_lossn	194
3.2.4	fcr_mmlkdrop	195
3.2.5	fcr_mmlwaitq	196
3.2.6	fcr_oqndrop	197
3.2.7	fcr_oqnwaitq	198
3.3	State probabilities	199
3.3.1	statepr_aggr	199
3.3.2	statepr_aggr_large	200
3.3.3	statepr_allprobs_fcfs	202
3.3.4	statepr_allprobs_ps	203
3.3.5	statepr_sys_aggr	203
3.3.6	statepr_sys_aggr_large	205
3.3.7	statepr_sys_marg	206
3.4	Initial state and warm start	207
3.4.1	init_state_fcfs_exp	208
3.4.2	init_state_fcfs_nonexp	209
3.4.3	init_state_ps	210
3.4.4	ldes_warmstart	212
3.5	State-dependent routing	213
3.5.1	sdroute_closed	213
3.5.2	sdroute_jsq	215
3.5.3	sdroute_krzesinski	216
3.5.4	sdroute_multibranch	218
3.5.5	sdroute_open	219
3.5.6	sdroute_twoclasses_closed	220
3.6	Cyclic polling	222
3.6.1	polling_exhaustive_det	222

3.6.2	polling_exhaustive_exp	223
3.6.3	polling_gated	224
3.6.4	polling_klimited	225
3.7	Switchover times	226
3.7.1	switchover_basic	226
3.8	Response time distribution	227
3.8.1	cdf_respt_closed	227
3.8.2	cdf_respt_closed_threeclasses	229
3.8.3	cdf_respt_distrib	230
3.8.4	cdf_respt_open_twoclasses	232
3.8.5	cdf_respt_populations	233
3.9	Busy periods	234
3.9.1	busyp_subnetwork	235
3.10	Stochastic network calculus	236
3.10.1	snc_delay_quantile	236
3.10.2	snc_tandem_multiclass	237
3.11	Random environments	239
3.11.1	example_mapqn2renv	239
3.11.2	renv_basic	240
3.11.3	renv_container_terminal	241
3.11.4	renv_fourstages_repairmen	242
3.11.5	renv_genqn	243
3.11.6	renv_lqn_twostages	244
3.11.7	renv_map_fallback	245
3.11.8	renv_node_breakdown	246
3.11.9	renv_rotterdam_blending	247
3.11.10	renv_threestages_repairmen	247
3.11.11	renv_twostages_repairmen	249
3.12	Layered cache-queueing models	250
3.12.1	lcq_async_cache	250
3.12.2	lcq_singlehost	252
3.12.3	lcq_threehosts	253
3.13	Pass and swap	254
3.13.1	pas_closed_tandem_fig6	255
3.13.2	pas_compatibility_5class	256
3.13.3	pas_cyclic_ctmc_vs_bruteforce	257
3.13.4	pas_cyclic_normconst_bruteforce	258
3.13.5	pas_mmk	259
3.13.6	pas_nc_sampling	260
3.13.7	pas_saturation	261
3.13.8	pas_selfloop	263
3.14	Reward models	264
3.14.1	rewardModel_aggregation	264
3.14.2	rewardModel_mmk	265
3.14.3	rewardModel_multiclass	266
3.14.4	rewardModel_templates	267
3.15	Agent models	269
3.15.1	ag_closed_network	269
3.15.2	ag_gnetwork	270
3.15.3	ag_jackson_network	271

3.15.4	ag_multiclass_closed	272
3.15.5	ag_tandem_open	273
3.15.6	ag_tandem_phasetype	275
3.16	Large-scale models	276
3.16.1	largescale_tbi	276
3.17	Custom solvers	277
3.17.1	solver_custom	277
3.17.2	solver_custom_analyzer	278
4	Model gallery	279
4.1	Reading the names	279
4.2	The catalogue	279
4.2.1	gallery_aphm1	280
4.2.2	gallery_cache_lru	281
4.2.3	gallery_cache_routing	282
4.2.4	gallery_coxm1	283
4.2.5	gallery_cqn	283
4.2.6	gallery_cqn_multiclass	284
4.2.7	gallery_detm1	285
4.2.8	gallery_dm1	285
4.2.9	gallery_erldk	286
4.2.10	gallery_erlerl1	287
4.2.11	gallery_erlerl1_reentrant	287
4.2.12	gallery_erlm1	288
4.2.13	gallery_erlm1_ps	289
4.2.14	gallery_erlm1_reentrant	289
4.2.15	gallery_erlm1ps	289
4.2.16	gallery_fcr	290
4.2.17	gallery_fj_closed	291
4.2.18	gallery_fj_open	292
4.2.19	gallery_fj_quorum	293
4.2.20	gallery_gamm1	294
4.2.21	gallery_hyperl1_feedback	294
4.2.22	gallery_hyperl1_reentrant	295
4.2.23	gallery_hyperlk	296
4.2.24	gallery_hyphyp1_linear	296
4.2.25	gallery_hyphyp1_reentrant	297
4.2.26	gallery_hyphyp1_tandem	297
4.2.27	gallery_hypm1	298
4.2.28	gallery_hypm1_reentrant	298
4.2.29	gallery_lqn_basic	299
4.2.30	gallery_lqn_random	300
4.2.31	gallery_lqn_workflows	301
4.2.32	gallery_lukumar_reentrant	303
4.2.33	gallery_mapm1	304
4.2.34	gallery_mapmk	304
4.2.35	gallery_mdk	305
4.2.36	gallery_merl1	305
4.2.37	gallery_merl1_linear	305
4.2.38	gallery_merl1_reentrant	306

4.2.39	gallery_merl1_tandem	306
4.2.40	gallery_merlk	308
4.2.41	gallery_mhyp1	308
4.2.42	gallery_mhyp1_linear	308
4.2.43	gallery_mhyp1_reentrant	309
4.2.44	gallery_mhyp1_tandem	309
4.2.45	gallery_mhypk	311
4.2.46	gallery_mm1	311
4.2.47	gallery_mm1_feedback	312
4.2.48	gallery_mm1_linear	312
4.2.49	gallery_mm1_multiclass	313
4.2.50	gallery_mm1_prio	314
4.2.51	gallery_mm1_ps	314
4.2.52	gallery_mm1_ps_feedback	315
4.2.53	gallery_mm1_ps_multiclass	316
4.2.54	gallery_mm1_ps_reentrant	316
4.2.55	gallery_mm1_reentrant	317
4.2.56	gallery_mm1_tandem	317
4.2.57	gallery_mm1_tandem_multiclass	318
4.2.58	gallery_mm1k	319
4.2.59	gallery_mmap1	319
4.2.60	gallery_mmap1_multiclass	320
4.2.61	gallery_mmapk	320
4.2.62	gallery_mmk	322
4.2.63	gallery_mpar1	322
4.2.64	gallery_multitier	323
4.2.65	gallery_multitier_storage	324
4.2.66	gallery_parm1	325
4.2.67	gallery_qn_random	326
4.2.68	gallery_renv_breakdown	327
4.2.69	gallery_repairmen	327
4.2.70	gallery_replayerm1	328
4.2.71	gallery_um1	329
5	Discrete-time models	330
5.1	The slotted models	330
5.1.1	dt_bernoulli_loaddep	330
5.1.2	dt_cycle	331
5.1.3	dt_cycle_loaddep	332
5.1.4	dt_cycle_multiclass	333
5.1.5	dt_geogeol	335
5.1.6	dt_geogeol_loss	335
6	Solver-specific examples	337
6.0.1	ctmc_spn_mm1	337
6.0.2	ctmc_spn_vs_nc_closed_qn	338
6.0.3	ctmc_tandem_mm1	340

7	Model interchange	342
7.1	The fixtures	342
7.1.1	json_balking_retrial	342
7.1.2	json_classcap_droprule	343
7.1.3	json_fcr_details	344
7.1.4	json_hetero_servers	346
7.1.5	json_join_deadline	347
7.1.6	json_signal_classes	348
8	Inference and estimation	350
8.1	The estimators	350
8.1.1	est_ekf_changepoint	350
8.1.2	est_ekf_closed	351
8.1.3	est_ekf_open	353
8.1.4	est_erps_closed	354
8.1.5	est_erps_open	355
8.1.6	est_mcmc_closed	356
8.1.7	est_mle_open	357
8.1.8	est_qmle_closed	358
8.1.9	est_trace_closed	359
8.1.10	est_ubo_changepoint	361
8.1.11	est_ubo_closed	362
8.1.12	est_ubo_open	363
8.1.13	est_ubo_variants_closed	364
8.1.14	est_ubr_changepoint	366
8.1.15	est_ubr_closed	367
8.1.16	est_ubr_open	368
8.1.17	est_vi_closed	369
9	Optimization	371
9.1	Sizing an M/M/c queue	371
9.2	Exact sizing by bisection	372
9.3	Continuous service rate tuning	372
9.4	Load balancing across heterogeneous servers	372
9.5	Population sizing in a closed network	373
9.6	Robust sizing under workload scenarios	373
9.7	Cost-performance tradeoff frontier	374
9.8	Decomposing a multi-variable problem	374
9.9	LQN host demand tuning	374
9.10	The models behind the tutorials	375
9.10.1	opt_bisection_sizing	375
9.10.2	opt_decomposition	376
9.10.3	opt_load_balancing	377
9.10.4	opt_lqn_host_demand	378
9.10.5	opt_pareto_frontier	379
9.10.6	opt_population_sizing	380
9.10.7	opt_robust_sizing	381
9.10.8	opt_sensitivity_report	382
9.10.9	opt_server_sizing	383
9.10.10	opt_service_rate	384

Chapter 1

About this manual

This manual documents the worked-example inventory of LINE, one edition per codebase. An entry is runnable only when this edition identifies a source implementation and does not mark the required operation as unsupported: `cpp/examples/`, and each chapter follows the organization of that tree. Script names agree across the four codebases, so an example can be read side by side between editions; where a codebase does not carry a family at all, the chapter says so rather than leaving the reader to infer it.

The reference material on model classes, solvers and options is in the LINE user manual, and the guided tour and the introductory tutorials are in the LINE primer. Neither is repeated here: this manual starts where the primer stops, at the point where one already knows how to build a model and wants to see the feature one needs.

Every model in the suite is written in the same four blocks, and it is worth learning to read them:

1. the nodes: stations, sources, sinks and the special nodes;
2. the classes and their arrival and service distributions;
3. the topology, as a routing matrix or a serial routing shorthand;
4. the solution: one or more solvers, and the metrics read from them.

The chapters are:

- *Basic models* (Chapter 2): the elementary model families, one per node or class feature.
- *Advanced models* (Chapter 3): features that leave the product-form setting.
- *Model gallery* (Chapter 4): a catalogue of small named models, in Kendall notation where one applies.
- *Discrete-time models* (Chapter 5): models on a slotted time scale.
- *Solver-specific examples* (Chapter 6): scripts written around one solver rather than one model.
- *Model interchange* (Chapter 7): the JSON model format and its round-trip fixtures.
- *Inference and estimation* (Chapter 8): fitting model parameters to measurements.
- *Optimization* (Chapter 9): design problems on queueing networks.

Each chapter opens with a worked example and then gives one section per script in the reference inventory. A complete entry contains a topology figure, a feature table, source code, and captured output. These components are generated from the harvested model, the implementation in the named language tree, and an actual run.

Chapter 2

Basic models

The basic examples cover one model family each: a node type, a class type or a scheduling feature, in the smallest model that shows it. They live under `cpp/examples/basic/`, one translation unit per family, and the script names agree across the four codebases, so a model can be compared line by line between editions.

2.1 Open queueing networks

An open network has an unbounded population: jobs enter at a `Source`, visit stations according to the routing matrix and leave at a `Sink`. Stability requires the utilization of every station to stay below one, which is a property of the arrival rates and the demands, not something the model declares.

The archetype is `oqn_basic`: a Poisson stream feeds a Delay station with hyperexponential think time and an FCFS queue with exponential service. Four blocks build it – nodes, classes and their distributions, the routing matrix, and the solve – and every model in this manual follows the same four blocks.

```
Net m("model");
// Block 1: nodes
const std::size_t delay = m.add_delay("Delay");
const std::size_t queue = m.add_queue("Queue1", SchedStrategy::FCFS);
const std::size_t source = m.add_source("Source");
const std::size_t sink = m.add_sink("Sink");
// Block 2: classes
const std::size_t c1 = m.add_open_class("Class1", 0);
m.set_service(delay, c1, D::hyperexp(0.5, 3.0, 10.0));
m.set_service(queue, c1, D::exp_rate(1.0));
m.set_arrival(source, c1, D::exp_rate(0.1));
// Block 3: topology
Routing P;
P.set(c1, c1, delay, queue, 1.0);
P.set(c1, c1, queue, sink, 1.0);
P.set(c1, c1, source, delay, 1.0);
m.link(P);
// Block 4: solution
Matrix<double> init;
print_avg(m.get_struct(), mva::solver_mva_run_analyzer(m.get_struct(), mva::MvaOptions(),
    init));
```

The scripts of this family are:

2.1.1 `mam_transient_mapmap1`

Transient analysis of a MAP/MAP/1 queue started empty, through the Laplace-domain transient QBD method that `getTranAvg` auto-selects when both the arrival and the service process are correlated.

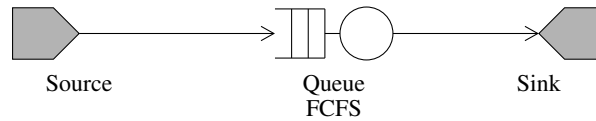


Figure 2.1: Topology of mam_transient_mapmap1: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: Class1 (open)
Scheduling	Queue: FCFS
Arrivals	Source – Class1: MAP
Service	Queue – Class1: MAP

Table 2.1: Features of mam_transient_mapmap1.

The example is built and solved as follows.

```
const mam::Map<double> a = arrival_map();
mam::Map<double> s = service_map();

// Scale service to rho = 0.6: both blocks by one factor, as the reference.
const double fac = (mam::map_lambda(a) / 0.6) / mam::map_lambda(s);
for (std::size_t i = 0; i < 2; ++i)
    for (std::size_t j = 0; j < 2; ++j) {
        s.D0(i, j) *= fac;
        s.D1(i, j) *= fac;
    }

Net m("MAP/MAP/1 transient");
Source source(m, "Source");
Queue queue(m, "Queue", SchedStrategy::FCFS);
Sink sink(m, "Sink");
OpenClass oclass(m, "Class1");
source.set_arrival(oclass, D::map_dist(a.D0, a.D1, lang::ProcessType::MAP));
queue.set_service(oclass, D::map_dist(s.D0, s.D1, lang::ProcessType::MAP));
Routing P;
serial(P, {source, queue, sink});
m.link(P);

mam::MamOptions opt;
opt.timespan_start = 0.0;
opt.timespan_end = 40.0;
const mam::TranResult<double> tr = mam::solver_mam_get_tran_avg(m.get_struct(), opt);

... (12 source lines omitted; full implementation: cpp/examples/basic/)

// so the derived reader anchors on this label exactly as it does on
// init_state_*'s SteadyStateQLen line. All four codebases print the same
// three labels so the cells line up.
std::printf("TranEndQLen[MAM/Queue]: %.6f\n", q.values.back());
std::printf("TranEndUtil[MAM/Queue]: %.6f\n", u.values.back());
std::printf("TranEndTput[MAM/Queue]: %.6f\n", t.values.back());
```

Running this edition prints

```
MAP/MAP/1 transient (rho=0.6), start empty:
t= 40.0 E[N]=1.54574 U=0.59999 Tput=1.87498
steady-state E[N] approaches 1.5458 as t -> inf.
TranEndQLen[MAM/Queue]: 1.545743
TranEndUtil[MAM/Queue]: 0.599994
TranEndTput[MAM/Queue]: 1.874982
```

2.1.2 oqn_basic

Source, Delay and an FCFS queue with one open class and a hyperexponential think time, solved with every solver that admits the model.

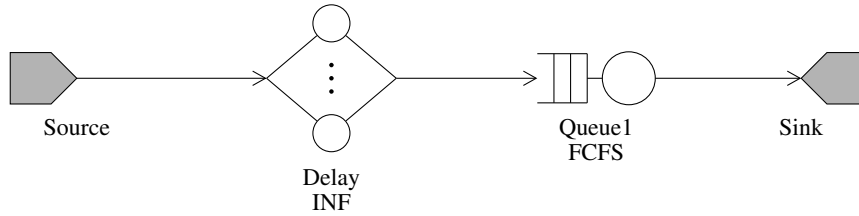


Figure 2.2: Topology of oqn_basic: an open network over 2 queueing stations.

Nodes	4: Source, Queue, Delay, Sink
Classes	1: Class1 (open)
Scheduling	Delay: INF; Queue1: FCFS
Arrivals	Source – Class1: Exp(0.1)
Service	Delay – Class1: HyperExp(0.5,0.5;3,10) Queue1 – Class1: Exp(1)
Solvers	CTMC, FLD, MVA, MAM, NC, JMT, SSA, LDES

Table 2.2: Features of oqn_basic.

The example is built and solved as follows.

```

Net m("model");
Delay delay(m, "Delay");
Queue queue(m, "Queue1", SchedStrategy::FCFS);
Source source(m, "Source");
Sink sink(m, "Sink");
OpenClass c1(m, "Class1", 0);

delay.set_service(c1, HyperExp(0.5, 3.0, 10.0));
queue.set_service(c1, Exp(1.0));
source.set_arrival(c1, Exp(0.1));

Routing P;
P.set(c1, c1, delay, queue, 1.0);
P.set(c1, c1, queue, sink, 1.0);
P.set(c1, c1, source, delay, 1.0);
m.link(P);
const Sn& sn = m.get_struct();

section("CTMC");
ctmc::CtmcOptions copt;
copt.cutoff = 10.0;
print_avg(sn, ctmc::solver_ctmc_run_analyzer(sn, copt));

section("FLD");
print_avg_sim(sn, fluid::solver_fluid_run_analyzer(sn, fluid::FluidOptions()));

... (15 source lines omitted; full implementation: cpp/examples/basic/)

sopt.seed = 23000;
sopt.samples = 50000;
print_avg_sim(sn, ssa::solver_ssa(sn, sopt));

section("LDES");
print_avg(m.get_struct(), ldes_avg(m, sim_opts(23000, 200000)));

```

Running this edition prints

SOLVER: CTMC						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Delay	Class1	0.021667	0.021667	0.21667	0.21667	0.1
0.1						
Queue1	Class1	0.11111	0.1	1.1111	1.1111	0.1
0.1						
Source	Class1	0	0	0	0	0
0.1						
SOLVER: FLD						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Delay	Class1	0.021667	0.021667	0.21667	0.21667	0.1
0.1						
Queue1	Class1	0.10021	0.1	1.0021	1.0021	0.1
0.1						
Source	Class1	0	0	0	0	0
0.1						
SOLVER: MVA						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Delay	Class1	0.021667	0.021667	0.21667	0.21667	0.1
0.1						
Queue1	Class1	0.11111	0.1	1.1111	1.1111	0.1
0.1						
... (23 captured-output lines omitted; rerun the source example for the full output)						
Queue1	Class1	0.11423	0.10225	1.1172	1.1172	0.10136
0.10225						
Source	Class1	0	0	0	0	0
0.1						
SOLVER: LDES						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Delay	Class1	0.021461	0.021697	0.21425	0.21425	0.10335
0.10014						
Queue1	Class1	0.11138	0.10026	1.1107	1.1107	0.10335
0.10014						
Source	Class1	0	0	0	0	0
0.1						

The rows repeat for each of the 8 solvers the script runs (CTMC, FLD, MVA, MAM, NC, JMT, SSA, LDES), which is the point of the example: the same model read by methods of different cost and different exactness.

2.1.3 oqn_bmmapt

A BATCH, MARKED, time-inhomogeneous MAP arrival stream: a block is indexed by segment, mark and batch size, so the schedule moves the class mix and the batch size together while the epoch rate stays put. The job rate therefore differs from the epoch rate, and per class within a segment.

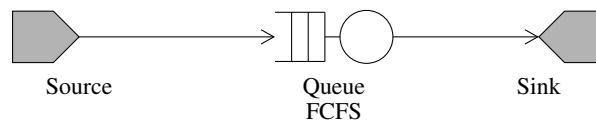


Figure 2.3: Topology of oqn_bmmapt: an open network over 1 queueing station.

The example is built and solved as follows.

```

Net m("model");
Source source(m, "Source");
Queue queue(m, "Queue", SchedStrategy::FCFS);
Sink sink(m, "Sink");
OpenClass premium(m, "Premium", 0);
OpenClass economy(m, "Economy", 0);

```

Nodes	3: Source, Queue, Sink
Classes	2: Premium (open), Economy (open)
Scheduling	Queue: FCFS
Arrivals	Source – Premium: BMMAPt(True); Economy: BMMAPt(True)
Service	Queue – Premium: Exp(16); Economy: Exp(16)
Solvers	LDES

Table 2.3: Features of `oqn_bmmapt`.

```
// Two twelve-hour segments repeating on a daily cycle. One phase, so the
// epoch stream is Poisson at rate 4 throughout and only the labels move.
const std::vector<double> bp = {0.0, 12.0, 24.0};
std::vector<Matrix<double>> D0;
D0.push_back(Matrix<double>(1, 1, -4.0));
D0.push_back(Matrix<double>(1, 1, -4.0));

// D1kb[mark][batch][segment]. The batch axis is DENSE: a mark that never
// releases a batch of that size still declares a zero block for it, exactly
// as BMAP's {D0, D1, ..., Dk} does.
//
// morning (segment 1): Premium in PAIRS at 3.6, Economy singly at 0.4
// evening (segment 2): Premium singly at 0.4, Economy in PAIRS at 3.6
auto one = [] (double v) { return Matrix<double>(1, 1, v); };
std::vector<std::vector<std::vector<Matrix<double>>>> D1kb(2);
D1kb[0].resize(2);
D1kb[1].resize(2);
D1kb[0][0] = {one(0.0), one(0.4)};
D1kb[0][1] = {one(3.6), one(0.0)};

... (11 source lines omitted; full implementation: cpp/examples/basic/)

// LDES simulates the batch marked schedule directly: the mark and the batch
// size are both decided by WHICH block's transition fired, inside the same
// competing-transitions draw that ends the interval, so neither label costs
// an extra random draw and the reduction to MMAPt at batch size 1 is exact.
print_avg(sn, ldes_avg(m, sim_opts(23000, 400000)));
```

Running this edition prints

Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Source	Premium	0	0	0	0	0
3.8						
Source	Economy	0	0	0	0	0
3.8						
Queue	Premium	0.67613	0.23914	0.17704	0.17704	3.8248
3.8209						
Queue	Economy	0.67338	0.23868	0.17699	0.17699	3.8088
3.805						

2.1.4 oqn_cs_routing

Three open classes over two PS queues, two of which switch class at a `ClassSwitch` node; the topology is imported from a JSIMgraph file.

The example is built and solved as follows.

```
Net m("myModel");
Source source(m, "Source 1");
Queue q1(m, "Queue 1", SchedStrategy::PS);
Sink sink(m, "Sink 1");
Queue q2(m, "Queue 2", SchedStrategy::PS);

OpenClass ca(m, "Class A", 0);
OpenClass cb(m, "Class B", 0);
```

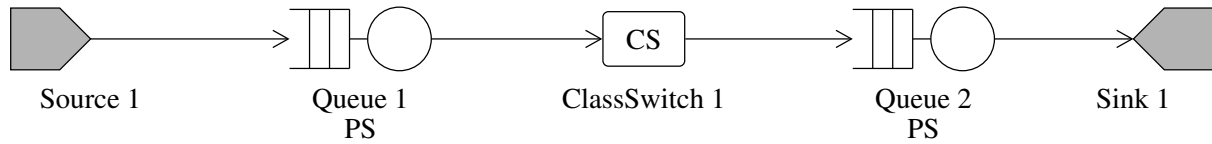


Figure 2.4: Topology of `oqn_cs_routing`: an open network over 2 queueing stations with a class-switch node.

Nodes	5: Source, Queue $\times$ 2, ClassSwitch, Sink
Classes	3: Class A (open), Class B (open), Class C (open)
Scheduling	Queue 1: PS; Queue 2: PS
Arrivals	Source 1 – Class A: Exp(2); Class B: Exp(1); Class C: Disabled
Service	Queue 1 – Class A: Exp(5); Class B: Exp(3.333); Class C: Exp(3) Queue 2 – Class A: Exp(1); Class B: Exp(1); Class C: Exp(6.667)
Solvers	CTMC, FLD, MVA, MAM, NC, JMT, SSA, LDES

Table 2.4: Features of `oqn_cs_routing`.

```

OpenClass cc(m, "Class C", 0);

Matrix<double> C(3, 3, 0.0);
for (std::size_t i = 0; i < 3; ++i) C(i, i) = 1.0;
const std::size_t cs = m.add_class_switch("ClassSwitch 1", C);

source.set_arrival(ca, D::exp_mean(0.5));
source.set_arrival(cb, D::exp_mean(1.0));
source.set_arrival(cc, Disabled());

q1.set_service(ca, D::exp_mean(0.2));
q1.set_service(cb, D::exp_mean(0.3));
q1.set_service(cc, D::exp_mean(0.333333));

q2.set_service(ca, D::exp_mean(1.0));
q2.set_service(cb, D::exp_mean(1.0));
q2.set_service(cc, D::exp_mean(0.15));

... (41 source lines omitted; full implementation: cpp/examples/basic/)

sopt.seed = 23000;
sopt.samples = 1000000;
print_avg_sim(sn, ssa::solver_ssa(sn, sopt));

section("LDES");
print_avg(m.get_struct(), ldes_avg(m, sim_opts(23000, 100000)));

```

Running this edition prints

```

SOLVER: CTMC
N/A: the reference passes options.cutoff as a (station x class) matrix; CtmcOptions carries a scalar and a
per-class vector only, and collapsing the matrix would enumerate a different state space

SOLVER: FLD
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
Tput
Source 1     Class A        0          0          0          0          0
2
Source 1     Class B        0          0          0          0          0
1
Queue 1      Class A      0.99406    0.4        0.49703    0.33135    2
2
Queue 1      Class B      0.74555    0.3        0.74555    0.24852    1
1
Queue 2      Class C      0.61889    0.45       0.2063     0.2063     3
3

SOLVER: MVA

```


Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Source 1	Class A	0	0	0	0	0
2						
Source 1	Class B	0	0	0	0	0
1						
Queue 1	Class A	1.3333	0.4	0.66667	0.44444	2
2						
... (29 captured-output lines omitted; rerun the source example for the full output)						
Source 1	Class A	0	0	0	0	0
2						
Source 1	Class B	0	0	0	0	0
1						
Queue 1	Class A	1.2717	0.4062	0.62613	0.41742	2
2.031						
Queue 1	Class B	0.90626	0.28948	0.93921	0.31307	1
0.96492						
Queue 2	Class C	0.82149	0.45235	0.27241	0.27241	2.9959
3.0156						
SOLVER: LDES						
N/A: SolverLDES(model, seed=23000, samples=100000) has no C++ engine in this port						

The rows repeat for each of the 8 solvers the script runs (CTMC, FLD, MVA, MAM, NC, JMT, SSA, LDES), which is the point of the example: the same model read by methods of different cost and different exactness.

2.1.5 oqn_fourqueues

A larger multiclass network over four stations mixing PS and FCFS scheduling.

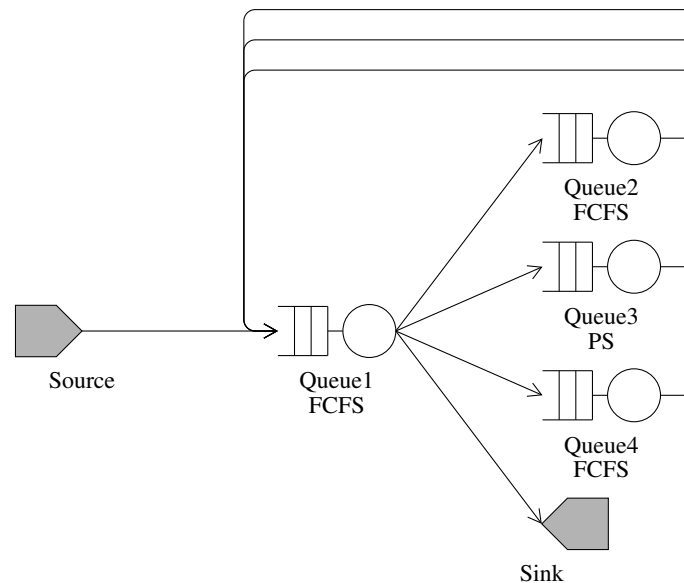


Figure 2.5: Topology of oqn_fourqueues: an open network over 4 queueing stations.

The example is built and solved as follows.

```

Net m("MyNetwork");
Source source(m, "Source");
Queue q1(m, "Queue1", SchedStrategy::FCFS);
Queue q2(m, "Queue2", SchedStrategy::FCFS);
Queue q3(m, "Queue3", SchedStrategy::PS);
Queue q4(m, "Queue4", SchedStrategy::FCFS);
Sink sink(m, "Sink");

const std::vector<std::size_t> cls = {m.add_open_class("Class1", 0),

```

Nodes	6: Source, Queue $\times$ 4, Sink
Classes	3: Class1 (open), Class2 (open), Class3 (open)
Scheduling	Queue1: FCFS; Queue2: FCFS; Queue3: PS; Queue4: FCFS
Arrivals	Source – Class1: Exp(0.2); Class2: Exp(0.125); Class3: Exp(0.1429)
Service	Queue1 – Class1: Exp(3.333); Class2: Exp(2); Class3: Exp(1.667) Queue2 – Class1: Exp(0.9091); Class2: Exp(0.7692); Class3: Exp(0.6667) Queue3 – Class1: Exp(0.5); Class2: Exp(0.4762); Class3: Exp(0.5263) Queue4 – Class1: Exp(0.6667); Class2: Exp(1.111); Class3: Exp(0.4348)
Solvers	CTMC, MVA, MAM, JMT, LDES

Table 2.5: Features of oqn_fourqueues.

```

                                m.add_open_class("Class2", 0),
                                m.add_open_class("Class3", 0));

const double arrival[3] = {5.0, 8.0, 7.0};
const double svc[3][4] = {{0.3, 1.1, 2.0, 1.5}, {0.5, 1.3, 2.1, 0.9}, {0.6, 1.5, 1.9, 2.3}};
const std::size_t queues[4] = {q1, q2, q3, q4};
for (std::size_t r = 0; r < 3; ++r) {
    source.set_arrival(cls[r], D::exp_mean(arrival[r]));
    for (std::size_t i = 0; i < 4; ++i) m.set_service(queues[i], cls[r], D::exp_mean(svc[r][i]));
}

Routing P;
for (std::size_t r = 0; r < 3; ++r) {
    P.set(cls[r], cls[r], source, q1, 1.0);
    P.set(cls[r], cls[r], q1, q2, 0.25);
    P.set(cls[r], cls[r], q1, q3, 0.25);
    P.set(cls[r], cls[r], q1, q4, 0.25);

    ... (18 source lines omitted; full implementation: cpp/examples/basic/)

print_avg(sn, mam::solver_mam_run_analyzer(sn, mam::MamOptions()));

section("JMT");
print_avg(m.get_struct(), jmt_avg(m, sim_opts(23000, 1000000)));
section("LDES");
print_avg(m.get_struct(), ldes_avg(m, sim_opts(23000, 1000000)));

```

Running this edition prints

SOLVER: CTMC						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Source	Class1	0	0	0	0	0
0.078166						
Source	Class2	0	0	0	0	0
0.061126						
Source	Class3	0	0	0	0	0
0.059853						
Queue1	Class1	0.14622	0.093799	0.46767	1.8707	0.31266
0.31266						
Queue1	Class2	0.15681	0.12225	0.64133	2.5653	0.24451
0.24451						
Queue1	Class3	0.16999	0.14365	0.71004	2.8402	0.23941
0.23941						
Queue2	Class1	0.10897	0.085983	1.394	1.394	0.078166
0.078166						
Queue2	Class2	0.095956	0.079464	1.5698	1.5698	0.061126
0.061126						
Queue2	Class3	0.10304	0.08978	1.7216	1.7216	0.059853
0.059853						
Queue3	Class1	0.20295	0.15633	2.5963	2.5963	0.078166
0.078166						
Queue3	Class2	0.17126	0.12837	2.8018	2.8018	0.061126
0.061126						
Queue3	Class3	0.15414	0.11372	2.5753	2.5753	0.059853
0.059853						
Queue4	Class1	0.15103	0.11725	1.9322	1.9322	0.078166

```

0.078166
Queue4      Class2      0.086961    0.055014    1.4226    1.4226    0.061126
0.061126

... (50 captured-output lines omitted; rerun the source example for the full output)

Queue3      Class2      3.8567    0.26258    30.078    30.078    0.12464
0.12464
Queue3      Class3      3.9759    0.27255    26.901    26.901    0.14254
0.14267
Queue4      Class1      1.3583    0.29939    6.6818    6.6818    0.19939
0.19939
Queue4      Class2      0.72756    0.1126    5.7367    5.7367    0.12507
0.1246
Queue4      Class3      1.0722    0.32813    7.4165    7.4165    0.14274
0.14286

SOLVER: LDES
N/A: SolverLDES(model, seed=23000, samples=1000000) has no C++ engine in this port

```

The rows repeat for each of the 5 solvers the script runs (CTMC, MVA, MAM, JMT, LDES), which is the point of the example: the same model read by methods of different cost and different exactness.

2.1.6 oqn_mapt

A time-inhomogeneous MAP arrival stream and the Ko-Pender fluid and diffusion limits returned by the kp fluid method.

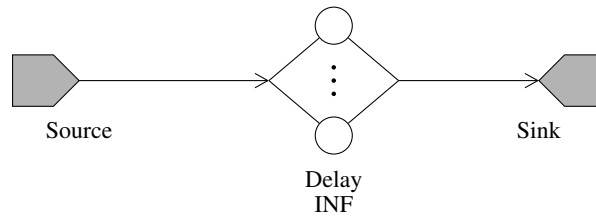


Figure 2.6: Topology of oqn_mapt: an open network over 1 queueing station.

Nodes	3: Source, Delay, Sink
Classes	1: OpenClass (open)
Scheduling	Delay: INF
Arrivals	Source – OpenClass: MAPt
Service	Delay – OpenClass: Exp(2)
Solvers	FLD

Table 2.6: Features of oqn_mapt.

The example is built and solved as follows.

```

Net m("model");
Source source(m, "Source");
Delay delay(m, "Delay");
Sink sink(m, "Sink");
OpenClass oc(m, "OpenClass", 0);

std::vector<double> bp(kMaptBreak, kMaptBreak + 3);
std::vector<Matrix<double>> D0segs, D1segs;
for (std::size_t k = 0; k < 2; ++k) {
    const mam::Map<double> s = mapt_segment(k);
    D0segs.push_back(s.D0);
    D1segs.push_back(s.D1);
}
source.set_arrival(oc, D::mapt(bp, D0segs, D1segs, true));
delay.set_service(oc, Exp(2.0));

```

```

Routing P;
serial(P, {source, delay, sink});
m.link(P);
const Sn& sn = m.get_struct();

fluid::FluidOptions opt;
opt.method = "kp";
opt.tol = 1e-9;
// The method is presentation: the golden keys this table plain `FLD`.
section("FLD (kp)", "FLD");

... (14 source lines omitted; full implementation: cpp/examples/basic/)

std::printf("\n  t      lambda(t)      QLen mean      QLen var\n");
const double probe[7] = {0.5, 1.0, 1.5, 2.5, 3.0, 4.0, 5.0};
for (std::size_t i = 0; i < 7; ++i)
    std::printf("%6.2f %10.4f %11.6f %11.6f\n", probe[i], mapt_rate_at(probe[i]),
        interp_at(probe[i], tran.t, mean), interp_at(probe[i], tran.t, var));

```

Running this edition prints

SOLVER: FLD (kp)						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Source	OpenClass	0	0	0	0	0
5.4301						
Delay	OpenClass	2.7151	2.7151	0.5	0.5	5.4301
5.4301						
t	lambda(t)	QLen mean	QLen var			
0.50	3.2000	1.011393	1.040372			
1.00	6.9167	1.383464	1.421803			
1.50	6.9167	2.698076	2.835061			
2.50	3.2000	3.355447	3.512559			
3.00	3.2000	2.242617	2.303137			
4.00	6.9167	2.864588	3.002254			
5.00	3.2000	3.377981	3.535106			

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by FLD.

2.1.7 oqn_mapt_service

A time-inhomogeneous MAP as the SERVICE process of a finite-buffer queue: the phase walk is taken from the service start instant and the modulating phase carries across successive services, so the stream is correlated as well as non-stationary.

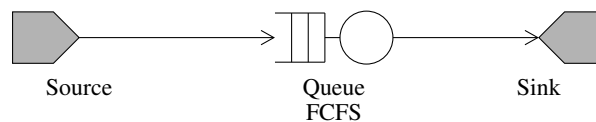


Figure 2.7: Topology of oqn_mapt_service: an open network over 1 queueing station.

The example is built and solved as follows.

```

Net m("model");
Source source(m, "Source");
Queue queue(m, "Queue", SchedStrategy::FCFS);
Sink sink(m, "Sink");
OpenClass oc(m, "OpenClass", 0);

// Three segments held for 1, 1, 2 time units, repeating with period 4.
const std::vector<double> bp = {0.0, 1.0, 2.0, 4.0};
source.set_arrival(oc, Exp(10.0));

```

Nodes	3: Source, Queue, Sink
Classes	1: OpenClass (open)
Scheduling	Queue: FCFS
Arrivals	Source – OpenClass: Exp(10)
Service	Queue – OpenClass: MAPt
Features	finite buffer 5 at Queue
Solvers	LDES

Table 2.7: Features of oqn_mapt_service.

```

queue.set_service(oc, D::mapt(bp, mapt_service_segs(false), mapt_service_segs(true), true));
queue.set_capacity(5);
queue.set_number_of_servers(1);

Routing P;
serial(P, {source, queue, sink});
m.link(P);
m.get_struct();

// The offered load exceeds what the server can clear, so the finite buffer
// drops: Tput falls short of the arrival rate and QLen sits just under the
// capacity. Both are signatures that the service process is being simulated
// -- a service that sampled as zero would report QLen = Util = 0, Tput = 10.
section("LDES");
print_avg(m.get_struct(), ldes_avg(m, sim_opts(1234, 1000000)));

// A MAP_t whose segments are identical carries no time dependence, so it
... (32 source lines omitted; full implementation: cpp/examples/basic/)

note("constant schedule vs ordinary MAP (must agree):");
section("LDES (constant MAPt schedule)");
print_avg(mf.get_struct(), ldes_avg(mf, sim_opts(1234, 400000)));
section("LDES (ordinary MAP)");
print_avg(mh.get_struct(), ldes_avg(mh, sim_opts(1234, 400000)));

```

Running this edition prints

```

SOLVER: LDES
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
  Tput
Source      OpenClass      0          0          0          0          0
  10
Queue      OpenClass      4.0818     0.97738   0.86824    0.86824    4.7059
  4.7012
constant schedule vs ordinary MAP (must agree):

SOLVER: LDES (constant MAPt schedule)
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
  Tput
Source      OpenClass      0          0          0          0          0
  1
Queue      OpenClass      0.17933    0.1505    0.1795     0.1795     1.0045
  0.99848

SOLVER: LDES (ordinary MAP)
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
  Tput
Source      OpenClass      0          0          0          0          0
  1
Queue      OpenClass      0.17852    0.14969   0.17882    0.17882    0.99943
  0.99843

```

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by LDES.

2.1.8 oqn_mmapt

A MARKED time-inhomogeneous MAP arrival stream: one modulating chain drives every marked class, the mark selects the class of the arriving job, and the schedule moves the class MIX while leaving the aggregate rate flat.

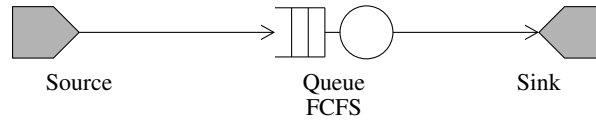


Figure 2.8: Topology of oqn_mmapt: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	2: Morning (open), Evening (open)
Scheduling	Queue: FCFS
Arrivals	Source – Morning: MMAPt(True); Evening: MMAPt(True)
Service	Queue – Morning: Exp(8); Evening: Exp(8)
Solvers	LDES

Table 2.8: Features of oqn_mmapt.

The example is built and solved as follows.

```
Net m("model");
Source source(m, "Source");
Queue queue(m, "Queue", SchedStrategy::FCFS);
Sink sink(m, "Sink");
OpenClass morningClass(m, "Morning", 0);
OpenClass eveningClass(m, "Evening", 0);

// Two twelve-hour segments repeating on a daily cycle. One phase, so the
// aggregate stream is Poisson at rate 4 throughout and ONLY the mix moves.
const std::vector<double> bp = {0.0, 12.0, 24.0};
std::vector<Matrix<double>> D0;
D0.push_back(Matrix<double>(1, 1, -4.0));
D0.push_back(Matrix<double>(1, 1, -4.0));

// Dlk[mark][segment]: mark 1 -> Morning, 9:1 by day and 1:9 by night.
auto one = [] (double v) { return Matrix<double>(1, 1, v); };
std::vector<std::vector<Matrix<double>>> Dlk(2);
Dlk[0] = {one(3.6), one(0.4)};
Dlk[1] = {one(0.4), one(3.6)};

const Dist arrival = D::mmapt(bp, D0, Dlk, true);
source.set_marked_arrival(arrival, {morningClass, eveningClass});
queue.set_service(morningClass, Exp(8.0));
queue.set_service(eveningClass, Exp(8.0));

Routing P;

... (13 source lines omitted; full implementation: cpp/examples/basic/)

"the marks from the segment in force rather than from the average.\n");

// LDES simulates the marked schedule directly: the mark is decided by WHICH
// block's transition fired, inside the same competing-transitions draw that
// ends the interval, so no extra randomness is introduced by the labelling.
print_avg(sn, ldes_avg(m, sim_opts(23000, 400000)));
```

Running this edition prints

```
Aggregate arrival rate over the cycle: 4
Per-mark rates over the cycle:      2.0000 2.0000 (they sum to the aggregate)
The cycle average is a 2:2 split, so a run that shows anything else is reading
```

the marks from the segment in force rather than from the average.

Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Source	Morning	0	0	0	0	0
2						
Source	Evening	0	0	0	0	0
2						
Queue	Morning	0.50187	0.25118	0.25014	0.25014	2.0086
2.0066						
Queue	Evening	0.50852	0.25199	0.25317	0.25317	2.0107
2.0088						

2.1.9 oqn_multichain_cs

Five classes grouped into three chains by class switching, the smallest model in which chains and classes differ.

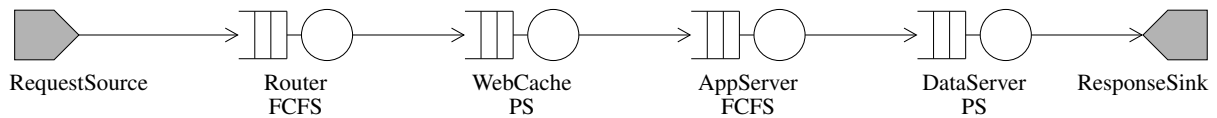


Figure 2.9: Topology of oqn_multichain_cs: an open network over 4 queueing stations.

Nodes	6: Source, Queue $\times$ 4, Sink
Classes	5: InteractiveA (open), InteractiveB (open), BatchA (open), BatchB (open), RealTime (open)
Scheduling	Router: FCFS; WebCache: PS; AppServer: FCFS; DataServer: PS
Arrivals	RequestSource – InteractiveA: Exp(0.3); InteractiveB: Exp(0.1); BatchA: Exp(0.15); BatchB: Exp(0.12); RealTime: Exp(0.2)
Service	Router – InteractiveA: Exp(6.667); InteractiveB: Exp(5.556); BatchA: Exp(5); BatchB: Exp(4.545); RealTime: Exp(10) WebCache – InteractiveA: Exp(3.333); InteractiveB: Exp(2.857); BatchA: Exp(1); BatchB: Exp(0.8333); RealTime: Exp(5) AppServer – InteractiveA: Exp(2); InteractiveB: Exp(1.667); BatchA: Exp(0.6667); BatchB: Exp(0.5556); RealTime: Exp(3.333) DataServer – InteractiveA: Exp(2.5); InteractiveB: Exp(2.222); BatchA: Exp(1.25); BatchB: Exp(1); RealTime: Exp(4)
Solvers	MVA, FLD, SSA

Table 2.9: Features of oqn_multichain_cs.

The example is built and solved as follows.

```
Net m("oqn_multichain");
Source source(m, "RequestSource");
Queue router(m, "Router", SchedStrategy::FCFS);
Queue cache(m, "WebCache", SchedStrategy::PS);
Queue app(m, "AppServer", SchedStrategy::FCFS);
Queue data(m, "DataServer", SchedStrategy::PS);
Sink sink(m, "ResponseSink");

const std::vector<std::size_t> cls = {
    m.add_open_class("InteractiveA", 0), m.add_open_class("InteractiveB", 0),
    m.add_open_class("BatchA", 0), m.add_open_class("BatchB", 0),
    m.add_open_class("RealTime", 0)};

const std::size_t st[4] = {router, cache, app, data};
const double svc[4][5] = {{0.15, 0.18, 0.2, 0.22, 0.1},
                          {0.3, 0.35, 1.0, 1.2, 0.2},
                          {0.5, 0.6, 1.5, 1.8, 0.3},
                          {0.4, 0.45, 0.8, 1.0, 0.25}};
for (std::size_t i = 0; i < 4; ++i)
    for (std::size_t r = 0; r < 5; ++r) m.set_service(st[i], cls[r], D::exp_mean(svc[i][r]));

const double lambda[5] = {0.3, 0.1, 0.15, 0.12, 0.2};
```

```

for (std::size_t r = 0; r < 5; ++r) source.set_arrival(cls[r], Exp(lambda[r]));

Routing P;
for (std::size_t r = 0; r < 5; ++r) {

    ... (28 source lines omitted; full implementation: cpp/examples/basic/)

print_avg_sim(sn, fluid::solver_fluid_run_analyzer(sn, fluid::FluidOptions()));

section("SSA");
ssa::SsaOptions sopt;
sopt.seed = 23000;
print_avg_sim(sn, ssa::solver_ssa(sn, sopt));

```

Running this edition prints

```

Open QN with 3 chains, 5 classes, and class switching:
- Chain 1: InteractiveA and InteractiveB (switch at Router)
- Chain 2: BatchA and BatchB (switch at Router)
- Chain 3: RealTime

SOLVER: MVA
Station      JobClass      QLen      Util      RespT      ResidT      ArrR
Tput
RequestSource InteractiveA      0          0          0          0          0
0.3
RequestSource InteractiveB      0          0          0          0          0
0.1
RequestSource BatchA      0          0          0          0          0
0.15
RequestSource BatchB      0          0          0          0          0
0.12
RequestSource RealTime      0          0          0          0          0
0.2
Router        InteractiveA      0.05287    0.045     0.17623    0.13217    0.3
0.3
Router        InteractiveB      0.021148  0.018     0.21148    0.05287    0.1
0.1
Router        BatchA      0.033956  0.03      0.22637    0.12576    0.15
0.15
Router        BatchB      0.029881  0.0264    0.24901    0.11067    0.12
0.12

... (64 captured-output lines omitted; rerun the source example for the full output)

AppServer     BatchA      0.46136    0.166     4.169      1.8992     0.10625
0.11066
AppServer     BatchB      0.72172    0.2733    4.7534     2.588      0.14985
0.15183
AppServer     RealTime    0.56093    0.062519  2.6916     2.6916     0.19198
0.2084
DataServer    InteractiveA 0.18431    0.089364  0.82499    0.43312    0.21823
0.22341
DataServer    InteractiveB 0.17774    0.08587   0.93144    0.44244    0.19128
0.19082
DataServer    BatchA      0.19004    0.099726  1.5245     0.6945     0.11066
0.12466
DataServer    BatchB      0.23906    0.13475   1.7741     0.96592    0.15183
0.13475
DataServer    RealTime    0.10682    0.043173  0.61854    0.61854    0.2084
0.17269

```

The rows repeat for each of the 3 solvers the script runs (MVA, FLD, SSA), which is the point of the example: the same model read by methods of different cost and different exactness.

2.1.10 oqn_nhpp

A cyclic non-homogeneous Poisson arrival stream, whose rate is a periodic function of time. The example is built and solved as follows.

```

Net m("model");
Source source(m, "Source");
Queue queue(m, "Queue", SchedStrategy::FCFS);

```

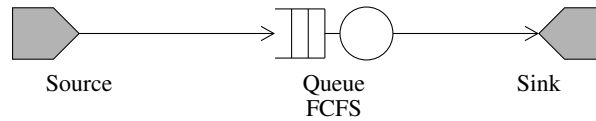



Figure 2.10: Topology of oqn_nhpp: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: OpenClass (open)
Scheduling	Queue: FCFS
Arrivals	Source – OpenClass: NHPP
Service	Queue – OpenClass: Exp(10)
Solvers	LDES

Table 2.10: Features of oqn_nhpp.

```

Sink sink(m, "Sink");
OpenClass oc(m, "OpenClass", 0);

// Rates 2, 8, 4 held for 3, 1, 2 time units, repeating cyclically.
const std::vector<double> bp = {0.0, 3.0, 4.0, 6.0};
const std::vector<double> rates = {2.0, 8.0, 4.0};
source.set_arrival(oc, D::nhpp(bp, rates, true));
queue.set_service(oc, Exp(10.0));

Routing P;
serial(P, {source, queue, sink});
m.link(P);
const Sn& sn = m.get_struct();

// The reference prints no banner here, so neither does this twin; the
// recorder still needs the attribution or the table is filed under no
// solver and dropped, and the row reads as "solver LDES missing".
attribute("LDES");
print_avg(sn, ldes_avg(m, sim_opts(1234, 100000)));

fluid::FluidOptions opt;
opt.timespan_end = 12.0;
const std::vector<fluid::FluidTranPoint> tran = fluid::solver_fluid_tran_avg(sn, opt);

... (6 source lines omitted; full implementation: cpp/examples/basic/)

note("FLD transient:");
const double probe[6] = {1.5, 3.5, 5.0, 7.5, 9.5, 11.0};
for (std::size_t i = 0; i < 6; ++i)
    std::printf(" t=%5.2f lambda(t)=%6.3f queue Tput=%6.3f\n", probe[i],
                rates[schedule_segment(probe[i], bp, true)], interp_at(probe[i], t, y));

```

Running this edition prints

Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Source	OpenClass	0	0	0	0	0
3.6667						
Queue	OpenClass	0.7244	0.36847	0.19681	0.19681	3.6846
3.6809						
FLD transient:						
t= 1.50 lambda(t)= 2.000 queue Tput= 2.000						
t= 3.50 lambda(t)= 8.000 queue Tput= 7.956						
t= 5.00 lambda(t)= 4.000 queue Tput= 4.000						
t= 7.50 lambda(t)= 2.000 queue Tput= 2.000						
t= 9.50 lambda(t)= 8.000 queue Tput= 7.956						
t=11.00 lambda(t)= 4.000 queue Tput= 4.000						

2.1.11 oqn_online

The same tandem written in one line from the arrival, demand and think time matrices, through `Network.tandemPsInf`.

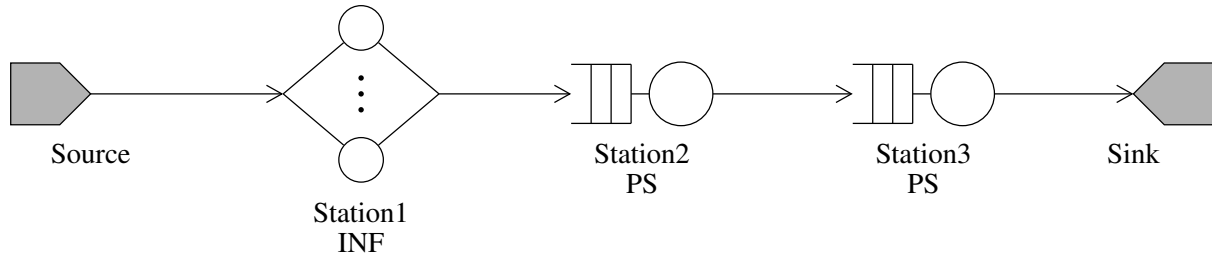


Figure 2.11: Topology of `oqn_online`: an open network over 3 queueing stations.

Nodes	5: Source, Queue $\times$ 2, Delay, Sink
Classes	2: Class1 (open), Class2 (open)
Scheduling	Station1: INF; Station2: PS; Station3: PS
Arrivals	Source – Class1: Exp(0.02); Class2: Exp(0.04)
Service	Station1 – Class1: Exp(0.01099); Class2: Exp(0.01087) Station2 – Class1: Exp(0.1); Class2: Exp(0.2) Station3 – Class1: Exp(0.2); Class2: Exp(0.1111)
Solvers	MVA

Table 2.11: Features of `oqn_online`.

The example is built and solved as follows.

```
note("This example shows a compact solution of a tandem open queueing network.");

const double lambda[2] = {1.0 / 50.0, 2.0 / 50.0};
const double Z[1][2] = {{91.0, 92.0}};
const double Dm[2][2] = {{10.0, 5.0}, {5.0, 9.0}};

Net m("Model");
std::vector<std::size_t> line;
line.push_back(m.add_source("Source"));
line.push_back(m.add_delay("Station1"));
line.push_back(m.add_queue("Station2", SchedStrategy::PS));
line.push_back(m.add_queue("Station3", SchedStrategy::PS));
line.push_back(m.add_sink("Sink"));

const std::vector<std::size_t> cls = {m.add_open_class("Class1"), m.add_open_class("Class2")};
for (std::size_t r = 0; r < 2; ++r) {
    m.set_arrival(line[0], cls[r], D::exp_mean(1.0 / lambda[r]));
    m.set_service(line[1], cls[r], D::exp_mean(Z[0][r]));
    for (std::size_t i = 0; i < 2; ++i) m.set_service(line[i + 2], cls[r], D::exp_mean(Dm[i][r]));
}

Routing P;
for (std::size_t r = 0; r < 2; ++r) serial(P, cls[r], line);
m.link(P);
const Sn& sn = m.get_struct();

section("MVA");
Matrix<double> init;
print_avg(sn, mva::solver_mva_run_analyzer(sn, mva::MvaOptions(), init));
```

Running this edition prints

```
This example shows a compact solution of a tandem open queueing network.
```

SOLVER: MVA						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Source	Class1	0	0	0	0	0
0.02						
Source	Class2	0	0	0	0	0
0.04						
Station1	Class1	1.82	1.82	91	91	0.02
0.02						
Station1	Class2	3.68	3.68	92	92	0.04
0.04						
Station2	Class1	0.33333	0.2	16.667	16.667	0.02
0.02						
Station2	Class2	0.33333	0.2	8.3333	8.3333	0.04
0.04						
Station3	Class1	0.18518	0.1	9.2592	9.2592	0.02
0.02						
Station3	Class2	0.66667	0.36	16.667	16.667	0.04
0.04						

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by MVA.

2.1.12 oqn_trace_driven

Arrivals and service times replayed from a measurement trace with Replayer.

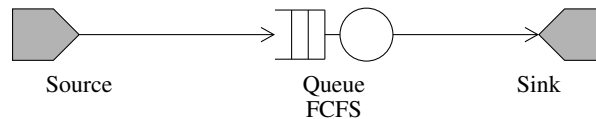


Figure 2.12: Topology of oqn_trace_driven: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: OpenClass (open)
Scheduling	Queue: FCFS
Arrivals	Source – OpenClass: Exp(1)
Service	Queue – OpenClass: Replayer(trace)
Solvers	JMT, LDES

Table 2.12: Features of oqn_trace_driven.

The example is built and solved as follows.

```

Net m("model");
Source source(m, "Source");
Queue queue(m, "Queue", SchedStrategy::FCFS);
Sink sink(m, "Sink");
OpenClass oc(m, "OpenClass", 0);

source.set_arrival(oc, Exp(1.0));
queue.set_service(oc,
    replayer_from_file(example_data_file("openQN/example_trace.txt")));

Routing P;
serial(P, {source, queue, sink});
m.link(P);
m.get_struct();

section("JMT");
print_avg(m.get_struct(), jmt_avg(m, sim_opts(23000)));
section("LDES");
print_avg(m.get_struct(), ldes_avg(m, sim_opts(23000)));

```

Running this edition prints

SOLVER: JMT						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Source	OpenClass	0	0	0	0	0
1.0205						
Queue	OpenClass	0.1128	0.10284	0.11275	0.11275	1.0205
1.0211						
SOLVER: LDES						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Source	OpenClass	0	0	0	0	0
1						
Queue	OpenClass	0.11241	0.10112	0.11236	0.11236	1.0014
1.0004						

The rows repeat for each of the 2 solvers the script runs (JMT, LDES), which is the point of the example: the same model read by methods of different cost and different exactness.

2.1.13 oqn_vsinks

Several sinks in one model, the virtual-sink idiom for measuring departure streams separately.

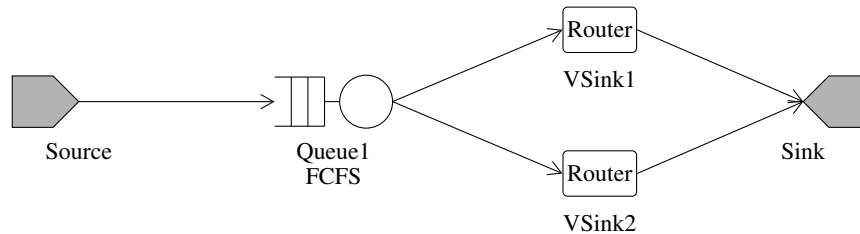


Figure 2.13: Topology of oqn_vsinks: an open network over 1 queueing station with a router.

Nodes	5: Source, Queue, Router × 2, Sink
Classes	2: Class1 (open), Class2 (open)
Scheduling	Queue1: FCFS
Arrivals	Source – Class1: Exp(1); Class2: Exp(1)
Service	Queue1 – Class1: Exp(100); Class2: Exp(100)
Solvers	MVA, MAM, NC

Table 2.13: Features of oqn_vsinks.

The example is built and solved as follows.

```

Net m("model");
Source source(m, "Source");
Queue queue(m, "Queue1", SchedStrategy::FCFS);
Sink sink(m, "Sink");
Router vsink1(m, "VSink1");
Router vsink2(m, "VSink2");

OpenClass c1(m, "Class1");
OpenClass c2(m, "Class2");

source.set_arrival(c1, Exp(1.0));
queue.set_service(c1, Exp(100.0));
source.set_arrival(c2, Exp(1.0));
queue.set_service(c2, Exp(100.0));

Routing P;
P.set(c1, c1, source, queue, 1.0);
P.set(c1, c1, queue, vsink1, 0.6);

```

```

P.set(c1, c1, queue, vsink2, 0.4);
P.set(c1, c1, vsink1, sink, 1.0);
P.set(c1, c1, vsink2, sink, 1.0);

P.set(c2, c2, source, queue, 1.0);
P.set(c2, c2, queue, vsink1, 0.1);
P.set(c2, c2, queue, vsink2, 0.9);
P.set(c2, c2, vsink1, sink, 1.0);

... (11 source lines omitted; full implementation: cpp/examples/basic/)

print_avg(sn, mam::solver_mam_run_analyzer(sn, mam::MamOptions()));
na("MAM getAvgNodeTable", "the node-level table has no C++ counterpart");

section("NC");
print_avg(sn, nc::solver_nc_run_analyzer(sn, nc::NcSolverOptions()));
na("NC getAvgNodeTable", "the node-level table has no C++ counterpart");

```

Running this edition prints

```

SOLVER: MVA
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
  Tput
Source      Class1          0          0          0          0          0
  1
Source      Class2          0          0          0          0          0
  1
Queue1      Class1      0.010204    0.01    0.010204    0.010204    1
  1
Queue1      Class2      0.010204    0.01    0.010204    0.010204    1
  1

SOLVER: MVA getAvgNodeTable
N/A: the node-level table, which is where the virtual sinks appear, has no C++ counterpart

SOLVER: MAM
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
  Tput
Source      Class1          0          0          0          0          0
  1
Source      Class2          0          0          0          0          0
  1
Queue1      Class1      0.010204    0.01    0.010204    0.010204    1
  1
Queue1      Class2      0.010204    0.01    0.010204    0.010204    1
  1

... (5 captured-output lines omitted; rerun the source example for the full output)

Station      JobClass      QLen      Util      RespT      ResidT      ArvR
  Tput
Source      Class1          0          0          0          0          0
  1
Source      Class2          0          0          0          0          0
  1
Queue1      Class1      0.010204    0.01    0.010204    0.010204    1
  1
Queue1      Class2      0.010204    0.01    0.010204    0.010204    1
  1

SOLVER: NC getAvgNodeTable
N/A: the node-level table has no C++ counterpart

```

The rows repeat for each of the 3 solvers the script runs (MVA, MAM, NC), which is the point of the example: the same model read by methods of different cost and different exactness.

2.2 Closed queueing networks

A closed network holds a fixed population per class. Each closed class names a reference station, at which its visit ratio is one and through which its throughput is measured; the population is the third argument of the class constructor. A Delay station with think time Z and a queue with demand D is the

interactive-system archetype, and the repairman model is its oldest form.

Because the population is finite, every metric is bounded and the network is stable by construction. What varies across the scripts below is the scheduling discipline, the service distribution and the number of servers, that is, exactly the assumptions under which a product-form solution does or does not hold.

2.2.1 cqn_bas_blocking

Blocking-after-service with finite buffers, where a completing job holds its server until the destination frees a slot.

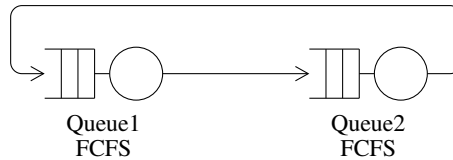


Figure 2.14: Topology of `cqn_bas_blocking`: a closed network over 2 queueing stations.

Nodes	2: Queue $\times$ 2
Classes	1: Class1 (closed, $N = 2$)
Scheduling	Queue1: FCFS; Queue2: FCFS
Service	Queue1 – Class1: Exp(1) Queue2 – Class1: Exp(0.8)
Features	drop rule {'Class1': 'blockingAfterService'} at Queue1; finite buffer 1 at Queue2
Solvers	MVA

Table 2.14: Features of `cqn_bas_blocking`.

The example is built and solved as follows.

```

Net m("cqn_bas_blocking");
Queue q1(m, "Queue1", SchedStrategy::FCFS);
Queue q2(m, "Queue2", SchedStrategy::FCFS);
ClosedClass c1(m, "Class1", 2.0, q1, 0);

q1.set_service(c1, Exp(1.0));
q2.set_service(c1, Exp(0.8));
q2.set_capacity(1.0);
q1.set_drop_rule(c1, lang::DropStrategy::BAS);

Routing P;
cyclic(P, c1, {q1, q2});
m.link(P);
const Sn& sn = m.get_struct();

// The method is presentation: the golden keys this table plain `MVA`.
section("MVA (method=sqd)", "MVA");
run_mva(sn, "sqd");

```

Running this edition prints

SOLVER: MVA (method=sqd)						
Station	JobClass	QLen	Util	RespT	ResidT	ArrR
Tput						
Queue1	Class1	0.91777	0.54051	1.698	1.698	0.54051
0.54051						
Queue2	Class1	1.0822	0.67563	2.0023	2.0023	0.54051
0.54051						

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by MVA.

2.2.2 cqn_bcmp_theorem

The product-form scheduling policies of the BCMP theorem (FCFS with class-independent exponential service, PS, LCFS-PR, INF) compared on one network.

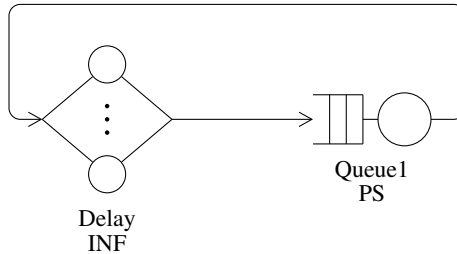


Figure 2.15: Topology of cqn_bcmp_theorem: a closed network over 2 queueing stations.

Nodes	2: Queue, Delay
Classes	2: Class1 (closed, $N = 2$), Class2 (closed, $N = 2$)
Scheduling	Delay: INF; Queue1: PS
Service	Delay – Class1: Erlang(3,2); Class2: HyperExp(0.5,0.5;3,10) Queue1 – Class1: Exp(1); Class2: Exp(1)
Solvers	CTMC

Table 2.15: Features of cqn_bcmp_theorem.

The example is built and solved as follows.

```
const char* names[3] = {"PS scheduling model", "FCFS scheduling model",
                        "LCFS-PR scheduling model"};
const SchedStrategy scheds[3] = {SchedStrategy::PS, SchedStrategy::FCFS,
                                  SchedStrategy::LCFS-PR};
for (std::size_t k = 0; k < 3; ++k) {
    Net m = bcmp_model(names[k], scheds[k]);
    const Sn& sn = m.get_struct();
    note(std::string("\nMODEL: ") + names[k]);
    // The reference prints the model name and the table, with no solver
    // banner, so the declaration is made to the recorder alone. All three
    // models answer identically -- that IS the BCMP theorem this example
    // demonstrates -- so which of the three the golden holds does not
    // change a digit.
    attribute("CTMC");
    run_ctmc(sn);
}
```

Running this edition prints

```
MODEL: PS scheduling model
Station  JobClass      QLen      Util      RespT      ResidT      ArvR
Tput
Delay    Class1         0.30861    0.30861    0.66667    0.66667    0.46291
0.46291
Delay    Class2         0.11625    0.11625    0.21667    0.21667    0.53653
0.53653
Queue1   Class1         1.6914     0.46291    3.6538     3.6538     0.46291
0.46291
Queue1   Class2         1.8838     0.53653    3.511      3.511      0.53653
0.53653

MODEL: FCFS scheduling model
Station  JobClass      QLen      Util      RespT      ResidT      ArvR
Tput
Delay    Class1         0.30861    0.30861    0.66667    0.66667    0.46291
0.46291
Delay    Class2         0.11625    0.11625    0.21667    0.21667    0.53653
0.53653
```

Queue1	Class1	1.6914	0.46291	3.6538	3.6538	0.46291
0.46291						
Queue1	Class2	1.8838	0.53653	3.511	3.511	0.53653
0.53653						
MODEL: LCFS-PR scheduling model						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Delay	Class1	0.30861	0.30861	0.66667	0.66667	0.46291
0.46291						
Delay	Class2	0.11625	0.11625	0.21667	0.21667	0.53653
0.53653						
Queue1	Class1	1.6914	0.46291	3.6538	3.6538	0.46291
0.46291						
Queue1	Class2	1.8838	0.53653	3.511	3.511	0.53653
0.53653						

2.2.3 cqn_lcfs_multiclass

Multiclass LCFS and LCFS-PR scheduling, and the difference preemption makes.

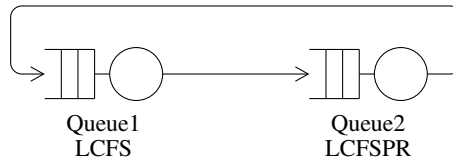


Figure 2.16: Topology of cqn_lcfs_multiclass: a closed network over 2 queueing stations.

Nodes	2: Queue $\times$ 2
Classes	3: Class1 (closed, $N = 1$), Class2 (closed, $N = 1$), Class3 (closed, $N = 1$)
Scheduling	Queue1: LCFS; Queue2: LCFS-PR
Service	Queue1 – Class1: Exp(1); Class2: Exp(0.3333); Class3: Exp(0.2) Queue2 – Class1: Exp(0.5); Class2: Exp(0.25); Class3: Exp(0.1667)
Solvers	MVA, CTMC

Table 2.16: Features of cqn_lcfs_multiclass.

The example is built and solved as follows.

```
// mu(i,r), the service RATE of class r at station i: 1./[[1,3,5],[2,4,6]].
const double mu[2][3] = {{1.0 / 1.0, 1.0 / 3.0, 1.0 / 5.0},
                          {1.0 / 2.0, 1.0 / 4.0, 1.0 / 6.0}};

Net m("LCFS Multiclass Model");
Queue q1(m, "Queue1", SchedStrategy::LCFS);
Queue q2(m, "Queue2", SchedStrategy::LCFS-PR);

std::vector<std::size_t> cls;
for (std::size_t r = 0; r < 3; ++r)
    cls.push_back(m.add_closed_class("Class" + std::to_string(r + 1), 1.0, q1, 0));
for (std::size_t r = 0; r < 3; ++r) {
    q1.set_service(cls[r], Exp(mu[0][r]));
    q2.set_service(cls[r], Exp(mu[1][r]));
}

Routing P;
for (std::size_t r = 0; r < 3; ++r) cyclic(P, cls[r], {q1, q2});
m.link(P);
const Sn& sn = m.get_struct();

note("Solving LCFS+LCFS-PR network with MVA...");
section("MVA");
run_mva(sn, "exact");
```



```

note("\nSolving with CTMC for validation...");
section("CTMC");
run_ctmc(sn);

note("\n=== Comparison ===");
note("MVA and CTMC results should match for this product-form network.");

```

Running this edition prints

```

Solving LCFS+LCFS-PR network with MVA...

SOLVER: MVA
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
  Tput
Queue1      Class1      0.46099    0.20432    2.2562     2.2562     0.20432
0.20432
Queue1      Class2      0.31239    0.18744     5         5         0.062479
0.062479
Queue1      Class3      0.2381     0.17055    6.9802     6.9802     0.03411
0.03411
Queue2      Class1      0.53901    0.40865     2.638     2.638     0.20432
0.20432
Queue2      Class2      0.68761    0.24992    11.005     11.005     0.062479
0.062479
Queue2      Class3      0.7619     0.20466    22.337     22.337     0.03411
0.03411

Solving with CTMC for validation...

SOLVER: CTMC
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
  Tput
Queue1      Class1      0.46099    0.20432    2.2562     2.2562     0.20432
0.20432
Queue1      Class2      0.31239    0.18744     5         5         0.062479
0.062479
Queue1      Class3      0.2381     0.17055    6.9802     6.9802     0.03411
0.03411
Queue2      Class1      0.53901    0.40865     2.638     2.638     0.20432
0.20432
Queue2      Class2      0.68761    0.24992    11.005     11.005     0.062479
0.062479
Queue2      Class3      0.7619     0.20466    22.337     22.337     0.03411
0.03411

=== Comparison ===
MVA and CTMC results should match for this product-form network.

```

The rows repeat for each of the 2 solvers the script runs (MVA, CTMC), which is the point of the example: the same model read by methods of different cost and different exactness.

2.2.4 cqn_mmpp2_service

MMPP(2) service times, a correlated service process that no product-form solver can represent exactly.

Nodes	3: Queue $\times$ 2, Delay
Classes	2: Class1 (closed, $N = 1$), Class2 (closed, $N = 1$)
Scheduling	Delay: INF; Queue1: PS; Queue2: PS
Service	Delay – Class1: Erlang(3,2); Class2: HyperExp(0.5,0.5;3,10) Queue1 – Class1: HyperExp(0.1,0.9;1,10); Class2: MMPP2 Queue2 – Class1: HyperExp(0.1,0.9;1,10); Class2: Erlang(1,2)
Features	class switching on 3 links
Solvers	JMT, LDES

Table 2.17: Features of cqn_mmpp2_service.

The example is built and solved as follows.

```

Net m("model");
Delay delay(m, "Delay");

```

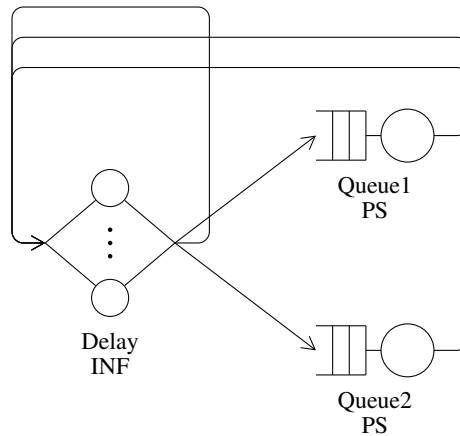


Figure 2.17: Topology of `cqn_mmpp2_service`: a closed network over 3 queueing stations.

```

Queue q1(m, "Queue1", SchedStrategy::PS);
Queue q2(m, "Queue2", SchedStrategy::PS);

ClosedClass c1(m, "Class1", 1.0, delay, 0);
ClosedClass c2(m, "Class2", 1.0, delay, 0);

delay.set_service(c1, Erlang(3.0, 2));
delay.set_service(c2, HyperExp(0.5, 3.0, 10.0));
q1.set_service(c1, HyperExp(0.1, 1.0, 10.0));
q1.set_service(c2, mmpp2(1.0, 2.0, 3.0, 4.0));
q2.set_service(c1, HyperExp(0.1, 1.0, 10.0));
q2.set_service(c2, Erlang(1.0, 2));

Routing P;
P.set(c1, c1, delay, q1, 0.3);
P.set(c1, c1, delay, q2, 0.7);
P.set(c1, c1, q1, delay, 1.0);
P.set(c1, c1, q2, delay, 1.0);
// RAND needs the CONNECTIONS only; the refresh replaces them by the uniform
// split, so the Delay's self-link is one of the three destinations.
m.set_routing(delay, c2, RoutingStrategy::RAND);
m.set_routing(q1, c2, RoutingStrategy::RAND);
m.set_routing(q2, c2, RoutingStrategy::RAND);
P.set(c2, c2, delay, delay, 1.0);

... (5 source lines omitted; full implementation: cpp/examples/basic/)

m.get_struct();

section("JMT");
print_avg(m.get_struct(), jmt_avg(m, sim_opts(23000)));
section("LDES");
print_avg(m.get_struct(), ldes_avg(m, sim_opts(23000)));

```

Running this edition prints

SOLVER: JMT						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Delay	Class1	0.70972	0.70972	0.67253	0.67253	1.0537
1.0537						
Delay	Class2	0.16657	0.16657	0.21263	0.21263	0.79236
0.79405						
Queue1	Class1	0.066141	0.058043	0.21991	0.065973	0.32032
0.32032						
Queue1	Class2	0.18831	0.18064	0.72928	0.24309	0.27172
0.27173						
Queue2	Class1	0.22291	0.13915	0.29634	0.20744	0.74075
0.74564						
Queue2	Class2	0.64052	0.55218	2.3056	0.76855	0.27048

0.27073

SOLVER: LDES

N/A: SolverLDES(model, seed=23000) has no C++ engine in this port

The rows repeat for each of the 2 solvers the script runs (JMT, LDES), which is the point of the example: the same model read by methods of different cost and different exactness.

2.2.5 cqn_multichain_cs

Five classes in three chains, the closed counterpart of oqn_multichain_cs.

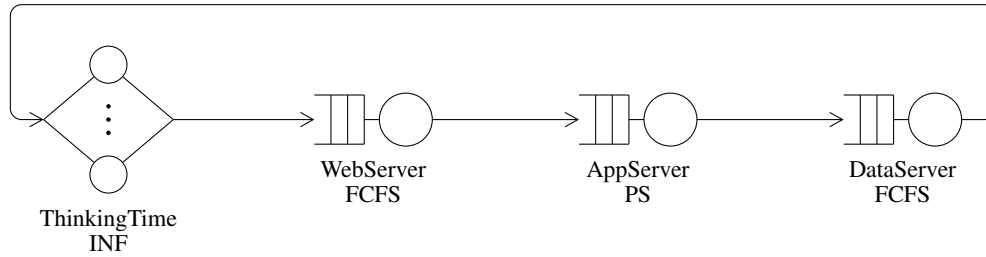


Figure 2.18: Topology of cqn_multichain_cs: a closed network over 4 queueing stations.

Nodes	4: Queue $\times$ 3, Delay
Classes	5: HighPriorityA (closed, $N = 2$), HighPriorityB (closed, $N = 1$), RegularA (closed, $N = 3$), RegularB (closed, $N = 2$), Background (closed, $N = 2$)
Scheduling	ThinkingTime: INF; WebServer: FCFS; AppServer: PS; DataServer: FCFS
Service	ThinkingTime – HighPriorityA: Exp(0.6667); HighPriorityB: Exp(0.625); RegularA: Exp(0.5); RegularB: Exp(0.4762); Background: Exp(0.3333) WebServer – HighPriorityA: Exp(2.5); HighPriorityB: Exp(2.381); RegularA: Exp(2); RegularB: Exp(1.923); Background: Exp(1.25) AppServer – HighPriorityA: Exp(1.429); HighPriorityB: Exp(1.333); RegularA: Exp(1.111); RegularB: Exp(1.053); Background: Exp(0.8333) DataServer – HighPriorityA: Exp(2); HighPriorityB: Exp(1.923); RegularA: Exp(1.667); RegularB: Exp(1.613); Background: Exp(1)
Solvers	MVA, SSA

Table 2.18: Features of cqn_multichain_cs.

The example is built and solved as follows.

```

Net m("cqn_multichain");
Delay think(m, "ThinkingTime");
Queue web(m, "WebServer", SchedStrategy::FCFS);
Queue app(m, "AppServer", SchedStrategy::PS);
Queue data(m, "DataServer", SchedStrategy::FCFS);

const double pop[5] = {2.0, 1.0, 3.0, 2.0, 2.0};
const char* names[5] = {"HighPriorityA", "HighPriorityB", "RegularA", "RegularB",
                        "Background"};
std::vector<std::size_t> cls;
for (std::size_t r = 0; r < 5; ++r) cls.push_back(m.add_closed_class(names[r], pop[r], think));

const std::size_t st[4] = {think, web, app, data};
const double svc[4][5] = {{1.5, 1.6, 2.0, 2.1, 3.0},
                          {0.4, 0.42, 0.5, 0.52, 0.8},
                          {0.7, 0.75, 0.9, 0.95, 1.2},
                          {0.5, 0.52, 0.6, 0.62, 1.0}};
for (std::size_t i = 0; i < 4; ++i)
    for (std::size_t r = 0; r < 5; ++r) m.set_service(st[i], cls[r], D::exp_mean(svc[i][r]));

Routing P;
for (std::size_t r = 0; r < 5; ++r) {

```

```

P.set(cls[r], cls[r], think, web, 1.0);
P.set(cls[r], cls[r], app, data, 1.0);
P.set(cls[r], cls[r], data, think, 1.0);
}

... (19 source lines omitted; full implementation: cpp/examples/basic/)

ssa::SsaOptions sopt;
sopt.samples = 200000;
sopt.seed = 23000;
const ssa::SsaSolution sr = ssa::solver_ssa(sn, sopt);
print_avg_chain(sn, solvers::avg_result_from_sim<double>(sn, sr.QN, sr.UN, sr.RN, sr.TN, sr.
    CN,
                                sr.XN, sr.method));

```

Running this edition prints

SOLVER: MVA							
Station	Chain	JobClasses	QLen	Util	RespT	ResidT	
ArvR	Tput						
ThinkingTime	Chain1	(HighPriorityA HighPriorityB)	0.573738	0.573738	1.55455		
1.55455	0.369071	0.369071					
ThinkingTime	Chain2	(RegularA RegularB)	1.0321	1.0321	2.05385	2.05385	
0.502518	0.502518						
ThinkingTime	Chain3	(Background)	0.458901	0.458901	3	3	
0.152967	0.152967						
WebServer	Chain1	(HighPriorityA HighPriorityB)	0.339963	0.151655	0.921131		
0.921131	0.369071	0.369071					
WebServer	Chain2	(RegularA RegularB)	0.510272	0.256671	1.01543	1.01543	
0.502518	0.502518						
WebServer	Chain3	(Background)	0.195577	0.122374	1.27856	1.27856	
0.152967	0.152967						
AppServer	Chain1	(HighPriorityA HighPriorityB)	1.54564	0.268415	4.18791		
4.18791	0.369071	0.369071					
AppServer	Chain2	(RegularA RegularB)	2.67318	0.465796	5.31957	5.31957	
0.502518	0.502518						
AppServer	Chain3	(Background)	1.05477	0.18356	6.89544	6.89544	
0.152967	0.152967						
DataServer	Chain1	(HighPriorityA HighPriorityB)	0.540664	0.188562	1.46493		
1.46493	0.369071	0.369071					
DataServer	Chain2	(RegularA RegularB)	0.784453	0.306923	1.56104	1.56104	
0.502518	0.502518						
DataServer	Chain3	(Background)	0.290748	0.152967	1.90072	1.90072	
0.152967	0.152967						
SOLVER: SSA							
... (5 captured-output lines omitted; rerun the source example for the full output)							
WebServer	Chain2	(RegularA RegularB)	0.573643	0.264198	1.10903	1.10903	
0.516371	0.517335						
WebServer	Chain3	(Background)	0.21746	0.128946	1.34916	1.34916	
0.157035	0.161182						
AppServer	Chain1	(HighPriorityA HighPriorityB)	1.44005	0.272338	3.84577		
3.84577	0.37893	0.374363					
AppServer	Chain2	(RegularA RegularB)	2.48111	0.482233	4.76915	4.76915	
0.517335	0.520204						
AppServer	Chain3	(Background)	0.991373	0.194652	6.11167	6.11167	
0.161182	0.16221						
DataServer	Chain1	(HighPriorityA HighPriorityB)	0.591666	0.192222	1.57251		
1.57251	0.374363	0.376379					
DataServer	Chain2	(RegularA RegularB)	0.884629	0.317389	1.70236	1.70236	
0.520204	0.519638						
DataServer	Chain3	(Background)	0.320063	0.158842	2.01497	2.01497	
0.16221	0.158842						

The rows repeat for each of the 2 solvers the script runs (MVA, SSA), which is the point of the example: the same model read by methods of different cost and different exactness.

2.2.6 cq_n_multiserver

A closed network with multiserver FCFS and PS stations, Erlang service and a class disabled at one station.

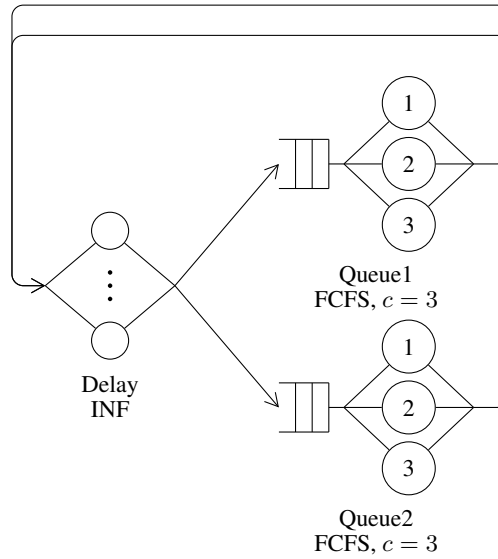


Figure 2.19: Topology of `cqn_multiserver`: a closed network over 3 queueing stations.

Nodes	3: Queue $\times$ 2, Delay
Classes	4: Class1 (closed, $N = 2$), Class2 (closed, $N = 2$), Class3 (closed, $N = 2$), Class4 (closed, $N = 1$)
Scheduling	Delay: INF; Queue1: FCFS ($c = 3$); Queue2: FCFS ($c = 3$)
Service	Delay – Class1: Exp(1); Class2: Exp(1); Class3: Exp(10); Class4: Exp(1) Queue1 – Class1: Exp(1); Class2: Erlang(1,2); Class3: Exp(10); Class4: Exp(1) Queue2 – Class1: Disabled; Class2: Disabled; Class3: Erlang(1,2); Class4: Exp(1)
Solvers	MVA

Table 2.19: Features of `cqn_multiserver`.

The example is built and solved as follows.

```

Net m("model");
Delay delay(m, "Delay");
Queue q1(m, "Queue1", SchedStrategy::FCFS);
Queue q2(m, "Queue2", SchedStrategy::FCFS);
q1.set_number_of_servers(3.0);
q2.set_number_of_servers(3.0);

const double pop[4] = {2.0, 2.0, 2.0, 1.0};
std::vector<std::size_t> cls;
for (std::size_t r = 0; r < 4; ++r)
    cls.push_back(m.add_closed_class("Class" + std::to_string(r + 1), pop[r], delay, 0));

delay.set_service(cls[0], Exp(1.0));
delay.set_service(cls[1], Exp(1.0));
delay.set_service(cls[2], Exp(10.0));
delay.set_service(cls[3], Exp(1.0));

q1.set_service(cls[0], Exp(1.0));
q1.set_service(cls[1], Erlang(1.0, 2));
q1.set_service(cls[2], Exp(10.0));
q1.set_service(cls[3], Exp(1.0));

q2.set_service(cls[0], Disabled());
q2.set_service(cls[1], Disabled());
q2.set_service(cls[2], Erlang(1.0, 2));
q2.set_service(cls[3], Exp(1.0));

... (32 source lines omitted; full implementation: cpp/examples/basic/)

const std::vector<std::size_t> started = {1, 1, 1, 1};
print_states("State (MarginalAndStarted)",
            qn::from_marginal_and_started(sn, ist, n, started, phases));

section("MVA");
run_mva(sn);

```

Running this edition prints

```

N/A: State.fromMarginalAndRunning has no C++ counterpart; State.fromMarginal and State.
fromMarginalAndStarted are ported
State (Marginal): 96 state(s)
[4 3 2 0 1 0 0]
[4 3 2 1 0 0 0]
[4 3 2 0 1 0 0]
[4 3 2 1 0 0 0]
[4 3 2 0 1 0 0]
[4 3 2 1 0 0 0]
[4 2 2 0 0 1 0]
[4 2 2 0 0 1 0]
[4 2 2 0 0 1 0]
[4 1 1 0 1 1 0]
[4 1 1 1 0 1 0]
[4 1 1 0 1 1 0]
[4 1 1 1 0 1 0]
[4 1 1 0 1 1 0]
[4 1 1 1 0 1 0]

... (87 captured-output lines omitted; rerun the source example for the full output)

Delay          Class2          0.47603    0.47603         1    0.33333    0.47603
0.47603
Delay          Class3          0.13232    0.13232        0.1    0.066667   1.3232
1.3232
Delay          Class4          0.66159    0.66159         1    0.33333    0.66159
0.66159
Queue1         Class1          1.286     0.31736       1.3507   0.90046    0.95207
0.95207
Queue1         Class2          1.286     0.31736       2.7014   0.90046    0.47603
0.47603
Queue1         Class3          0.19955    0.03308       0.20108   0.10054    0.99239
0.99239
Queue1         Class4          1.3303    0.22053       2.0108   0.67026    0.66159
0.66159

```

Queue2	Class3	0.67621	0.22053	2.0442	0.3407	0.3308
0.3308						

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by MVA.

2.2.7 cqn_multiserver_nc

Delay plus a multiserver FCFS queue, solved by the normalizing constant method.

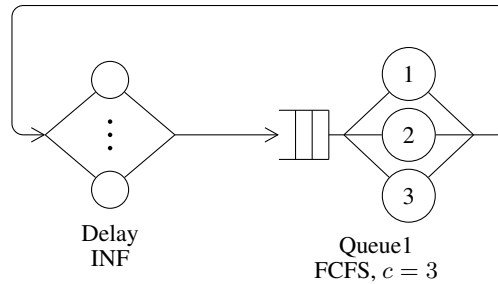


Figure 2.20: Topology of `cqn_multiserver_nc`: a closed network over 2 queueing stations.

Nodes	2: Queue, Delay
Classes	1: Class1 (closed, $N = 5$)
Scheduling	Delay: INF; Queue1: FCFS ($c = 3$)
Service	Delay – Class1: Exp(1) Queue1 – Class1: Exp(1.25)
Solvers	CTMC, MVA, NC

Table 2.20: Features of `cqn_multiserver_nc`.

The example is built and solved as follows.

```
Net m("model");
Delay delay(m, "Delay");
Queue q1(m, "Queue1", SchedStrategy::FCFS);
q1.set_number_of_servers(3.0);

ClosedClass c1(m, "Class1", 5.0, delay, 0);
delay.set_service(c1, D::exp_mean(1.0));
q1.set_service(c1, D::exp_mean(0.8));

Routing P;
cyclic(P, c1, {delay, q1});
m.link(P);
const Sn& sn = m.get_struct();

section("CTMC");
run_ctmc(sn);
section("MVA");
run_mva(sn);
section("NC");
run_nc(sn, "exact");
```

Running this edition prints

SOLVER: CTMC						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Delay	Class1	2.6613	2.6613	1	1	2.6613
2.6613						
Queue1	Class1	2.3387	0.70969	0.87875	0.87875	2.6613
2.6613						

SOLVER: MVA						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Delay	Class1	2.6613	2.6613	1	1	2.6613
2.6613						
Queue1	Class1	2.3387	0.70969	0.87875	0.87875	2.6613
2.6613						
SOLVER: NC						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Delay	Class1	2.6613	2.6613	1	1	2.6613
2.6613						
Queue1	Class1	2.3387	0.70969	0.87875	0.87875	2.6613
2.6613						

The rows repeat for each of the 3 solvers the script runs (CTMC, MVA, NC), which is the point of the example: the same model read by methods of different cost and different exactness.

2.2.8 cqn_online

The closed counterpart of the one-line construction, from the demand and think time matrices.

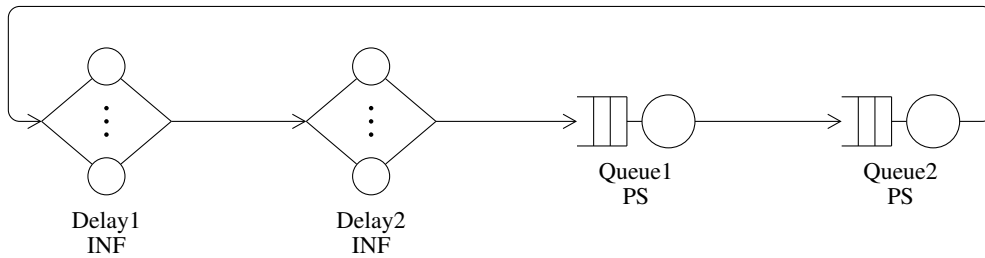


Figure 2.21: Topology of `cqn_online`: a closed network over 4 queueing stations.

Nodes	4: Queue $\times$ 2, Delay $\times$ 2
Classes	2: Class1 (closed, $N = 1$), Class2 (closed, $N = 2$)
Scheduling	Delay1: INF; Delay2: INF; Queue1: PS; Queue2: PS
Service	Delay1 – Class1: Exp(0.01099); Class2: Exp(0.01087) Delay2 – Class1: Exp(0.01075); Class2: Exp(0.01064) Queue1 – Class1: Exp(0.1); Class2: Exp(0.2) Queue2 – Class1: Exp(0.2); Class2: Exp(0.1111)
Solvers	MVA

Table 2.21: Features of `cqn_online`.

The example is built and solved as follows.

```
const double Dm[2][2] = {{10.0, 5.0}, {5.0, 9.0}};
const double N[2] = {1.0, 2.0};
const double Z[2][2] = {{91.0, 92.0}, {93.0, 94.0}};

Net m("Model");
std::vector<std::size_t> st;
st.push_back(m.add_delay("Delay1"));
st.push_back(m.add_delay("Delay2"));
st.push_back(m.add_queue("Queue1", SchedStrategy::PS));
st.push_back(m.add_queue("Queue2", SchedStrategy::PS));

std::vector<std::size_t> cls;
for (std::size_t r = 0; r < 2; ++r)
    cls.push_back(m.add_closed_class("Class" + std::to_string(r + 1), N[r], st[0]));
for (std::size_t r = 0; r < 2; ++r)
    for (std::size_t i = 0; i < 2; ++i) {
        m.set_service(st[i], cls[r], Exp(1.0 / Z[i][r]));
        m.set_service(st[i + 2], cls[r], Exp(1.0 / Dm[i][r]));
    }
```



```

    }

    Routing P;
    for (std::size_t r = 0; r < 2; ++r) cyclic(P, cls[r], st);
    m.link(P);
    const Sn& sn = m.get_struct();

    section("MVA");
    run_mva(sn, "exact");

```

Running this edition prints

SOLVER: MVA						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Delay1	Class1	0.45505	0.45505	91	91	0.0050005
0.0050005						
Delay1	Class2	0.91525	0.91525	92	92	0.0099484
0.0099484						
Delay2	Class1	0.46505	0.46505	93	93	0.0050005
0.0050005						
Delay2	Class2	0.93515	0.93515	94	94	0.0099484
0.0099484						
Queue1	Class1	0.052561	0.050005	10.511	10.511	0.0050005
0.0050005						
Queue1	Class2	0.053601	0.049742	5.3879	5.3879	0.0099484
0.0099484						
Queue2	Class1	0.027348	0.025002	5.469	5.469	0.0050005
0.0050005						
Queue2	Class2	0.096001	0.089535	9.6499	9.6499	0.0099484
0.0099484						

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by MVA.

2.2.9 cqn_repairmen

The machine repairman model: a Delay station of thinking machines and one FCFS repair queue.

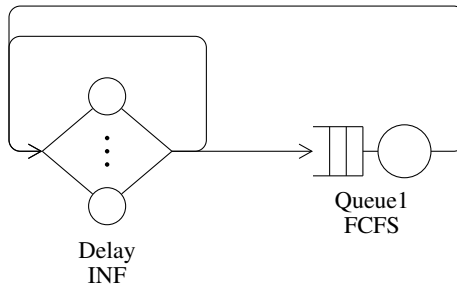


Figure 2.22: Topology of cqn_repairmen: a closed network over 2 queueing stations.

Nodes	2: Queue, Delay
Classes	1: Class1 (closed, $N = 10$)
Scheduling	Delay: INF; Queue1: FCFS
Service	Delay – Class1: Exp(1) Queue1 – Class1: Exp(0.6667)
Solvers	CTMC, JMT, SSA, FLD, MVA, NC, MAM, LDES

Table 2.22: Features of cqn_repairmen.

The example is built and solved as follows.

```

Net m("model");
Delay delay(m, "Delay");
Queue q1(m, "Queue1", SchedStrategy::FCFS);

```

```

ClosedClass c1(m, "Class1", 10.0, delay, 0);

delay.set_service(c1, D::exp_mean(1.0));
q1.set_service(c1, D::exp_mean(1.5));

Routing P;
P.set(c1, c1, delay, delay, 0.7);
P.set(c1, c1, delay, q1, 0.3);
P.set(c1, c1, q1, delay, 1.0);
m.link(P);
const Sn& sn = m.get_struct();

section("CTMC");
run_ctmc(sn);
section("JMT");
print_avg(m.get_struct(), jmt_avg(m, sim_opts(23000)));
section("SSA");
run_ssa(sn, 5000, 23000);
section("FLD");
run_fluid(sn);
section("MVA");
run_mva(sn);
section("NC");
run_nc(sn, "exact");
section("MAM");
run_mam(sn);
section("LDES");
print_avg(m.get_struct(), ldes_avg(m, sim_opts(23000, 5000)));

```

Running this edition prints

```

SOLVER: CTMC
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
  Tput
Delay        Class1      2.222      2.222      1          1          2.222
  2.222
Queue1       Class1      7.778      0.99991    11.668      3.5004      0.66661
  0.66661

SOLVER: JMT
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
  Tput
Delay        Class1      2.1909     2.1909     1.0081      1.0081      2.2124
  2.2129
Queue1       Class1      7.7331     0.99977    11.573      3.472       0.67574
  0.68022

SOLVER: SSA
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
  Tput
Delay        Class1      2.2376     2.2376     1          1          2.2325
  2.2376
Queue1       Class1      7.7624     0.99925    11.652      3.4957      0.67127
  0.66617

SOLVER: FLD
... (13 captured-output lines omitted; rerun the source example for the full output)

SOLVER: MAM
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
  Tput
Delay        Class1      2.222      2.222      1          1          2.222
  2.222
Queue1       Class1      7.778      0.99991    11.668      3.5004      0.66661
  0.66661

SOLVER: LDES
N/A: SolverLDES(model, seed=23000, samples=5000) has no C++ engine in this port

```

The rows repeat for each of the 8 solvers the script runs (CTMC, JMT, SSA, FLD, MVA, NC, MAM, LDES), which is the point of the example: the same model read by methods of different cost and different exactness.

2.2.10 cqn_repairmen_multi

The repairman model with several repair stations.

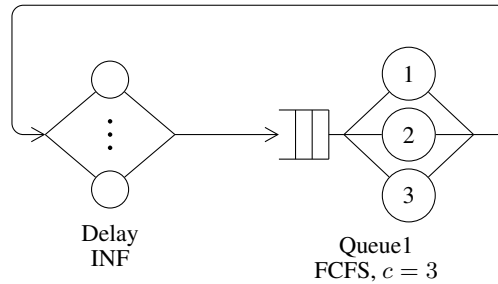


Figure 2.23: Topology of cqn_repairmen_multi: a closed network over 2 queueing stations.

Nodes	2: Queue, Delay
Classes	2: Class1 (closed, $N = 4$), Class2 (closed, $N = 2$)
Scheduling	Delay: INF; Queue1: FCFS ($c = 3$)
Service	Delay – Class1: Exp(1); Class2: Exp(1) Queue1 – Class1: Exp(1); Class2: Exp(10)
Solvers	CTMC, LQNS, MVA, NC

Table 2.23: Features of cqn_repairmen_multi.

The example is built and solved as follows.

```
Net m("model");
Delay delay(m, "Delay");
Queue q1(m, "Queue1", SchedStrategy::FCFS);
q1.set_number_of_servers(3.0);

ClosedClass c1(m, "Class1", 4.0, delay, 0);
ClosedClass c2(m, "Class2", 2.0, delay, 0);

delay.set_service(c1, Exp(1.0));
delay.set_service(c2, Exp(1.0));
q1.set_service(c1, Exp(1.0));
q1.set_service(c2, Exp(10.0));

Routing P;
cyclic(P, c1, {delay, q1});
cyclic(P, c2, {delay, q1});
m.link(P);
const Sn& sn = m.get_struct();

// The reference catches each solver's failure and prints it, so a refusal
// does not cost the rows below it.
guarded("CTMC", [&]() { run_ctmc(sn); }, " Error: ");

// The reference guards these four with `if LQNS.hasQnsolver()`, and so does
// this: LINE ships no qnsolver binary, so on a host without one the rows are
// reported absent rather than left to fail.

... (16 source lines omitted; full implementation: cpp/examples/basic/)

// what carries the plain key and seidmann keeps its qualified name.
guarded("MVA (multiserver=softmin)", [&]() { run_mva(sn, "default", "softmin"); },
    " Error: ", 0, "MVA");
guarded("MVA (multiserver=seidmann)", [&]() { run_mva(sn, "default", "seidmann"); },
    " Error: ");
guarded("NC", [&]() { run_nc(sn); }, " Error: ");
```

Running this edition prints

SOLVER: CTMC						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Delay	Class1	1.9455	1.9455	1	1	1.9455
1.9455						
Delay	Class2	1.6383	1.6383	1	1	1.6383
1.6383						
Queue1	Class1	2.0545	0.64849	1.0561	1.0561	1.9455
1.9455						
Queue1	Class2	0.36172	0.054609	0.22079	0.22079	1.6383
1.6383						
SOLVER: LQNS (qns.conway)						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Delay	Class1	1.8692	1.8692	1	1	1.8692
1.8692						
Delay	Class2	1.613	1.613	1	1	1.613
1.613						
Queue1	Class1	2.1308	0.62307	1.1399	1.1399	1.8692
1.8692						
Queue1	Class2	0.38701	0.053766	0.23994	0.23994	1.613
1.613						
SOLVER: LQNS (qns.reiser)						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
... (31 captured-output lines omitted; rerun the source example for the full output)						
Queue1	Class2	0.7007	0.04331	0.53929	0.53929	1.2993
1.2993						
SOLVER: NC						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Delay	Class1	1.7852	1.7852	1	1	1.7852
1.7852						
Delay	Class2	1.7418	1.7418	1	1	1.7418
1.7418						
Queue1	Class1	2.2148	0.58399	1.2406	1.2406	1.7852
1.7852						
Queue1	Class2	0.2582	0.063355	0.14824	0.14824	1.7418
1.7418						

The rows repeat for each of the 4 solvers the script runs (CTMC, LQNS, MVA, NC), which is the point of the example: the same model read by methods of different cost and different exactness.

2.2.11 cqn_scheduling_dps

Discriminatory processor sharing, with the class weights that PS ignores.

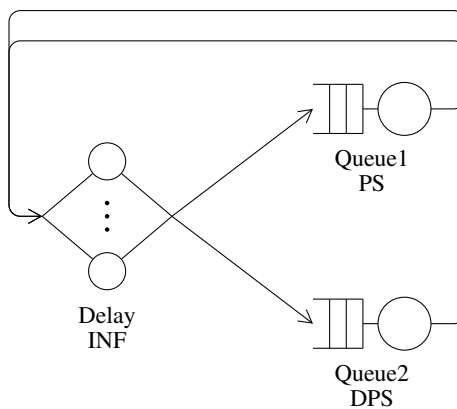


Figure 2.24: Topology of `cqn_scheduling_dps`: a closed network over 3 queueing stations.

The example is built and solved as follows.

Nodes	3: Queue $\times$ 2, Delay
Classes	2: Class1 (closed, $N = 2$), Class2 (closed, $N = 1$)
Scheduling	Delay: INF; Queue1: PS; Queue2: DPS
Service	Delay – Class1: Exp(3); Class2: Exp(0.5) Queue1 – Class1: Exp(0.1); Class2: Exp(1) Queue2 – Class1: Exp(0.1); Class2: Exp(1)
Solvers	CTMC, JMT, FLD, MVA, LDES

Table 2.24: Features of `cqn_scheduling_dps`.

```

Net m("model");
Delay delay(m, "Delay");
Queue q1(m, "Queue1", SchedStrategy::PS);
Queue q2(m, "Queue2", SchedStrategy::DPS);

ClosedClass c1(m, "Class1", 2.0, delay, 0);
ClosedClass c2(m, "Class2", 1.0, delay, 0);

note("Network created with mixed scheduling:");
note("  Delay: Delay node");
note("  Queue1: PS (Processor Sharing)");
note("  Queue2: DPS (Discriminatory Processor Sharing)");
note("\nClass populations: Class1=2, Class2=1");

delay.set_service(c1, D::exp_mean(1.0 / 3.0));
delay.set_service(c2, D::exp_mean(1.0 / 0.5));

q1.set_service(c1, D::exp_mean(1.0 / 0.1));
q1.set_sched_param(c1, 5.0);
q1.set_service(c2, D::exp_mean(1.0 / 1.0));
q1.set_sched_param(c2, 1.0);

q2.set_service(c1, D::exp_mean(1.0 / 0.1));
q2.set_sched_param(c1, 1.0);
q2.set_service(c2, D::exp_mean(1.0 / 1.0));
q2.set_sched_param(c2, 5.0);

... (30 source lines omitted; full implementation: cpp/examples/basic/)

section("JMT");
print_avg(m.get_struct(), jmt_avg(m, sim_opts(23000, 10000)));
guarded("FLD", [&]() { run_fluid(sn); }, refused);
guarded("MVA", [&]() { run_mva(sn); }, refused);
section("LDES");
print_avg(m.get_struct(), ldes_avg(m, sim_opts(23000, 10000)));

```

Running this edition prints

```

Network created with mixed scheduling:
  Delay: Delay node
  Queue1: PS (Processor Sharing)
  Queue2: DPS (Discriminatory Processor Sharing)

Class populations: Class1=2, Class2=1
Service parameters configured:
  Delay: Class1=Exp(3), Class2=Exp(0.5)
  Queue1 (PS): Class1=Exp(0.1), Class2=Exp(1) - weights ignored
  Queue2 (DPS): Class1=Exp(0.1,w=1), Class2=Exp(1,w=5) - Class2 has priority
Routing configured:
  Class1: 30% Queue1, 70% Queue2 (more traffic to DPS queue)
  Class2: 70% Queue1, 30% Queue2 (more traffic to PS queue)
  All queues return 100% to Delay
=== Multi-Solver Analysis ===
Configured 5 solvers for comparison

... (31 captured-output lines omitted; rerun the source example for the full output)

Delay          Class2          0.59007    0.59007         2         2    0.29503
  0.29503
Queue1         Class1          0.42397    0.28152       15.06      4.518    0.028152

```

0.028152						
Queue1	Class2	0.29407	0.20652	1.4239	0.99675	0.20652
0.20652						
Queue2	Class1	1.5447	0.65688	23.517	16.462	0.065688
0.065688						
Queue2	Class2	0.11586	0.08851	1.3089	0.39268	0.08851
0.08851						

SOLVER: LDES

N/A: SolverLDES(model, samples=10000, seed=23000) has no C++ engine in this port

The rows repeat for each of the 5 solvers the script runs (CTMC, JMT, FLD, MVA, LDES), which is the point of the example: the same model read by methods of different cost and different exactness.

2.2.12 cqn_threeclass_hyperl

The three-class version, with multiserver stations.

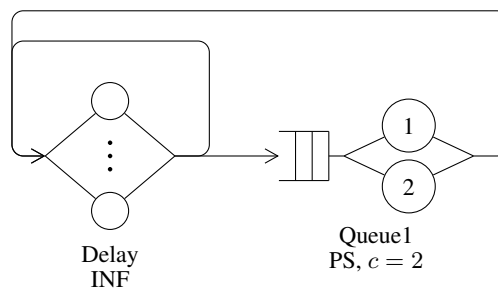


Figure 2.25: Topology of cqn_threeclass_hyperl: a closed network over 2 queueing stations.

Nodes	2: Queue, Delay
Classes	3: Class1 (closed, $N = 2$), Class2 (closed, $N = 0$), Class3 (closed, $N = 1$)
Scheduling	Delay: INF; Queue1: PS ($c = 2$)
Service	Delay – Class1: Erlang(3,2); Class2: HyperExp(0.5,0.5;3,10); Class3: Exp(1) Queue1 – Class1: HyperExp(0.1,0.9;1,10); Class2: Exp(2); Class3: Exp(3)
Solvers	CTMC, JMT, SSA, FLD, MVA, NC, MAM, LDES

Table 2.25: Features of cqn_threeclass_hyperl.

The example is built and solved as follows.

```

Net m("model");
Delay delay(m, "Delay");
Queue q1(m, "Queue1", SchedStrategy::PS);
q1.set_number_of_servers(2.0);

ClosedClass c1(m, "Class1", 2.0, delay, 0);
ClosedClass c2(m, "Class2", 0.0, delay, 0);
ClosedClass c3(m, "Class3", 1.0, delay, 0);

delay.set_service(c1, Erlang(3.0, 2));
delay.set_service(c2, HyperExp(0.5, 3.0, 10.0));
delay.set_service(c3, Exp(1.0));
q1.set_service(c1, HyperExp(0.1, 1.0, 10.0));
q1.set_service(c2, Exp(2.0));
q1.set_service(c3, Exp(3.0));

Routing P;
P.set(c1, c1, delay, delay, 0.3);
P.set(c1, c1, delay, q1, 0.1);
P.set(c1, c1, q1, delay, 0.2);
P.set(c1, c2, delay, delay, 0.6);
P.set(c1, c2, q1, delay, 0.8);

```

```

P.set(c2, c1, q1, delay, 1.0);
P.set(c2, c2, delay, q1, 1.0);
cyclic(P, c3, {delay, q1});
m.link(P);

... (9 source lines omitted; full implementation: cpp/examples/basic/)

guarded("MVA", [&]() { run_mva(sn); }, "Error with MVA: ", 100);
guarded("NC", [&]() { run_nc(sn, "exact"); }, "Error with NC: ", 100);
guarded("MAM", [&]() { run_mam(sn); }, "Error with MAM: ", 100);
guarded("", [&]() { run_auto(sn, 23000); }, "Error with LINE: ", 100);
section("LDES");
print_avg(m.get_struct(), ldes_avg(m, sim_opts(23000, 5000)));

```

Running this edition prints

```

SOLVER: CTMC
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
Tput
Delay        Class1        1.1235     1.1235     0.66667     0.39683     1.6853
1.6853
Delay        Class2        0.2483     0.2483     0.21667     0.087698    1.146
1.146
Delay        Class3        0.74132    0.74132     1           1           0.74132
0.74132
Queue1       Class1        0.033246   0.01601    0.19727     0.011742    0.16853
0.16853
Queue1       Class2        0.59492    0.2865     0.51913     0.21012     1.146
1.146
Queue1       Class3        0.25868    0.12355    0.34894     0.34894     0.74132
0.74132

SOLVER: JMT
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
Tput
Delay        Class1        1.1301     1.1301     0.66008     0.3929      1.7071
1.7092
Delay        Class2        0.24356    0.24356    0.20999     0.084996    1.1618
1.1618
Delay        Class3        0.73801    0.73801     1.0024      1.0024      0.74033
0.74028
Queue1       Class1        0.038038   0.014299   0.20157     0.011999    0.16551
0.16612
Queue1       Class2        0.60099    0.30343    0.51428     0.20816     1.1618
1.1622

... (50 captured-output lines omitted; rerun the source example for the full output)

Delay        Class2        0.2483     0.2483     0.21667     0.087698    1.146
1.146
Delay        Class3        0.74132    0.74132     1           1           0.74132
0.74132
Queue1       Class1        0.033246   0.01601    0.19727     0.011742    0.16853
0.16853
Queue1       Class2        0.59492    0.2865     0.51913     0.21012     1.146
1.146
Queue1       Class3        0.25868    0.12355    0.34894     0.34894     0.74132
0.74132

SOLVER: LDES
N/A: SolverLDES(model, seed=23000, samples=5000) has no C++ engine in this port

```

The rows repeat for each of the 9 solvers the script runs (CTMC, JMT, SSA, FLD, MVA, NC, MAM, AUTO, LDES), which is the point of the example: the same model read by methods of different cost and different exactness.

2.2.13 cqn_twoclass_erl

Two classes with Erlang service over PS and INF stations, exercising the RAND, round-robin and weighted round-robin routing policies.

The example is built and solved as follows.

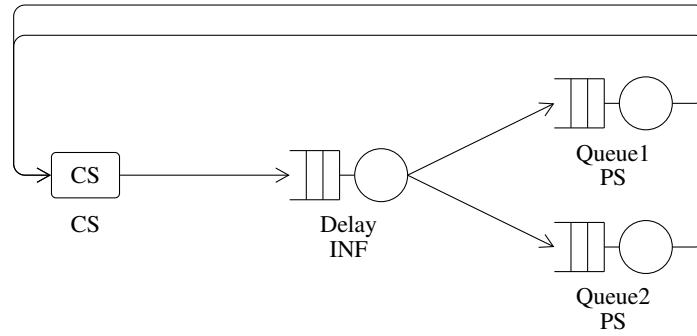


Figure 2.26: Topology of `cqn_twoclass_erl`: a closed network over 3 queueing stations with a class-switch node.

Nodes	4: Queue $\times$ 3, ClassSwitch
Classes	2: Class1 (closed, $N = 15$), Class2 (closed, $N = 5$)
Scheduling	Queue1: PS; Queue2: PS; Delay: INF
Service	Queue1 – Class1: Exp(0.6667); Class2: Erlang(1.333,2) Queue2 – Class1: Erlang(1.333,2); Class2: Exp(0.6667) Delay – Class1: Exp(1); Class2: Exp(1)
Features	class switching on 3 links
Solvers	JMT, LDES

Table 2.26: Features of `cqn_twoclass_erl`.

```

Net m("model");
Queue q1(m, "Queue1", SchedStrategy::PS);
Queue q2(m, "Queue2", SchedStrategy::PS);
Queue delay(m, "Delay", SchedStrategy::INF);
m.raw_struct().stations[m.station_index(delay) - 1].nservers =
    std::numeric_limits<double>::infinity();

ClosedClass c1(m, "Class1", 15.0, delay, 0);
ClosedClass c2(m, "Class2", 5.0, delay, 0);

Matrix<double> C(2, 2, 0.0);
C(0, 1) = 1.0;
C(1, 0) = 1.0;
const std::size_t cs = m.add_class_switch("CS", C);

q1.set_service(c1, D::exp_mean(1.5));
q1.set_service(c2, lang::erlang_fit_mean_order<double>(1.5, 2));
q2.set_service(c1, lang::erlang_fit_mean_order<double>(1.5, 2));
q2.set_service(c2, D::exp_mean(1.5));
delay.set_service(c1, D::exp_mean(1.0));
delay.set_service(c2, D::exp_mean(1.0));

Routing P;
for (std::size_t r = 0; r < 2; ++r) {
    const std::size_t cr = r == 0 ? c1 : c2;
    P.set(cr, cr, q1, cs, 1.0);

    ... (20 source lines omitted; full implementation: cpp/examples/basic/)

m.get_struct();

section("JMT");
print_avg(m.get_struct(), jmt_avg(m, sim_opts(23000)));
section("LDES");
print_avg(m.get_struct(), ldes_avg(m, sim_opts(23000)));

```

Running this edition prints

SOLVER: JMT

Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Queue1	Class1	0.90422	0.4215	3.0843	0.77107	0.28506
0.28397						
Queue1	Class2	0.5383	0.28473	2.8414	0.47356	0.19131
0.18984						
Queue2	Class1	7.4591	0.44033	26.209	6.5522	0.287
0.28698						
Queue2	Class2	9.92	0.55965	26.615	8.8718	0.3808
0.37815						
Delay	Class1	0.57561	0.57561	1.0061	0.50304	0.56853
0.56405						
Delay	Class2	0.54055	0.54055	0.97771	0.48885	0.56807
0.56532						
SOLVER: LDES						
N/A: SolverLDES(model, seed=23000) has no C++ engine in this port						

The rows repeat for each of the 2 solvers the script runs (JMT, LDES), which is the point of the example: the same model read by methods of different cost and different exactness.

2.2.14 cqn_twoclass_hyper1

Two classes with hyperexponential and Erlang service on PS stations.

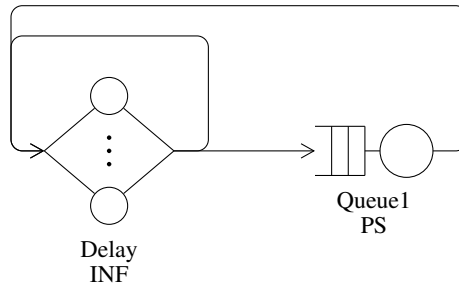


Figure 2.27: Topology of cqn_twoclass_hyper1: a closed network over 2 queueing stations.

Nodes	2: Queue, Delay
Classes	2: Class1 (closed, $N = 2$), Class2 (closed, $N = 2$)
Scheduling	Delay: INF; Queue1: PS
Service	Delay – Class1: Erlang(3,2); Class2: HyperExp(0.5,0.5;3,10) Queue1 – Class1: HyperExp(0.1,0.9;1,10); Class2: Exp(1)
Solvers	CTMC, JMT, SSA, FLD, MVA, NC, MAM, LDES

Table 2.27: Features of cqn_twoclass_hyper1.

The example is built and solved as follows.

```

Net m("model");
Delay delay(m, "Delay");
Queue q1(m, "Queue1", SchedStrategy::PS);

ClosedClass c1(m, "Class1", 2.0, delay, 0);
ClosedClass c2(m, "Class2", 2.0, delay, 0);

delay.set_service(c1, Erlang(3.0, 2));
delay.set_service(c2, HyperExp(0.5, 3.0, 10.0));
q1.set_service(c1, HyperExp(0.1, 1.0, 10.0));
q1.set_service(c2, Exp(1.0));

Routing P;
P.set(c1, c1, delay, delay, 0.3);
P.set(c1, c1, delay, q1, 0.1);
P.set(c1, c1, q1, delay, 0.2);

```

```

P.set(c1, c2, delay, delay, 0.6);
P.set(c1, c2, q1, delay, 0.8);
P.set(c2, c1, q1, delay, 1.0);
P.set(c2, c2, delay, q1, 1.0);
m.link(P);
const Sn& sn = m.get_struct();

section("CTMC");
run_ctmc(sn);
section("JMT");

... (7 source lines omitted; full implementation: cpp/examples/basic/)

section("NC");
run_nc(sn, "exact");
section("MAM");
run_mam(sn);
section("LDES");
print_avg(m.get_struct(), ldes_avg(m, sim_opts(23000, 5000)));

```

Running this edition prints

```

SOLVER: CTMC
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
  Tput
Delay        Class1        0.93078    0.93078    0.66667    0.39683    1.3962
  1.3962
Delay        Class2        0.2057     0.2057     0.21667    0.087698   0.94939
  0.94939
Queue1       Class1        0.077835   0.026527   0.55749    0.033184   0.13962
  0.13962
Queue1       Class2        2.7857     0.94939    2.9342     1.1876     0.94939
  0.94939

SOLVER: JMT
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
  Tput
Delay        Class1        0.93008    0.93008    0.66993    0.39877    1.4084
  1.4078
Delay        Class2        0.20202    0.20202    0.21432    0.086747   0.9506
  0.95064
Queue1       Class1        0.074672   0.018632   0.54991    0.032733   0.13725
  0.13724
Queue1       Class2        2.8076     0.95301    2.9302     1.186      0.95064
  0.95063

SOLVER: SSA
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
  Tput
... (27 captured-output lines omitted; rerun the source example for the full output)

Station      JobClass      QLen      Util      RespT      ResidT      ArvR
  Tput
Delay        Class1        0.93078    0.93078    0.66667    0.39683    1.3962
  1.3962
Delay        Class2        0.2057     0.2057     0.21667    0.087698   0.94939
  0.94939
Queue1       Class1        0.077835   0.026527   0.55749    0.033184   0.13962
  0.13962
Queue1       Class2        2.7857     0.94939    2.9342     1.1876     0.94939
  0.94939

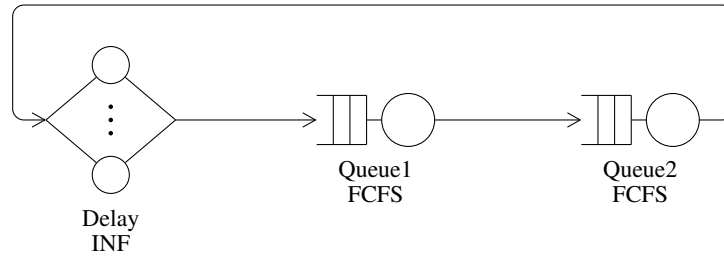
SOLVER: LDES
N/A: SolverLDES(model, seed=23000, samples=5000) has no C++ engine in this port

```

The rows repeat for each of the 8 solvers the script runs (CTMC, JMT, SSA, FLD, MVA, NC, MAM, LDES), which is the point of the example: the same model read by methods of different cost and different exactness.

2.2.15 cqn_twoqueues_multi

Two FCFS queues and a Delay with several closed classes.

Figure 2.28: Topology of `cqn_twoqueues_multi`: a closed network over 3 queueing stations.

Nodes	3: Queue $\times$ 2, Delay
Classes	2: Class1 (closed, $N = 10$), Class2 (closed, $N = 10$)
Scheduling	Delay: INF; Queue1: FCFS; Queue2: FCFS
Service	Delay – Class1: Exp(1); Class2: Exp(1) Queue1 – Class1: Exp(0.6667); Class2: Exp(0.6667) Queue2 – Class1: Exp(0.3333); Class2: Exp(0.3333)
Solvers	MVA

Table 2.28: Features of `cqn_twoqueues_multi`.

The example is built and solved as follows.

```

Net m("model");
Delay delay(m, "Delay");
Queue q1(m, "Queue1", SchedStrategy::FCFS);
Queue q2(m, "Queue2", SchedStrategy::FCFS);

ClosedClass c1(m, "Class1", 10.0, delay, 0);
ClosedClass c2(m, "Class2", 10.0, delay, 0);

const std::size_t st[3] = {delay, q1, q2};
const double svc[3] = {1.0, 1.5, 3.0};
for (std::size_t i = 0; i < 3; ++i) {
    m.set_service(st[i], c1, D::exp_mean(svc[i]));
    m.set_service(st[i], c2, D::exp_mean(svc[i]));
}

Routing P;
cyclic(P, c1, {delay, q1, q2});
cyclic(P, c2, {delay, q1, q2});
m.link(P);
const Sn& sn = m.get_struct();

section("MVA");
run_mva(sn);
  
```

Running this edition prints

SOLVER: MVA	Station	JobClass	QLen	Util	RespT	ResidT	ArrR
	Tput						
	Delay	Class1	0.16667	0.16667	1	1	0.16667
	0.16667						
	Delay	Class2	0.16667	0.16667	1	1	0.16667
	0.16667						
	Queue1	Class1	0.49999	0.25	3	3	0.16667
	0.16667						
	Queue1	Class2	0.49999	0.25	3	3	0.16667
	0.16667						
	Queue2	Class1	9.3333	0.5	56	56	0.16667
	0.16667						
	Queue2	Class2	9.3333	0.5	56	56	0.16667
	0.16667						

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by MVA.

2.3 Mixed networks

A mixed network carries open and closed classes at the same stations. The open classes contribute a fixed arrival rate, the closed ones a fixed population, and the two compete for the same servers, so neither can be solved in isolation.

The scripts pair a single-server and a multiserver station under FCFS and under PS. They are deliberately awkward instances: the load-dependent rates they induce are sparse, and they are the models on which the approximate solvers disagree first.

2.3.1 mqn_basic

Open and closed classes at the same stations, the smallest mixed model.

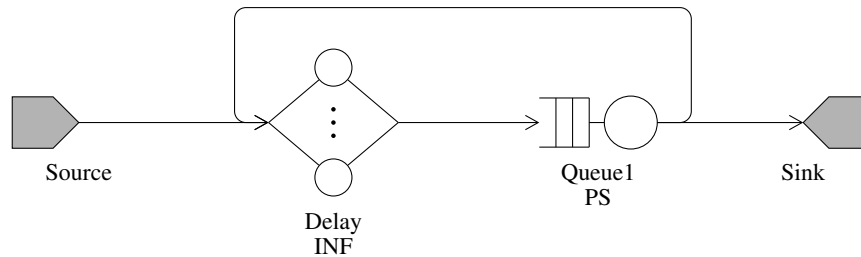


Figure 2.29: Topology of mqn_basic: an open network over 2 queueing stations.

Nodes	4: Source, Queue, Delay, Sink
Classes	2: ClosedClass (closed, $N = 2$), OpenClass (open)
Scheduling	Delay: INF; Queue1: PS
Arrivals	Source – ClosedClass: Disabled; OpenClass: Exp(0.1)
Service	Delay – ClosedClass: Erlang(3,2); OpenClass: HyperExp(0.5,0.5;3,10) Queue1 – ClosedClass: HyperExp(0.1,0.9;1,10); OpenClass: Exp(1)
Features	class switching on 1 link
Solvers	CTMC, JMT, SSA, MVA, LDES

Table 2.29: Features of mqn_basic.

The example is built and solved as follows.

```
Net model("model");
Delay delay(model, "Delay");
Queue queue1(model, "Queue1", SchedStrategy::PS);
Source source(model, "Source");
Sink sink(model, "Sink");

ClosedClass closed(model, "ClosedClass", 2, delay, 0);
OpenClass open(model, "OpenClass", 0);

delay.set_service(closed, Erlang(3.0, 2));
delay.set_service(open, HyperExp(0.5, 3.0, 10.0));
queue1.set_service(closed, HyperExp(0.1, 1.0, 10.0));
queue1.set_service(open, Exp(1.0));
source.set_arrival(open, Exp(0.1));

Routing P;
P.set(closed, closed, delay, queue1, 1.0);
P.set(closed, closed, queue1, delay, 1.0);
P.set(open, open, source, delay, 1.0);
P.set(open, open, delay, queue1, 1.0);
P.set(open, open, queue1, sink, 1.0);
model.link(P);
```

```
SolverOpts ctmc;
ctmc.cutoff = 3.0;
ctmc.seed = 23000;

... (10 source lines omitted; full implementation: cpp/examples/basic/)

section("MVA");
print_avg(solve_avg("MVA", model));

section("LDES");
print_avg(solve_avg("LDES", model, sim_opts(23000)));
```

Running this edition prints

```
SOLVER: CTMC
SolverCTMC method=default
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
  Tput
Delay        ClosedClass    1.4362     1.4362     0.666667    0.666667    2.1543
  2.1543
Delay        OpenClass      0.0216066  0.0216066  0.216667    0.216667    0.0997229
  0.0997229
Queue1       ClosedClass    0.563803   0.409316   0.261711    0.261711    2.1543
  2.1543
Queue1       OpenClass      0.172234   0.0997229   1.72712     1.72712     0.0997229
  0.0997229
Source       OpenClass      0          0          0           0           0
  0.0997229

SOLVER: JMT
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
  Tput
Delay        ClosedClass    1.4528     1.4528     0.66644     0.66644     2.168
  2.1536
Delay        OpenClass      0.021961   0.021961   0.21581     0.21581     0.10001
  0.10001
Queue1       ClosedClass    0.54717    0.40216    0.25983     0.25983     2.1536
  2.1602
Queue1       OpenClass      0.16391    0.095781   1.711       1.711       0.10001
  0.10001
Source       OpenClass      0          0          0           0           0
  0.10001

... (13 captured-output lines omitted; rerun the source example for the full output)

Delay        ClosedClass    1.43593    1.43593    0.666667    0.666667    2.15389
  2.15389
Delay        OpenClass      0.0216667  0.0216667  0.216667    0.216667    0.1
  0.1
Queue1       ClosedClass    0.564071   0.40924    0.261885    0.261885    2.15389
  2.15389
Queue1       OpenClass      0.173786   0.1        1.73786     1.73786     0.1
  0.1
Source       OpenClass      0          0          0           0           0
  0.1

SOLVER: LDES
N/A: SolverLDES is the SSJ simulation engine of the JAR and has no C++ port
```

The rows repeat for each of the 5 solvers the script runs (CTMC, JMT, SSA, MVA, LDES), which is the point of the example: the same model read by methods of different cost and different exactness.

2.3.2 mqn_multichain_cs

A mixed network whose classes are grouped into several chains by class switching. The example is built and solved as follows.

```
Net model("mqn_multichain");
Delay think(model, "ThinkingTime");
Queue web(model, "WebServer", SchedStrategy::FCFS);
Queue app(model, "AppServer", SchedStrategy::PS);
Queue data(model, "DataServer", SchedStrategy::FCFS);
```

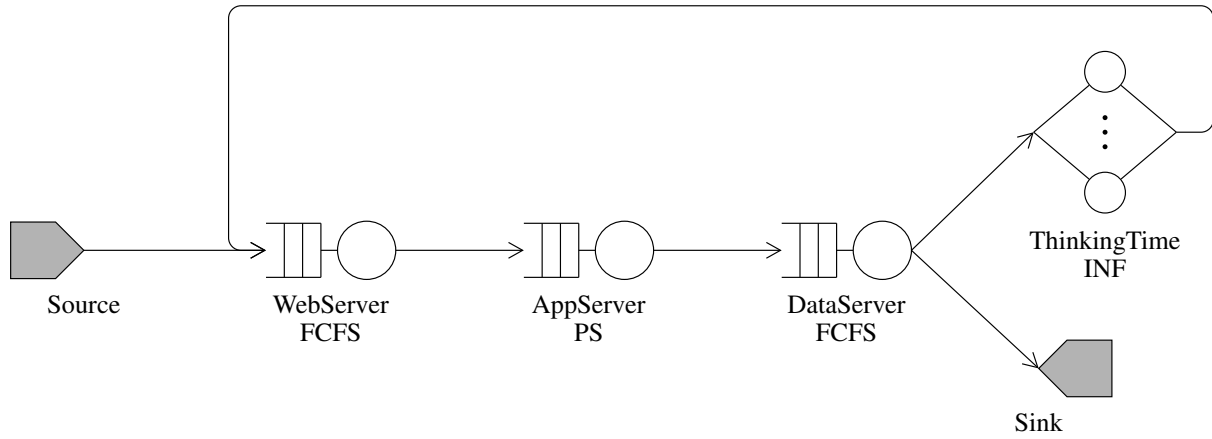


Figure 2.30: Topology of `mqn_multichain_cs`: an open network over 4 queueing stations.

Nodes	6: Source, Queue $\times$ 3, Delay, Sink
Classes	5: InteractiveA (closed, $N = 3$), InteractiveB (closed, $N = 2$), BatchA (open), BatchB (open), ExternalLoad (open)
Scheduling	ThinkingTime: INF; WebServer: FCFS; AppServer: PS; DataServer: FCFS
Arrivals	Source – InteractiveA: Disabled; InteractiveB: Disabled; BatchA: Exp(0.1); BatchB: Exp(0.05); ExternalLoad: Exp(0.05)
Service	ThinkingTime – InteractiveA: Exp(0.6667); InteractiveB: Exp(0.5); BatchA: Disabled; BatchB: Disabled; ExternalLoad: Disabled WebServer – InteractiveA: Exp(2.5); InteractiveB: Exp(2); BatchA: Exp(3.333); BatchB: Exp(2.857); ExternalLoad: Exp(5) AppServer – InteractiveA: Exp(1.25); InteractiveB: Exp(1); BatchA: Exp(1.667); BatchB: Exp(1.429); ExternalLoad: Exp(2) DataServer – InteractiveA: Exp(2); InteractiveB: Exp(1.667); BatchA: Exp(0.8333); BatchB: Exp(0.6667); ExternalLoad: Exp(1.25)
Features	drop rule {'BatchA': 'waitingQueue', 'BatchB': 'waitingQueue', 'ExternalLoad': 'waitingQueue'} at ThinkingTime; class switching on 1 link
Solvers	MVA, SSA

Table 2.30: Features of `mqn_multichain_cs`.

```

Source source(model, "Source");
Sink sink(model, "Sink");

ClosedClass ia(model, "InteractiveA", 3, think);
ClosedClass ib(model, "InteractiveB", 2, think);
OpenClass ba(model, "BatchA", 0);
OpenClass bb(model, "BatchB", 0);
OpenClass ext(model, "ExternalLoad", 0);

think.set_service(ia, D::exp_mean(1.5));
think.set_service(ib, D::exp_mean(2.0));

web.set_service(ia, D::exp_mean(0.4));
web.set_service(ib, D::exp_mean(0.5));
web.set_service(ba, D::exp_mean(0.3));
web.set_service(bb, D::exp_mean(0.35));
web.set_service(ext, D::exp_mean(0.2));

app.set_service(ia, D::exp_mean(0.8));
app.set_service(ib, D::exp_mean(1.0));
app.set_service(ba, D::exp_mean(0.6));

... (57 source lines omitted; full implementation: cpp/examples/basic/)

SolverOpts ssa;
ssa.seed = 23000;
// The reference also passes cutoff=5 to SSA; neither codebase's simulator
// reads it (Python carries it into HandlerOptions and no engine consumes it).
section("SSA");
print_avg_chain(sn, solve_avg("SSA", model, ssa));

```

Running this edition prints

```

Mixed QN with 3 chains, 5 classes, and class switching:
- Chain 1 (Closed): InteractiveA and InteractiveB (switch at WebServer)
- Chain 2 (Open): BatchA and BatchB (switch at WebServer)
- Chain 3 (Open): ExternalLoad

SOLVER: MVA
SolverMVA method=egflin

```

Station	Chain	QLen	Util	RespT	ResidT	ArvR
ThinkingTime 0.716883	Chain1	1.29039	1.29039	1.8	1.8	0.716883
ThinkingTime 0	Chain2	0	0	0	0	0
ThinkingTime 0	Chain3	0	0	0	0	0
WebServer 0.716883	Chain1	0.478625	0.329766	0.667647	0.667647	0.716883
WebServer 0.15	Chain2	0.0852521	0.0475	0.568348	0.568348	0.15
WebServer 0.05	Chain3	0.022584	0.01	0.451681	0.451681	0.05
AppServer 0.716883	Chain1	1.97634	0.659533	2.75684	2.75684	0.716883
AppServer 0.15	Chain2	0.330704	0.0975	2.20469	2.20469	0.15
AppServer 0.141496	Chain2	0.299124	0.0928382	2.10279	2.10279	0.167002
AppServer 0.0416083	Chain3	0.0711661	0.0208042	1.71038	1.71038	0.0533922
DataServer 0.775042	Chain1	1.27481	0.438689	1.64089	1.64089	0.752371
DataServer 0.157442	Chain2	0.449629	0.215016	2.8223	2.8223	0.141496
DataServer 0.0413179	Chain3	0.0966255	0.0330543	2.33859	2.33859	0.0416083
Source 0	Chain1	0	0	0	0	0
Source 0.15	Chain2	0	0	0	0	0

```

... (18 captured-output lines omitted; rerun the source example for the full output)

```

Source	Chain3	0	0	0	0	0
0.05						

The rows repeat for each of the 2 solvers the script runs (MVA, SSA), which is the point of the example: the same model read by methods of different cost and different exactness.

2.3.3 mqn_multiserver_fcfs

The multiserver FCFS version of the same model.

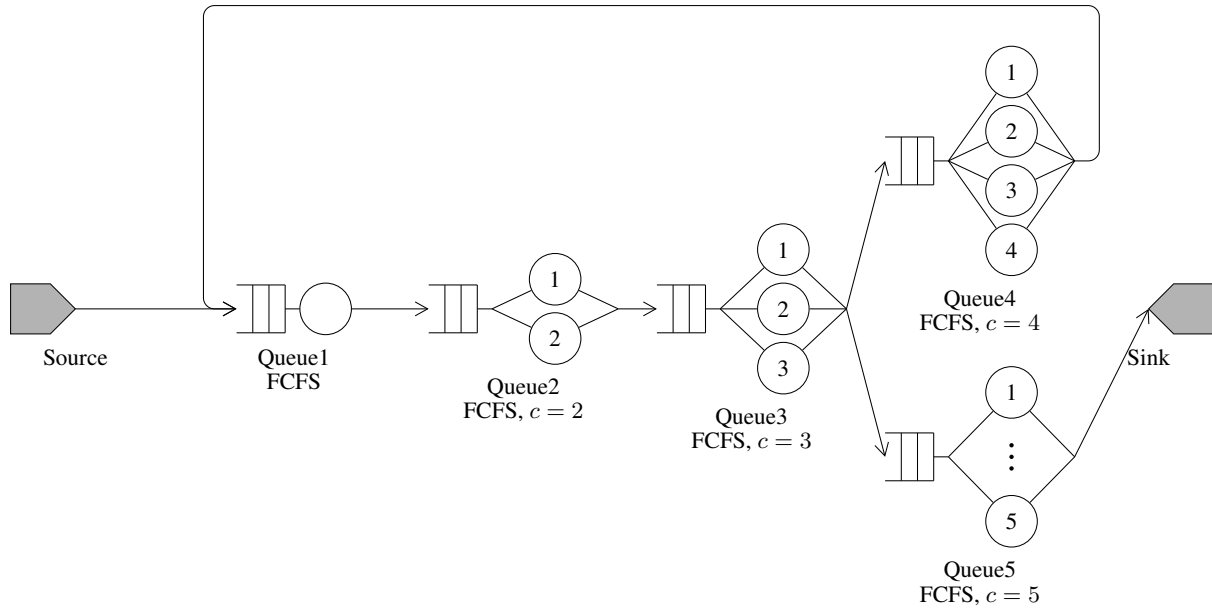


Figure 2.31: Topology of mqn_multiserver_fcfs: an open network over 5 queueing stations.

Nodes	7: Source, Queue $\times$ 5, Sink
Classes	2: ClosedClass (closed, $N = 3$), OpenClass (open)
Scheduling	Queue1: FCFS; Queue2: FCFS ($c = 2$); Queue3: FCFS ($c = 3$); Queue4: FCFS ($c = 4$); Queue5: FCFS ($c = 5$)
Arrivals	Source – ClosedClass: Disabled; OpenClass: Exp(0.3)
Service	Queue1 – ClosedClass: Exp(1); OpenClass: Exp(1) Queue2 – ClosedClass: Exp(2); OpenClass: Exp(1.414) Queue3 – ClosedClass: Exp(3); OpenClass: Exp(1.732) Queue4 – ClosedClass: Exp(4); OpenClass: Exp(2) Queue5 – ClosedClass: Exp(5); OpenClass: Exp(2.236)
Features	class switching on 3 links
Solvers	CTMC, JMT, SSA, MVA, LDES

Table 2.31: Features of mqn_multiserver_fcfs.

The example is built and solved as follows.

```
Net model = multiserver_model(SchedStrategy::FCFS);

note(
    "This example shows the execution of the solver on a 2-class mixed model with 5 "
    "multi-server FCFS nodes.");

SolverOpts ctmc;
ctmc.cutoff = 3.0;
ctmc.seed = 23000;
section("CTMC");
print_avg(solve_avg("CTMC", model, ctmc));
```



```

section("JMT");
print_avg(model.get_struct(), jmt_avg(model, sim_opts(23000, 100000)));

SolverOpts ssa;
ssa.seed = 23000;
section("SSA");
print_avg(solve_avg("SSA", model, ssa));

section("MVA");
print_avg(solve_avg("MVA", model));

section("LDES");
print_avg(solve_avg("LDES", model, sim_opts(23000, 100000)));

```

Running this edition prints

This example shows the execution of the solver on a 2-class mixed model with 5 multi-server FCFS nodes.

```

SOLVER: CTMC
SolverCTMC method=default
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
Tput
Queue1      ClosedClass      2.19691    0.720003    3.05125    3.05125    0.720003
0.720003
Queue1      OpenClass      0.953383    0.247626    3.8501     3.8501     0.247626
0.247626
Queue2      ClosedClass      0.38259    0.180001    0.531373    0.531373    0.720003
0.720003
Queue2      OpenClass      0.184099    0.0875489    0.743458    0.743458    0.247626
0.247626
Queue3      ClosedClass      0.240498    0.0800004    0.334024    0.334024    0.720003
0.720003
Queue3      OpenClass      0.14315     0.0476556    0.578089    0.578089    0.247626
0.247626
Queue4      ClosedClass      0.180001    0.0450002    0.25        0.25        0.720003
0.720003
Queue5      OpenClass      0.110742    0.0221483    0.447214    0.447214    0.247626
0.247626
Source      OpenClass      0          0          0          0          0
0.247626

SOLVER: JMT
... (31 captured-output lines omitted; rerun the source example for the full output)

Queue3      ClosedClass      0.197353    0.0652807    0.335905    0.335905    0.587527
0.587527
Queue3      OpenClass      0.174215    0.057735     0.580717    0.580717     0.3
0.3
Queue4      ClosedClass      0.14697     0.0367204    0.25015     0.25015     0.587527
0.587527
Queue5      OpenClass      0.134261    0.0268328    0.447536    0.447536     0.3
0.3
Source      OpenClass      0          0          0          0          0
0.3

SOLVER: LDES
N/A: SolverLDES is the SSJ simulation engine of the JAR and has no C++ port

```

The rows repeat for each of the 5 solvers the script runs (CTMC, JMT, SSA, MVA, LDES), which is the point of the example: the same model read by methods of different cost and different exactness.

2.3.4 mqn_multiserver_ps

The multiserver PS version.

The example is built and solved as follows.

```

Net model = multiserver_model(SchedStrategy::PS);

SolverOpts ctmc;
ctmc.cutoff = 3.0;
ctmc.seed = 23000;
section("CTMC");

```

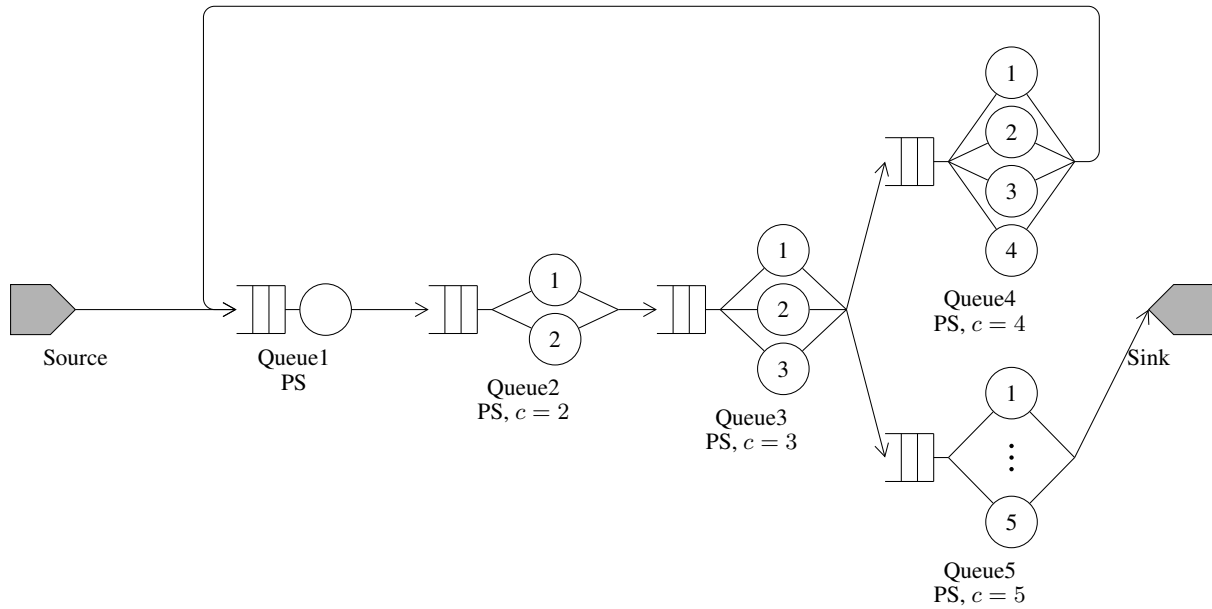


Figure 2.32: Topology of `mqn_multiserver_ps`: an open network over 5 queueing stations.

Nodes	7: Source, Queue $\times$ 5, Sink
Classes	2: ClosedClass (closed, $N = 3$), OpenClass (open)
Scheduling	Queue1: PS; Queue2: PS ($c = 2$); Queue3: PS ($c = 3$); Queue4: PS ($c = 4$); Queue5: PS ($c = 5$)
Arrivals	Source – ClosedClass: Disabled; OpenClass: Exp(0.3)
Service	Queue1 – ClosedClass: Exp(1); OpenClass: Exp(1) Queue2 – ClosedClass: Exp(2); OpenClass: Exp(1.414) Queue3 – ClosedClass: Exp(3); OpenClass: Exp(1.732) Queue4 – ClosedClass: Exp(4); OpenClass: Exp(2) Queue5 – ClosedClass: Exp(5); OpenClass: Exp(2.236)
Features	class switching on 3 links
Solvers	CTMC, JMT, MVA, LDES

Table 2.32: Features of `mqn_multiserver_ps`.

```

print_avg(solve_avg("CTMC", model, ctmc));

section("JMT");
print_avg(model.get_struct(), jmt_avg(model, sim_opts(23000, 100000)));

SolverOpts mva;
mva.method = "exact";
section("MVA");
print_avg(solve_avg("MVA", model, mva));

section("LDES");
print_avg(solve_avg("LDES", model, sim_opts(23000, 100000)));

```

Running this edition prints

```

SOLVER: CTMC
SolverCTMC method=default
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
  Tput
Queue1      ClosedClass      2.19929    0.720637    3.05187    3.05187    0.720637
0.720637
Queue1      OpenClass      0.953403    0.24747    3.85259    3.85259    0.24747
0.24747
Queue2      ClosedClass      0.379979    0.180159    0.527282    0.527282    0.720637
0.720637
Queue2      OpenClass      0.186486    0.087494    0.75357    0.75357    0.24747
0.24747
Queue3      ClosedClass      0.240574    0.0800708    0.333835    0.333835    0.720637
0.720637
Queue3      OpenClass      0.143158    0.0476257    0.578485    0.578485    0.24747
0.24747
Queue4      ClosedClass      0.180159    0.0450398    0.25        0.25        0.720637
0.720637
Queue5      OpenClass      0.110672    0.0221344    0.447214    0.447214    0.24747
0.24747
Source      OpenClass      0          0          0          0          0
0.24747

SOLVER: JMT
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
  Tput
Queue1      ClosedClass      2.2615     0.67284    3.3567     3.3567     0.6722
0.67218

... (16 captured-output lines omitted; rerun the source example for the full output)

Queue3      ClosedClass      0.224725    0.0747669    0.333963    0.333963    0.672902
0.672902
Queue3      OpenClass      0.173682    0.057735    0.578939    0.578939    0.3
0.3
Queue4      ClosedClass      0.168226    0.0420564    0.25        0.25        0.672902
0.672902
Queue5      OpenClass      0.134164    0.0268328    0.447214    0.447214    0.3
0.3
Source      OpenClass      0          0          0          0          0
0.3

SOLVER: LDES
N/A: SolverLDES is the SSJ simulation engine of the JAR and has no C++ port

```

The rows repeat for each of the 4 solvers the script runs (CTMC, JMT, MVA, LDES), which is the point of the example: the same model read by methods of different cost and different exactness.

2.3.5 mqn_singleserver_fcfs

A single-server FCFS station under a mixed workload, a sparse load-dependent multiclass instance that stresses the solvers.

The example is built and solved as follows.

```

Net model = singleserver_model(SchedStrategy::FCFS, lang::aph_fit_mean_scv<double>(3.0, 64.0)
);
section("JMT");

```

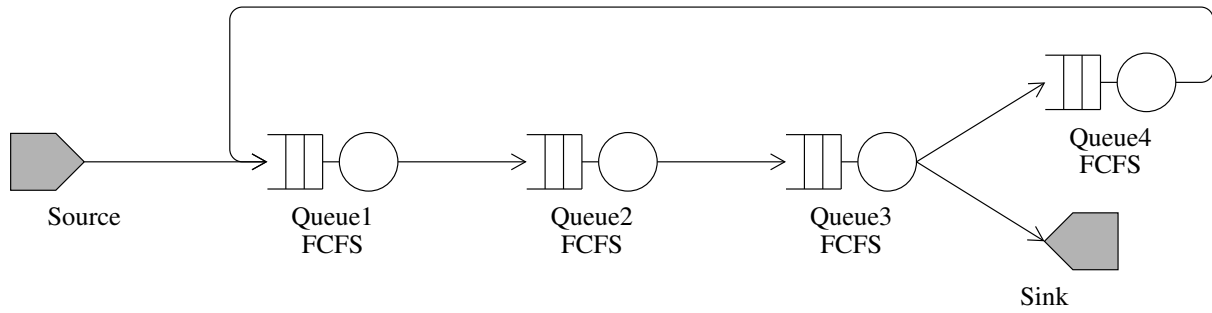


Figure 2.33: Topology of mqn_singleserver_fcfs: an open network over 4 queueing stations.

Nodes	6: Source, Queue $\times$ 4, Sink
Classes	2: ClosedClass (closed, $N = 100$), OpenClass (open)
Scheduling	Queue1: FCFS; Queue2: FCFS; Queue3: FCFS; Queue4: FCFS
Arrivals	Source – ClosedClass: Disabled; OpenClass: APH
Service	Queue1 – ClosedClass: Exp(1); OpenClass: Exp(1) Queue2 – ClosedClass: Exp(2); OpenClass: Exp(1.414) Queue3 – ClosedClass: Exp(3); OpenClass: Exp(1.732) Queue4 – ClosedClass: Exp(4); OpenClass: Exp(2)
Features	class switching on 2 links
Solvers	JMT, MVA, NC, LDES

Table 2.33: Features of mqn_singleserver_fcfs.

```

print_avg(model.get_struct(), jmt_avg(model, sim_opts(23000, 20000)));

// The reference leaves its SolverFLD block commented out: the ODEs are too
// stiff with APH arrivals in a mixed network.

SolverOpts mva;
mva.method = "lin";
section("MVA");
print_avg(solve_avg("MVA", model, mva));

SolverOpts nc;
nc.cutoff = 3.0;
nc.seed = 23000;
section("NC");
print_avg(solve_avg("NC", model, nc));

section("LDES");
print_avg(solve_avg("LDES", model, sim_opts(23000, 20000)));

```

Running this edition prints

SOLVER: JMT	Station	JobClass	QLen	Util	RespT	ResidT	ArvR
	Tput						
	Queue1	ClosedClass	98.44	0.68116	143.75	143.75	0.69274
	0.68865						
	Queue1	OpenClass	56.624	0.29771	187.54	187.54	0.30425
	0.30471						
	Queue2	ClosedClass	0.87067	0.3381	1.2547	1.2547	0.68865
	0.68863						
	Queue2	OpenClass	0.53673	0.19108	1.7508	1.7508	0.30471
	0.30472						
	Queue3	ClosedClass	0.43728	0.20633	0.62634	0.62634	0.68863
	0.69273						
	Queue3	OpenClass	0.29725	0.18077	0.96959	0.96959	0.30472
	0.29542						
	Queue4	ClosedClass	0.21893	0.1596	0.31419	0.31419	0.69273
	0.69274						
	Source	OpenClass	0	0	0	0	0
	0.30425						
SOLVER: MVA							

```

SolverMVA method=lin
Station      JobClass      QLen      Util      RespT      ResidT      ArrvR
Tput
Queue1       ClosedClass    98.5326    0.664353    148.314    148.314    0.664353
0.664353
Queue1       OpenClass      49.7663    0.333333    149.299    149.299    0.333333
0.333333
... (13 captured-output lines omitted; rerun the source example for the full output)
Queue2       OpenClass      0.548281    0.235195    1.64484    1.64484    0.333333
0.333333
Queue3       ClosedClass    0.38844    0.222423    0.582257    0.582257    0.667127
0.667127
Queue3       OpenClass      0.330783    0.192403    0.992348    0.992348    0.333333
0.333333
Queue4       ClosedClass    0.207511    0.166782    0.311052    0.311052    0.667127
0.667127
Source       OpenClass      0          0          0          0          0
0.333333

SOLVER: LDES
N/A: SolverLDES is the SSJ simulation engine of the JAR and has no C++ port

```

The rows repeat for each of the 4 solvers the script runs (JMT, MVA, NC, LDES), which is the point of the example: the same model read by methods of different cost and different exactness.

2.3.6 mqn_singleserver_ps

The processor sharing counterpart.

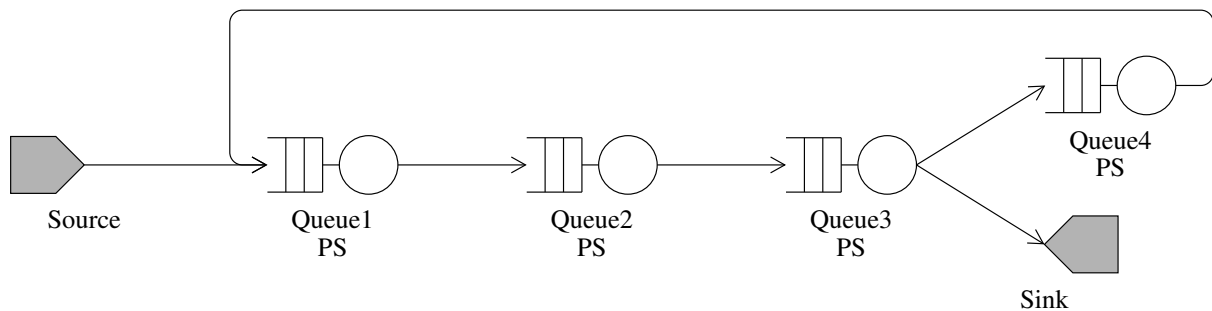


Figure 2.34: Topology of mqn_singleserver_ps: an open network over 4 queueing stations.

Nodes	6: Source, Queue $\times$ 4, Sink
Classes	2: ClosedClass (closed, $N = 100$), OpenClass (open)
Scheduling	Queue1: PS; Queue2: PS; Queue3: PS; Queue4: PS
Arrivals	Source – ClosedClass: Disabled; OpenClass: Exp(0.3)
Service	Queue1 – ClosedClass: Exp(1); OpenClass: Exp(1) Queue2 – ClosedClass: Exp(2); OpenClass: Exp(1.414) Queue3 – ClosedClass: Exp(3); OpenClass: Exp(1.732) Queue4 – ClosedClass: Exp(4); OpenClass: Exp(2)
Features	class switching on 2 links
Solvers	JMT, SSA, FLD, MVA, NC, LDES

Table 2.34: Features of mqn_singleserver_ps.

The example is built and solved as follows.

```

Net model = singleserver_model(SchedStrategy::PS, Exp(0.3));

section("JMT");
print_avg(model.get_struct(), jmt_avg(model, sim_opts(23000, 100000)));

SolverOpts ssa;

```

```

ssa.seed = 23000;
ssa.samples = 100000;
section("SSA");
print_avg(solve_avg("SSA", model, ssa));

SolverOpts fld;
fld.seed = 23000;
// The reference also passes cutoff=3 to FLD; a fluid limit enumerates no
// state space, and neither codebase's fluid options carry the field.
section("FLD");
print_avg(solve_avg("FLD", model, fld));

SolverOpts mva;
mva.method = "lin";
section("MVA");
print_avg(solve_avg("MVA", model, mva));

SolverOpts nc;
nc.cutoff = 3.0;
nc.seed = 23000;
section("NC");
print_avg(solve_avg("NC", model, nc));

section("LDES");
print_avg(solve_avg("LDES", model, sim_opts(23000, 100000)));

```

Running this edition prints

```

SOLVER: JMT
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
  Tput
Queue1      ClosedClass    98.568    0.69772    141.74     141.74     0.70515
  0.69804
Queue1      OpenClass      43.069    0.3019     143.85     143.85     0.30007
  0.29886
Queue2      ClosedClass    0.82325    0.35985    1.1341     1.1341     0.69804
  0.69752
Queue2      OpenClass      0.48659    0.20797    1.618      1.618      0.29886
  0.29886
Queue3      ClosedClass    0.39465    0.23505    0.55513    0.55513    0.69752
  0.69751
Queue3      OpenClass      0.28616    0.1685     0.96528    0.96528    0.29886
  0.29905
Queue4      ClosedClass    0.21695    0.17845    0.30292    0.30292    0.69751
  0.69752
Source      OpenClass      0          0          0          0          0
  0.30007

SOLVER: SSA
SolverSSA method=nrm
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
  Tput
Queue1      ClosedClass    98.5861    0.711774    138.508    138.508    0.710698
  0.711774
Queue1      OpenClass      40.3313    0.288226    139.93     139.93     0.3
  0.288226

... (37 captured-output lines omitted; rerun the source example for the full output)

Queue2      OpenClass      0.48647    0.212132    1.62157    1.62157     0.3
  0.3
Queue3      ClosedClass    0.401179    0.233477    0.572761    0.572761    0.70043
  0.70043
Queue3      OpenClass      0.293533    0.173205    0.978442    0.978442     0.3
  0.3
Queue4      ClosedClass    0.219237    0.175107    0.313004    0.313004    0.70043
  0.70043
Source      OpenClass      0          0          0          0          0
  0.3

SOLVER: LDES
N/A: SolverLDES is the SSJ simulation engine of the JAR and has no C++ port

```

The rows repeat for each of the 6 solvers the script runs (JMT, SSA, FLD, MVA, NC, LDES), which is the point of the example: the same model read by methods of different cost and different exactness.

2.4 Class switching

A job may change class as it moves. Switching is expressed either by off-diagonal blocks of the routing matrix, which give the probability of leaving node i as class r and arriving at node j as class s , or by a dedicated `ClassSwitch` node that performs the change in place.

Switching is what makes classes and chains differ: the classes that can reach one another form a chain, and it is the chain, not the class, that carries a conserved population. The diamond examples build a switching chain that is reducible on purpose, so that transient classes appear.

2.4.1 `cs_implicit`

A `ClassSwitch` node declared explicitly, against the implicit switching that a routing matrix with off-diagonal class blocks already expresses.

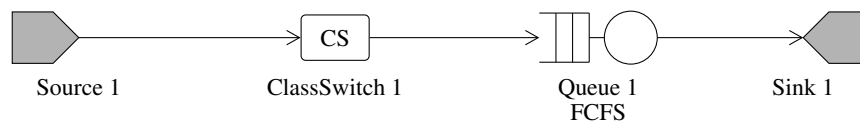


Figure 2.35: Topology of `cs_implicit`: an open network over 1 queueing station with a class-switch node.

Nodes	4: Source, Queue, ClassSwitch, Sink
Classes	2: Class1 (open), Class2 (open)
Scheduling	Queue 1: FCFS
Arrivals	Source 1 – Class1: Exp(0.1); Class2: Exp(0.5)
Service	Queue 1 – Class1: Exp(1); Class2: Exp(1)
Solvers	MVA

Table 2.35: Features of `cs_implicit`.

The example is built and solved as follows.

```
Net model("mmlcs");
Source source(model, "Source 1");
Queue queue(model, "Queue 1", SchedStrategy::FCFS);
Sink sink(model, "Sink 1");

OpenClass class1(model, "Class1", 0);
OpenClass class2(model, "Class2", 0);

source.set_arrival(class1, D::exp_mean(10.0));
source.set_arrival(class2, D::exp_mean(2.0));
queue.set_service(class1, D::exp_mean(1.0));
queue.set_service(class2, D::exp_mean(1.0));

// The matrix is declared after the classes, so the node can be too.
Matrix<double> csmatrix(2, 2, 0.0);
csmatrix(class1 - 1, class1 - 1) = 0.3;
csmatrix(class1 - 1, class2 - 1) = 0.7;
csmatrix(class2 - 1, class1 - 1) = 1.0;
const std::size_t cs = model.add_class_switch("ClassSwitch 1", csmatrix);

Routing P;
P.set(class1, class1, source, cs, 1.0);
P.set(class1, class1, queue, sink, 1.0);
P.set(class1, class1, cs, queue, 1.0);
P.set(class2, class2, source, cs, 1.0);
P.set(class2, class2, queue, sink, 1.0);
P.set(class2, class2, cs, queue, 1.0);
model.link(P);
```

```
const Sn& sn = model.get_struct();
print_routing_matrix(sn);

section("MVA");
print_avg_chain(sn, solve_avg("MVA", model));
```

Running this edition prints

```
Source 1 [Class1] => ClassSwitch 1 [Class1] : Pr=1.000000
Source 1 [Class2] => ClassSwitch 1 [Class2] : Pr=1.000000
Queue 1 [Class1] => Sink 1 [Class1] : Pr=1.000000
Queue 1 [Class2] => Sink 1 [Class2] : Pr=1.000000
ClassSwitch 1 [Class1] => Queue 1 [Class1] : Pr=0.300000
ClassSwitch 1 [Class1] => Queue 1 [Class2] : Pr=0.700000
ClassSwitch 1 [Class2] => Queue 1 [Class1] : Pr=1.000000
```

SOLVER: MVA
SolverMVA method=egflin

Station	Chain	QLen	Util	RespT	ResidT	ArvR
Source 1	Chain1	0	0	0	0	0
0.6						
Queue 1	Chain1	1.5	0.6	2.5	2.5	0.6
0.6						

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by MVA.

2.4.2 cs_multi_diamond

The same pattern with several branches.

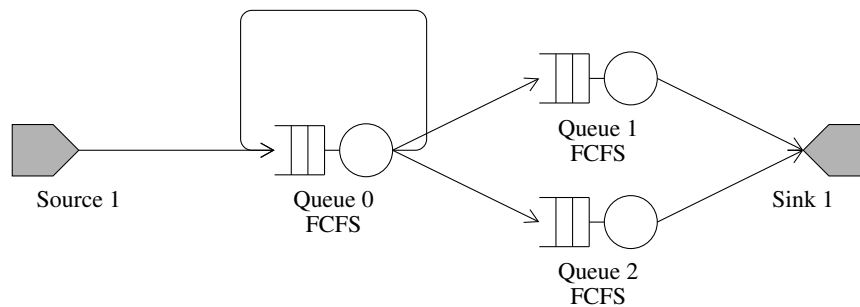


Figure 2.36: Topology of cs_multi_diamond: an open network over 3 queueing stations.

Nodes	5: Source, Queue $\times$ 3, Sink
Classes	3: Class1 (open), Class2 (open), Class3 (open)
Scheduling	Queue 0: FCFS; Queue 1: FCFS; Queue 2: FCFS
Arrivals	Source 1 – Class1: Exp(1); Class2: Disabled; Class3: Disabled
Service	Queue 0 – Class1: Exp(10); Class2: Disabled; Class3: Disabled Queue 1 – Class1: Disabled; Class2: Exp(20); Class3: Disabled Queue 2 – Class1: Disabled; Class2: Disabled; Class3: Exp(30)
Features	drop rule {'Class2': 'waitingQueue', 'Class3': 'waitingQueue'} at Queue 0; drop rule {'Class1': 'waitingQueue', 'Class3': 'waitingQueue'} at Queue 1; drop rule {'Class1': 'waitingQueue', 'Class2': 'waitingQueue'} at Queue 2; class switching on 1 link
Solvers	MVA

Table 2.36: Features of cs_multi_diamond.

The example is built and solved as follows.

```
Net model("mm1cs");
```



```

Source source(model, "Source 1");
Queue queue0(model, "Queue 0", SchedStrategy::FCFS);
Queue queue1(model, "Queue 1", SchedStrategy::FCFS);
Queue queue2(model, "Queue 2", SchedStrategy::FCFS);
Sink sink(model, "Sink 1");

OpenClass class1(model, "Class1", 0);
OpenClass class2(model, "Class2", 0);
OpenClass class3(model, "Class3", 0);

source.set_arrival(class1, Exp(1.0));
queue0.set_service(class1, Exp(10.0));
queue1.set_service(class2, Exp(20.0));
queue2.set_service(class3, Exp(30.0));

Routing P;
P.set(class1, class1, source, queue0, 1.0);
P.set(class1, class1, queue0, queue0, 0.2);
P.set(class1, class2, queue0, queue1, 0.3);
P.set(class1, class3, queue0, queue2, 0.5);
P.set(class2, class2, queue1, sink, 1.0);
P.set(class3, class3, queue2, sink, 1.0);
model.link(P);

const Sn& sn = model.get_struct();
print_routing_matrix(sn);

section("MVA");
print_avg_chain(sn, solve_avg("MVA", model));

```

Running this edition prints

```

Source 1 [Class1] => Queue 0 [Class1] : Pr=1.000000
Source 1 [Class2] => Queue 0 [Class2] : Pr=1.000000
Source 1 [Class3] => Queue 0 [Class3] : Pr=1.000000
Queue 0 [Class1] => Queue 0 [Class1] : Pr=0.200000
Queue 0 [Class1] => CS_Queue 0_to_Queue 1 [Class1] : Pr=0.300000
Queue 0 [Class1] => CS_Queue 0_to_Queue 2 [Class1] : Pr=0.500000
Queue 0 [Class2] => Queue 0 [Class2] : Pr=0.333333
Queue 0 [Class2] => CS_Queue 0_to_Queue 1 [Class2] : Pr=0.333333
Queue 0 [Class2] => CS_Queue 0_to_Queue 2 [Class2] : Pr=0.333333
Queue 0 [Class3] => Queue 0 [Class3] : Pr=0.333333
Queue 0 [Class3] => CS_Queue 0_to_Queue 1 [Class3] : Pr=0.333333
Queue 0 [Class3] => CS_Queue 0_to_Queue 2 [Class3] : Pr=0.333333
Queue 1 [Class1] => Sink 1 [Class1] : Pr=1.000000
Queue 1 [Class2] => Sink 1 [Class2] : Pr=1.000000
Queue 1 [Class3] => Sink 1 [Class3] : Pr=1.000000
Queue 2 [Class1] => Sink 1 [Class1] : Pr=1.000000

... (8 captured-output lines omitted; rerun the source example for the full output)

SOLVER: MVA
SolverMVA method=egflin
Station      Chain      QLen      Util      RespT      ResidT      ArrvR
Tput
Source 1      Chain1      0          0          0          0          0
1
Queue 0      Chain1      0.142857   0.125      0.114286   0.142857   1.25
1.25
Queue 1      Chain1      0.0191083  0.01875   0.0509554  0.0191083  0.375
0.375
Queue 2      Chain1      0.0212766  0.0208333  0.0340425  0.0212766  0.625
0.625

```

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by MVA.

2.4.3 cs_single_diamond

Class switching driven by a reducible Markov chain over the classes, the diamond pattern with one branch.

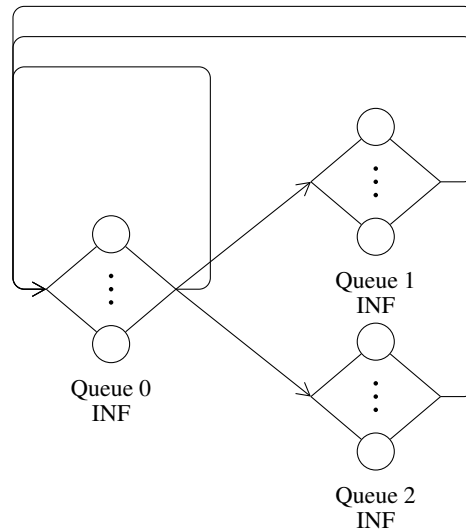


Figure 2.37: Topology of `cs_single_diamond`: a closed network over 3 queueing stations.

Nodes	3: Delay $\times$ 3
Classes	3: Class1 (closed, $N = 1$), Class2 (closed, $N = 0$), Class3 (closed, $N = 0$)
Scheduling	Queue 0: INF; Queue 1: INF; Queue 2: INF
Service	Queue 0 – Class1: Exp(1); Class2: Disabled; Class3: Disabled Queue 1 – Class1: Disabled; Class2: Exp(0.5); Class3: Disabled Queue 2 – Class1: Disabled; Class2: Disabled; Class3: Exp(0.3333)
Features	drop rule {'Class2': 'waitingQueue', 'Class3': 'waitingQueue'} at Queue 0; drop rule {'Class1': 'waitingQueue', 'Class3': 'waitingQueue'} at Queue 1; drop rule {'Class1': 'waitingQueue', 'Class2': 'waitingQueue'} at Queue 2
Solvers	MVA

Table 2.37: Features of `cs_single_diamond`.

The example is built and solved as follows.

```
Net model("mmlcs");
Delay queue0(model, "Queue 0");
Delay queue1(model, "Queue 1");
Delay queue2(model, "Queue 2");

ClosedClass class1(model, "Class1", 1, queue0);
ClosedClass class2(model, "Class2", 0, queue0);
ClosedClass class3(model, "Class3", 0, queue0);

queue0.set_service(class1, D::exp_mean(1.0));
queue1.set_service(class2, D::exp_mean(2.0));
queue2.set_service(class3, D::exp_mean(3.0));

Routing P;
P.set(class1, class1, queue0, queue0, 0.2);
P.set(class1, class2, queue0, queue1, 0.3);
P.set(class1, class3, queue0, queue2, 0.5);
P.set(class2, class1, queue1, queue0, 1.0);
P.set(class3, class1, queue2, queue0, 1.0);
model.link(P);

const Sn& sn = model.get_struct();
print_routing_matrix(sn);

section("MVA");
print_avg_chain(sn, solve_avg("MVA", model));
```

Running this edition prints

```
Queue 0 [Class1] => Queue 0 [Class1] : Pr=0.200000
Queue 0 [Class1] => CS_Queue 0_to_Queue 1 [Class1] : Pr=0.300000
Queue 0 [Class1] => CS_Queue 0_to_Queue 2 [Class1] : Pr=0.500000
Queue 0 [Class2] => Queue 0 [Class2] : Pr=0.333333
Queue 0 [Class2] => CS_Queue 0_to_Queue 1 [Class2] : Pr=0.333333
Queue 0 [Class2] => CS_Queue 0_to_Queue 2 [Class2] : Pr=0.333333
Queue 0 [Class3] => Queue 0 [Class3] : Pr=0.333333
Queue 0 [Class3] => CS_Queue 0_to_Queue 1 [Class3] : Pr=0.333333
Queue 0 [Class3] => CS_Queue 0_to_Queue 2 [Class3] : Pr=0.333333
Queue 1 [Class1] => CS_Queue 1_to_Queue 0 [Class1] : Pr=1.000000
Queue 1 [Class2] => CS_Queue 1_to_Queue 0 [Class2] : Pr=1.000000
Queue 1 [Class3] => CS_Queue 1_to_Queue 0 [Class3] : Pr=1.000000
Queue 2 [Class1] => CS_Queue 2_to_Queue 0 [Class1] : Pr=1.000000
Queue 2 [Class2] => CS_Queue 2_to_Queue 0 [Class2] : Pr=1.000000
Queue 2 [Class3] => CS_Queue 2_to_Queue 0 [Class3] : Pr=1.000000
CS_Queue 0_to_Queue 1 [Class1] => Queue 1 [Class2] : Pr=1.000000

... (10 captured-output lines omitted; rerun the source example for the full output)

CS_Queue 2_to_Queue 0 [Class3] => Queue 0 [Class1] : Pr=1.000000

SOLVER: MVA
SolverMVA method=exact
Station      Chain      QLen      Util      RespT      ResidT      ArrvR
Tput
Queue 0      Chain1      0.322581   0.322581   1           1           0.322581
0.322581
Queue 1      Chain1      0.193548   0.193548   2           0.6         0.0967742
0.0967742
Queue 2      Chain1      0.483871   0.483871   3           1.5         0.16129
0.16129
```

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by MVA.

2.4.4 cs_transient_class

A class that is transient in the switching chain, so it is visited but never returned to.
The example is built and solved as follows.

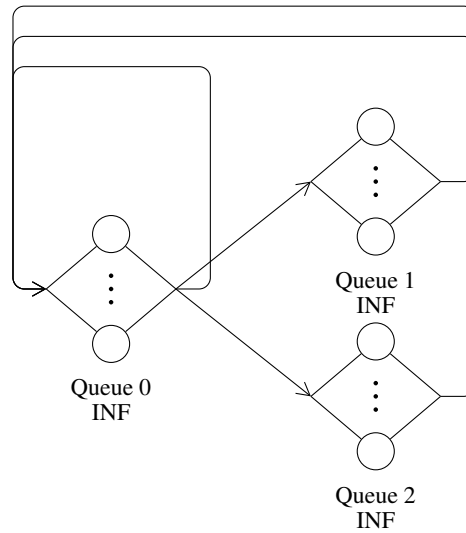


Figure 2.38: Topology of `cs_transient_class`: a closed network over 3 queueing stations.

Nodes	3: Delay $\times$ 3
Classes	3: Class1 (closed, $N = 1$), Class2 (closed, $N = 0$), Class3 (closed, $N = 0$)
Scheduling	Queue 0: INF; Queue 1: INF; Queue 2: INF
Service	Queue 0 – Class1: Exp(1); Class2: Exp(1); Class3: Exp(1) Queue 1 – Class1: Disabled; Class2: Exp(1); Class3: Disabled Queue 2 – Class1: Disabled; Class2: Disabled; Class3: Exp(1)
Features	drop rule {'Class1': 'waitingQueue', 'Class3': 'waitingQueue'} at Queue 1; drop rule {'Class1': 'waitingQueue', 'Class2': 'waitingQueue'} at Queue 2
Solvers	MVA

Table 2.38: Features of `cs_transient_class`.

```

Net model("reducible_cs");
Delay queue0(model, "Queue 0");
Delay queue1(model, "Queue 1");
Delay queue2(model, "Queue 2");

ClosedClass class1(model, "Class1", 1, queue0);
ClosedClass class2(model, "Class2", 0, queue0);
ClosedClass class3(model, "Class3", 0, queue0);

queue0.set_service(class1, D::exp_mean(1.0));
queue0.set_service(class2, D::exp_mean(1.0));
queue0.set_service(class3, D::exp_mean(1.0));
queue1.set_service(class2, D::exp_mean(1.0));
queue2.set_service(class3, D::exp_mean(1.0));

Routing P;
P.set(class1, class1, queue0, queue0, 0.2);
P.set(class1, class2, queue0, queue1, 0.3);
P.set(class1, class3, queue0, queue2, 0.5);
P.set(class2, class2, queue0, queue1, 1.0);
P.set(class2, class2, queue1, queue0, 1.0);
P.set(class3, class3, queue0, queue2, 1.0);
P.set(class3, class3, queue2, queue0, 1.0);
model.link(P);

const Sn& sn = model.get_struct();
print_routing_matrix(sn);

section("MVA");
print_avg_chain(sn, solve_avg("MVA", model));

```

Running this edition prints

```

Queue 0 [Class1] => Queue 0 [Class1] : Pr=0.200000
Queue 0 [Class1] => CS_Queue 0_to_Queue 1 [Class1] : Pr=0.300000
Queue 0 [Class1] => CS_Queue 0_to_Queue 2 [Class1] : Pr=0.500000
Queue 0 [Class2] => CS_Queue 0_to_Queue 1 [Class2] : Pr=1.000000
Queue 0 [Class3] => CS_Queue 0_to_Queue 2 [Class3] : Pr=1.000000
Queue 1 [Class1] => Queue 0 [Class1] : Pr=1.000000
Queue 1 [Class2] => Queue 0 [Class2] : Pr=1.000000
Queue 1 [Class3] => Queue 0 [Class3] : Pr=1.000000
Queue 2 [Class1] => Queue 0 [Class1] : Pr=1.000000
Queue 2 [Class2] => Queue 0 [Class2] : Pr=1.000000
Queue 2 [Class3] => Queue 0 [Class3] : Pr=1.000000
CS_Queue 0_to_Queue 1 [Class1] => Queue 1 [Class2] : Pr=1.000000
CS_Queue 0_to_Queue 1 [Class2] => Queue 1 [Class2] : Pr=1.000000
CS_Queue 0_to_Queue 1 [Class3] => Queue 1 [Class3] : Pr=1.000000
CS_Queue 0_to_Queue 2 [Class1] => Queue 2 [Class3] : Pr=1.000000
CS_Queue 0_to_Queue 2 [Class2] => Queue 2 [Class2] : Pr=1.000000
CS_Queue 0_to_Queue 2 [Class3] => Queue 2 [Class3] : Pr=1.000000

SOLVER: MVA
SolverMVA method=exact

```

Station	Chain	QLen	Util	RespT	ResidT	ArrvR
Queue 0	Chain1	0.5	0.5	1	1	0.5
Queue 1	Chain1	0.1875	0.1875	1	0.375	0.1875
Queue 2	Chain1	0.3125	0.3125	1	0.625	0.3125

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by MVA.

2.5 Priority models

Priorities are declared on the class and enforced by the discipline of the station. Head-of-line (HOL) service is non-preemptive: a low-priority job in service completes, and only the waiting order changes. The preemptive variants interrupt it instead, and the priority processor sharing family divides the capacity in

proportion to the priority weights.

Priority breaks product form, so these models are the province of the exact CTMC solver, of simulation, and of the approximations that the MVA solver offers for HOL.

2.5.1 prio_hol_closed

The closed counterpart.

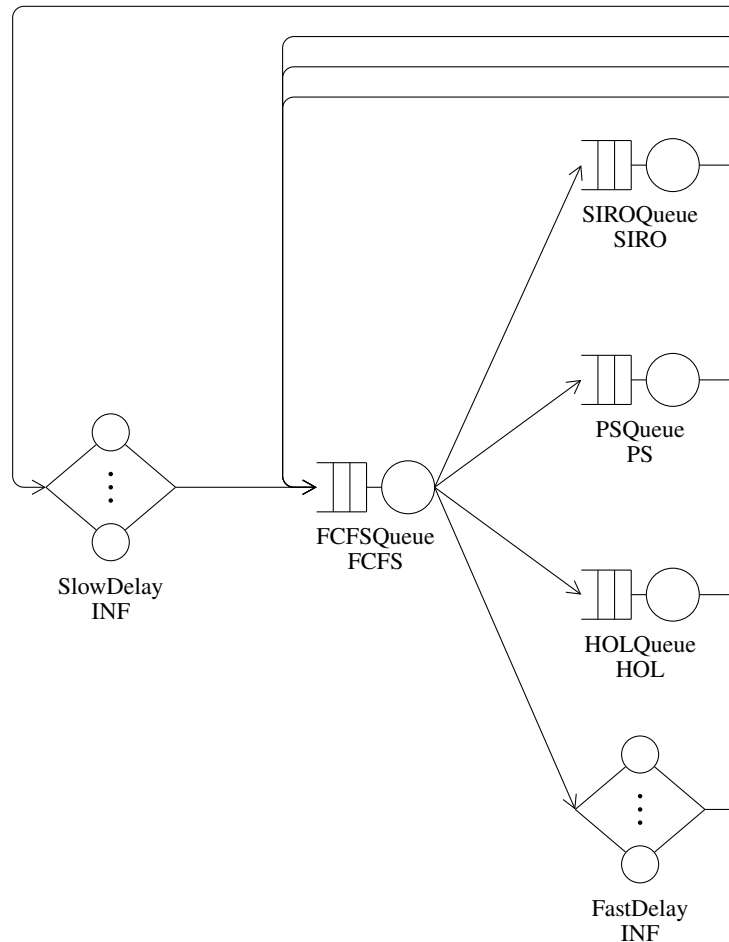


Figure 2.39: Topology of prio_hol_closed: a closed network over 6 queueing stations.

The example is built and solved as follows.

```
void prio_hol_closed() {
    Net m = prio_hol_closed_model();
    section("MVA");
    print_avg(m.get_struct(), mva_run(m));
    section("JMT");
    print_avg(m.get_struct(), jmt_avg(m, sim_opts(23000, 10000)));
    section("SSA");
    print_avg_sim(m.get_struct(), ssa_run(m, 10000, 23000));
    // The fourth block, which used to say this port had no LDES engine and no
    // client for one. It has both, and `ldes_station.h` serves HOL directly.
    section("LDES");
    print_avg(m.get_struct(), ldes_avg(m, sim_opts(23000, 10000)));
}
```

Running this edition prints

```
SOLVER: MVA
```

Nodes	6: Queue $\times$ 4, Delay $\times$ 2
Classes	3: Class1 (closed, $N = 18$), Class2 (closed, $N = 18$), Class3 (closed, $N = 18$)
Scheduling	SlowDelay: INF; FCFSQueue: FCFS; SIROQueue: SIRO; PSQueue: PS; HOLQueue: HOL; FastDelay: INF
Service	SlowDelay – Class1: Exp(0.1); Class2: Exp(0.1); Class3: Exp(0.1) FCFSQueue – Class1: Exp(3.333); Class2: Exp(2); Class3: Exp(1.667) SIROQueue – Class1: Exp(0.9091); Class2: Exp(0.7692); Class3: Exp(0.6667) PSQueue – Class1: Exp(1); Class2: Exp(0.9091); Class3: Exp(0.5263) HOLQueue – Class1: Exp(0.4); Class2: Exp(0.5263); Class3: Exp(0.2326) FastDelay – Class1: Exp(1); Class2: Exp(1); Class3: Exp(1)
Solvers	MVA, JMT, SSA, LDES

Table 2.39: Features of prio_hol_closed.

Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
SlowDelay 0.14877	Class1	1.4877	1.4877	10	10	0.14877
SlowDelay 0.00078005	Class2	0.0078005	0.0078005	10	10	0.00078005
SlowDelay 0.1447	Class3	1.447	1.447	10	10	0.1447
FCFSQueue 0.59509	Class1	0.49316	0.17853	0.8287	3.3148	0.59509
FCFSQueue 0.0031202	Class2	0.0032351	0.0015601	1.0368	4.1473	0.0031202
FCFSQueue 0.5788	Class3	0.6456	0.34728	1.1154	4.4616	0.5788
SIROQueue 0.14877	Class1	0.28096	0.16365	1.8885	1.8885	0.14877
SIROQueue 0.00078005	Class2	0.0016424	0.0010141	2.1056	2.1056	0.00078005
SIROQueue 0.1447	Class3	0.32966	0.21705	2.2782	2.2782	0.1447
PSQueue 0.14877	Class1	0.25406	0.14877	1.7077	1.7077	0.14877
PSQueue 0.00078005	Class2	0.0014773	0.00085805	1.8939	1.8939	0.00078005
PSQueue 0.1447	Class3	0.46625	0.27493	3.2222	3.2222	0.1447
HOLQueue 0.14877	Class1	15.335	0.37193	103.08	103.08	0.14877
HOLQueue 0.00078005	Class2	17.985	0.0014821	23056	23056	0.00078005
... (40 captured-output lines omitted; rerun the source example for the full output)						
HOLQueue -05	Class2	17.635	0.00011093	3.0204e+05	3.0204e+05	0.0050024 5.8386e
HOLQueue 0.14713	Class3	14.466	0.63266	98.322	98.322	0.14746
FastDelay 0.14944	Class1	0.14944	0.14944	1	1	0.1525
FastDelay 0.0056507	Class2	0.0056507	0.0056507	1	1	0.0050024
FastDelay 0.1554	Class3	0.1554	0.1554	1	1	0.14746
SOLVER: LDES						
N/A: SolverLDES is the SSJ-based discrete-event engine of the JAR, driven over model.json by the MATLAB and Python clients; the C++ port carries no LDES engine and no client for it, and SSA above is a different simulator, not this one						

The rows repeat for each of the 4 solvers the script runs (MVA, JMT, SSA, LDES), which is the point of the example: the same model read by methods of different cost and different exactness.

2.5.2 prio_hol_open

Head-of-line non-preemptive priority in an open network, with PS, SIRO and FCFS stations alongside. The example is built and solved as follows.

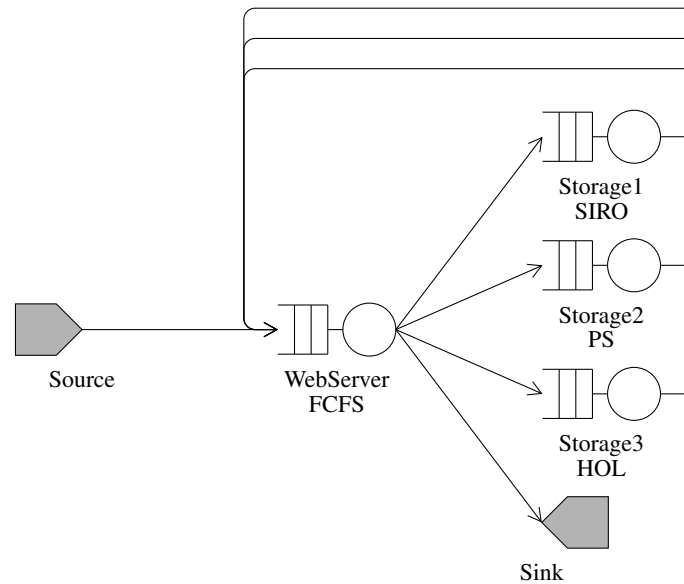


Figure 2.40: Topology of `prio_hol_open`: an open network over 4 queueing stations.

Nodes	6: Source, Queue $\times$ 4, Sink
Classes	3: Class1 (open), Class2 (open), Class3 (open)
Scheduling	WebServer: FCFS; Storage1: SIRO; Storage2: PS; Storage3: HOL
Arrivals	Source – Class1: Exp(0.1); Class2: Exp(0.1); Class3: Exp(0.1)
Service	WebServer – Class1: Exp(3.333); Class2: Exp(2); Class3: Exp(1.667) Storage1 – Class1: Exp(0.9091); Class2: Exp(0.7692); Class3: Exp(0.6667) Storage2 – Class1: Exp(0.5); Class2: Exp(0.4762); Class3: Exp(0.5263) Storage3 – Class1: Exp(0.4); Class2: Exp(0.5263); Class3: Exp(0.2326)
Solvers	CTMC, MVA, JMT, SSA

Table 2.40: Features of `prio_hol_open`.


```

Net m = prio_hol_open_model();
section("CTMC");
print_avg(m.get_struct(), ctmc_run(m, 1.0));
// The MVA-specific iter_tol (1e-6), not the generic 1e-4: the looser one
// stopped the AMVA fixed point 2.9e-5 short on Storage3 and split this row
// from the engines that solve at the solver's own tolerance.
section("MVA");
print_avg(m.get_struct(), mva_run(m));
section("JMT");
print_avg(m.get_struct(), jmt_avg(m, sim_opts(23000, 10000)));
section("SSA");
print_avg_sim(m.get_struct(), ssa_run(m, 10000, 23000));

```

Running this edition prints

SOLVER: CTMC						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Source	Class1	0	0	0	0	0
0.052281						
Source	Class2	0	0	0	0	0
0.050449						
Source	Class3	0	0	0	0	0
0.046082						
WebServer	Class1	0.090117	0.062737	0.43092	1.7237	0.20912
0.20912						
WebServer	Class2	0.12132	0.1009	0.60119	2.4048	0.20179
0.20179						
WebServer	Class3	0.12559	0.1106	0.68136	2.7255	0.18433
0.18433						
Storage1	Class1	0.06951	0.057509	1.3295	1.3295	0.052281
0.052281						
Storage1	Class2	0.075291	0.065583	1.4924	1.4924	0.050449
0.050449						
Storage1	Class3	0.076531	0.069123	1.6607	1.6607	0.046082
0.046082						
Storage2	Class1	0.12865	0.10456	2.4607	2.4607	0.052281
0.052281						
Storage2	Class2	0.1301	0.10594	2.5789	2.5789	0.050449
0.050449						
Storage2	Class3	0.10964	0.087556	2.3793	2.3793	0.046082
0.046082						
Storage3	Class1	0.18891	0.1307	3.6134	3.6134	0.052281
0.052281						
Storage3	Class2	0.1688	0.095853	3.3459	3.3459	0.050449
0.050449						
... (47 captured-output lines omitted; rerun the source example for the full output)						
Storage1	Class2	0.23072	0.12843	2.3354	2.3354	0.092582
0.098791						
Storage1	Class3	0.24995	0.15804	2.3724	2.3724	0.10206
0.10536						
Storage2	Class1	0.96429	0.24142	7.9887	7.9887	0.1111
0.12071						
Storage2	Class2	0.82108	0.205	8.4112	8.4112	0.092582
0.097617						
Storage2	Class3	0.86533	0.22195	7.4076	7.4076	0.10206
0.11682						
Storage3	Class1	1.4084	0.27523	12.793	12.793	0.1111
0.11009						
Storage3	Class2	10.126	0.17467	110.15	110.15	0.092582
0.091933						
Storage3	Class3	1.6252	0.44369	15.751	15.751	0.10206
0.10318						

The rows repeat for each of the 4 solvers the script runs (CTMC, MVA, JMT, SSA), which is the point of the example: the same model read by methods of different cost and different exactness.

2.5.3 prio_identical

Priority classes with identical parameters, the sanity check that priority alone does not move the aggregate metrics.

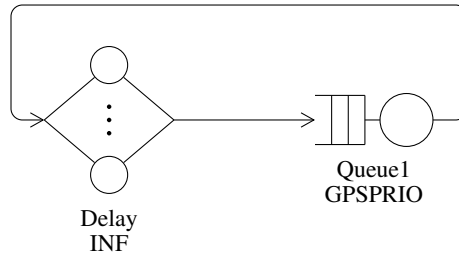


Figure 2.41: Topology of prio_identical: a closed network over 2 queueing stations.

Nodes	2: Queue, Delay
Classes	4: Class1 (closed, $N = 6$), Class2 (closed, $N = 4$), Class3 (closed, $N = 4$), Class4 (closed, $N = 1$)
Scheduling	Delay: INF; Queue1: GPSPRIO
Service	Delay – Class1: Erlang(3,2); Class2: Exp(1); Class3: Exp(1); Class4: Exp(2) Queue1 – Class1: Exp(30); Class2: Exp(2); Class3: Exp(12); Class4: Exp(1)
Solvers	CTMC, JMT

Table 2.41: Features of prio_identical.

The example is built and solved as follows.

```
void prio_identical() {
    Net m = prio_identical_model();
    section("CTMC");
    print_avg(m.get_struct(), ctmc_run(m));
    section("JMT");
    print_avg(m.get_struct(), jmt_avg(m, sim_opts(23000, 10000)));
}
```

Running this edition prints

SOLVER: CTMC	Station	JobClass	QLen	Util	RespT	ResidT	ArvR
	Tput						
Delay	8.4558	Class1	5.6372	5.6372	0.66667	0.66667	8.4558
Delay	0.91384	Class2	0.91384	0.91384	1	1	0.91384
Delay	3.0493	Class3	3.0493	3.0493	1	1	3.0493
Delay	0.0063909	Class4	0.0031954	0.0031954	0.5	0.5	0.0063909
Queue1	8.4558	Class1	0.3628	0.28186	0.042905	0.042905	8.4558
Queue1	0.91384	Class2	3.0862	0.45692	3.3772	3.3772	0.91384
Queue1	3.0493	Class3	0.9507	0.25411	0.31177	0.31177	3.0493
Queue1	0.0063909	Class4	0.9968	0.0063909	155.97	155.97	0.0063909
SOLVER: JMT							
Station	JobClass	QLen	Util	RespT	ResidT	ArvR	
Tput							
Delay	8.5533	Class1	5.6487	5.6487	0.66214	0.66214	8.5567
Delay	0.90847	Class2	0.89602	0.89602	0.99227	0.99227	0.89476
Delay	3.0983	Class3	3.0308	3.0308	0.97343	0.97343	3.1087
Delay	0.0065312	Class4	0.0031932	0.0031932	0.50558	0.50558	0.0065312
Queue1	8.5531	Class1	0.35125	0.28302	0.040881	0.040881	8.5533
Queue1	0.90646	Class2	3.104	0.42925	3.417	3.417	0.90847
Queue1	3.0462	Class3	0.96924	0.27143	0.30885	0.30885	3.0983

Queue1	Class4	0.99678	0.0065642	152.63	152.63	0.0065312
0.0065303						

The rows repeat for each of the 2 solvers the script runs (CTMC, JMT), which is the point of the example: the same model read by methods of different cost and different exactness.

2.5.4 prio_psprio

Priority processor sharing, where the share of a class is set by its priority.

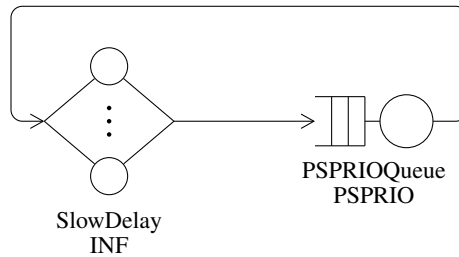


Figure 2.42: Topology of prio_psprio: a closed network over 2 queueing stations.

Nodes	2: Queue, Delay
Classes	2: Class1 (closed, $N = 2$), Class2 (closed, $N = 2$)
Scheduling	SlowDelay: INF; PSPRIOQueue: PSPRIO
Service	SlowDelay – Class1: Erlang(3,2); Class2: HyperExp(0.5,0.5;3,10) PSPRIOQueue – Class1: HyperExp(0.1,0.9;1,10); Class2: Exp(1)
Solvers	CTMC, JMT, SSA

Table 2.42: Features of prio_psprio.

The example is built and solved as follows.

```
void prio_psprio() {
    Net m = prio_psprio_model();
    section("CTMC");
    print_avg(m.get_struct(), ctmc_run(m));
    section("JMT");
    print_avg(m.get_struct(), jmt_avg(m, sim_opts(23000, 5000)));
    section("SSA");
    print_avg_sim(m.get_struct(), ssa_run(m, 5000, 23000));
}
```

Running this edition prints

SOLVER: CTMC						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
SlowDelay	Class1	1.4834	1.4834	0.66667	0.66667	2.2252
2.2252						
SlowDelay	Class2	0.12349	0.12349	0.21667	0.21667	0.56996
0.56996						
PSPRIOQueue	Class1	0.51655	0.42278	0.23214	0.23214	2.2252
2.2252						
PSPRIOQueue	Class2	1.8765	0.56996	3.2923	3.2923	0.56996
0.56996						
SOLVER: JMT						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
SlowDelay	Class1	1.5161	1.5161	0.66563	0.66563	2.2873
2.2388						
SlowDelay	Class2	0.12717	0.12717	0.20933	0.20933	0.58227
0.58227						
PSPRIOQueue	Class1	0.48386	0.40341	0.21719	0.21719	2.2388
2.2385						

PSPRIOQueue 0.57109	Class2	1.87	0.59305	3.179	3.179	0.58227
SOLVER: SSA						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
SlowDelay 2.2523	Class1	1.4694	1.4694	0.65239	0.65239	2.2491
SlowDelay 0.55257	Class2	0.11132	0.11132	0.20147	0.20147	0.55726
PSPRIOQueue 2.2491	Class1	0.53064	0.42793	0.23593	0.23593	2.2523
PSPRIOQueue 0.55726	Class2	1.8887	0.55257	3.3892	3.3892	0.55257

The rows repeat for each of the 3 solvers the script runs (CTMC, JMT, SSA), which is the point of the example: the same model read by methods of different cost and different exactness.

2.6 Fork-join networks

A `Fork` node splits an arriving job into sibling tasks, one per outgoing branch, and a `Join` node recombines them. The response time of the parent job is the maximum over its siblings, which is why fork-join has no product-form solution and why the join is where the modeling subtleties live: how many siblings are required (the quorum), whether the join is present at all, and what happens when branches share a station.

The family is the largest in the basic set because each of those questions has its own script. Nesting a fork inside a branch, three or more branches, asymmetric branch demands and class switching on the siblings are all covered separately, so a failure can be localized.

2.6.1 fj_asymm

Asymmetric branches, whose response time is set by the slowest one.

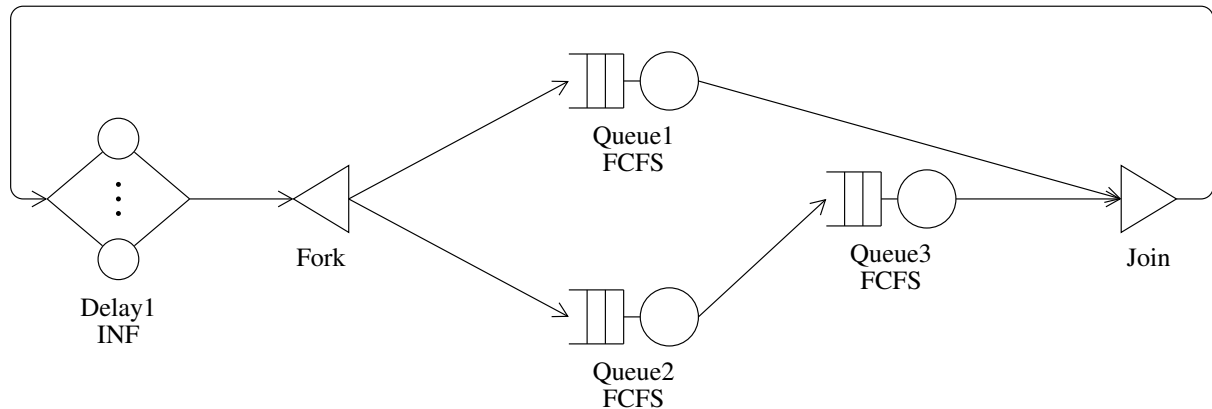


Figure 2.43: Topology of `fj_asymm`: a closed network over 4 queueing stations with a fork-join block.

Nodes	6: Queue $\times$ 3, Delay, Fork, Join
Classes	1: class1 (closed, $N = 10$)
Scheduling	Delay1: INF; Queue1: FCFS; Queue2: FCFS; Queue3: FCFS
Service	Delay1 – class1: Exp(0.5) Queue1 – class1: Exp(1) Queue2 – class1: Exp(2) Queue3 – class1: Exp(1)
Solvers	JMT, MVA

Table 2.43: Features of `fj_asymm`.

The example is built and solved as follows.

```
void fj_asymm() {
    Net m = fj_asymm_model();
    section("JMT");
    print_avg(m.get_struct(), jmt_avg(m, sim_opts(23000)));
    section("MVA");
    print_avg(m.get_struct(), mva_run(m));
}
```

Running this edition prints

SOLVER: JMT						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Delay1	class1	1.8108	1.8108	1.9614	1.9614	0.94914
0.94982						
Queue1	class1	6.1513	0.95281	6.5892	6.5892	0.94982
0.94864						
Queue2	class1	0.82403	0.47064	0.8557	0.8557	0.94982
0.95192						
Queue3	class1	5.1269	0.93083	5.4049	5.4049	0.95192
0.94412						
Join	class1	4.293	0	2.2396	2.2396	1.8954
0.94685						
SOLVER: MVA						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Delay1	class1	1.7139	1.7139	2	2	0.85693
0.85693						
Queue1	class1	5.2237	0.85693	6.0958	6.0958	0.85693
0.85693						
Queue2	class1	0.73602	0.42846	0.85891	0.85891	0.85693
0.85693						
Queue3	class1	5.2237	0.85693	6.0958	6.0958	0.85693
0.85693						
Join	class1	5.6159	0	3.2768	3.2768	1.7139
0.85693						

The rows repeat for each of the 2 solvers the script runs (JMT, MVA), which is the point of the example: the same model read by methods of different cost and different exactness.

2.6.2 fj_basic_closed

The closed counterpart, on PS branches.

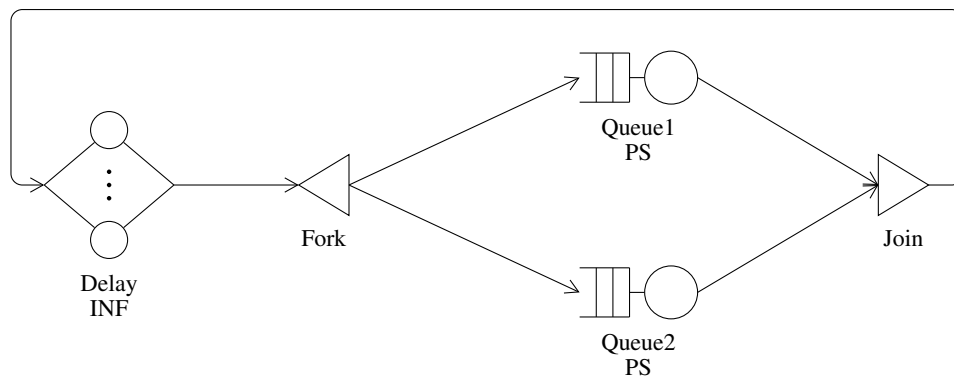


Figure 2.44: Topology of `fj_basic_closed`: a closed network over 3 queueing stations with a fork-join block.

The example is built and solved as follows.

```
Net m = fj_basic_closed_model();
section("JMT");
print_avg(m.get_struct(), jmt_avg(m, sim_opts(23000)));
```

Nodes	5: Queue $\times$ 2, Delay, Fork, Join
Classes	1: class1 (closed, $N = 5$)
Scheduling	Delay: INF; Queue1: PS; Queue2: PS
Service	Delay – class1: Exp(1) Queue1 – class1: Exp(1) Queue2 – class1: Exp(1)
Solvers	JMT, MVA

Table 2.44: Features of fj_basic_closed.

```
section("MVA");
print_avg(m.get_struct(), mva_run(m, "amva", "ht"));
```

Running this edition prints

SOLVER: JMT						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Delay	class1	0.86625	0.86625	0.99611	0.99611	0.89442
0.89768						
Queue1	class1	2.6475	0.88812	3.0945	3.0945	0.89768
0.89761						
Queue2	class1	2.5863	0.89336	2.815	2.815	0.89768
0.89757						
Join	class1	2.9633	0	1.685	1.685	1.724
0.88799						
SOLVER: MVA						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Delay	class1	0.94049	0.94049	1	1	0.94049
0.94049						
Queue1	class1	2.7063	0.94049	2.8776	2.8776	0.94049
0.94049						
Queue2	class1	2.7063	0.94049	2.8776	2.8776	0.94049
0.94049						
Join	class1	2.7063	0	1.4388	1.4388	1.881
0.94049						

The rows repeat for each of the 2 solvers the script runs (JMT, MVA), which is the point of the example: the same model read by methods of different cost and different exactness.

2.6.3 fj_basic_nesting

A fork inside a branch of another fork, with open and closed classes.

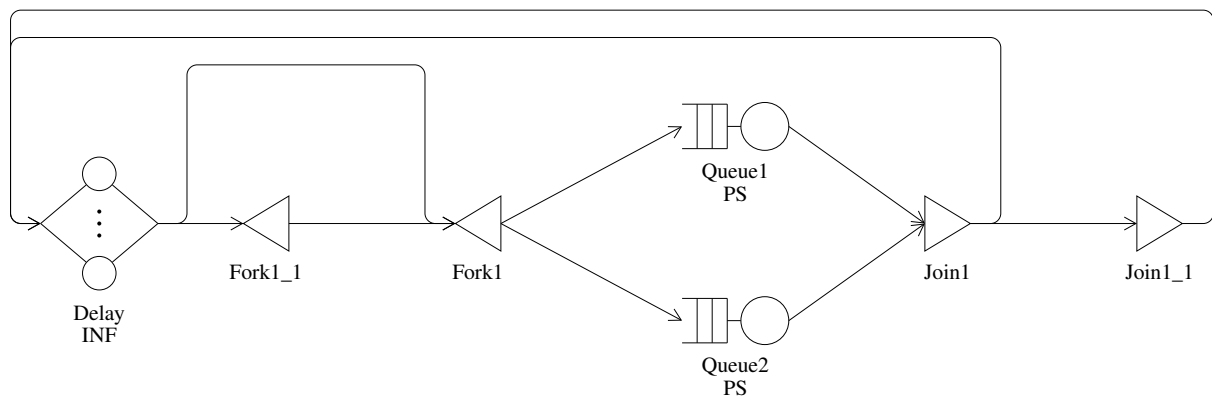


Figure 2.45: Topology of fj_basic_nesting: a closed network over 3 queueing stations with a fork-join block.

The example is built and solved as follows.

```
void fj_basic_nesting() {
```

Nodes	7: Queue $\times$ 2, Delay, Fork $\times$ 2, Join $\times$ 2
Classes	2: class1 (closed, $N = 5$), class2 (closed, $N = 2$)
Scheduling	Delay: INF; Queue1: PS; Queue2: PS
Service	Delay – class1: Exp(0.25); class2: Exp(0.25) Queue1 – class1: Exp(1); class2: Exp(2) Queue2 – class1: Exp(0.75); class2: Exp(2)
Features	class switching on 2 links
Solvers	JMT, MVA

Table 2.45: Features of fj_basic_nesting.

```

Net m = fj_basic_nesting_model();
section("JMT");
print_avg(m.get_struct(), jmt_avg(m, sim_opts(23000)));
section("MVA");
print_avg(m.get_struct(), mva_run(m));

```

Running this edition prints

SOLVER: JMT						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Delay	class1	2.0368	2.0368	4.0283	4.0283	0.50637
0.50487						
Delay	class2	0.98642	0.98642	3.9138	3.9138	0.25427
0.2541						
Join1	class1	2.2026	0	2.1777	2.1777	0.99731
0.50798						
Join1	class2	1.0479	0	1.0011	1.0011	1.0445
0.51183						
Join1_1	class2	0.54106	0	1.0555	1.0555	0.51183
0.2541						
Queue1	class1	1.3447	0.49509	2.6485	2.6485	0.50487
0.5051						
Queue1	class2	0.82969	0.26813	1.6297	1.6297	0.51178
0.51204						
Queue2	class1	2.3299	0.6631	4.6827	4.6827	0.50487
0.50447						
Queue2	class2	1.1149	0.25869	2.1947	2.1947	0.51178
0.51352						
SOLVER: MVA						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Delay	class1	1.762	1.762	4	4	0.44051
0.44051						
Delay	class2	0.93501	0.93501	4	4	0.23375
0.23375						
Join1	class1	2.4047	0	2.7294	2.7294	0.88102
0.44051						
Join1	class2	0.99834	0	0.85419	0.85419	0.93501
0.4675						
Join1_1	class2	0.71667	0	1.533	1.533	0.4675
0.23375						
Queue1	class1	1.2807	0.44051	2.9074	2.9074	0.44051
0.44051						
Queue1	class2	0.68934	0.23375	1.4745	1.4745	0.23375
0.4675						
Queue2	class1	2.9009	0.58734	6.5853	6.5853	0.44051
0.44051						
Queue2	class2	1.179	0.23375	2.5219	2.5219	0.23375
0.4675						

The rows repeat for each of the 2 solvers the script runs (JMT, MVA), which is the point of the example: the same model read by methods of different cost and different exactness.

2.6.4 fj_basic_open

One fork, two parallel FCFS branches and a join, in an open network.
The example is built and solved as follows.

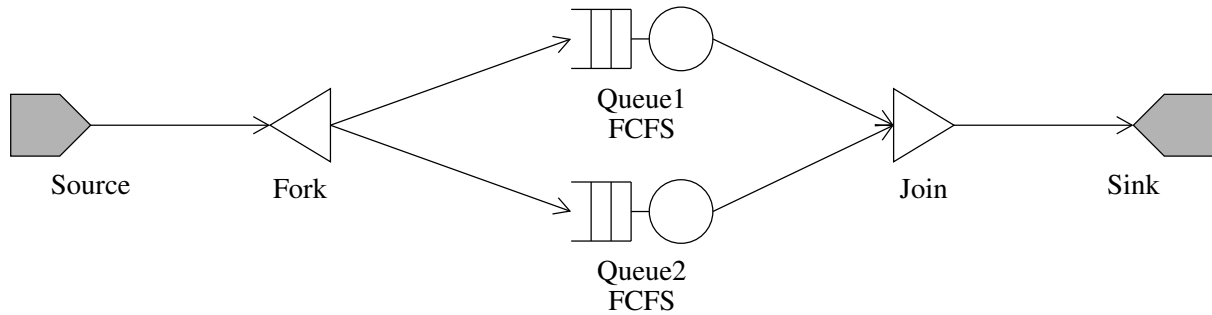


Figure 2.46: Topology of `fj_basic_open`: an open network over 2 queueing stations with a fork-join block.

Nodes	6: Source, Queue $\times$ 2, Fork, Join, Sink
Classes	1: class1 (open)
Scheduling	Queue1: FCFS; Queue2: FCFS
Arrivals	Source – class1: Exp(0.05)
Service	Queue1 – class1: Exp(1) Queue2 – class1: Exp(2)
Solvers	JMT, MVA, LDES, MAM

Table 2.46: Features of `fj_basic_open`.

```
void fj_basic_open() {
    Net m = fj_basic_open_model();
    section("JMT");
    print_avg(m.get_struct(), jmt_avg(m, sim_opts(23000)));
    section("MVA");
    print_avg(m.get_struct(), mva_run(m));
    // `LDES(model, seed=23000)`, the reference's third solver. It was left out,
    // so the golden's whole LDES row went unmeasured; the engine is driven the
    // same way the other simulation blocks in this corpus drive it.
    section("LDES");
    print_avg(m.get_struct(), ldes_avg(m, sim_opts(23000)));
}
```

Running this edition prints

SOLVER: JMT						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Source	class1	0	0	0	0	0
0.050494						
Queue1	class1	0.051923	0.049524	1.0478	1.0478	0.050494
0.050812						
Queue2	class1	0.026691	0.026329	0.50973	0.50973	0.050494
0.050896						
Join	class1	0.044462	0	0.43237	0.43237	0.10395
0.050857						

SOLVER: MVA						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Source	class1	0	0	0	0	0
0.05						
Queue1	class1	0.052631	0.05	1.0526	1.0526	0.05
0.05						
Queue2	class1	0.025641	0.025	0.51282	0.51282	0.05
0.05						
Join	class1	0.043789	0	0.43789	0.43789	0.1
0.05						

SOLVER: LDES						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Source	class1	0	0	0	0	0
0.05						
Queue1	class1	0.052923	0.050267	1.0563	1.0563	0.051673
0.050071						

Queue2	class1	0.025718	0.025066	0.51298	0.51298	0.051673
0.050071						
Join	class1	0.043779	0	0.43706	0.43706	0.10335
0.050071						

The rows repeat for each of the 3 solvers the script runs (JMT, MVA, LDES), which is the point of the example: the same model read by methods of different cost and different exactness.

2.6.5 fj_complex_serial

Several fork-join blocks in series inside a larger network.

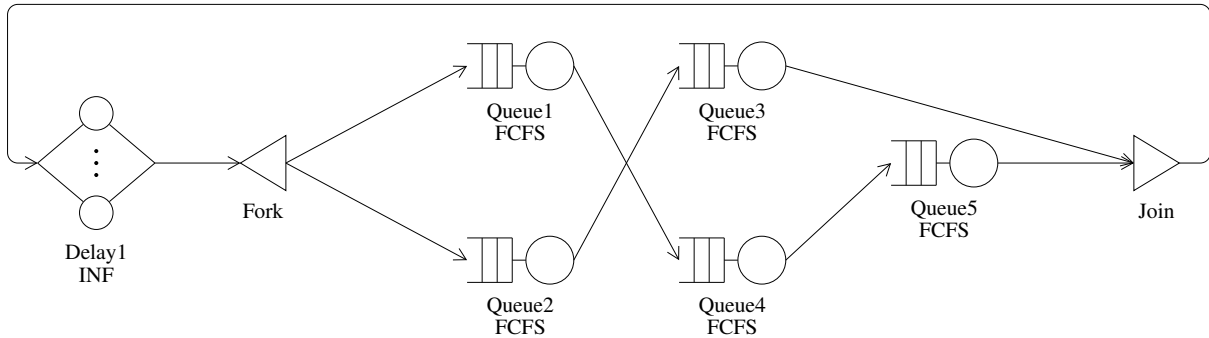


Figure 2.47: Topology of fj_complex_serial: a closed network over 6 queueing stations with a fork-join block.

Nodes	8: Queue × 5, Delay, Fork, Join
Classes	1: class1 (closed, $N = 10$)
Scheduling	Delay1: INF; Queue1: FCFS; Queue2: FCFS; Queue3: FCFS; Queue4: FCFS; Queue5: FCFS
Service	Delay1 – class1: Exp(0.5) Queue1 – class1: Exp(1) Queue2 – class1: Exp(2) Queue3 – class1: Exp(1) Queue4 – class1: Exp(3) Queue5 – class1: Exp(0.8)
Solvers	JMT, MVA

Table 2.47: Features of fj_complex_serial.

The example is built and solved as follows.

```
void fj_complex_serial() {
    Net m = fj_complex_serial_model();
    section("JMT");
    print_avg(m.get_struct(), jmt_avg(m, sim_opts(23000)));
    section("MVA");
    print_avg(m.get_struct(), mva_run(m));
}
```

Running this edition prints

SOLVER: JMT						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Delay1	class1	1.5109	1.5109	1.9611	1.9611	0.76579
0.75823						
Queue1	class1	2.5753	0.76218	3.3293	3.3293	0.75823
0.75929						
Queue2	class1	0.5914	0.38738	0.79434	0.79434	0.75823
0.76578						
Queue3	class1	2.7793	0.76354	3.4472	3.4472	0.76578
0.75992						
Queue4	class1	0.32517	0.24092	0.41843	0.41843	0.75929
0.75933						
Queue5	class1	5.3388	0.95413	7.2633	7.2633	0.75933
0.76579						
Join	class1	5.3359	0	3.4698	3.4698	1.5125
0.75865						

SOLVER: MVA						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Delay1	class1	1.3867	1.3867	2	2	0.69335
0.69335						
Queue1	class1	2.142	0.69335	3.0894	3.0894	0.69335
0.69335						
Queue2	class1	0.52374	0.34667	0.75538	0.75538	0.69335
0.69335						
Queue3	class1	2.142	0.69335	3.0894	3.0894	0.69335
0.69335						
Queue4	class1	0.29836	0.23112	0.43031	0.43031	0.69335
0.69335						
Queue5	class1	5.6088	0.86669	8.0894	8.0894	0.69335
0.69335						
Join	class1	6.7098	0	4.8387	4.8387	1.3867
0.69335						

The rows repeat for each of the 2 solvers the script runs (JMT, MVA), which is the point of the example: the same model read by methods of different cost and different exactness.

2.6.6 fj_cs_multi_visits

Class switching with several visits per branch.

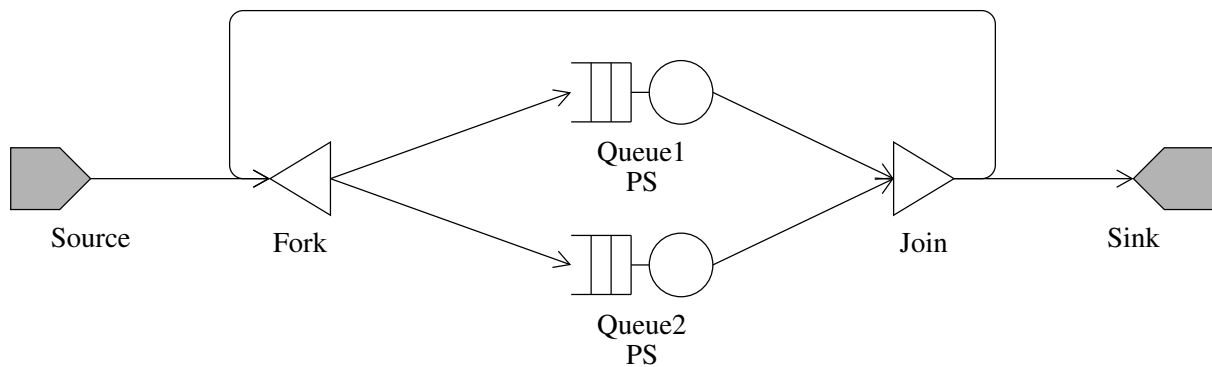


Figure 2.48: Topology of fj_cs_multi_visits: an open network over 2 queueing stations with a fork-join block.

Nodes	6: Source, Queue × 2, Fork, Join, Sink
Classes	2: class1 (open), class2 (open)
Scheduling	Queue1: PS; Queue2: PS
Arrivals	Source – class1: Exp(0.1); class2: Disabled
Service	Queue1 – class1: Exp(1); class2: Exp(1) Queue2 – class1: Exp(1); class2: Exp(1)
Features	class switching on 1 link
Solvers	MVA, JMT

Table 2.48: Features of fj_cs_multi_visits.

The example is built and solved as follows.

```

void fj_cs_multi_visits() {
    Net m = fj_cs_multi_visits_model();
    section("MVA");
    print_avg(m.get_struct(), mva_run(m));
    section("JMT");
    print_avg(m.get_struct(), jmt_avg(m, sim_opts(23000)));
}

```

Running this edition prints

SOLVER: MVA						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Source	class1	0	0	0	0	0
0.1						
Queue1	class1	0.12474	0.1	1.2474	1.2474	0.1
0.1						
Queue1	class2	0.12474	0.1	1.2474	1.2474	0.1
0.1						
Queue2	class1	0.12474	0.1	1.2474	1.2474	0.1
0.1						
Queue2	class2	0.12474	0.1	1.2474	1.2474	0.1
0.1						
Join	class1	0.12474	0	0.62368	0.62368	0.2
0.1						
Join	class2	0.12474	0	0.62368	0.62368	0.2
0.1						
SOLVER: JMT						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Source	class1	0	0	0	0	0
0.099935						
Queue1	class1	0.12819	0.09701	1.2308	1.2308	0.099935
0.099937						
Queue1	class2	0.1269	0.1011	1.248	1.248	0.099931
0.099928						
Queue2	class1	0.12642	0.10181	1.244	1.244	0.099935
0.099931						
Queue2	class2	0.1243	0.1007	1.2318	1.2318	0.099931
0.099981						
Join	class1	0.13493	0	0.66288	0.66288	0.19881
0.099931						
Join	class2	0.12888	0	0.6401	0.6401	0.19882
0.099924						

The rows repeat for each of the 2 solvers the script runs (MVA, JMT), which is the point of the example: the same model read by methods of different cost and different exactness.

2.6.7 fj_cs_postfork

Class switching after the fork, on the sibling tasks.

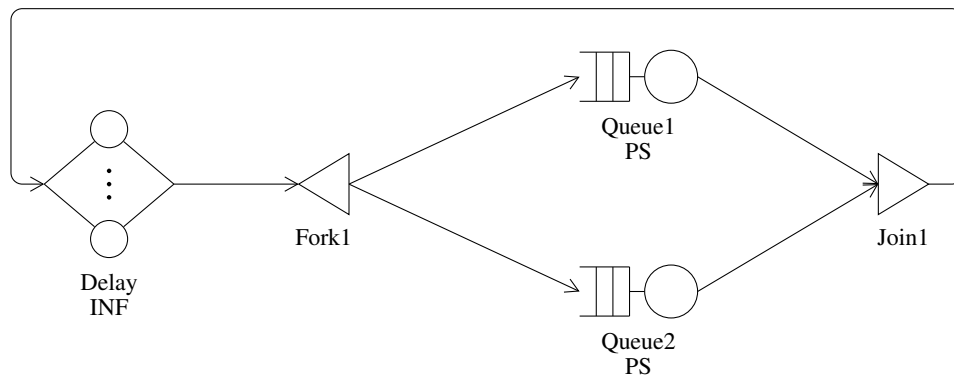


Figure 2.49: Topology of fj_cs_postfork: a closed network over 3 queueing stations with a fork-join block.

The example is built and solved as follows.

```
void fj_cs_postfork() {
    Net m = fj_cs_postfork_model();
    section("MVA");
    print_avg(m.get_struct(), mva_run(m));
    section("JMT");
    print_avg(m.get_struct(), jmt_avg(m, sim_opts(23000)));
}
```

Nodes	5: Queue $\times$ 2, Delay, Fork, Join
Classes	2: class1 (closed, $N = 1$), class2 (closed, $N = 1$)
Scheduling	Delay: INF; Queue1: PS; Queue2: PS
Service	Delay – class1: Exp(0.25); class2: Exp(0.25) Queue1 – class1: Exp(2); class2: Exp(2) Queue2 – class1: Exp(2); class2: Exp(2)
Features	class switching on 3 links
Solvers	MVA, JMT

Table 2.49: Features of `fj_cs_postfork`.

Running this edition prints

SOLVER: MVA						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Delay	class1	1.6364	1.6364	4	4	0.40911
0.40911						
Join1	class1	0.24644	0	0.30119	0.30119	0.81822
0.40911						
Queue1	class1	0.24644	0.20455	0.60238	0.60238	0.40911
0.40911						
Queue2	class1	0.24644	0.20455	0.60238	0.60238	0.40911
0.40911						
SOLVER: JMT						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Delay	class1	0.83919	0.83919	4.0008	4.0008	0.20697
0.20977						
Delay	class2	0.82894	0.82894	3.8875	0	0.20948
0.21192						
Join1	class1	0.21374	0	0.2649	0.2649	0.8241
0.20774						
Join1	class2	0	0	0	0	0
0.21192						
Queue1	class1	0.21284	0.19531	0.5414	0.5414	0.41471
0.41463						
Queue2	class1	0.22108	0.19926	0.54182	0.54182	0.41471
0.4147						

The rows repeat for each of the 2 solvers the script runs (MVA, JMT), which is the point of the example: the same model read by methods of different cost and different exactness.

2.6.8 `fj_cs_prefork`

Class switching before the fork.

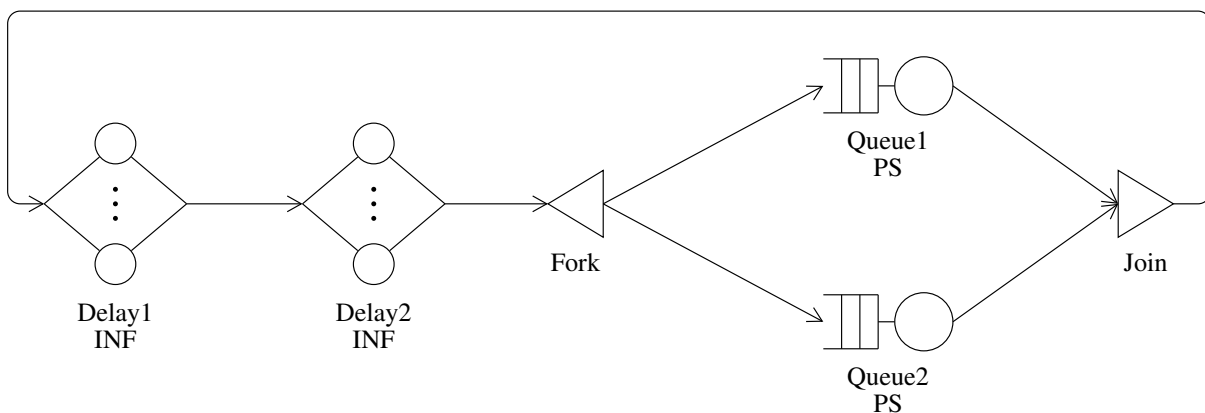


Figure 2.50: Topology of `fj_cs_prefork`: a closed network over 4 queueing stations with a fork-join block.

Nodes	6: Queue $\times$ 2, Delay $\times$ 2, Fork, Join
Classes	2: class1 (closed, $N = 10$), class2 (closed, $N = 10$)
Scheduling	Delay1: INF; Delay2: INF; Queue1: PS; Queue2: PS
Service	Delay1 – class1: Exp(0.5); class2: Exp(0.5) Delay2 – class1: Disabled; class2: Exp(2) Queue1 – class1: Exp(1); class2: Disabled Queue2 – class1: Exp(1); class2: Disabled
Features	drop rule {'class1': 'waitingQueue'} at Delay2; drop rule {'class2': 'waitingQueue'} at Queue1; drop rule {'class2': 'waitingQueue'} at Queue2; class switching on 5 links
Solvers	JMT, MVA

Table 2.50: Features of fj_cs_prefork.

The example is built and solved as follows.

```
void fj_cs_prefork() {
    Net m = fj_cs_prefork_model();
    section("JMT");
    print_avg(m.get_struct(), jmt_avg(m, sim_opts(23000)));
    section("MVA");
    print_avg(m.get_struct(), mva_run(m));
}
```

Running this edition prints

SOLVER: JMT						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Delay1	class1	1.9083	1.9083	2.0226	2.0226	0.9777
0.97739						
Delay2	class2	0.48212	0.48212	0.49535	0.49535	0.97739
0.97737						
Queue1	class1	12.772	0.97594	12.462	12.462	0.97737
0.97823						
Queue2	class1	8.7839	0.94243	8.9988	8.9988	0.97737
0.97343						
Join	class1	13.632	0	7.0837	7.0837	1.92
0.9777						
SOLVER: MVA						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Delay1	class1	1.8659	1.8659	2	2	0.93293
0.93293						
Delay2	class2	0.46646	0.46646	0.5	0.5	0.93293
0.93293						
Queue1	class1	11.87	0.93293	12.723	12.723	0.93293
0.93293						
Queue2	class1	11.87	0.93293	12.723	12.723	0.93293
0.93293						
Join	class1	11.87	0	6.3615	6.3615	1.8659
0.93293						

The rows repeat for each of the 2 solvers the script runs (JMT, MVA), which is the point of the example: the same model read by methods of different cost and different exactness.

2.6.9 fj_deep_nesting

Several nesting levels, the stress case for the join bookkeeping.

Nodes	9: Queue $\times$ 4, Delay, Fork $\times$ 2, Join $\times$ 2
Classes	1: class1 (closed, $N = 1$)
Scheduling	Delay1: INF; Queue1: FCFS; Queue2: FCFS; Queue3: FCFS; Queue4: FCFS
Service	Delay1 – class1: Exp(0.5) Queue1 – class1: Exp(1) Queue2 – class1: Exp(1) Queue3 – class1: Exp(2) Queue4 – class1: Exp(2)
Solvers	JMT, MVA

Table 2.51: Features of fj_deep_nesting.

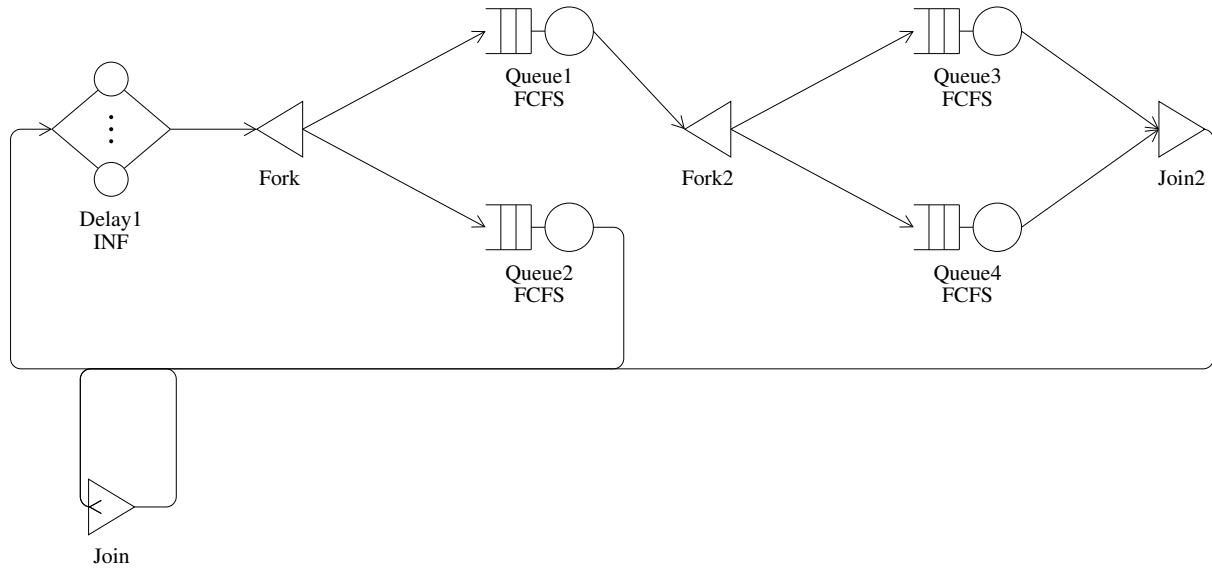


Figure 2.51: Topology of `fj_deep_nesting`: a closed network over 5 queueing stations with a fork-join block.

The example is built and solved as follows.

```

void fj_deep_nesting() {
    Net m = fj_deep_nesting_model();
    section("JMT");
    print_avg(m.get_struct(), jmt_avg(m, sim_opts(23000)));
    section("MVA");
    print_avg(m.get_struct(), mva_run(m));
}

```

Running this edition prints

SOLVER: JMT						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Delay1	class1	0.50301	0.50301	1.9945	1.9945	0.2517
Queue1	class1	0.2479	0.2479	1.0192	1.0192	0.24907
Queue2	class1	0.24341	0.24341	0.95781	0.95781	0.24907
Join	class1	0.30974	0	0.62676	0.62676	0.50588
Queue3	class1	0.12468	0.12468	0.50511	0.50511	0.24906
Queue4	class1	0.12783	0.12783	0.50431	0.50431	0.24906
Join2	class1	0.12602	0	0.24695	0.24695	0.50768
SOLVER: MVA						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Delay1	class1	0.45192	0.45192	2	2	0.22596
Queue1	class1	0.27306	0.22596	1.2084	1.2084	0.22596
Queue2	class1	0.27306	0.22596	1.2084	1.2084	0.22596
Join	class1	0.39228	0	0.86802	0.86802	0.45192
Queue3	class1	0.12639	0.11298	0.55933	0.55933	0.22596
Queue4	class1	0.12639	0.11298	0.55933	0.55933	0.22596
Join2	class1	0.12639	0	0.22373	0.22373	0.45192

The rows repeat for each of the 2 solvers the script runs (JMT, MVA), which is the point of the example: the same model read by methods of different cost and different exactness.

2.6.10 fj_delays

Delay stations inside the branches.

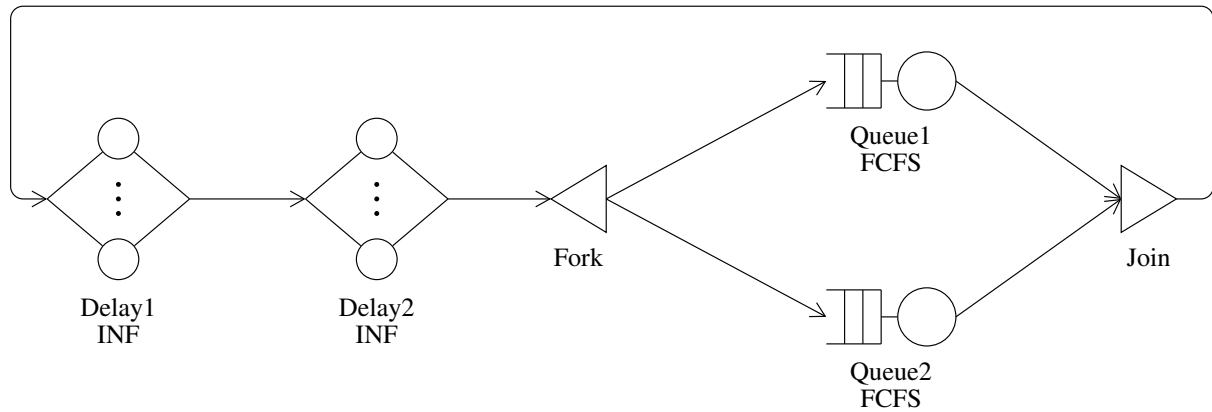


Figure 2.52: Topology of fj_delays: a closed network over 4 queueing stations with a fork-join block.

Nodes	6: Queue $\times$ 2, Delay $\times$ 2, Fork, Join
Classes	1: class1 (closed, $N = 10$)
Scheduling	Delay1: INF; Delay2: INF; Queue1: FCFS; Queue2: FCFS
Service	Delay1 – class1: Exp(0.5) Delay2 – class1: Exp(2) Queue1 – class1: Exp(1) Queue2 – class1: Exp(1)
Solvers	JMT, MVA

Table 2.52: Features of fj_delays.

The example is built and solved as follows.

```

void fj_delays() {
    Net m = fj_delays_model();
    section("JMT");
    print_avg(m.get_struct(), jmt_avg(m, sim_opts(23000)));
    section("MVA");
    print_avg(m.get_struct(), mva_run(m));
}

```

Running this edition prints

SOLVER: JMT						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Delay1	class1	1.9728	1.9728	2.035	2.035	0.96734
0.95415						
Delay2	class1	0.48219	0.48219	0.49433	0.49433	0.95415
0.95412						
Queue1	class1	6.0429	0.95704	6.081	6.081	0.95412
0.96494						
Queue2	class1	5.1463	0.94515	5.7192	5.7192	0.95412
0.96616						
Join	class1	3.9749	0	2.0321	2.0321	1.9436
0.96744						
SOLVER: MVA						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Delay1	class1	1.7184	1.7184	2	2	0.85919
0.85919						

Delay2	class1	0.4296	0.4296	0.5	0.5	0.85919
0.85919						
Queue1	class1	5.3091	0.85919	6.1792	6.1792	0.85919
0.85919						
Queue2	class1	5.3091	0.85919	6.1792	6.1792	0.85919
0.85919						
Join	class1	5.3091	0	3.0896	3.0896	1.7184
0.85919						

The rows repeat for each of the 2 solvers the script runs (JMT, MVA), which is the point of the example: the same model read by methods of different cost and different exactness.

2.6.11 fj_mixed_openclosed

Open and closed classes through one fork-join block, with a class disabled on a branch.

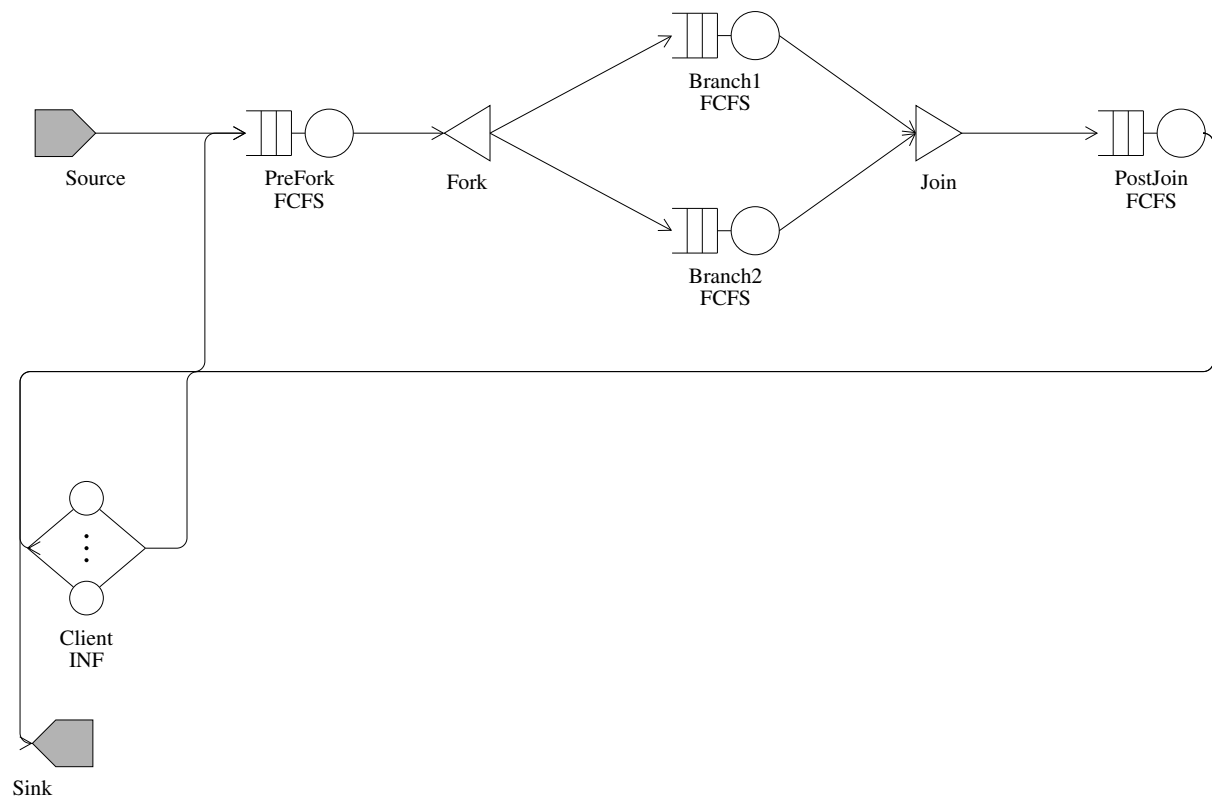


Figure 2.53: Topology of `fj_mixed_openclosed`: an open network over 5 queueing stations with a fork-join block.

Nodes	9: Source, Queue $\times$ 4, Delay, Fork, Join, Sink
Classes	2: Open (open), Closed (closed, $N = 1$)
Scheduling	Client: INF; PreFork: FCFS; Branch1: FCFS; Branch2: FCFS; PostJoin: FCFS
Arrivals	Source – Open: Exp(0.1); Closed: Disabled
Service	Client – Open: Disabled; Closed: Exp(1) PreFork – Open: Exp(5); Closed: Exp(5) Branch1 – Open: Exp(3.333); Closed: Exp(3.333) Branch2 – Open: Exp(2.5); Closed: Exp(2.5) PostJoin – Open: Exp(10); Closed: Exp(10)
Features	class switching on 2 links
Solvers	MVA, JMT

Table 2.53: Features of `fj_mixed_openclosed`.

The example is built and solved as follows.


```
void fj_mixed_openclosed() {
    Net m = fj_mixed_openclosed_model();
    section("MVA");
    print_avg(m.get_struct(), mva_run(m));
    section("JMT");
    print_avg(m.get_struct(), jmt_avg(m, sim_opts(23000, 200000)));
}
```

Running this edition prints

```
SOLVER: MVA
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
Tput
Source      Open          0          0          0          0          0
0.1
Client      Closed        0.51751    0.51751    1          1          0.51751
0.51751
PreFork      Open          0.022568    0.02      0.22568    0.22568    0.1
0.1
PreFork      Closed        0.10584    0.1035    0.20451    0.20451    0.51751
0.51751
Branch1      Open          0.03657    0.03      0.3657     0.3657     0.1
0.1
Branch1      Closed        0.18244    0.15525    0.35253    0.35253    0.51751
0.51751
Branch2      Open          0.052445    0.04      0.52445    0.52445    0.1
0.1
Branch2      Closed        0.25868    0.207     0.49985    0.49985    0.51751
0.51751
PostJoin     Open          0.010629    0.01      0.10629    0.10629    0.1
0.1
PostJoin     Closed        0.052301    0.051751    0.10106    0.10106    0.51751
0.51751
Join         Open          0.045923    0          0.22961    0.22961    0.2
0.1
Join         Closed        0.22715    0          0.21946    0.21946    1.035
0.51751

SOLVER: JMT
... (5 captured-output lines omitted; rerun the source example for the full output)

Branch1      Open          0.035936    0.029862    0.36021    0.36021    0.10008
0.10008
Branch1      Closed        0.16655    0.16185    0.30807    0.30807    0.54074
0.54099
Branch2      Open          0.05106    0.039888    0.50925    0.50925    0.10008
0.10008
Branch2      Closed        0.22565    0.21675    0.41709    0.41709    0.54074
0.54099
PostJoin     Open          0.010513    0.0099568    0.10573    0.10573    0.10008
0.10008
PostJoin     Closed        0.054457    0.05398    0.10078    0.10078    0.54099
0.54099
Join         Open          0.042761    0          0.21454    0.21454    0.19947
0.10008
Join         Closed        0.19875    0          0.18427    0.18427    1.0827
0.54099
```

The rows repeat for each of the 2 solvers the script runs (MVA, JMT), which is the point of the example: the same model read by methods of different cost and different exactness.

2.6.12 fj_nojoin

A fork with no join, so the siblings leave through the sink independently.
The example is built and solved as follows.

```
Net m = fj_nojoin_model();
section("MVA");
print_avg(m.get_struct(), mva_run(m));
section("JMT");
print_avg(m.get_struct(), jmt_avg(m, sim_opts(23000)));
```

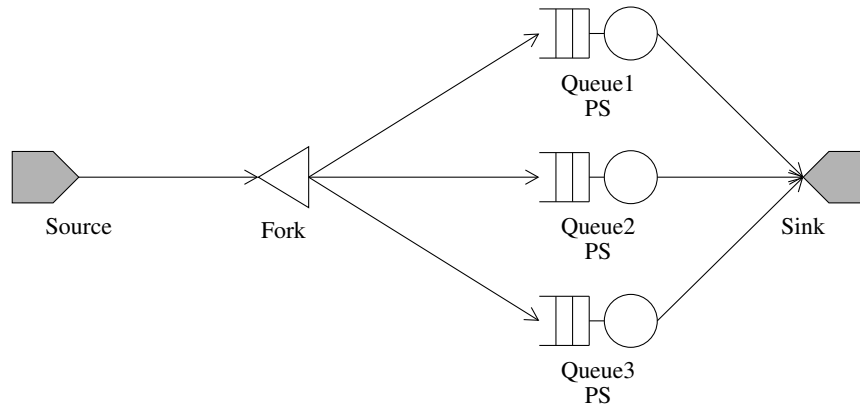


Figure 2.54: Topology of `fj_nojoin`: an open network over 3 queueing stations with a fork-join block.

Nodes	6: Source, Queue $\times$ 3, Fork, Sink		
Classes	1: class1 (open)		
Scheduling	Queue1: PS; Queue2: PS; Queue3: PS		
Arrivals	Source – class1: Exp(0.5)		
Service	Queue1 – class1: Exp(1)	Queue2 – class1: Exp(2)	Queue3 – class1: Exp(3)
Solvers	MVA, JMT		

Table 2.54: Features of `fj_nojoin`.

Running this edition prints

SOLVER: MVA						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Source	class1	0	0	0	0	0
0.5						
Queue1	class1	1	0.5	2	2	0.5
0.5						
Queue2	class1	0.33314	0.25	0.66629	0.66629	0.5
0.5						
Queue3	class1	0.19913	0.16667	0.39826	0.39826	0.5
0.5						
SOLVER: JMT						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Source	class1	0	0	0	0	0
0.5062						
Queue1	class1	0.97824	0.49952	1.9229	1.9229	0.5062
0.50103						
Queue2	class1	0.32722	0.24452	0.6673	0.6673	0.5062
0.50398						
Queue3	class1	0.2038	0.16807	0.41146	0.41146	0.5062
0.50621						

The rows repeat for each of the 2 solvers the script runs (MVA, JMT), which is the point of the example: the same model read by methods of different cost and different exactness.

2.6.13 `fj_route_overlap`

Branches that share a station, so their visits overlap.

The example is built and solved as follows.

```
void fj_route_overlap() {
    Net m = fj_route_overlap_model();
    section("JMT");
    print_avg(m.get_struct(), jmt_avg(m, sim_opts(23000)));
}
```

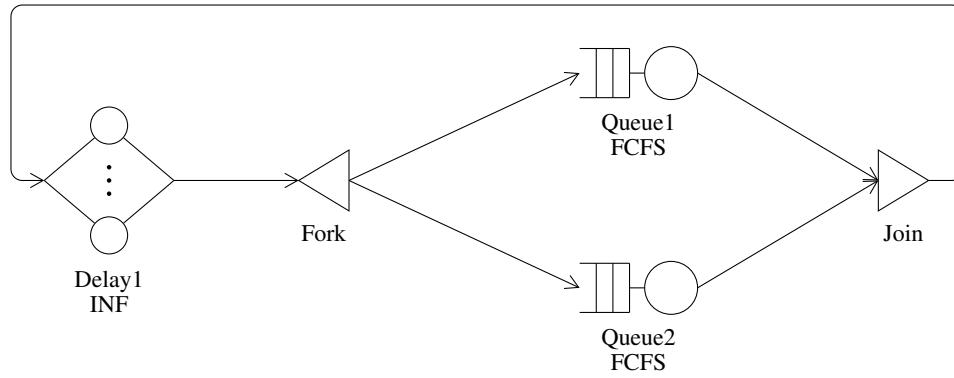


Figure 2.55: Topology of `fj_route_overlap`: a closed network over 3 queueing stations with a fork-join block.

Nodes	5: Queue $\times$ 2, Delay, Fork, Join		
Classes	2: class1 (closed, $N = 10$), class2 (closed, $N = 10$)		
Scheduling	Delay1: INF; Queue1: FCFS; Queue2: FCFS		
Service	Delay1 – class1: Exp(0.5); class2: Exp(0.2)	Queue1 – class1: Exp(1); class2: Exp(1)	Queue2 – class1: Exp(2); class2: Exp(2)
Solvers	JMT, MVA		

Table 2.55: Features of `fj_route_overlap`.

```

section("MVA");
print_avg(m.get_struct(), mva_run(m));

```

Running this edition prints

SOLVER: JMT						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Delay1	class1	1.0434	1.0434	1.9568	1.9568	0.53561
	class2	2.2482	2.2482	4.9237	4.9237	0.46316
Queue1	class1	8.9566	0.53833	16.567	16.567	0.53361
	class2	7.7518	0.45808	16.743	16.743	0.46313
Queue2	class1	0.52817	0.27261	0.99797	0.99797	0.53361
	class2	0.47122	0.22608	1.0143	1.0143	0.46313
Join	class1	8.4186	0	7.8315	7.8315	1.0916
	class2	7.2339	0	7.7245	7.7245	0.91758
SOLVER: MVA						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Delay1	class1	1.0241	1.0241	2	2	0.51206
	class2	2.215	2.215	5	5	0.443
Queue1	class1	9.0096	0.51206	17.595	17.595	0.51206
	class2	7.8075	0.443	17.624	17.624	0.443
Queue2	class1	0.48432	0.25603	0.94582	0.94582	0.51206
	class2	0.41936	0.2215	0.94663	0.94663	0.443
Join	class1	8.5747	0	8.3727	8.3727	1.0241
	class2	7.4309	0	8.387	8.387	0.886

0.443

The rows repeat for each of the 2 solvers the script runs (JMT, MVA), which is the point of the example: the same model read by methods of different cost and different exactness.

2.6.14 fj_serialfjs_closed

The closed version.

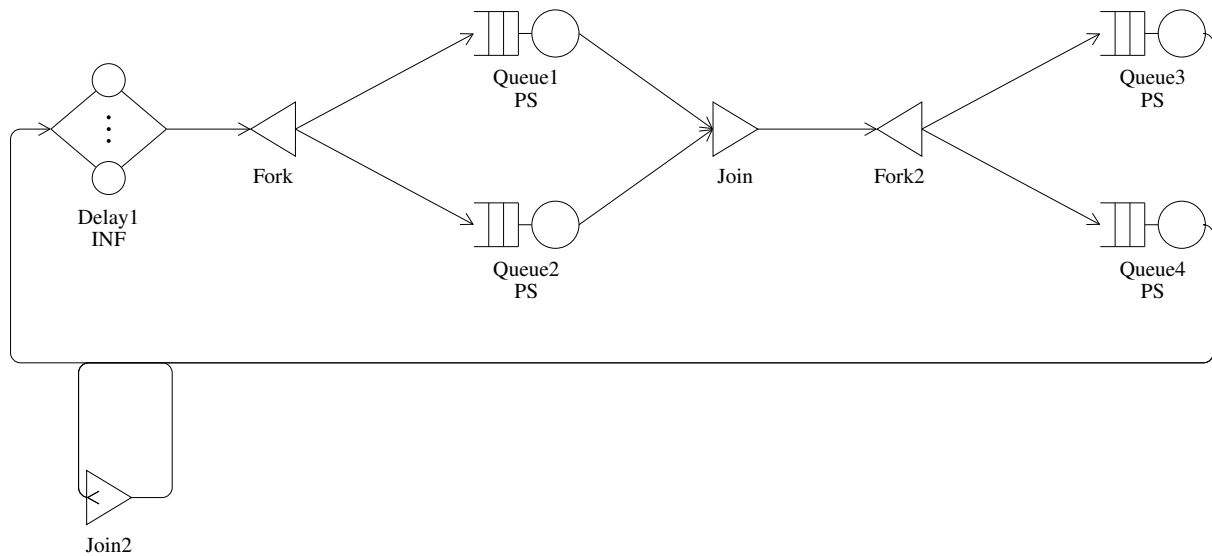


Figure 2.56: Topology of fj_serialfjs_closed: a closed network over 5 queueing stations with a fork-join block.

Nodes	9: Queue $\times$ 4, Delay, Fork $\times$ 2, Join $\times$ 2
Classes	1: class1 (closed, $N = 10$)
Scheduling	Delay1: INF; Queue1: PS; Queue2: PS; Queue3: PS; Queue4: PS
Service	Delay1 – class1: Exp(0.5) Queue1 – class1: Exp(1) Queue2 – class1: Exp(1) Queue3 – class1: Exp(1) Queue4 – class1: Exp(1)
Solvers	JMT, MVA

Table 2.56: Features of fj_serialfjs_closed.

The example is built and solved as follows.

```
void fj_serialfjs_closed() {
    Net m = fj_serialfjs_closed_model();
    section("JMT");
    print_avg(m.get_struct(), jmt_avg(m, sim_opts(23000)));
    section("MVA");
    print_avg(m.get_struct(), mva_run(m));
}
```

Running this edition prints

SOLVER: JMT	Station	JobClass	QLen	Util	RespT	ResidT	ArvR
	Tput						
	Delay1	class1	1.5478	1.5478	2.0184	2.0184	0.80819
	0.81366						
	Queue1	class1	2.6984	0.81744	3.4341	3.4341	0.81366
	0.80809						
	Queue2	class1	2.8035	0.80343	3.4227	3.4227	0.81366
	0.81267						

Join	class1	3.1856	0	1.9318	1.9318	1.588
0.80888						
Queue3	class1	2.5598	0.80584	3.1944	3.1944	0.80888
0.80815						
Queue4	class1	2.6804	0.80079	3.3994	3.3994	0.80888
0.80952						
Join2	class1	2.8828	0	1.7538	1.7538	1.6209
0.808						
SOLVER: MVA						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Delay1	class1	1.5101	1.5101	2	2	0.75503
0.75503						
Queue1	class1	2.8654	0.75503	3.7951	3.7951	0.75503
0.75503						
Queue2	class1	2.8654	0.75503	3.7951	3.7951	0.75503
0.75503						
Join	class1	2.8654	0	1.8975	1.8975	1.5101
0.75503						
Queue3	class1	2.8654	0.75503	3.7951	3.7951	0.75503
0.75503						
Queue4	class1	2.8654	0.75503	3.7951	3.7951	0.75503
0.75503						
Join2	class1	2.8654	0	1.8975	1.8975	1.5101
0.75503						

The rows repeat for each of the 2 solvers the script runs (JMT, MVA), which is the point of the example: the same model read by methods of different cost and different exactness.

2.6.15 fj_serialfjs_open

A chain of fork-join blocks, open.

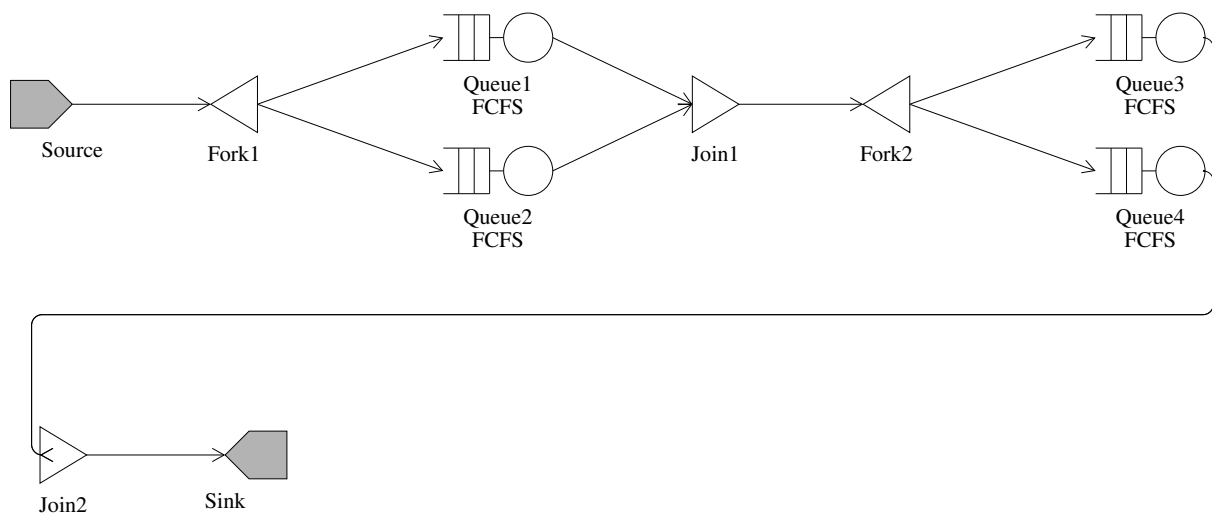


Figure 2.57: Topology of fj_serialfjs_open: an open network over 4 queueing stations with a fork-join block.

Nodes	10: Source, Queue $\times$ 4, Fork $\times$ 2, Join $\times$ 2, Sink
Classes	1: class1 (open)
Scheduling	Queue1: FCFS; Queue2: FCFS; Queue3: FCFS; Queue4: FCFS
Arrivals	Source – class1: Exp(0.4)
Service	Queue1 – class1: Exp(1) Queue2 – class1: Exp(1) Queue3 – class1: Exp(1) Queue4 – class1: Exp(1)
Solvers	JMT, MVA

Table 2.57: Features of fj_serialfjs_open.

The example is built and solved as follows.

```
void fj_serialfjs_open() {
    Net m = fj_serialfjs_open_model();
    section("JMT");
    print_avg(m.get_struct(), jmt_avg(m, sim_opts(23000)));
    section("MVA");
    print_avg(m.get_struct(), mva_run(m));
}
```

Running this edition prints

SOLVER: JMT						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Source	class1	0	0	0	0	0
0.40474						
Queue1	class1	0.68628	0.40101	1.6675	1.6675	0.40474
0.4092						
Queue2	class1	0.69566	0.40012	1.7616	1.7616	0.40474
0.40471						
Join1	class1	0.59155	0	0.76312	0.76312	0.83292
0.40867						
Queue3	class1	0.68976	0.41292	1.6883	1.6883	0.40867
0.40872						
Queue4	class1	0.6731	0.41363	1.6489	1.6489	0.40867
0.41236						
Join2	class1	0.61305	0	0.7252	0.7252	0.83343
0.41225						
SOLVER: MVA						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Source	class1	0	0	0	0	0
0.4						
Queue1	class1	0.66666	0.4	1.6667	1.6667	0.4
0.4						
Queue2	class1	0.66666	0.4	1.6667	1.6667	0.4
0.4						
Join1	class1	0.66666	0	0.83333	0.83333	0.8
0.4						
Queue3	class1	0.66666	0.4	1.6667	1.6667	0.4
0.4						
Queue4	class1	0.66666	0.4	1.6667	1.6667	0.4
0.4						
Join2	class1	0.66666	0	0.83333	0.83333	0.8
0.4						

The rows repeat for each of the 2 solvers the script runs (JMT, MVA), which is the point of the example: the same model read by methods of different cost and different exactness.

2.6.16 fj_threebranches

Three parallel branches rather than two.

Nodes	6: Queue $\times$ 3, Delay, Fork, Join
Classes	2: class1 (closed, $N = 10$), class2 (closed, $N = 10$)
Scheduling	Delay1: INF; Queue1: PS; Queue2: PS; Queue3: PS
Service	Delay1 – class1: Exp(0.5); class2: Exp(0.8) Queue1 – class1: Exp(1.5); class2: Exp(2.8) Queue2 – class1: Exp(1.1); class2: Exp(3) Queue3 – class1: Exp(2.5); class2: Exp(1)
Solvers	MVA

Table 2.58: Features of fj_threebranches.

The example is built and solved as follows.

```
void fj_threebranches() {
    Net m = fj_threebranches_model();
    section("MVA");
    print_avg(m.get_struct(), mva_run(m));
}
```

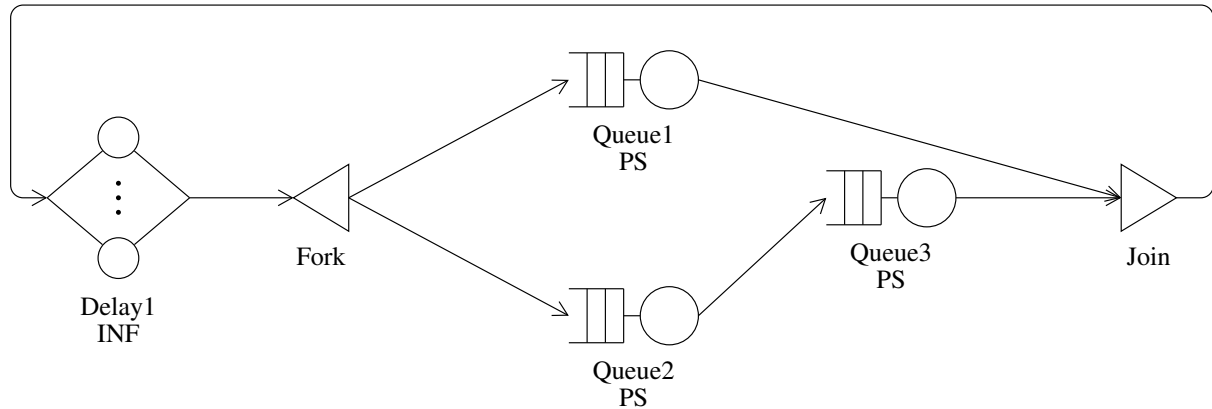


Figure 2.58: Topology of `fj_threebranches`: a closed network over 4 queueing stations with a fork-join block.

Running this edition prints

SOLVER: MVA						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Delay1	class1	1.4591	1.4591	2	2	0.72955
Delay1	class2	0.79755	0.79755	1.25	1.25	0.63804
Queue1	class1	1.6413	0.48637	2.2497	2.2497	0.72955
Queue1	class2	0.77382	0.22787	1.2128	1.2128	0.63804
Queue2	class1	4.8036	0.66323	6.5844	6.5844	0.72955
Queue2	class2	1.5573	0.21268	2.4407	2.4407	0.63804
Queue3	class1	3.529	0.29182	4.8372	4.8372	0.72955
Queue3	class2	7.6515	0.63804	11.992	11.992	0.63804
Join	class1	7.2315	0	4.9561	4.9561	1.4591
Join	class2	8.555	0	6.7041	6.7041	1.2761

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by MVA.

2.6.17 `fj_tiny_closed`

The smallest closed fork-join model, small enough for an exact CTMC solve.

Nodes	5: Queue $\times$ 2, Delay, Fork, Join
Classes	1: Class1 (closed, $N = 1$)
Scheduling	Delay1: INF; Queue1: FCFS; Queue2: FCFS
Service	Delay1 – Class1: Exp(1) Queue1 – Class1: Exp(2) Queue2 – Class1: Exp(3)
Solvers	CTMC, SSA, JMT

Table 2.59: Features of `fj_tiny_closed`.

The example is built and solved as follows.

```

Net m = fj_tiny_closed_model();
section("CTMC");
ctmc::CtmcOptions copt;
print_avg(m.get_struct(), ctmc::solver_ctmc_run_analyzer(m.get_struct(), copt));

```

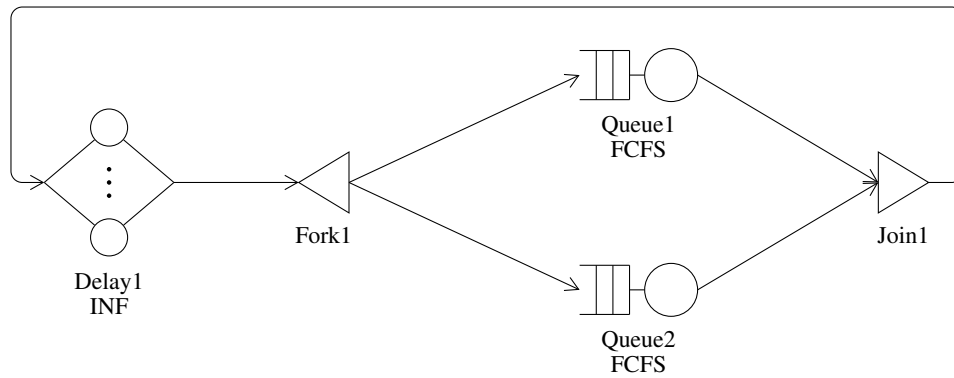


Figure 2.59: Topology of `fj_tiny_closed`: a closed network over 3 queueing stations with a fork-join block.

```
section("SSA");
ssa::SsaOptions sopt;
sopt.seed = 23000;
sopt.samples = 50000;
print_avg_sim(m.get_struct(), ssa::solver_ssa(m.get_struct(), sopt));

section("JMT");
print_avg(m.get_struct(), jmt_avg(m, sim_opts(23000)));
```

Running this edition prints

SOLVER: CTMC						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Delay1	Class1	0.61224	0.61224	1	1	0.61224
Queue1	Class1	0.30612	0.30612	0.5	0.5	0.61224
Queue2	Class1	0.20408	0.20408	0.33333	0.33333	0.61224
Join1	Class1	0.26531	0	0.21667	0.21667	1.2245
SOLVER: SSA						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Delay1	Class1	0.60986	0.60986	1	1	0.62171
Queue1	Class1	0.30719	0.30759	0.5	0.5	0.60986
Queue2	Class1	0.20832	0.20506	0.33333	0.33333	0.60986
Join1	Class1	0.26476	0.26476	0.21363	0.21363	1.2394
SOLVER: JMT						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Delay1	Class1	0.61067	0.61067	0.98692	0.98692	0.61699
Queue1	Class1	0.31288	0.31288	0.49922	0.49922	0.61569
Queue2	Class1	0.2034	0.2034	0.33361	0.33361	0.61569
Join1	Class1	0.26125	0	0.21207	0.21207	1.2554

The rows repeat for each of the 3 solvers the script runs (CTMC, SSA, JMT), which is the point of the example: the same model read by methods of different cost and different exactness.

2.6.18 fj_twoclasses_forked

Two classes forked independently through the same block.

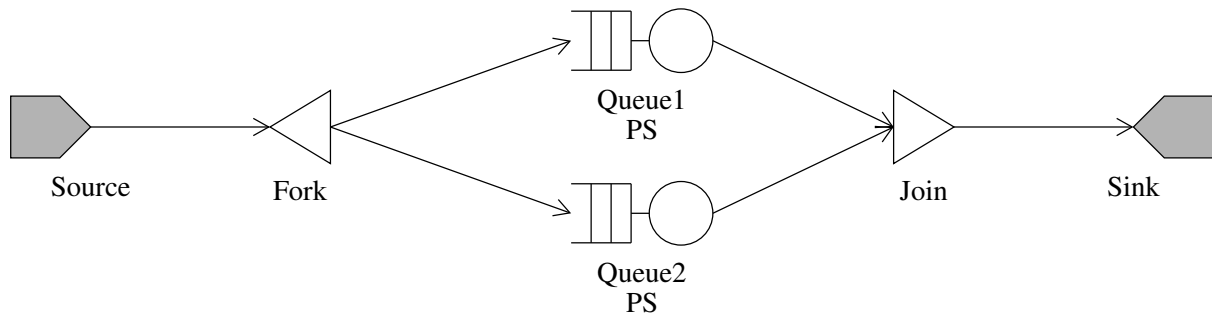


Figure 2.60: Topology of fj_twoclasses_forked: an open network over 2 queueing stations with a fork-join block.

Nodes	6: Source, Queue $\times$ 2, Fork, Join, Sink
Classes	2: class1 (open), class2 (open)
Scheduling	Queue1: PS; Queue2: PS
Arrivals	Source – class1: Exp(0.25); class2: Exp(0.25)
Service	Queue1 – class1: Exp(1); class2: Immediate Queue2 – class1: Exp(0.75); class2: Exp(2)
Solvers	JMT, MVA

Table 2.60: Features of fj_twoclasses_forked.

The example is built and solved as follows.

```

void fj_twoclasses_forked() {
    Net m = fj_twoclasses_forked_model();
    section("JMT");
    print_avg(m.get_struct(), jmt_avg(m, sim_opts(23000)));
    section("MVA");
    print_avg(m.get_struct(), mva_run(m));
}
  
```

Running this edition prints

SOLVER: JMT						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Source	class1	0	0	0	0	0
0.25128						
Source	class2	0	0	0	0	0
0.25184						
Queue1	class1	1.4252	0.49231	2.9402	2.9402	0.50253
0.49004						
Queue1	class2	0	0	0	0	0.50531
0.50531						
Queue2	class1	8.7958	0.65567	17.406	17.406	0.50253
0.50863						
Queue2	class2	3.4512	0.25836	6.7395	6.7395	0.50531
0.50463						
Join	class1	14.468	0	14.673	14.673	0.9868
0.24763						
Join	class2	6.754	0	6.305	6.305	1.028
0.25154						
SOLVER: MVA						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Source	class1	0	0	0	0	0
0.25						
Source	class2	0	0	0	0	0
0.25						
Queue1	class1	1	0.5	2	2	0.25
0.5						

Queue1	class2	0	0	2e-08	0	0.25
0.5						
Queue2	class1	8	0.66667	16	16	0.25
0.5						
Queue2	class2	3	0.25	6	6	0.25
0.5						
Join	class1	15.082	0	15.082	15.082	1
0.25						
Join	class2	6	0	6	6	1
0.25						

The rows repeat for each of the 2 solvers the script runs (JMT, MVA), which is the point of the example: the same model read by methods of different cost and different exactness.

2.6.19 fj_variable_fanout

A forking level that is not one fixed number: a per-link vector, a random degree, or a branch that fires only part of the time.

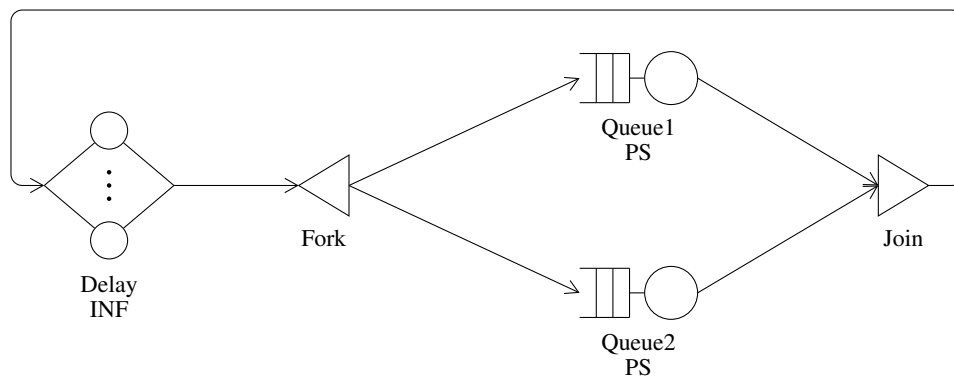


Figure 2.61: Topology of fj_variable_fanout: a closed network over 3 queueing stations with a fork-join block.

Nodes	5: Queue $\times$ 2, Delay, Fork, Join
Classes	1: class1 (closed, $N = 4$)
Scheduling	Delay: INF; Queue1: PS; Queue2: PS
Service	Delay – class1: Exp(1) Queue1 – class1: Exp(2) Queue2 – class1: Exp(2)
Solvers	JMT

Table 2.61: Features of fj_variable_fanout.

The example is built and solved as follows.

```
void fj_variable_fanout() {
    static const char* names[4] = {"fixed", "vector", "random", "prob"};
    for (int mode = 0; mode < 4; ++mode) {
        Net m = fj_variable_fanout_model(mode);
        section(std::string("mode = ") + names[mode] + " / JMT");
        print_avg(m.get_struct(), jmt_avg(m, sim_opts(23000)));
    }
}
```

Running this edition prints

SOLVER: mode = fixed / JMT						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Delay	class1	1.5341	1.5341	1.0133	1.0133	1.5446
1.5448						
Queue1	class1	1.5889	0.7622	1.0174	1.0174	1.5448
1.5455						

Queue2	class1	1.6127	0.77458	1.0392	1.0392	1.5448
1.547						
Join	class1	1.7718	0	0.56524	0.56524	3.0925
1.5608						
SOLVER: mode = vector / JMT						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Delay	class1	0.64773	0.64773	1.0005	1.0005	0.65557
0.65538						
Queue1	class1	0.44604	0.32456	0.67224	0.67224	0.65538
0.65424						
Queue2	class1	5.9736	0.99253	2.9998	2.9998	1.9632
1.9594						
Join	class1	6.893	0	2.539	2.539	2.6684
0.65666						
SOLVER: mode = random / JMT						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
... (3 captured-output lines omitted; rerun the source example for the full output)						
Join	class1	7.0001	0	2.0119	2.0119	3.3855
0.82206						
SOLVER: mode = prob / JMT						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Delay	class1	1.5688	1.5688	1.0401	1.0401	1.5284
1.5083						
Queue1	class1	1.6277	0.79454	1.1405	1.1405	1.5083
1.4271						
Queue2	class1	0.61004	0.43665	0.79765	0.79765	0.76875
0.7648						
Join	class1	0	0	0	0	2.1407
1.452						

2.7 Layered queueing networks

A layered queueing network (LQN) models software resources explicitly. A `Processor` hosts `Tasks`, each task exposes `Entry` points, and each entry runs an `Activity` graph that consumes host demand and makes synchronous or asynchronous calls to the entries of other tasks. A synchronous call blocks the caller until the reply arrives, which is what creates the layers: a task is simultaneously a server to its callers and a client of the tasks it calls.

The models are solved by `SolverLN`, which decomposes the layers into queueing submodels and iterates to a fixed point, and by the external `lqns` and `lqsim` tools. The same model is also readable from, and writable to, the `.lqn` XML interchange format.

2.7.1 lqn_basic

One reference task calling one server task, the smallest layered model.

Processors	2: P1 (ps, $m = 2$), P2 (ps, $m = 3$)
Tasks	3: T1 (ref, $m = 50$), T2 (fcfs, $m = 50$), T3 (fcfs, $m = 25$)
Entries	3: E1, E2, E3
Activities	3, host demands AS1: 0.1, AS2: 0.05, AS3: 0.02
Calls	2 (2 synchronous, 0 asynchronous)
Think times	T1: 2.0

Table 2.62: Features of `lqn_basic`.

The example is built and solved as follows.

```
void lqn_basic() { solve_ln(lqn_basic_model()); }
```

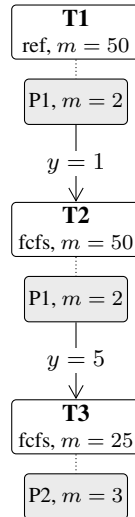


Figure 2.62: Call graph of `lqn_basic`: 3 tasks over 2 processors, drawn with the reference task on top and a called task below its caller; the shaded box under each task is the processor it runs on.

Running this edition prints

SolverLN(SolverMVA) arith=double type=approximate,deterministic layers=4 iterations=52 converged=1						
Node	RespT	ResidT	Tput	NodeType	QLen	Util
P1	NaN	NaN	NaN	Processor	NaN	0.997107
P2	NaN	NaN	NaN	Processor	NaN	0.443159
T1	NaN	1.08477	13.3	RefTask	23.4	0.664738
T2	NaN	0.556066	13.3	Task	8.72587	0.332369
T3	NaN	0.02	66.5	Task	1.32948	0.443159
E1	1.76088	NaN	13.3	Entry	23.4	0.664738
E2	0.656339	NaN	13.3	Entry	8.72587	0.332369
E3	0.02	NaN	66.5	Entry	1.32948	0.443159
AS1	1.76088	1.08477	13.3	Activity	23.4	0.664738
AS2	0.656339	0.556066	13.3	Activity	8.72587	0.332369
AS3	0.02	0.02	66.5	Activity	1.32948	0.443159

2.7.2 lqn_bpmn

A BPMN process translated into a layered model with forks, joins and immediate activities. The example is built and solved as follows.

```

const Lqn b = lqn_bpmn_model();
const LqnS l = b.build();
std::printf("This example illustrates the solution of a complex layered queueing network "
            "extracted from a BPMN model.\n");
solve_lqns(b.model());
solve_ln(l);

```

Running this edition prints

```

This example illustrates the solution of a complex layered queueing network extracted from a BPMN model.

```

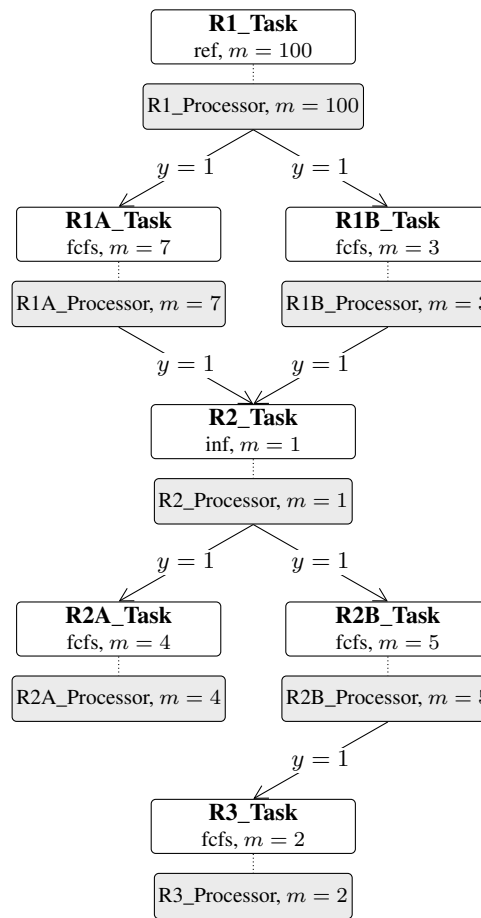


Figure 2.63: Call graph of 1qn_bpmn: 7 tasks over 7 processors, drawn with the reference task on top and a called task below its caller; the shaded box under each task is the processor it runs on.

Processors	7: R1_Processor (fcfs, $m = 100$), R2_Processor (inf, $m = 1$), R3_Processor (fcfs, $m = 2$), R1A_Processor (fcfs, $m = 7$), R1B_Processor (fcfs, $m = 3$), R2A_Processor (fcfs, $m = 4$), R2B_Processor (fcfs, $m = 5$)
Tasks	7: R1_Task (ref, $m = 100$), R2_Task (inf, $m = 1$), R3_Task (fcfs, $m = 2$), R1A_Task (fcfs, $m = 7$), R1B_Task (fcfs, $m = 3$), R2A_Task (fcfs, $m = 4$), R2B_Task (fcfs, $m = 5$)
Entries	16: R1_Ref_Entry, R2_Synch_A2_Entry, R2_Synch_A5_Entry, R3_Synch_A9_Entry, R1A_Synch_A1_Entry, R1A_Synch_A2_Entry, R1A_Synch_A3_Entry, R1B_Synch_A4_Entry, R1B_Synch_A5_Entry, R1B_Synch_A6_Entry, R2A_Synch_A7_Entry, R2A_Synch_A8_Entry, R2A_Synch_A11_Entry, R2B_Synch_A9_Entry, R2B_Synch_A10_Entry, R2B_Synch_A12_Entry
Activities	29, host demands A1_Empty: 0.0, A2_Empty: 0.0, A5_Empty: 0.0, A6_Empty: 0.0, A3_Empty: 0.0, A4_Empty: 0.0, E4_Empty: 0.0, A7_Empty: 0.0, ...
Calls	15 (15 synchronous, 0 asynchronous)
Think times	R1_Task: 20.0

Table 2.63: Features of 1qn_bpmn.

```

SOLVER: LQNS
N/A: the reference solves this model with the external lqns binary when it is installed; this port does
not carry it
SolverLN(SolverMVA) arith=double type=approximate,deterministic layers=13 iterations=52 converged=1
Node      RespT      ResidT      Tput      NodeType      QLen      Util
R1_Processor      NaN      NaN      NaN      Processor      NaN      0
R2_Processor      NaN      NaN      NaN      Processor      NaN      0
R3_Processor      NaN      NaN      NaN      Processor      NaN      0.504458
R1A_Processor      NaN      NaN      NaN      Processor      NaN      0.213958
R1B_Processor      NaN      NaN      NaN      Processor      NaN      0.470555
R2A_Processor      NaN      NaN      NaN      Processor      NaN      0.460057
R2B_Processor      NaN      NaN      NaN      Processor      NaN      0.363153
R1_Task      NaN      0      0.106962      RefTask      97.9      0
R2_Task      NaN      0      0.213924      Task      6.77907      0
R3_Task      NaN      10      0.100892      Task      1.00892      0.504458

... (41 captured-output lines omitted; rerun the source example for the full output)

A5_Res_Empty      13.0286      0      0.106944      Activity      1.39334      0
A7      6.81087      2.27029      0.102235      Activity      0.696307      0.153352
A8      9.08116      3.02705      0.102235      Activity      0.92841      0.20447
A11      4.54058      1.51353      0.102235      Activity      0.464205      0.102235
A9      4.36873      1.45641      0.100891      Activity      0.440765      0.0807127
A10      6.5531      2.18461      0.100891      Activity      0.661148      0.121069
A12      8.73747      2.91184      0.100857      Activity      0.881236      0.161371
A9_Res_Empty      11.9455      0      0.100891      Activity      1.20519      0

```

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by LQNS.

2.7.3 lqn_bpmn_trace

The BPMN translation with the activity trace printed.

Processors	7: R1_Processor (fcfs, $m = 100$), R2_Processor (inf, $m = 1$), R3_Processor (fcfs, $m = 2$), R1A_Processor (fcfs, $m = 7$), R1B_Processor (fcfs, $m = 3$), R2A_Processor (fcfs, $m = 4$), R2B_Processor (fcfs, $m = 5$)
Tasks	7: R1_Task (ref, $m = 100$), R2_Task (inf, $m = 1$), R3_Task (fcfs, $m = 2$), R1A_Task (fcfs, $m = 7$), R1B_Task (fcfs, $m = 3$), R2A_Task (fcfs, $m = 4$), R2B_Task (fcfs, $m = 5$)
Entries	16: R1_Ref_Entry, R2_Synch_A2_Entry, R2_Synch_A5_Entry, R3_Synch_A9_Entry, R1A_Synch_A1_Entry, R1A_Synch_A2_Entry, R1A_Synch_A3_Entry, R1B_Synch_A4_Entry, R1B_Synch_A5_Entry, R1B_Synch_A6_Entry, R2A_Synch_A7_Entry, R2A_Synch_A8_Entry, R2A_Synch_A11_Entry, R2B_Synch_A9_Entry, R2B_Synch_A10_Entry, R2B_Synch_A12_Entry
Activities	29, host demands A1_Empty: 0.0, A2_Empty: 0.0, A5_Empty: 0.0, A6_Empty: 0.0, A3_Empty: 0.0, A4_Empty: 0.0, E4_Empty: 0.0, A7_Empty: 0.0, ...
Calls	15 (15 synchronous, 0 asynchronous)
Think times	R1_Task: 20.0

Table 2.64: Features of lqn_bpmn_trace.

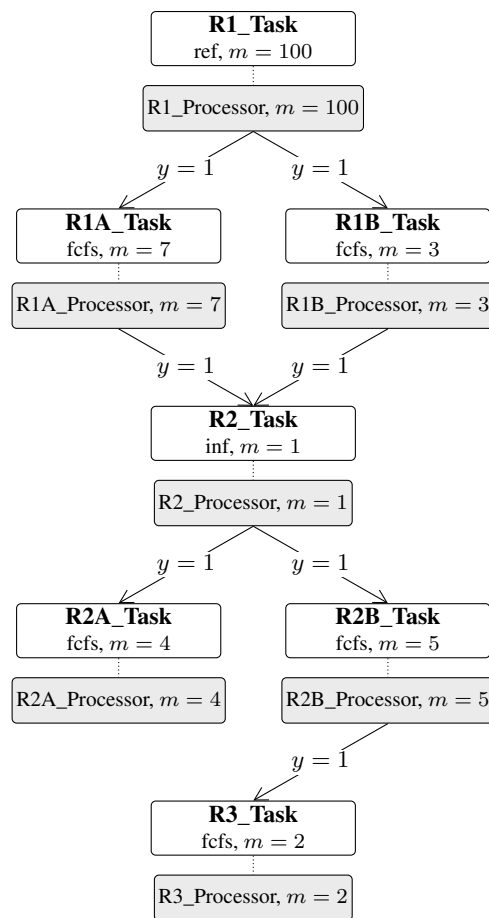


Figure 2.64: Call graph of `lqn_bpmn_trace`: 7 tasks over 7 processors, drawn with the reference task on top and a called task below its caller; the shaded box under each task is the processor it runs on.

The example is built and solved as follows.

```
note("=== Running LN solver with tracing ===");
const LqnS l = lqn_bpmn_model().build();
ln::LnOptions opt;
opt.iter_max = 10;
ln::SolverLN<double> s(l, opt);
const ln::LnSolution<double> sol = s.get_ensemble_avg();

const std::vector<std::vector<ln::LayerResult<double> > >& hist = s.iteration_results();
const std::size_t shown = std::min<std::size_t>(5, hist.size());
for (std::size_t it = 0; it < shown; ++it) {
    std::printf("=== Iteration %zu ===\n", it + 1);
    for (std::size_t e = 0; e < hist[it].size(); ++e) {
        const double tn = mat_max(hist[it][e].TN), qn = mat_max(hist[it][e].QN);
        if (tn > 1e-10 || qn > 1e-10)
            std::printf("  Layer %zu: max(TN)=%.4e, max(QN)=%.4e\n", e + 1, tn, qn);
    }
}

note("=== Final check: Getting AvgTable ===");
const char* tasks[7] = {"R:R1_Task", "T:R2_Task", "T:R3_Task", "T:R1A_Task",
                        "T:R1B_Task", "T:R2A_Task", "T:R2B_Task"};
for (int k = 0; k < 7; ++k) {
    const std::size_t i = idx_of(l, tasks[k]);
    std::printf("%s: Tput=%.6f\n", l.names[i].c_str(), ln_sanitize(sol.TN[i]));
}
```

Running this edition prints

```
=== Running LN solver with tracing ===
=== Iteration 1 ===
Layer 1: max(TN)=5.0000e+00, max(QN)=1.0000e+02
Layer 2: max(TN)=5.0000e+08, max(QN)=0.0000e+00
Layer 3: max(TN)=1.0000e+08, max(QN)=0.0000e+00
Layer 4: max(TN)=3.5000e+08, max(QN)=0.0000e+00
Layer 5: max(TN)=1.5000e+08, max(QN)=0.0000e+00
Layer 6: max(TN)=2.0000e+08, max(QN)=0.0000e+00
Layer 7: max(TN)=2.5000e+08, max(QN)=0.0000e+00
Layer 8: max(TN)=3.5000e+08, max(QN)=0.0000e+00
Layer 9: max(TN)=1.8692e+08, max(QN)=0.0000e+00
Layer 10: max(TN)=5.0000e+00, max(QN)=1.0000e+02
Layer 11: max(TN)=5.0000e+00, max(QN)=1.0000e+02
Layer 12: max(TN)=3.8760e+08, max(QN)=0.0000e+00
Layer 13: max(TN)=4.5042e+08, max(QN)=0.0000e+00
=== Iteration 2 ===

... (55 captured-output lines omitted; rerun the source example for the full output)

=== Final check: Getting AvgTable ===
R1_Task: Tput=0.103955
R2_Task: Tput=0.212614
R3_Task: Tput=0.099427
R1A_Task: Tput=0.268079
R1B_Task: Tput=0.255731
R2A_Task: Tput=0.301283
R2B_Task: Tput=0.300000
```

2.7.4 lqn_flatph

Method `flat.ph`: the squashed layering carrying the same composed phase-type encoding, one sub-model holding every processor and called task.

The example is built and solved as follows.

```
const LqnS l = lqn_flatph_model();
for (const std::string& m : {std::string("srvn.cs"), std::string("srvn.ph"),
                             std::string("flat.cs"), std::string("flat.ph")})
    solve_ln_labelled(l, m);
```

Running this edition prints

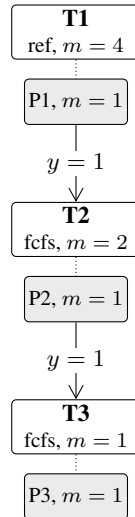


Figure 2.65: Call graph of `lqn_flatph`: 3 tasks over 3 processors, drawn with the reference task on top and a called task below its caller; the shaded box under each task is the processor it runs on.

Processors	3: P1 (ps, $m = 1$), P2 (ps, $m = 1$), P3 (ps, $m = 1$)
Tasks	3: T1 (ref, $m = 4$), T2 (fcfs, $m = 2$), T3 (fcfs, $m = 1$)
Entries	3: E1, E2, E3
Activities	3, host demands A1: 0.2, A2: 0.2, A3: 0.2
Calls	2 (2 synchronous, 0 asynchronous)
Think times	T1: 1.0

Table 2.65: Features of `lqn_flatph`.

```

--- method=svrn.cs (built as svrn.cs, 5 submodel(s))
SolverLN(SolverMVA) arith=double type=approximate,deterministic layers=5 iterations=52 converged=1
Node      RespT      ResidT      Tput      NodeType      QLen      Util
P1         NaN         NaN         NaN        Processor      NaN        0.436785
P2         NaN         NaN         NaN        Processor      NaN        0.436785
P3         NaN         NaN         NaN        Processor      NaN        0.436785
T1         NaN         0.283523    2.18393    RefTask        1.81607    0.436785
T2         NaN         0.245988    2.18393    Task           1.07444    0.436785
T3         NaN         0.2         2.18393    Task           0.436785    0.436785
E1         0.831563      NaN         2.18393    Entry          1.81607    0.436785
E2         0.491976      NaN         2.18393    Entry          1.07444    0.436785
E3         0.2          NaN         2.18393    Entry          0.436785    0.436785
A1         0.831563      0.283523    2.18393    Activity        1.81607    0.436785
A2         0.491976      0.245988    2.18393    Activity        1.07444    0.436785
A3         0.2          0.2         2.18393    Activity        0.436785    0.436785

... (39 captured-output lines omitted; rerun the source example for the full output)

T2         NaN         0.247509    2.12644    Task           1.06646    0.425288
T3         NaN         0.2         2.12667    Task           0.425333    0.425333
E1         NaN         0.2         2.12667    Entry          1.87356    0.425288
  
```

E2	0.881077	NaN	2.12644			
				Entry	1.06646	0.425288
E3	0.501524	NaN	2.12644			
				Entry	0.425333	0.425333
A1	0.2	NaN	2.12667			
				Activity	1.87356	0.425288
A2	0.881077	0.284068	2.12644			
				Activity	1.06646	0.425288
A3	0.501524	0.247509	2.12644			
				Activity	0.425333	0.425333
	0.2	0.2	2.12667			

2.7.5 lqn_fork_open_arrival

An AND-fork under an open arrival stream.

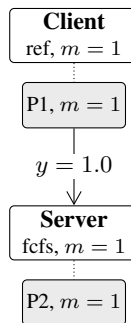


Figure 2.66: Call graph of `lqn_fork_open_arrival`: 2 tasks over 2 processors, drawn with the reference task on top and a called task below its caller; the shaded box under each task is the processor it runs on.

Processors	2: P1 (inf, $m = 1$), P2 (inf, $m = 1$)
Tasks	2: Client (ref, $m = 1$), Server (fcfs, $m = 1$)
Entries	3: CE, SE, OE
Activities	9, host demands CA: 0.5, RA1: 0.2, RA2: 0.3, RA3: 0.4, RA4: 0.1, OA1: 0.2, OA2: 0.3, OA3: 0.4, ...
Calls	1 (1 synchronous, 0 asynchronous)
Think times	Client: 1.0

Table 2.66: Features of `lqn_fork_open_arrival`.

The example is built and solved as follows.

```

Lqn b;
b.processor("P1", std::numeric_limits<double>::infinity(), SchedStrategy::INF);
b.task("Client", 1, SchedStrategy::REF, "P1");
b.think_time("Client", Em(1.0));
b.entry("CE", "Client");
b.processor("P2", std::numeric_limits<double>::infinity(), SchedStrategy::INF);
b.task("Server", 1, SchedStrategy::FCFS, "P2");
b.think_time("Server", Immediate());
b.entry("SE", "Server");
b.entry("OE", "Server");
b.open_arrival("OE", Er(0.1));

b.activity("CA", Em(0.5), "Client");
b.bound_to("CA", "CE");
b.sync_call("CA", "SE", 1.0);

b.activity("RA1", Em(0.2), "Server");
b.bound_to("RA1", "SE");
b.activity("RA2", Em(0.3), "Server");

```

```

b.activity("RA3", Em(0.4), "Server");
b.activity("RA4", Em(0.1), "Server");
b.replies_to("RA4", "SE");
b.and_fork("RA1", {"RA2", "RA3"});
b.and_join({"RA2", "RA3"}, "RA4");

b.activity("OA1", Em(0.2), "Server");

... (7 source lines omitted; full implementation: cpp/examples/basic/)

try {
    solve_ln(1);
} catch (const std::exception& e) {
    std::printf("LN refuses this model: %s\n", e.what());
}

```

Running this edition prints

SolverLN(SolverMVA) arith=double type=approximate,deterministic layers=3 iterations=52 converged=1						
Node	RespT	ResidT	Tput	NodeType	Qlen	Util
P1				Processor	NaN	0.2
P2	NaN	NaN	NaN	Processor	NaN	0.554545
Client	NaN	NaN	NaN	RefTask	0.6	0.2
Server	NaN	0.5	0.4	Task	0.554545	0.554545
CE	NaN	1	0.554545	Entry	0.6	0.2
SE	1.5	NaN	0.4	Entry	0.443636	0.443636
OE	1	NaN	0.443636	Entry	0.110909	0.110909
CA	1	NaN	0.110909	Activity	0.6	0.2
RA1	1.5	0.5	0.4	Activity	0.0887273	0.0887273
RA2	0.2	0.16	0.443636	Activity	0.133091	0.133091
RA3	0.3	0.24	0.443636	Activity	0.177455	0.177455
RA4	0.4	0.32	0.443636	Activity	0.0443636	0.0443636
OA1	0.1	0.08	0.443636	Activity	0.0221818	0.0221818
OA2	0.2	0.04	0.110909	Activity	0.0332727	0.0332727
OA3	0.3	0.06	0.110909	Activity	0.0443636	0.0443636
OA4	0.4	0.08	0.110909	Activity	0.0110909	0.0110909
	0.1	0.02	0.110909			

2.7.6 lqn_init

A layered model whose SolverLN run is warm-started from the LQNS solution, and the elapsed time that initialization saves.

Nodes	2: Queue, Delay
Classes	1: T1 (closed, $N = 10$)
Scheduling	Clients: INF; P1: PS
Service	Clients – T1: Exp(0.008971) P1 – T1: APH

Table 2.67: Features of lqn_init.

The example is built and solved as follows.

```
std::printf("This example illustrates the initialization of LN using the output of LQNS.\n");
```

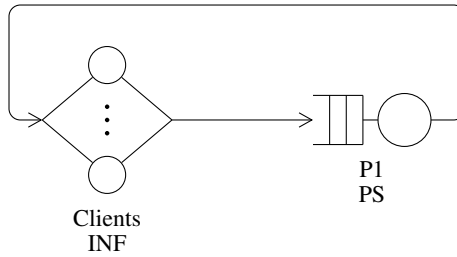


Figure 2.67: Topology of `lqn_init`: a closed network over 2 queueing stations.

```
const lqn::LqnModel<double> mdl = lqn::read_lqnx_model<double>(layered_file("lqn_serial.xml")
);
const LqnS l = lqn::lqn_finalize(mdl);

solve_lqns(mdl);

std::printf("\nSolve with LN without initialization:\n");
ln::LnOptions opt;
const auto t0 = std::chrono::steady_clock::now();
ln::SolverLN<double> s(l, opt);
const ln::LnSolution<double> sol = s.get_ensemble_avg();
const double elapsed = std::chrono::duration<double>(std::chrono::steady_clock::now() - t0)
    .count();
print_ln(l, sol, opt, s.nlayers());
std::printf("Time elapsed: %.3fs\n", elapsed);

std::printf("\nWe now obtain the CDF of response times:\n");
if (s.nlayers() < 3) {
    std::printf("Model ensemble not available or insufficient layers\n");
    return;
}
const qn::Layer<double>& L = s.layers()[2];
fluid::FluidOptions fopt;
const std::vector<std::vector<fluid::FluidPassage>> > RD =
    fluid::solver_fluid_cdf_respt<double>(L, fopt);
for (std::size_t i = 0; i < RD.size(); ++i)
    for (std::size_t c = 0; c < RD[i].size(); ++c) {
        if (RD[i][c].t.empty()) continue;
        const fluid::FluidPassage& p = RD[i][c];
        std::printf("RD[%s][%s]: %zu points, t in [%.6g, %.6g], F in [%.6g, %.6g]\n",
            L.stations[i].name.c_str(), L.classes[c].name.c_str(), p.t.size(),
            p.t.front(), p.t.back(), p.cdf.front(), p.cdf.back());
    }
}
```

Running this edition prints

This example illustrates the initialization of LN using the output of LQNS.

SOLVER: LQNS

N/A: the reference initializes LN from the external `lqns` binary's answer; this port carries neither `lqns` nor its output, so there is nothing to initialize from

Solve with LN without initialization:

SolverLN(SolverMVA) arith=double type=approximate,deterministic layers=3 iterations=52 converged=1

Node	RespT	ResidT	Tput	NodeType	Qlen	Util
P1	NaN	NaN	NaN	Processor	NaN	0.141226
P2	NaN	NaN	NaN	Processor	NaN	0.529599
T1	NaN	1.82889	0.0882665	RefTask	1.17335	0.141226
T2	NaN	6	0.0882665	Task	0.529599	0.529599
E1	13.3	NaN	0.0882665	Entry	1.17335	0.141226
E2	6	NaN	0.0882665	Entry	0.529599	0.529599

AS1	1.82889	1.82889	0.0882665	Activity	0.16143	0.141226
AS2	11.4644	0	0.0882665	Activity	1.01192	0
AS3	5	5	0.0882665	Activity	0.441333	0.441333
AS4	1	1	0.0882665	Activity	0.0882665	0.0882665
Time elapsed: 0.054s						
We now obtain the CDF of response times:						
RD[Clients][R:T1]: 17377 points, t in [0, 10182.9], F in [0, 1]						
RD[T:T2][R:T1]: 13843 points, t in [0, 10182.9], F in [0, 1]						

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by `LQNS`.

2.7.7 `lqn_jsq`

Join-the-shortest-queue dispatching between replicated tasks.

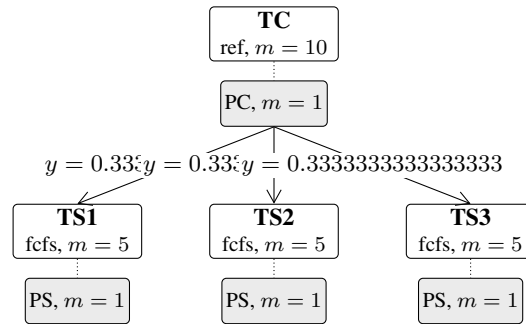


Figure 2.68: Call graph of `lqn_jsq`: 4 tasks over 2 processors, drawn with the reference task on top and a called task below its caller; the shaded box under each task is the processor it runs on.

Processors	2: PC (inf, $m = 1$), PS (ps, $m = 1$)
Tasks	4: TC (ref, $m = 10$), TS1 (fcfs, $m = 5$), TS2 (fcfs, $m = 5$), TS3 (fcfs, $m = 5$)
Entries	4: EC, ES1, ES2, ES3
Activities	4, host demands AC: 0.5, AS1: 1.0, AS2: 1.0, AS3: 1.0
Calls	3 (3 synchronous, 0 asynchronous)
Think times	TC: 5.0

Table 2.68: Features of `lqn_jsq`.

The example is built and solved as follows.

```
solve_ln(routed_group_model(true), routed_group_options());
```

Running this edition prints

SolverLN(SolverSSA) arith=double type=exact,deterministic layers=1 iterations=11 converged=1						
Node	RespT	ResidT	Tput	NodeType	QLen	Util
PC				Processor	NaN	0.5
	NaN	NaN	NaN			
PS				Processor	NaN	0.9348
	NaN	NaN	NaN			
TC				RefTask	5.14404	0.5
	NaN	0.502333	0.99265			
TS1				Task	1.54801	0.317936
	NaN	4.8	0.320381			
TS2				Task	1.45702	0.310026
	NaN	4.75088	0.306496			

TS3				Task	1.4596	0.306837
	NaN	4.765	0.306247			
EC				Entry	5.14404	0.5
	5.08286	NaN	1.01204			
ES1				Entry	1.54801	0.317936
	4.75401	NaN	0.325622			
ES2				Entry	1.45702	0.310026
	4.7494	NaN	0.306779			
ES3				Entry	1.4596	0.306837
	4.69437	NaN	0.310925			
AC				Activity	2.02291	0.5
	2.02382	0.502333	1			
AS1				Activity	1.64056	0.317936
	5.16002	4.8	0.317936			
AS2				Activity	1.56367	0.310026
	5.04365	4.75088	0.310026			
AS3				Activity	1.5848	0.306837
	5.16495	4.765	0.306837			

2.7.8 lqn_moment3

The third moment of the entry service time, and how it propagates up the layers.

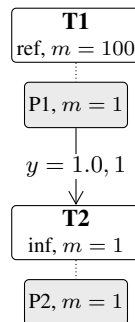


Figure 2.69: Call graph of `lqn_moment3`: 2 tasks over 2 processors, drawn with the reference task on top and a called task below its caller; the shaded box under each task is the processor it runs on.

Processors	2: P1 (ps, $m = 1$), P2 (ps, $m = 1$)
Tasks	2: T1 (ref, $m = 100$), T2 (inf, $m = 1$)
Entries	3: E1, E2, E3
Activities	5, host demands A1: 1.0, A20: 1.0, A21: 1.0, A22: 1.0, A3: 1.0
Calls	2 (2 synchronous, 0 asynchronous)
Think times	T1: 10.0
Solvers	FLD

Table 2.69: Features of `lqn_moment3`.

The example is built and solved as follows.

```

const LqnS l = lqn_twotasks_model().build();
solve_ln(l, nc_layers());

ln::LnOptions opt = nc_layers();
opt.method = "moment3";
try {
    solve_ln(l, opt);
} catch (const std::exception& e) {
    std::fprintf(stderr, "\nLN(moment3) failed: %s\n", e.what());
    solve_ln(l, nc_layers());
}

```

Running this edition prints

```

lsoda -- at t = 0.0350432 and step size h_ = 1.33679e-07, the
        corrector convergence failed repeatedly or
        with fabs(h_) = hmin
lsoda -- at t = 0.0350432 and step size h_ = 1.33679e-07, the
        corrector convergence failed repeatedly or
        with fabs(h_) = hmin
lsoda -- at t = 1935.59 and step size h_ = 0.778401, the
        corrector convergence failed repeatedly or
        with fabs(h_) = hmin
lsoda -- at t = 2915.04 and step size h_ = 0.00011676, the
        corrector convergence failed repeatedly or
        with fabs(h_) = hmin
lsoda -- at t = 3206.55 and step size h_ = 0.00012788, the
        corrector convergence failed repeatedly or
        with fabs(h_) = hmin
lsoda -- at t = 3570.93 and step size h_ = 1.39, the

... (578 captured-output lines omitted; rerun the source example for the full output)

E1          389.7          NaN          0.25          Entry          97.4          0.25
E2          285.6          NaN          0.25          Entry          71.4          0.75
E3          95.2          NaN          0.25          Entry          23.8          0.25
A1          390          1.32747          0.25          Activity          97.5          0.25
A20         97.2          48.6          0.25          Activity          24.3          0.25
A21         97.2          48.6          0.25          Activity          24.3          0.25
A22         97.2          48.6          0.25          Activity          24.3          0.25
A3          97.2          48.6          0.25          Activity          24.3          0.25

```

2.7.9 lqn_multi_solvers

One layered model solved by every layered solver, side by side.

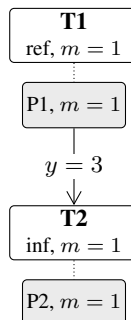


Figure 2.70: Call graph of `lqn_multi_solvers`: 2 tasks over 2 processors, drawn with the reference task on top and a called task below its caller; the shaded box under each task is the processor it runs on.

Processors	2: P1 (inf, $m = 1$), P2 (inf, $m = 1$)
Tasks	2: T1 (ref, $m = 1$), T2 (inf, $m = 1$)
Entries	2: E1, E2
Activities	2, host demands A1: 1.0, A2: 1.0
Calls	1 (1 synchronous, 0 asynchronous)
Think times	T1: 0.0001

Table 2.70: Features of `lqn_multi_solvers`.

The example is built and solved as follows.

```

Lqn b;
b.processor("P1", std::numeric_limits<double>::infinity(), SchedStrategy::INF);
b.task("T1", 1, SchedStrategy::REF, "P1");
b.entry("E1", "T1");
b.processor("P2", std::numeric_limits<double>::infinity(), SchedStrategy::INF);
b.task("T2", std::numeric_limits<double>::infinity(), SchedStrategy::INF, "P2");
b.entry("E2", "T2");
b.think_time("T1", lang::erlang_fit_mean_order<double>(0.0001, 2));
b.activity("A1", Er(1.0), "T1");
b.bound_to("A1", "E1");
b.sync_call("A1", "E2", 3.0);
b.activity("A2", lang::aph_fit_mean_scv<double>(1.0, 10.0), "T2");
b.bound_to("A2", "E2");
b.replies_to("A2", "E2");
const LqnS l = b.build();

solve_lqns(b.model());
// The reference also sets lnoptions.seed = 2300; it selects nothing here,
// since neither MVA nor NC layers draw random numbers.
solve_ln(l);
solve_ln(l, nc_layers());

```

Running this edition prints

```

SOLVER: LQNS
N/A: the reference's first solver is the external lqns binary, which this port does not carry
SolverLN(SolverMVA) arith=double type=approximate,deterministic layers=3 iterations=52 converged=1
Node      RespT      ResidT      Tput      NodeType      QLen      Util
P1         NaN         NaN         NaN        Processor      NaN        0.249994
P2         NaN         NaN         NaN        Processor      NaN        0.749981
T1         NaN         1          0.249994   RefTask        1          0.249994
T2         NaN         1          0.749981   Task           0.749981   0.749981
E1         4           NaN         0.249994   Entry          1          0.249994
E2         1           NaN         0.749981   Entry          0.749981   0.749981
A1         4           1          0.249994   Activity        1          0.249994
A2         1           1          0.749981   Activity        0.749981   0.749981
SolverLN(SolverNC) arith=double type=approximate,deterministic layers=3 iterations=52 converged=1
Node      RespT      ResidT      Tput      NodeType      QLen      Util
P1         NaN         NaN         NaN        Processor      NaN        0.249994
P2         NaN         NaN         NaN        Processor      NaN        0.749981
T1         NaN         1          0.249994   RefTask        1          0.249994
T2         NaN         1          0.749981   Task           0.749981   0.749981
E1         4           NaN         0.249994   Entry          1          0.249994
E2         1           NaN         0.749981   Entry          0.749981   0.749981
A1         4           1          0.249994   Activity        1          0.249994
A2         1           1          0.749981   Activity        0.749981   0.749981

```

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by LQNS.

2.7.10 lqn_ofbiz

The Apache OFBiz case study, read from its .lqnx file.
The example is built and solved as follows.

Processors	9: FrontEnd_CPU_Processor (ps, $m = 1$), USAGE_DELAY_Processor (inf, $m = 1$), UsageScenario_userType1_1_Processor (fcfs, $m = 1$), RequestHandler_HandlerIF_main_345_Processor (fcfs, $m = 1$), RequestHandler_HandlerIF_login_345_Processor (fcfs, $m = 1$), RequestHandler_HandlerIF_checkLogin_345_Processor (fcfs, $m = 1$), RequestHandler_HandlerIF_logout_345_Processor (fcfs, $m = 1$), UsageScenario_userType2_7_Processor (fcfs, $m = 1$), RequestHandler_HandlerIF_quickadd_345_Processor (fcfs, $m = 1$)
Tasks	9: FrontEnd_CPU_Task (inf, $m = 1$), USAGE_DELAY_Task (inf, $m = 1$), UsageScenario_userType1_1_Task (ref, $m = 10$), RequestHandler_HandlerIF_main_345_Task (inf, $m = 1$), RequestHandler_HandlerIF_login_345_Task (inf, $m = 1$), RequestHandler_HandlerIF_checkLogin_345_Task (inf, $m = 1$), RequestHandler_HandlerIF_logout_345_Task (inf, $m = 1$), UsageScenario_userType2_7_Task (ref, $m = 10$), RequestHandler_HandlerIF_quickadd_345_Task (inf, $m = 1$)
Entries	14: FrontEnd_CPU_Entry, InternalAction_main_Iu1-wMhoEeKON4DtRoKCMw_34_50_Entry, InternalAction_login_YgxRGw26EeSPwb7XgvxhWQ_34_50_Entry, InternalAction_CheckLogin_vHMZsMhoEeKON4DtRoKCMw_34_50_Entry, InternalAction_logout_j-2E4MhrEeKON4DtRoKCMw_34_50_Entry, InternalAction_addcart_5JEHQMhoEeKON4DtRoKCMw_34_50_Entry, USAGE_DELAY0_Entry, UsageScenario_userType1_1_Entry, RequestHandler_HandlerIF_main_345_Entry, RequestHandler_HandlerIF_login_345_Entry, RequestHandler_HandlerIF_checkLogin_345_Entry, RequestHandler_HandlerIF_logout_345_Entry, UsageScenario_userType2_7_Entry, RequestHandler_HandlerIF_quickadd_345_Entry
Activities	40, host demands FrontEnd_CPU_Activity: 0.0, InternalAction_main_Iu1-wMhoEeKON4DtRoKCMw_34_50_Activity: 0.01, InternalAction_login_YgxRGw26EeSPwb7XgvxhWQ_34_50_Activity: 0.01, InternalAction_CheckLogin_vHMZsMhoEeKON4DtRoKCMw_34_50_Activity: 0.01, InternalAction_logout_j-2E4MhrEeKON4DtRoKCMw_34_50_Activity: 0.01, InternalAction_addcart_5JEHQMhoEeKON4DtRoKCMw_34_50_Activity: 0.01, USAGE_DELAY0_Activity: 0.0, Start2: 0.0, ...
Calls	19 (19 synchronous, 0 asynchronous)
Think times	UsageScenario_userType1_1_Task: 10.0, UsageScenario_userType2_7_Task: 14.0

Table 2.71: Features of `lqn_ofbiz`.

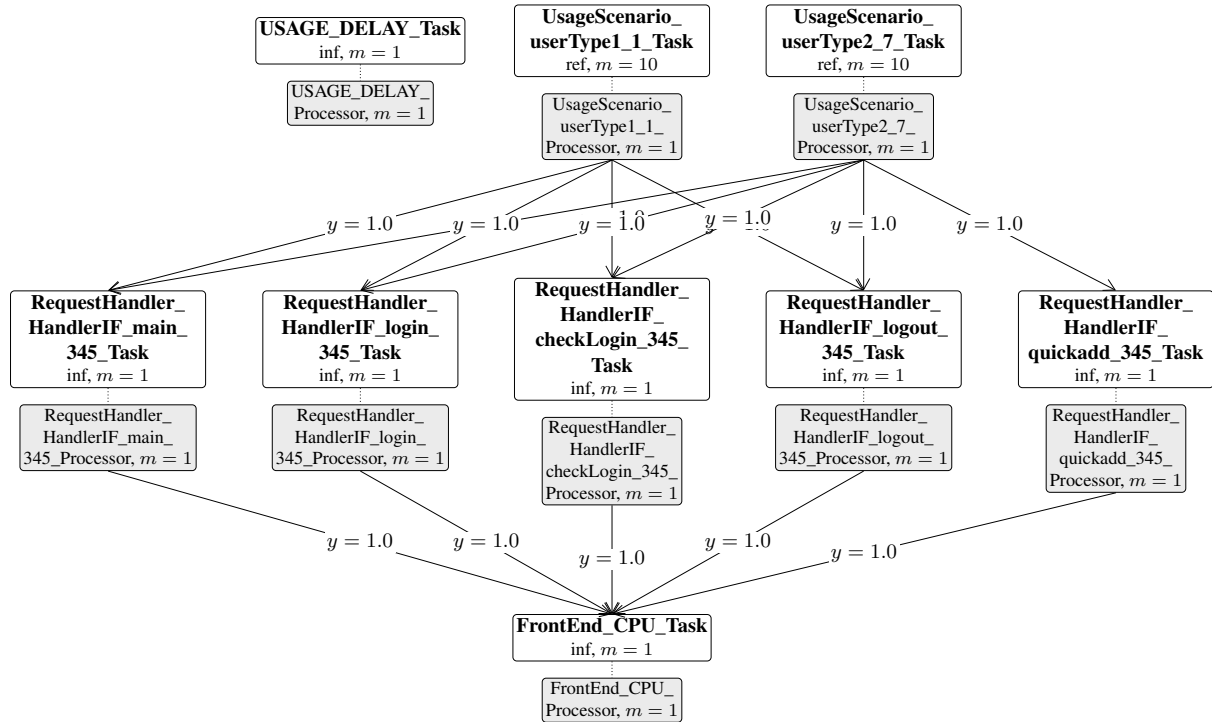


Figure 2.71: Call graph of lqn_ofbiz: 9 tasks over 9 processors, drawn with the reference task on top and a called task below its caller; the shaded box under each task is the processor it runs on.

```
std::printf("This example illustrates the solution of a moderately large LQN.\n");
const LqnS l = lqn::read_lqn<double>(layered_file("lqn_ofbiz.xml"));
const ln::LnOptions opt = nc_layers();
const auto t0 = std::chrono::steady_clock::now();
ln::SolverLN<double> s(l, opt);
const ln::LnSolution<double> sol = s.get_ensemble_avg();
const double tnoinit = std::chrono::duration<double>(std::chrono::steady_clock::now() - t0)
    .count();
print_ln(l, sol, opt, s.nlayers());
std::printf("Tnoinit = %.6f\n", tnoinit);
```

Running this edition prints

```
This example illustrates the solution of a moderately large LQN.
SolverLN(SolverNC) arith=double type=approximate,deterministic layers=14 iterations=52 converged=1
Node      RespT      ResidT      Tput      NodeType      QLen      Util
FrontEnd_CPU_Processor      Processor      NaN      0.116372
NaN      NaN      NaN
USAGE_DELAY_Processor      Processor      NaN      0
NaN      NaN      NaN
UsageScenario_userType1_1_Processor      Processor      NaN      0
NaN      NaN      NaN
RequestHandler_HandlerIF_main_345_Processor      Processor      NaN      0
NaN      NaN      NaN
RequestHandler_HandlerIF_login_345_Processor      Processor      NaN      0
NaN      NaN      NaN
RequestHandler_HandlerIF_checkLogin_345_Processor      Processor      NaN      0
NaN      NaN      NaN
RequestHandler_HandlerIF_logout_345_Processor      Processor      NaN      0
NaN      NaN      NaN
UsageScenario_userType2_7_Processor      Processor      NaN      0
NaN      NaN      NaN
RequestHandler_HandlerIF_quickadd_345_Processor      Processor      NaN      0
NaN      NaN      NaN
FrontEnd_CPU_Task      Task      0.131481      0.116372
NaN      0.0112983      11.6372
USAGE_DELAY_Task      Task      0      0
```

NaN	0	0			
UsageScenario_userType1_1_Task			RefTask	0.0673337	0
NaN	0	0.993267			
RequestHandler_HandlerIF_main_345_Task			Task	0.0465	0
NaN	0	4.11564			
... (52 captured-output lines omitted; rerun the source example for the full output)					
EntryLevelSystemCallcall12quickadd			Activity	0.00801847	0
0.0112983	0	0.709704			
EntryLevelSystemCallcall12logout			Activity	0.00801847	0
0.0112983	0	0.709704			
EntryLevelSystemCallcall12main3			Activity	0.00801847	0
0.0112983	0	0.709704			
Stop9			Activity	0	0
0	0	0.709704			
StartAction_start__EkBkIMhoEeKON4DtRoKCMw_34_5			Activity	0	0
0	0	0.709704			
InternalAction_addcart__5JEHQMhoEeKON4DtRoKCMw_34_50			Activity	0.00801845	0
0.0112983	0	0.709704			
StopAction_stop__EkCLMMhoEeKON4DtRoKCMw_34_5			Activity	0	0
0	0	0.709704			
Tnoinit = 0.376400					

2.7.11 lqn_open_arrival

An open arrival stream into an entry, instead of a closed reference task.

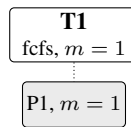


Figure 2.72: Call graph of `lqn_open_arrival`: 1 task over 1 processor, drawn with the reference task on top and a called task below its caller; the shaded box under each task is the processor it runs on.

Processors	1: P1 (ps, $m = 1$)
Tasks	1: T1 (fcfs, $m = 1$)
Entries	1: E1
Activities	1, host demands A1: 1.6

Table 2.72: Features of `lqn_open_arrival`.

The example is built and solved as follows.

```

Lqn b;
b.processor("P1", 1, SchedStrategy::PS);
b.task("T1", 1, SchedStrategy::FCFS, "P1");
b.think_time("T1", Immediate());
b.entry("E1", "T1");
b.open_arrival("E1", Er(0.2));
b.activity("A1", Em(1.6), "T1");
b.bound_to("A1", "E1");
b.replies_to("A1", "E1");
solve_ln(b.build());

```

Running this edition prints

SolverLN(SolverMVA) arith=double type=approximate,deterministic layers=1 iterations=52 converged=1						
Node	RespT	ResidT	Tput	NodeType	Qlen	Util
P1				Processor	NaN	0.32
T1	NaN	NaN	NaN	Task	0.32	0.32
	NaN	1.6	0.2			

E1				Entry	0.32	0.32
A1	1.6	NaN	0.2			
	1.6	1.6	0.2	Activity	0.32	0.32

2.7.12 lqn_paramident

Parameter identification on a layered model, fitting host demands to measurements.

No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

The example is built and solved as follows.

```
const std::size_t nsteps = 16, np = 2, no = 3;
LqnS l = paramident_model();

const std::size_t t1 = infer::infer_lqn_findbyname(l.names, "T1");
const std::size_t as3 = infer::infer_lqn_findbyname(l.names, "AS3");
if (t1 == infer::INFER_LQN_NOT_FOUND || as3 == infer::INFER_LQN_NOT_FOUND)
    throw InputError("lqn_paramident: the hidden parameters name no element of the LQN");

std::vector<infer::LqnObsSpec> obs(3);
obs[0].metric = infer::LqnMetric::RespT;
obs[0].name = "E1";
obs[1].metric = infer::LqnMetric::Util;
obs[1].name = "P1";
obs[2].metric = infer::LqnMetric::Util;
obs[2].name = "P2";

// infer_lqn_setparams: a mean is injected as an exponential, i.e. SCV = 1
auto hfun = [&](const std::vector<double>& a) {
    l.think[t1] = Em(a[0]);
    l.hostdem[as3] = Em(a[1]);
    ln::LnOptions opt;
    ln::SolverLN<double> s(l, opt);
    const ln::LnSolution<double> sol = s.get_ensemble_avg();
    infer::LqnMetrics<double> mt;
    mt.QLen = sol.QN;
    mt.Util = sol.UN;

    ... (46 source lines omitted; full implementation: cpp/examples/basic/)
}

std::printf("\nFinal: Z = %.4f (true %.4f), Sd = %.5f (true %.5f)\n",
            info.ahat(0, nsteps - 1), a_true(0, nsteps - 1), info.ahat(1, nsteps - 1),
            a_true(1, nsteps - 1));
std::printf("RMS tracking Ea = %.4g, prediction Er = %.4g\n", info.has_Ea ? info.Ea : 0.0,
            info.Er);
```

Running this edition prints

```
Step | Z_true | Z_hat | Sd_true | Sd_hat | ||e||
1 | 0.500 | 0.498 | 0.0200 | 0.0196 | 0.124
2 | 0.500 | 0.474 | 0.0200 | 0.0197 | 0.0335
3 | 0.500 | 0.484 | 0.0200 | 0.0200 | 0.0126
4 | 0.500 | 0.478 | 0.0200 | 0.0195 | 0.0198
5 | 0.500 | 0.433 | 0.0400 | 0.0302 | 0.43
6 | 0.500 | 0.440 | 0.0400 | 0.0351 | 0.19
7 | 0.500 | 0.446 | 0.0400 | 0.0376 | 0.0941
8 | 0.500 | 0.439 | 0.0400 | 0.0376 | 0.0216
9 | 1.000 | 0.663 | 0.0400 | 0.0370 | 0.285
10 | 1.000 | 0.770 | 0.0400 | 0.0373 | 0.132
11 | 1.000 | 0.862 | 0.0400 | 0.0371 | 0.115
12 | 1.000 | 0.907 | 0.0400 | 0.0376 | 0.058
13 | 1.000 | 1.025 | 0.0200 | 0.0290 | 0.341
14 | 1.000 | 1.062 | 0.0200 | 0.0250 | 0.146
15 | 1.000 | 1.065 | 0.0200 | 0.0224 | 0.0854
16 | 1.000 | 1.044 | 0.0200 | 0.0212 | 0.0422
```

```
Final: Z = 1.0445 (true 1.0000), Sd = 0.02120 (true 0.02000)
RMS tracking Ea = 0.2814, prediction Er = 0.1029
```

2.7.13 lqn_rrabin

Round-robin dispatching between replicated tasks.

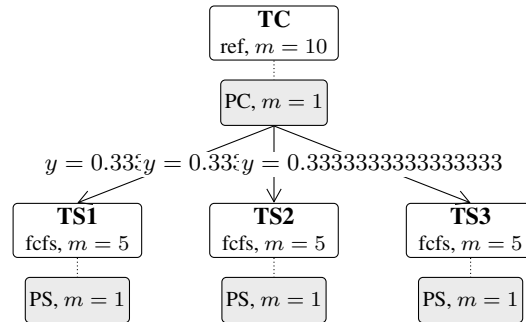


Figure 2.73: Call graph of `lqn_rrabin`: 4 tasks over 2 processors, drawn with the reference task on top and a called task below its caller; the shaded box under each task is the processor it runs on.

Processors	2: PC (inf, $m = 1$), PS (ps, $m = 1$)
Tasks	4: TC (ref, $m = 10$), TS1 (fcfs, $m = 5$), TS2 (fcfs, $m = 5$), TS3 (fcfs, $m = 5$)
Entries	4: EC, ES1, ES2, ES3
Activities	4, host demands AC: 0.5, AS1: 1.0, AS2: 1.0, AS3: 1.0
Calls	3 (3 synchronous, 0 asynchronous)
Think times	TC: 5.0

Table 2.73: Features of `lqn_rrabin`.

The example is built and solved as follows.

```
solve_ln(routed_group_model(false), routed_group_options());
```

Running this edition prints

SolverLN(SolverSSA) arith=double type=exact,deterministic layers=1 iterations=10 converged=1						
Node	RespT	ResidT	Tput	NodeType	QLen	Util
PC				Processor	NaN	0.494377
	NaN	NaN	NaN			
PS				Processor	NaN	0.945844
	NaN	NaN	NaN			
TC				RefTask	5.18129	0.494377
	NaN	0.515198	0.968992			
TS1				Task	1.53199	0.31035
	NaN	4.858	0.315137			
TS2				Task	1.51053	0.310902
	NaN	4.9	0.308171			
TS3				Task	1.58723	0.324593
	NaN	5.06021	0.309728			
EC				Entry	5.18129	0.494377
	5.32976	NaN	0.972143			
ES1				Entry	1.53199	0.31035
	4.67543	NaN	0.327667			
ES2				Entry	1.51053	0.310902
	4.65833	NaN	0.324264			
ES3				Entry	1.58723	0.324593
	4.85156	NaN	0.327158			
AC				Activity	2.10549	0.494377
	2.12943	0.515198	0.988754			
AS1				Activity	1.59641	0.31035
	5.1439	4.858	0.31035			

AS2	5.13027	4.9	0.310902	Activity	1.59501	0.310902
AS3	5.03294	5.06021	0.324593	Activity	1.63366	0.324593

2.7.14 lqn_serial

A serial call chain across three layers, with an immediate activity.

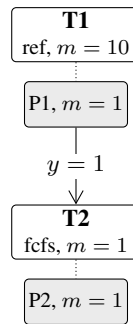


Figure 2.74: Call graph of `lqn_serial`: 2 tasks over 2 processors, drawn with the reference task on top and a called task below its caller; the shaded box under each task is the processor it runs on.

Processors	2: P1 (ps, $m = 1$), P2 (ps, $m = 1$)
Tasks	2: T1 (ref, $m = 10$), T2 (fcfs, $m = 1$)
Entries	2: E1, E2
Activities	4, host demands AS1: 1.6, AS2: 0.0, AS3: 5.0, AS4: 1.0
Calls	1 (1 synchronous, 0 asynchronous)
Think times	T1: 100.0

Table 2.74: Features of `lqn_serial`.

The example is built and solved as follows.

```

Lqn b;
b.processor("P1", 1, SchedStrategy::PS);
b.processor("P2", 1, SchedStrategy::PS);
b.task("T1", 10, SchedStrategy::REF, "P1");
b.think_time("T1", Em(100));
b.task("T2", 1, SchedStrategy::FCFS, "P2");
b.think_time("T2", Immediate());
b.entry("E1", "T1");
b.entry("E2", "T2");
b.activity("AS1", Em(1.6), "T1");
b.bound_to("AS1", "E1");
b.activity("AS2", Immediate(), "T1");
b.sync_call("AS2", "E2", 1.0);
b.activity("AS3", Em(5), "T2");
b.bound_to("AS3", "E2");
b.activity("AS4", Em(1), "T2");
b.replies_to("AS4", "E2");
b.serial("AS1", "AS2");
b.serial("AS3", "AS4");
const LqnS l = b.build();

solve_lqns(b.model());
solve_ln(l);
  
```

Running this edition prints

```
SOLVER: LQNS
```

```
N/A: the reference solves this model with the external lqns binary, which this port does not carry;
      SolverLN below is a different algorithm, not a stand-in
SolverLN(SolverMVA) arith=double type=approximate,deterministic layers=3 iterations=52 converged=1
Node      RespT      ResidT      Tput      NodeType      QLen      Util
P1         NaN         NaN         NaN      Processor      NaN      0.141226
P2         NaN         NaN         NaN      Processor      NaN      0.529599
T1         NaN      1.82889      0.0882665  RefTask      1.17335      0.141226
T2         NaN         6      0.0882665  Task         0.529599      0.529599
E1         13.3         NaN      0.0882665  Entry        1.17335      0.141226
E2         6           NaN      0.0882665  Entry        0.529599      0.529599
AS1        1.82889      1.82889      0.0882665  Activity     0.16143      0.141226
AS2        11.4644         0      0.0882665  Activity     1.01192         0
AS3         5           5      0.0882665  Activity     0.441333      0.441333
AS4         1           1      0.0882665  Activity     0.0882665      0.0882665
```

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by LQNS.

2.7.15 lqn_server_pools

A layered model with heterogeneous server pools, compared with the compatibility representation that treats the servers as interchangeable.

No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

The example is built and solved as follows.

```
ln::LnOptions opt;
// a compatibility declaration is a station rate law, which only the
// class-switching layerings can carry
opt.method = "srvn.cs";
std::cout << "--- homogeneous pool on P1 ---" << std::endl;
solve_ln(lqn_server_pools_model(false), opt);
std::cout << "--- compatibility pool on P1 ---" << std::endl;
solve_ln(lqn_server_pools_model(true), opt);
```

Running this edition prints

```
--- homogeneous pool on P1 ---
SolverLN(SolverMVA) arith=double type=approximate,deterministic layers=4 iterations=52 converged=1
Node      RespT      ResidT      Tput      NodeType      QLen      Util
P0         NaN         NaN         NaN      Processor      NaN      0.679022
P1         NaN         NaN         NaN      Processor      NaN      1.13156
T1         NaN      0.811272      1.35804  RefTask      1.64196      0.679022
T2         NaN      0.1641      1.35796  Task         0.222842      0.452654
T3         NaN      0.232651      1.35781  Task         0.315895      0.678904
E1        1.20906         NaN      1.35804  Entry        1.64196      0.679022
E2         0.1641         NaN      1.35796  Entry        0.222842      0.452654
E3         0.232651         NaN      1.35781  Entry        0.315895      0.678904
A1         1.20958      0.811272      1.35804  Activity     1.64266      0.679022
```

A2	0.1641	0.1641	1.35796	Activity	0.222842	0.452654
A3	0.232651	0.232651	1.35781	Activity	0.315895	0.678904
--- compatibility pool on P1 ---						
SolverLN(SolverMVA) arith=double type=approximate,deterministic layers=4 iterations=52 converged=1						
... (4 captured-output lines omitted; rerun the source example for the full output)						
T2	NaN	0.225579	1.27904	Task	0.288525	0.426347
T3	NaN	0.332217	1.27867	Task	0.424795	0.639334
E1	1.3452	NaN	1.27921	Entry	1.72079	0.639605
E2	0.225579	NaN	1.27904	Entry	0.288525	0.426347
E3	0.332217	NaN	1.27867	Entry	0.424795	0.639334
A1	1.3465	0.784788	1.27921	Activity	1.72246	0.639605
A2	0.225579	0.225579	1.27904	Activity	0.288525	0.426347
A3	0.332217	0.332217	1.27867	Activity	0.424795	0.639334

2.7.16 lqn_setup

Task setup and initialization activities before service.

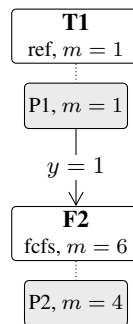


Figure 2.75: Call graph of `lqn_setup`: 2 tasks over 2 processors, drawn with the reference task on top and a called task below its caller; the shaded box under each task is the processor it runs on.

Processors	2: P1 (inf, $m = 1$), P2 (fcfs, $m = 4$)
Tasks	2: T1 (ref, $m = 1$), F2 (fcfs, $m = 6$)
Entries	2: E1, E2
Activities	2, host demands A1: 1.0, A2: 0.3333333333333333
Calls	1 (1 synchronous, 0 asynchronous)

Table 2.75: Features of `lqn_setup`.

The example is built and solved as follows.

```

std::printf("Example of layered model with a task paying setup and delay-off times\n");
Lqn b;
b.processor("P1", lang::GlobalConstants::MaxInt, SchedStrategy::INF);
b.task("T1", 1, SchedStrategy::REF, "P1");
b.entry("E1", "T1");
b.processor("P2", 4, SchedStrategy::FCFS);
b.task("F2", 6, SchedStrategy::FCFS, "P2");
b.think_time("F2", Em(8.0));
b.setup_time("F2", Er(1.0), Er(2.0));
b.entry("E2", "F2");

```



```

b.activity("A1", Er(1.0), "T1");
b.bound_to("A1", "E1");
b.sync_call("A1", "E2", 1.0);
b.activity("A2", Er(3.0), "F2");
b.bound_to("A2", "E2");
b.replies_to("A2", "E2");
// lnoptions.seed = 23000 in the reference selects nothing over MVA layers.
solve_ln(b.build());

```

Running this edition prints

```

Example of layered model with a task paying setup and delay-off times
SolverLN(SolverMVA) arith=double type=approximate,deterministic layers=3 iterations=52 converged=1
Node                                     NodeType      QLen      Util
P1                                     Processor      NaN      0.5
  RespT      ResidT      Tput
P2                                     Processor      NaN      0.0416667
  RespT      ResidT      Tput
T1                                     RefTask        1      0.5
  RespT      ResidT      Tput
F2                                     Task           0.5      0.0416667
  RespT      ResidT      Tput
E1                                     Entry           1      0.5
  RespT      ResidT      Tput
E2                                     Entry           0.5      0.0416667
  RespT      ResidT      Tput
A1                                     Activity         1      0.5
  RespT      ResidT      Tput
A2                                     Activity      0.166667      0.0416667
  RespT      ResidT      Tput
0.333334      0.333334      0.5

```

2.7.17 lqn_sockshop

The Sock Shop microservice benchmark as a layered model.

Processors	7: P1 (inf, $m = 1$), P2_1 (ps, $m = 1$), P2_2 (ps, $m = 1$), P2_3 (ps, $m = 1$), P3_1 (ps, $m = 1$), P3_2 (ps, $m = 1$), P3_3 (ps, $m = 1$)
Tasks	7: T0 (ref, $m = 1000$), T1 (fcfs, $m = 24$), T2 (fcfs, $m = 21$), T6 (fcfs, $m = 100$), T3 (fcfs, $m = 139$), T4 (fcfs, $m = 16$), T5 (fcfs, $m = 151$)
Entries	12: E0, E1, E2, E3, E4, E11, E5, E6, E7, E8, E9, E10
Activities	24, host demands AS: 5e-08, AS0: 5e-08, AS1: 5e-08, AS2: 0.0022216, AH1: 5e-08, AH2: 0.0021319, AH3: 5e-08, AH4: 0.0037561, ...
Calls	14 (14 synchronous, 0 asynchronous)
Think times	T0: 7.0

Table 2.76: Features of lqn_sockshop.

The example is built and solved as follows.

```

Lqn b;
b.processor("P1", 1, SchedStrategy::INF);
b.processor("P2_1", 1, SchedStrategy::PS, 2.0);
b.processor("P2_2", 1, SchedStrategy::PS);
b.processor("P2_3", 1, SchedStrategy::PS);
b.processor("P3_1", 1, SchedStrategy::PS);
b.processor("P3_2", 1, SchedStrategy::PS);
b.processor("P3_3", 1, SchedStrategy::PS);

b.task("T0", 1000, SchedStrategy::REF, "P1");
b.think_time("T0", Em(7.0));
b.task("T1", 24, SchedStrategy::FCFS, "P2_1");
b.think_time("T1", Immediate());
b.task("T2", 21, SchedStrategy::FCFS, "P2_2");
b.think_time("T2", Immediate());
b.task("T6", 100, SchedStrategy::FCFS, "P2_3");
b.think_time("T6", Immediate());
b.task("T3", 139, SchedStrategy::FCFS, "P3_1");

```

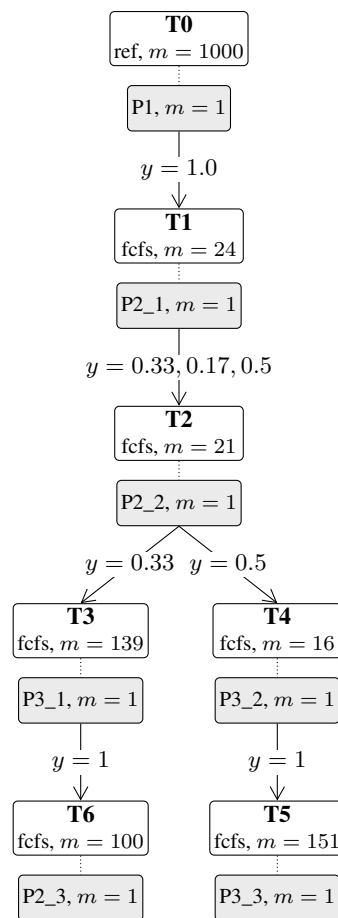


Figure 2.76: Call graph of `lqn_sockshop`: 7 tasks over 7 processors, drawn with the reference task on top and a called task below its caller; the shaded box under each task is the processor it runs on.

```

b.think_time("T3", Immediate());
b.task("T4", 16, SchedStrategy::FCFS, "P3_2");
b.think_time("T4", Immediate());
b.task("T5", 151, SchedStrategy::FCFS, "P3_3");
b.think_time("T5", Immediate());

b.entry("E0", "T0");
b.entry("E1", "T1");

... (104 source lines omitted; full implementation: cpp/examples/basic/)

m.tasks[task_slot("T3")].fanin.push_back(std::make_pair(std::string("T2"), 1.0));
m.tasks[task_slot("T4")].fanout.push_back(std::make_pair(std::string("T5"), 1.0));
m.tasks[task_slot("T4")].fanin.push_back(std::make_pair(std::string("T2"), 1.0));
m.tasks[task_slot("T5")].fanin.push_back(std::make_pair(std::string("T4"), 1.0));

solve_ln(lqn::lqn_finalize(m));

```

Running this edition prints

```

SolverLN(SolverMVA) arith=double type=approximate,deterministic layers=13 iterations=52 converged=1
Node
  RespT      ResidT      Tput      NodeType      QLen      Util
P1           NaN        NaN        Processor      NaN      1.42444e-05
P2_1         NaN        NaN        Processor      NaN      0.316466
P2_2         NaN        NaN        Processor      NaN      0.559962
P2_3         NaN        NaN        Processor      NaN      0.284615
P3_1         NaN        NaN        Processor      NaN      0.437663
P3_2         NaN        NaN        Processor      NaN      0.0788156
P3_3         NaN        NaN        Processor      NaN      0.078558
T0           NaN        1e-07      142.4      RefTask      2.89345      1.42444e-05
T1           NaN      0.00261499      142.4      Task         2.88237      0.316466
T2           NaN      0.00799778      142.5      Task         2.47617      0.559962
T6           NaN      0.00561016       70.5      Task         0.395673      0.284615
T3           NaN      0.0109294       70.5      Task         1.16642      0.437663
T4           NaN      0.00351255       24.2      Task         0.170267      0.0788156
T5           NaN      0.00351779       24.2      Task         0.0852041      0.078558

... (28 captured-output lines omitted; rerun the source example for the full output)

AH11         0          0        23.5      Activity      2.0698e-08      1.17533e-08
AH12      0.0115082      0.00196579      23.5      Activity      0.270519      0.0787191
AS3          0          0        12.1      Activity      0          0
AS4      0.00728746      0.0018848      12.1      Activity      0.0882458      0.0422916
AS5          0          0        12.1      Activity      0          0
AS6      0.00677337      0.00162775      12.1      Activity      0.0820205      0.036524
AH13         0          0        24.2      Activity      1.3135e-08      1.21104e-08
AH14      0.00351779      0.00351779      24.2      Activity      0.0852038      0.078558

```

2.7.18 lqn_srvnph

Method `srvnph`: each entry activity graph composed into one phase-type service law, so an AND fork-join and a loop survive as a distribution rather than as routing.

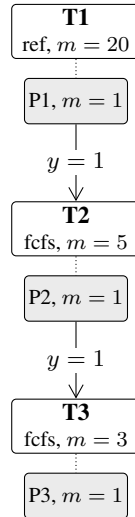


Figure 2.77: Call graph of `lqn_srvnph`: 3 tasks over 3 processors, drawn with the reference task on top and a called task below its caller; the shaded box under each task is the processor it runs on.

Processors	3: P1 (inf, $m = 1$), P2 (ps, $m = 1$), P3 (ps, $m = 1$)
Tasks	3: T1 (ref, $m = 20$), T2 (fcfs, $m = 5$), T3 (fcfs, $m = 3$)
Entries	4: E1, E2, E3, E4
Activities	10, host demands A1: 0.1, A20: 0.2, A21: 0.3, A22: 0.2, A23: 0.1, A30: 0.1, A31: 0.2, A32: 0.1, ...
Calls	3 (3 synchronous, 0 asynchronous)
Think times	T1: 1.0

Table 2.77: Features of `lqn_srvnph`.

The example is built and solved as follows.

```

const LqnS l = lqn_srvnph_model();
double secs[2] = {0.0, 0.0};
std::size_t nclasses[2] = {0, 0};
const char* label[2] = {"default", "srvn.ph"};
const char* method[2] = {"default", "srvn.ph"};

for (int k = 0; k < 2; ++k) {
    ln::LnOptions opt;
    opt.method = method[k];
    const std::chrono::steady_clock::time_point t0 = std::chrono::steady_clock::now();
    ln::SolverLN<double> s(l, opt);
    const ln::LnSolution<double> sol = s.get_ensemble_avg();
    secs[k] = std::chrono::duration<double>(std::chrono::steady_clock::now() - t0).count();
    for (std::size_t e = 0; e < s.layers().size(); ++e)
        nclasses[k] += s.layers()[e].nclasses;
    std::printf("\nLN(%s) Results [%0.3f s]:\n", label[k], secs[k]);
    print_ln(l, sol, opt, s.nlayers());
}
std::printf("\nLayer classes: default %zu, srvn.ph %zu. Runtime: %0.3fs vs %0.3fs (%0.2fx).\n",
    nclasses[0], nclasses[1], secs[0], secs[1],
    secs[1] > 0.0 ? secs[0] / secs[1] : 0.0);
  
```

Running this edition prints

```

LN(default) Results [0.180 s]:
SolverLN(SolverMVA) arith=double type=approximate,deterministic layers=5 iterations=52 converged=1
Node      RespT      ResidT      Tput      NodeType      QLen      Util
P1         NaN         NaN         NaN      Processor      NaN      0.0534882
P2         NaN         NaN         NaN      Processor      NaN      1
P3         NaN         NaN         NaN      Processor      NaN      0.0801795
T1         NaN         0.1        0.534882  RefTask       19.4651   0.0534882
T2         NaN         4.63789    1.05263   Task          4.88199   1
T3         NaN         0.158239   0.53453   Task          0.0845833 0.0801795
E1         36.4         NaN         0.534882  Entry         19.4651   0.0534882
E2         3.90559    NaN         0.526316  Entry         2.05557   0.421053
E3         5.37019    NaN         0.526316  Entry         2.82641   0.578947
E4         0.158239    NaN         0.53453   Entry         0.0845833 0.0801795
A1         36.4         0.1        0.534882  Activity      19.4651   0.0534882
A20        0.976398     0.488199   0.526316  Activity      0.513893   0.105263
A21        1.4646      0.732298   0.526316  Activity      0.77084    0.157895
... (25 captured-output lines omitted; rerun the source example for the full output)
A23        0.488199     0.244099   0.526316  Activity      0.256947   0.0526316
A30        0.488199     0.244099   0.526316  Activity      0.256947   0.0526316
A31        0.976398     1.4646     1.57895   Activity      1.54168    0.315789
A32        0.488199     0.732298   1.57895   Activity      0.77084    0.157895
A33        0.488199     0.244099   0.526316  Activity      0.256947   0.0526316
A4         0.158239     0.158239   0.53453   Activity      0.0845833 0.0801795

Layer classes: default 5, srvn.ph 5. Runtime: 0.180s vs 0.113s (1.59x).

```

2.7.19 lqn_transient

The transient response of a layered model, through `getTranAvg`.

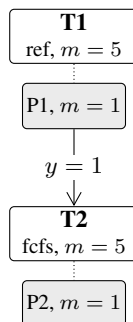


Figure 2.78: Call graph of `lqn_transient`: 2 tasks over 2 processors, drawn with the reference task on top and a called task below its caller; the shaded box under each task is the processor it runs on.

The example is built and solved as follows.

```
Lqn b;
```

Processors	2: P1 (ps, $m = 1$), P2 (ps, $m = 1$)
Tasks	2: T1 (ref, $m = 5$), T2 (fcfs, $m = 5$)
Entries	2: E1, E2
Activities	2, host demands A1: 0.5, A2: 0.3333333333333333
Calls	1 (1 synchronous, 0 asynchronous)
Think times	T1: 1.0

Table 2.78: Features of `lqn_transient`.

```

b.processor("P1", 1, SchedStrategy::PS);
b.processor("P2", 1, SchedStrategy::PS);
b.task("T1", 5, SchedStrategy::REF, "P1");
b.think_time("T1", Er(1.0));
b.task("T2", 5, SchedStrategy::FCFS, "P2");
b.think_time("T2", Er(1.0));
b.entry("E1", "T1");
b.entry("E2", "T2");
b.activity("A1", Er(2.0), "T1");
b.bound_to("A1", "E1");
b.sync_call("A1", "E2", 1.0);
b.activity("A2", Er(3.0), "T2");
b.bound_to("A2", "E2");
b.replies_to("A2", "E2");
const LqnS l = b.build();

ln::LnOptions opt;
opt.layer_solver = "fluid";
ln::SolverLN<double> s(l, opt);
const ln::LnTranSolution tr = s.get_tran_avg();
const std::vector<qn::Layer<double>> & layers = s.layers();

std::printf("LN.getTranAvg returned transient traces for %zu layers.\n", tr.layers.size());
std::printf("SolverLN(SolverFluid) getTranAvg arith=double mode=%s layers=%zu "
            "iterations=%ld gap=%.3e\n",

    ... (13 source lines omitted; full implementation: cpp/examples/basic/)

            layers[e].stations[i].name.c_str(), layers[e].classes[r].name.c_str()
        ,
            ln_sanitiz(e.front()), ln_sanitiz(e.back()),
            ln_sanitiz(tr.layers[e].UN[i][r].back()),
            ln_sanitiz(tr.layers[e].TN[i][r].back()));
    }
}

```

Running this edition prints

```

LN.getTranAvg returned transient traces for 3 layers.
SolverLN(SolverFluid) getTranAvg arith=double mode=decoupled layers=3 iterations=0 gap=0.000e+00

Layer P:P1 (101 points on [0, 45.7008])
Station      JobClass      QLen(0)    QLen(end)    Util(end)    Tput(end)
)
Clients      R:T1          5          3.04672      3.04672
  2
P:P1         R:T1          0          1.95328      1
  2

Layer P:P2 (101 points on [0, 69.1953])
Station      JobClass      QLen(0)    QLen(end)    Util(end)    Tput(end)
)
Clients      T:T2          5          4.36865      4.36865
  1.89405
P:P2         T:T2          0          0.63135      0.63135
  1.89405

Layer T:T2 (101 points on [0, 69.1953])
Station      JobClass      QLen(0)    QLen(end)    Util(end)    Tput(end)
)
Clients      R:T1          5          4.0753       4.0753

```

1.76687				
T:T2	R:T1	0	0.924704	0.184941
1.76687				

2.7.20 lqn_twotasks

Two tasks over separate processors, the two-layer archetype.

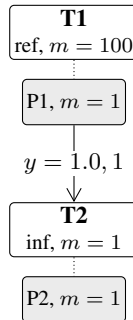


Figure 2.79: Call graph of `lqn_twotasks`: 2 tasks over 2 processors, drawn with the reference task on top and a called task below its caller; the shaded box under each task is the processor it runs on.

Processors	2: P1 (ps, $m = 1$), P2 (ps, $m = 1$)
Tasks	2: T1 (ref, $m = 100$), T2 (inf, $m = 1$)
Entries	3: E1, E2, E3
Activities	5, host demands A1: 1.0, A20: 1.0, A21: 1.0, A22: 1.0, A3: 1.0
Calls	2 (2 synchronous, 0 asynchronous)
Think times	T1: 10.0

Table 2.79: Features of `lqn_twotasks`.

The example is built and solved as follows.

```

const Lqn b = lqn_twotasks_model();
const LqnS l = b.build();
solve_lqns(b.model());
solve_ln(l, nc_layers());

```

Running this edition prints

```

SOLVER: LQNS
N/A: the reference solves this model with the external lqns binary, which this port does not carry
SolverLN(SolverNC) arith=double type=approximate,deterministic layers=3 iterations=52 converged=1
Node      RespT      ResidT      Tput      NodeType      QLen      Util
P1         NaN         NaN         NaN        Processor      NaN        0.25
P2         NaN         NaN         NaN        Processor      NaN        1
T1         NaN         1.32747    0.25       RefTask        97.5       0.25
T2         NaN         194.3      0.5        Task           97.2       1
E1         390         NaN         0.25       Entry          97.5       0.25
E2         291.5       NaN         0.25       Entry          72.9       0.75
E3         97.2        NaN         0.25       Entry          24.3       0.25
A1         390         1.32747    0.25       Activity       97.5       0.25
A20        97.2        48.6       0.25       Activity       24.3       0.25

```

A21				Activity	24.3	0.25
	97.2	48.6	0.25			
A22				Activity	24.3	0.25
	97.2	48.6	0.25			
A3				Activity	24.3	0.25
	97.2	48.6	0.25			

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by `LQNS`.

2.7.21 `lqn_workflows`

Workflow patterns (sequence, AND-fork and join, loop) expressed as activity graphs.

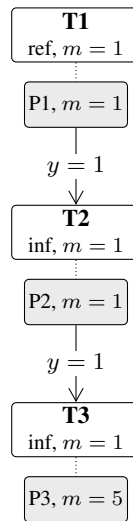


Figure 2.80: Call graph of `lqn_workflows`: 3 tasks over 3 processors, drawn with the reference task on top and a called task below its caller; the shaded box under each task is the processor it runs on.

Processors	3: P1 (inf, $m = 1$), P2 (inf, $m = 1$), P3 (ps, $m = 5$)
Tasks	3: T1 (ref, $m = 1$), T2 (inf, $m = 1$), T3 (inf, $m = 1$)
Entries	3: Entry, E2, E1
Activities	14, host demands A1: 1.0, A2: 2.0, A3: 3.0, B1: 0.1, B2: 0.2, B3: 0.3, B4: 0.4, B5: 0.5, ...
Calls	2 (2 synchronous, 0 asynchronous)
Think times	T1: 0.0

Table 2.80: Features of `lqn_workflows`.

The example is built and solved as follows.

```

std::printf("This example illustrates a layered network with a loop.\n");
Lqn b;
b.processor("P1", lang::GlobalConstants::MaxInt, SchedStrategy::INF);
b.task("T1", 1, SchedStrategy::REF, "P1");
b.think_time("T1", Immediate());
b.entry("Entry", "T1");
b.processor("P2", lang::GlobalConstants::MaxInt, SchedStrategy::INF);
b.task("T2", 1, SchedStrategy::INF, "P2");
b.think_time("T2", Immediate());
b.entry("E2", "T2");
b.processor("P3", 5, SchedStrategy::PS);
b.task("T3", 20, SchedStrategy::INF, "P3");
b.think_time("T3", Em(10));
b.entry("E1", "T3");

```



```

b.activity("A1", Em(1), "T1");
b.bound_to("A1", "Entry");
b.activity("A2", Em(2), "T1");
b.activity("A3", Em(3), "T1");
b.sync_call("A3", "E2", 1.0);

b.activity("B1", Em(0.1), "T2");
b.bound_to("B1", "E2");
b.activity("B2", Em(0.2), "T2");
b.activity("B3", Em(0.3), "T2");
b.activity("B4", Em(0.4), "T2");

... (19 source lines omitted; full implementation: cpp/examples/basic/)

b.or_fork("C1", {"C2", "C3", "C4"}, {0.3, 0.3, 0.4});
b.or_join({"C2", "C3", "C4"}, "C5");
const LqnS l = b.build();

solve_lqns(b.model());
solve_ln(l);

```

Running this edition prints

This example illustrates a layered network with a loop.

SOLVER: LQNS
N/A: the reference solves this model with the external lqns binary when it is installed; this port does not carry it
SolverLN(SolverMVA) arith=double type=approximate,deterministic layers=5 iterations=52 converged=1

Node	RespT	ResidT	Tput	NodeType	QLen	Util
P1				Processor	NaN	0.795425
P2	NaN	NaN	NaN	Processor	NaN	0.163074
P3	NaN	NaN	NaN	Processor	NaN	0.071629
T1	NaN	10	0.0795425	RefTask	1	0.795425
T2	NaN	2.1	0.0776545	Task	0.19972	0.163074
T3	NaN	0.91	0.0787132	Task	0.071629	0.071629
Entry	12.5719	NaN	0.0795425	Entry	1	0.795425
E2	2.5719	NaN	0.0776545	Entry	0.19972	0.163074
E1	0.91	NaN	0.0787132	Entry	0.071629	0.071629
A1	1	1	0.0795425	Activity	0.0795425	0.0795425
... (5 captured-output lines omitted; rerun the source example for the full output)						
B4	0.4	0.4	0.0776545	Activity	0.0310618	0.0310618
B5	0.5	0.5	0.0776545	Activity	0.0388273	0.0388273
B6	1.51	0.6	0.0776545	Activity	0.117258	0.0465927
C1	0.1	0.1	0.0787132	Activity	0.00787132	0.00787132
C2	0.2	0.06	0.023614	Activity	0.00472279	0.00472279
C3	0.3	0.09	0.023614	Activity	0.00708419	0.00708419
C4	0.4	0.16	0.0314853	Activity	0.0125941	0.0125941
C5	0.5	0.5	0.0787132	Activity	0.0393566	0.0393566

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by LQNS.

2.8 Cache models

A `Cache` node holds a finite set of items under a replacement policy. Each request names an item drawn from a popularity distribution, typically Zipf; a hit is served at once, a miss must be fetched and inserted, which evicts an item. The metric of interest is the hit ratio, and everything else in the model follows from it.

The first group of scripts fixes the workload and varies the replacement policy (LRU, FIFO, RR, CLIMB, H-LRU, Q-LRU), so the hit ratios are directly comparable. The second group builds retrieval systems, in which the cache sits in front of a backing store and the miss path is itself a queueing network, and the delayed-hit effect appears: a request for an item already being fetched waits for that fetch rather than starting another.

2.8.1 `cache_compare_replc`

RR, FIFO and LRU on one item popularity. CTMC and MVA report all three hit ratios; NC does not support LRU and reports that cell as NaN rather than implying support.

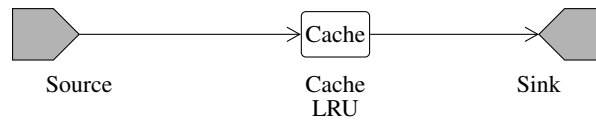


Figure 2.81: Topology of `cache_compare_replc`: an open network with a cache node.

Nodes	3: Source, Cache, Sink
Classes	3: InitClass (open), HitClass (open), MissClass (open)
Arrivals	Source – InitClass: Exp(2); HitClass: Disabled; MissClass: Disabled
Features	cache: LRU replacement, 5 items, capacity 2,1; class switching on 2 links

Table 2.81: Features of `cache_compare_replc`.

The example is built and solved as follows.

```

const std::size_t n = 5;
const double alpha = 1.0;
const std::vector<int> m{2, 1};
const ReplacementStrategy strat[3] = {ReplacementStrategy::RR, ReplacementStrategy::FIFO,
                                       ReplacementStrategy::LRU};
const char* names[3] = {"rr", "fifo", "lru"};

for (int s = 0; s < 3; ++s) {
    Net net = open_cache_model(strat[s], zipf_pmf(alpha, n), m, 2.0);
    const Sn& sn = net.get_struct();

    // `cacheNode.getHitRatio` AFTER the CTMC solve, which is the hit share
    // the analyzer writes back into `actualhitprob`: the hit-class
    // departure rate at the cache over the hit- plus miss-class one. It is
    // read off the solved chain, not off the routing matrix, whose cache
    // entries `refresh_routing` resolved to a uniform split.
    double ctmc_hr = kNaN;
    if (gate("SolverCTMC", qn::ctmc_feature_set("exact"), sn)) {
        ctmc::CtmcOptions copt;
        copt.method = "exact";
        copt.cutoff = 1.0;
        const ctmc::CtmcSolution<double> cr = ctmc::solver_ctmc_analyzer(sn, copt);
        if (!cr.cache.caches.empty() && !cr.cache.caches[0].hitprob.empty())
            ctmc_hr = cr.cache.caches[0].hitprob[0];
    }
}

```

```

... (17 source lines omitted; full implementation: cpp/examples/basic/)

// `CACHE`: a hit ratio compared across three solvers is not any one
// solver's row.
const double column[3] = {ctmc_hr, mva_hr, nc_hr};
for (int k = 0; k < 3; ++k)
    derived("CACHE", names[s], "item" + std::to_string(k), column[k]);
}

```

Running this edition prints

```

rr: 0.71338230, 0.69673077, 0.70305657
fifo: 0.71338230, 0.69673077, 0.70305657

SOLVER: SolverNC
N/A: SolverNC: this model uses the LRU cache replacement policy, which this solver does not support
lru: 0.72430609, 0.72522055, nan

```

2.8.2 cache_itemsize_costcap

Per-item storage costs and a cap on the cost each list may hold, so an insertion that would breach the cap serves the request without changing the cache state.

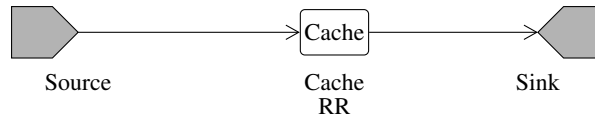


Figure 2.82: Topology of cache_itemsize_costcap: an open network with a cache node.

Nodes	3: Source, Cache, Sink
Classes	3: InitClass (open), HitClass (open), MissClass (open)
Arrivals	Source – InitClass: Exp(2); HitClass: Disabled; MissClass: Disabled
Features	cache: RR replacement, 6 items, capacity 1,1; class switching on 2 links
Solvers	NC

Table 2.82: Features of cache_itemsize_costcap.

The example is built and solved as follows.

```

const std::size_t n = 6;
Net net = open_cache_model(ReplacementStrategy::RR, uniform_pmf(n),
                           std::vector<int>{1, 1}, 2.0,
                           std::vector<int>{1, 1, 1, 2, 2, 2}, std::vector<int>{2, 1});
const Sn& sn = net.get_struct();
if (!gate("SolverNC", qn::nc_feature_set("exact"), sn)) return;

section("NC");
nc::NcSolverOptions opt;
opt.method = "exact";
print_avg(sn, nc::solver_nc_run_analyzer(sn, opt));

const nc::NcCacheSolution<double> r = nc::solver_nc_cache_analyzer(sn, opt);
const CacheInfo ci = cache_info(sn);
const double lam = sn.rates(sn.sourceIdx - 1, 0);

// The reference's per-class hit ratio: one over each read stream, zero where
// the class never reads, which is what the miss RATE over the read rate is.
std::vector<double> hit(sn.nclasses, 0.0);
for (std::size_t c = 0; c < sn.nclasses && c < r.missrate.size(); ++c)
    if (c == 0 && lam > 0.0) hit[c] = 1.0 - r.missrate[c] / lam;
const double miss = 1.0 - hit[0];

note("Cache table (NC):");

```

```

cache_header();
cache_row(ci.name, sn.classes[0].name, 0, ci.totcap, ci.nitems, hit[0], kNaN, miss);

... (7 source lines omitted; full implementation: cpp/examples/basic/)

print_ratio("Hit Ratio", hit);
print_ratio("Mean per-list storage cost", r.listcost);
if (r.costcap_method_switched)
    note("(the requested method had no cost-capped counterpart and was switched)");
if (!r.costcap_blocked.empty())
    note("(a cost cap blocks a promotion path: cross-check with SolverLDES)");

```

Running this edition prints

```

SOLVER: NC
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
Tput
Source      InitClass      0          0          0          0          0
2
Cache table (NC):
Node      JobClass      List  ListCap  Items      HitProb  DelayedHitP      MissProb      ArvR
ResidT
Cache      InitClass      0          2          6      0.333333      nan      0.666667      nan
nan
Cache      InitClass      1          nan          6      0.166667      nan      nan      nan
nan
Cache      InitClass      2          nan          6      0.166667      nan      nan      nan
nan
Item table (NC):
Item      Miss      List1      List2
1          0.533333      0.133333      0.333333
2          0.533333      0.133333      0.333333
3          0.533333      0.133333      0.333333
4          0.8          0.2          0
5          0.8          0.2          0
6          0.8          0.2          0
Hit Ratio:      0.333333 0 0
Mean per-list storage cost: 1.6 1

```

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by NC.

2.8.3 cache_mmap_rr_env

A marked MAP request stream into a random-replacement cache inside a random environment.

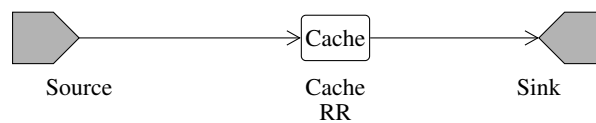


Figure 2.83: Topology of cache_mmap_rr_env: an open network with a cache node.

Nodes	3: Source, Cache, Sink
Classes	6: Read1 (open), Read2 (open), Hit1 (open), Miss1 (open), Hit2 (open), Miss2 (open)
Arrivals	Source – Read1: MMAP; Read2: MMAP; Hit1: Disabled; Miss1: Disabled; Hit2: Disabled; Miss2: Disabled
Features	cache: RR replacement, 4 items, capacity 2; class switching on 2 links
Solvers	LDES, CTMC, FLD

Table 2.83: Features of cache_mmap_rr_env.

The example is built and solved as follows.

```

const std::size_t n = 4; // number of items
const std::vector<int> m{2}; // cache capacity

```

```
// Per-class item-popularity distributions (deliberately different)
const std::vector<double> p1{8.0 / 15, 4.0 / 15, 2.0 / 15, 1.0 / 15};
const std::vector<double> p2{1.0 / 15, 2.0 / 15, 4.0 / 15, 8.0 / 15};

// Marked MMPP2 (M3A layout D = {D0, D11, D12}); D1 = D11 + D12 diagonal.
// Phase 1 bursty (rate 4, 90% class1); phase 2 calm (rate 1, 80% class2).
// Off-diagonal of D0 are the phase-switch rates (both 0.5).
Matrix<double> D0(2, 2), D11(2, 2, 0.0), D12(2, 2, 0.0);
D0(0, 0) = -4.5; D0(0, 1) = 0.5; D0(1, 0) = 0.5; D0(1, 1) = -1.5;
D11(0, 0) = 3.6; D11(1, 1) = 0.2; // class-1 arrivals per phase
D12(0, 0) = 0.4; D12(1, 1) = 0.8; // class-2 arrivals per phase

// `buildCacheModel` with the arrival law of one stage, or the MMAP itself.
struct Build {
    static Net run(std::size_t n, const std::vector<int>& m, const std::vector<double>& p1,
                  const std::vector<double>& p2, std::size_t* cache_out,
                  std::size_t* source_out) {
        Net net("MMAPCache");
        Source source(net, "Source");
        qn::CacheParam<double> ch;
        ch.nitems = n;
        ch.itemcap = m;
        ch.replacestrat = ReplacementStrategy::RR;
        ch.pread = std::vector<std::vector<double>> >{p1, p2, {}, {}, {}, {}};

        ... (138 source lines omitted; full implementation: cpp/examples/basic/)

        env_cache_rows(r, e.stage(0).model);
    } catch (const std::exception& ex) {
        na("ENV (mean field over FLD stages)", ex.what());
    }
}
(void)cache_node;
```

Running this edition prints

```
SOLVER: LDES
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
Tput
Source      Read1      0          0          0          0          0
1.9
Source      Read2      0          0          0          0          0
0.6

SOLVER: CTMC
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
Tput
Source      Read1      0          0          0          0          0
1.9
Source      Read2      0          0          0          0          0
0.6

SOLVER: ENV (avg)
Node      JobClass      List  ListCap  Items      HitProb  DelayedHitP  MissProb  ArvR
ResidT
Cache      Read1      0      2      4      0.566588      nan      0.433412      nan
nan
Cache      Read2      0      2      4      0.450462      nan      0.549538      nan
nan

SOLVER: ENV (dec)
... (5 captured-output lines omitted; rerun the source example for the full output)

Node      JobClass      List  ListCap  Items      HitProb  DelayedHitP  MissProb  ArvR
ResidT
Cache      Read1      0      2      4      0.596258      nan      0.403742      nan
nan
Cache      Read2      0      2      4      0.477143      nan      0.522857      nan
nan

SOLVER: ENV (mean field over FLD stages)
Node      JobClass      List  ListCap  Items      HitProb  DelayedHitP  MissProb  ArvR
ResidT
Cache      Read1      0      2      4      0.573977      nan      0.426023      nan
```

Cache	nan							
	Read2	0	2	4	0.482485	nan	0.517515	nan
	nan							

The rows repeat for each of the 3 solvers the script runs (LDES, CTMC, ENV), which is the point of the example: the same model read by methods of different cost and different exactness.

2.8.4 cache_replc_climb

The CLIMB policy, which promotes a hit by one list only.

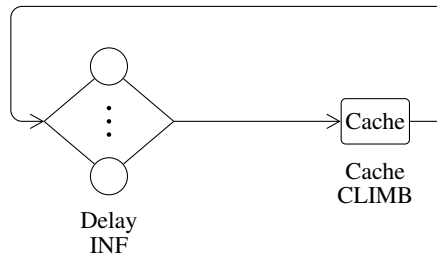


Figure 2.84: Topology of `cache_replc_climb`: a closed network over 1 queueing station with a cache node.

Nodes	2: Delay, Cache
Classes	3: JobClass (closed, $N = 1$), HitClass (closed, $N = 0$), MissClass (closed, $N = 0$)
Scheduling	Delay: INF
Service	Delay – JobClass: Exp(1); HitClass: Disabled; MissClass: Disabled
Features	drop rule { 'HitClass': 'waitingQueue', 'MissClass': 'waitingQueue' } at Delay; cache: CLIMB replacement, 5 items, capacity 2; class switching on 2 links
Solvers	CTMC

Table 2.84: Features of `cache_replc_climb`.

The example is built and solved as follows.

```
const std::size_t n = 5;
Net net =
    closed_cache_model(ReplacementStrategy::CLIMB, zipf_pmf(1.2, n), std::vector<int>{2});
const Sn& sn = net.get_struct();
run_ctmc(sn, -1.0, "exact");
```

Running this edition prints

SOLVER: CTMC						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Delay	JobClass	1	1	1	1	1
Cache	JobClass	0	0	0	0	1
Cache	HitClass	0	0	0	0	0
Cache	MissClass	0	0	0	0	0
CS_Cache_to_Delay	JobClass	0	0	0	0	0
CS_Cache_to_Delay	HitClass	0	0	0	0	0.566295
CS_Cache_to_Delay	MissClass	0	0	0	0	0.433705

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by CTMC.

2.8.5 cache_replc_fifo

First-in first-out replacement.

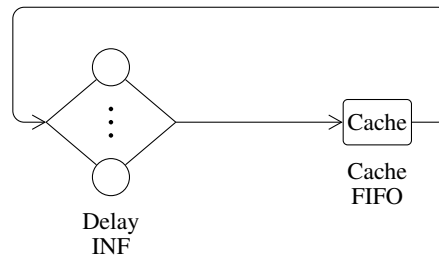


Figure 2.85: Topology of `cache_replc_fifo`: a closed network over 1 queueing station with a cache node.

Nodes	2: Delay, Cache
Classes	3: JobClass (closed, $N = 1$), HitClass (closed, $N = 0$), MissClass (closed, $N = 0$)
Scheduling	Delay: INF
Service	Delay – JobClass: Exp(1); HitClass: Disabled; MissClass: Disabled
Features	drop rule {'HitClass': 'waitingQueue', 'MissClass': 'waitingQueue'} at Delay; cache: FIFO replacement, 5 items, capacity 2; class switching on 2 links
Solvers	CTMC, SSA, MVA, FLD

Table 2.85: Features of `cache_replc_fifo`.

The example is built and solved as follows.

```
const std::size_t n = 5;
Net net =
    closed_cache_model(ReplacementStrategy::FIFO, uniform_pmf(n), std::vector<int>{2});
const Sn& sn = net.get_struct();
run_ctmc(sn, -1.0);
run_ssa(sn, 100000, 23000);
run_mva(sn);
run_fluid_rmf(sn, true);
```

Running this edition prints

```
SOLVER: CTMC
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
  Tput
Delay        JobClass      1          1          1          1          1
  1
Cache        JobClass      0          0          0          0          1
  0
Cache        HitClass      0          0          0          0          0
  0.4
Cache        MissClass     0          0          0          0          0
  0.6
CS_Cache_to_Delay JobClass      0          0          0          0          0
  1
CS_Cache_to_Delay HitClass      0          0          0          0          0.4
  0
CS_Cache_to_Delay MissClass     0          0          0          0          0.6
  0

SOLVER: SSA
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
  Tput
Delay        JobClass      1          1          1          1          1
  1
Cache        JobClass      0          0          0          0          1
  0
Cache        HitClass      0          0          0          0          0
  0.395422
```

Cache	MissClass	0	0	0	0	0
0.604578						
... (17 captured-output lines omitted; rerun the source example for the full output)						
Cache	JobClass	0	0	0	0	1
0						
Cache	HitClass	0	0	0	0	0
0.4						
Cache	MissClass	0	0	0	0	0
0.6						
CS_Cache_to_Delay	JobClass	0	0	0	0	0
1						
CS_Cache_to_Delay	HitClass	0	0	0	0	0.4
0						
CS_Cache_to_Delay	MissClass	0	0	0	0	0.6
0						
Hit Ratio:		0.4	0	0		
Miss Ratio:		0.6	0	0		

The rows repeat for each of the 4 solvers the script runs (CTMC, SSA, MVA, FLD), which is the point of the example: the same model read by methods of different cost and different exactness.

2.8.6 cache_replc_hlru

H-LRU, the multi-list generalization of LRU.

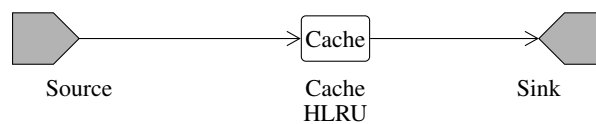


Figure 2.86: Topology of `cache_replc_hlru`: an open network with a cache node.

Nodes	3: Source, Cache, Sink
Classes	3: InitClass (open), HitClass (open), MissClass (open)
Arrivals	Source – InitClass: Exp(1); HitClass: Disabled; MissClass: Disabled
Features	cache: HLRU replacement, 6 items, capacity 2,1; class switching on 2 links
Solvers	CTMC, MVA, SSA

Table 2.86: Features of `cache_replc_hlru`.

The example is built and solved as follows.

```

const std::size_t n = 6;
Net net = open_cache_model(ReplacementStrategy::HLRU, zipf_pmf(1.2, n),
                           std::vector<int>{2, 1}, 1.0);

const Sn& sn = net.get_struct();
run_ctmc(sn, 1.0);
run_mva(sn);
run_ssa(sn, 10000, 23000);

```

Running this edition prints

SOLVER: CTMC						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Source	InitClass	0	0	0	0	0
1						
Cache	InitClass	0	0	0	0	1
0						
Cache	HitClass	0	0	0	0	0
0.700849						
Cache	MissClass	0	0	0	0	0
0.299151						
Sink	HitClass	0	0	0	0	0.700849
0						

Sink		MissClass	0	0	0	0	0.299151
0							
SOLVER: MVA							
Station		JobClass	QLen	Util	RespT	ResidT	ArvR
Tput							
Source		InitClass	0	0	0	0	0
1							
Cache		InitClass	0	0	0	0	1
0							
Cache		HitClass	0	0	0	0	0
0.701566							
Cache		MissClass	0	0	0	0	0
0.298434							
Sink		HitClass	0	0	0	0	0.701566
0							
Sink		MissClass	0	0	0	0	0.298434
0							
SOLVER: SSA							
Station		JobClass	QLen	Util	RespT	ResidT	ArvR
Tput							
Source		InitClass	0	0	0	0	0
1							
Cache		InitClass	0	0	0	0	1
0							
Cache		HitClass	0	0	0	0	0
0.704278							
Cache		MissClass	0	0	0	0	0
0.295722							
Sink		HitClass	0	0	0	0	0.704278
0							
Sink		MissClass	0	0	0	0	0.295722
0							

The rows repeat for each of the 3 solvers the script runs (CTMC, MVA, SSA), which is the point of the example: the same model read by methods of different cost and different exactness.

2.8.7 cache_replc_lru

Least-recently-used replacement, with a closed class circulating between a Delay and the cache.

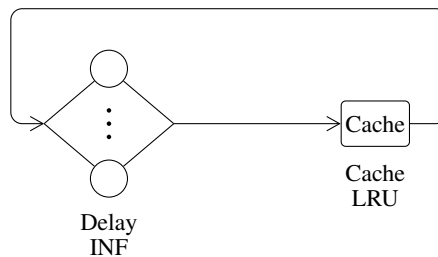


Figure 2.87: Topology of `cache_replc_lru`: a closed network over 1 queueing station with a cache node.

Nodes	2: Delay, Cache
Classes	3: JobClass (closed, $N = 1$), HitClass (closed, $N = 0$), MissClass (closed, $N = 0$)
Scheduling	Delay: INF
Service	Delay – JobClass: Exp(1); HitClass: Disabled; MissClass: Disabled
Features	drop rule { 'HitClass': 'waitingQueue', 'MissClass': 'waitingQueue' } at Delay; cache: LRU replacement, 5 items, capacity 2; class switching on 2 links
Solvers	CTMC, SSA, MVA

Table 2.87: Features of `cache_replc_lru`.

The example is built and solved as follows.

```

const std::size_t n = 5;
Net net =
    closed_cache_model(ReplacementStrategy::LRU, uniform_pmf(n), std::vector<int>{2});
const Sn& sn = net.get_struct();
run_ctmc(sn, -1.0);
run_ssa(sn, 100000, 23000);
run_mva(sn);

```

Running this edition prints

```

SOLVER: CTMC
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
  Tput
Delay        JobClass      1          1          1          1          1
  1
Cache        JobClass      0          0          0          0          1
  0
Cache        HitClass      0          0          0          0          0
  0.4
Cache        MissClass     0          0          0          0          0
  0.6
CS_Cache_to_Delay JobClass      0          0          0          0          0
  1
CS_Cache_to_Delay HitClass      0          0          0          0          0.4
  0
CS_Cache_to_Delay MissClass     0          0          0          0          0.6
  0

SOLVER: SSA
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
  Tput
Delay        JobClass      1          1          1          1          1
  1
Cache        JobClass      0          0          0          0          1
  0
Cache        HitClass      0          0          0          0          0
  0.397235
Cache        MissClass     0          0          0          0          0
  0.602764

... (5 captured-output lines omitted; rerun the source example for the full output)

Station      JobClass      QLen      Util      RespT      ResidT      ArvR
  Tput
Delay        JobClass      1          1          1          1          1
  1
Cache        JobClass      0          0          0          0          1
  0
Cache        HitClass      0          0          0          0          0
  0.4
Cache        MissClass     0          0          0          0          0
  0.6
CS_Cache_to_Delay JobClass      0          0          0          0          0
  1
CS_Cache_to_Delay HitClass      0          0          0          0          0.4
  0
CS_Cache_to_Delay MissClass     0          0          0          0          0.6
  0

```

The rows repeat for each of the 3 solvers the script runs (CTMC, SSA, MVA), which is the point of the example: the same model read by methods of different cost and different exactness.

2.8.8 cache_replc_qlru

Q-LRU, which promotes a hit with probability q .

The example is built and solved as follows.

```

const std::size_t n = 5;
Net net = closed_cache_model(ReplacementStrategy::QLRU, zipf_pmf(1.2, n),
                             std::vector<int>{2}, 0.5);
const Sn& sn = net.get_struct();
run_ctmc(sn, -1.0, "exact");

```

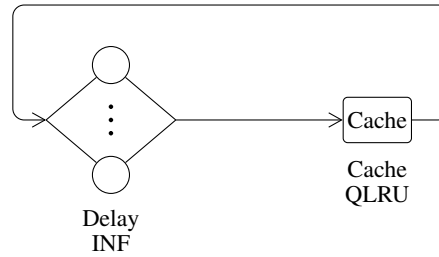


Figure 2.88: Topology of `cache_replc_qlru`: a closed network over 1 queueing station with a cache node.

Nodes	2: Delay, Cache
Classes	3: JobClass (closed, $N = 1$), HitClass (closed, $N = 0$), MissClass (closed, $N = 0$)
Scheduling	Delay: INF
Service	Delay – JobClass: Exp(1); HitClass: Disabled; MissClass: Disabled
Features	drop rule {'HitClass': 'waitingQueue', 'MissClass': 'waitingQueue'} at Delay; cache: QLRU replacement, 5 items, capacity 2; class switching on 2 links
Solvers	CTMC

Table 2.88: Features of `cache_replc_qlru`.

Running this edition prints

SOLVER: CTMC						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Delay	JobClass	1	1	1	1	1
1						
Cache	JobClass	0	0	0	0	1
0						
Cache	HitClass	0	0	0	0	0
0.575143						
Cache	MissClass	0	0	0	0	0
0.424857						
CS_Cache_to_Delay	JobClass	0	0	0	0	0
1						
CS_Cache_to_Delay	HitClass	0	0	0	0	0.575143
0						
CS_Cache_to_Delay	MissClass	0	0	0	0	0.424857
0						

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by CTMC.

2.8.9 `cache_replc_routing`

Cache-aware routing: hits and misses leave the cache on different arcs, under RAND and round-robin routing.

Nodes	6: Source, Delay $\times$ 2, Cache, Router, Sink
Classes	3: InitClass (open), HitClass (open), MissClass (open)
Scheduling	Delay1: INF; Delay2: INF
Arrivals	Source – InitClass: Exp(2)
Service	Delay1 – HitClass: Exp(10); MissClass: Exp(1) Delay2 – HitClass: Exp(20); MissClass: Exp(2)
Features	cache: FIFO replacement, 5 items, capacity 2; class switching on 6 links
Solvers	CTMC, SSA, MVA, NC, FLD

Table 2.89: Features of `cache_replc_routing`.

The example is built and solved as follows.

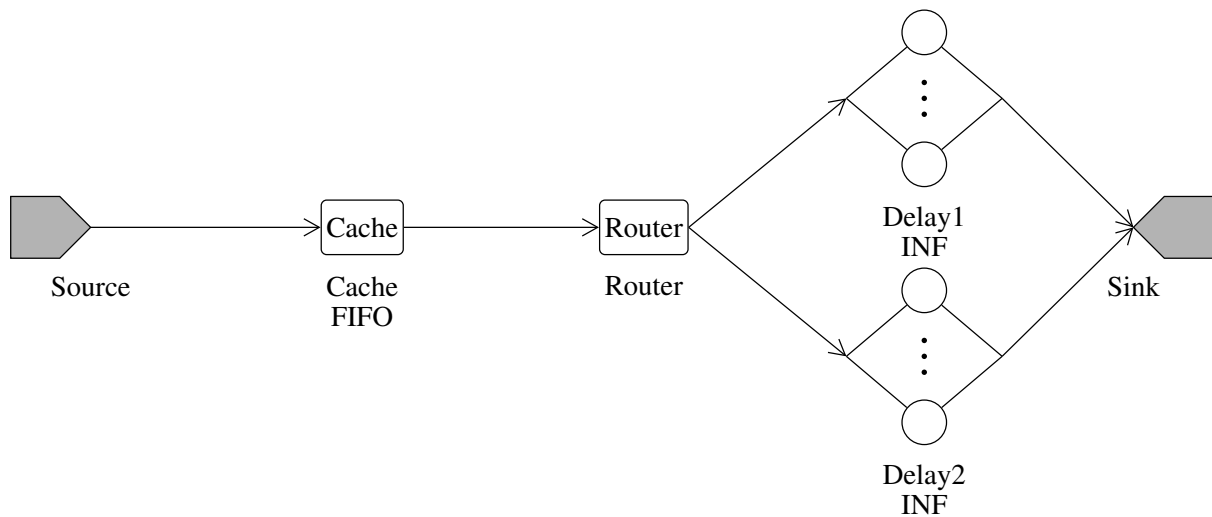


Figure 2.89: Topology of `cache_replc_routing`: an open network over 2 queueing stations with a cache node and a router.

```

const std::size_t n = 5;
Net net("model");
Source source(net, "Source");
qn::CacheParam<double> ch;
ch.nitems = n;
ch.itemcap = std::vector<int>(2);
ch.replacestrat = ReplacementStrategy::FIFO;
ch.pread = std::vector<std::vector<double>>{uniform_pmf(n), {}, {}};
ch.hitclass = std::vector<std::size_t>(2, 0, 0);
ch.missclass = std::vector<std::size_t>(3, 0, 0);
const std::size_t cache = net.add_cache("Cache", ch);
Router router(net, "Router");
Delay delay1(net, "Delay1");
Delay delay2(net, "Delay2");
Sink sink(net, "Sink");

OpenClass init(net, "InitClass");
OpenClass hit(net, "HitClass");
OpenClass miss(net, "MissClass");

source.set_arrival(init, Exp(2.0));
delay1.set_service(hit, Exp(10.0));
delay1.set_service(miss, Exp(1.0));
delay2.set_service(hit, Exp(20.0));
delay2.set_service(miss, Exp(2.0));

... (19 source lines omitted; full implementation: cpp/examples/basic/)

const Sn& sn = net.get_struct();
run_ctmc(sn, 1.0);
run_ssa(sn, 10000, 23000);
run_mva(sn);
run_nc(sn);
run_fluid_rmf(sn, true);

```

Running this edition prints

SOLVER: CTMC						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Source	InitClass	0	0	0	0	0
1.86858						
Cache	InitClass	0	0	0	0	1.86858

0						
Cache	HitClass	0	0	0	0	0
0.747431						
Cache	MissClass	0	0	0	0	0
1.12115						
Router	HitClass	0	0	0	0	0.934289
0.934289						
Router	MissClass	0	0	0	0	0.934289
0.934289						
Delay1	HitClass	0.0367654	0.0367654	0.1	0.025	0.467144
0.367654						
Delay1	MissClass	0.457218	0.457218	1	0.25	0.467144
0.457218						
Delay2	HitClass	0.0189889	0.0189889	0.05	0.0125	0.467144
0.379777						
Delay2	MissClass	0.331964	0.331964	0.5	0.125	0.467144
0.663929						
Sink	HitClass	0	0	0	0	0.747431
0						
Sink	MissClass	0	0	0	0	1.12115
0						
SOLVER: SSA						
... (52 captured-output lines omitted; rerun the source example for the full output)						
Delay1	HitClass	0	0	0	0	0.4
0						
Delay1	MissClass	0	0	0	0	0.6
0						
Delay2	HitClass	0	0	0	0	0.4
0						
Delay2	MissClass	0	0	0	0	0.6
0						
Sink	HitClass	0	0	0	0	0.8
0						
Sink	MissClass	0	0	0	0	1.2
0						
Hit Ratio:		0.4	0	0		
Miss Ratio:		0.6	0	0		

The rows repeat for each of the 5 solvers the script runs (CTMC, SSA, MVA, NC, FLD), which is the point of the example: the same model read by methods of different cost and different exactness.

2.8.10 cache_replc_rr

Random replacement.

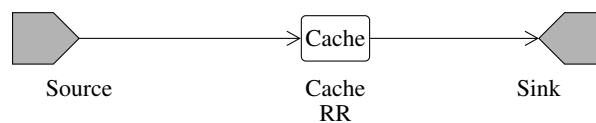


Figure 2.90: Topology of `cache_replc_rr`: an open network with a cache node.

Nodes	3: Source, Cache, Sink
Classes	3: InitClass (open), HitClass (open), MissClass (open)
Arrivals	Source – InitClass: Exp(2); HitClass: Disabled; MissClass: Disabled
Features	cache: RR replacement, 5 items, capacity 2; class switching on 2 links
Solvers	CTMC, SSA, MVA, NC, FLD

Table 2.90: Features of `cache_replc_rr`.

The example is built and solved as follows.

```

const std::size_t n = 5;
Net net = open_cache_model(ReplacementStrategy::RR, zipf_pmf(1.4, n),
                           std::vector<int>{2}, 2.0);

```

```
const Sn& sn = net.get_struct();
run_ctmc(sn, 1.0);
run_ssa(sn, 10000, 23000);
run_mva(sn);
run_nc(sn);
run_fluid_rmf(sn, true);
```

Running this edition prints

```
SOLVER: CTMC
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
  Tput
Source      InitClass      0          0          0          0          0
  2
Cache      InitClass      0          0          0          0          2
  0
Cache      HitClass      0          0          0          0          0
  1.18079
Cache      MissClass      0          0          0          0          0
  0.819208
Sink      HitClass      0          0          0          0          1.18079
  0
Sink      MissClass      0          0          0          0          0.819208
  0

SOLVER: SSA
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
  Tput
Source      InitClass      0          0          0          0          0
  2
Cache      InitClass      0          0          0          0          2
  0
Cache      HitClass      0          0          0          0          0
  1.14603
Cache      MissClass      0          0          0          0          0
  0.853967
Sink      HitClass      0          0          0          0          1.14603
  0

... (22 captured-output lines omitted; rerun the source example for the full output)

Source      InitClass      0          0          0          0          0
  2
Cache      InitClass      0          0          0          0          2
  0
Cache      HitClass      0          0          0          0          0
  1.12458
Cache      MissClass      0          0          0          0          0
  0.875416
Sink      HitClass      0          0          0          0          1.12458
  0
Sink      MissClass      0          0          0          0          0.875416
  0
Hit Ratio:      0.562292 0 0
Miss Ratio:      0.437708 0 0
```

The rows repeat for each of the 5 solvers the script runs (CTMC, SSA, MVA, NC, FLD), which is the point of the example: the same model read by methods of different cost and different exactness.

2.8.11 cache_rmf_transient

The transient hit ratio from the refined mean field approximation.

No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

The example is built and solved as follows.

```
const std::size_t n = 10;
const std::vector<int> m{3, 2};
const double alpha = 0.8;
const std::vector<double> p = zipf_pmf(alpha, n);
```

```
std::printf("Cache parameters: n=%zu, m=[%d, %d], Zipf(%.1f)\n", n, m[0], m[1], alpha);

Matrix<double> lambda(1, n, 0.0);
for (std::size_t i = 0; i < n; ++i) lambda(0, i) = p[i];
const cache::CacheMissRmfResult<double> r =
    cache::cache_miss_rmf(std::vector<double>(), m, lambda, 10000.0);

std::printf("\nSteady-state results (refined mean field):\n");
double total_hit = 0.0;
for (std::size_t k = 1; k <= m.size(); ++k) {
    const double hr = cache::rmf_detail::hit_rate(r.xss, p, k, n);
    total_hit += hr;
    std::printf(" Hit rate (list %zu): %.6f\n", k, hr);
    // The example SUMS `hit_rate` over the lists and forms the transient
    // state as `X[t] + V[t]/n` itself, so these totals exist nowhere else --
    // no result table carries them and no getter returns them. The shape key
    // is `CACHE`; no solver produced these.
    derived("CACHE", "HitRate_L" + std::to_string(k), "Steady", hr);
}
const double miss_rate = cache::rmf_detail::hit_rate(r.xss, p, std::size_t(0), n);
std::printf(" Miss rate: %.6f\n", miss_rate);

... (21 source lines omitted; full implementation: cpp/examples/basic/)

// Keyed by the time as the golden spells it, `t=%.3f`: the grid point
// is the row label, and there is no table these belong to.
char label[32];
std::snprintf(label, sizeof(label), "t=%.3f", tr.t[j]);
derived("CACHE", label, "Transient", hr);
}
```

Running this edition prints

```
Cache parameters: n=10, m=[3, 2], Zipf(0.8)

Steady-state results (refined mean field):
Hit rate (list 1): 0.312854
Hit rate (list 2): 0.301606
Miss rate: 0.385540
Total hit prob: 0.614460
Total miss prob: 0.385540

Transient hit rates (refined, N=10):
t= 0.000: hit_rate=0.728003
t= 5.025: hit_rate=0.645750
t= 12.563: hit_rate=0.624411
t= 25.126: hit_rate=0.616758
t= 50.000: hit_rate=0.614587
```

2.8.12 cachenet_tandem

A tandem of two caches over one item set, where a miss at the first is a read of the same item at the second. Item identity is carried by a dedicated job class per item, so no class switching happens on the arc between the caches.

The example is built and solved as follows.

```
const std::size_t n = 3;
const double pAccess[3] = {0.5, 0.3, 0.2};

Net net("CacheTandem");
Delay think(net, "Think");

qn::CacheParam<double> c1p;
c1p.nitems = n;
c1p.itemcap = std::vector<int>{1};
c1p.replacestrat = ReplacementStrategy::LRU;
const std::size_t cachel = net.add_cache("Cache1", c1p);

qn::CacheParam<double> c2p;
```

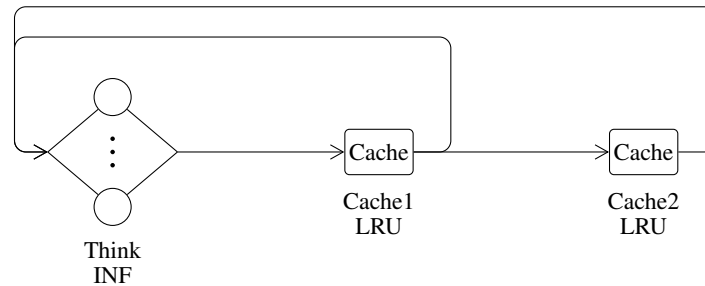


Figure 2.91: Topology of `cachenet_tandem`: a closed network over 1 queueing station with a cache node.

Nodes	3: Delay, Cache $\times$ 2
Classes	15: Read1 (closed, $N = 1$), Hit1_1 (closed, $N = 0$), Hit2_1 (closed, $N = 0$), Miss_1 (closed, $N = 0$), Read2 (closed, $N = 1$), Hit1_2 (closed, $N = 0$), Hit2_2 (closed, $N = 0$), Miss_2 (closed, $N = 0$), Read3 (closed, $N = 1$), Hit1_3 (closed, $N = 0$), Hit2_3 (closed, $N = 0$), Miss_3 (closed, $N = 0$), Cache2_item1 (closed, $N = 0$), Cache2_item2 (closed, $N = 0$), Cache2_item3 (closed, $N = 0$)
Scheduling	Think: INF
Service	Think – Read1: Exp(0.5); Hit1_1: Disabled; Hit2_1: Disabled; Miss_1: Disabled; Read2: Exp(0.3); Hit1_2: Disabled; Hit2_2: Disabled; Miss_2: Disabled; Read3: Exp(0.2); Hit1_3: Disabled; Hit2_3: Disabled; Miss_3: Disabled; Cache2_item1: Disabled; Cache2_item2: Disabled; Cache2_item3: Disabled
Features	drop rule { 'Hit1_1': 'waitingQueue', 'Hit2_1': 'waitingQueue', 'Miss_1': 'waitingQueue', 'Hit1_2': 'waitingQueue', 'Hit2_2': 'waitingQueue', 'Miss_2': 'waitingQueue', 'Hit1_3': 'waitingQueue', 'Hit2_3': 'waitingQueue', 'Miss_3': 'waitingQueue', 'Cache2_item1': 'waitingQueue', 'Cache2_item2': 'waitingQueue', 'Cache2_item3': 'waitingQueue' } at Think; cache: LRU replacement, 3 items, capacity 1; cache: LRU replacement, 3 items, capacity 2; class switching on 3 links
Solvers	CTMC, SSA, LDES

Table 2.91: Features of `cachenet_tandem`.


```

c2p.nitems = n;
c2p.itemcap = std::vector<int>(2);
c2p.replacestrat = ReplacementStrategy::LRU;
const std::size_t cache2 = net.add_cache("Cache2", c2p);

// one dedicated class per item for each role
std::vector<std::size_t> read(n), hit1(n), hit2(n), miss(n);
for (std::size_t f = 0; f < n; ++f) {
    read[f] = net.add_closed_class("Read" + std::to_string(f + 1), 1.0, think);
    hit1[f] = net.add_closed_class("Hit1_" + std::to_string(f + 1), 0.0, think);
    hit2[f] = net.add_closed_class("Hit2_" + std::to_string(f + 1), 0.0, think);
    miss[f] = net.add_closed_class("Miss_" + std::to_string(f + 1), 0.0, think);
    think.set_service(read[f], Exp(pAccess[f]));

    ... (33 source lines omitted; full implementation: cpp/examples/basic/)

    for (std::size_t c = 0; c < sn.nclasses; ++c) {
        const double h = r.cache[k].hitprob[c], mi = r.cache[k].missprob[c];
        if (std::isnan(h) && std::isnan(mi)) continue;
        cache_row(sn.nodes[nd - 1].name, sn.classes[c].name, 0, cap, cp.nitems, h, kNaN, mi);
    }
}

```

Running this edition prints

SOLVER: SSA									
Node	JobClass	List	ListCap	Items	HitProb	DelayedHitP	MissProb	ArvR	
	ResidT								
Cache1	Read1	0	1	3	0.500988	nan	0.499012	nan	
	nan								
Cache1	Read2	0	1	3	0.301921	nan	0.698079	nan	
	nan								
Cache1	Read3	0	1	3	0.197777	nan	0.802223	nan	
	nan								
Cache2	Cache2_item1	0	2	3	0.681806	nan	0.318194	nan	
	nan								
Cache2	Cache2_item2	0	2	3	0.539506	nan	0.460494	nan	
	nan								
Cache2	Cache2_item3	0	2	3	0.353163	nan	0.646837	nan	
	nan								

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by SSA.

2.8.13 retrieval_chain

A chain of caches, each backing the one before it.

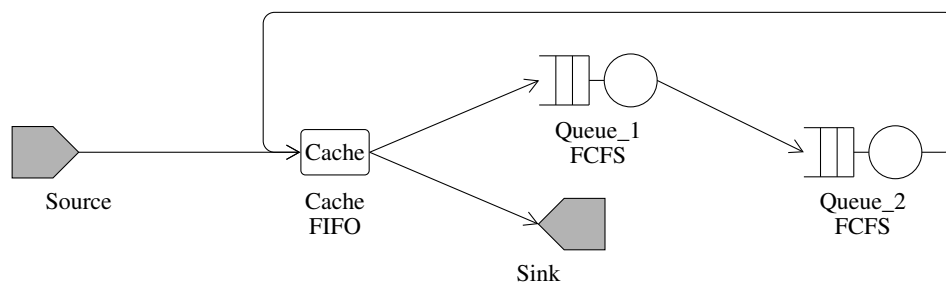


Figure 2.92: Topology of retrieval_chain: an open network over 2 queueing stations with a cache node.

The example is built and solved as follows.

```

const std::vector<double> access{0.6, 0.3, 0.1};
const std::size_t n = access.size();

Net net("Chain Model");

```

Nodes	5: Source, Queue $\times$ 2, Cache, Sink
Classes	6: InitClass (open), HitClass (open), MissClass (open), InitClass_retrievalClass_1 (open), InitClass_retrievalClass_2 (open), InitClass_retrievalClass_3 (open)
Scheduling	Queue_1: FCFS; Queue_2: FCFS
Arrivals	Source – InitClass: Exp(1); HitClass: Disabled; MissClass: Disabled; InitClass_retrievalClass_1: Disabled; InitClass_retrievalClass_2: Disabled; InitClass_retrievalClass_3: Disabled
Service	Queue_1 – InitClass: Exp(2); HitClass: Disabled; MissClass: Disabled; InitClass_retrievalClass_1: Exp(2); InitClass_retrievalClass_2: Exp(2); InitClass_retrievalClass_3: Exp(2) Queue_2 – InitClass: Exp(3); HitClass: Disabled; MissClass: Disabled; InitClass_retrievalClass_1: Exp(3); InitClass_retrievalClass_2: Exp(3); InitClass_retrievalClass_3: Exp(3)
Features	cache: FIFO replacement, 3 items, capacity 1; drop rule {'HitClass': 'waitingQueue', 'MissClass': 'waitingQueue'} at Queue_1; drop rule {'HitClass': 'waitingQueue', 'MissClass': 'waitingQueue'} at Queue_2; class switching on 4 links
Solvers	SSA, LDES, MVA, NC

Table 2.92: Features of retrieval_chain.

```

Source source(net, "Source");
qn::CacheParam<double> ch;
ch.nitems = n;
ch.itemcap = std::vector<int>{1};
ch.replacestrat = ReplacementStrategy::FIFO;
ch.pread = std::vector<std::vector<double>>{access, {}, {}};
ch.hitclass = std::vector<std::size_t>{2, 0, 0};
ch.missclass = std::vector<std::size_t>{3, 0, 0};
const std::size_t cache = net.add_cache("Cache", ch);
Queue q1(net, "Queue_1", SchedStrategy::FCFS);
Queue q2(net, "Queue_2", SchedStrategy::FCFS);
Sink sink(net, "Sink");

OpenClass init(net, "InitClass");
OpenClass hit(net, "HitClass");
OpenClass miss(net, "MissClass");
source.set_arrival(init, Exp(1.0));
q1.set_service(init, Exp(2.0));
q2.set_service(init, Exp(3.0));
net.set_retrieval_system(cache, init, miss, std::vector<std::size_t>{q1, q2});

Routing P;

... (7 source lines omitted; full implementation: cpp/examples/basic/)

const Sn& sn = net.get_struct();
retrieval_ssa_cache_table(sn);
retrieval_ldes_cache_table(sn);
retrieval_mva_cache_table(sn, init);
retrieval_nc_tables(sn);

```

Running this edition prints

```

SOLVER: SSA
Node      JobClass      List  ListCap  Items      HitProb  DelayedHitP  MissProb      ArvR
ResidT
Cache      InitClass          0      1      3      0.441096  0.146871  0.412033  0.558904
nan

SOLVER: LDES
Node      JobClass      List  ListCap  Items      HitProb  DelayedHitP  MissProb      ArvR
ResidT
Cache      InitClass          0      1      3      0.436541  0.144565  0.418893  0.563459
0.837526

SOLVER: MVA
Node      JobClass      ListCap  Items  Hit+DelayedHitProb  MissProb
Cache      InitClass          1      3      0.570337  0.429663
N/A: HitProb and DelayedHitProb separately -- the C++ open delayed-hit MVA analyzer returns only the

```

metric table, whose hit-class throughput sums the two									
SOLVER: NC									
Node	ResidT	JobClass	List	ListCap	Items	HitProb	DelayedHitP	MissProb	ArvR
Cache	InitClass		0	1	3	0.437293	0.144658	0.418049	0.562707
	nan								
Cache	InitClass		1	nan	3	0.437293	nan	nan	nan
	nan								
Item table (NC):									
Item	Miss	List1							
1	0.298607	0.543464							
2	0.530856	0.327804							
3	0.796284	0.128733							

The rows repeat for each of the 4 solvers the script runs (SSA, LDES, MVA, NC), which is the point of the example: the same model read by methods of different cost and different exactness.

2.8.14 retrieval_default

The same system with a PS backing store and the default retrieval parameters.

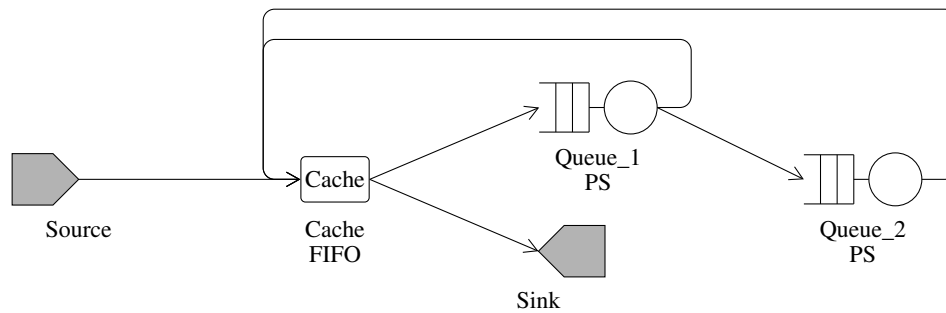


Figure 2.93: Topology of retrieval_default: an open network over 2 queueing stations with a cache node.

Nodes	5: Source, Queue $\times$ 2, Cache, Sink
Classes	6: InitClass (open), HitClass (open), MissClass (open), InitClass_retrievalClass_1 (open), InitClass_retrievalClass_2 (open), InitClass_retrievalClass_3 (open)
Scheduling	Queue_1: PS; Queue_2: PS
Arrivals	Source – InitClass: Exp(1); HitClass: Disabled; MissClass: Disabled; InitClass_retrievalClass_1: Disabled; InitClass_retrievalClass_2: Disabled; InitClass_retrievalClass_3: Disabled
Service	Queue_1 – InitClass: Exp(2); HitClass: Disabled; MissClass: Disabled; InitClass_retrievalClass_1: Exp(5); InitClass_retrievalClass_2: Exp(2); InitClass_retrievalClass_3: Exp(2) Queue_2 – InitClass: Exp(3); HitClass: Disabled; MissClass: Disabled; InitClass_retrievalClass_1: Exp(3); InitClass_retrievalClass_2: Exp(3); InitClass_retrievalClass_3: Exp(3)
Features	cache: FIFO replacement, 3 items, capacity 1; drop rule {'HitClass': 'waitingQueue', 'MissClass': 'waitingQueue'} at Queue_1; drop rule {'HitClass': 'waitingQueue', 'MissClass': 'waitingQueue'} at Queue_2; class switching on 4 links
Solvers	SSA, LDES, MVA, NC

Table 2.93: Features of retrieval_default.

The example is built and solved as follows.

```
const std::vector<double> access{0.6, 0.3, 0.1};
const std::size_t n = access.size();

Net net("DelayedHits");
Source source(net, "Source");
qn::CacheParam<double> ch;
```

```

ch.nitems = n;
ch.itemcap = std::vector<int>{1};
ch.replacestrat = ReplacementStrategy::FIFO;
ch.pread = std::vector<std::vector<double>>{access, {}, {}};
ch.hitclass = std::vector<std::size_t>{2, 0, 0};
ch.missclass = std::vector<std::size_t>{3, 0, 0};
const std::size_t cache = net.add_cache("Cache", ch);
Queue q1(net, "Queue_1", SchedStrategy::PS);
Queue q2(net, "Queue_2", SchedStrategy::PS);
Sink sink(net, "Sink");

OpenClass init(net, "InitClass");
OpenClass hit(net, "HitClass");
OpenClass miss(net, "MissClass");
source.set_arrival(init, Exp(1.0));
q1.set_service(init, Exp(2.0));
q2.set_service(init, Exp(3.0));
net.set_retrieval_system(cache, init, miss, std::vector<std::size_t>(q1, q2));
const std::vector<std::size_t> rc = retrieval_classes(miss, n);

... (14 source lines omitted; full implementation: cpp/examples/basic/)

const Sn& sn = net.get_struct();
retrieval_ssa_cache_table(sn);
retrieval_ldes_cache_table(sn);
retrieval_mva_cache_table(sn, init);
retrieval_nc_tables(sn);

```

Running this edition prints

SOLVER: SSA									
Node	JobClass	List	ListCap	Items	HitProb	DelayedHitP	MissProb	ArvR	
Cache	InitClass	0	1	3	0.465442	0.0774529	0.457105	0.534558	
	nan								

SOLVER: LDES									
Node	JobClass	List	ListCap	Items	HitProb	DelayedHitP	MissProb	ArvR	
Cache	InitClass	0	1	3	0.463086	0.0786008	0.458313	0.536914	
	0.575359								

SOLVER: MVA						
Node	JobClass	ListCap	Items	Hit+DelayedHitProb	MissProb	
Cache	InitClass	1	3	0.50853	0.49147	

N/A: HitProb and DelayedHitProb separately -- the C++ open delayed-hit MVA analyzer returns only the metric table, whose hit-class throughput sums the two

SOLVER: NC									
Node	JobClass	List	ListCap	Items	HitProb	DelayedHitP	MissProb	ArvR	
Cache	InitClass	0	1	3	0.463832	0.0784562	0.457711	0.536168	
	nan								
Cache	InitClass	1	nan	3	0.463832	nan	nan	nan	
	nan								

Item table (NC):		
Item	Miss	List1
1	0.339213	0.617368
2	0.574401	0.275742
3	0.818634	0.10689

The rows repeat for each of the 4 solvers the script runs (SSA, LDES, MVA, NC), which is the point of the example: the same model read by methods of different cost and different exactness.

2.8.15 retrieval_ps

The PS variant, with the delayed-hit behaviour made explicit.

The example is built and solved as follows.

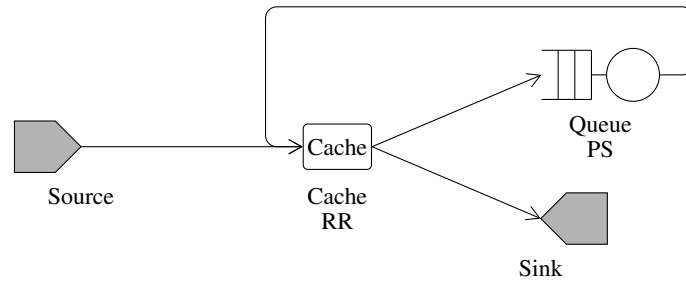


Figure 2.94: Topology of `retrieval_ps`: an open network over 1 queueing station with a cache node.

Nodes	4: Source, Queue, Cache, Sink
Classes	10: InitClass (open), HitClass (open), MissClass (open), InitClass_retrievalClass_1 (open), InitClass_retrievalClass_2 (open), InitClass_retrievalClass_3 (open), InitClass_retrievalClass_4 (open), InitClass_retrievalClass_5 (open), InitClass_retrievalClass_6 (open), InitClass_retrievalClass_7 (open)
Scheduling	Queue: PS
Arrivals	Source – InitClass: Exp(1); HitClass: Disabled; MissClass: Disabled; InitClass_retrievalClass_1: Disabled; InitClass_retrievalClass_2: Disabled; InitClass_retrievalClass_3: Disabled; InitClass_retrievalClass_4: Disabled; InitClass_retrievalClass_5: Disabled; InitClass_retrievalClass_6: Disabled; InitClass_retrievalClass_7: Disabled
Service	Queue – InitClass: Exp(1); HitClass: Disabled; MissClass: Disabled; InitClass_retrievalClass_1: Exp(1); InitClass_retrievalClass_2: Exp(1); InitClass_retrievalClass_3: Exp(1); InitClass_retrievalClass_4: Exp(1); InitClass_retrievalClass_5: Exp(1); InitClass_retrievalClass_6: Exp(1); InitClass_retrievalClass_7: Exp(1)
Features	cache: RR replacement, 7 items, capacity 6; drop rule {'HitClass': 'waitingQueue', 'MissClass': 'waitingQueue'} at Queue; class switching on 3 links
Solvers	SSA, LDES, MVA, NC

Table 2.94: Features of `retrieval_ps`.

```

const double w[7] = {49, 49, 49, 49, 7, 1, 1};
std::vector<double> access(7, 0.0);
for (std::size_t i = 0; i < 7; ++i) access[i] = w[i] / 205.0;
const std::size_t n = access.size();

Net net("PS Model");
Source source(net, "Source");
qn::CacheParam<double> ch;
ch.nitems = n;
ch.itemcap = std::vector<int>(6);
ch.replacestrat = ReplacementStrategy::RR;
ch.pread = std::vector<std::vector<double>>(access, {}, {});
ch.hitclass = std::vector<std::size_t>(2, 0, 0);
ch.missclass = std::vector<std::size_t>(3, 0, 0);
const std::size_t cache = net.add_cache("Cache", ch);
Queue queue(net, "Queue", SchedStrategy::PS);
Sink sink(net, "Sink");

OpenClass init(net, "InitClass");
OpenClass hit(net, "HitClass");
OpenClass miss(net, "MissClass");
source.set_arrival(init, Exp(1.0));
queue.set_service(init, Exp(1.0));
net.set_retrieval_system(cache, init, miss, std::vector<std::size_t>(queue));

Routing P;

... (6 source lines omitted; full implementation: cpp/examples/basic/)

const Sn& sn = net.get_struct();
retrieval_ssa_cache_table(sn);
retrieval_ldes_cache_table(sn);
retrieval_mva_cache_table(sn, init);
retrieval_nc_tables(sn);

```

Running this edition prints

SOLVER: SSA								
Node	JobClass	List	ListCap	Items	HitProb	DelayedHitP	MissProb	ArvR
Cache	InitClass	0	6	7	0.984482	0.00196457	0.0135531	0.0155176
	nan							

SOLVER: LDES								
Node	JobClass	List	ListCap	Items	HitProb	DelayedHitP	MissProb	ArvR
Cache	InitClass	0	6	7	0.983265	0.00205473	0.0146804	0.0167351
	0.989232							

SOLVER: MVA					
Node	JobClass	ListCap	Items	Hit+DelayedHitProb	MissProb
Cache	InitClass	6	7	0.979174	0.0208264

N/A: HitProb and DelayedHitProb separately -- the C++ open delayed-hit MVA analyzer returns only the metric table, whose hit-class throughput sums the two

SOLVER: NC								
Node	JobClass	List	ListCap	Items	HitProb	DelayedHitP	MissProb	ArvR
Cache	InitClass	0	6	7	0.982722	0.00215973	0.0151181	0.0172779
	nan							
Cache	InitClass	1	nan	7	0.982722	nan	nan	nan
	nan							

Item table (NC):

Item	Miss	List1
1	0.00903561	0.988805
2	0.00903561	0.988805
3	0.00903561	0.988805
4	0.00903561	0.988805
5	0.0632493	0.934591
6	0.442745	0.555095
7	0.442745	0.555095

The rows repeat for each of the 4 solvers the script runs (SSA, LDES, MVA, NC), which is the point of

the example: the same model read by methods of different cost and different exactness.

2.8.16 retrieval_routing

Retrieval with routing that separates the hit and miss paths.

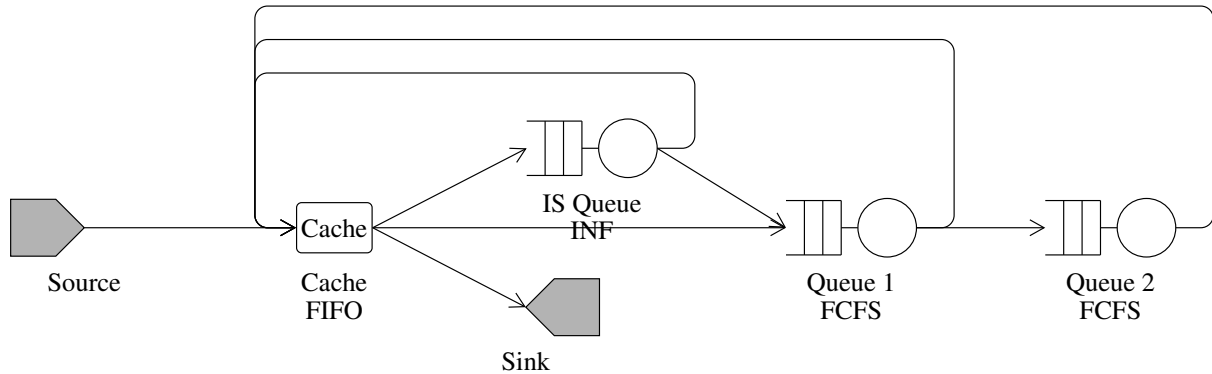


Figure 2.95: Topology of retrieval_routing: an open network over 3 queueing stations with a cache node.

Nodes	6: Source, Queue $\times$ 3, Cache, Sink
Classes	6: InitClass (open), HitClass (open), MissClass (open), InitClass_retrievalClass_1 (open), InitClass_retrievalClass_2 (open), InitClass_retrievalClass_3 (open)
Scheduling	IS Queue: INF; Queue 1: FCFS; Queue 2: FCFS
Arrivals	Source – InitClass: Exp(1); HitClass: Disabled; MissClass: Disabled; InitClass_retrievalClass_1: Disabled; InitClass_retrievalClass_2: Disabled; InitClass_retrievalClass_3: Disabled
Service	IS Queue – InitClass: Exp(2); HitClass: Disabled; MissClass: Disabled; InitClass_retrievalClass_1: Exp(2); InitClass_retrievalClass_2: Exp(2); InitClass_retrievalClass_3: Exp(2) Queue 1 – InitClass: Exp(3); HitClass: Disabled; MissClass: Disabled; InitClass_retrievalClass_1: Exp(3); InitClass_retrievalClass_2: Exp(3); InitClass_retrievalClass_3: Exp(3) Queue 2 – InitClass: Exp(3); HitClass: Disabled; MissClass: Disabled; InitClass_retrievalClass_1: Exp(3); InitClass_retrievalClass_2: Exp(3); InitClass_retrievalClass_3: Exp(3)
Features	cache: FIFO replacement, 3 items, capacity 2; drop rule {'HitClass': 'waitingQueue', 'MissClass': 'waitingQueue'} at IS Queue; drop rule {'HitClass': 'waitingQueue', 'MissClass': 'waitingQueue'} at Queue 1; drop rule {'HitClass': 'waitingQueue', 'MissClass': 'waitingQueue'} at Queue 2; class switching on 5 links
Solvers	SSA, LDES, MVA, NC

Table 2.95: Features of retrieval_routing.

The example is built and solved as follows.

```
const std::vector<double> access{0.6, 0.3, 0.1};
const std::size_t n = access.size();

Net net("Probabilistic Routing");
Source source(net, "Source");
qn::CacheParam<double> ch;
ch.nitems = n;
ch.itemcap = std::vector<int>{2};
ch.replacestrat = ReplacementStrategy::FIFO;
ch.pread = std::vector<std::vector<double>>{access, {}, {}};
ch.hitclass = std::vector<std::size_t>{2, 0, 0};
ch.missclass = std::vector<std::size_t>{3, 0, 0};
const std::size_t cache = net.add_cache("Cache", ch);
Queue isq(net, "IS Queue", SchedStrategy::INF);
Queue ql(net, "Queue 1", SchedStrategy::FCFS);
```

```

Queue q2(net, "Queue 2", SchedStrategy::FCFS);
Sink sink(net, "Sink");
const std::vector<std::size_t> queues{isq, q1, q2};

OpenClass init(net, "InitClass");
OpenClass hit(net, "HitClass");
OpenClass miss(net, "MissClass");
source.set_arrival(init, Exp(1.0));
isq.set_service(init, Exp(2.0));
q1.set_service(init, Exp(3.0));
q2.set_service(init, Exp(3.0));

... (33 source lines omitted; full implementation: cpp/examples/basic/)

// Keep 1e5: the SSA estimator's ~2.4% rel sd here is absorbed by a widened gate on
// the retrieval_routing/SSA golden cell, because 1e6 costs MATLAB ~4h a row.
retrieval_ssa_cache_table(sn);
retrieval_ldes_cache_table(sn);
retrieval_mva_cache_table(sn, init);
retrieval_nc_tables(sn);

```

Running this edition prints

```

SOLVER: SSA
Node      JobClass      List  ListCap  Items      HitProb  DelayedHitP      MissProb      ArvR
  ResidT
Cache      InitClass      0      2      3      0.783788  0.0391277      0.177084      0.216212
  nan

SOLVER: LDES
Node      JobClass      List  ListCap  Items      HitProb  DelayedHitP      MissProb      ArvR
  ResidT
Cache      InitClass      0      2      3      0.783847  0.0385158      0.177637      0.216153
  0.614128

SOLVER: MVA
Node      JobClass      ListCap  Items  Hit+DelayedHitProb      MissProb
Cache      InitClass      2      3      0.795994      0.204006
N/A: HitProb and DelayedHitProb separately -- the C++ open delayed-hit MVA analyzer returns only the
metric table, whose hit-class throughput sums the two

SOLVER: NC
Node      JobClass      List  ListCap  Items      HitProb  DelayedHitP      MissProb      ArvR
  ResidT
Cache      InitClass      0      2      3      0.783523  0.0386644      0.177813      0.216477
  nan
Cache      InitClass      1      nan      3      0.783523      nan      nan      nan
  nan

Item table (NC):
Item      Miss      List1
1      0.0987849      0.858639
2      0.19757      0.771016
3      0.59271      0.370345

```

The rows repeat for each of the 4 solvers the script runs (SSA, LDES, MVA, NC), which is the point of the example: the same model read by methods of different cost and different exactness.

2.8.17 retrieval_simple

A retrieval system: a cache in front of an infinite-server backing store, hits served at once and misses fetched.

The example is built and solved as follows.

```

const std::vector<double> access{0.6, 0.3, 0.1};
const std::size_t n = access.size();
Net net("Simple Model");
Source source(net, "Source");
qn::CacheParam<double> ch;
ch.nitems = n;
ch.itemcap = std::vector<int>{1};
ch.replacestrat = ReplacementStrategy::FIFO;

```

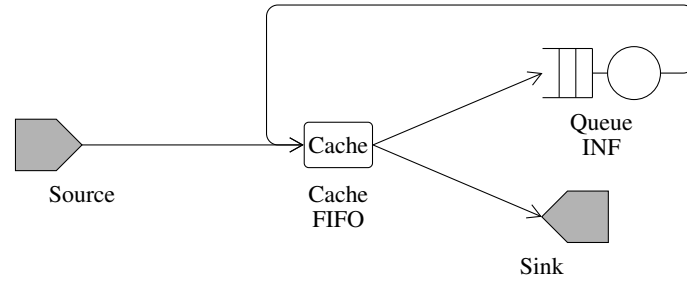



Figure 2.96: Topology of `retrieval_simple`: an open network over 1 queueing station with a cache node.

Nodes	4: Source, Queue, Cache, Sink
Classes	6: InitClass (open), HitClass (open), MissClass (open), InitClass_retrievalClass_1 (open), InitClass_retrievalClass_2 (open), InitClass_retrievalClass_3 (open)
Scheduling	Queue: INF
Arrivals	Source – InitClass: Exp(1); HitClass: Disabled; MissClass: Disabled; InitClass_retrievalClass_1: Disabled; InitClass_retrievalClass_2: Disabled; InitClass_retrievalClass_3: Disabled
Service	Queue – InitClass: Exp(2); HitClass: Disabled; MissClass: Disabled; InitClass_retrievalClass_1: Exp(2); InitClass_retrievalClass_2: Exp(2); InitClass_retrievalClass_3: Exp(2)
Features	cache: FIFO replacement, 3 items, capacity 1; drop rule {'HitClass': 'waitingQueue', 'MissClass': 'waitingQueue'} at Queue; class switching on 3 links
Solvers	SSA, LDES, MVA, NC

Table 2.96: Features of `retrieval_simple`.

```

ch.pread = std::vector<std::vector<double>>{access, {}, {}};
ch.hitclass = std::vector<std::size_t>{2, 0, 0};
ch.missclass = std::vector<std::size_t>{3, 0, 0};
const std::size_t cache = net.add_cache("Cache", ch);
Queue queue(net, "Queue", SchedStrategy::INF);
Sink sink(net, "Sink");

OpenClass init(net, "InitClass");
OpenClass hit(net, "HitClass");
OpenClass miss(net, "MissClass");
source.set_arrival(init, Exp(1.0));
queue.set_service(init, Exp(2.0));
net.set_retrieval_system(cache, init, miss, std::vector<std::size_t>(queue));

Routing P;
P.set(init, init, source, cache, 1.0);
P.set(init, init, cache, queue, 1.0);
P.set(init, init, queue, cache, 1.0);

... (3 source lines omitted; full implementation: cpp/examples/basic/)

const Sn& sn = net.get_struct();
retrieval_ssa_cache_table(sn);
retrieval_ldes_cache_table(sn);
retrieval_mva_cache_table(sn, init);
retrieval_nc_tables(sn);

```

Running this edition prints

```

SOLVER: SSA
Node      JobClass      List  ListCap  Items      HitProb  DelayedHitP      MissProb      ArvR
  ResidT
Cache      InitClass        0      1      3      0.449431  0.0928304      0.457739      0.550569
      nan

SOLVER: LDES
Node      JobClass      List  ListCap  Items      HitProb  DelayedHitP      MissProb      ArvR

```

```

ResidT
Cache      InitClass      0      1      3      0.446722      0.0913677      0.46191      0.553278
0.499896

SOLVER: MVA
Node      JobClass      ListCap  Items  Hit+DelayedHitProb      MissProb
Cache      InitClass      1      3      0.513818      0.486182
N/A: HitProb and DelayedHitProb separately -- the C++ open delayed-hit MVA analyzer returns only the
metric table, whose hit-class throughput sums the two

SOLVER: NC
Node      JobClass      List  ListCap  Items      HitProb  DelayedHitP      MissProb      ArvR
ResidT
Cache      InitClass      0      1      3      0.446046      0.091157      0.462797      0.553954
nan
Cache      InitClass      1      nan      3      0.446046      nan      nan      nan
nan
Item table (NC):
Item      Miss      List1
1      0.335021      0.564472
2      0.592131      0.319049
3      0.841449      0.116478

```

The rows repeat for each of the 4 solvers the script runs (SSA, LDES, MVA, NC), which is the point of the example: the same model read by methods of different cost and different exactness.

2.9 Cluster models

A cluster is a dispatcher in front of a pool of servers. It is not a new node type: the shorthand builds a Router and a set of stations, and the interest is in the dispatching policy and in how the response time scales with the pool size.

2.9.1 cl_basic

A dispatcher in front of a pool of identical servers, built with the cluster shorthand.

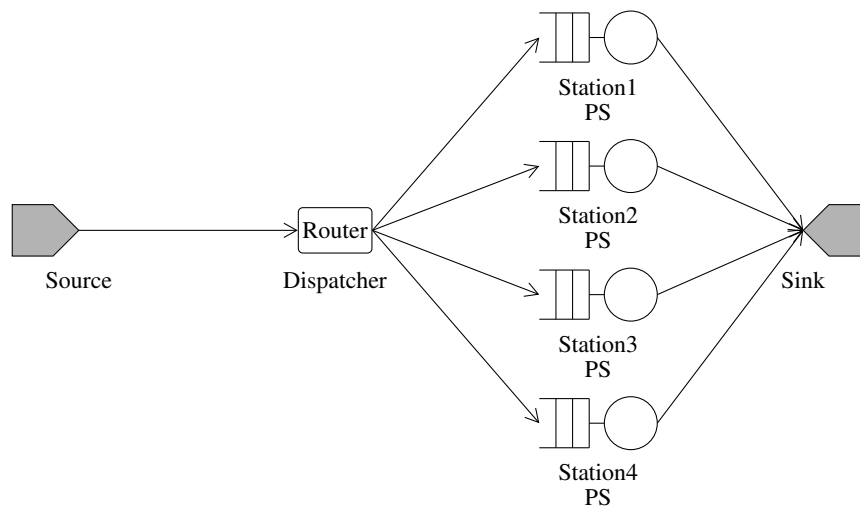


Figure 2.97: Topology of `cl_basic`: an open network over 4 queueing stations with a router.

The example is built and solved as follows.

```

const std::vector<double> lambda(1, 0.4);
const Demands d = uniform_demands(4, 1.0);
Net model = cluster_open(lambda, d, same_sched(4, SchedStrategy::PS), single_servers(4),
    RoutingStrategy::RAND);
print_avg(solve_avg("MVA", model));

```

Nodes	7: Source, Queue $\times$ 4, Router, Sink			
Classes	1: Class1 (open)			
Scheduling	Station1: PS; Station2: PS; Station3: PS; Station4: PS			
Arrivals	Source – Class1: Exp(0.4)			
Service	Station1 – Class1: Exp(1)	Station2 – Class1: Exp(1)	Station3 – Class1: Exp(1)	Station4 – Class1: Exp(1)
Features	class switching on 7 links			
Solvers	MVA			

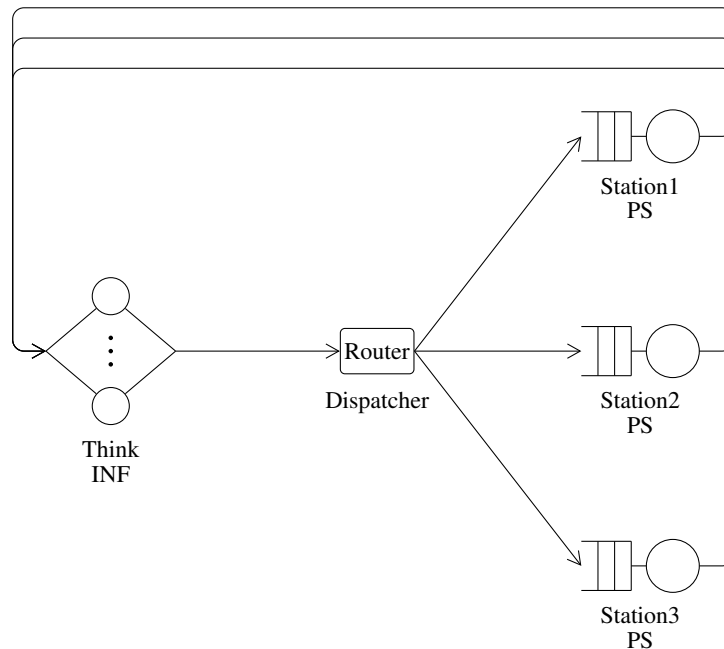
Table 2.97: Features of `cl_basic`.

Running this edition prints

SolverMVA method=egflin						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Source	Class1	0	0	0	0	0
0.4						
Station1	Class1	0.10977	0.1	1.0977	0.274425	0.1
0.1						
Station2	Class1	0.10977	0.1	1.0977	0.274425	0.1
0.1						
Station3	Class1	0.10977	0.1	1.0977	0.274425	0.1
0.1						
Station4	Class1	0.10977	0.1	1.0977	0.274425	0.1
0.1						

2.9.2 `cl_closed`

The closed version, with a fixed population of requests.

Figure 2.98: Topology of `cl_closed`: a closed network over 4 queueing stations with a router.

The example is built and solved as follows.

```
const std::vector<double> N(1, 8.0), Z(1, 1.0);
const Demands d = uniform_demands(3, 1.0);
Net model = cluster_closed(N, Z, d, same_sched(3, SchedStrategy::PS), single_servers(3),
    RoutingStrategy::RAND);
```

Nodes	5: Queue $\times$ 3, Delay, Router
Classes	1: Class1 (closed, $N = 8$)
Scheduling	Think: INF; Station1: PS; Station2: PS; Station3: PS
Service	Think – Class1: Exp(1) Station1 – Class1: Exp(1) Station2 – Class1: Exp(1) Station3 – Class1: Exp(1)
Features	class switching on 5 links
Solvers	MVA

Table 2.98: Features of `cl_closed`.

```
print_avg(solve_avg("MVA", model));
```

Running this edition prints

SolverMVA method=exact						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Think	Class1	2.19978	2.19978	1	1	2.19978
2.19978						
Station1	Class1	1.93341	0.733261	2.63672	0.878907	0.733261
0.733261						
Station2	Class1	1.93341	0.733261	2.63672	0.878907	0.733261
0.733261						
Station3	Class1	1.93341	0.733261	2.63672	0.878907	0.733261
0.733261						

2.9.3 `cl_compare`

RAND against round-robin dispatching on one cluster.

No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

Solvers	SSA
----------------	-----

Table 2.99: Features of `cl_compare`.

The example is built and solved as follows.

```
const std::vector<double> lambda(1, 1.0);
const Demands d = uniform_demands(4, 1.0 / 0.4);

SolverOpts ssa;
ssa.seed = 23000;
ssa.samples = 2000;

const RoutingStrategy policies[2] = {RoutingStrategy::RAND, RoutingStrategy::RROBIN};
const char* policy_name[2] = {"RoutingStrategy.RAND", "RoutingStrategy.RROBIN"};
for (std::size_t p = 0; p < 2; ++p) {
    std::printf("\n=== Dispatching: %s ===\n", policy_name[p]);
    Net model = cluster_open(lambda, d, same_sched(4, SchedStrategy::PS), single_servers(4),
                             policies[p]);
    print_avg(solve_avg("SSA", model, ssa));
}
```

Running this edition prints

```
=== Dispatching: RoutingStrategy.RAND ===
SolverSSA method=nrm
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
Tput
Source      Class1      0          0          0          0          0
1
```

Station1 0.272376	Class1	2.36347	0.680939	8.67725	2.16931	0.25
Station2 0.203399	Class1	0.915806	0.508497	4.50252	1.12563	0.25
Station3 0.256631	Class1	1.46177	0.641578	5.696	1.424	0.25
Station4 0.270701	Class1	1.96221	0.676753	7.24863	1.81216	0.25
=== Dispatching: RoutingStrategy.RROBIN === SolverSSA method=nrm						
Station Tput	JobClass	QLen	Util	RespT	ResidT	ArvR
Source 1	Class1	0	0	0	0	0
Station1 0.239019	Class1	0.937304	0.597547	3.92146	0.980366	0.25
Station2 0.219365	Class1	0.807772	0.548411	3.68233	0.920582	0.25
Station3 0.194489	Class1	0.691625	0.486223	3.55611	0.889028	0.25
Station4 0.228052	Class1	1.00852	0.570131	4.42232	1.10558	0.25

2.9.4 cl_multiclass

Several request classes through one cluster.

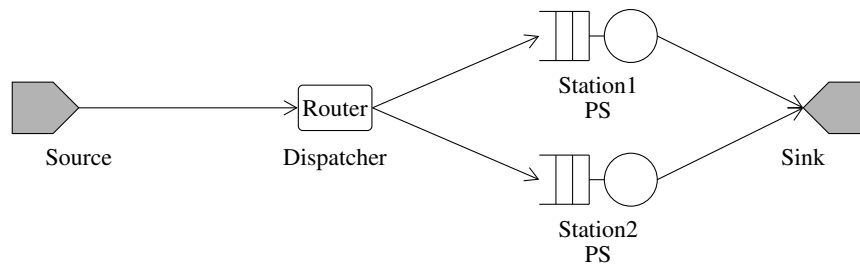


Figure 2.99: Topology of `cl_multiclass`: an open network over 2 queueing stations with a router.

Nodes	5: Source, Queue × 2, Router, Sink
Classes	2: Class1 (open), Class2 (open)
Scheduling	Station1: PS; Station2: PS
Arrivals	Source – Class1: Exp(0.3); Class2: Exp(0.2)
Service	Station1 – Class1: Exp(1); Class2: Exp(2) Station2 – Class1: Exp(1); Class2: Exp(2)
Features	class switching on 5 links
Solvers	MVA

Table 2.100: Features of `cl_multiclass`.

The example is built and solved as follows.

```

std::vector<double> lambda;
lambda.push_back(0.3);
lambda.push_back(0.2);
Demands d(2, std::vector<double>());
for (std::size_t i = 0; i < 2; ++i) {
    d[i].push_back(1.0);
    d[i].push_back(0.5);
}
Net model = cluster_open(lambda, d, same_sched(2, SchedStrategy::PS), single_servers(2),
    RoutingStrategy::RAND);
print_avg(solve_avg("MVA", model));
  
```

Running this edition prints

SolverMVA method=egflin						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Source	Class1	0	0	0	0	0
0.3						
Source	Class2	0	0	0	0	0
0.2						
Station1	Class1	0.187105	0.15	1.24737	0.623683	0.15
0.15						
Station1	Class2	0.0623683	0.05	0.623683	0.311842	0.1
0.1						
Station2	Class1	0.187105	0.15	1.24737	0.623683	0.15
0.15						
Station2	Class2	0.0623683	0.05	0.623683	0.311842	0.1
0.1						

2.9.5 cl_scheduling

The same cluster under FCFS and under PS.

No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

Solvers	SSA
----------------	-----

Table 2.101: Features of `cl_scheduling`.

The example is built and solved as follows.

```
const std::size_t M = 3;
const double lambda = 0.9, mu = 0.5, scv = 4.0;

SolverOpts ssa;
ssa.seed = 23000;
ssa.samples = 20000;

const SchedStrategy disciplines[2] = {SchedStrategy::FCFS, SchedStrategy::PS};
const char* discipline_name[2] = {"FCFS", "PS"};
for (std::size_t k = 0; k < 2; ++k) {
    std::printf("\n=== Scheduling: %s ===\n", discipline_name[k]);
    Net model = cluster_open(std::vector<double>(1, lambda), uniform_demands(M, 1.0 / mu),
                           same_sched(M, disciplines[k]), single_servers(M),
                           RoutingStrategy::RAND, scv);
    print_avg(solve_avg("SSA", model, ssa));
}
```

Running this edition prints

```
=== Scheduling: FCFS ===
SolverSSA method=nrm
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
Tput
Source       Class1      0          0          0          0          0
0.9
Station1     Class1      3.68632    0.639344   12.9853    4.32844    0.3
0.283884
Station2     Class1      2.79057    0.629293   9.16608    3.05536    0.3
0.304446
Station3     Class1      2.3576     0.543987   7.83853    2.61284    0.3
0.300771

=== Scheduling: PS ===
SolverSSA method=nrm
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
Tput
Source       Class1      0          0          0          0          0
0.9
Station1     Class1      1.42936    0.599483   4.65866    1.55289    0.3
0.306818
```

Station2	Class1	1.41494	0.618972	4.4949	1.4983	0.3
0.314789						
Station3	Class1	1.30372	0.534523	4.3655	1.45517	0.3
0.298643						

2.9.6 cl_stations

Clusters whose members are declared station by station.

No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

Solvers	MVA
----------------	-----

Table 2.102: Features of `cl_stations`.

The example is built and solved as follows.

```
const double lambda = 1.6, mu = 1.0;
const std::size_t counts[4] = {2, 3, 4, 6};
for (std::size_t k = 0; k < 4; ++k) {
    const std::size_t M = counts[k];
    Net model = cluster_open(std::vector<double>(1, lambda), uniform_demands(M, 1.0 / mu),
                             same_sched(M, SchedStrategy::PS), single_servers(M),
                             RoutingStrategy::RAND);
    const AvgTable t = solve_avg("MVA", model);
    double sum_respt = 0.0, max_util = 0.0;
    std::size_t n = 0;
    for (std::size_t i = 0; i < t.Station.size(); ++i) {
        if (t.Station[i] == "Source") continue;
        sum_respt += t.RespT[i];
        max_util = std::max(max_util, t.Util[i]);
        ++n;
    }
    std::printf("\n=== M = %zu (mean RespT %.4f, max Util %.4f) ===\n", M,
               n ? sum_respt / static_cast<double>(n) : 0.0, max_util);
    print_avg(t);
}
```

Running this edition prints

```
=== M = 2 (mean RespT 5.0000, max Util 0.8000) ===
SolverMVA method=egflin
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
  Tput
Source       Class1          0          0          0          0          0
  1.6
Station1     Class1          4          0.8          5          2.5          0.8
  0.8
Station2     Class1          4          0.8          5          2.5          0.8
  0.8

=== M = 3 (mean RespT 2.1429, max Util 0.5333) ===
SolverMVA method=egflin
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
  Tput
Source       Class1          0          0          0          0          0
  1.6
Station1     Class1          1.14286    0.533333    2.14286    0.714285    0.533333
  0.533333
Station2     Class1          1.14286    0.533333    2.14286    0.714285    0.533333
  0.533333
Station3     Class1          1.14286    0.533333    2.14286    0.714285    0.533333
  0.533333

=== M = 4 (mean RespT 1.6667, max Util 0.4000) ===
... (10 captured-output lines omitted; rerun the source example for the full output)
```

Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Source	Class1	0	0	0	0	0
1.6						
Station1	Class1	0.363511	0.266667	1.36317	0.227194	0.266667
0.266667						
Station2	Class1	0.363511	0.266667	1.36317	0.227194	0.266667
0.266667						
Station3	Class1	0.363511	0.266667	1.36317	0.227194	0.266667
0.266667						
Station4	Class1	0.363511	0.266667	1.36317	0.227194	0.266667
0.266667						
Station5	Class1	0.363511	0.266667	1.36317	0.227194	0.266667
0.266667						
Station6	Class1	0.363511	0.266667	1.36317	0.227194	0.266667
0.266667						

2.9.7 cl_sweep

A sweep over the cluster size, and the response time curve it traces.

No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

Solvers	MVA
---------	-----

Table 2.103: Features of `cl_sweep`.

The example is built and solved as follows.

```
const Demands d = uniform_demands(2, 1.0);
const double rates[4] = {0.2, 0.5, 0.9, 1.5};
for (std::size_t k = 0; k < 4; ++k) {
    std::printf("\n=== lambda = %g ===\n", rates[k]);
    const std::vector<double> lambda(1, rates[k]);
    Net model = cluster_open(lambda, d, same_sched(2, SchedStrategy::PS), single_servers(2),
                             RoutingStrategy::RAND);
    print_avg(solve_avg("MVA", model));
}
```

Running this edition prints

```
=== lambda = 0.2 ===
SolverMVA method=egflin
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
Tput
Source       Class1      0          0          0          0          0
0.2
Station1     Class1      0.10977    0.1        1.0977     0.548849    0.1
0.1
Station2     Class1      0.10977    0.1        1.0977     0.548849    0.1
0.1

=== lambda = 0.5 ===
SolverMVA method=egflin
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
Tput
Source       Class1      0          0          0          0          0
0.5
Station1     Class1      0.333144   0.25       1.33258    0.666288    0.25
0.25
Station2     Class1      0.333144   0.25       1.33258    0.666288    0.25
0.25

=== lambda = 0.9 ===
SolverMVA method=egflin
... (3 captured-output lines omitted; rerun the source example for the full output)
```


Station2 0.45	Class1	0.818181	0.45	1.81818	0.90909	0.45
=== lambda = 1.5 ===						
SolverMVA method=egflin						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Source	Class1	0	0	0	0	0
1.5						
Station1	Class1	3	0.75	4	2	0.75
0.75						
Station2	Class1	3	0.75	4	2	0.75
0.75						

2.9.8 dispatch_closed

The closed version.

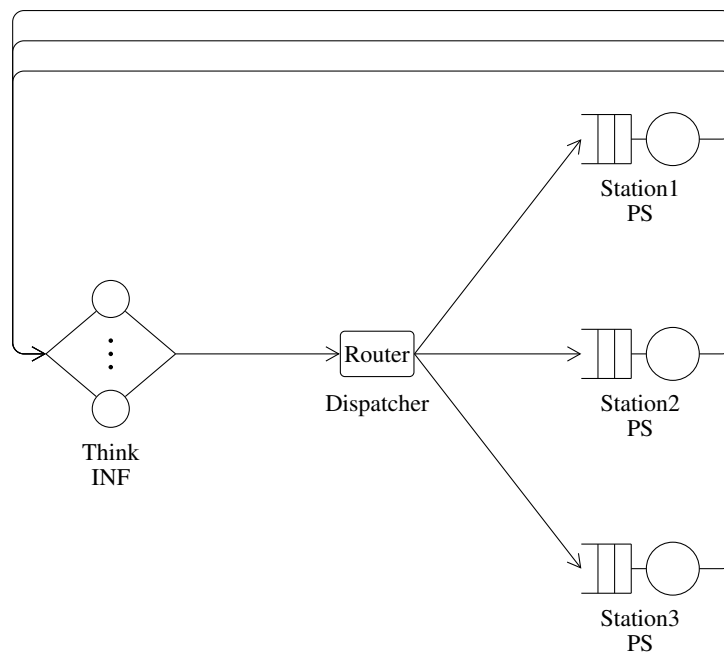


Figure 2.100: Topology of dispatch_closed: a closed network over 4 queueing stations with a router.

Nodes	5: Queue $\times$ 3, Delay, Router
Classes	1: Class1 (closed, $N = 10$)
Scheduling	Think: INF; Station1: PS; Station2: PS; Station3: PS
Service	Think – Class1: Exp(1) Station1 – Class1: Exp(1) Station2 – Class1: Exp(1) Station3 – Class1: Exp(1)
Features	class switching on 5 links
Solvers	MVA, SSA

Table 2.104: Features of dispatch_closed.

The example is built and solved as follows.

```

const std::vector<double> N(1, 10.0), Z(1, 1.0);
const Demands d = uniform_demands(3, 1.0);
Net model = cluster_closed(N, Z, d, same_sched(3, SchedStrategy::PS), single_servers(3),
                           RoutingStrategy::RAND);
print_avg(solve_avg("MVA", model));

```

```
SolverOpts ssa;
ssa.seed = 23000;
ssa.samples = 20000;
print_avg(solve_avg("SSA", model, ssa));
```

Running this edition prints

SolverMVA method=exact						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Think	Class1	2.35999	2.35999	1	1	2.35999
2.35999						
Station1	Class1	2.54667	0.786664	3.2373	1.0791	0.786664
0.786664						
Station2	Class1	2.54667	0.786664	3.2373	1.0791	0.786664
0.786664						
Station3	Class1	2.54667	0.786664	3.2373	1.0791	0.786664
0.786664						
SolverSSA method=nrm						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Think	Class1	2.3439	2.3439	1	1	2.3578
2.3439						
Station1	Class1	2.73716	0.794505	3.44512	1.14837	0.781299
0.794505						
Station2	Class1	2.33698	0.762899	3.06329	1.0211	0.781299
0.762899						
Station3	Class1	2.58195	0.800401	3.22583	1.07528	0.781299
0.800401						

2.9.9 dispatch_mixed

The mixed version, open and closed classes through the same dispatcher.

No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

Solvers	MVA, LDES
----------------	-----------

Table 2.105: Features of `dispatch_mixed`.

The example is built and solved as follows.

```
const std::vector<double> lambda(1, 0.5), N(1, 3.0), Z(1, 1.0);
Demands d(2, std::vector<double>());
for (std::size_t i = 0; i < 2; ++i) {
    d[i].push_back(1.0 / 2.0); // open class, mu = 2.0
    d[i].push_back(1.0 / 1.5); // closed class, mu = 1.5
}
Net model = cluster_mixed(lambda, N, Z, d, same_sched(2, SchedStrategy::PS), single_servers
(2),
                        RoutingStrategy::RAND);
print_avg(solve_avg("MVA", model));

print_avg(solve_avg("LDES", model, sim_opts(23000)));
```

Running this edition prints

SolverMVA method=exact						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Source	Class1	0	0	0	0	0
0.5						
Think	Class2	1.40954	1.40954	1	1	1.40954
1.40954						
Station1	Class1	0.256461	0.125	1.02585	0.512923	0.25
0.25						

Station1	Class2	0.795229	0.469847	1.12835	0.564176	0.704771
0.704771						
Station2	Class1	0.256461	0.125	1.02585	0.512923	0.25
0.25						
Station2	Class2	0.795229	0.469847	1.12835	0.564176	0.704771
0.704771						

SOLVER: LDES

N/A: SolverLDES is the SSJ simulation engine of the JAR and has no C++ port

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by LDES.

2.9.10 dispatch_open

The dispatcher idiom written out in full, open workload.

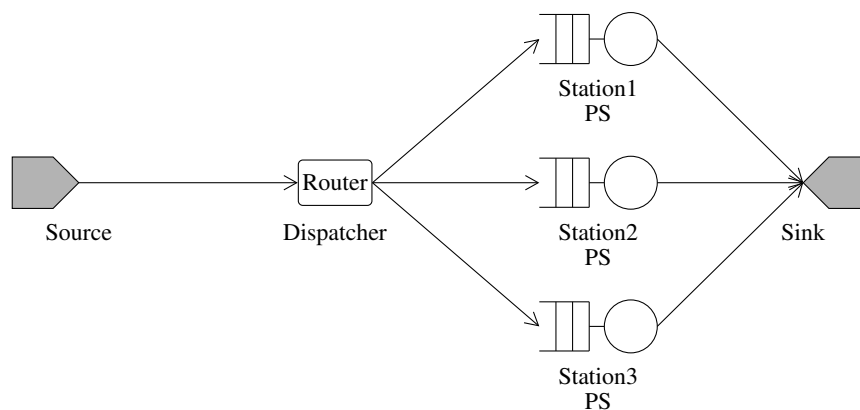


Figure 2.101: Topology of dispatch_open: an open network over 3 queueing stations with a router.

Nodes	6: Source, Queue × 3, Router, Sink
Classes	1: Class1 (open)
Scheduling	Station1: PS; Station2: PS; Station3: PS
Arrivals	Source – Class1: Exp(0.4)
Service	Station1 – Class1: Exp(1) Station2 – Class1: Exp(1) Station3 – Class1: Exp(1)
Features	class switching on 6 links
Solvers	MVA, SSA

Table 2.106: Features of dispatch_open.

The example is built and solved as follows.

```

const std::vector<double> lambda(1, 0.4);
const Demands d = uniform_demands(3, 1.0);
Net model = cluster_open(lambda, d, same_sched(3, SchedStrategy::PS), single_servers(3),
                          RoutingStrategy::RAND);
print_avg(solve_avg("MVA", model));

SolverOpts ssa;
ssa.seed = 23000;
ssa.samples = 20000;
print_avg(solve_avg("SSA", model, ssa));
  
```

Running this edition prints

SolverMVA method=egflin						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						

Source	Class1	0	0	0	0	0
0.4						
Station1	Class1	0.152633	0.133333	1.14475	0.381583	0.133333
0.133333						
Station2	Class1	0.152633	0.133333	1.14475	0.381583	0.133333
0.133333						
Station3	Class1	0.152633	0.133333	1.14475	0.381583	0.133333
0.133333						
SolverSSA method=nrm						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Source	Class1	0	0	0	0	0
0.4						
Station1	Class1	0.152855	0.133089	1.14851	0.382837	0.133333
0.133089						
Station2	Class1	0.166642	0.143651	1.16004	0.386681	0.133333
0.143651						
Station3	Class1	0.156008	0.133557	1.1681	0.389367	0.133333
0.133557						

2.10 Stochastic Petri nets

A stochastic Petri net is built from `Place` and `Transition` nodes. A transition fires when its input places hold enough tokens, consuming and producing tokens according to the arc weights; timed transitions fire after a random delay, immediate transitions fire at once, and an inhibitor arc disables a transition while its place is marked. A transition may have several firing modes, each with its own rate and arc weights.

Petri nets and queueing models are two notations for the same processes, and two of the scripts make that explicit by building the same system both ways and comparing the generators. Because the firing time need not be exponential, the family also carries the non-Markovian cases.

The figures of this section are drawn in the notation of the net rather than in that of a queueing network. A place is a circle, and the tokens or the number inside it are the marking the model starts from; a transition is a bar, hollow when it fires after a random delay and solid when it fires at once; an arc is plain when it moves one token and labelled with its multiplicity otherwise, and an arc ending in a small circle is an inhibitor, which holds its transition disabled for as long as its place is marked. A transition with more than one firing mode is named with the number of modes, since no single distribution describes it. A circle bisected by a bar is a queueing place, whose tokens are served by a scheduling station embedded in the place.

2.10.1 `spn_basic_closed`

The closed counterpart, a token circulating between two places.

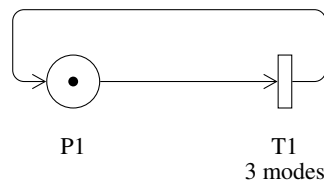


Figure 2.102: Topology of `spn_basic_closed`: a stochastic Petri net over 1 place and 1 transition; a place is a circle holding its initial marking and a transition a bar, hollow when its firing time is random and solid when it fires at once; a transition offering several firing modes is named with their number in place of a distribution.

The example is built and solved as follows.

```
void spn_basic_closed() {
```

Nodes	2: Place, Transition
Classes	1: Class1 (closed, $N = 1$)
Features	1 transition with 3 firing modes each
Solvers	JMT

Table 2.107: Features of `spn_basic_closed`.

```
Net m = spn_basic_closed_model();
print_spn(m);
section("JMT");
print_avg(m.get_struct(), jmt_avg(m, sim_opts(23000)));
```

Running this edition prints

```
MODEL: model (2 nodes, 1 classes)
Class1      Closed  population=1      priority=0
Place      P1
Transition T1      marking: Class1=1
              3 mode(s)
  Model1      TIMED      servers=1      weight=1      prio=1      Exp mean=1
enabling:    P1 x1
firing:      P1 x1
  Mode2      TIMED      servers=1      weight=1      prio=1      Erlang mean=1
enabling:    P1 x1
firing:      P1 x1
  Mode3      TIMED      servers=1      weight=1      prio=1      HyperExp mean=1
enabling:    P1 x1
firing:      P1 x1

SOLVER: JMT
Station    JobClass      QLen      Util      RespT      ResidT      ArrvR
Tput
P1         Class1        1          0          0.35308    0.35308    2.835
2.8322
```

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by JMT.

2.10.2 `spn_basic_open`

One place and one timed transition fed by an open class, the Petri net form of an M/M/1 queue.

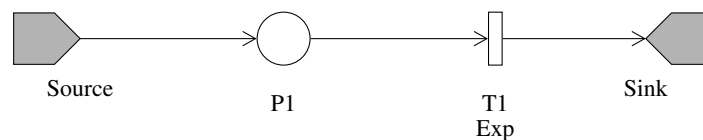


Figure 2.103: Topology of `spn_basic_open`: a stochastic Petri net over 1 place and 1 transition; a place is a circle holding its initial marking and a transition a bar, hollow when its firing time is random and solid when it fires at once; the net is open, fed by a Source and drained into a Sink.

Nodes	4: Source, Place, Transition, Sink
Classes	1: Class1 (open)
Arrivals	Source – Class1: Exp(1)
Features	1 transition with 1 firing mode each
Solvers	JMT

Table 2.108: Features of `spn_basic_open`.

The example is built and solved as follows.

```
void spn_basic_open() {
```

```

Net m = spn_basic_open_model();
print_spn(m);
section("JMT");
print_avg(m.get_struct(), jmt_avg(m, sim_opts(23000)));

```

Running this edition prints

```

MODEL: model (4 nodes, 1 classes)
Class1      Open      population=inf      priority=0
Place       P1        marking: (from the reference station)
Transition T1        1 mode(s)
Model       TIMED      servers=MaxInt      weight=1      prio=1      Exp mean=0.25
enabling:    P1 x1
firing:      Sink x1

SOLVER: JMT
Station      JobClass      QLen      Util      RespT      ResidT      ArrvR
Tput
Source       Class1        0          0          0          0          0
1.0312
P1           Class1        0.25966    0          0.2503     0.2503     1.0312
1.0427

```

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by JMT.

2.10.3 spn_closed_fourplaces

Four places with Erlang, Coxian and hyperexponential firing times.

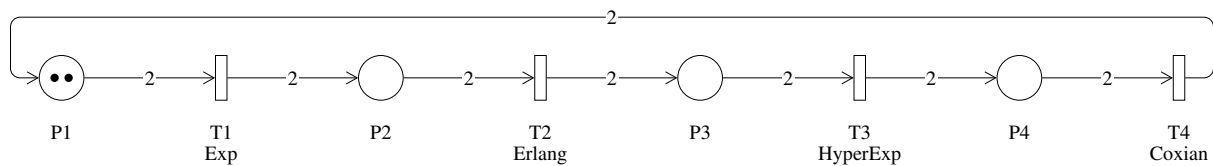


Figure 2.104: Topology of `spn_closed_fourplaces`: a stochastic Petri net over 4 places and 4 transitions; a place is a circle holding its initial marking and a transition a bar, hollow when its firing time is random and solid when it fires at once; an arc that moves more than one token carries its multiplicity.

Nodes	8: Place $\times$ 4, Transition $\times$ 4
Classes	1: Class1 (closed, $N = 2$)
Features	4 transitions with 1 firing mode each
Solvers	JMT

Table 2.109: Features of `spn_closed_fourplaces`.

The example is built and solved as follows.

```

void spn_closed_fourplaces() {
    Net m = spn_closed_fourplaces_model();
    print_spn(m);
    section("JMT");
    print_avg(m.get_struct(), jmt_avg(m, sim_opts(23000)));
}

```

Running this edition prints

```

MODEL: model (8 nodes, 1 classes)
Class1      Closed     population=2      priority=0
Place       P1         marking: Class1=2
Place       P2         marking: Class1=0
Place       P3         marking: Class1=0
Place       P4         marking: Class1=0
Transition T1          1 mode(s)

```

Model1	TIMED	servers=1	weight=1	prio=1	Exp mean=0.5	
enabling:	P1 x2					
firing:	P2 x2					
Transition T2		1 mode(s)				
Mode2	TIMED	servers=1	weight=1	prio=1	Erlang mean=1.33333	
enabling:	P2 x2					
firing:	P3 x2					
Transition T3		1 mode(s)				
Mode3	TIMED	servers=1	weight=1	prio=1	HyperExp mean=0.433333	
... (5 captured-output lines omitted; rerun the source example for the full output)						
firing:	P1 x2					
SOLVER: JMT						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
P1	Class1	0.27748	0	0.48217	0.48217	0.57461
0.57432						
P2	Class1	0.77676	0	1.3441	1.3441	0.57432
0.57273						
P3	Class1	0.25164	0	0.43035	0.43035	0.57273
0.57294						
P4	Class1	0.69386	0	1.2301	1.2301	0.57294
0.5744						

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by JMT.

2.10.4 spn_closed_twoplaces

A closed net over two places with Erlang firing times.

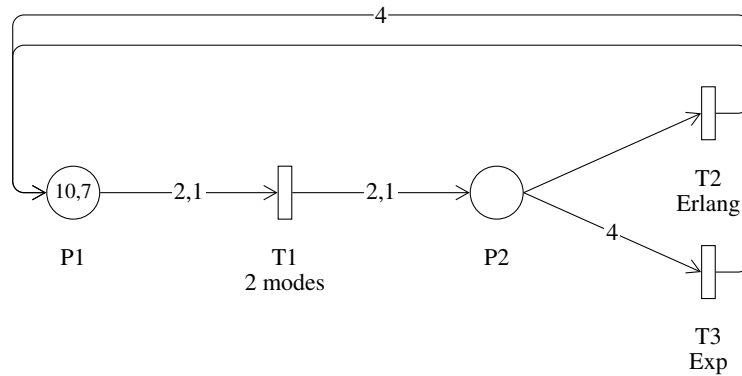


Figure 2.105: Topology of `spn_closed_twoplaces`: a stochastic Petri net over 2 places and 3 transitions; a place is a circle holding its initial marking and a transition a bar, hollow when its firing time is random and solid when it fires at once; an arc that moves more than one token carries its multiplicity; a transition offering several firing modes is named with their number in place of a distribution.

Nodes	5: Place $\times$ 2, Transition $\times$ 3
Classes	2: Class1 (closed, $N = 10$), Class2 (closed, $N = 7$)
Features	3 transitions, 1 to 2 firing modes; class switching on 2 links
Solvers	JMT

Table 2.110: Features of `spn_closed_twoplaces`.

The example is built and solved as follows.

```
Net m = spn_closed_twoplaces_model();
print_spn(m);
section("JMT");
```

```
print_avg(m.get_struct(), jmt_avg(m, sim_opts(23000)));
```

Running this edition prints

```
MODEL: model (5 nodes, 2 classes)
Class1      Closed  population=10      priority=0
Class2      Closed  population=7      priority=0
Place       P1      marking: Class1=10 Class2=7
Place       P2      marking: Class1=0 Class2=0
Transition T1      2 mode(s)
  Model      TIMED  servers=1      weight=1      prio=1      Exp mean=0.5
  enabling:   P1 x2 Class1
  firing:     P2 x2 Class1
  Mode2      TIMED  servers=1      weight=1      prio=1      Exp mean=0.333333
  enabling:   P1 x1 Class2
  firing:     P2 x1 Class2
Transition T2      1 mode(s)
  Mode3      TIMED  servers=1      weight=1      prio=1      Erlang mean=1.33333
  enabling:   P2 x1 Class1
  firing:     P1 x1 Class1

... (3 captured-output lines omitted; rerun the source example for the full output)

  firing:     P1 x4 Class2

SOLVER: JMT
Station      JobClass      QLen      Util      RespT      ResidT      ArrvR
Tput
P1           Class1      0.88109      0      1.1688      1.1688      0.7533
0.75488
P1           Class2      1.708      0      1.1162      1.1162      1.5906
1.6042
P2           Class1      9.1189      0      12.1      12.1      0.75488
0.75495
P2           Class2      5.292      0      3.3423      3.3423      1.6042
1.5821
```

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by JMT.

2.10.5 spn_colored_gspn

A colored generalized stochastic Petri net: one firing mode per token color, an immediate transition whose inhibitor arcs make the server exclusive, and firing weights that arbitrate the two colors competing for it.

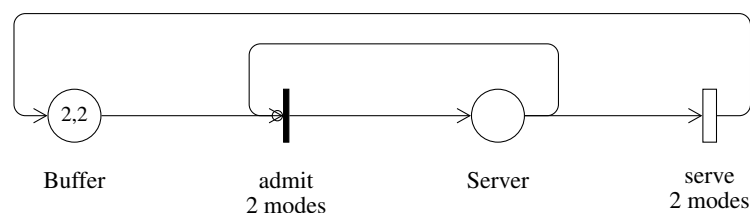


Figure 2.106: Topology of `spn_colored_gspn`: a stochastic Petri net over 2 places and 2 transitions; a place is a circle holding its initial marking and a transition a bar, hollow when its firing time is random and solid when it fires at once; the arc ending in a circle is an inhibitor and holds admit disabled while Server is marked; a transition offering several firing modes is named with their number in place of a distribution.

The example is built and solved as follows.

```
Net m = spn_colored_gspn_model();
print_spn(m);
section("CTMC");
SolverOpts ctmc;
ctmc.cutoff = 4;
```


Nodes	4: Place $\times$ 2, Transition $\times$ 2
Classes	2: Gold (closed, $N = 2$), Silver (closed, $N = 2$)
Features	2 transitions with 2 firing modes each
Solvers	CTMC, SSA

Table 2.111: Features of `spn_colored_gspn`.

```
print_avg(solve_avg("CTMC", m, ctmc));
section("SSA");
SolverOpts ssa;
ssa.samples = 200000;
ssa.seed = 23000;
print_avg(solve_avg("SSA", m, ssa));
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

2.10.6 `spn_fluid_dae`

Fluid DAE analysis of stochastic Petri nets, including invariant constraints, immediate-transition flows, bounded markings and marking covariances.

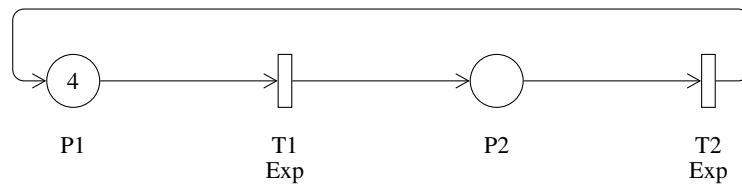


Figure 2.107: Topology of `spn_fluid_dae`: a stochastic Petri net over 2 places and 2 transitions; a place is a circle holding its initial marking and a transition a bar, hollow when its firing time is random and solid when it fires at once.

Nodes	4: Place $\times$ 2, Transition $\times$ 2
Classes	1: Class1 (closed, $N = 4$)
Features	2 transitions with 1 firing mode each
Solvers	FLD, CTMC

Table 2.112: Features of `spn_fluid_dae`.

The example is built and solved as follows.

```
note("-- a closed net whose fluid answer is EXACT --");
Net exact = spn_fluid_exact_model();
const Sn& sn_exact = exact.get_struct();
section("FLD");
print_avg_sim(sn_exact, fluid::solver_fluid_run_analyzer(sn_exact, fluid::FluidOptions()));
ctmc::CtmcOptions copt;
copt.cutoff = 6.0;
section("CTMC (cutoff = 6)");
print_avg(sn_exact, ctmc::solver_ctmc_run_analyzer(sn_exact, copt));

const fluid::petri::PetriSolution ps =
    fluid::petri::solver_fluid_petri(sn_exact, fluid::petri::PetriOptions());
std::printf("marking variance: %.6f %.6f (exact 0.96)\n", ps.petri.marking_var(0, 0),
    ps.petri.marking_var(1, 0));
std::printf("invariant \"%s\" = %g, error %.2e\n", ps.petri.invariant_label[0].c_str(),
    ps.petri.invariant_value[0], ps.petri.invariant_error[0]);

note("\n-- an immediate transition, as an algebraic flow --");
```

```

Net imm = spn_fluid_immediate_model();
const Sn& sn_imm = imm.get_struct();
section("FLD");
print_avg_sim(sn_imm, fluid::solver_fluid_run_analyzer(sn_imm, fluid::FluidOptions()));

note("\n-- the same net with the immediate transition eliminated by hand --");
Net red = spn_fluid_reduced_model();
const Sn& sn_red = red.get_struct();
section("FLD");
print_avg_sim(sn_red, fluid::solver_fluid_run_analyzer(sn_red, fluid::FluidOptions()));

```

Running this edition prints

```

-- a closed net whose fluid answer is EXACT --

SOLVER: FLD
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
  Tput
P1      4.8      Class1      2.4      2.4      0.5      0.5      4.8
P2      4.8      Class1      1.6      1.6      0.33333      0.33333      4.8

SOLVER: CTMC (cutoff = 6)
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
  Tput
P1      4.8      Class1      2.4      2.4      0.5      0.5      4.8
P2      4.8      Class1      1.6      1.6      0.33333      0.33333      4.8
marking variance: 0.960000 0.960000 (exact 0.96)
invariant "P1(class 1) + P2(class 1)" = 4, error 0.00e+00

-- an immediate transition, as an algebraic flow --

... (4 captured-output lines omitted; rerun the source example for the full output)
P3      1.9288      Class1      1.247      1.247      0.64652      0.64652      1.9288

-- the same net with the immediate transition eliminated by hand --

SOLVER: FLD
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
  Tput
P1      1.9288      Class1      2.753      2.753      1.4273      1.4273      1.9288
P3      1.9288      Class1      1.247      1.247      0.64652      0.64652      1.9288

```

The rows repeat for each of the 2 solvers the script runs (FLD, CTMC), which is the point of the example: the same model read by methods of different cost and different exactness.

2.10.7 spn_fourmodes

Four firing modes, and the mode selection probabilities.

Nodes	7: Place $\times$ 3, Transition $\times$ 4
Classes	1: Class1 (closed, $N = 8$)
Features	4 transitions with 1 firing mode each
Solvers	JMT

Table 2.113: Features of spn_fourmodes.

The example is built and solved as follows.

```

void spn_fourmodes() {
    Net m = spn_fourmodes_model();
    print_spn(m);
}

```

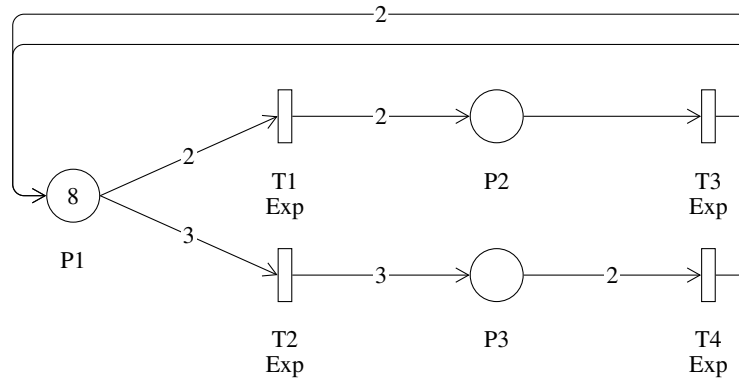


Figure 2.108: Topology of `spn_fourmodes`: a stochastic Petri net over 3 places and 4 transitions; a place is a circle holding its initial marking and a transition a bar, hollow when its firing time is random and solid when it fires at once; an arc that moves more than one token carries its multiplicity.

```
section("JMT");
print_avg(m.get_struct(), jmt_avg(m, sim_opts(23000)));
```

Running this edition prints

```
MODEL: model (7 nodes, 1 classes)
Class1      Closed  population=8      priority=0
Place      P1      marking: Class1=8
Place      P2      marking: Class1=0
Place      P3      marking: Class1=0
Transition T1      1 mode(s)
  Model     TIMED   servers=1      weight=1      prio=1      Exp mean=0.5
  enabling:  P1 x2
  firing:    P2 x2
Transition T2      1 mode(s)
  Model     TIMED   servers=1      weight=1      prio=1      Exp mean=1
  enabling:  P1 x3
  firing:    P3 x3
Transition T3      1 mode(s)
  Model     TIMED   servers=1      weight=1      prio=1      Exp mean=0.25
  enabling:  P2 x1

... (3 captured-output lines omitted; rerun the source example for the full output)

  enabling:  P3 x2
  firing:    P1 x2

SOLVER: JMT
Station    JobClass      QLen      Util      RespT      ResidT      ArrvR
Tput
P1         Class1        3.4344     0         0.72032    0.72032    4.796
P2         Class1        2.5589     0         0.86649    0.86649    2.9984
P3         Class1        1.9935     0         1.1405     1.1405     1.8044
1.742
```

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by JMT.

2.10.8 `spn_inhibiting`

An inhibitor arc, which disables a transition while its place is marked. The example is built and solved as follows.

```
void spn_inhibiting() {
  Net m = spn_inhibiting_model();
  print_spn(m);
  section("JMT");
```

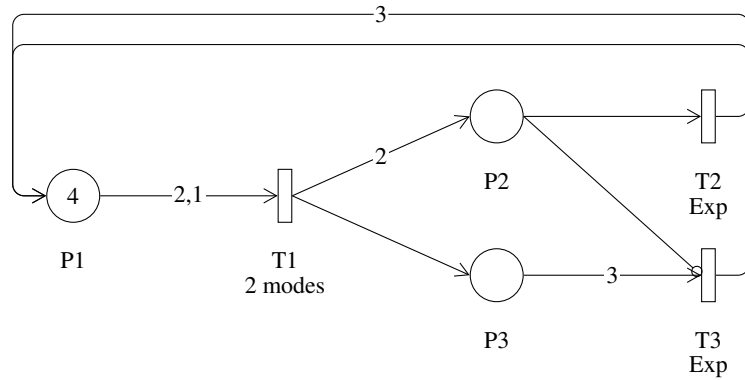


Figure 2.109: Topology of `spn_inhibiting`: a stochastic Petri net over 3 places and 3 transitions; a place is a circle holding its initial marking and a transition a bar, hollow when its firing time is random and solid when it fires at once; an arc that moves more than one token carries its multiplicity; the arc ending in a circle is an inhibitor and holds T3 disabled while P2 is marked; a transition offering several firing modes is named with their number in place of a distribution.

Nodes	6: Place $\times$ 3, Transition $\times$ 3
Classes	1: Class1 (closed, $N = 4$)
Features	3 transitions, 1 to 2 firing modes
Solvers	JMT

Table 2.114: Features of `spn_inhibiting`.

```
print_avg(m.get_struct(), jmt_avg(m, sim_opts(23000)));
```

Running this edition prints

```
MODEL: model (6 nodes, 1 classes)
  Class1      Closed  population=4      priority=0
  Place      P1      marking: Class1=4
  Place      P2      marking: Class1=0
  Place      P3      marking: Class1=0
  Transition T1      2 mode(s)
    Model     TIMED   servers=1      weight=1      prio=1      Exp mean=0.5
    enabling:  P1 x2
    firing:    P2 x2
    Mode2     TIMED   servers=1      weight=1      prio=1      Exp mean=1
    enabling:  P1 x1
    firing:    P3 x1
  Transition T2      1 mode(s)
    Mode3     TIMED   servers=1      weight=1      prio=1      Exp mean=0.25
    enabling:  P2 x1
    firing:    P1 x1

... (3 captured-output lines omitted; rerun the source example for the full output)

  inhibiting: P2 x1
  firing:     P1 x3

SOLVER: JMT
Station  Tput      JobClass      QLen      Util      RespT      ResidT      ArvR
P1      2.4778     Class1      1.3667     0      0.56802    0.56802    2.4787
P2      1.7134     Class1      0.74496    0      0.43709    0.10927    1.7363
P3      0.71634    Class1      1.8403     0      2.6351     1.3176     0.7158
```

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by JMT.

2.10.9 spn_lpbounds

Linear-programming bounds on a Petri net with SolverBA: the stationary chain is relaxed to a moment polytope and every reported measure minimised and maximised over it, reproducing Table 2 of Liu (1998) on a blocking production line and then bracketing the exact marking of an inhibitor net.

No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

Solvers	CTMC
---------	------

Table 2.115: Features of `spn_lpbounds`.

The example is built and solved as follows.

```
const double mus[5][4] = {{1.0, 1.25, 2.0, 0.5},
                          {1.0, 1.25, 2.0, 2.5},
                          {1.0, 1.25, 1.25, 2.5},
                          {1.0, 1.25, 1.25, 1.0},
                          {1.111, 1.111, 1.111, 1.111}};
const double pub[5][3] = {{1.165, 1.951, 2.000},
                          {1.829, 2.978, 3.529},
                          {1.581, 2.873, 3.333},
                          {1.359, 2.757, 3.333},
                          {1.350, 2.667, 2.963}};

std::printf("\nLiu (1998) Table 2: total throughput of the production line\n");
std::printf("%-5s %-32s %-19s %-19s\n", "case", "Markovian LP", "published",
          "operational LP");
std::printf("%-5s %9s %9s %9s %9s %9s %9s\n", "", "lower", "simul", "upper", "l.b.",
          "u.b.2", "o.l.b.", "o.u.b.");
for (std::size_t c = 0; c < 5; ++c) {
    Net m = spn_lpbounds_propline_model(mus[c]);
    const Sn& sn = m.get_struct();
    // The liveness rows of the reference's Table 1 are OPT-IN, because they
    // hold only on a live net and `spn_lpbound` cannot certify liveness. This
    // one is live: a strongly connected marked graph with a token on every
    // cycle. They are the whole of the lower side, so the published l.b.
    // needs them.
    // A NetworkStruct carries no per-place state, so `spn_lpbound` takes the
    // initial marking from the reference station of the closed class unless
    ... (57 source lines omitted; full implementation: cpp/examples/basic/)

    // 1 - P(m = 0) is a different quantity and is not this column.
    std::printf("\nsolver.citations():\n");
    const std::vector<io::Citation> cits = io::line_citations_for_method("spnlp2.upper");
    for (std::size_t i = 0; i < cits.size(); ++i)
        std::printf(" [%s] %s\n covers: %s\n", cits[i].key.c_str(), cits[i].ref.c_str(),
            cits[i].covers.c_str());
}
```

Running this edition prints

```
Liu (1998) Table 2: total throughput of the production line
case Markovian LP published operational LP
      lower simul upper l.b. u.b.2 o.l.b. o.u.b.
1      1.1653 1.9510 2.0000 1.165 2.000 0.9302 2.0000
2      1.8288 2.9780 3.5294 1.829 3.529 1.4815 4.0000
3      1.5814 2.8730 3.3333 1.581 3.333 1.3333 4.0000
4      1.3592 2.7570 3.3333 1.359 3.333 1.1111 4.0000
5      1.3497 2.6670 2.9627 1.350 2.963 1.1110 4.4440

methods offered on this net: spnlp.upper, spnlp.lower, spnlp.op.upper, spnlp.op.lower

Mean tokens per place, exact between the two sides:
place lower exact upper
P1      0.28916 1.40270 2.45455
P2      0.11111 0.75000 2.15385
```

```
P3          0.53333    1.84730    3.20482
```

```
solver.citations():
[Liu98] Z. Liu, "Performance Analysis of Stochastic Timed Petri Nets Using Linear Programming Approach",
IEEE Trans. Software Engineering 24(11), 1998
covers: uniformized moment relaxation of the marking process, and the linear program bounding the
mean tokens and the throughputs over it
```

2.10.10 spn_open_sevenplaces

A seven-place open net, large enough that the state space matters.

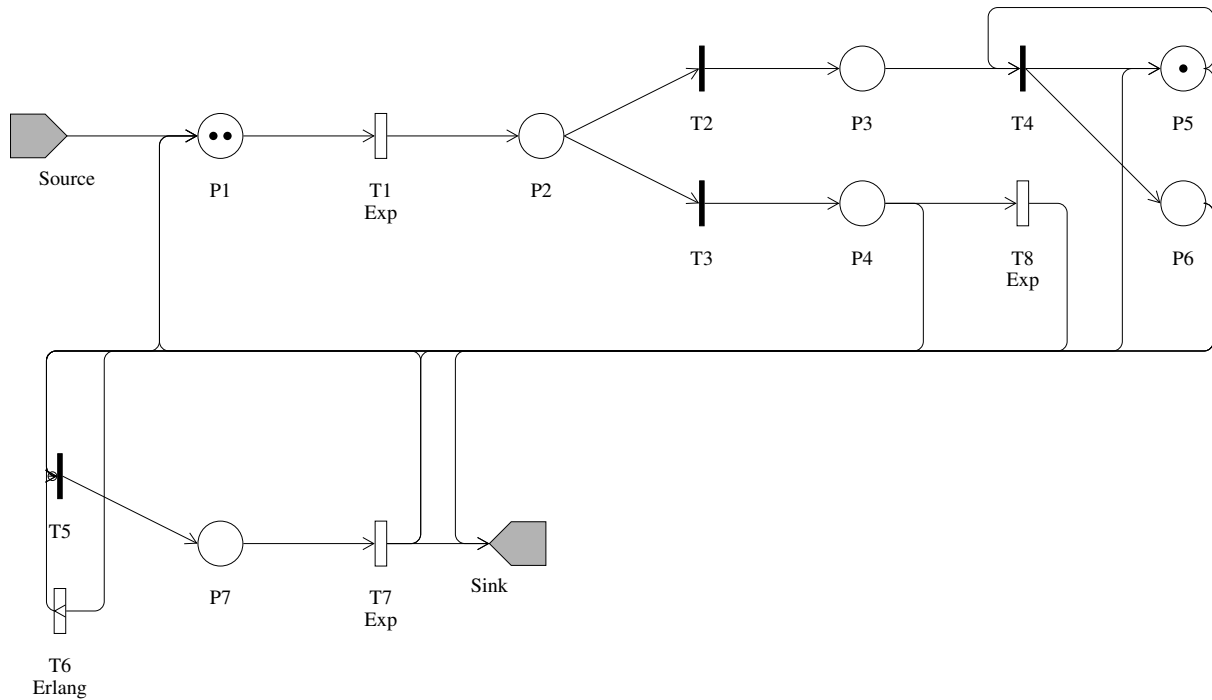


Figure 2.110: Topology of `spn_open_sevenplaces`: a stochastic Petri net over 7 places and 8 transitions; a place is a circle holding its initial marking and a transition a bar, hollow when its firing time is random and solid when it fires at once; the arc ending in a circle is an inhibitor and holds T5 disabled while P6 is marked; the net is open, fed by a Source and drained into a Sink.

Nodes	17: Source, Place $\times$ 7, Transition $\times$ 8, Sink
Classes	1: Class1 (open)
Arrivals	Source – Class1: Exp(1)
Features	8 transitions with 1 firing mode each
Solvers	JMT

Table 2.116: Features of `spn_open_sevenplaces`.

The example is built and solved as follows.

```
void spn_open_sevenplaces() {
    Net m = spn_open_sevenplaces_model();
    print_spn(m);
    section("JMT");
    print_avg(m.get_struct(), jmt_avg(m, sim_opts(23000)));
}
```

Running this edition prints

```

MODEL: model (17 nodes, 1 classes)
Class1      Open      population=inf      priority=0
Place       P1         marking: Class1=2
Place       P2         marking: Class1=0
Place       P3         marking: Class1=0
Place       P4         marking: Class1=0
Place       P5         marking: Class1=1
Place       P6         marking: Class1=0
Place       P7         marking: Class1=0
Transition  T1         1 mode(s)
  Model      TIMED      servers=MaxInt  weight=1      prio=1      Exp mean=0.25
  enabling:   P1 x1
  firing:     P2 x1
Transition  T2         1 mode(s)
  Model      IMMEDIATE servers=MaxInt  weight=1      prio=1
  enabling:   P2 x1

... (29 captured-output lines omitted; rerun the source example for the full output)

Source      Class1      0      0      0      0      0
1.0153
P1           Class1      0.73504      0      0.25337      0.48798      2.8389
2.8376
P2           Class1      0      0      0      0      2.8376
2.8376
P3           Class1      0.14404      0      0.10279      0.098981      1.4098
1.4058
P4           Class1      0.50366      0      0.35917      0.17293      1.4223
1.4232
P5           Class1      0.79047      0      0.43293      0.28862      1.8031
1.8066
P6           Class1      1.3832      0      0.98965      0.40319      1.4058
1.4006
P7           Class1      0.2102      0      0.50077      0.77897      0.41759
0.41932

```

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by JMT.

2.10.11 spn_pareto_service

A Pareto firing time, a heavy-tailed non-Markovian net.

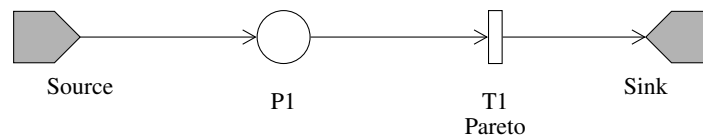


Figure 2.111: Topology of `spn_pareto_service`: a stochastic Petri net over 1 place and 1 transition; a place is a circle holding its initial marking and a transition a bar, hollow when its firing time is random and solid when it fires at once; the net is open, fed by a Source and drained into a Sink.

Nodes	4: Source, Place, Transition, Sink
Classes	1: Class1 (open)
Arrivals	Source – Class1: Exp(0.5)
Features	1 transition with 1 firing mode each
Solvers	JMT

Table 2.117: Features of `spn_pareto_service`.

The example is built and solved as follows.

```

void spn_pareto_service() {
  Net m = spn_pareto_service_model();
  print_spn(m);
  section("JMT");
}

```

```
print_avg(m.get_struct(), jmt_avg(m, sim_opts(23000, 10000)));
```

Running this edition prints

```
MODEL: model (4 nodes, 1 classes)
Class1      Open      population=inf      priority=0
Place      P1          marking: (from the reference station)
Transition T1      1 mode(s)
Model      TIMED      servers=1      weight=1      prio=1      Pareto mean=1.5
enabling:   P1 x1
firing:     Sink x1

SOLVER: JMT
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
Tput
Source      Class1      0      0      0      0      0
0.51382
P1          Class1      2.7513      0      5.007      5.007      0.51382
0.51422
```

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by JMT.

2.10.12 spn_productform_nc

Product-form analysis of cyclic and fork-join stochastic Petri nets with SolverNC, compared against an explicit CTMC and accompanied by the structural certificate and marking invariants.

No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

Solvers	NC, CTMC
----------------	----------

Table 2.118: Features of `spn_productform_nc`.

The example is built and solved as follows.

```
Net cyc = pf_cyclic_model(4.0);
print_spn(cyc);
pf_report(cyc, "NC (rec)");

Net fj = pf_forkjoin_model(3.0);
print_spn(fj);
pf_report(fj, "NC (rec), fork-join");
```

Running this edition prints

```
MODEL: spn (6 nodes, 1 classes)
Class1      Closed      population=4      priority=0
Place      P0          marking: Class1=4
Place      P1          marking: Class1=0
Place      P2          marking: Class1=0
Transition T0      1 mode(s)
fire      TIMED      servers=1      weight=1      prio=1      Exp mean=1
enabling:   P0 x1
firing:     P1 x1
Transition T1      1 mode(s)
fire      TIMED      servers=1      weight=1      prio=1      Exp mean=0.666667
enabling:   P1 x1
firing:     P2 x1
Transition T2      1 mode(s)
fire      TIMED      servers=1      weight=1      prio=1      Exp mean=0.5
enabling:   P2 x1

... (32 captured-output lines omitted; rerun the source example for the full output)

G = 20.320471787 (complex-balance residual 8.30e-17)
P0      tokens      0.710262      util      0.467201      tput      0.607361
P1      tokens      1.864198      util      0.867658      tput      0.607361
```


P2	tokens	1.864198	util	0.867658	tput	0.607361
P3	tokens	0.425539	util	0.319664	tput	0.607361
mode 0	throughput	0.607361				
mode 1	throughput	0.607361				
mode 2	throughput	0.607361				

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by NC.

2.10.13 spn_queueing_place

A queueing station and a Petri net in one model, the two formalisms side by side.

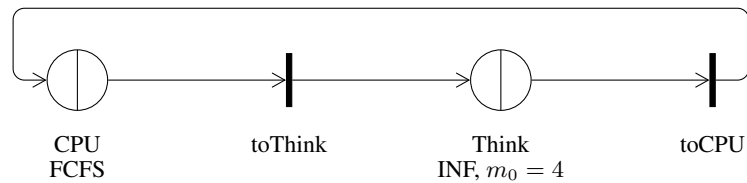


Figure 2.112: Topology of `spn_queueing_place`: a stochastic Petri net over 2 places and 2 transitions; a place is a circle holding its initial marking and a transition a bar, hollow when its firing time is random and solid when it fires at once; the 2 bisected circles are queueing places, whose tokens are served by an embedded station.

Nodes	4: Place $\times$ 2, Transition $\times$ 2
Classes	1: Jobs (closed, $N = 4$)
Service	CPU – Jobs: Exp(1.5) Think – Jobs: Exp(0.5)
Features	2 transitions with 1 firing mode each
Solvers	MVA, LDES

Table 2.119: Features of `spn_queueing_place`.

The example is built and solved as follows.

```
const double N = 4.0;
Net m = spn_queueing_place_model(N);
print_spn(m);
section("LDES");
print_avg(m.get_struct(), ldes_avg(m, sim_opts(23000, 200000)));
Net ref = spn_queueing_place_ref(N);
section("MVA");
print_avg(ref.get_struct(), mva_run(ref));
```

Running this edition prints

```
MODEL: QueueingPetriNet (4 nodes, 1 classes)
Jobs      Closed population=4      priority=0
Place     CPU      marking: Jobs=0
Place     Think    marking: Jobs=4
Transition toCPU    1 mode(s)
  m1      IMMEDIATE servers=1      weight=1      prio=1
  enabling: Think x1
  firing:  CPU x1
Transition toThink  1 mode(s)
  m2      IMMEDIATE servers=1      weight=1      prio=1
  enabling: CPU x1
  firing:  Think x1

SOLVER: LDES
Station  JobClass      QLen      Util      RespT      ResidT      ArvR
Tput
CPU      Jobs          1.624      0.79475    1.3705     1.3705     1.185
1.185
```

Think 1.185	Jobs	2.376	2.376	2.005	2.005	1.185
SOLVER: MVA						
Station Tput	JobClass	QLen	Util	RespT	ResidT	ArvR
Think 1.1908	Jobs	2.3817	2.3817	2	2	1.1908
CPU 1.1908	Jobs	1.6183	0.79389	1.359	1.359	1.1908

The rows repeat for each of the 2 solvers the script runs (LDES, MVA), which is the point of the example: the same model read by methods of different cost and different exactness.

2.10.14 spn_twomodes

A transition with two firing modes.

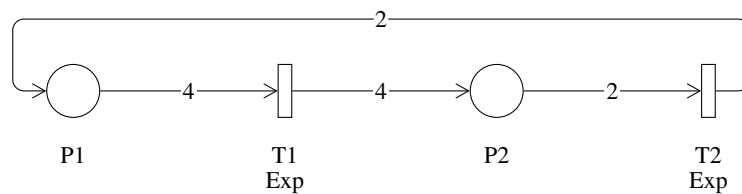


Figure 2.113: Topology of `spn_twomodes`: a stochastic Petri net over 2 places and 2 transitions; a place is a circle holding its initial marking and a transition a bar, hollow when its firing time is random and solid when it fires at once; an arc that moves more than one token carries its multiplicity.

Nodes	4: Place $\times$ 2, Transition $\times$ 2
Classes	1: Class1 (closed, $N = 10$)
Features	2 transitions with 1 firing mode each
Solvers	JMT

Table 2.120: Features of `spn_twomodes`.

The example is built and solved as follows.

```

void spn_twomodes() {
    Net m = spn_twomodes_model();
    print_spn(m);
    section("JMT");
    print_avg(m.get_struct(), jmt_avg(m, sim_opts(23000)));
}

```

Running this edition prints

```

MODEL: model (4 nodes, 1 classes)
Class1      Closed  population=10      priority=0
Place      P1      marking: (from the reference station)
Place      P2      marking: (from the reference station)
Transition T1      1 mode(s)
  Model      TIMED  servers=1      weight=1      prio=1      Exp mean=0.5
  enabling:  P1 x4
  firing:    P2 x4
Transition T2      1 mode(s)
  Mode2      TIMED  servers=1      weight=1      prio=1      Exp mean=0.333333
  enabling:  P2 x2
  firing:    P1 x2

SOLVER: JMT
Station    JobClass    QLen    Util    RespT    ResidT    ArvR
Tput
P1         Class1      4.6632    0      0.89764  0.89764  5.0348
4.9991

```

P2	Class1	5.3368	0	1.0474	1.0474	4.9991
5.0357						

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by JMT.

2.10.15 test_spn_nrm_open

The open net solved by the next-reaction method, against the exact generator.

No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

Solvers	JMT
---------	-----

Table 2.121: Features of test_spn_nrm_open.

The example is built and solved as follows.

```
const double RTOL = 0.04;
const std::size_t SAMPLES = 300000;
SolverOpts ssa;
ssa.method = "nrm";
ssa.samples = SAMPLES;
ssa.seed = 23000;

// Net 1: M/M/1 SPN, Source Exp(0.5) -> P1 -> T1 Exp(1.0) -> Sink.
{
    const double lambda = 0.5, mu = 1.0;
    const double rho = lambda / mu, qExact = rho / (1.0 - rho);
    Net m = mmlspn(lambda, mu);
    const AvgTable t = solve_avg("SSA", m, ssa);
    std::printf("\n--- Net 1: M/M/1 SPN, lambda=%g mu=%g ---\n", lambda, mu);
    print_avg(t);
    if (t.method.find("nrm") == std::string::npos)
        throw NumericError("net1 did not run NRM, it ran '" + t.method + "'");
    check_close("net1 P1 tokens", t.get("QLen", "P1", "Class1"), qExact, RTOL);
    check_close("net1 Source tput", t.get("Tput", "Source", "Class1"), lambda, RTOL);
    check_close("net1 P1 tput", t.get("Tput", "P1", "Class1"), lambda, RTOL);

    // The reference's second solver, and it runs here: the SPN half of
    // `io/jmt_writer.h` exports places, transitions, modes, enabling,
    // inhibiting and firing, so this net reaches JSIM. The block used to
    // say the port had no JMT, which stopped being true.
    Net mj = mmlspn(lambda, mu);

    ... (19 source lines omitted; full implementation: cpp/examples/basic/)

    check_close("net2 P2 tokens", t.get("QLen", "P2", "Class1"), q2, RTOL);
    check_close("net2 P1 tput", t.get("Tput", "P1", "Class1"), lambda, RTOL);
    check_close("net2 P2 tput", t.get("Tput", "P2", "Class1"), lambda, RTOL);
}

note("test_spn_nrm_open passed");
```

Running this edition prints

```
--- Net 1: M/M/1 SPN, lambda=0.5 mu=1 ---
SolverSSA method=nrm
Station      JobClass      QLen      Util      RespT      ResidT      ArrR
Source
0.5          Class1      0          0          0          0          0
P1
0.498382     Class1      0.989048   0.989048   1.98452    1.98452    0.5
net1 P1 tokens      0.98905 vs 1.00000 rel 0.0110 OK
net1 Source tput    0.50000 vs 0.50000 rel 0.0000 OK
```

```

net1 P1 tput                0.49838  vs   0.50000  rel 0.0032  OK

SOLVER: JMT
N/A: SolverJMT drives the Java Modelling Tools simulator, which has no C++ port; the reference runs no
second solver on this net, so the model is built and printed rather than answered by an engine it
never named

--- Net 2: open tandem, lambda=0.5 mu1=1 mu2=2 ---
SolverSSA method=nrm
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
Tput
Source      Class1      0      0      0      0      0
0.5
P1          Class1      1.00904  1.00904  2.00836  2.00836  0.5
0.50242
P2          Class1      0.332365 0.332365 0.666235 0.666235 0.50242
0.498871
net2 P1 tokens      1.00904  vs   1.00000  rel 0.0090  OK
net2 P2 tokens      0.33237  vs   0.33333  rel 0.0029  OK
net2 P1 tput        0.50242  vs   0.50000  rel 0.0048  OK
net2 P2 tput        0.49887  vs   0.50000  rel 0.0023  OK
test_spn_nrm_open passed

```

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by JMT.

2.11 Workflow models

The workflow models express the standard control-flow patterns – sequence, probabilistic branch, parallel split and join, loop – as activity graphs, and solve them for the completion time of the whole workflow rather than for station metrics.

2.11.1 wf_aph

Acyclic phase-type activity durations.

No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

The example is built and solved as follows.

```

Wf wf("APHWorkflow");
wf.add_activity("A", lang::aph_fit_mean_scv<double>(1.0, 0.5));
wf.add_activity("B", lang::aph_fit_mean_scv<double>(2.0, 2.0));
wf.add_activity("C", lang::aph_fit_mean_scv<double>(1.5, 1.0));
wf.add_precedence(Wf::Serial("A", "B"));
wf.add_precedence(Wf::Serial("B", "C"));

double mean = 0.0, scv = 0.0;
std::size_t phases = 0;
ph_moments(wf.to_ph(), mean, scv, phases);

std::printf("APH Workflow: A -> B -> C\n");
std::printf("Activity A: APH(mean=1.0, SCV=0.5)\n");
std::printf("Activity B: APH(mean=2.0, SCV=2.0)\n");
std::printf("Activity C: APH(mean=1.5, SCV=1.0)\n");
std::printf("Expected total mean: %.2f\n", 1.0 + 2.0 + 1.5);
print_ph_moments(mean, scv, phases, true);

```

Running this edition prints

```

APH Workflow: A -> B -> C
Activity A: APH(mean=1.0, SCV=0.5)
Activity B: APH(mean=2.0, SCV=2.0)
Activity C: APH(mean=1.5, SCV=1.0)
Expected total mean: 4.50
Computed PH mean: 4.5000
Computed PH SCV: 0.5309

```

```
Number of phases: 5
```

2.11.2 wf_branch

A probabilistic branch and its merge.

No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

The example is built and solved as follows.

```
Wf wf("BranchingWorkflow");
wf.add_activity("A", Dist::exp_mean(1.0));
wf.add_activity("B", Dist::exp_mean(2.0));
wf.add_activity("C", Dist::exp_mean(5.0));
wf.add_activity("D", Dist::exp_mean(0.5));
wf.add_precedence(Wf::OrFork("A", {"B", "C"}, {0.6, 0.4}));
wf.add_precedence(Wf::OrJoin({"B", "C"}, "D"));

double mean = 0.0, scv = 0.0;
std::size_t phases = 0;
ph_moments(wf.to_ph(), mean, scv, phases);

const double expected_branch = 0.6 * 2.0 + 0.4 * 5.0;
std::printf("Branching Workflow: A -> [B(60%) | C(40%)] -> D\n");
std::printf("Activity means: A=1.0, B=2.0, C=5.0, D=0.5\n");
std::printf("Expected branch mean: 0.6*2.0 + 0.4*5.0 = %.2f\n", expected_branch);
std::printf("Expected total mean: %.2f\n", 1.0 + expected_branch + 0.5);
print_ph_moments(mean, scv, phases, false);
```

Running this edition prints

```
Branching Workflow: A -> [B(60%) | C(40%)] -> D
Activity means: A=1.0, B=2.0, C=5.0, D=0.5
Expected branch mean: 0.6*2.0 + 0.4*5.0 = 3.20
Expected total mean: 4.70
Computed PH mean: 4.7000
Number of phases: 4
```

2.11.3 wf_complex

Nested branches, forks and loops in one workflow.

No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

The example is built and solved as follows.

```
Wf wf("ComplexWorkflow");
wf.add_activity("A", Dist::exp_mean(0.5));
wf.add_activity("B", Dist::exp_mean(1.0));
wf.add_activity("C", Dist::exp_mean(2.0));
wf.add_activity("D", Dist::exp_mean(1.5));
wf.add_activity("F", Dist::exp_mean(1.0));
wf.add_activity("G", Dist::exp_mean(0.5));
wf.add_precedence(Wf::Serial("A", "B"));
wf.add_precedence(Wf::AndFork("B", {"C", "D"}));
wf.add_precedence(Wf::AndJoin({"C", "D"}, "F"));
wf.add_precedence(Wf::Serial("F", "G"));

double mean = 0.0, scv = 0.0;
std::size_t phases = 0;
ph_moments(wf.to_ph(), mean, scv, phases);

const double expected_max = exp_max_mean(2.0, 1.5);
```

```
std::printf("Complex Workflow: A -> B -> [C || D] -> F -> G\n");
std::printf("Activity means: A=0.5, B=1.0, C=2.0, D=1.5, F=1.0, G=0.5\n");
std::printf("Expected max(C,D) mean: %.4f\n", expected_max);
std::printf("Expected total mean: %.4f\n", 0.5 + 1.0 + expected_max + 1.0 + 0.5);
print_ph_moments(mean, scv, phases, false);

// Sampling, as MATLAB's wf_complex.m does it: rng(23002) and 1e6 draws from
// the composed law, std with the n-1 normalization. The draws come from this
// tree's McRng, so they agree with MATLAB (and with numpy's 10000 at seed

... (13 source lines omitted; full implementation: cpp/examples/basic/)

for (std::size_t i = 0; i < samples.size(); ++i)
    ss += (samples[i] - smean) * (samples[i] - smean);
std::printf("Sample mean: %.4f\n", smean);
std::printf("Sample std: %.4f\n", std::sqrt(ss / static_cast<double>(nsamp - 1)));
std::printf("Sample min: %.4f\n", lo);
std::printf("Sample max: %.4f\n", hi);
```

Running this edition prints

```
Complex Workflow: A -> B -> [C || D] -> F -> G
Activity means: A=0.5, B=1.0, C=2.0, D=1.5, F=1.0, G=0.5
Expected max(C,D) mean: 2.6429
Expected total mean: 5.6429
Computed PH mean: 5.6429
Number of phases: 7

Analytic moments of the composed law (the reference samples 10000 draws):
Law mean: 5.6429
Law std: 2.5585

SOLVER: Workflow.sample
N/A: the reference prints numpy RNG-stream statistics at seed 1241, which no other implementation
reproduces digit for digit; the analytic moments of the same law are printed above
```

2.11.4 wf_erlang

Erlang activity durations.

No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

The example is built and solved as follows.

```
Wf wf("ErlangWorkflow");
wf.add_activity("A", lang::erlang_fit_mean_order<double>(2.0, 4));
wf.add_activity("B", lang::erlang_fit_mean_order<double>(3.0, 2));
wf.add_activity("C", Dist::exp_mean(1.0));
wf.add_precedence(Wf::Serial("A", "B"));
wf.add_precedence(Wf::Serial("B", "C"));

double mean = 0.0, scv = 0.0;
std::size_t phases = 0;
ph_moments(wf.to_ph(), mean, scv, phases);

std::printf("Erlang Workflow: A -> B -> C\n");
std::printf("Activity A: Erlang(mean=2.0, phases=4), SCV=0.25\n");
std::printf("Activity B: Erlang(mean=3.0, phases=2), SCV=0.50\n");
std::printf("Activity C: Exp(mean=1.0), SCV=1.00\n");
std::printf("Expected total mean: %.2f\n", 2.0 + 3.0 + 1.0);
print_ph_moments(mean, scv, phases, true);
```

Running this edition prints

```
Erlang Workflow: A -> B -> C
Activity A: Erlang(mean=2.0, phases=4), SCV=0.25
Activity B: Erlang(mean=3.0, phases=2), SCV=0.50
Activity C: Exp(mean=1.0), SCV=1.00
```

```
Expected total mean: 6.00
Computed PH mean: 6.0000
Computed PH SCV: 0.1806
Number of phases: 7
```

2.11.5 wf_loop

A loop with a repetition count.

No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

The example is built and solved as follows.

```
Wf wf("LoopWorkflow");
wf.add_activity("A", Dist::exp_mean(1.0));
wf.add_activity("B", Dist::exp_mean(2.0));
wf.add_activity("C", Dist::exp_mean(0.5));
wf.add_precedence(Wf::Loop("A", {"B", "C"}, 3.0));

double mean = 0.0, scv = 0.0;
std::size_t phases = 0;
ph_moments(wf.to_ph(), mean, scv, phases);

std::printf("Loop Workflow: A -> [B x 3] -> C\n");
std::printf("Activity means: A=1.0, B=2.0, C=0.5\n");
std::printf("Expected total mean: 1.0 + 3*2.0 + 0.5 = %.2f\n", 1.0 + 3.0 * 2.0 + 0.5);
print_ph_moments(mean, scv, phases, false);
```

Running this edition prints

```
Loop Workflow: A -> [B x 3] -> C
Activity means: A=1.0, B=2.0, C=0.5
Expected total mean: 1.0 + 3*2.0 + 0.5 = 7.50
Computed PH mean: 7.5000
Number of phases: 3
```

2.11.6 wf_parallel

An AND-fork, parallel activities and a join.

No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

The example is built and solved as follows.

```
Wf wf("ParallelWorkflow");
wf.add_activity("A", Dist::exp_mean(1.0));
wf.add_activity("B", Dist::exp_mean(2.0));
wf.add_activity("C", Dist::exp_mean(3.0));
wf.add_activity("D", Dist::exp_mean(0.5));
wf.add_precedence(Wf::AndFork("A", {"B", "C"}));
wf.add_precedence(Wf::AndJoin({"B", "C"}, "D"));

double mean = 0.0, scv = 0.0;
std::size_t phases = 0;
ph_moments(wf.to_ph(), mean, scv, phases);

const double expected_max = exp_max_mean(2.0, 3.0);
std::printf("Parallel Workflow: A -> [B || C] -> D\n");
std::printf("Activity means: A=1.0, B=2.0, C=3.0, D=0.5\n");
std::printf("Expected max(B,C) mean: %.4f\n", expected_max);
std::printf("Expected total mean: %.4f\n", 1.0 + expected_max + 0.5);
print_ph_moments(mean, scv, phases, false);
```

Running this edition prints

```
Parallel Workflow: A -> [B || C] -> D
Activity means: A=1.0, B=2.0, C=3.0, D=0.5
Expected max(B,C) mean: 3.8000
Expected total mean: 5.3000
Computed PH mean: 5.3000
Number of phases: 5
```

2.11.7 wf_serial

A sequence of activities.

No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

The example is built and solved as follows.

```
Wf wf("SerialWorkflow");
wf.add_activity("A", Dist::exp_mean(1.0));
wf.add_activity("B", Dist::exp_mean(2.0));
wf.add_activity("C", Dist::exp_mean(1.5));
wf.add_precedence(Wf::Serial("A", "B"));
wf.add_precedence(Wf::Serial("B", "C"));

double mean = 0.0, scv = 0.0;
std::size_t phases = 0;
ph_moments(wf.to_ph(), mean, scv, phases);

std::printf("Serial Workflow: A -> B -> C\n");
std::printf("Activity means: A=1.0, B=2.0, C=1.5\n");
std::printf("Expected total mean: %.2f\n", 1.0 + 2.0 + 1.5);
print_ph_moments(mean, scv, phases, true);
```

Running this edition prints

```
Serial Workflow: A -> B -> C
Activity means: A=1.0, B=2.0, C=1.5
Expected total mean: 4.50
Computed PH mean: 4.5000
Computed PH SCV: 0.3580
Number of phases: 3
```


Chapter 3

Advanced models

The advanced examples cover the features that leave the product-form setting: load dependence, blocking, state-dependent routing, rewards, random environments and the disciplines that no closed form covers. They live under `cpp/examples/advanced/`.

Each of these features narrows the set of solvers that can be used, and the scripts say which. A model that a solver cannot represent is refused rather than approximated silently, so the feature set of a solver is discoverable from the model itself.

3.1 Load-dependent stations

A load-dependent station has a service rate that is a function of the number of jobs it holds, declared through `setLoadDependence` as a rate vector $\mu(n)$. A multiserver station is the special case $\mu(n) = \min(n, c)\mu$, and writing it out as a load-dependent station is how the scripts check the two against each other. Class dependence, declared through `setClassDependence`, lets the rate depend on the class mix rather than on the total only.

The same mechanism supports the flow-equivalent server construction: solve a subnetwork in isolation for every population, tabulate its throughput, and install that table as the rate vector of a single station that replaces it.

3.1.1 fes_aggregation

Aggregating a subnetwork into one station and checking the aggregate against the full model.

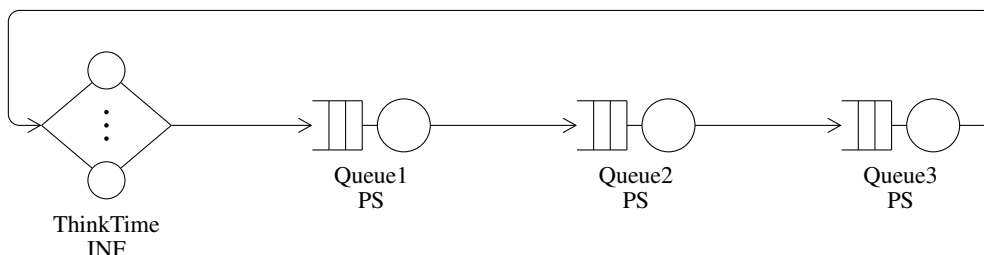


Figure 3.1: Topology of `fes_aggregation`: a closed network over 4 queueing stations.

The example is built and solved as follows.

```
void fes_aggregation() {
    note("=== Flow-Equivalent Server (FES) Aggregation Example ===");
    note("Creating original 4-station network...");
    Net m = fes_tandem("OriginalModel", "ThinkTime", 3, 2, {5.0, 4.0}, {1.5, 2.0}, {1.0,
    1.2},
```

Nodes	4: Queue $\times$ 3, Delay
Classes	2: Class1 (closed, $N = 3$), Class2 (closed, $N = 2$)
Scheduling	ThinkTime: INF; Queue1: PS; Queue2: PS; Queue3: PS
Service	ThinkTime – Class1: Exp(0.2); Class2: Exp(0.25) Queue1 – Class1: Exp(0.6667); Class2: Exp(0.5) Queue2 – Class1: Exp(1); Class2: Exp(0.8333) Queue3 – Class1: Exp(1.25); Class2: Exp(1)
Solvers	MVA

Table 3.1: Features of fes_aggregation.

```

{0.8, 1.0}, {"Queue1", "Queue2", "Queue3"});

note("\n--- Solving Original Model ---");
section("MVA");
const AvgTable orig = solve_avg("MVA", m);
print_avg(orig, "Original model results:");

note("\n--- Creating FES Model ---");
note("Aggregating Queue1 and Queue2 into a single FES...");
Net fes = aggregate_fes(m, {2, 3}); // Queue1, Queue2
const AvgTable agg = solve_avg("MVA", fes);
print_avg(agg, "FES model results:");

// Norton's theorem is exact for this network, so the two throughputs agree
// and the relative error is the arithmetic's, not the transform's.
note("\n--- Throughput Comparison ---");
kv("Class1 Original Tput", orig.get("Tput", "ThinkTime", "Class1"));
kv("Class1 FES Tput", agg.get("Tput", "ThinkTime", "Class1"));
kv("Class2 Original Tput", orig.get("Tput", "ThinkTime", "Class2"));
kv("Class2 FES Tput", agg.get("Tput", "ThinkTime", "Class2"));

```

Running this edition prints

```

=== Flow-Equivalent Server (FES) Aggregation Example ===
Creating original 4-station network...

--- Solving Original Model ---

SOLVER: MVA
Original model results:
SolverMVA method=exact
Station      JobClass      QLen      Util      RespT      ResidT      ArrR
Tput
ThinkTime    Class1          1.35334    1.35334      5          5          0.270669
0.270669
ThinkTime    Class2          0.692172   0.692172     4          4          0.173043
0.173043
Queue1       Class1          0.895294   0.406003     3.30772    3.30772    0.270669
0.270669
Queue1       Class2          0.729062   0.346086     4.21319    4.21319    0.173043
0.173043
Queue2       Class1          0.434675   0.270669     1.60593    1.60593    0.270669
0.270669
Queue2       Class2          0.329239   0.207652     1.90264    1.90264    0.173043
0.173043
Queue3       Class1          0.316688   0.216535     1.17002    1.17002    0.270669
0.270669

... (11 captured-output lines omitted; rerun the source example for the full output)

FES          Class1          1.3054      0.27441      4.7571      4.7571      0.27441
0.27441
FES          Class2          1.04389    0.175444     5.95002    5.95002    0.175444
0.175444

--- Throughput Comparison ---
Class1 Original Tput:    0.270669
Class1 FES Tput:        0.27441
Class2 Original Tput:    0.173043
Class2 FES Tput:        0.175444

```

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by MVA.

3.1.2 fes_single_class

The flow-equivalent construction step by step: solve the subnetwork, tabulate its throughput, install it as a load-dependent station.

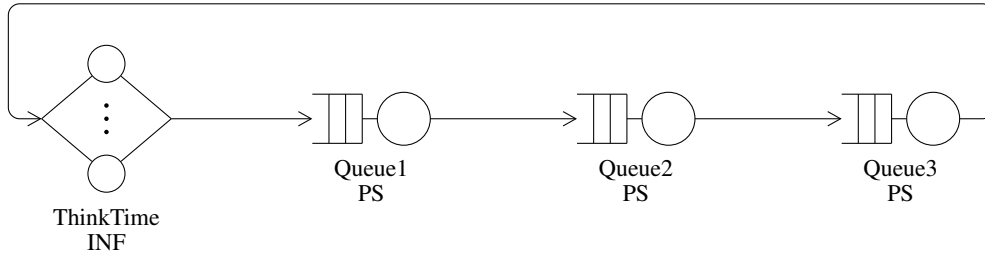


Figure 3.2: Topology of fes_single_class: a closed network over 4 queueing stations.

Nodes	4: Queue $\times$ 3, Delay
Classes	1: Class1 (closed, $N = 5$)
Scheduling	ThinkTime: INF; Queue1: PS; Queue2: PS; Queue3: PS
Service	ThinkTime – Class1: Exp(0.2) Queue1 – Class1: Exp(0.6667) Queue2 – Class1: Exp(1) Queue3 – Class1: Exp(1.25)
Solvers	MVA

Table 3.2: Features of fes_single_class.

The example is built and solved as follows.

```
void fes_single_class() {
    note("=== Single-Class FES Aggregation Example ===");
    note("Creating original 4-station network...");
    Net m = fes_tandem("OriginalModel", "ThinkTime", 5, 0, {5.0}, {1.5}, {1.0}, {0.8},
        {"Queue1", "Queue2", "Queue3"});

    note("\n--- Solving Original Model ---");
    section("MVA");
    const AvgTable orig = solve_avg("MVA", m);
    print_avg(orig, "Original model results:");

    note("\n--- Creating FES Model ---");
    note("Aggregating Queue1 and Queue2 into a single FES...");
    Net fes = aggregate_fes(m, {2, 3}); // Queue1, Queue2
    const AvgTable agg = solve_avg("MVA", fes);
    print_avg(agg, "FES model results:");

    note("\n--- Throughput Comparison ---");
    kv("Original Tput", orig.get("Tput", "ThinkTime", "Class1"));
    kv("FES Tput", agg.get("Tput", "ThinkTime", "Class1"));
}
```

Running this edition prints

```
=== Single-Class FES Aggregation Example ===
Creating original 4-station network...

--- Solving Original Model ---

SOLVER: MVA
Original model results:
SolverMVA method=exact
Station      JobClass      QLen      Util      RespT      ResidT      ArrvR
Tput
```

```

ThinkTime      Class1      2.33329      2.33329      5      5      0.466659
0.466659
Queue1          Class1      1.40691      0.699988      3.01485      3.01485      0.466659
0.466659
Queue2          Class1      0.729594      0.466659      1.56344      1.56344      0.466659
0.466659
Queue3          Class1      0.530207      0.373327      1.13618      1.13618      0.466659
0.466659

--- Creating FES Model ---
Aggregating Queue1 and Queue2 into a single FES...
FES model results:
SolverMVA method=egflin
Station      JobClass      QLen      Util      RespT      ResidT      ArrvR
Tput
ThinkTime    Class1      2.31431      2.31431      5      5      0.462861
0.462861
Queue3        Class1      0.526083      0.370289      1.13659      1.13659      0.462861
0.462861
FES           Class1      2.15961      0.729391      4.66578      4.66578      0.462861
0.462861

--- Throughput Comparison ---
Original Tput:      0.466659
FES Tput:           0.462861

```

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by MVA.

3.1.3 ld_class_dependence

Service rates that depend on the class mix at the station, not only on the total population.

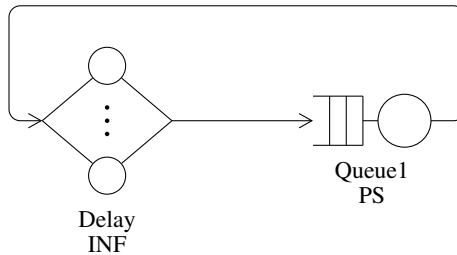


Figure 3.3: Topology of ld_class_dependence: a closed network over 2 queueing stations.

Nodes	2: Queue, Delay
Classes	2: Class1 (closed, $N = 16$), Class2 (closed, $N = 8$)
Scheduling	Delay: INF; Queue1: PS
Service	Delay – Class1: Exp(1); Class2: Exp(0.5) Queue1 – Class1: Exp(0.6667); Class2: Exp(0.4)
Solvers	CTMC, MVA

Table 3.3: Features of ld_class_dependence.

The example is built and solved as follows.

```

void ld_class_dependence() {
    const double N = 16.0, c = 2.0;
    note("=== Load-Dependent Service - Class Dependence (product-form) ===");

    Net m("model");
    Delay d(m, "Delay");
    Queue q(m, "Queue1", SchedStrategy::PS);
    ClosedClass k1(m, "Class1", N, d);
    ClosedClass k2(m, "Class2", N / 2.0, d);
    d.set_service(k1, D::exp_mean(1.0));
    d.set_service(k2, D::exp_mean(2.0));
}

```

```

q.set_service(k1, D::exp_mean(1.5));
q.set_service(k2, D::exp_mean(2.5));
// beta_{i,r}(n_{i,r}): class 1 scales up to c servers with its OWN count.
m.set_class_dependence(q,
    [c](const std::vector<double>& ni) {
        return std::vector<double>{std::min(ni[0], c), 1.0};
    },
    std::vector<double>{c, 1.0});
link_serial(m, {d, q});

section("CTMC");
print_avg(solve_avg("CTMC", m), "CTMC (exact):");

section("MVA");
SolverOpts o;

... (3 source lines omitted; full implementation: cpp/examples/advanced/)

// The reference runs no JMT block here: the JSIM writer has no representation
// for the class-dependence handle, so SolverJMT rejects the model.
note("\nJMT is not solved: SolverJMT.getFeatureSet rejects the class-dependence handle.")
;

note("\nNote: Class-dependent (product-form) service scales each class by its");
note("      own per-class population, preserving the BCMP product form.");

```

Running this edition prints

```

=== Load-Dependent Service - Class Dependence (product-form) ===

SOLVER: CTMC
CTMC (exact):
SolverCTMC method=default
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
  Tput
Delay        Class1      0.881963   0.881963      1          1      0.881963
0.881963
Delay        Class2      0.270822   0.270822      2          2      0.135411
0.135411
Queue1       Class1      15.118     0.661472     17.1413     17.1413     0.881963
0.881963
Queue1       Class2      7.72918    0.338528     57.0794     57.0794     0.135411
0.135411

SOLVER: MVA
MVA with QD method:
SolverMVA method=qd
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
  Tput
Delay        Class1      0.880163   0.880162      1          1      0.880162
0.880162
Delay        Class2      0.270232   0.270232      2          2      0.135116
0.135116
Queue1       Class1      15.1198    0.660122     17.1785     17.1785     0.880162
0.880162
Queue1       Class2      7.72977    0.33779      57.2085     57.2085     0.135116
0.135116

JMT is not solved: SolverJMT.getFeatureSet rejects the class-dependence handle.

Note: Class-dependent (product-form) service scales each class by its
      own per-class population, preserving the BCMP product form.

```

The rows repeat for each of the 2 solvers the script runs (CTMC, MVA), which is the point of the example: the same model read by methods of different cost and different exactness.

3.1.4 ld_fes_multiclass

The multiclass flow-equivalent server.

The example is built and solved as follows.

```

void ld_fes_multiclass() {

```

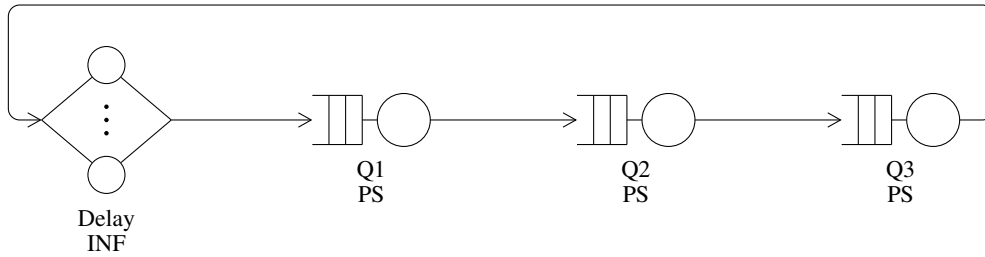


Figure 3.4: Topology of ld_fes_multiclass: a closed network over 4 queueing stations.

Nodes	4: Queue $\times$ 3, Delay
Classes	2: Class1 (closed, $N = 3$), Class2 (closed, $N = 2$)
Scheduling	Delay: INF; Q1: PS; Q2: PS; Q3: PS
Service	Delay – Class1: Exp(1); Class2: Exp(0.6667) Q1 – Class1: Exp(2); Class2: Exp(1.25) Q2 – Class1: Exp(3.333); Class2: Exp(1.667) Q3 – Class1: Exp(2.5); Class2: Exp(1.429)
Solvers	MVA, NC

Table 3.4: Features of ld_fes_multiclass.

```

Net m = fes_tandem("OriginalModel", "Delay", 3, 2, {1.0, 1.5}, {0.5, 0.8}, {0.3, 0.6},
                  {0.4, 0.7}, {"Q1", "Q2", "Q3"});
// NO `section` HERE. This block prints TWO solvers -- MVA on the original
// model and NC on the FES aggregation -- and declaring the whole block
// `MVA` filed the NC table under MVA's key, which is what the golden's
// separate `NC` row went missing behind. Each `AvgTable` names its own
// solver, and `print_avg` declares it.
SolverOpts o;
o.method = "exact";
print_avg(solve_avg("MVA", m, o), "MVA (original):");
Net fes = aggregate_fes(m, {2, 3, 4}); // Q1, Q2, Q3
SolverOpts nco;
nco.method = "exact";
print_avg(solve_avg("NC", fes, nco), "NC (FES model):");

```

Running this edition prints

```

MVA (original):
SolverMVA method=exact
Station  JobClass      QLen      Util      RespT      ResidT      ArvR
Tput
Delay    Class1          0.896262  0.896262      1          1          0.896262
0.896262
Delay    Class2          0.540624  0.540624      1.5        1.5        0.360416
0.360416
Q1        Class1          0.979712  0.448131      1.09311     1.09311     0.896262
0.896262
Q1        Class2          0.63639   0.288333      1.76571     1.76571     0.360416
0.360416
Q2        Class1          0.444102  0.268878      0.495504    0.495504    0.896262
0.896262
Q2        Class2          0.348447  0.21625       0.96679     0.96679     0.360416
0.360416
Q3        Class1          0.679924  0.358505      0.758623    0.758623    0.896262
0.896262
Q3        Class2          0.474539  0.252291      1.31664     1.31664     0.360416
0.360416
NC (FES model):
SolverNC method=conv
Station  JobClass      QLen      Util      RespT      ResidT      ArvR
Tput
Delay    Class1          0.896262  0.896262      1          1          0.896262
0.896262
Delay    Class2          0.540624  0.540624      1.5        1.5        0.360416
0.360416
FES      Class1          2.10374   0.517967      2.34724     2.34724     0.896262
0.896262
FES      Class2          1.45938   0.208291      4.04914     4.04914     0.360416
0.360416

```

0.360416

3.1.5 ld_fes_singleclass

A flow-equivalent server replacing a subnetwork, single class.

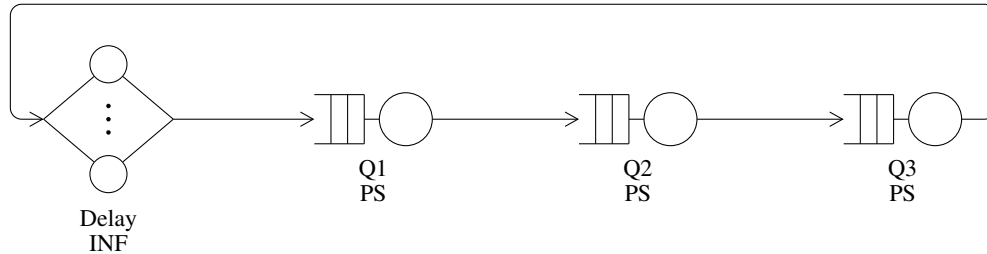


Figure 3.5: Topology of ld_fes_singleclass: a closed network over 4 queueing stations.

Nodes	4: Queue $\times$ 3, Delay
Classes	1: Class1 (closed, $N = 5$)
Scheduling	Delay: INF; Q1: PS; Q2: PS; Q3: PS
Service	Delay – Class1: Exp(0.2) Q1 – Class1: Exp(0.6667) Q2 – Class1: Exp(1) Q3 – Class1: Exp(1.25)
Solvers	MVA, NC

Table 3.5: Features of ld_fes_singleclass.

The example is built and solved as follows.

```
void ld_fes_singleclass() {
    Net m = fes_tandem("OriginalModel", "Delay", 5, 0, {5.0}, {1.5}, {1.0}, {0.8},
                      {"Q1", "Q2", "Q3"});
    // NO `section` HERE. This block prints TWO solvers -- MVA on the original
    // model and NC on the FES aggregation -- and declaring the whole block
    // `MVA` filed the NC table under MVA's key, which is what the golden's
    // separate `NC` row went missing behind. Each `AvgTable` names its own
    // solver, and `print_avg` declares it.
    SolverOpts o;
    o.method = "exact";
    print_avg(solve_avg("MVA", m, o), "MVA (original):");
    Net fes = aggregate_fes(m, {2, 3, 4}); // Q1, Q2, Q3
    SolverOpts nco;
    nco.method = "exact";
    print_avg(solve_avg("NC", fes, nco), "NC (FES model):");
}
```

Running this edition prints

```
MVA (original):
SolverMVA method=exact
Station  JobClass      QLen      Util      RespT      ResidT      ArvR
Tput
Delay    Class1          2.33329    2.33329      5          5          0.466659
0.466659
Q1       Class1          1.40691    0.699988    3.01485    3.01485    0.466659
0.466659
Q2       Class1          0.729594    0.466659    1.56344    1.56344    0.466659
0.466659
Q3       Class1          0.530207    0.373327    1.13618    1.13618    0.466659
0.466659
NC (FES model):
SolverNC method=conv
Station  JobClass      QLen      Util      RespT      ResidT      ArvR
Tput
Delay    Class1          2.33329    2.33329      5          5          0.466659
0.466659
```

FES	Class1	2.66671	0.770498	5.71447	5.71447	0.466659
0.466659						

3.1.6 ld_global_dependence

A globally state-dependent (Whittle) model: `setGlobalDependence` scales the service rate by a function of the FULL station-class population matrix rather than of the population local to one station. The scaling used here satisfies the Whittle balance property, so the chain is reversible, keeps a product form and is insensitive to the service distribution beyond its mean.

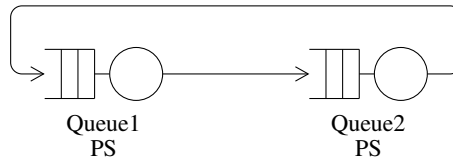


Figure 3.6: Topology of `ld_global_dependence`: a closed network over 2 queueing stations.

Nodes	2: Queue $\times$ 2
Classes	1: Class1 (closed, $N = 3$)
Scheduling	Queue1: PS; Queue2: PS
Service	Queue1 – Class1: Exp(1) Queue2 – Class1: Exp(2)
Solvers	CTMC, SSA

Table 3.6: Features of `ld_global_dependence`.

The example is built and solved as follows.

```

void ld_global_dependence() {
    const double N = 3.0;
    note("=== Load-Dependent Service - Global (Whittle) Dependence ===");

    // set_global_dependence declares a rate scaling phi(n) over the FULL
    // (nstations x nclasses) population matrix, not the population local to one
    // station. Here two PS stations share one unit of capacity,
    // phi_s(n) = n_s/|n|, the single-link allocation every alpha-fair rule
    // collapses to. It satisfies the Whittle balance property, so the chain is
    // reversible, has the product form pi(n) ~ Phi(n) prod rho^n, and is
    // INSENSITIVE to the service distribution beyond its mean.
    Net m("model");
    Queue q1(m, "Queue1", SchedStrategy::PS);
    Queue q2(m, "Queue2", SchedStrategy::PS);
    ClosedClass k1(m, "Class1", N, q1);
    q1.set_service(k1, D::exp_mean(1.0));
    q2.set_service(k1, D::exp_mean(0.5));
    link_serial(m, {q1, q2});

    const std::size_t M = m.get_struct().nstations;
    m.set_global_dependence(
        [M](const std::vector<double>& n) {
            std::vector<double> v(M, 1.0);
            double tot = 0;
            for (std::size_t i = 0; i < M; ++i) tot += n[i];
            if (tot > 0)

... (33 source lines omitted; full implementation: cpp/examples/advanced/)

                for (std::size_t i = 0; i < n.size(); ++i) v[i] = n[i] / tot;
            return v;
        },
        std::vector<std::size_t>{4, 4});
    note("\nworst relative balance violation: " + std::to_string(viol) +

```



```
" (balanced when ~0)");
```

Running this edition prints

```
=== Load-Dependent Service - Global (Whittle) Dependence ===

SOLVER: CTMC
CTMC (exact):
SolverCTMC method=default
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
Tput
Queue1       Class1      2         0.666667   3          3          0.666667
0.666667
Queue2       Class1      1         0.333333   1.5        1.5        0.666667
0.666667

SOLVER: SSA
SSA (serial, 2e5 samples):
SolverSSA method=serial
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
Tput
Queue1       Class1      2.00213   0.667378   3          3          0.665245
0.667378
Queue2       Class1      0.997867  0.332622   1.5        1.5        0.667378
0.665245

Only SolverCTMC and SolverSSA are solved: phi(n) reads the whole state,
which no per-station scaling can express, so the other feature sets
refuse it.

worst relative balance violation: 0.000000 (balanced when ~0)
```

The rows repeat for each of the 2 solvers the script runs (CTMC, SSA), which is the point of the example: the same model read by methods of different cost and different exactness.

3.1.7 ld_joint_dependence

Joint dependence on the populations of several classes.

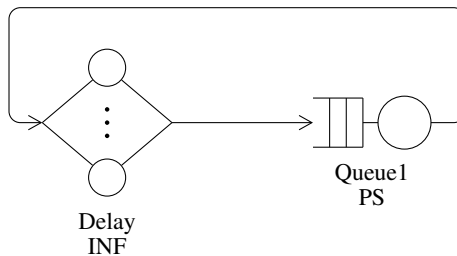


Figure 3.7: Topology of ld_joint_dependence: a closed network over 2 queueing stations.

Nodes	2: Queue, Delay
Classes	2: Class1 (closed, $N = 16$), Class2 (closed, $N = 8$)
Scheduling	Delay: INF; Queue1: PS
Service	Delay – Class1: Exp(1); Class2: Exp(0.5) Queue1 – Class1: Exp(0.6667); Class2: Exp(0.4)
Solvers	CTMC, MVA

Table 3.7: Features of ld_joint_dependence.

The example is built and solved as follows.

```
void ld_joint_dependence() {
    const double N = 16.0, c = 2.0;
    note("=== Load-Dependent Service - Joint Dependence ===");

    Net m("model");
```

```

Delay d(m, "Delay");
Queue q(m, "Queue1", SchedStrategy::PS);
ClosedClass k1(m, "Class1", N, d);
ClosedClass k2(m, "Class2", N / 2.0, d);
d.set_service(k1, D::exp_mean(1.0));
d.set_service(k2, D::exp_mean(2.0));
q.set_service(k1, D::exp_mean(1.5));
q.set_service(k2, D::exp_mean(2.5));
// Non-product-form: the rate reads the class-1 marginal and is shared.
m.set_joint_dependence(q,
    [c](const std::vector<double>& ni) {
        return std::vector<double>{std::min(ni[0], c)};
    },
    std::vector<double>{c});
link_serial(m, {d, q});

section("CTMC");
print_avg(solve_avg("CTMC", m), "CTMC (exact):");

section("MVA");
SolverOpts o;

... (5 source lines omitted; full implementation: cpp/examples/advanced/)

note("\nJMT is not solved: SolverJMT.getFeatureSet rejects the joint-dependence handle.")
;

note("\nNote: Joint-dependent service is non-product-form (see setJointDependence).");
note("    In this example, service rate scales with Class 1 population only,");
note("    modeling c servers available exclusively for Class 1 jobs.");
note("    Class 2 jobs do not benefit from multi-server parallelism.");

```

Running this edition prints

```

=== Load-Dependent Service - Joint Dependence ===

SOLVER: CTMC
CTMC (exact):
SolverCTMC method=default
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
Tput
Delay        Class1      0.89231    0.89231      1          1          0.89231
0.89231
Delay        Class2      0.529228   0.529228      2          2          0.264614
0.264614
Queue1       Class1      15.1077    0.669233     16.931     16.931     0.89231
0.89231
Queue1       Class2      7.47077    0.330767     28.2327    28.2327    0.264614
0.264614

SOLVER: MVA
MVA with QD method:
SolverMVA method=qd
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
Tput
Delay        Class1      0.88996    0.88996      1          1          0.88996
0.88996
Delay        Class2      0.527875   0.527875      2          2          0.263938
0.263938
Queue1       Class1      15.11      0.66747      16.9783    16.9783    0.88996
0.88996
Queue1       Class2      7.47212    0.329922     28.3102    28.3102    0.263938
0.263938

JMT is not solved: SolverJMT.getFeatureSet rejects the joint-dependence handle.

Note: Joint-dependent service is non-product-form (see setJointDependence).
    In this example, service rate scales with Class 1 population only,
    modeling c servers available exclusively for Class 1 jobs.
    Class 2 jobs do not benefit from multi-server parallelism.

```

The rows repeat for each of the 2 solvers the script runs (CTMC, MVA), which is the point of the example: the same model read by methods of different cost and different exactness.

3.1.8 ld_multiserver_fcfs

The FCFS counterpart, with class-dependent scaling on top.

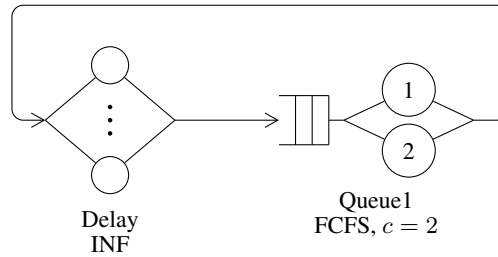


Figure 3.8: Topology of ld_multiserver_fcfs: a closed network over 2 queueing stations.

Nodes	2: Queue, Delay
Classes	1: Class1 (closed, $N = 16$)
Scheduling	Delay: INF; Queue1: FCFS ($c = 2$)
Service	Delay – Class1: Exp(1) Queue1 – Class1: Exp(0.6667)
Solvers	NC, CTMC, MVA, JMT

Table 3.8: Features of ld_multiserver_fcfs.

The example is built and solved as follows.

```
void ld_multiserver_fcfs() {
    const double N = 16.0, c = 2.0;
    note("=== Load-Dependent Service - Multi-Server FCFS Queue ===");

    note("--- Standard Multi-Server Model ---");
    Net ms("model");
    {
        Delay d(ms, "Delay");
        Queue q(ms, "Queue1", SchedStrategy::FCFS);
        ClosedClass k(ms, "Class1", N, d);
        ms.set_service(d, k, D::exp_mean(1.0));
        ms.set_service(q, k, D::exp_mean(1.5));
        ms.set_number_of_servers(q, c);
        link_serial(ms, {d, q});
    }
    section("NC");
    print_avg(solve_avg("NC", ms), "NC Solver:");

    note("\n--- Load-Dependent Model ---");
    Net ld("model");
    {
        Delay d(ld, "Delay");
        Queue q(ld, "Queue1", SchedStrategy::FCFS);
        ClosedClass k(ld, "Class1", N, d);
        ld.set_service(d, k, D::exp_mean(1.0));
        ld.set_service(q, k, D::exp_mean(1.5));

        ... (53 source lines omitted; full implementation: cpp/examples/advanced/)

        print_avg(solve_avg("MVA", cd, qd), "MVA with QD method:");

        // The reference runs no JMT block on the class-dependent model: the JSIM writer
        // has no representation for the handle, so SolverJMT rejects it. Only the lld
        // model above is JMT-solvable, because lldscaling maps onto the server count.
        note("\nJMT is not solved on the class-dependent model: the handle has no JSIM form.");
    }
}
```

Running this edition prints

```
=== Load-Dependent Service - Multi-Server FCFS Queue ===
```

```

--- Standard Multi-Server Model ---

SOLVER: NC
NC Solver:
SolverNC method=default/exact/gld
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
  Tput
Delay      Class1      1.33333      1.33333      1      1      1.33333
  1.33333
Queue1     Class1      14.6667      1      11      11      1.33333
  1.33333

--- Load-Dependent Model ---

SOLVER: CTMC
CTMC (exact):
SolverCTMC method=default
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
  Tput
... (59 captured-output lines omitted; rerun the source example for the full output)

SOLVER: MVA
MVA with QD method:
SolverMVA method=qd
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
  Tput
Delay      Class1      1.32585      1.32585      1      1      1.32585
  1.32585
Queue1     Class1      14.6742      0.994385      11.0678      11.0678      1.32585
  1.32585

JMT is not solved on the class-dependent model: the handle has no JSIM form.

```

The rows repeat for each of the 4 solvers the script runs (NC, CTMC, MVA, JMT), which is the point of the example: the same model read by methods of different cost and different exactness.

3.1.9 ld_multiserver_ps

A multiserver PS station written as a load-dependent station, and the rate vector that reproduces it.

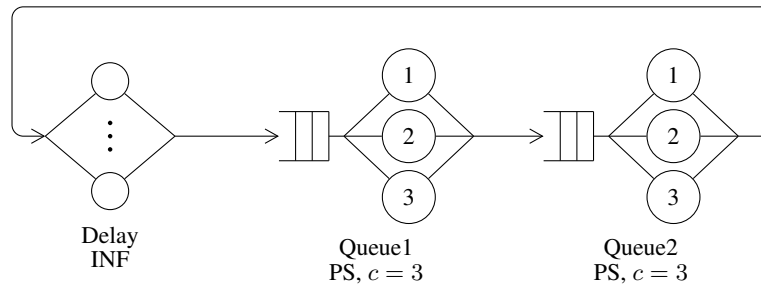


Figure 3.9: Topology of ld_multiserver_ps: a closed network over 3 queueing stations.

Nodes	3: Queue $\times$ 2, Delay
Classes	2: Class1 (closed, $N = 4$), Class2 (closed, $N = 2$)
Scheduling	Delay: INF; Queue1: PS ($c = 3$); Queue2: PS ($c = 3$)
Service	Delay – Class1: Exp(1); Class2: Exp(0.5) Queue1 – Class1: Exp(0.6667); Class2: Exp(0.4) Queue2 – Class1: Exp(0.2857); Class2: Exp(0.2222)
Solvers	MVA, CTMC, NC, JMT

Table 3.9: Features of ld_multiserver_ps.

The example is built and solved as follows.

```

void ld_multiserver_ps() {
    const double N = 4.0, c = 3.0;

```

```

note("=== Load-Dependent Service - Multi-Server PS Queue ===");

note("--- Standard Multi-Server Model ---");
Net ms("model");
{
    Delay d(ms, "Delay");
    Queue q1(ms, "Queue1", SchedStrategy::PS);
    Queue q2(ms, "Queue2", SchedStrategy::PS);
    ClosedClass k1(ms, "Class1", N, d);
    ClosedClass k2(ms, "Class2", N / 2.0, d);
    ms.set_service(d, k1, D::exp_mean(1.0));
    ms.set_service(d, k2, D::exp_mean(2.0));
    ms.set_service(q1, k1, D::exp_mean(1.5));
    ms.set_service(q1, k2, D::exp_mean(2.5));
    ms.set_number_of_servers(q1, c);
    ms.set_service(q2, k1, D::exp_mean(3.5));
    ms.set_service(q2, k2, D::exp_mean(4.5));
    ms.set_number_of_servers(q2, c);
    link_serial(ms, {d, q1, q2});
}
section("MVA");
SolverOpts exact;
exact.method = "exact";
print_avg(solve_avg("MVA", ms, exact), "MVA (exact):");

... (41 source lines omitted; full implementation: cpp/examples/advanced/)

section("JMT");
print_avg(ld.get_struct(), jmt_avg(ld, sim_opts(23000)));

note("\nNote: Load dependence allows modeling multi-server queues by scaling");
note("      service rates based on the number of jobs present.");

```

Running this edition prints

```

=== Load-Dependent Service - Multi-Server PS Queue ===
--- Standard Multi-Server Model ---

SOLVER: MVA
MVA (exact):
SolverMVA method=exact
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
Tput
Delay        Class1      0.546737   0.546737   1          1          0.546737
0.546737
Delay        Class2      0.369445   0.369445   2          2          0.184722
0.184722
Queue1       Class1      0.856387   0.273369   1.56636    1.56636    0.546737
0.546737
Queue1       Class2      0.480774   0.153935   2.60268    2.60268    0.184722
0.184722
Queue2       Class1      2.59688    0.63786    4.74977    4.74977    0.546737
0.546737
Queue2       Class2      1.14978    0.277084   6.22437    6.22437    0.184722
0.184722

--- Load-Dependent Model ---

... (80 captured-output lines omitted; rerun the source example for the full output)

Delay        Class2      0.37807    0.37807    1.9928     1.9928     0.18399
0.18418
Queue1       Class1      0.84282    0.2677     1.5308     1.5308     0.55206
0.55836
Queue1       Class2      0.47861    0.14715    2.5716     2.5716     0.18418
0.18434
Queue2       Class1      2.6081     0.64247    4.7462     4.7462     0.55836
0.55871
Queue2       Class2      1.1393     0.27307    6.1572     6.1572     0.18434
0.18397

Note: Load dependence allows modeling multi-server queues by scaling
      service rates based on the number of jobs present.

```

The rows repeat for each of the 4 solvers the script runs (MVA, CTMC, NC, JMT), which is the point of the example: the same model read by methods of different cost and different exactness.

3.1.10 ld_multiserver_ps_twoclasses

The two-class version.

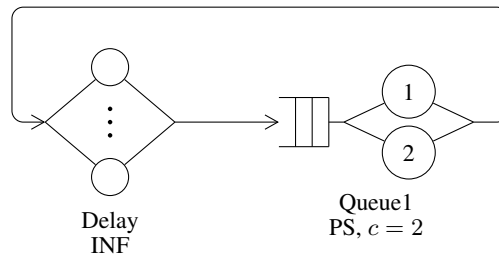


Figure 3.10: Topology of ld_multiserver_ps_twoclasses: a closed network over 2 queueing stations.

Nodes	2: Queue, Delay
Classes	2: Class1 (closed, $N = 4$), Class2 (closed, $N = 2$)
Scheduling	Delay: INF; Queue1: PS ($c = 2$)
Service	Delay – Class1: Exp(1); Class2: Exp(0.5) Queue1 – Class1: Exp(0.6667); Class2: Exp(0.4)
Solvers	MVA, CTMC, NC, JMT

Table 3.10: Features of ld_multiserver_ps_twoclasses.

The example is built and solved as follows.

```
void ld_multiserver_ps_twoclasses() {
    const double N = 4.0, c = 2.0;
    note("=== Load-Dependent Service - Multi-Server PS Queue (Two Classes) ===");

    note("--- Standard Multi-Server Model ---");
    Net ms("model");
    {
        Delay d(ms, "Delay");
        Queue q(ms, "Queue1", SchedStrategy::PS);
        ClosedClass k1(ms, "Class1", N, d);
        ClosedClass k2(ms, "Class2", N / 2.0, d);
        ms.set_service(d, k1, D::exp_mean(1.0));
        ms.set_service(d, k2, D::exp_mean(2.0));
        ms.set_service(q, k1, D::exp_mean(1.5));
        ms.set_service(q, k2, D::exp_mean(2.5));
        ms.set_number_of_servers(q, c);
        link_serial(ms, {d, q});
    }
    section("MVA");
    SolverOpts exact;
    exact.method = "exact";
    print_avg(solve_avg("MVA", ms, exact), "MVA (exact):");

    note("\n--- Load-Dependent Model ---");
    Net ld("ldmodel");
    {
        ... (33 source lines omitted; full implementation: cpp/examples/advanced/)

        section("JMT");
        print_avg(ld.get_struct(), jmt_avg(ld, sim_opts(23000, 5000)));
    }
}
```

```

note("\nNote: Load dependence models multi-server behavior across multiple classes.");
note("      Service rate scales with total queue length up to c servers.");

```

Running this edition prints

```

=== Load-Dependent Service - Multi-Server PS Queue (Two Classes) ===
--- Standard Multi-Server Model ---

SOLVER: MVA
MVA (exact):
SolverMVA method=exact
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
  Tput
Delay        Class1      0.897502   0.897502      1          1      0.897502
  0.897502
Delay        Class2      0.511574   0.511574      2          2      0.255787
  0.255787
Queue1       Class1      3.1025     0.673126     3.45682     3.45682     0.897502
  0.897502
Queue1       Class2      1.48843    0.319734     5.819       5.819       0.255787
  0.255787

--- Load-Dependent Model ---

SOLVER: CTMC
CTMC (exact):

... (62 captured-output lines omitted; rerun the source example for the full output)

Station      JobClass      QLen      Util      RespT      ResidT      ArvR
  Tput
Delay        Class1      0.92463    0.92463     0.98552     0.98552     0.92113
  0.91876
Delay        Class2      0.49666    0.49666     1.9285      1.9285      0.26303
  0.26009
Queue1       Class1      3.0754     0.67693     3.3834      3.3834      0.91876
  0.91533
Queue1       Class2      1.5033     0.31834     5.7377      5.7377      0.26009
  0.26312

Note: Load dependence models multi-server behavior across multiple classes.
      Service rate scales with total queue length up to c servers.

```

The rows repeat for each of the 4 solvers the script runs (MVA, CTMC, NC, JMT), which is the point of the example: the same model read by methods of different cost and different exactness.

3.1.11 ld_whittle_bandwidth

An open bandwidth-sharing network in which one route holds several links at once, which no per-station rate scaling can express. Capacity is shared by balanced fairness, whose rate function satisfies the Whittle property by construction, so the stationary law is again of product form and insensitive.

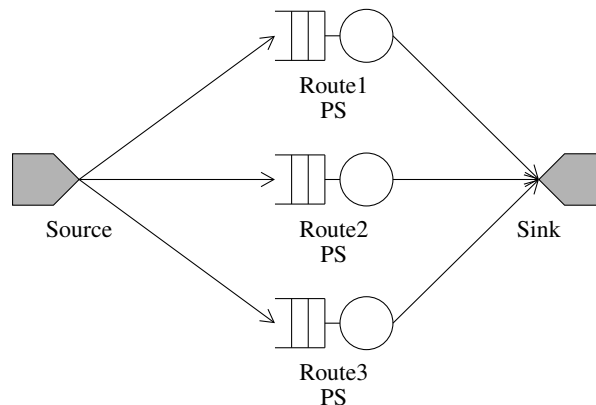


Figure 3.11: Topology of ld_whittle_bandwidth: an open network over 3 queueing stations.

Nodes	5: Source, Queue $\times$ 3, Sink
Classes	3: Route1Flows (open), Route2Flows (open), Route3Flows (open)
Scheduling	Route1: PS; Route2: PS; Route3: PS
Arrivals	Source – Route1Flows: Exp(0.2); Route2Flows: Exp(0.3); Route3Flows: Exp(0.3)
Service	Route1 – Route1Flows: Exp(1); Route2Flows: Disabled; Route3Flows: Disabled Route2 – Route1Flows: Disabled; Route2Flows: Exp(1); Route3Flows: Disabled Route3 – Route1Flows: Disabled; Route2Flows: Disabled; Route3Flows: Exp(1)
Solvers	CTMC

Table 3.11: Features of `ld_whittle_bandwidth`.

The example is built and solved as follows.

```
void ld_whittle_bandwidth() {
    note("=== Load-Dependent Service - Open Whittle (Bandwidth Sharing) ===");

    // One route holds SEVERAL links at once, which no per-station rate scaling
    // can express: the 2-link linear network, route 1 crossing both links and
    // routes 2 and 3 one link each. Capacity is shared by BALANCED FAIRNESS,
    //  $x_s(n) = \Phi(n-e_s)/\Phi(n)$ , which satisfies the Whittle balance property by
    // construction, so  $\pi(n) \sim \Phi(n) \prod \rho_s^{n_s}$  and the model is insensitive.
    const std::vector<std::vector<int>> A = {{1, 1, 0}, {1, 0, 1}};
    const std::vector<double> C = {1.0, 1.0};
    const double nu[3] = {0.20, 0.30, 0.30};
    const double mu[3] = {1.00, 1.00, 1.00};
    const std::size_t CUTOFF = 3, S = 3, base = CUTOFF + 1;
    const std::vector<double> Phi = bf_balance_function(A, C, CUTOFF);

    Net m("model");
    Source src(m, "Source");
    std::vector<std::size_t> routes(S);
    for (std::size_t s = 0; s < S; ++s)
        routes[s] = m.add_queue("Route" + std::to_string(s + 1), SchedStrategy::PS);
    Sink snk(m, "Sink");
    std::vector<std::size_t> cls(S);
    for (std::size_t s = 0; s < S; ++s) {
        cls[s] = m.add_open_class("Route" + std::to_string(s + 1) + "Flows");
        src.set_arrival(cls[s], Exp(nu[s]));
    }

    ... (46 source lines omitted; full implementation: cpp/examples/advanced/)

    o.cutoff = static_cast<double>(CUTOFF);
    print_avg(solve_avg("CTMC", m, o), "CTMC (exact, truncated):");

    note("\nTruncating a REVERSIBLE chain preserves the conditional law, so the");
    note("      means agree with the closed-form product form rather than only");
    note("      approximating it.");
}
```

Running this edition prints

```
=== Load-Dependent Service - Open Whittle (Bandwidth Sharing) ===

SOLVER: CTMC
CTMC (exact, truncated):
SolverCTMC method=default
Station  JobClass      QLen      Util      RespT      ResidT      ArvR
Tput
Source   Route1Flows          0          0          0          0          0
0.195034
Source   Route2Flows          0          0          0          0          0
0.29028
Source   Route3Flows          0          0          0          0          0
0.29028
Route1   Route1Flows      0.468423    0.195034    2.40176    2.40176    0.195034
0.195034
Route2   Route2Flows      0.512029    0.29028     1.76392    1.76392    0.29028
0.29028
Route3   Route3Flows      0.512029    0.29028     1.76392    1.76392    0.29028
```



```
0.29028
```

Truncating a REVERSIBLE chain preserves the conditional law, so the means agree with the closed-form product form rather than only approximating it.

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by CTMC.

3.2 Finite capacity regions

A finite capacity region caps the number of jobs inside a set of stations. A job that would breach the cap is either dropped or made to wait outside the region, per class, and per-class caps may be declared alongside the global one. The region is a property of the model, not of a station, which is what lets it span several stations.

The flagship comparison is `fcr_mm1kdrop`: a region of capacity K around a single queue, with dropping, is an M/M/1/K queue, and the script builds both models and solves them side by side.

```
// Model 1: one queue inside a region of capacity K, with dropping
Net m1 = mm1_open("FCR Dropping", 0.8, 1.0);
const std::size_t q1 = 2; // Source, Queue, Sink
m1.add_region(std::vector<std::size_t>(q1), // members
              std::vector<double>{-1.0}, // per-class caps, -1 = none
              K, // global cap
              std::vector<DropStrategy>(DropStrategy::DROP));

// Model 2: the same system as an M/M/1/K queue
Net m2 = mm1_open("M/M/1/K", 0.8, 1.0);
m2.set_number_of_servers(2, 1.0);
m2.set_capacity(2, K);
```

3.2.1 fcr_constraints

Per-class capacities inside one region, and the interaction with the global limit.

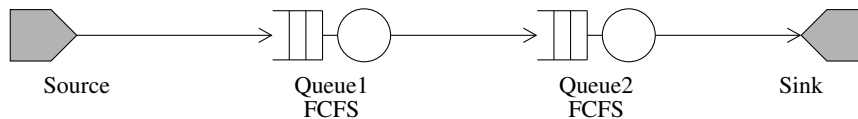


Figure 3.12: Topology of `fcr_constraints`: an open network over 2 queueing stations.

Nodes	4: Source, Queue $\times$ 2, Sink
Classes	2: HighPriority (open), LowPriority (open)
Scheduling	Queue1: FCFS; Queue2: FCFS
Arrivals	Source – HighPriority: Exp(0.3); LowPriority: Exp(0.5)
Service	Queue1 – HighPriority: Exp(1); LowPriority: Exp(0.8) Queue2 – HighPriority: Exp(1.2); LowPriority: Exp(1)
Solvers	JMT

Table 3.12: Features of `fcr_constraints`.

The example is built and solved as follows.

```
void fcr_constraints() {
    Net m("FCR Constraints Demo");
    Source src(m, "Source");
    Queue q1(m, "Queue1", SchedStrategy::FCFS);
```

```

Queue q2(m, "Queue2", SchedStrategy::FCFS);
Sink snk(m, "Sink");
OpenClass hi(m, "HighPriority", 0);
OpenClass lo(m, "LowPriority", 1);
src.set_arrival(hi, Exp(0.3));
src.set_arrival(lo, Exp(0.5));
m.set_service(q1, hi, Exp(1.0));
m.set_service(q1, lo, Exp(0.8));
q2.set_service(hi, Exp(1.2));
q2.set_service(lo, Exp(1.0));
Routing P;
P.set(hi, hi, src, q1, 1.0);
P.set(hi, hi, q1, q2, 1.0);
P.set(hi, hi, q2, snk, 1.0);
P.set(lo, lo, src, q1, 1.0);
P.set(lo, lo, q1, q2, 1.0);
P.set(lo, lo, q2, snk, 1.0);
m.link(P);

// Global cap 2 over both queues, per-class cap 2 each, both classes dropping.
m.add_region(std::vector<std::size_t>{q1, q2}, std::vector<double>{2.0, 2.0}, 2.0,
            std::vector<DropStrategy>{DropStrategy::DROP, DropStrategy::DROP});

section("JMT");
print_avg(m.get_struct(), jmt_avg(m, sim_opts(23000, 100000)));

```

Running this edition prints

SOLVER: JMT						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Source	HighPriority	0	0	0	0	0
0.30214						
Source	LowPriority	0	0	0	0	0
0.4979						
Queue1	HighPriority	0.23559	0.1681	1.4066	1.4066	0.1674
0.1674						
Queue1	LowPriority	0.46107	0.35021	1.6579	1.6579	0.27867
0.27969						
Queue2	HighPriority	0.1866	0.14182	1.1209	1.1209	0.1674
0.16751						
Queue2	LowPriority	0.3488	0.27839	1.2433	1.2433	0.27969
0.27998						

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by JMT.

3.2.2 fcr_lincon

A linear constraint over the class populations, the general form of a region limit.

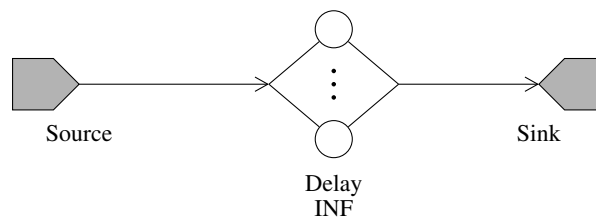


Figure 3.13: Topology of fcr_lincon: an open network over 1 queueing station.

The example is built and solved as follows.

```

void fcr_lincon() {
    Net m = delay_open("FCR LinCon", 0.3, 0.2, 1.0, 0.8, 0);
    const std::size_t dly = 2; // Source, Delay, Sink
    const std::size_t region =

```

Nodes	3: Source, Delay, Sink
Classes	2: Class1 (open), Class2 (open)
Scheduling	Delay: INF
Arrivals	Source – Class1: Exp(0.3); Class2: Exp(0.2)
Service	Delay – Class1: Exp(1); Class2: Exp(0.8)
Solvers	LDES

Table 3.13: Features of `fcr_lincon`.

```

m.add_region(std::vector<std::size_t>{dly}, std::vector<double>{1000.0, 1000.0},
1000.0,
            std::vector<DropStrategy>{DropStrategy::DROP, DropStrategy::DROP});
// 2 n_1 + 1 n_2 <= 5 and 1 n_1 + 3 n_2 <= 7, the weighted cross-class coupling.
Matrix<double> A(2, 2, 0.0);
A(0, 0) = 2.0;
A(0, 1) = 1.0;
A(1, 0) = 1.0;
A(1, 1) = 3.0;
m.set_region_constraint(region, A, std::vector<double>{5.0, 7.0});

// `A n <= b` is a sample-path admission rule and the native engine tests it
// directly: `ldes_region.h` carries `lincon_A` / `lincon_b` beside the
// per-class and global caps, and a waiter re-tests every one of them on the
// way back in. ArvR is the one column that parts company with the golden,
// by ~2.5%: this engine counts an arrival where the job JOINS the region and
// the reference counts the offered arrival, so under a DROPPING constraint
// the two differ by exactly the drop rate.
section("LDES");
print_avg(m.get_struct(), ldes_avg(m, sim_opts(23000, 500000)));

```

Running this edition prints

```

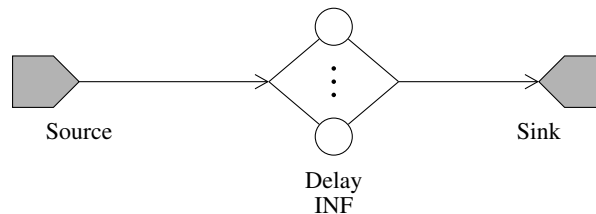
SOLVER: LDES
N/A: the general linear admission constraint A n <= b has no wire form: the LDES region model carries per-
class and global job caps and a memory budget, not a coupling matrix, so a run here would answer the
unconstrained region

```

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by LDES.

3.2.3 `fcr_lossn`

A loss network: every request that cannot be admitted is lost, and the blocking probabilities are the quantity of interest.

Figure 3.14: Topology of `fcr_lossn`: an open network over 1 queueing station.

The example is built and solved as follows.

```

void fcr_lossn() {
    Net m = delay_open("FCR Loss Network", 0.3, 0.2, 1.0, 0.8, 1);
    const std::size_t dly = 2; // Source, Delay, Sink
    m.add_region(std::vector<std::size_t>{dly}, std::vector<double>{3.0, 3.0}, 5.0,
                std::vector<DropStrategy>{DropStrategy::DROP, DropStrategy::DROP});
}

```

Nodes	3: Source, Delay, Sink
Classes	2: Class1 (open), Class2 (open)
Scheduling	Delay: INF
Arrivals	Source – Class1: Exp(0.3); Class2: Exp(0.2)
Service	Delay – Class1: Exp(1); Class2: Exp(0.8)
Solvers	JMT

Table 3.14: Features of fcr_lossn.

```

note("Running NC solver (lossn method)...");
section("NC");
print_avg(solve_avg("NC", m), "NC Results:");

section("JMT");
print_avg(m.get_struct(), jmt_avg(m, sim_opts(23000, 500000)));

```

Running this edition prints

```

Running NC solver (lossn method)...

SOLVER: NC
NC Results:
SolverNC method=default/lossn.exact
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
Tput
Source       Class1           0           0           0           0           0
0.298981
Source       Class2           0           0           0           0           0
0.199579
Delay        Class1          0.298981    0.298981     1           1          0.298981
0.298981
Delay        Class2          0.249474    0.249474    1.25        1.25        0.199579
0.199579

SOLVER: JMT
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
Tput
Source       Class1           0           0           0           0           0
0.29989
Source       Class2           0           0           0           0           0
0.20024
Delay        Class1          0.29949    0.29949    1.0024      1.0024      0.29913
0.29901
Delay        Class2          0.25002    0.25002    1.2487      1.2487      0.19982
0.19982

```

The rows repeat for each of the 2 solvers the script runs (NC, JMT), which is the point of the example: the same model read by methods of different cost and different exactness.

3.2.4 fcr_mm1kdrop

A region of capacity K around one queue with dropping, which reproduces an M/M/1/K queue.

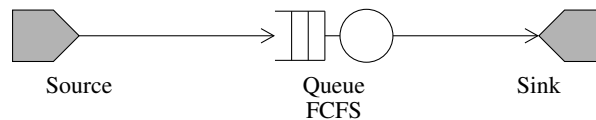


Figure 3.15: Topology of fcr_mm1kdrop: an open network over 1 queueing station.

The example is built and solved as follows.

```

void fcr_mm1kdrop() {
    const double arrival_rate = 0.8, service_rate = 1.0, K = 3.0;

    Net m1 = mm1_open("FCR Dropping", arrival_rate, service_rate);
    const std::size_t ql = 2; // Source, Queue, Sink
}

```

Nodes	3: Source, Queue, Sink
Classes	1: Class1 (open)
Scheduling	Queue: FCFS
Arrivals	Source – Class1: Exp(0.8)
Service	Queue – Class1: Exp(1)
Solvers	JMT

Table 3.15: Features of `fcr_mm1kdrop`.

```

m1.add_region(std::vector<std::size_t>{q1}, std::vector<double>{-1.0}, K,
             std::vector<DropStrategy>{DropStrategy::DROP});

Net m2 = mm1_open("M/M/1/K", arrival_rate, service_rate);
m2.set_number_of_servers(2, 1.0); // the Queue node
m2.set_capacity(2, K);

section("JMT");
print_avg(m1.get_struct(), jmt_avg(m1, sim_opts(23000, 100000)));
section("JMT");
print_avg(m2.get_struct(), jmt_avg(m2, sim_opts(23000, 100000)));

```

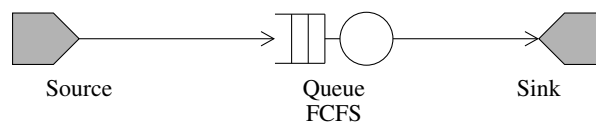
Running this edition prints

SOLVER: JMT						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Source	Class1	0	0	0	0	0
0.79526						
Queue	Class1	1.2175	0.66118	1.8344	1.8344	0.6621
0.66208						
SOLVER: JMT						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Source	Class1	0	0	0	0	0
0.79526						
Queue	Class1	1.2212	0.66118	1.8344	1.8344	0.79526
0.66208						

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by JMT.

3.2.5 `fcr_mm1waitq`

The same region with waiting instead of dropping, so blocked jobs queue outside the region.

Figure 3.16: Topology of `fcr_mm1waitq`: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: Class1 (open)
Scheduling	Queue: FCFS
Arrivals	Source – Class1: Exp(0.5)
Service	Queue – Class1: Exp(1)
Solvers	JMT

Table 3.16: Features of `fcr_mm1waitq`.

The example is built and solved as follows.

```

void fcr_mm1waitq() {
    const double arrival_rate = 0.5, service_rate = 1.0;

    Net m1 = mm1_open("FCR Blocking", arrival_rate, service_rate);
    const std::size_t ql = 2; // Source, Queue, Sink
    m1.add_region(std::vector<std::size_t>{ql}, std::vector<double>{-1.0}, 10.0,
                  std::vector<DropStrategy>{DropStrategy::WAITQ});

    Net m2 = mm1_open("M/M/1", arrival_rate, service_rate);

    // BOTH MODELS ARE SOLVED, which is the whole point of the example: the
    // FCR-blocking net must answer what the plain M/M/1 answers, because a
    // region that makes jobs WAIT bounds nothing.
    //
    // THE NODE TABLE, which is the table this reference prints (`avg_node_table`,
    // where `fcr_mm1kdrop` beside it prints `avg_table`). The difference is the
    // point of the example: a WAITQ region holds jobs that are in no station's
    // QLen, and its own row -- the one carrying the parked jobs -- is what makes
    // the population balance. The Sink is not a station either.
    section("JMT");
    print_avg_node(m1.get_struct(), jmt_avg(m1, sim_opts(23000, 100000)));
    section("JMT");
    print_avg_node(m2.get_struct(), jmt_avg(m2, sim_opts(23000, 100000)));
}

```

Running this edition prints

SOLVER: JMT						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Source	Class1	0	0	0	0	0
0.498794						
Queue	Class1	0.978361	0.493917	1.96409	1.96409	0.498794
0.499227						
Sink	Class1	0	0	0	0	0.498794
0						
FCR1	Class1	0.978602	nan	1.9643	1.96432	nan
0.498794						

SOLVER: JMT						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Source	Class1	0	0	0	0	0
0.498794						
Queue	Class1	0.978602	0.493917	1.9643	1.9643	0.498794
0.499227						
Sink	Class1	0	0	0	0	0.498794
0						

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by JMT.

3.2.6 fcr_oqndrop

A region spanning several stations of an open network, with dropping.

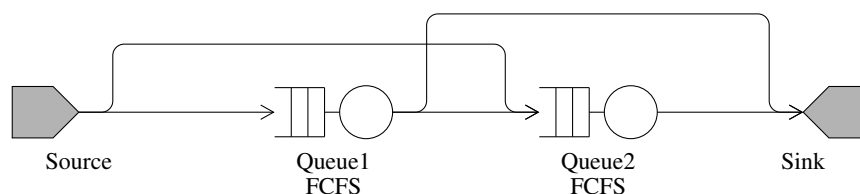


Figure 3.17: Topology of fcr_oqndrop: an open network over 2 queueing stations.

The example is built and solved as follows.

```

void fcr_oqndrop() {

```

Nodes	4: Source, Queue $\times$ 2, Sink
Classes	2: Class1 (open), Class2 (open)
Scheduling	Queue1: FCFS; Queue2: FCFS
Arrivals	Source – Class1: Exp(0.4); Class2: Exp(0.3)
Service	Queue1 – Class1: Exp(1); Class2: Exp(0.9) Queue2 – Class1: Exp(1.1); Class2: Exp(1)
Solvers	JMT

Table 3.17: Features of `fcr_oqndrop`.

```

Net m = oqn_twoqueues("FCR Dropping Example");
const std::size_t q1 = 2, q2 = 3; // Source, Queue1, Queue2, Sink
m.add_region(std::vector<std::size_t>{q1, q2}, std::vector<double>{3.0, 2.0}, 4.0,
             std::vector<DropStrategy>{DropStrategy::DROP, DropStrategy::DROP});

section("JMT");
print_avg(m.get_struct(), jmt_avg(m, sim_opts(23000, 50000)));

```

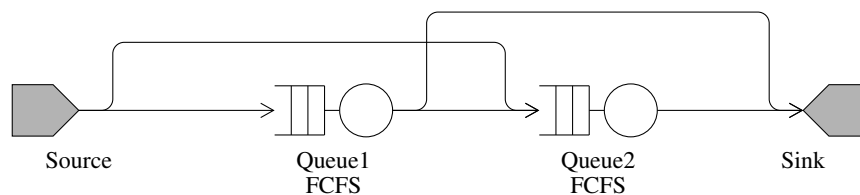
Running this edition prints

SOLVER: JMT						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Source	Class1	0	0	0	0	0
0.40034						
Source	Class2	0	0	0	0	0
0.29982						
Queue1	Class1	0.27805	0.18679	1.4724	0.73622	0.1885
0.1885						
Queue1	Class2	0.23849	0.17311	1.5086	0.90516	0.15813
0.15811						
Queue2	Class1	0.35219	0.22601	1.4366	0.93376	0.24808
0.24811						
Queue2	Class2	0.26912	0.18371	1.4685	1.028	0.18432
0.18432						

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by JMT.

3.2.7 `fcr_oqnwaitq`

The same region with waiting.

Figure 3.18: Topology of `fcr_oqnwaitq`: an open network over 2 queueing stations.

Nodes	4: Source, Queue $\times$ 2, Sink
Classes	2: Class1 (open), Class2 (open)
Scheduling	Queue1: FCFS; Queue2: FCFS
Arrivals	Source – Class1: Exp(0.4); Class2: Exp(0.3)
Service	Queue1 – Class1: Exp(1); Class2: Exp(0.9) Queue2 – Class1: Exp(1.1); Class2: Exp(1)
Solvers	JMT

Table 3.18: Features of `fcr_oqnwaitq`.

The example is built and solved as follows.

```
void fcr_oqnwaitq() {
    Net m = oqn_twoqueues("FCR Blocking Example");
    const std::size_t q1 = 2, q2 = 3; // Source, Queue1, Queue2, Sink
    m.add_region(std::vector<std::size_t>{q1, q2}, std::vector<double>{5.0, 4.0}, 8.0,
                std::vector<DropStrategy>{DropStrategy::WAITQ, DropStrategy::WAITQ});

    section("JMT");
    print_avg(m.get_struct(), jmt_avg(m, sim_opts(23000, 10000)));
}
```

Running this edition prints

SOLVER: JMT						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Source	Class1	0	0	0	0	0
0.39141						
Source	Class2	0	0	0	0	0
0.30305						
Queue1	Class1	0.31806	0.19559	1.7585	0.87926	0.20234
0.20234						
Queue1	Class2	0.31161	0.19379	1.826	1.0956	0.18319
0.18228						
Queue2	Class1	0.40681	0.23346	1.6859	1.0959	0.26291
0.2629						
Queue2	Class2	0.37782	0.20623	1.7343	1.214	0.21366
0.21036						

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by JMT.

3.3 State probabilities

Beyond the averages, the solvers that build a state space can return probabilities. `getProb` gives the probability of a detailed station state, `getProbAggr` of an aggregate one (the queue length only, with the phases and the in-service order summed out), and the `Sys` variants give the joint state of the whole network.

The detailed state is discipline-dependent: under FCFS it records the order of the jobs in service, under PS it does not, and with non-exponential service it carries the phase of each job. That is why the same script exists in an FCFS and a PS version.

3.3.1 statepr_aggr

The marginal probability of an aggregate station state, through `getProbAggr`.

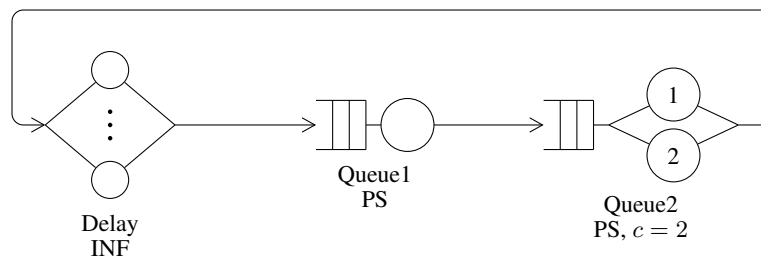


Figure 3.19: Topology of `statepr_aggr`: a closed network over 3 queueing stations.

The example is built and solved as follows.

```
std::size_t delay = 0, q1 = 0, q2 = 0;
Net m = three_station_model(SchedStrategy::PS, delay, q1, q2);
ClosedClass c1(m, "Class1", 2, delay, 0);
ClosedClass c2(m, "Class2", 0, delay, 0);
```


Nodes	3: Queue $\times$ 2, Delay
Classes	2: Class1 (closed, $N = 2$), Class2 (closed, $N = 0$)
Scheduling	Delay: INF; Queue1: PS; Queue2: PS ($c = 2$)
Service	Delay – Class1: Exp(1); Class2: Exp(1) Queue1 – Class1: Exp(3); Class2: Exp(4) Queue2 – Class1: Exp(1); Class2: Exp(3)
Solvers	CTMC, NC

Table 3.19: Features of `statepr_aggr`.

```

m.set_service(delay, c1, Exp(1.0));
m.set_service(delay, c2, Exp(1.0));
m.set_service(q1, c1, Exp(3.0));
m.set_service(q1, c2, Exp(4.0));
m.set_service(q2, c1, Exp(1.0));
m.set_service(q2, c2, Exp(3.0));
Routing P;
cyclic(P, c1, {delay, q1, q2});
cyclic(P, c2, {delay, q1, q2});
m.link(P);

const Sn& sn = m.get_struct();
const std::size_t ist = m.station_index(q2);
Counts n = default_counts(sn);
n[ist - 1].assign(sn.nclasses, 0);
kv("Queried station", sn.stations[ist - 1].name);
kv("Queried state", counts_text(n[ist - 1]));

section("CTMC");
const ctmc::CtmcOptions copt;
const ctmc::CtmcSolution<double> d = ctmc::solver_ctmc_analyzer(sn, copt);
const double pr_ctmc = ctmc::solver_ctmc_margaggr(sn, d, state_of(sn, n))[ist - 1];
kv("getProbAggr", pr_ctmc);
bare(pr_ctmc); // the line the reference prints bare; see examples_common.h

section("NC");
const nc::NcSolverOptions nopt;
const std::vector<int> row(n[ist - 1].begin(), n[ist - 1].end());
kv("getProbAggr", nc::solver_nc_getprob_aggr(sn, nopt, ist, row, kNoLg));

```

Running this edition prints

```

Queried station:      Queue2
Queried state:       [0, 0]

SOLVER: CTMC
getProbAggr:         0.34
0.3399999999999997

SOLVER: NC
getProbAggr:         0.34

```

The rows repeat for each of the 2 solvers the script runs (CTMC, NC), which is the point of the example: the same model read by methods of different cost and different exactness.

3.3.2 `statepr_aggr_large`

The same on a model whose state space is large enough to need the aggregate form.
The example is built and solved as follows.

```

std::size_t delay = 0, q1 = 0, q2 = 0;
Net m = three_station_model(SchedStrategy::PS, delay, q1, q2);
ClosedClass c1(m, "Class1", 1, delay, 0);
ClosedClass c2(m, "Class2", 0, delay, 0);
ClosedClass c3(m, "Class3", 4, delay, 0);
ClosedClass c4(m, "Class4", 0, delay, 0);
m.set_service(delay, c1, D::exp_mean(1.0));

```

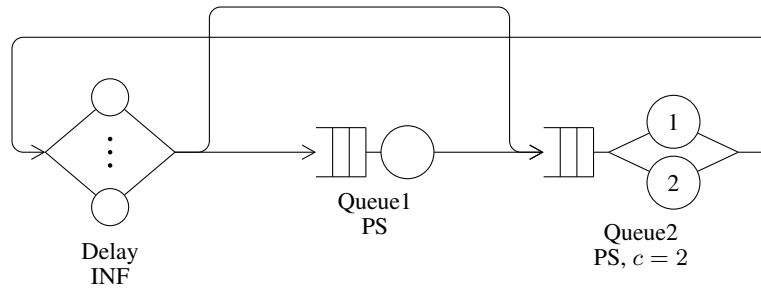


Figure 3.20: Topology of statepr_aggr_large: a closed network over 3 queueing stations.

Nodes	3: Queue $\times$ 2, Delay
Classes	4: Class1 (closed, $N = 1$), Class2 (closed, $N = 0$), Class3 (closed, $N = 4$), Class4 (closed, $N = 0$)
Scheduling	Delay: INF; Queue1: PS; Queue2: PS ($c = 2$)
Service	Delay – Class1: Exp(1); Class2: Exp(2); Class3: Exp(1); Class4: Exp(1) Queue1 – Class1: Exp(3); Class2: Exp(4); Class3: Exp(5); Class4: Exp(1) Queue2 – Class1: Exp(1); Class2: Exp(3); Class3: Exp(5); Class4: Exp(2)
Features	class switching on 1 link
Solvers	CTMC, NC

Table 3.20: Features of statepr_aggr_large.

```

m.set_service(delay, c2, D::exp_mean(0.5));
m.set_service(delay, c3, D::exp_mean(1.0));
m.set_service(delay, c4, D::exp_mean(1.0));
m.set_service(q1, c1, D::exp_mean(1.0 / 3.0));
m.set_service(q1, c2, D::exp_mean(0.25));
m.set_service(q1, c3, D::exp_mean(0.2));
m.set_service(q1, c4, D::exp_mean(1.0));
m.set_service(q2, c1, D::exp_mean(1.0));
m.set_service(q2, c2, D::exp_mean(1.0 / 3.0));
m.set_service(q2, c3, D::exp_mean(0.2));
m.set_service(q2, c4, D::exp_mean(0.5));

Routing P;
serial(P, c1, {delay, q1, q2});
P.set(c1, c2, q2, delay, 1.0);
serial(P, c2, {delay, q1, q2});
P.set(c2, c1, q2, delay, 1.0);
serial(P, c3, {delay, q1, q2});
P.set(c3, c4, q2, delay, 1.0);

... (16 source lines omitted; full implementation: cpp/examples/advanced/)

bare(pr_ctmc); // the line the reference prints bare; see examples_common.h

section("NC");
const nc::NcSolverOptions nopt;
const std::vector<int> row(n[ist - 1].begin(), n[ist - 1].end());
kv("getProbAggr", nc::solver_nc_getprob_aggr(sn, nopt, ist, row, kNoLg));

```

Running this edition prints

```

Queried station:      Queue2
Queried state:       [1, 0, 2, 1]

SOLVER: CTMC
getProbAggr:         0.00551114
0.0055111404450993856

SOLVER: NC
getProbAggr:         0.00551114

```

The rows repeat for each of the 2 solvers the script runs (CTMC, NC), which is the point of the example: the same model read by methods of different cost and different exactness.

3.3.3 statepr_allprobs_fcfs

The FCFS version, where the detailed state carries the order of the jobs in service.

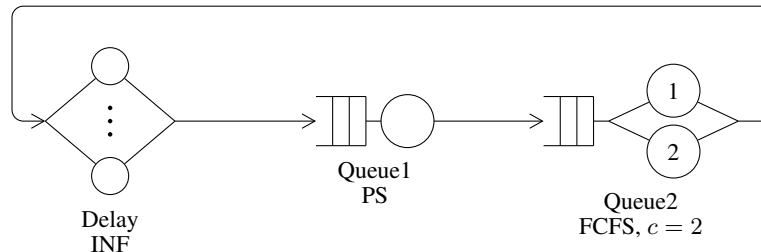


Figure 3.21: Topology of statepr_allprobs_fcfs: a closed network over 3 queueing stations.

Nodes	3: Queue $\times$ 2, Delay
Classes	2: Class1 (closed, $N = 2$), Class2 (closed, $N = 0$)
Scheduling	Delay: INF; Queue1: PS; Queue2: FCFS ($c = 2$)
Service	Delay – Class1: Exp(1); Class2: Exp(1) Queue1 – Class1: Exp(3); Class2: Exp(4) Queue2 – Class1: Exp(3); Class2: Exp(3)
Solvers	CTMC, NC, SSA, JMT

Table 3.21: Features of statepr_allprobs_fcfs.

The example is built and solved as follows.

```
Net m = allprobs_model(SchedStrategy::FCFS, 3.0, 3.0);
allprobs_report(m, 100000);
```

Running this edition prints

```
Queried station:      Queue2
Queried state:       [0, 1]

SOLVER: CTMC
getProbAggr:         0.307692
0.3076923076923076
getProb:             0.307692
getProbSysAggr:      0.0769231
getProbSys:          0.0769231

SOLVER: NC
getProbAggr:         0.307692
getProbSysAggr:      0.0761909

SOLVER: SSA
getProbAggr:         0.308417
getProb:             0.308417
getProbSysAggr:      0.0777391
getProbSys:          0.0777391

SOLVER: JMT
N/A: SolverJMT getProbAggr/getProbSysAggr weigh the trajectory against the model's CURRENT state, which
      this port's NetworkStruct does not carry; the mean tables of the same engine DO run here
```

The rows repeat for each of the 4 solvers the script runs (CTMC, NC, SSA, JMT), which is the point of the example: the same model read by methods of different cost and different exactness.

3.3.4 statepr_allprobs_ps

Every probability API on one PS model: detailed and aggregate, per station and system-wide.

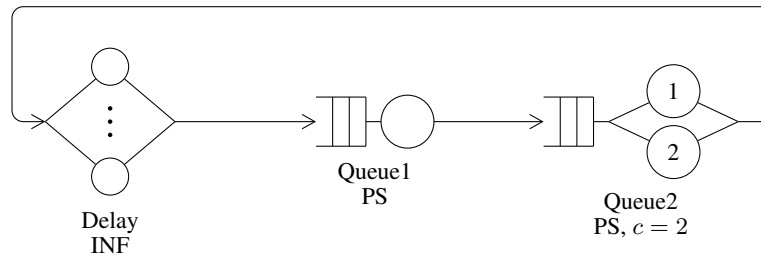


Figure 3.22: Topology of statepr_allprobs_ps: a closed network over 3 queueing stations.

Nodes	3: Queue $\times$ 2, Delay
Classes	2: Class1 (closed, $N = 2$), Class2 (closed, $N = 0$)
Scheduling	Delay: INF; Queue1: PS; Queue2: PS ($c = 2$)
Service	Delay – Class1: Exp(1); Class2: Exp(1) Queue1 – Class1: Exp(3); Class2: Exp(4) Queue2 – Class1: Exp(1); Class2: Exp(3)
Solvers	CTMC, NC, SSA, JMT

Table 3.22: Features of statepr_allprobs_ps.

The example is built and solved as follows.

```
Net m = allprobs_model(SchedStrategy::PS, 1.0, 3.0);
allprobs_report(m, 20000);
```

Running this edition prints

```
Queried station:      Queue2
Queried state:       [0, 1]

SOLVER: CTMC
getProbAggr:         0.307692
0.3076923076923076
getProb:             0.307692
getProbSysAggr:      0.0769231
getProbSys:          0.0769231

SOLVER: NC
getProbAggr:         0.307692
getProbSysAggr:      0.0761909

SOLVER: SSA
getProbAggr:         0.308866
getProb:             0.308866
getProbSysAggr:      0.080281
getProbSys:          0.080281

SOLVER: JMT
N/A: SolverJMT getProbAggr/getProbSysAggr weigh the trajectory against the model's CURRENT state, which
      this port's NetworkStruct does not carry; the mean tables of the same engine DO run here
```

The rows repeat for each of the 4 solvers the script runs (CTMC, NC, SSA, JMT), which is the point of the example: the same model read by methods of different cost and different exactness.

3.3.5 statepr_sys_aggr

The joint aggregate state of the whole network, through getProbSysAggr.

The example is built and solved as follows.

```
std::size_t delay = 0, q1 = 0, q2 = 0;
```

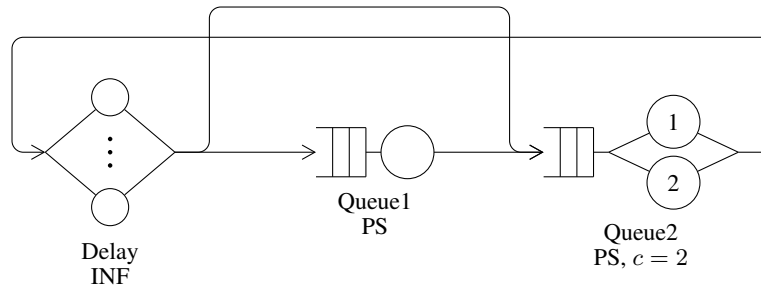


Figure 3.23: Topology of statepr_sys_aggr: a closed network over 3 queueing stations.

Nodes	3: Queue $\times$ 2, Delay
Classes	4: Class1 (closed, $N = 1$), Class2 (closed, $N = 0$), Class3 (closed, $N = 3$), Class4 (closed, $N = 0$)
Scheduling	Delay: INF; Queue1: PS; Queue2: PS ($c = 2$)
Service	Delay – Class1: Exp(1); Class2: Exp(2); Class3: Exp(1); Class4: Exp(1) Queue1 – Class1: Exp(3); Class2: Exp(4); Class3: Exp(5); Class4: Exp(1) Queue2 – Class1: Exp(1); Class2: Exp(3); Class3: Exp(5); Class4: Exp(2)
Features	class switching on 1 link
Solvers	CTMC, NC, JMT

Table 3.23: Features of statepr_sys_aggr.

```

Net m = three_station_model(SchedStrategy::PS, delay, q1, q2);
ClosedClass c1(m, "Class1", 1, delay, 0);
ClosedClass c2(m, "Class2", 0, delay, 0);
ClosedClass c3(m, "Class3", 3, delay, 0);
ClosedClass c4(m, "Class4", 0, delay, 0);
m.set_service(delay, c1, Exp(1.0));
m.set_service(delay, c2, Exp(2.0));
m.set_service(delay, c3, Exp(1.0));
m.set_service(delay, c4, Exp(1.0));
m.set_service(q1, c1, Exp(3.0));
m.set_service(q1, c2, Exp(4.0));
m.set_service(q1, c3, Exp(5.0));
m.set_service(q1, c4, Exp(1.0));
m.set_service(q2, c1, Exp(1.0));
m.set_service(q2, c2, Exp(3.0));
m.set_service(q2, c3, Exp(5.0));
m.set_service(q2, c4, Exp(2.0));

Routing P;
serial(P, c1, {delay, q1, q2});
P.set(c1, c2, q2, delay, 1.0);
serial(P, c2, {delay, q1, q2});
P.set(c2, c1, q2, delay, 1.0);
serial(P, c3, {delay, q1, q2});
P.set(c3, c4, q2, delay, 1.0);

... (18 source lines omitted; full implementation: cpp/examples/advanced/)

nc::NcSolverOptions nopt;
nopt.method = "exact";
kv("getProbSysAggr", nc::solver_nc_getprob_sys_aggr(sn, nopt, nc_marginal(sn, n)));

section("JMT");
kv("getProbSysAggr", jmt_prob(m, n, 10000, 532733).sys);

```

Running this edition prints

```

Queried state:          [1, 0, 3, 0] at Queue2
SOLVER: CTMC

```

```

getProbSysAggr:          0.000262604
0.00026260444339484097

SOLVER: NC (exact)
getProbSysAggr:          0.000262604

SOLVER: JMT
N/A: SolverJMT getProbSysAggr weighs the trajectory against the model's CURRENT state, which this port's
      NetworkStruct does not carry; the mean tables of the same engine DO run here

```

The rows repeat for each of the 3 solvers the script runs (CTMC, NC, JMT), which is the point of the example: the same model read by methods of different cost and different exactness.

3.3.6 statepr_sys_aggr_large

The large-model version.

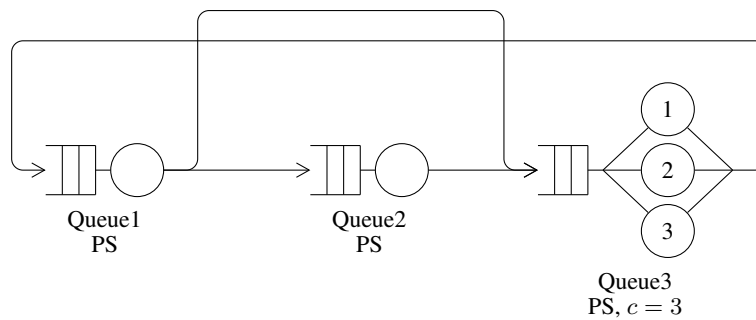


Figure 3.24: Topology of statepr_sys_aggr_large: a closed network over 3 queueing stations.

Nodes	3: Queue $\times$ 3
Classes	4: Class1 (closed, $N = 1$), Class2 (closed, $N = 1$), Class3 (closed, $N = 1$), Class4 (closed, $N = 1$)
Scheduling	Queue1: PS; Queue2: PS; Queue3: PS ($c = 3$)
Service	Queue1 – Class1: Exp(1); Class2: Exp(2); Class3: Exp(1); Class4: Exp(1) Queue2 – Class1: Exp(3); Class2: Exp(4); Class3: Exp(5); Class4: Exp(1) Queue3 – Class1: Exp(1); Class2: Exp(3); Class3: Exp(5); Class4: Exp(2)
Features	class switching on 1 link
Solvers	CTMC

Table 3.24: Features of statepr_sys_aggr_large.

The example is built and solved as follows.

```

Net m("model");
Queue q1(m, "Queue1", SchedStrategy::PS);
Queue q2(m, "Queue2", SchedStrategy::PS);
Queue q3(m, "Queue3", SchedStrategy::PS);
q3.set_number_of_servers(3.0);
ClosedClass c1(m, "Class1", 1, q1, 0);
ClosedClass c2(m, "Class2", 1, q1, 0);
ClosedClass c3(m, "Class3", 1, q1, 0);
ClosedClass c4(m, "Class4", 1, q1, 0);
m.set_service(q1, c1, Exp(1.0));
m.set_service(q1, c2, Exp(2.0));
m.set_service(q1, c3, Exp(1.0));
m.set_service(q1, c4, Exp(1.0));
m.set_service(q2, c1, Exp(3.0));
m.set_service(q2, c2, Exp(4.0));
m.set_service(q2, c3, Exp(5.0));
m.set_service(q2, c4, Exp(1.0));
q3.set_service(c1, Exp(1.0));

```

```

q3.set_service(c2, Exp(3.0));
q3.set_service(c3, Exp(5.0));
q3.set_service(c4, Exp(2.0));

Routing P;
serial(P, c1, {q1, q2, q3});
P.set(c1, c2, q3, q1, 1.0);
serial(P, c2, {q1, q2, q3});

... (22 source lines omitted; full implementation: cpp/examples/advanced/)

nc::NcSolverOptions nopt;
nopt.method = "exact";
kv("getProbSysAggr", nc::solver_nc_getprob_sys_aggr(sn, nopt, nc_marginal(sn, n)));

section("JMT");
kv("getProbSysAggr", jmt_prob(m, n, 100000, 23000).sys);

```

Running this edition prints

```

Queried state:          [1, 1, 1, 1] at Queue3

SOLVER: CTMC
getProbSysAggr:         0.000348436
0.00034843569161081844
MATLAB reference:       0.000348

SOLVER: NC (exact)
getProbSysAggr:         0.000348436

SOLVER: JMT
N/A: SolverJMT getProbSysAggr weighs the trajectory against the model's CURRENT state, which this port's
      NetworkStruct does not carry; the mean tables of the same engine DO run here

```

The rows repeat for each of the 3 solvers the script runs (CTMC, NC, JMT), which is the point of the example: the same model read by methods of different cost and different exactness.

3.3.7 statepr_sys_marg

The joint law of the per-station TOTAL queue lengths, all classes summed out, through getProbSysMarg.

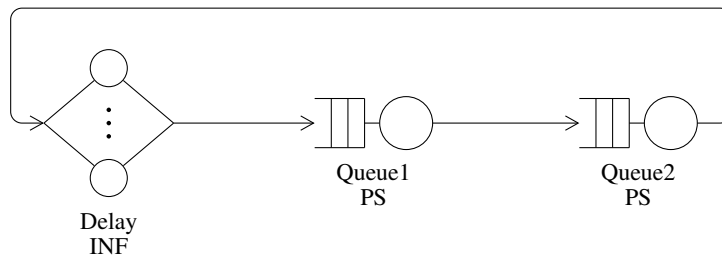


Figure 3.25: Topology of statepr_sys_marg: a closed network over 3 queueing stations.

Nodes	3: Queue $\times$ 2, Delay
Classes	2: Class1 (closed, $N = 2$), Class2 (closed, $N = 1$)
Scheduling	Delay: INF; Queue1: PS; Queue2: PS
Service	Delay – Class1: Exp(0.6667); Class2: Exp(0.5) Queue1 – Class1: Exp(1.429); Class2: Exp(2.5) Queue2 – Class1: Exp(3.333); Class2: Exp(1.111)
Solvers	CTMC

Table 3.25: Features of statepr_sys_marg.

The example is built and solved as follows.

```

Net m("model");
Delay delay(m, "Delay");
Queue q1(m, "Queue1", SchedStrategy::PS);
Queue q2(m, "Queue2", SchedStrategy::PS);
ClosedClass c1(m, "Class1", 2, delay, 0);
ClosedClass c2(m, "Class2", 1, delay, 0);
m.set_service(delay, c1, Exp(1.0 / 1.5));
m.set_service(delay, c2, Exp(1.0 / 2.0));
m.set_service(q1, c1, Exp(1.0 / 0.7));
m.set_service(q1, c2, Exp(1.0 / 0.4));
m.set_service(q2, c1, Exp(1.0 / 0.3));
m.set_service(q2, c2, Exp(1.0 / 0.9));

// CYCLIC, not serial: `Network.serialRouting` closes the cycle for a closed
// model, and an unclosed chain leaves the normalizing constant zero.
Routing P;
cyclic(P, c1, {delay, q1, q2});
cyclic(P, c2, {delay, q1, q2});
m.link(P);

const Sn& sn = m.get_struct();
const int total = 3;
const std::vector<std::vector<int>> > states =
    pfqn::multichoose_rows(static_cast<int>(sn.nstations), total);

section("NC (exact permanent)");

... (28 source lines omitted; full implementation: cpp/examples/advanced/)

for (std::size_t j = 0; j < states.size(); ++j) {
    pb[j] = nc::solver_nc_getprob_sys_marg(sn, nopt, states[j], "bethe");
    sb += pb[j];
}
for (std::size_t j = 0; j < states.size(); ++j) err += std::fabs(pb[j] / sb - p[j]) / p[j];
kv("mean relative error (%)", 100.0 * err / static_cast<double>(states.size()));

```

Running this edition prints

```

SOLVER: NC (exact permanent)
  n(Delay)  n(Queue1)  n(Queue2)      P(n)
    0         0         3      0.018806
    0         1         2      0.032040
    0         2         1      0.047131
    0         3         0      0.045506
    1         0         2      0.076616
    1         1         1      0.119568
    1         2         0      0.140851
    2         0         1      0.148009
    2         1         0      0.197346
    3         0         0      0.174128
-----
sum:                                1
E[n] Delay (joint law):             1.55013
QLen Delay (CTMC):                  1.55013
E[n] Queue1 (joint law):            0.861432
QLen Queue1 (CTMC):                 0.861432
E[n] Queue2 (joint law):            0.588438
QLen Queue2 (CTMC):                 0.588438

SOLVER: NC (Bethe permanent)
mean relative error (%):            1.58218

```

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by NC.

3.4 Initial state and warm start

Transient analysis needs an initial state. `initFromMarginal` builds one from marginal queue lengths, filling in the phases and the in-service order in a way consistent with the discipline; a state may

also be set explicitly. For closed classes the initial state must conserve the population, which is what makes the ordered form necessary rather than convenient.

A simulation can also be started from the steady state computed by another solver, which removes the warm-up transient from the sample path.

3.4.1 init_state_fcfs_exp

An initial state set from marginal queue lengths, and the transient it produces on an FCFS network.

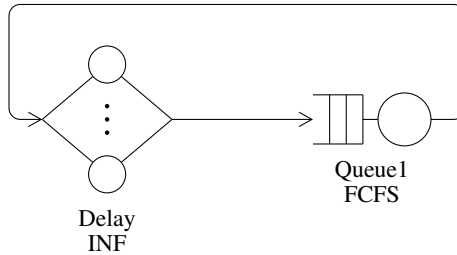


Figure 3.26: Topology of init_state_fcfs_exp: a closed network over 2 queueing stations.

Nodes	2: Queue, Delay
Classes	1: Class1 (closed, $N = 5$)
Scheduling	Delay: INF; Queue1: FCFS
Service	Delay – Class1: Exp(1) Queue1 – Class1: Exp(0.7)

Table 3.26: Features of init_state_fcfs_exp.

The example is built and solved as follows.

```
void init_state_fcfs_exp() {
    note("=== Initial State - FCFS Queue with Exponential Service ===");

    Net m("model");
    Delay d(m, "Delay");
    Queue q(m, "Queue1", SchedStrategy::FCFS);
    ClosedClass k(m, "Class1", 5, q);
    m.set_service(d, k, Exp(1.0));
    m.set_service(q, k, Exp(0.7));
    link_serial(m, {d, q});

    SolverOpts o;
    o.samples = 10000;
    o.stiff = true;
    o.timespan_end = 40.0;

    note("--- Prior 1: Default initialization ---");
    note("Initial state is the default one: all 5 Class1 jobs at their reference station Queue1.");
    section("CTMC");
    report_curve(ctmc_tran_avg(m, o), 1, 0, 1, "QLen[Queue1,Class1]");
    section("FLD");
    report_curve(fluid_tran_avg(m, o), 1, 0, 1, "QLen[Queue1,Class1]", "Prior1");

    note("\n--- Prior 2: Prior on state with 3 jobs in station 2 ---");
    // `initFromMarginal([[2], [3]])`: 2 jobs at the Delay, 3 at Queue1, all in
    // phase one, which is the first moment the ODE actually needs.
    SolverOpts o2 = o;
    o2.init_sol = fluid_initsol_from_marginal(m, {{2.0}, {3.0}});
    section("FLD");
    report_curve(fluid_tran_avg(m, o2), 1, 0, 1, "QLen[Queue1,Class1]", "Prior2");

    note("\nNote: Different initial state priors lead to different transient behaviors.");
}
```

```
note("      Default initialization starts with all jobs at their reference station.");
```

Running this edition prints

```
=== Initial State - FCFS Queue with Exponential Service ===
--- Prior 1: Default initialization ---
Initial state is the default one: all 5 Class1 jobs at their reference station Queue1.

SOLVER: CTMC
N/A: getTranAvg has no entry point in example_util.h; the library carries ctmc::
      solver_ctmc_transient_analyzer but the example facade exposes only the fluid transient

SOLVER: FLD
QLen[Queue1,Class1]: 100 time points
QLen[Queue1,Class1] at t=0: 5
QLen[Queue1,Class1] at t=end: 4.3
SteadyStateQLen[FLD/Prior1]: 4.300000

--- Prior 2: Prior on state with 3 jobs in station 2 ---

SOLVER: FLD
QLen[Queue1,Class1]: 100 time points
QLen[Queue1,Class1] at t=0: 3
QLen[Queue1,Class1] at t=end: 4.3
SteadyStateQLen[FLD/Prior2]: 4.300000

Note: Different initial state priors lead to different transient behaviors.
      Default initialization starts with all jobs at their reference station.
```

The rows repeat for each of the 2 solvers the script runs (CTMC, FLD), which is the point of the example: the same model read by methods of different cost and different exactness.

3.4.2 init_state_fcfs_nonexp

The same with non-exponential service, where the phase of the job in service is part of the state.

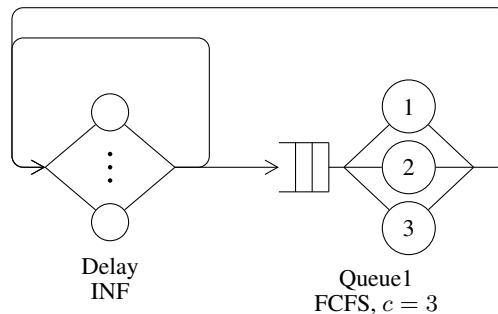


Figure 3.27: Topology of `init_state_fcfs_nonexp`: a closed network over 2 queueing stations.

Nodes	2: Queue, Delay
Classes	2: Class1 (closed, $N = 3$), Class2 (closed, $N = 2$)
Scheduling	Delay: INF; Queue1: FCFS ($c = 3$)
Service	Delay – Class1: Exp(1); Class2: Exp(1) Queue1 – Class1: Exp(1.2); Class2: Erlang(2,2)
Solvers	CTMC, FLD

Table 3.27: Features of `init_state_fcfs_nonexp`.

The example is built and solved as follows.

```
void init_state_fcfs_nonexp() {
    note("=== Initial State - FCFS Queue with Non-Exponential Service ===");

    Net m("model");
    Delay d(m, "Delay");
```

```

Queue q(m, "Queue1", SchedStrategy::FCFS);
m.set_number_of_servers(q, 3.0);
ClosedClass k1(m, "Class1", 3, q);
ClosedClass k2(m, "Class2", 2, q);
m.set_service(d, k1, Exp(1.0));
m.set_service(d, k2, Exp(1.0));
m.set_service(q, k1, Exp(1.2));
m.set_service(q, k2, erlang_fit(1.0, 0.5));
link_classswitch(m, k1, k2, d, q);

SolverOpts o;
o.samples = 10000;
o.stiff = true;
o.timespan_end = 5.0;

note("--- Prior 1: Default initialization ---");
note("Initial state is the default one: every closed class at its reference station
Queue1.");
section("CTMC");
report_curve(ctmc_tran_avg(m, o), 1, 0, 2, "QLen[Queue1,Class1]");
section("FLD");
report_curve(fluid_tran_avg(m, o), 1, 0, 2, "QLen[Queue1,Class1]", "Prior1");

... (11 source lines omitted; full implementation: cpp/examples/advanced/)

report_curve(fluid_tran_avg(m, o3), 1, 0, 2, "QLen[Queue1,Class1]", "Prior3");

note("\nNote: This example shows three types of initial state specification:");
note(" 1. Default: All jobs at reference stations");
note(" 2. Marginal: First state matching the marginal distribution");
note(" 3. Uniform: Uniform distribution over all states with given marginal");

```

Running this edition prints

```

=== Initial State - FCFS Queue with Non-Exponential Service ===
--- Prior 1: Default initialization ---
Initial state is the default one: every closed class at its reference station Queue1.

SOLVER: CTMC
N/A: getTranAvg has no entry point in example_util.h; the library carries ctmc::
      solver_ctmc_transient_analyzer but the example facade exposes only the fluid transient

SOLVER: FLD
QLen[Queue1,Class1]: 100 time points
QLen[Queue1,Class1] at t=0: 3
QLen[Queue1,Class1] at t=end: 0.175046
SteadyStateQLen[FLD/Prior1]: 0.175046

--- Prior 2: First state with marginal [0,0; 4,1] ---

SOLVER: FLD

... (9 captured-output lines omitted; rerun the source example for the full output)

QLen[Queue1,Class1] at t=0: 4
QLen[Queue1,Class1] at t=end: 0.175767
SteadyStateQLen[FLD/Prior3]: 0.175767

Note: This example shows three types of initial state specification:
 1. Default: All jobs at reference stations
 2. Marginal: First state matching the marginal distribution
 3. Uniform: Uniform distribution over all states with given marginal

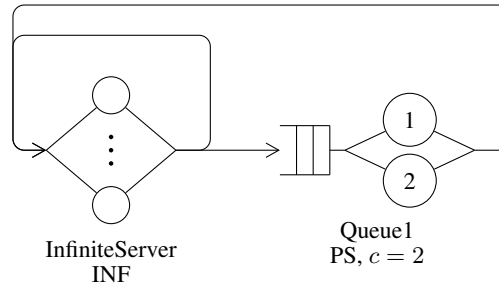
```

The rows repeat for each of the 2 solvers the script runs (CTMC, FLD), which is the point of the example: the same model read by methods of different cost and different exactness.

3.4.3 init_state_ps

The processor sharing version.

The example is built and solved as follows.

Figure 3.28: Topology of `init_state_ps`: a closed network over 2 queueing stations.

Nodes	2: Queue, Delay
Classes	2: Class1 (closed, $N = 3$), Class2 (closed, $N = 1$)
Scheduling	InfiniteServer: INF; Queue1: PS ($c = 2$)
Service	InfiniteServer – Class1: Exp(3); Class2: Exp(0.5) Queue1 – Class1: Exp(0.1); Class2: Exp(1)
Solvers	CTMC, JMT, SSA, FLD, MVA, NC

Table 3.28: Features of `init_state_ps`.

```
void init_state_ps() {
    note("=== Initial State - PS Queue with Class Switching ===");
    note("This example shows solver execution on a 2-class 2-node class-switching model");
    note("with specified initial state.");

    Net m("model");
    Delay d(m, "InfiniteServer");
    Queue q(m, "Queue1", SchedStrategy::PS);
    m.set_number_of_servers(q, 2.0);
    ClosedClass k1(m, "Class1", 3, d);
    ClosedClass k2(m, "Class2", 1, d);
    m.set_service(d, k1, Exp(3.0));
    m.set_service(d, k2, Exp(0.5));
    m.set_service(q, k1, Exp(0.1));
    m.set_service(q, k2, Exp(1.0));
    link_classswitch(m, k1, k2, d, q);

    note("\nInitial state specification the reference sets:");
    note("  Marginal: [[2,1], [1,0]] - Job distribution across stations and classes");
    note("  Started:  [[0,0], [1,0]] - Number of jobs in service at each station");
    init_from_marginal_and_started(m, {{2, 1}, {1, 0}}, {{0, 0}, {1, 0}});

    SolverOpts o;
    o.seed = 23000;
    o.samples = 100000;

    ... (14 source lines omitted; full implementation: cpp/examples/advanced/)

    section("NC");
    print_avg(solve_avg("NC", m, o));

    note("\nNote: Initial state affects transient behavior but not steady-state metrics.");
    note("      MVA and NC compute steady-state only, so initial state has no effect.");
}
```

Running this edition prints

```
=== Initial State - PS Queue with Class Switching ===
This example shows solver execution on a 2-class 2-node class-switching model
with specified initial state.

Initial state specification the reference sets:
Marginal: [[2,1], [1,0]] - Job distribution across stations and classes
Started:  [[0,0], [1,0]] - Number of jobs in service at each station
```

```

SOLVER: Network.initFromMarginalAndStarted
N/A: this port's NetworkStruct carries no 'state' field, so initFromMarginalAndStarted([[2,1], [1,0]],
[[0,0], [1,0]]) has no counterpart; every solve here starts from the default initial state (each
closed class at its reference station, in phase one)
The blocks below therefore run from the default initial state. That leaves the CTMC,
MVA and NC answers unchanged (they are steady-state on an irreducible chain) and makes
the SSA and FLD blocks start from a different point of the same model.

SOLVER: CTMC
SolverCTMC method=default

... (39 captured-output lines omitted; rerun the source example for the full output)

Station      JobClass      QLen      Util      RespT      ResidT      ArvR
  Tput
InfiniteServer Class1      0.33653    0.33653    0.333333    0.198413    1.00959
1.00959
InfiniteServer Class2      1.37304    1.37304      2      0.809524    0.686521
0.686521
Queue1         Class1      1.36335    0.504795    13.504      0.80381    0.100959
0.100959
Queue1         Class2      0.927078    0.343261    1.3504      0.546591    0.686521
0.686521

Note: Initial state affects transient behavior but not steady-state metrics.
MVA and NC compute steady-state only, so initial state has no effect.

```

The rows repeat for each of the 6 solvers the script runs (CTMC, JMT, SSA, FLD, MVA, NC), which is the point of the example: the same model read by methods of different cost and different exactness.

3.4.4 ldes_warmstart

Seeding a simulation from the steady state of another solver, so the warm-up phase can be skipped.

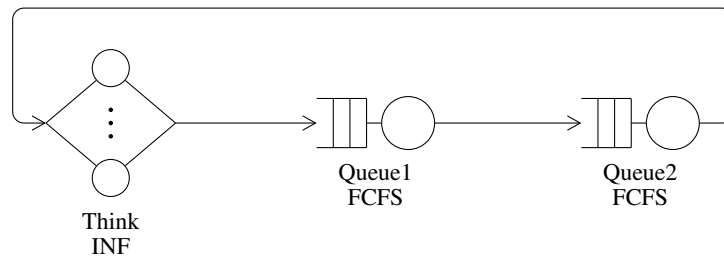


Figure 3.29: Topology of ldes_warmstart: a closed network over 3 queueing stations.

Nodes	3: Queue $\times$ 2, Delay
Classes	1: Jobs (closed, $N = 100$)
Scheduling	Think: INF; Queue1: FCFS; Queue2: FCFS
Service	Think – Jobs: Exp(1) Queue1 – Jobs: Exp(1) Queue2 – Jobs: Exp(0.98)
Solvers	MVA, CTMC, LDES

Table 3.29: Features of ldes_warmstart.

The example is built and solved as follows.

```

void ldes_warmstart() {
    const double N = 100;
    note("=== LDES warm start from an auxiliary solver ===");

    Net exact_model = warmstart_tandem(N);
    SolverOpts o;
    o.method = "exact";
    const AvgTable exact = solve_avg("MVA", exact_model, o);
    print_vector("Exact mean queue lengths", exact.column("QLen"));
}

```

```

// SolverLDES(m, SolverMVA(m, method='exact')): the warm placement is computed
// ONCE from the auxiliary solve, and the derived init_sol reused across runs.
const auto t0 = std::chrono::steady_clock::now();
Net m = warmstart_tandem(N);
mva::MvaOptions mo;
mo.method = "exact";
Matrix<double> none;
const mva::AvgResult<double> aux = mva::solver_mva_run_analyzer(m.get_struct(), mo, none)
;
ldes::LdesOptions warm;
ldes::init_from_placement(warm, ldes::warm_start_placement_from_qlen(m.get_struct(), aux.
QN));
const double init_time =
    std::chrono::duration<double>(std::chrono::steady_clock::now() - t0).count();
char label[64];
std::snprintf(label, sizeof label, "Warm placement from SolverMVA (%.3fs)", init_time);
print_vector(label, warm.init_sol);

... (42 source lines omitted; full implementation: cpp/examples/advanced/)

        wt);
    if (cold_at != 0 && warm_at != 0) break;
}
auto at = [] (std::size_t b) { return b ? std::to_string(b) : std::string("None"); };
std::printf("\nSamples to reach 10%% error: COLD=%s, WARM=%s\n", at(cold_at).c_str(),
            at(warm_at).c_str());

```

Running this edition prints

```

=== LDES warm start from an auxiliary solver ===
Exact mean queue lengths = 0.976943 33.7139 65.3091

SOLVER: LDES
N/A: the warm placement is SolverLDES(model, SolverMVA) writing options.init_sol, and this example facade
exposes no init_sol argument; the engine itself runs (see the other LDES blocks) and `line-cli -s
ldes --ldes-init_sol` takes the placement

```

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by LDES.

3.5 State-dependent routing

Under RAND or probabilistic routing the destination is drawn from a fixed distribution. State-dependent policies read the network state instead: JSQ sends the job to the shortest queue, k -choices samples k queues and picks the shortest of those, and round-robin and its weighted form cycle deterministically. Because the routing depends on the state, the network is no longer product-form, and JSQ in particular is only available to the simulators.

3.5.1 sdroute_closed

The closed counterpart.

Nodes	3: Queue $\times$ 2, Delay
Classes	1: Class1 (closed, $N = 1$)
Scheduling	Delay: INF; Queue1: PS; Queue2: PS
Service	Delay – Class1: HyperExp(0.99,0.01;1.534,0.02819) Queue1 – Class1: Exp(1) Queue2 – Class1: Exp(2)
Features	class switching on 1 link
Solvers	CTMC, JMT, SSA

Table 3.30: Features of sdroute_closed.

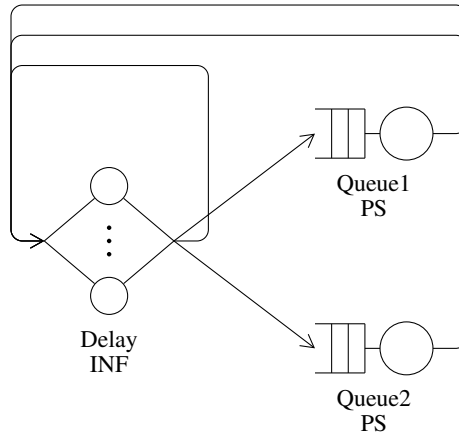


Figure 3.30: Topology of sdroute_closed: a closed network over 3 queueing stations.

The example is built and solved as follows.

```
Net m("model");
Delay delay(m, "Delay");
Queue q1(m, "Queue1", SchedStrategy::PS);
Queue q2(m, "Queue2", SchedStrategy::PS);
ClosedClass c1(m, "Class1", 1, delay, 0);
delay.set_service(c1, lang::hyperexp_fit_mean_scv<double>(1.0, 25.0));
q1.set_service(c1, Exp(1.0));
q2.set_service(c1, Exp(2.0));

// The links of `addLink`: the Delay reaches itself and both queues, and each
// queue returns to it with probability one.
Routing P;
P.set(c1, c1, delay, delay, 1.0);
P.set(c1, c1, delay, q1, 1.0);
P.set(c1, c1, delay, q2, 1.0);
P.set(c1, c1, q1, delay, 1.0);
P.set(c1, c1, q2, delay, 1.0);
m.set_routing(delay, c1, RoutingStrategy::RROBIN);
m.link(P);

const Sn& sn = m.get_struct();

section("CTMC");
ctmc_table(sn, ctmc::CtmcOptions());

section("JMT");
print_avg(m.get_struct(), jmt_avg(m, sim_opts(23000, 100000)));

section("SSA");
ssa_table(sn, 10000);
```

Running this edition prints

SOLVER: CTMC						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Delay	Class1	0.66667	0.66667	1	1	0.66667
0.66667						
Queue1	Class1	0.22222	0.22222	1	0.33333	0.22222
0.22222						
Queue2	Class1	0.11111	0.11111	0.5	0.16667	0.22222
0.22222						
SOLVER: JMT						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Delay	Class1	0.6634	0.6634	0.99642	0.99642	0.67275
0.66595						

Queue1	Class1	0.2227	0.2227	0.99408	0.33136	0.22136
0.22132						
Queue2	Class1	0.11295	0.11295	0.50054	0.16685	0.22136
0.22136						
SOLVER: SSA						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Delay	Class1	0.63744	0.63744	0.84254	0.84254	0.73498
0.75657						
Queue1	Class1	0.24233	0.24233	1	0.33333	0.25219
0.24233						
Queue2	Class1	0.12023	0.12023	0.5	0.16667	0.25219
0.24046						
Note: the serial SSA engine simulates the routing table it is given, so an RROBIN dispatcher reaches it as its uniform expansion and the split is random rather than cyclic.						

The rows repeat for each of the 3 solvers the script runs (CTMC, JMT, SSA), which is the point of the example: the same model read by methods of different cost and different exactness.

3.5.2 sdroute_jsq

Join-the-shortest-queue dispatching over a pool of FCFS queues.

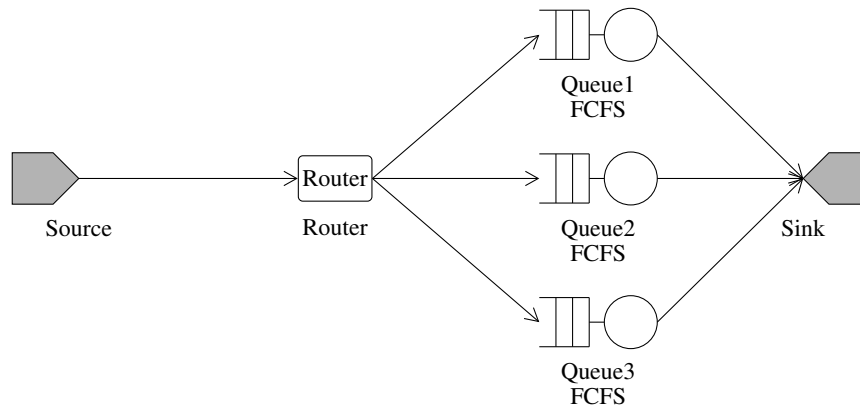


Figure 3.31: Topology of sdroute_jsq: an open network over 3 queueing stations with a router.

Nodes	6: Source, Queue × 3, Router, Sink
Classes	1: Class1 (open)
Scheduling	Queue1: FCFS; Queue2: FCFS; Queue3: FCFS
Arrivals	Source – Class1: Exp(2.1)
Service	Queue1 – Class1: Exp(1) Queue2 – Class1: Exp(1) Queue3 – Class1: Exp(1)
Features	class switching on 6 links
Solvers	JMT, LDES

Table 3.31: Features of sdroute_jsq.

The example is built and solved as follows.

```

const std::size_t N = 3;
const double rho = 0.7, mu = 1.0;
const double lam = static_cast<double>(N) * rho * mu;

Net m("myModel");
Source source(m, "Source");
Router router(m, "Router");
std::vector<std::size_t> queue;
for (std::size_t i = 0; i < N; ++i)
    queue.push_back(m.add_queue("Queue" + std::to_string(i + 1), SchedStrategy::FCFS));
Sink sink(m, "Sink");

```



```

OpenClass c1(m, "Class1");
source.set_arrival(c1, Exp(lam));
for (std::size_t i = 0; i < N; ++i) m.set_service(queue[i], c1, Exp(mu));

Routing P;
P.set(c1, c1, source, router, 1.0);
for (std::size_t i = 0; i < N; ++i) {
    P.set(c1, c1, router, queue[i], 1.0);
    P.set(c1, c1, queue[i], sink, 1.0);
}
// `addLink` LEAVES A NODE ON RAND, and this is not cosmetic: JMT draws from
// that node's stream for a random split even when it has ONE destination,
// so a model built with an explicit routing matrix (PROB) consumes the
// stream differently and its sample path parts company with the
// reference's at the same seed. The reference links every node this way.

... (10 source lines omitted; full implementation: cpp/examples/advanced/)

// NODE tables, which is what the reference prints: the Router is the
// dispatcher under test and is not a station.
section("JMT");
print_avg_node(m.get_struct(), jmt_avg(m, sim_opts(23000)));
section("LDES");
print_avg_node(m.get_struct(), ldes_avg(m, sim_opts(23000)));

```

Running this edition prints

SOLVER: JMT						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Source	Class1	0	0	0	0	0
2.14241						
Router	Class1	0	0	0	0	2.14241
2.14241						
Queue1	Class1	1.1546	0.699035	1.67728	0.559094	0.70971
0.703301						
Queue2	Class1	1.19603	0.706251	1.66381	0.554602	0.703128
0.697342						
Queue3	Class1	1.14024	0.706213	1.67411	0.558038	0.700538
0.702699						
Sink	Class1	0	0	0	0	2.14241
0						
SOLVER: LDES						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Source	Class1	0	0	0	0	0
2.1						
Router	Class1	0	0	0	0	2.1
2.1						
Queue1	Class1	1.20338	0.700436	1.72414	0.574712	0.698574
0.697848						
Queue2	Class1	1.20348	0.700877	1.71324	0.571079	0.703064
0.702401						
Queue3	Class1	1.20648	0.702813	1.72187	0.573955	0.701371
0.700614						
Sink	Class1	0	0	0	0	2.1
0						

The rows repeat for each of the 2 solvers the script runs (JMT, LDES), which is the point of the example: the same model read by methods of different cost and different exactness.

3.5.3 sdroute_krzesinski

Adaptive routing with balanced loading (Krzesinski 1987): a central server and four single-center branches whose routing probabilities depend on the branch populations, and which keep a product form. The example is built and solved as follows.

```

const double mu[3] = {1.0, 0.8, 0.5};
Net m("sdrCentralServer");
Queue cpu(m, "CPU", SchedStrategy::FCFS);

```

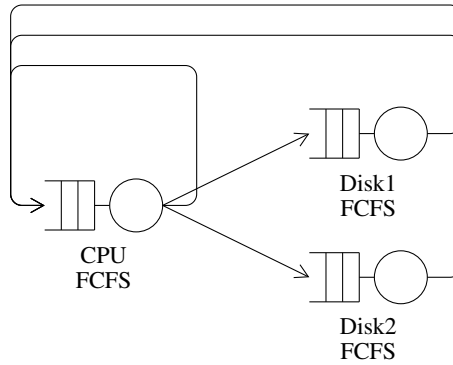


Figure 3.32: Topology of sdroute_krzesinski: a closed network over 3 queueing stations.

Nodes	3: Queue $\times$ 3
Classes	1: Class1 (closed, $N = 3$)
Scheduling	CPU: FCFS; Disk1: FCFS; Disk2: FCFS
Service	CPU – Class1: Exp(1) Disk1 – Class1: Exp(0.8) Disk2 – Class1: Exp(0.5)
Solvers	NC, CTMC

Table 3.32: Features of sdroute_krzesinski.

```

Queue d1(m, "Disk1", SchedStrategy::FCFS);
Queue d2(m, "Disk2", SchedStrategy::FCFS);
ClosedClass c1(m, "Class1", 3, cpu, 0);
cpu.set_service(c1, Exp(mu[0]));
d1.set_service(c1, Exp(mu[1]));
d2.set_service(c1, Exp(mu[2]));

Routing P;
P.set(c1, c1, cpu, cpu, 1.0 / 3); // denied entry: the busy form of waiting
P.set(c1, c1, cpu, d1, 1.0 / 3);
P.set(c1, c1, cpu, d2, 1.0 / 3);
P.set(c1, c1, d1, cpu, 1.0);
P.set(c1, c1, d2, cpu, 1.0);
m.link(P);

Matrix<double> dcoeff(2, 3, 0.0);
dcoeff(0, 1) = 1.0;
dcoeff(0, 2) = 2.0;
dcoeff(1, 2) = 3.0;
m.set_state_dep_routing(cpu, cpu, {{}, {d1}, {d2}}, {0, 1, 2}, {-1, -1}, dcoeff);

const Sn& sn = m.get_struct();

... (16 source lines omitted; full implementation: cpp/examples/advanced/)

// The generator carries eq. (10) per state through `rt_state`, so this is
// the same model solved twice by unrelated routes: an exact chain and a
// product form.
section("CTMC");
ctmc_table(sn, ctmc::CtmcOptions());

```

Running this edition prints

```

SDR routing probabilities (Table 1, eq. 10)
(m2,m3)=(0,0)  P12=0.3333  P13=0.6667  P11=0.0000
(m2,m3)=(0,1)  P12=0.5000  P13=0.5000  P11=0.0000
(m2,m3)=(1,0)  P12=0.0000  P13=1.0000  P11=0.0000
(m2,m3)=(0,2)  P12=1.0000  P13=0.0000  P11=0.0000
(m2,m3)=(1,1)  P12=0.0000  P13=1.0000  P11=0.0000
(m2,m3)=(1,2)  P12=0.0000  P13=0.0000  P11=1.0000

```

SOLVER: NC						
SolverNC method=default/sdr						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
CPU	Class1	1.31646	0.746835	1.76271	1.76271	0.995781
0.746835						
Disk1	Class1	0.443038	0.443038	1.25	0.416667	0.248945
0.35443						
Disk2	Class1	1.24051	0.78481	3.16129	1.05376	0.248945
0.392405						
SOLVER: CTMC						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
CPU	Class1	1.3165	0.74684	1.7627	1.7627	0.99578
0.74684						
Disk1	Class1	0.44304	0.44304	1.25	0.41667	0.24895
0.35443						
Disk2	Class1	1.2405	0.78481	3.1613	1.0538	0.24895
0.39241						

The rows repeat for each of the 2 solvers the script runs (NC, CTMC), which is the point of the example: the same model read by methods of different cost and different exactness.

3.5.4 sdroute_multibranch

The same product form on branches holding several centers, where the coefficients are the branch traffic equations.

No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

Solvers	CTMC
---------	------

Table 3.33: Features of `sdroute_multibranch`.

The example is built and solved as follows.

```
const double mu[4] = {1.0, 0.9, 0.7, 0.5}; // centres 1, 2a, 2b, 3
const std::size_t N = 3;
const double pbacks[2] = {0.0, 0.5};
const char* labels[2] = {"A: branch 2 = 2a -> 2b (series)",
                          "B: branch 2 = 2a -> 2b, 2b -> 2a w.p. 0.5 (feedback onto the "
                          "branch departure)"};

std::printf("\n==== SDR with multi-centre branches ====\n");
for (int cs = 0; cs < 2; ++cs) {
    const double pback = pbacks[cs];
    Net m("sdr_multi");
    Queue cpu(m, "CPU", SchedStrategy::FCFS);
    Queue b2a(m, "B2a", SchedStrategy::FCFS);
    Queue b2b(m, "B2b", SchedStrategy::FCFS);
    Queue b3(m, "B3", SchedStrategy::FCFS);
    ClosedClass c1(m, "Class1", N, cpu, 0);
    const std::size_t centre[4] = {cpu, b2a, b2b, b3};
    for (int i = 0; i < 4; ++i) m.set_service(centre[i], c1, Exp(mu[i]));

    Routing P;
    P.set(c1, c1, cpu, cpu, 1.0); // denied entry: the busy form of waiting
    P.set(c1, c1, cpu, b2a, 1.0);
    P.set(c1, c1, cpu, b3, 1.0);
    P.set(c1, c1, b2a, b2b, 1.0);
    if (pback > 0) {
        P.set(c1, c1, b2b, b2a, pback);
    }

    ... (42 source lines omitted; full implementation: cpp/examples/advanced/)
```

```

Matrix<double> xiflat(4, 1, 1.0);
const pfqn::SdrResult<double> flat =
    pfqn::pfqn_sdr<double>(S, xiflat, std::vector<std::size_t>(1, N), sn.sdr);
std::printf("    xi==1 everywhere: max|dQ| = %.3e    max|dX| = %.3e\n",
            max_abs_diff(ct.QN, flat.QN), max_abs_diff(ct.TN, flat.XN));
}

```

Running this edition prints

```

==== SDR with multi-centre branches ====

A: branch 2 = 2a -> 2b (series)
xi          = [1 1 1 1]
pfqn_sdr Q  = [1.09799 0.416774 0.5591 0.926134]
CTMC       Q  = [1.09799 0.416774 0.5591 0.926134]
pfqn_sdr X  = [0.660243 0.318141 0.318141 0.342103]
CTMC       X  = [0.660243 0.318141 0.318141 0.342103]
max|dQ| = 2.220e-16    max|dX| = 1.110e-16
xi==1 everywhere: max|dQ| = 2.220e-16    max|dX| = 1.110e-16

B: branch 2 = 2a -> 2b, 2b -> 2a w.p. 0.5 (feedback onto the branch departure)
xi          = [1 2 2 1]
pfqn_sdr Q  = [0.800493 0.561502 0.763721 0.874284]
CTMC       Q  = [0.800493 0.561502 0.763721 0.874284]
pfqn_sdr X  = [0.545258 0.402965 0.402965 0.343775]
CTMC       X  = [0.545258 0.402965 0.402965 0.343775]
max|dQ| = 5.551e-16    max|dX| = 3.331e-16
xi==1 everywhere: max|dQ| = 2.975e-01    max|dX| = 1.150e-01

```

3.5.5 sdroute_open

Round-robin dispatching from a Router node in an open network.

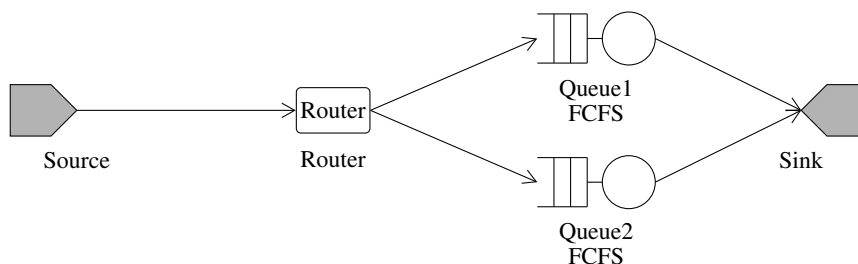


Figure 3.33: Topology of sdroute_open: an open network over 2 queueing stations with a router.

Nodes	5: Source, Queue × 2, Router, Sink
Classes	1: Class1 (open)
Scheduling	Queue1: FCFS; Queue2: FCFS
Arrivals	Source – Class1: Exp(1)
Service	Queue1 – Class1: Exp(2) Queue2 – Class1: Exp(2)
Features	class switching on 5 links
Solvers	JMT, CTMC

Table 3.34: Features of sdroute_open.

The example is built and solved as follows.

```

Net m("myModel");
Source source(m, "Source");
Router router(m, "Router");
Queue q1(m, "Queue1", SchedStrategy::FCFS);
Queue q2(m, "Queue2", SchedStrategy::FCFS);
Sink sink(m, "Sink");
OpenClass cl(m, "Class1");

```

```

source.set_arrival(c1, Exp(1.0));
q1.set_service(c1, Exp(2.0));
q2.set_service(c1, Exp(2.0));

Routing P;
P.set(c1, c1, source, router, 1.0);
P.set(c1, c1, router, q1, 1.0);
P.set(c1, c1, router, q2, 1.0);
P.set(c1, c1, q1, sink, 1.0);
P.set(c1, c1, q2, sink, 1.0);
// RAND on the linked nodes, as `addLink` leaves them: JMT draws from the
// node's own stream for a random split even at one destination, so PROB
// there would shift the whole sample path at the same seed.
m.set_routing(source, c1, RoutingStrategy::RAND);
m.set_routing(q1, c1, RoutingStrategy::RAND);
m.set_routing(q2, c1, RoutingStrategy::RAND);
m.set_routing(sink, c1, RoutingStrategy::RAND);
m.set_routing(router, c1, RoutingStrategy::RROBIN);
m.link(P);

... (7 source lines omitted; full implementation: cpp/examples/advanced/)

// table `CTMC`, and filing it under the decorated spelling would read as a
// solver that produced nothing. See `section(title, golden_key)`.
section("CTMC (cutoff = 5)", "CTMC");
ctmc::CtmcOptions copt;
copt.cutoff = 5.0;
ctmc_node_table(sn, copt);

```

Running this edition prints

SOLVER: JMT						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Source	Class1	0	0	0	0	0
1.01198						
Router	Class1	0	0	0	0	1.01198
1.01198						
Queue1	Class1	0.302398	0.262182	0.576111	0.288056	0.503514
0.505329						
Queue2	Class1	0.29317	0.253607	0.569511	0.284756	0.505312
0.507191						
Sink	Class1	0	0	0	0	1.01198
0						
SOLVER: CTMC (cutoff = 5)						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Source	Class1	0	0	0	0	0
0.998444						
Router	Class1	0	0	0	0	0.998444
0.998444						
Queue1	Class1	0.287012	0.249611	0.574918	0.287459	0.499222
0.499222						
Queue2	Class1	0.287012	0.249611	0.574918	0.287459	0.499222
0.499222						
Sink	Class1	0	0	0	0	0.998444
0						

The rows repeat for each of the 2 solvers the script runs (JMT, CTMC), which is the point of the example: the same model read by methods of different cost and different exactness.

3.5.6 sdroute_twoclasses_closed

Two classes with MAP, MMPP(2) and phase-type service under round-robin dispatching. The example is built and solved as follows.

```

Net m("model");
Delay delay(m, "Delay");
Queue q1(m, "Queue1", SchedStrategy::PS);
Queue q2(m, "Queue2", SchedStrategy::FCFS);

```

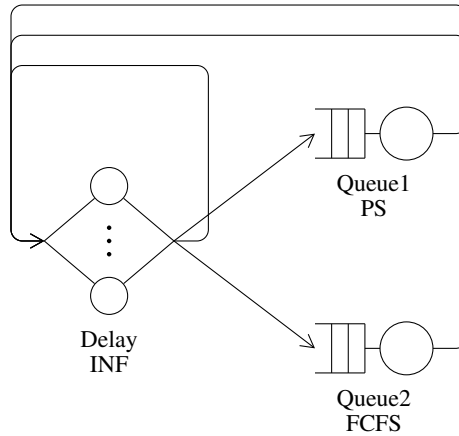


Figure 3.34: Topology of `sdroute_twoclasses_closed`: a closed network over 3 queueing stations.

Nodes	3: Queue $\times$ 2, Delay
Classes	2: Class1 (closed, $N = 1$), Class2 (closed, $N = 2$)
Scheduling	Delay: INF; Queue1: PS; Queue2: FCFS
Service	Delay – Class1: HyperExp(0.99,0.01;1.534,0.02819); Class2: HyperExp(0.99,0.01;1.534,0.02819) Queue1 – Class1: APH; Class2: PH Queue2 – Class1: MAP; Class2: MAP
Features	class switching on 1 link
Solvers	CTMC, JMT, SSA

Table 3.35: Features of `sdroute_twoclasses_closed`.

```

ClosedClass c1(m, "Class1", 1, delay, 0);
ClosedClass c2(m, "Class2", 2, delay, 0);

Matrix<double> A21(2, 2, 0.0);
A21(0, 0) = -2.0;
A21(0, 1) = 2.0;
A21(1, 1) = -0.5;
const std::vector<double> alpha21{1.0, 0.0};
Matrix<double> A22(1, 1, -1.0);
const std::vector<double> alpha22{1.0};

delay.set_service(c1, lang::hyperexp_fit_mean_scv<double>(1.0, 25.0));
q1.set_service(c1, APH(alpha21, A21));
q2.set_service(c1, map2(-20.0, 0.0, 0.0, -1.0, 0.0, 20.0, 0.8, 0.2));
delay.set_service(c2, lang::hyperexp_fit_mean_scv<double>(1.0, 25.0));
q1.set_service(c2, PH(alpha22, A22));
q2.set_service(c2, map2(-4.0, 3.0, 4.0, -6.0, 1.0, 0.0, 0.0, 2.0));

Routing P;
for (std::size_t r : {c1, c2}) {
    P.set(r, r, delay, delay, 1.0);
    P.set(r, r, delay, q1, 1.0);

    ... (11 source lines omitted; full implementation: cpp/examples/advanced/)

section("JMT");
print_avg(m.get_struct(), jmt_avg(m, sim_opts(23000, 100000)));

section("SSA");
ssa_table(sn, 5000);

```

Running this edition prints

SOLVER: CTMC						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Delay	Class1	0.40007	0.40007	1	1	0.40007
0.40007						
Delay	Class2	1.1107	1.1107	1	1	1.1107
1.1107						
Queue1	Class1	0.4964	0.33339	3.7224	1.2408	0.13336
0.13336						
Queue1	Class2	0.59143	0.37022	1.5975	0.5325	0.37022
0.37022						
Queue2	Class1	0.10353	0.07705	0.77636	0.25879	0.13336
0.13336						
Queue2	Class2	0.29791	0.25915	0.80468	0.26823	0.37022
0.37022						
SOLVER: JMT						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Delay	Class1	0.40801	0.40801	1.0367	1.0367	0.39368
0.39368						
Delay	Class2	1.104	1.104	1.021	1.021	1.1026
1.1029						
Queue1	Class1	0.49768	0.33572	3.7386	1.2462	0.13251
0.13251						
Queue1	Class2	0.59452	0.37038	1.6028	0.53427	0.37491
0.3749						
Queue2	Class1	0.10247	0.075542	0.77111	0.25704	0.13251
0.13251						
... (3 captured-output lines omitted; rerun the source example for the full output)						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Delay	Class1	0.40167	0.40167	1.0474	1.0474	0.38198
0.38349						
Delay	Class2	1.0854	1.0854	0.91858	0.91858	1.1505
1.1816						
Queue1	Class1	0.50454	0.32772	3.761	1.2537	0.12783
0.13415						
Queue1	Class2	0.60363	0.37219	1.6218	0.54061	0.39387
0.37219						
Queue2	Class1	0.093791	0.068341	0.78162	0.26054	0.12783
0.12						
Queue2	Class2	0.31096	0.27327	0.80886	0.26962	0.39387
0.38444						
Note: the serial SSA engine simulates the routing table it is given, so an RROBIN dispatcher reaches it as its uniform expansion and the split is random rather than cyclic.						

The rows repeat for each of the 3 solvers the script runs (CTMC, JMT, SSA), which is the point of the example: the same model read by methods of different cost and different exactness.

3.6 Cyclic polling

A polling server visits its queues in a cycle. Under gated service it serves the jobs that were waiting when it arrived, under exhaustive service it stays until the queue empties, and under k -limited service it serves at most k jobs per visit. The discipline is declared as `SchedStrategy.POLLING` and the variant is a parameter of the station.

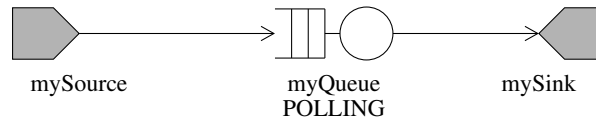
The polling server parks at a queue rather than roving: the visit order and the ratio between the queue lengths are what a polling model must be validated on, not the aggregate throughput, which is insensitive to the discipline.

3.6.1 polling_exhaustive_det

Exhaustive service with deterministic (immediate) switchover.

The example is built and solved as follows.

```
std::size_t queue = 0, c1 = 0, c2 = 0;
Net m = polling_model("M[2]/M[2]/1-Gated", Det(1.0), Det(0.001), Det(1.0),
```

Figure 3.35: Topology of `polling_exhaustive_det`: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	2: myClass1 (open), myClass2 (open)
Scheduling	myQueue: POLLING
Arrivals	mySource – myClass1: Det(1); myClass2: Det(1)
Service	myQueue – myClass1: Det(0.001); myClass2: Det(0.001)
Solvers	JMT

Table 3.36: Features of `polling_exhaustive_det`.

```

        Det(0.001), queue, c1, c2);
m.set_polling_type(queue, lang::PollingType::EXHAUSTIVE);
m.set_switchover(queue, c1, Immediate());
m.set_switchover(queue, c2, Immediate());
polling_mva(m);
polling_jmt(m, 100000);

```

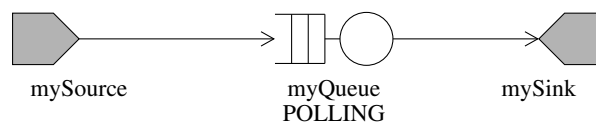
Running this edition prints

SOLVER: MVA						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
mySource	myClass1	0	0	0	0	0
1						
mySource	myClass2	0	0	0	0	0
1						
myQueue	myClass1	0.0010011	0.001	0.0010011	0.0010011	1
1						
myQueue	myClass2	0.0010011	0.001	0.0010011	0.0010011	1
1						
SOLVER: JMT						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
mySource	myClass1	0	0	0	0	0
1						
mySource	myClass2	0	0	0	0	0
1						
myQueue	myClass1	0.00099993	0.00099993	0.001	0.001	1
1						
myQueue	myClass2	0.0019999	0.00099993	0.002	0.002	1
1						

The rows repeat for each of the 2 solvers the script runs (MVA, JMT), which is the point of the example: the same model read by methods of different cost and different exactness.

3.6.2 `polling_exhaustive_exp`

Exhaustive service with exponential switchover times.

Figure 3.36: Topology of `polling_exhaustive_exp`: an open network over 1 queueing station.

The example is built and solved as follows.

Nodes	3: Source, Queue, Sink
Classes	2: myClass1 (open), myClass2 (open)
Scheduling	myQueue: POLLING
Arrivals	mySource – myClass1: Exp(0.1); myClass2: Exp(0.1)
Service	myQueue – myClass1: Exp(1); myClass2: Exp(1.5)
Solvers	JMT

Table 3.37: Features of polling_exhaustive_exp.

```
std::size_t queue = 0, c1 = 0, c2 = 0;
Net m = polling_model("M[2]/M[2]/1-Gated", Exp(0.1), Exp(1.0),
                     Exp(0.1), Exp(1.5), queue, c1, c2);
m.set_polling_type(queue, lang::PollingType::EXHAUSTIVE);
polling_mva(m);
polling_jmt(m, 0);
```

Running this edition prints

SOLVER: MVA						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
mySource	myClass1	0	0	0	0	0
0.1						
mySource	myClass2	0	0	0	0	0
0.1						
myQueue	myClass1	0.11685	0.1	1.1685	1.1685	0.1
0.1						
myQueue	myClass2	0.084724	0.066667	0.84724	0.84724	0.1
0.1						
SOLVER: JMT						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
mySource	myClass1	0	0	0	0	0
0.099696						
mySource	myClass2	0	0	0	0	0
0.10064						
myQueue	myClass1	0.11377	0.099441	1.1502	1.1502	0.099696
0.099232						
myQueue	myClass2	0.082842	0.064638	0.8347	0.8347	0.10064
0.09965						

The rows repeat for each of the 2 solvers the script runs (MVA, JMT), which is the point of the example: the same model read by methods of different cost and different exactness.

3.6.3 polling_gated

A gated polling server: at each visit it serves the jobs that were waiting when it arrived.

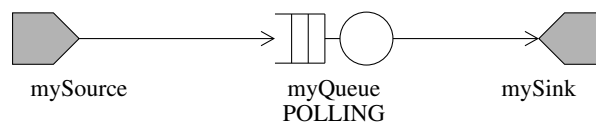


Figure 3.37: Topology of polling_gated: an open network over 1 queueing station.

The example is built and solved as follows.

```
std::size_t queue = 0, c1 = 0, c2 = 0;
Net m = polling_model("M[2]/M[2]/1-Exhaustive", Exp(1.0), Exp(4.0),
                     Exp(0.8), Exp(1.5), queue, c1, c2);
m.set_polling_type(queue, lang::PollingType::GATED);
m.set_switchover(queue, c1, Exp(1.0));
m.set_switchover(queue, c2, Exp(0.5));
polling_mva(m);
```

Nodes	3: Source, Queue, Sink
Classes	2: myClass1 (open), myClass2 (open)
Scheduling	myQueue: POLLING
Arrivals	mySource – myClass1: Exp(1); myClass2: Exp(0.8)
Service	myQueue – myClass1: Exp(4); myClass2: Exp(1.5)
Solvers	JMT

Table 3.38: Features of `polling_gated`.

```
polling_jmt(m, 1000000);
```

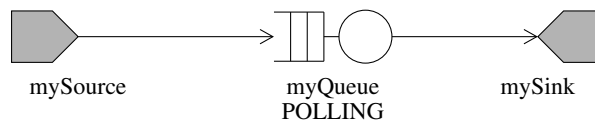
Running this edition prints

SOLVER: MVA						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
mySource	myClass1	0	0	0	0	0
1						
mySource	myClass2	0	0	0	0	0
0.8						
myQueue	myClass1	11.22	0.25	11.22	11.22	1
1						
myQueue	myClass2	11.403	0.53333	14.254	14.254	0.8
0.8						
SOLVER: JMT						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
mySource	myClass1	0	0	0	0	0
1.0012						
mySource	myClass2	0	0	0	0	0
0.80026						
myQueue	myClass1	11.31	0.24989	11.34	11.34	1.0012
1.0008						
myQueue	myClass2	11.312	0.53337	14.149	14.149	0.80026
0.79995						

The rows repeat for each of the 2 solvers the script runs (MVA, JMT), which is the point of the example: the same model read by methods of different cost and different exactness.

3.6.4 `polling_klimited`

k -limited service, at most k jobs per visit.

Figure 3.38: Topology of `polling_klimited`: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	2: myClass1 (open), myClass2 (open)
Scheduling	myQueue: POLLING
Arrivals	mySource – myClass1: Exp(0.2); myClass2: Exp(0.3)
Service	myQueue – myClass1: Exp(1); myClass2: Exp(1.5)
Solvers	JMT

Table 3.39: Features of `polling_klimited`.

The example is built and solved as follows.

```
std::size_t queue = 0, c1 = 0, c2 = 0;
```

```

Net m = polling_model("M[2]/M[2]/1-Gated", Exp(0.2), Exp(1.0),
                     Exp(0.3), Exp(1.5), queue, c1, c2);
m.set_polling_type(queue, lang::PollingType::KLIMITED, 1);
m.set_switchover(queue, c1, Exp(1.0));
m.set_switchover(queue, c2, Immediate());
polling_mva(m);
polling_jmt(m, 100000);

```

Running this edition prints

SOLVER: MVA						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
mySource	myClass1	0	0	0	0	0
0.2						
mySource	myClass2	0	0	0	0	0
0.3						
myQueue	myClass1	0.66667	0.2	3.3333	3.3333	0.2
0.2						
myQueue	myClass2	1.1333	0.2	3.7778	3.7778	0.3
0.3						
SOLVER: JMT						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
mySource	myClass1	0	0	0	0	0
0.19924						
mySource	myClass2	0	0	0	0	0
0.30098						
myQueue	myClass1	0.78887	0.19856	3.9795	3.9795	0.19924
0.20042						
myQueue	myClass2	1.3909	0.2001	4.7392	4.7392	0.30098
0.30096						

The rows repeat for each of the 2 solvers the script runs (MVA, JMT), which is the point of the example: the same model read by methods of different cost and different exactness.

3.7 Switchover times

Moving the server between queues costs time. A switchover distribution is declared per arc of the polling cycle, and its mean adds directly to the cycle time, so the waiting time grows even though no extra work is done.

3.7.1 switchover_basic

Pairwise switchover metadata on an ordinary FCFS queue. The JMT run warns that non-polling queues do not consume these times and solves the underlying saturated two-class queue without them; use the cyclic-polling examples for honoured switchover delays.

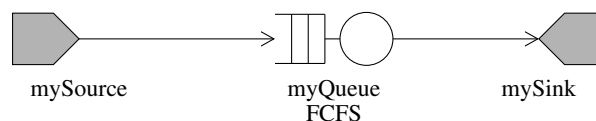


Figure 3.39: Topology of switchover_basic: an open network over 1 queueing station.

The example is built and solved as follows.

```

Net m("M[2]/M[2]/1-Gated");
Source src(m, "mySource");
Queue q(m, "myQueue", SchedStrategy::FCFS);
Sink snk(m, "mySink");

OpenClass c1(m, "myClass1", 0);
src.set_arrival(c1, Exp(0.2));

```

Nodes	3: Source, Queue, Sink
Classes	2: myClass1 (open), myClass2 (open)
Scheduling	myQueue: FCFS
Arrivals	mySource – myClass1: Exp(0.2); myClass2: Exp(0.8)
Service	myQueue – myClass1: Exp(0.1); myClass2: Exp(1.5)
Solvers	JMT

Table 3.40: Features of `switchover_basic`.

```
q.set_service(c1, Exp(0.1));

OpenClass c2(m, "myClass2", 0);
src.set_arrival(c2, Exp(0.8));
q.set_service(c2, Exp(1.5));

// Switching from class 1 to class 2 costs an Exp(1); the other way round an
// Erlang of two phases of rate 1.
q.set_switchover(c1, c2, Exp(1.0));
q.set_switchover(c2, c1, Erlang(1.0, 2));

Routing P;
P.set(c1, c1, src, q, 1.0);
P.set(c1, c1, q, snk, 1.0);
P.set(c2, c2, src, q, 1.0);
P.set(c2, c2, q, snk, 1.0);
m.link(P);

section("JMT");
print_avg(m.get_struct(), jmt_avg(m, sim_opts(23000, 10000)));
```

Running this edition prints

```
SOLVER: JMT
N/A: the MODEL, not the engine: `jmt_avg` drives the same JMT.jar the reference drives, but the model
below cannot be built in this port at all, so there is nothing to hand it
N/A: Queue.setSwitchover(fromClass, toClass, dist) has no counterpart in this port. Network::
set_switchover declares one distribution per class and only at a POLLING station, so the pairwise
switchover of this model cannot be expressed.
```

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by JMT.

3.8 Response time distribution

`getCdfRespT` returns the cumulative distribution of the response time, per station and per class, rather than its mean. The distribution is what a percentile service level objective is written against, and it is far more sensitive to the service distribution than the mean is: two models with the same mean response time can have very different tails.

3.8.1 `cdf_respt_closed`

The response time CDF at a station of a closed PS network.

Nodes	2: Queue, Delay
Classes	1: Class1 (closed, $N = 1$)
Scheduling	Delay: INF; Queue2: PS
Service	Delay – Class1: Exp(10) Queue2 – Class1: Erlang(3,3)
Solvers	JMT, FLD

Table 3.41: Features of `cdf_respt_closed`.

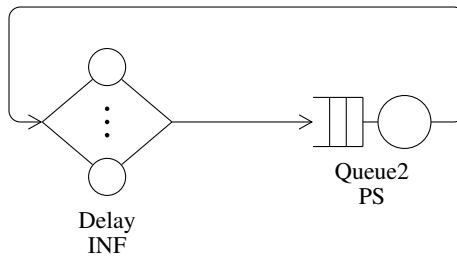


Figure 3.40: Topology of `cdf_respt_closed`: a closed network over 2 queueing stations.

The example is built and solved as follows.

```
void cdf_respt_closed() {
    note("=== CDF Response Time - Closed Network ===");

    const D serv1 = Exp(1.0 / 0.1);
    const D serv2 = erlang_fit(1.0, 1.0 / 3.0);

    Net m("model");
    Delay d(m, "Delay");
    Queue q(m, "Queue2", SchedStrategy::PS);
    ClosedClass k(m, "Class1", 1, d);
    d.set_service(k, serv1);
    q.set_service(k, serv2);
    link_serial(m, {d, q});

    note("Computing CDF with JMT simulation...");
    section("JMT");
    const std::vector<std::vector<CdfCurve> > rd_sim =
        cdf_respt("JMT", m, sim_opts(23000, 10000));
    print_cdf_stats(m.get_struct(), rd_sim, "Response-time CDF (Simulation):");

    note("Computing CDF with Fluid solver...");
    section("FLD");
    const std::vector<std::vector<CdfCurve> > rd = cdf_respt("FLD", m);
    print_cdf_stats(m.get_struct(), rd, "Response-time CDF (Fluid):", CDF_QUAD_MIDPOINT);

    print_cdf_stats_pair(m.get_struct(), rd_sim, " (Simulation)");
    print_cdf_stats_pair(m.get_struct(), rd, " (Fluid)", CDF_QUAD_MIDPOINT);

    note("\nSince there is a single job, mean and squared coefficient of variation");
    note("of response times are close, up to fluid approximation precision, to those");
    note("of the service time distribution.");
    print_vector("Average Response Time from Theory", {serv1.mean, serv2.mean});
    print_vector("Squared Coefficient of Variation from Theory", {serv1.scv, serv2.scv});
}
```

Running this edition prints

```
=== CDF Response Time - Closed Network ===
Computing CDF with JMT simulation...

SOLVER: JMT
Response-time CDF (Simulation):
Station      JobClass      MeanFromCdf      ScvFromCdf      Points
Delay        Class1         0.099998         1.01102         12000
Queue2       Class1         0.997706         0.336454         12000
Computing CDF with Fluid solver...

SOLVER: FLD
Response-time CDF (Fluid):
Station      JobClass      MeanFromCdf      ScvFromCdf      Points
Delay        Class1         0.100089         1.00588         11126
Queue2       Class1         1.00044          0.334119         5882

... (9 captured-output lines omitted; rerun the source example for the full output)

Squared Coefficient of Variation from CDF (Fluid):
```

```
[[1.00587805], [0.3341186668]]
```

Since there is a single job, mean and squared coefficient of variation of response times are close, up to fluid approximation precision, to those of the service time distribution.

Average Response Time from Theory = 0.1 1

Squared Coefficient of Variation from Theory = 1 0.333333

The rows repeat for each of the 2 solvers the script runs (JMT, FLD), which is the point of the example: the same model read by methods of different cost and different exactness.

3.8.2 cdf_respt_closed_threeclasses

The three-class version, with Erlang service.

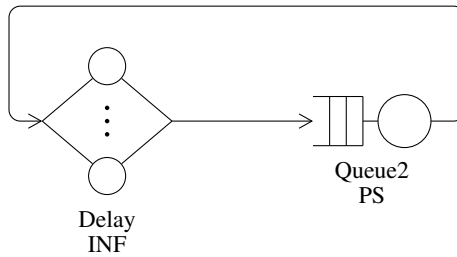


Figure 3.41: Topology of `cdf_respt_closed_threeclasses`: a closed network over 2 queueing stations.

Nodes	2: Queue, Delay
Classes	3: Class1 (closed, $N = 1$), Class2 (closed, $N = 0$), Class3 (closed, $N = 0$)
Scheduling	Delay: INF; Queue2: PS
Service	Delay – Class1: Exp(1); Class2: Exp(1); Class3: Exp(1) Queue2 – Class1: Exp(1); Class2: Erlang(0.5,2); Class3: Exp(100)
Solvers	FLD

Table 3.42: Features of `cdf_respt_closed_threeclasses`.

The example is built and solved as follows.

```
void cdf_respt_closed_threeclasses() {
    note("=== CDF Response Time - Closed Network with Three Classes ===");

    Net m("model");
    Delay d(m, "Delay");
    Queue q(m, "Queue2", SchedStrategy::PS);
    ClosedClass k1(m, "Class1", 1, d);
    ClosedClass k2(m, "Class2", 0, d);
    ClosedClass k3(m, "Class3", 0, d);
    // Class 1 never completes, so the population is carried by the switch cycle.
    m.raw_struct().classes[k1 - 1].completes = false;
    d.set_service(k1, Exp(1.0));
    d.set_service(k2, Exp(1.0));
    d.set_service(k3, Exp(1.0));
    q.set_service(k1, Exp(1.0));
    q.set_service(k2, Erlang(0.5, 2));
    q.set_service(k3, Exp(1.0 / 0.01));

    Routing P;
    P.set(k1, k1, d, q, 1.0);
    P.set(k1, k2, q, d, 1.0);
    P.set(k2, k2, d, q, 1.0);
    P.set(k2, k1, q, d, 1.0);
    P.set(k3, k3, d, q, 1.0);
    P.set(k3, k3, q, d, 1.0);
}
```

```

m.link(P);

... (11 source lines omitted; full implementation: cpp/examples/advanced/)

print_cdf_stats_pair(m.get_struct(), rd, "");

note("\nNote: This example demonstrates class-switching behavior:");
note(" - Class 1 jobs switch to Class 2 at the queue");
note(" - Class 2 jobs switch back to Class 1 at the delay");
note(" - Class 3 jobs follow a simple circular route without switching");

```

Running this edition prints

```

=== CDF Response Time - Closed Network with Three Classes ===
Computing with Fluid solver (state-dependent method)...

SOLVER: FLD
Average Response Time = 1 1 1 4
Response-time CDF (Fluid):
Station      JobClass      MeanFromCdf      ScvFromCdf      Points
Delay        Class1          1.00071          1.00462          14337
Delay        Class2          1.00071          1.00462          14337
Delay        Class3           0              0              2
Queue2       Class1          1.00071          1.00462          14337
Queue2       Class2          4.00159          0.500846         4780
Queue2       Class3           0              0              2

Average Response Time from CDF:
[[1.000706563, 1.000706563, 0], [1.000706563, 4.001591728, 0]]

Squared Coefficient of Variation from CDF:
[[1.004616231, 1.004616231, 0], [1.004616231, 0.500846445, 0]]

Note: This example demonstrates class-switching behavior:
- Class 1 jobs switch to Class 2 at the queue
- Class 2 jobs switch back to Class 1 at the delay
- Class 3 jobs follow a simple circular route without switching

```

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by FLD.

3.8.3 cdf_respt_distrib

The distribution object returned by `getCdfRespt` and the percentiles read from it.

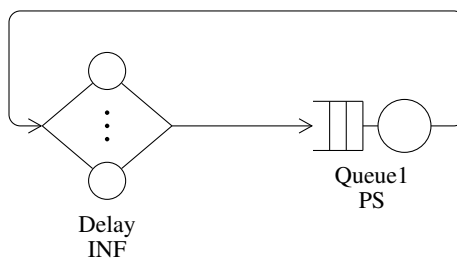


Figure 3.42: Topology of `cdf_respt_distrib`: a closed network over 2 queueing stations.

Nodes	2: Queue, Delay
Classes	2: Class1 (closed, $N = 1$), Class2 (closed, $N = 3$)
Scheduling	Delay: INF; Queue1: PS
Service	Delay – Class1: Exp(1); Class2: Erlang(0.5,2) Queue1 – Class1: Exp(0.5); Class2: Hyper-Exp(0.99,0.01;0.324,0.005143)
Solvers	FLD, JMT

Table 3.43: Features of `cdf_respt_distrib`.

The example is built and solved as follows.

```
void cdf_respt_distrib() {
    note("=== CDF Response Time - Different Service Distributions ===");

    Net m("model");
    Delay d(m, "Delay");
    Queue q(m, "Queue1", SchedStrategy::PS);
    ClosedClass k1(m, "Class1", 1, d);
    d.set_service(k1, D::exp_mean(1.0));
    q.set_service(k1, D::exp_mean(2.0));
    ClosedClass k2(m, "Class2", 3, d);
    d.set_service(k2, erlang_fit_order(4.0, 2));
    // HyperExp.fitMeanAndSCV is map_hyperexp(mean, scv, p=0.99); example_util.h's
    // `hyperexp_fit` is the BALANCED-means fit, which is a different law.
    q.set_service(k2, lang::hyperexp_fit_mean_scv<double>(5.0, 30.0));
    link_ring(m, {d, q});

    note("Computing CDF with Fluid solver (steady-state)...");
    section("FLD");
    const std::vector<std::vector<CdfCurve> > rd = cdf_respt("FLD", m);
    print_cdf_stats(m.get_struct(), rd, "Response-time CDF (Fluid):");

    section("JMT");
    const std::vector<std::vector<CdfCurve> > rd_sim =
        cdf_respt("JMT", m, sim_opts(23000, 100000));
    print_cdf_stats(m.get_struct(), rd_sim, "Response-time CDF (Simulation):");

    ... (12 source lines omitted; full implementation: cpp/examples/advanced/)

    : std::string("N/A"));
}

note("\nNote: Class 1 uses Exponential service distributions.");
note("      Class 2 uses Erlang (delay) and HyperExponential (queue) distributions.");
note("      This shows how different service distributions affect response time CDFs.");
```

Running this edition prints

```
=== CDF Response Time - Different Service Distributions ===
Computing CDF with Fluid solver (steady-state)...

SOLVER: FLD
Response-time CDF (Fluid):
Station      JobClass      MeanFromCdf      ScvFromCdf      Points
Delay        Class1          1.00103          1.00904          11791
Delay        Class2          4.00195          0.501874         13235
Queue1       Class1          6.57588          1.00893          16047
Queue1       Class2          9.35805          0.975141         17700

SOLVER: JMT
Response-time CDF (Simulation):
Station      JobClass      MeanFromCdf      ScvFromCdf      Points
Delay        Class1          0.99895          0.990581         101568
Delay        Class2          4.00026          0.504333         108432

... (16 captured-output lines omitted; rerun the source example for the full output)

proc[Delay,Class1]:      Exp
proc[Delay,Class2]:      Erlang
proc[Queue1,Class1]:     Exp
proc[Queue1,Class2]:     HyperExp

Note: Class 1 uses Exponential service distributions.
      Class 2 uses Erlang (delay) and HyperExponential (queue) distributions.
      This shows how different service distributions affect response time CDFs.
```

The rows repeat for each of the 2 solvers the script runs (FLD, JMT), which is the point of the example: the same model read by methods of different cost and different exactness.

3.8.4 cdf_respt_open_twoclasses

The open two-class version, on FCFS stations.

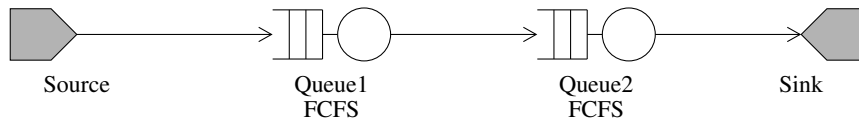


Figure 3.43: Topology of `cdf_respt_open_twoclasses`: an open network over 2 queueing stations.

Nodes	4: Source, Queue $\times$ 2, Sink
Classes	2: Class1 (open), Class2 (open)
Scheduling	Queue1: FCFS; Queue2: FCFS
Arrivals	Source – Class1: Exp(0.25); Class2: Exp(0.25)
Service	Queue1 – Class1: Exp(1); Class2: Exp(1) Queue2 – Class1: Exp(1); Class2: Exp(1)
Solvers	FLD, JMT

Table 3.44: Features of `cdf_respt_open_twoclasses`.

The example is built and solved as follows.

```
void cdf_respt_open_twoclasses() {
    note("=== CDF Response Time - Open Network with Two Classes ===");

    Net m("myModel");
    Source src(m, "Source");
    Queue q1(m, "Queue1", SchedStrategy::FCFS);
    Queue q2(m, "Queue2", SchedStrategy::FCFS);
    Sink snk(m, "Sink");
    OpenClass k1(m, "Class1", 0);
    OpenClass k2(m, "Class2", 0);
    src.set_arrival(k1, D::exp_mean(4.0));
    src.set_arrival(k2, D::exp_mean(4.0));
    q1.set_service(k1, D::exp_mean(1.0));
    q1.set_service(k2, D::exp_mean(1.0));
    q2.set_service(k1, D::exp_mean(1.0));
    q2.set_service(k2, D::exp_mean(1.0));

    // Each class switches into the other on every hop, as P{r,s}(i,j) declares.
    Routing P;
    P.set(k1, k1, src, q1, 1.0);
    P.set(k1, k2, q1, q2, 1.0);
    P.set(k2, k1, q2, snk, 1.0);
    P.set(k2, k2, src, q1, 1.0);
    P.set(k2, k1, q1, q2, 1.0);
    P.set(k1, k2, q2, snk, 1.0);
    m.link(P);

    ... (12 source lines omitted; full implementation: cpp/examples/advanced/)

    print_cdf_stats_pair(m.get_struct(), rd, " (Fluid)");
    print_cdf_stats_pair(m.get_struct(), rd_sim, " (Simulation)");

    note("\nNote: Response time tail distributions are available in the curves above");
    note("      for detailed analysis and plotting.");
}
```

Running this edition prints

```
=== CDF Response Time - Open Network with Two Classes ===
Computing CDF with Fluid solver...

SOLVER: FLD
Response-time CDF (Fluid):
```

Station	JobClass	MeanFromCdf	ScvFromCdf	Points
Queue1	Class1	1.50186	1.00603	12513
Queue1	Class2	1.50186	1.00603	12513
Queue2	Class1	1.50186	1.00603	12513
Queue2	Class2	1.50186	1.00603	12513

SOLVER: JMT

Response-time CDF (Simulation):

Station	JobClass	MeanFromCdf	ScvFromCdf	Points
Queue1	Class1	1.93571	1.01073	11978
Queue1	Class2	1.95017	1.02247	12024

... (9 captured-output lines omitted; rerun the source example for the full output)

Average Response Time from CDF (Simulation):

```
[[0, 0], [1.935708052, 1.950174785], [2.09039405, 2.068766301]]
```

Squared Coefficient of Variation from CDF (Simulation):

```
[[0, 0], [1.010732255, 1.022469029], [1.016930295, 1.003423044]]
```

Note: Response time tail distributions are available in the curves above for detailed analysis and plotting.

The rows repeat for each of the 2 solvers the script runs (FLD, JMT), which is the point of the example: the same model read by methods of different cost and different exactness.

3.8.5 cdf_respt_populations

How the response time distribution moves as the closed population grows.

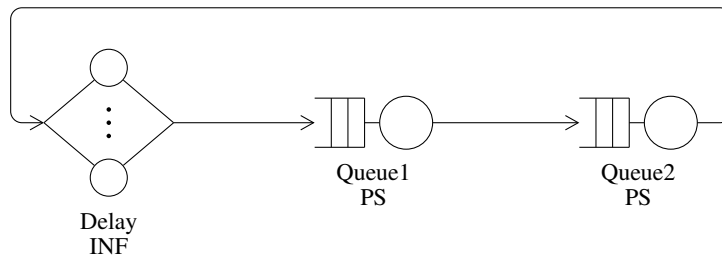


Figure 3.44: Topology of `cdf_respt_populations`: a closed network over 3 queueing stations.

Nodes	3: Queue × 2, Delay
Classes	1: Class1 (closed, $N = 8$)
Scheduling	Delay: INF; Queue1: PS; Queue2: PS
Service	Delay – Class1: Exp(1) Queue1 – Class1: Exp(0.5) Queue2 – Class1: Exp(0.5)
Solvers	FLD

Table 3.45: Features of `cdf_respt_populations`.

The example is built and solved as follows.

```

void cdf_respt_populations() {
    const double n_jobs[3] = {1, 4, 8};
    note("=== CDF Response Time - Varying Population Sizes ===");

    for (int i = 0; i < 3; ++i) {
        std::printf("\n--- Population N = %g jobs ---\n", n_jobs[i]);
        Net m = populations_model(n_jobs[i]);

        SolverOpts o;
        o.iter_max = 100;
        section("FLD");
        const AvgTable t = solve_avg("FLD", m, o);
        // The reference prints the vector twice: once plain, once tagged with N as
        // its codebase-independent parity target.
    }
}

```

```

print_vector("Average Response Time", t.column("RespT"));
// `FLD getAvgRespT N=<k>: v1 v2 ...`, character for character as the
// reference prints it: this is the ONE line the parity parser reads for
// this example (parser.parse_direct_respt), and `print_vector` writes
// `label = ...`, which that pattern does not match.
const std::vector<double> respt = t.column("RespT");
std::printf("FLD getAvgRespT N=%d:", static_cast<int>(n_jobs[i]));
for (std::size_t k = 0; k < respt.size(); ++k) std::printf(" %.6f", respt[k]);
std::printf("\n");
// THE TWO AGGREGATE ROWS ARE WHAT THE GOLDEN ASSERTS AT THE LARGER
// POPULATIONS, and deliberately: once every queue saturates the fluid
// equilibrium is a SET, and which member an engine's ODE trajectory

... (13 source lines omitted; full implementation: cpp/examples/advanced/)

print_cdf_stats(m.get_struct(), cdf_respt("FLD", m, o), "Response-time CDF (Fluid):")
;
}

note("\n=== Summary ===");
note("As population increases, response times increase due to queueing effects.");
note("The CDF shifts to the right, showing higher probability of longer response times.");
;

```

Running this edition prints

```

=== CDF Response Time - Varying Population Sizes ===

--- Population N = 1 jobs ---

SOLVER: FLD
Average Response Time = 1 2 2
FLD getAvgRespT N=1: 1.000000 2.000000 2.000000
Response-time CDF (Fluid):
Station      JobClass      MeanFromCdf  ScvFromCdf  Points
Delay        Class1          1.00072     1.00469     14318
Queue1       Class1          2.00146     1.00468     17377
Queue2       Class1          2.00146     1.00468     17377

--- Population N = 4 jobs ---

SOLVER: FLD

... (14 captured-output lines omitted; rerun the source example for the full output)

Station      JobClass      MeanFromCdf  ScvFromCdf  Points
Delay        Class1          1.00074     1.00499     14337
Queue1       Class1          8.4034     1.00332     8979
Queue2       Class1          8.4034     1.00331     8979

=== Summary ===
As population increases, response times increase due to queueing effects.
The CDF shifts to the right, showing higher probability of longer response times.

```

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by FLD.

3.9 Busy periods

A busy period of order n runs from the instant an arrival raises the number of jobs held by the target to n until it falls back below n . The target may be a station, a station-class pair, or a declared set of stations, and the order n ranges over $1..K$. The analytical reference is the busy period formula of Daduna, exposed by the normalizing constant solver; the simulator measures the same quantity from the sample path.

3.9.1 busyp_subnetwork

Mean busy periods of order n for a station, for a station-class pair and for a declared subnetwork of stations.

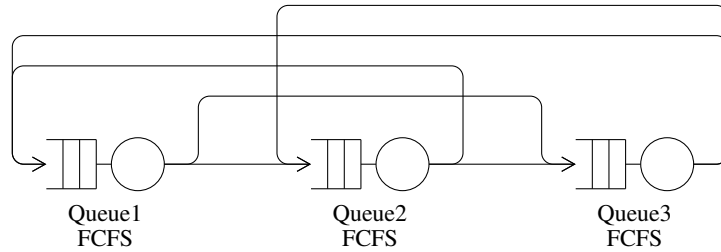


Figure 3.45: Topology of busyp_subnetwork: a closed network over 3 queueing stations.

Nodes	3: Queue $\times$ 3
Classes	1: Class1 (closed, $N = 5$)
Scheduling	Queue1: FCFS; Queue2: FCFS; Queue3: FCFS
Service	Queue1 – Class1: Exp(1.5) Queue2 – Class1: Exp(0.9) Queue3 – Class1: Exp(2)
Solvers	LDES

Table 3.46: Features of busyp_subnetwork.

The example is built and solved as follows.

```
const double N = 5.0;
Net m("busyPeriodModel");
Queue q1(m, "Queue1", SchedStrategy::FCFS);
Queue q2(m, "Queue2", SchedStrategy::FCFS);
Queue q3(m, "Queue3", SchedStrategy::FCFS);
ClosedClass c1(m, "Class1", N, q1, 0);

const double rate[3] = {1.5, 0.9, 2.0};
q1.set_service(c1, Exp(rate[0]));
q2.set_service(c1, Exp(rate[1]));
q3.set_service(c1, Exp(rate[2]));

Routing P;
P.set(c1, c1, q1, q2, 0.6);
P.set(c1, c1, q1, q3, 0.4);
P.set(c1, c1, q2, q1, 0.7);
P.set(c1, c1, q2, q3, 0.3);
P.set(c1, c1, q3, q1, 0.5);
P.set(c1, c1, q3, q2, 0.5);
m.link(P);
const Sn& sn = m.get_struct();

// alpha are the chain visit ratios; the closed formula is invariant to their
// scale, so the reference's unnormalized visits serve as they are
const mva::ChainDemands<double> d = mva::sn_get_demands_chain(sn);
std::vector<double> alpha;

... (40 source lines omitted; full implementation: cpp/examples/advanced/)

for (std::size_t k = 0; k < orders.size(); ++k)
    b[k] = (orders[k] <= tgt.mean.size()) ? tgt.mean[orders[k] - 1] : 0.0;
break;
}
busyp_row(labels[s], b);
}
```

Running this edition prints

SOLVER: Mean busy period of order n (exact, pfqn_busyp)				
subnetwork	n=1	n=3	n=5	
[1]	1.5803	1.2492	0.6667	
[2]	11.9318	4.1381	1.1111	
[3]	0.7101	0.6567	0.5000	
[1 2]	502.3221	25.5284	1.5979	
SOLVER: Mean busy period of order n (measured, LDES)				
subnetwork	n=1	n=3	n=5	
[1]	1.5782	1.2547	0.6628	
[2]	11.9004	4.1368	1.1112	
[3]	0.7107	0.6642	0.5012	
[1 2]	487.3380	25.5168	1.5978	

3.10 Stochastic network calculus

Every other family in these examples reports a mean. The stochastic network calculus reports a *tail*: given a violation probability ε it returns a delay or a queue length that is exceeded with probability at most ε , under any work-conserving scheduling policy at the station. Each flow is carried by a moment-generating-function envelope and each server by the counting process of its service; the min-plus operations compose them across a feed-forward network, at the cost of the burstiness a server adds to the flow it forwards. The bound on the mean that follows by integration is loose, and the decay rate of the tail is asymptotically exact, so the quantile is what these examples read.

3.10.1 snc_delay_quantile

A delay quantile with a certified violation probability, and how it tightens deep in the tail.

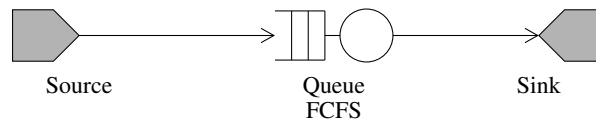


Figure 3.46: Topology of snc_delay_quantile: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: Class1 (open)
Scheduling	Queue: FCFS
Arrivals	Source – Class1: Exp(0.6)
Service	Queue – Class1: Exp(1)

Table 3.47: Features of snc_delay_quantile.

The example is built and solved as follows.

```
void snc_delay_quantile() {
    note("=== Stochastic network calculus: delay quantile ===");

    const double lambda = 0.6, mu = 1.0;
    Net m("SncQuantile");
    Source src(m, "Source");
    Queue q(m, "Queue", SchedStrategy::FCFS);
    Sink snk(m, "Sink");
    OpenClass c(m, "Class1", 0);
    src.set_arrival(c, Exp(lambda));
    q.set_service(c, Exp(mu));
    Routing P;
    P.set(src, q, 1.0);
    P.set(q, snk, 1.0);
}
```

```

m.link(P);
const Sn sn = m.get_struct();

// The quantile is the native output of the family: the smallest d and b for
// which  $P\{D>d\} \leq \text{eps}$  and  $P\{Q>b\} \leq \text{eps}$  are certified.
section("Quantiles at eps = 1e-3 (snc.upper)");
const ba::SncPercentiles p3 = ba::solver_ba_snc_perc(sn, 1e-3);
std::printf("%-10s %-10s %12s %12s\n", "Station", "JobClass", "RespTPerc", "QLenPerc");
std::printf("%-10s %-10s %12.5f %12.5f\n", "Queue", "Class1", p3.D(1, 0), p3.B(1, 0));

// The exact M/M/1 sojourn tail is  $\exp(-(\mu-\lambda)*d)$  and the exact
// queue-length tail is  $\rho^{(n+1)}$ . The bound reproduces both DECAY RATES

... (36 source lines omitted; full implementation: cpp/examples/advanced/)

std::printf("    the optimal theta approaches  $\log(\mu/\lambda) = %.4f$ , which is\n",
            std::log(mu / lambda));
std::printf("    what makes the backlog decay rate exact\n");
const snc::SncResult back = snc::snc_bound_delay(arv, srv, d.value);
std::printf("api:  $P\{D > %.4f\} \leq %.3e$  (theta = %.4f), exact tail  $%.3e$ \n", d.value, back.
            value,
            back.theta, std::exp(-(\mu - lambda) * d.value));

```

Running this edition prints

```

=== Stochastic network calculus: delay quantile ===

SOLVER: Quantiles at eps = 1e-3 (snc.upper)
Station  JobClass  RespTPerc  QLenPerc
Queue    Class1    29.60819   23.64494

SOLVER: How the bound tightens as the guarantee gets stricter
eps      d bound    d exact    ratio      n bound    n exact    ratio
1e-02    23.3454    11.5129    2.028      18.7266    8.0152     2.336
1e-03    29.6082    17.2694    1.714      23.6449    12.5227    1.888
1e-06    47.9527    34.5388    1.388      38.0313    26.0455    1.460
1e-09    65.9597    51.8082    1.273      52.1418    39.5682    1.318
1e-12    83.7930    69.0776    1.213      66.1120    53.0909    1.245

SOLVER: Mean columns (snc.upper), and why they are the loose end
exact M/M/1: R = 2.5000, Q = 1.5000
snc.upper   : R = 13.2051 (5.3x), Q = 7.9231 (5.3x)

SOLVER: The same answer from the api
api: d(1e-3) = 29.6082 at theta = 0.4632
      the optimal theta approaches  $\log(\mu/\lambda) = 0.5108$ , which is
      what makes the backlog decay rate exact
api:  $P\{D > 29.6082\} \leq 1.000e-03$  (theta = 0.4632), exact tail 7.187e-06

```

3.10.2 snc_tandem_multiclass

Envelope propagation across a feed-forward tandem, pay-bursts-only-once, blind multiplexing, and the model classes the family refuses.

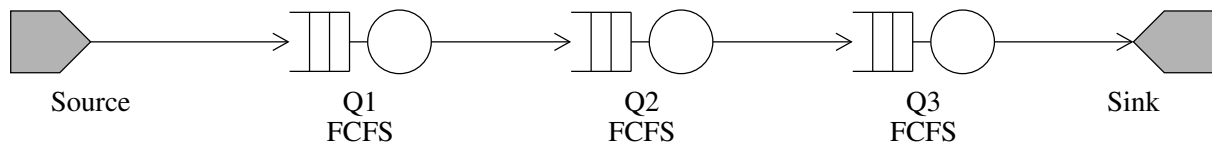


Figure 3.47: Topology of snc_tandem_multiclass: an open network over 3 queueing stations.

The example is built and solved as follows.

```

void snc_tandem_multiclass() {
    note("=== Stochastic network calculus: tandem and shared server ===");

    const double lambda = 0.6;
    const double rates[3] = {1.5, 1.2, 1.0};
}

```

Nodes	5: Source, Queue $\times$ 3, Sink
Classes	1: Class1 (open)
Scheduling	Q1: FCFS; Q2: FCFS; Q3: FCFS
Arrivals	Source – Class1: Exp(0.6)
Service	Q1 – Class1: Exp(1.5) Q2 – Class1: Exp(1.2) Q3 – Class1: Exp(1)

Table 3.48: Features of `snc_tandem_multiclass`.

```

Net m("SncTandem");
Source src(m, "Source");
Queue q1(m, "Q1", SchedStrategy::FCFS);
Queue q2(m, "Q2", SchedStrategy::FCFS);
Queue q3(m, "Q3", SchedStrategy::FCFS);
Sink snk(m, "Sink");
OpenClass c(m, "Class1", 0);
src.set_arrival(c, Exp(lambda));
q1.set_service(c, Exp(rates[0]));
q2.set_service(c, Exp(rates[1]));
q3.set_service(c, Exp(rates[2]));
Routing P;
P.set(src, q1, 1.0);
P.set(q1, q2, 1.0);
P.set(q2, q3, 1.0);
P.set(q3, snk, 1.0);
m.link(P);

// Each hop replaces the arrival envelope by the DEPARTURE envelope of the
// station upstream, which carries the burst the server has added. The exact
// answer does not degrade this way -- by Burke's theorem the departure

... (82 source lines omitted; full implementation: cpp/examples/advanced/)

try {
    run_snc(u.get_struct());
    std::printf("    unequal service rates at a shared station: NO REFUSAL (unexpected)\n");
} catch (const std::exception& e) {
    std::printf("    unequal service rates at a shared station: %s\n", e.what());
}

```

Running this edition prints

```

=== Stochastic network calculus: tandem and shared server ===

SOLVER: Tandem: what envelope propagation costs
station   R bound   R exact   ratio
Q1        3.9606   1.1111    3.56
Q2        10.7164   1.6667    6.43
Q3        24.7385   2.5000    9.90

SOLVER: End-to-end: pay bursts only once
concatenated (snc_conv) : 42.5195 at theta = 0.4793
summed per hop          : 65.5128
pay bursts once saves   : 35.1%

SOLVER: Two classes sharing one server (blind multiplexing)
JobClass   R bound   RespTPerc  QLenPerc
ClassA     24.0302   55.9018   23.6449
ClassB     24.0302   55.9018   23.6449
exact per-class response time (aggregate M/M/1, lambda=0.6, mu=1): 2.5000

SOLVER: What the family refuses, and why
unequal service rates at a shared station: solver_ba_snc: method 'snc.upper' requires the classes
sharing a station to have equal service rates: station 2 carries rates in [1.000000, 2.000000]

```

3.11 Random environments

A random environment is a Markov process over stages, with one model per stage. The environment moves between stages according to a transition process, and the network parameters change with it; the metrics reported are the environment average, weighted by the stage probabilities, but the transient handoff between stages matters when the stage sojourn is comparable with the relaxation time of the network.

Server breakdown and repair is the smallest instance: two stages, one with the server up and one with it down. The case studies at the end of the family are larger, and are the models on which the environment solver is validated.

3.11.1 example_mapqn2renv

Converting a queueing network with MMPP(2) service into an equivalent random environment model.

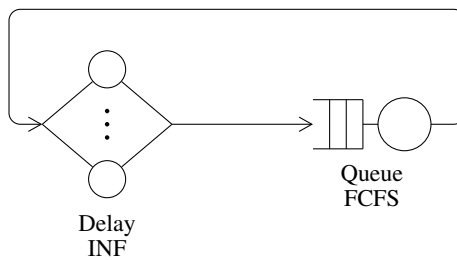


Figure 3.48: Topology of example_mapqn2renv: a closed network over 2 queueing stations.

Nodes	2: Queue, Delay
Classes	1: Class1 (closed, $N = 5$)
Scheduling	Delay: INF; Queue: FCFS
Service	Delay – Class1: Exp(1) Queue – Class1: MMPP2
Solvers	JMT

Table 3.49: Features of example_mapqn2renv.

The example is built and solved as follows.

```
note("=== MMPP2 Closed QN to Random Environment Transformation ===");
Net m = mmpp_closed("MMPP_ClosedQN");
kv("Population", 5.0);
note("Topology: Delay -> MMPP2 Queue -> Delay (cyclic), think time Exp(1.0)");

io::Map2RenvInfo<double> info;
Env e = io::mapqn2renv(m.get_struct(), &info);
stage_table(e);

env::EnvOptions o;
o.iter_max = 100;
o.iter_tol = 0.01;
o.timespan_end = 100.0;

section("ENV (FLD)", "ENV");
const env::EnvAnalyzerSolution<double> rf = env::solver_env(e, o);
env_banner(e, o, rf);
print_env_avg(e.stage(0).model, rf);

section("ENV (CTMC)", "ENV");
env::EnvOptions oc = o;
oc.stage_solver = "ctmc";
oc.stage_cutoff = 100.0;
```



```

const env::EnvAnalyzerSolution<double> rc = env::solver_env(e, oc);
env_banner(e, oc, rc);
print_env_avg(e.stage(0).model, rc);

    ... (10 source lines omitted; full implementation: cpp/examples/advanced/)
}

// The reference point the environment solves are read against: the ORIGINAL
// MMPP2 model, simulated.
section("JMT");
print_avg(m.get_struct(), jmt_avg(m));

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.11.2 renv_basic

The smallest random environment: two stages, one model per stage, and the stage transition process. No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

Solvers	FLD, MVA
---------	----------

Table 3.50: Features of `renv_basic`.

The example is built and solved as follows.

```

Net fast = renv_basic_stage(4.0);
Net slow = renv_basic_stage(1.0);

Env e("ServerModes", 2);
e.set_stage(0, "Fast", "operational", fast.get_struct());
e.set_stage(1, "Slow", "degraded", slow.get_struct());
e.add_transition(0, 1, Exp(0.5));
e.add_transition(1, 0, Exp(1.0));

note("Environment stages:");
stage_table(e);

env::EnvOptions o;
o.method = "meanfield";
o.timespan_end = 100.0;
const env::EnvAnalyzerSolution<double> r = env::solver_env(e, o);

note("\n--- Environment-Averaged Results ---");
env_banner(e, o, r);
// The reference prints these two blocks with no solver banner, so the
// declaration is made to the recorder alone. An ENSEMBLE IS NAMED BY ITS
// MEMBER: `ENV(FLD)` is a mean-field coupling over fluid stages, which is
// what the golden's `FLD` row holds and what the comparator reconciles the
// two spellings of.
attribute("ENV(FLD)");
print_env_avg(e.stage(0).model, r);

    ... (6 source lines omitted; full implementation: cpp/examples/advanced/)

Matrix<double> init;
// One table per stage under one key; the golden holds the FIRST, which
// is why `renv_basic` is listed in `corpus.json`'s multi-model set.
attribute("MVA");
print_avg(sn, mva::solver_mva_run_analyzer(sn, mo, init));
}

```

Running this edition prints

```
Environment stages:
Stage   Name      Type      Model
1       Fast      operational BaseModel
2       Slow      degraded  BaseModel

--- Environment-Averaged Results ---
SolverENV method=meanfield stages=2 horizon=100 points=1001 iters=13
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
Tput
ThinkTime    Jobs          3.0002     3.0002     nan         1         nan
3.0002
Fast/Slow Server Jobs          1.9998     1         nan        0.66659     nan
3

--- Individual Stage Analysis (MVA) ---

Stage 0:
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
Tput
ThinkTime    Jobs          3.2037     3.2037     1          1         3.2037
3.2037
Fast/Slow Server Jobs          1.7963     0.80093    0.56068    0.56068    3.2037
3.2037

Stage 1:
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
Tput
ThinkTime    Jobs          0.99693    0.99693    1          1         0.99693
0.99693
Fast/Slow Server Jobs          4.0031     0.99693    4.0154     4.0154     0.99693
0.99693
```

3.11.3 renv_container_terminal

A container terminal case study: MAP arrivals, multiserver and load-dependent stations, several environment stages.

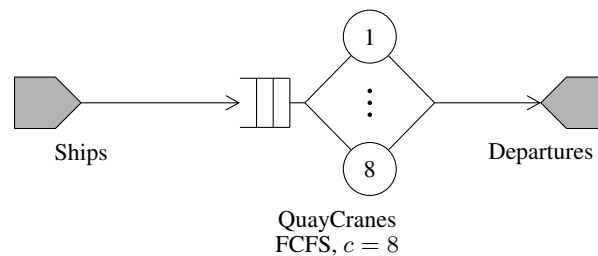


Figure 3.49: Topology of renv_container_terminal: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: Containers (open)
Scheduling	QuayCranes: FCFS ($c = 8$)
Arrivals	Ships – Containers: MAP
Service	QuayCranes – Containers: Exp(16)

Table 3.51: Features of renv_container_terminal.

The example is built and solved as follows.

```
const double daily_rate_hr[24] = {6, 30, 40, 62, 76, 79, 119, 164, 152, 130, 79, 70,
                                   57, 57, 113, 130, 162, 202, 148, 118, 92, 62, 36, 8};
const double daily_duration_hr[24] = {0.51, 0.84, 0.76, 0.98, 0.67, 2.33, 1.80, 0.62,
                                       0.36, 1.30, 1.02, 0.98, 0.86, 1.56, 1.30, 1.57,
                                       0.34, 0.22, 0.47, 1.33, 1.56, 0.93, 0.71, 1.11};
```

```

const std::size_t E = 24;           // hourly environment stages
const double N = 6.0;              // straddle carriers circulating (the closed tokens)
const double n_cranes = 2.0;       // quay cranes, a multi-server FCFS station
const double crane_rate = 16.0;    // moves per hour served by one crane

std::vector<Net> stage;
std::vector<double> env_rate(E);
for (std::size_t h = 0; h < E; ++h) {
    // The per-token yard completion rate of hour h: the closed-network
    // image of that hour's external demand intensity.
    stage.push_back(terminal_model(daily_rate_hr[h] / N, crane_rate, n_cranes, N));
    env_rate[h] = 1.0 / daily_duration_hr[h];
}
Env e = daily_cycle("RotterdamDailyCycle", stage, env_rate);

std::printf("Rotterdam container terminal: %zu hourly stages, %g carriers, %g cranes.\n", E,
            N,
            n_cranes);

// The reference's stage solver is `CTMC(m, 'exact', 'timespan', [0,T])` with
// T = 12 hours, which covers the hour-duration CDF past 99.9% for every
// stage, so the quadrature below is not truncating the blend.

... (82 source lines omitted; full implementation: cpp/examples/advanced/)

const double mean_lambda = num / den;
std::printf("MMPP mean arrival = %.2f/hr, %g cranes @ %.0f/hr, rho = %.3f\n", mean_lambda,
            c_open, crane_rate, mean_lambda / (c_open * crane_rate));
section("MAM");
print_avg(open_model.get_struct(),
            mam::solver_mam_run_analyzer(open_model.get_struct(), mam::MamOptions()));

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.11.4 renv_fourstages_repairmen

The four-stage version.

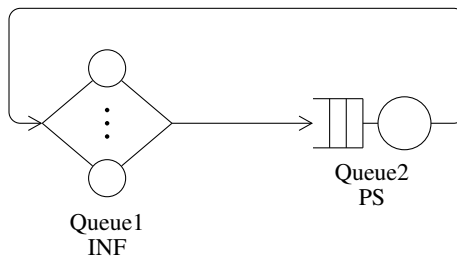


Figure 3.50: Topology of renv_fourstages_repairmen: a closed network over 2 queueing stations.

Nodes	2: Queue, Delay
Classes	1: Class1 (closed, $N = 30$)
Scheduling	Queue1: INF; Queue2: PS
Service	Queue1 – Class1: Exp(4) Queue2 – Class1: Exp(1)
Solvers	ENV

Table 3.52: Features of renv_fourstages_repairmen.

The example is built and solved as follows.

```

const std::size_t E = 4;

```

```

const double N = 30.0;
const char* env_name[4] = {"Stage1", "Stage2", "Stage3", "Stage4"};
const char* env_type[4] = {"UP", "DOWN", "FAST", "SLOW"};

// rate = ones(M,E); rate(M,:) = 1:E; rate(1,:) = E:-1:1, with M = 3: the
// generator reads rows 1 and 2, so the queue runs at 1 in every stage.
const double rate_delay[4] = {4.0, 3.0, 2.0, 1.0};
const double rate_queue[4] = {1.0, 1.0, 1.0, 1.0};

note("Rate matrix:");
for (std::size_t s = 0; s < E; ++s)
    std::printf("  stage %zu: Queue1 %.6g Queue2 %.6g\n", s + 1, rate_delay[s],
                rate_queue[s]);

Env e("MyEnv", E);
std::vector<Net> sub;
for (std::size_t s = 0; s < E; ++s) sub.push_back(renv_genqn_model(rate_delay[s], rate_queue[
s], N));
for (std::size_t s = 0; s < E; ++s)
    e.set_stage(s, env_name[s], env_type[s], sub[s].get_struct());

const double env_rates[4][4] = {{0.0, 0.5, 0.0, 0.0},
                                {0.0, 0.0, 0.5, 0.5},
                                {0.5, 0.0, 0.0, 0.5},
                                {0.5, 0.5, 0.0, 0.0}};
note("Environment transition rates:");

... (22 source lines omitted; full implementation: cpp/examples/advanced/)

const env::EnvAnalyzerSolution<double> r = env::solver_env(e, o);

note("AvgTable =");
env_banner(e, o, r);
attribute("ENV (FLD) ");
print_env_avg(e.stage(0).model, r);

```

Running this edition prints

```

Rate matrix:
  stage 1: Queue1 4 Queue2 1
  stage 2: Queue1 3 Queue2 1
  stage 3: Queue1 2 Queue2 1
  stage 4: Queue1 1 Queue2 1
Environment transition rates:
0 0.5 0 0
0 0 0.5 0.5
0.5 0 0 0.5
0.5 0.5 0 0
The metasolver considers an environment with 4 stages and a queueing network with 3 stations.
Every time the stage changes, the queueing network will modify the service rates of the stations.
Stage   Name      Type      Model
1       Stage1    UP       qn1
2       Stage2    DOWN     qn1
3       Stage3    FAST     qn1
4       Stage4    SLOW     qn1
AvgTable =
SolverENV method=meanfield stages=4 horizon=100 points=1001 iters=4
Station  JobClass      QLen      Util      RespT      ResidT      ArvR
Tput
Queue1   Class1          0.43983    0.43983    nan        0.45267    nan
0.97164
Queue2   Class1          29.56      1          nan        29.56      nan
1

```

3.11.5 renv_genqn

A randomly generated network placed in an environment.

The example is built and solved as follows.

```

Net m = renv_genqn_model(1.0, 0.5, 4.0);
const Sn& sn = m.get_struct();

```

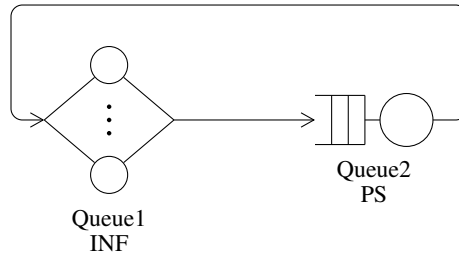


Figure 3.51: Topology of renv_genqn: a closed network over 2 queueing stations.

Nodes	2: Queue, Delay
Classes	1: Class1 (closed, $N = 4$)
Scheduling	Queue1: INF; Queue2: PS
Service	Queue1 – Class1: Exp(1) Queue2 – Class1: Exp(0.5)
Solvers	MVA

Table 3.53: Features of renv_genqn.

```
mva::MvaOptions mo;
Matrix<double> init;
attribute("MVA");
print_avg(sn, mva::solver_mva_run_analyzer(sn, mo, init));
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.11.6 renv_lqn_twostages

A layered model in a two-stage environment, including its transient.

No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

The example is built and solved as follows.

```
const double T = 50.0;

Env e = renv_lqn_env("DBReliability", 0.2, 1.0);
note("Environment stages:");
stage_table(e);

const env::EnvOptions o = renv_lqn_options(T, 20, 0.02);
const env::EnvAnalyzerSolution<double> r = env::solver_env(e, o);

note("\n--- Environment-Averaged Results ---");
env_banner(e, o, r);
std::printf("ENV over LQN ran: aggregate size = %zu x %zu\n", r.QN.rows(), r.QN.cols());
// The labels come from a solver built on the SAME stage model, so they are
// the labels of the very blocks the aggregate was assembled from.
ln::LnOptions lo;
lo.layer_solver = "fluid";
lo.timespan_end = T;
ln::SolverLN<double> labels(renv_lqn_model(0.8), lo);
attribute("ENV(LN(FLD))");
print_env_avg_lqn(labels, r);

// (1) Finite, and the closed population is conserved: the aggregate sums
// each layer's own five client jobs, and the environment moves none of them.
double upQ = 0.0, upT = 0.0, dnQ = 0.0, dnT = 0.0;
renv_lqn_stage_alone(renv_lqn_model(0.8), T, upQ, upT);
renv_lqn_stage_alone(renv_lqn_model(3.0), T, dnQ, dnT);
```

```

... (36 source lines omitted; full implementation: cpp/examples/advanced/)

const bool bracketed3 = x3 >= lo_x - tol && x3 <= hi_x + tol;

note(conerved && bracketed && monotone && bracketed3
    ? "PASS: ENV-over-LQN meanfield ran; throughput bracketed, monotone in P(UP), "
      "3-stage bracketed."
    : "FAIL: ENV-over-LQN violated one of conservation, bracketing or monotonicity.");

```

Running this edition prints

```

SOLVER: SolverENV over LayeredNetwork
N/A: the reference runs SolverENV over LQN stages through SolverLN layer solvers; an Environment stage in
this port holds a Network struct, so a LayeredNetwork stage cannot be declared, and the mean-field
coupling refuses LayeredNetwork stages by name

```

3.11.7 renv_map_fallback

An MMPP(2) stage transition process, and the fallback taken when it cannot be represented exactly.

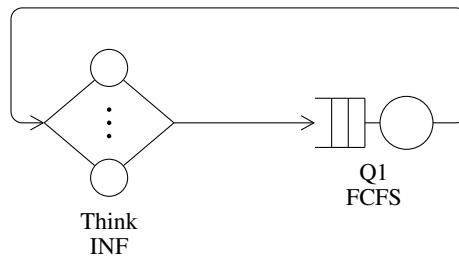


Figure 3.52: Topology of renv_map_fallback: a closed network over 2 queueing stations.

Nodes	2: Queue, Delay
Classes	1: C1 (closed, $N = 5$)
Scheduling	Think: INF; Q1: FCFS
Service	Think – C1: Exp(1) Q1 – C1: MMPP2
Solvers	MVA, ENV, CTMC

Table 3.54: Features of renv_map_fallback.

The example is built and solved as follows.

```

Net m = mmpp_closed("mmppClosed");
io::Map2RenvInfo<double> info;
Env e = io::map2renv(m.get_struct(), &info);

std::printf("Random-environment image: %zu stages, phase orders %s, MMPP image: %d\n",
    info.nstages, info.orders.empty() ? "[ ]" : "[2]",
    info.is_mmpp ? 1 : 0);
stage_table(e);

section("CTMC");
ctmc::CtmcOptions copt;
print_avg(m.get_struct(), ctmc::solver_ctmc_run_analyzer(m.get_struct(), copt));

// The two environment LIMITS the reference reaches through
// `options.config.map_env_method`, here with the port's DEFAULT stage
// backend, which is still the fluid one. For a five-job exponential closed
// stage the fluid steady state is not the exact MVA one, so these two
// tables and the `map_env` run below differ by the stage solver alone.
for (const char* limit : {"dec", "avg"}) {
    env::EnvOptions o;

```

```

o.method = limit;
o.iter_max = 100;
o.iter_tol = 0.01;
section(std::string("ENV(") + limit + ", fluid stages)", "ENV");
const env::EnvAnalyzerSolution<double> r = env::solver_env(e, o);
env_banner(e, o, r);

... (36 source lines omitted; full implementation: cpp/examples/advanced/)

},
solvers::mva_stage_fn(mopt), mopt.method);
note("map_env='off': the model was accepted, which it should not have been");
} catch (const std::exception& ex) {
    std::printf("map_env='off' -> %s\n", ex.what());
}

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.11.8 renv_node_breakdown

Server breakdown and repair as an environment over a queueing model.

No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

Solvers	FLD, ENV
---------	----------

Table 3.55: Features of `renv_node_breakdown`.

The example is built and solved as follows.

```

try {
    renv_node_breakdown_blocks();
} catch (const std::exception& ex) {
    std::printf("Error running examples: %s\n", ex.what());
}

```

Running this edition prints

```

=====
Example 1: Using addNodeFailureRepair with node object
=====

Stage table for env1:
Stage   Name           Type           Model
1       UP              operational    ServerWithFailures
2       DOWN_Server       failed        ServerWithFailures

=====
Example 2: Using separate breakdown and repair calls with node object
=====

Stage table for env2:
Stage   Name           Type           Model
1       UP              operational    ServerWithFailures

... (33 captured-output lines omitted; rerun the source example for the full output)

- DOWN state: Server processes jobs at reduced rate 0.5
- Breakdown occurs at rate 0.1 (mean time to failure = 10 time units)
- Repair occurs at rate 1.0 (mean time to repair = 1 time unit)
- Results show averaged performance across both states

=====
All examples completed successfully!
=====

```

3.11.9 renv_rotterdam_blending

The Rotterdam blending case study.

No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

The example is built and solved as follows.

```
const double rate_hr[24] = {6, 30, 40, 62, 76, 79, 119, 164, 152, 130, 79, 70,
                          57, 57, 113, 130, 162, 202, 148, 118, 92, 62, 36, 8};
const double dur_hr[24] = {0.51, 0.84, 0.76, 0.98, 0.67, 2.33, 1.80, 0.62,
                          0.36, 1.30, 1.02, 0.98, 0.86, 1.56, 1.30, 1.57,
                          0.34, 0.22, 0.47, 1.33, 1.56, 0.93, 0.71, 1.11};

const std::size_t K = 24;
std::vector<double> lam(K), dur_min(K), frac_w(K);
double tot_dur = 0.0;
for (std::size_t h = 0; h < K; ++h) {
    lam[h] = rate_hr[h] * 0.5 / 60.0; // arrivals per MINUTE
    dur_min[h] = dur_hr[h] * 60.0;
    tot_dur += dur_min[h];
}
for (std::size_t h = 0; h < K; ++h) frac_w[h] = dur_min[h] / tot_dur;

// The study's simulated ground truth, N = 24..34 (Table 13).
const double sim_W[11] = {154.0661, 118.3936, 99.962, 88.2379, 80.9272, 75.0887,
                          70.745, 67.4326, 64.9719, 62.7913, 61.2534};
const double sim_Qex[11] = {89.8508, 63.3970, 49.3650, 40.8783, 35.3546, 30.6543,
                          27.4111, 24.7218, 22.3622, 20.3893, 18.9252};
const std::size_t tail_factor = 15; // M_trunc = N*(1 + tailFactor)
const SoqnParams p;

std::printf("Rotterdam SOQN exponential blending vs Dhingra simulation (tailFactor=%zu)\n",
            tail_factor);
std::printf("%4s | %10s %10s %7s | %10s %10s %7s\n", "N", "W_blend", "W_sim", "err%", "Qex_bl",
            "Qex_sim", "err%", "Qex_bl", "Qex_sim", "err%");

... (6 source lines omitted; full implementation: cpp/examples/advanced/)

const BlendResult r = blend_soqn(N, lam, dur_min, frac_w, mu1, mu2, tail_factor);
const std::size_t j = N - 24;
std::printf("%4zu | %10.4f %10.4f %6.2f | %10.4f %10.4f %6.2f\n", N, r.W, sim_W[j],
            100.0 * std::fabs(r.W - sim_W[j]) / sim_W[j], r.Qex, sim_Qex[j],
            100.0 * std::fabs(r.Qex - sim_Qex[j]) / sim_Qex[j]);
}
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.11.10 renv_threestages_repairmen

The three-stage version.

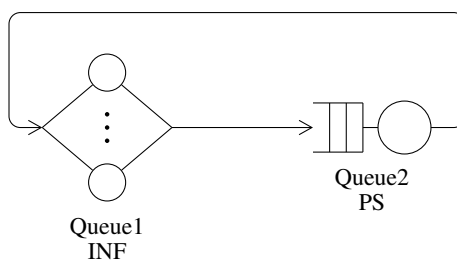


Figure 3.53: Topology of `renv_threestages_repairmen`: a closed network over 2 queueing stations.

Nodes	2: Queue, Delay
Classes	1: Class1 (closed, $N = 2$)
Scheduling	Queue1: INF; Queue2: PS
Service	Queue1 – Class1: Exp(3) Queue2 – Class1: Exp(1)
Solvers	ENV

Table 3.56: Features of `renv_threestages_repairmen`.

The example is built and solved as follows.

```
const std::size_t E = 3;
const double N = 2.0;
const char* env_name[3] = {"Stage1", "Stage2", "Stage3"};
const char* env_type[3] = {"UP", "DOWN", "FAST"};

// rate = ones(M,E); rate(M,:) = 1:E; rate(1,:) = E:-1:1, with M = 2.
const double rate_delay[3] = {3.0, 2.0, 1.0};
const double rate_queue[3] = {1.0, 2.0, 3.0};

note("Rate matrix:");
for (std::size_t s = 0; s < E; ++s)
    std::printf("  stage %zu: Queue1 %.6g Queue2 %.6g\n", s + 1, rate_delay[s],
               rate_queue[s]);

Env e("MyEnv", E);
std::vector<Net> sub;
for (std::size_t s = 0; s < E; ++s) sub.push_back(renv_genqn_model(rate_delay[s], rate_queue[
s], N));
for (std::size_t s = 0; s < E; ++s)
    e.set_stage(s, env_name[s], env_type[s], sub[s].get_struct());

// circul(3) is the right circular shift squared: 1 -> 2 -> 3 -> 1 at rate 1.
const double env_rates[3][3] = {{0.0, 1.0, 0.0}, {0.0, 0.0, 1.0}, {1.0, 0.0, 0.0}};
note("Environment transition rates (circulant matrix):");
for (std::size_t a = 0; a < E; ++a)
    std::printf("  %.6g %.6g %.6g\n", env_rates[a][0], env_rates[a][1], env_rates[a][2]);
for (std::size_t a = 0; a < E; ++a)

    ... (27 source lines omitted; full implementation: cpp/examples/advanced/)

// into a printed line would report `ok` with no table. Letting it throw is
// what `line-examples` reports as FAILED and the harvest records as an error.
const env::EnvAnalyzerSolution<double> r = env::solver_env(e, o);
section("ENV (CTMC)", "ENV");
env_banner(e, o, r);
print_env_avg(e.stage(0).model, r);
```

Running this edition prints

```
Rate matrix:
  stage 1: Queue1 3 Queue2 1
  stage 2: Queue1 2 Queue2 2
  stage 3: Queue1 1 Queue2 3
Environment transition rates (circulant matrix):
  0 1 0
  0 0 1
  1 0 0
The metasolver considers an environment with 3 stages and a queueing network with 2 stations.
This example illustrates the computation of the infinitesimal generator of the system.
Stage  Name      Type      Model
1      Stage1     UP       qn1
2      Stage2     DOWN     qn1
3      Stage3     FAST     qn1

SOLVER: SolverENV.getGenerator
N/A: the joint (environment, network) infinitesimal generator and the per-stage generators are read off
enumerated CTMC stage solves; this port carries no getGenerator on the ENV path

SOLVER: ENV (CTMC)
SolverENV method=meanfield stages=3 horizon=1000 points=1001 iters=3
```

Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Queue1	Class1	0.83053	0.83053	nan	0.62305	nan
1.333						
Queue2	Class1	1.1695	0.7581	nan	0.85696	nan
1.3647						

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by ENV.

3.11.11 renv_twostages_repairmen

A repairman model in a two-stage random environment.

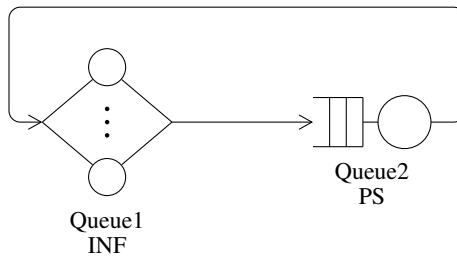


Figure 3.54: Topology of renv_twostages_repairmen: a closed network over 2 queueing stations.

Nodes	2: Queue, Delay
Classes	1: Class1 (closed, $N = 1$)
Scheduling	Queue1: INF; Queue2: PS
Service	Queue1 – Class1: Exp(2) Queue2 – Class1: Exp(1)
Solvers	ENV

Table 3.57: Features of renv_twostages_repairmen.

The example is built and solved as follows.

```
const std::size_t E = 2;
const double N = 1.0;
const double rate[2][2] = {{2.0, 1.0}, {1.0, 2.0}};
const char* env_name[2] = {"Stage1", "Stage2"};
const char* env_type[2] = {"UP", "DOWN"};

Env e("MyEnv", E);
std::vector<Net> sub;
for (std::size_t s = 0; s < E; ++s) sub.push_back(renv_genqn_model(rate[0][s], rate[1][s], N)
);
for (std::size_t s = 0; s < E; ++s)
    e.set_stage(s, env_name[s], env_type[s], sub[s].get_struct());

const double env_rates[2][2] = {{0.0, 1.0}, {0.5, 0.5}};
for (std::size_t a = 0; a < E; ++a)
    for (std::size_t b = 0; b < E; ++b)
        if (env_rates[a][b] > 0.0) e.add_transition(a, b, Exp(env_rates[a][b]));

note("Stage Table:");
stage_table(e);

env::EnvOptions o;
o.method = "meanfield";
o.iter_max = 100;
o.iter_tol = 0.01;
// The reference's ENV timespan is [0, Inf]; the horizon that reaches a
// stage is the fluid solver's own, which it sets to 1e3.
```

```

... (22 source lines omitted; full implementation: cpp/examples/advanced/)

// NOT a gap in this port: the reference leaves its CTMC alternative
// commented out, as the MATLAB script it follows does, so there is no call
// to record and nothing was refused.
note("\n## Alternative: CTMC Solver (Commented)");
note("The MATLAB version also shows an alternative using CTMC, which is commented out in "
     "the original.");

```

Running this edition prints

```

Stage Table:
Stage  Name          Type      Model
1      Stage1         UP        qn1
2      Stage2         DOWN      qn1

SOLVER: ENV
SolverENV method=meanfield stages=2 horizon=1000 points=1001 iters=5
QN[Queue1,Class1] = 0.555562  UN = 0.555562  TN = 0.691323
QN[Queue2,Class1] = 0.444438  UN = 0.444438  TN = 0.691303

Average Table:
Station  JobClass      QLen      Util      RespT      ResidT      ArvR
Tput
Queue1   Class1           0.55556   0.55556   nan        0.80362   nan
0.69132
Queue2   Class1           0.44444   0.44444   nan        0.6429    nan
0.6913

## Alternative: CTMC Solver (Commented)
The MATLAB version also shows an alternative using CTMC, which is commented out in the original.

```

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by ENV.

3.12 Layered cache-queueing models

A `CacheTask` inside a layered model exposes `ItemEntry` points, and a call to one is a cache request whose hit or miss outcome selects the downstream branch. The replacement policy is simulated directly, so hit and miss are exact sample-path events rather than a hit-ratio parameter.

3.12.1 `lcq_async_cache`

An ASYNCHRONOUS (non-blocking) call into the cache task, as a prefetch or a warm-up makes: the client continues without waiting for the reply while the request still resolves through `POST_CACHE` hit/miss branching.

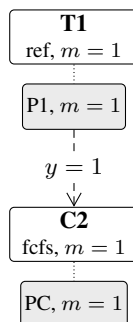


Figure 3.55: Call graph of `lcq_async_cache`: 2 tasks over 2 processors, drawn with the reference task on top and a called task below its caller; the shaded box under each task is the processor it runs on.

Processors	2: P1 (ps, $m = 1$), PC (ps, $m = 1$)
Tasks	2: T1 (ref, $m = 1$), C2 (fcfs, $m = 1$)
Entries	2: E1, I2
Activities	4, host demands A1: 0.0, AC2: 0.0, AC2h: 1.0, AC2m: 2.0
Calls	1 (0 synchronous, 1 asynchronous)

Table 3.58: Features of `lcq_async_cache`.

The example is built and solved as follows.

```
const std::size_t totalitems = 4;

lqn::LqnBuilder<double> b;
b.processor("P1", 1, SchedStrategy::PS);
b.task("T1", 1, SchedStrategy::REF, "P1");
b.entry("E1", "T1");

b.processor("PC", 1, SchedStrategy::PS);
b.cache_task("C2", 1, SchedStrategy::FCFS, "PC", totalitems, {2}, ReplacementStrategy::LRU);
b.item_entry("I2", "C2", totalitems, uniform_access(totalitems));

b.activity("A1", Immediate(), "T1");
b.bound_to("A1", "E1");
b.async_call("A1", "I2", 1.0);

b.activity("AC2", Immediate(), "C2");
b.bound_to("AC2", "I2");
b.activity("AC2h", Exp(1.0), "C2");
b.activity("AC2m", Exp(0.5), "C2");
b.cache_access("AC2", "AC2h", "AC2m");
b.replies_to("AC2h", "I2");
b.replies_to("AC2m", "I2");

const Lqn l = b.build();
note("=== Solving Async Cache Model ===");
section("LN (MVA)");

... (3 source lines omitted; full implementation: cpp/examples/advanced/)

note("To compare async vs sync cache access, run lcq_singlehost.");
note("");
note("Expected differences:");
note("- Async: Lower client response time (no blocking)");
note("- Async: Higher client throughput (no cache bottleneck)");
note("- Similar cache hit/miss ratios (same cache logic)");
```

Running this edition prints

```
=== Solving Async Cache Model ===

SOLVER: LN (MVA)
SolverLN(layers=mva) layers=3 iterations=52 converged=1
Node      NodeType      QLen      Util      RespT      ResidT      Tput
P1         Processor      NaN        0         NaN         NaN         NaN
PC         Processor      NaN        0.5       NaN         NaN         NaN
T1         RefTask        0          0         NaN         0          3.33333e+07
C2         Task           0.5        0.5       NaN         1.5         0.333333
E1         Entry          0          0         0           NaN         3.33333e+07
I2         Entry          0.5        0.5       1.5         NaN         0.333333
A1         Activity       0.333333   0         1e-08        0          3.33333e+07
AC2        Activity       0          0         0           0           0
AC2h       Activity       0.166667   0.166667   1           0.5         0.166667
AC2m       Activity       0.333333   0.333333   2           1           0.166667

=== Compare with Synchronous Version ===
To compare async vs sync cache access, run lcq_singlehost.

Expected differences:
- Async: Lower client response time (no blocking)
- Async: Higher client throughput (no cache bottleneck)
```

- Similar cache hit/miss ratios (same cache logic)

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by LN.

3.12.2 lcq_singlehost

A cache task and its item entries inside a layered model on one processor.

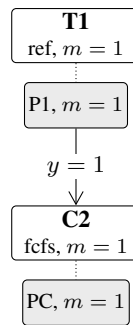


Figure 3.56: Call graph of lcq_singlehost: 2 tasks over 2 processors, drawn with the reference task on top and a called task below its caller; the shaded box under each task is the processor it runs on.

Processors	2: P1 (ps, $m = 1$), PC (ps, $m = 1$)
Tasks	2: T1 (ref, $m = 1$), C2 (fcfs, $m = 1$)
Entries	2: E1, I2
Activities	4, host demands A1: 0.0, AC2: 0.0, AC2h: 1.0, AC2m: 2.0
Calls	1 (1 synchronous, 0 asynchronous)

Table 3.59: Features of lcq_singlehost.

The example is built and solved as follows.

```

const std::size_t totalitems = 4;

lqn::LqnBuilder<double> b;
b.processor("P1", 1, SchedStrategy::PS);
b.task("T1", 1, SchedStrategy::REF, "P1");
b.entry("E1", "T1");

b.processor("PC", 1, SchedStrategy::PS);
b.cache_task("C2", 1, SchedStrategy::FCFS, "PC", totalitems, {2}, ReplacementStrategy::RR);
b.item_entry("I2", "C2", totalitems, uniform_access(totalitems));

b.activity("A1", Immediate(), "T1");
b.bound_to("A1", "E1");
b.sync_call("A1", "I2", 1.0);

b.activity("AC2", Immediate(), "C2");
b.bound_to("AC2", "I2");
b.activity("AC2h", Exp(1.0), "C2");
b.activity("AC2m", Exp(0.5), "C2");
b.cache_access("AC2", "AC2h", "AC2m");
b.replies_to("AC2h", "I2");
b.replies_to("AC2m", "I2");

const Lqn l = b.build();
section("LN (MVA)");
run_ln(l, "mva");

```

Running this edition prints

SOLVER: LN (MVA)						
SolverLN(layers=mva) layers=3 iterations=52 converged=1						
Node	NodeType	QLen	Util	RespT	ResidT	Tput
P1	Processor	NaN	0	NaN	NaN	NaN
PC	Processor	NaN	1	NaN	NaN	NaN
T1	RefTask	1	0	NaN	0	0.666667
C2	Task	1	1	NaN	1.5	0.666667
E1	Entry	1	0	1.5	NaN	0.666667
I2	Entry	1	1	1.5	NaN	0.666667
A1	Activity	1	0	1.5	0	0.666667
AC2	Activity	0	0	0	0	0
AC2h	Activity	0.333333	0.333333	1	0.5	0.333333
AC2m	Activity	0.666667	0.666667	2	1	0.333333

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by LN.

3.12.3 lcq_threehosts

The three-processor version, where the cache and its clients are on separate hosts.

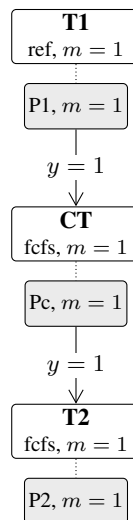


Figure 3.57: Call graph of `lcq_threehosts`: 3 tasks over 3 processors, drawn with the reference task on top and a called task below its caller; the shaded box under each task is the processor it runs on.

Processors	3: P1 (ps, $m = 1$), Pc (ps, $m = 1$), P2 (ps, $m = 1$)
Tasks	3: T1 (ref, $m = 1$), CT (fcfs, $m = 1$), T2 (fcfs, $m = 1$)
Entries	3: E1, IE, E2
Activities	5, host demands A1: 0.0, Ac: 0.0, Ac_hit: 1.0, Ac_miss: 2.0, A2: 0.2
Calls	2 (2 synchronous, 0 asynchronous)

Table 3.60: Features of `lcq_threehosts`.

The example is built and solved as follows.

```

const std::size_t totalitems = 4;
const double nusers = 1.0, ntokens = 1.0;

lqn::LqnBuilder<double> b;
b.processor("P1", 1, SchedStrategy::PS);
b.task("T1", nusers, SchedStrategy::REF, "P1");
b.entry("E1", "T1");

// Two cache lists of one item each: the multi-level cache of the reference.

```

```

b.processor("Pc", 1, SchedStrategy::PS);
b.cache_task("CT", ntokens, SchedStrategy::FCFS, "Pc", totalitems, {1, 1},
    ReplacementStrategy::RR);
b.item_entry("IE", "CT", totalitems, uniform_access(totalitems));

b.processor("P2", 1, SchedStrategy::PS);
b.task("T2", 1, SchedStrategy::FCFS, "P2");
b.entry("E2", "T2");
b.activity("A2", Exp(5.0), "T2");
b.bound_to("A2", "E2");
b.replies_to("A2", "E2");

b.activity("A1", Immediate(), "T1");
b.bound_to("A1", "E1");
b.sync_call("A1", "IE", 1.0);

b.activity("Ac", Immediate(), "CT");

    ... (7 source lines omitted; full implementation: cpp/examples/advanced/)

const Lqn l = b.build();
section("LN(NC)");
run_ln(l, "nc");
section("LN(MVA)");
run_ln(l, "mva");

```

Running this edition prints

```

SOLVER: LN(NC)
SolverLN(layers=nc) layers=5 iterations=52 converged=1
Node      NodeType      QLen      Util      RespT      ResidT      Tput
P1         Processor      NaN        0          NaN         NaN         NaN
Pc         Processor      NaN        0.936212   NaN         NaN         NaN
P2         Processor      NaN        0.0625     NaN         NaN         NaN
T1         RefTask        1          0          NaN         0          0.618548
CT         Task           0.99794    0.936212   NaN         1.51669    0.617274
T2         Task           0.0625     0.0625     NaN         0.2         0.3125
E1         Entry          1          0          1.61669     NaN         0.618548
IE         Entry          0.99794    0.936212   1.61669     NaN         0.617274
E2         Entry          0.0625     0.0625     0.2         NaN         0.3125
A2         Activity       0.0625     0.0625     0.2         0.2         0.3125
A1         Activity       1          0          1.61669     0          0.618548
Ac         Activity       0          0          0           0           0
Ac_hit     Activity       0.298335   0.298335   1           0.483311    0.298335

... (11 captured-output lines omitted; rerun the source example for the full output)

E1         Entry          1          0          1.6         NaN         0.625
IE         Entry          1          0.9375     1.6         NaN         0.625
E2         Entry          0.0625     0.0625     0.2         NaN         0.3125
A2         Activity       0.0625     0.0625     0.2         0.2         0.3125
A1         Activity       1          0          1.6         0          0.625
Ac         Activity       0          0          0           0           0
Ac_hit     Activity       0.3125     0.3125     1           0.5         0.3125
Ac_miss    Activity       0.6875     0.625      2.2         1           0.3125

```

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by LN.

3.13 Pass and swap

Under the pass-and-swap discipline a job that finds an incompatible job in service swaps with it rather than waiting, subject to a compatibility relation between classes. The initial placement of a closed population is therefore part of the model: the state must be set in the order the discipline assumes, and a placement that ignores it changes the answer.

3.13.1 pas_closed_tandem_fig6

The closed tandem of the reference paper, figure 6.

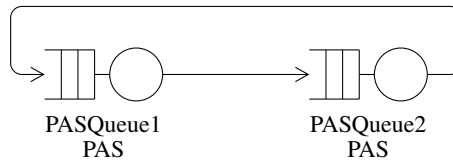


Figure 3.58: Topology of pas_closed_tandem_fig6: a closed network over 2 queueing stations.

Nodes	2: Queue $\times$ 2
Classes	6: Class1 (closed, $N = 1$), Class2 (closed, $N = 1$), Class3 (closed, $N = 1$), Class4 (closed, $N = 1$), Class5 (closed, $N = 1$), Class6 (closed, $N = 1$)
Scheduling	PASQueue1: PAS; PASQueue2: PAS
Service	PASQueue1 – Class1: Exp(1); Class2: Exp(1); Class3: Exp(1); Class4: Exp(1); Class5: Exp(1); Class6: Exp(1) PASQueue2 – Class1: Exp(1.3); Class2: Exp(1.3); Class3: Exp(1.3); Class4: Exp(1.3); Class5: Exp(1.3); Class6: Exp(1.3)
Features	finite buffer 6 at PASQueue1; finite buffer 6 at PASQueue2; class switching on 2 links
Solvers	CTMC

Table 3.61: Features of pas_closed_tandem_fig6.

The example is built and solved as follows.

```
const BoolGraph G = fig5_graph();
const PasRate mu1 = [] (const std::vector<std::size_t>&) { return kFigMu1; };
const PasRate mu2 = [] (const std::vector<std::size_t>&) { return kFigMu2; };

Net m("PASclosedTandem");
Queue q1(m, "PASQueue1", SchedStrategy::PAS);
Queue q2(m, "PASQueue2", SchedStrategy::PAS);
std::vector<std::size_t> cls;
for (std::size_t r = 0; r < 6; ++r)
    cls.push_back(m.add_closed_class("Class" + std::to_string(r + 1), 1.0, q1));
declare_pas(m, q1, mu1, G);
declare_pas(m, q2, mu2, G);
q1.set_number_of_servers(1.0);
q1.set_capacity(6.0);
q2.set_number_of_servers(1.0);
q2.set_capacity(6.0);
Routing P;
for (std::size_t r = 0; r < 6; ++r) cyclic(P, cls[r], {q1, q2});
m.link(P);

note("NOTE: the reference selects the recurrent component with "
     "q1.setState([1,2,3,4,5,6]);");
note("    this port has no PAS setState, and a PAS station's state space is built");
note("    count-shaped rather than as the ordered list the walk reads.");

const Sn& sn = m.get_struct();

... (21 source lines omitted; full implementation: cpp/examples/advanced/)

err = std::max(err, std::fabs(Qr2[i] - v));
}
std::printf("\nCTMC vs reference: max|dQ| = %.3e\n", err);
note(err <= 1e-6 ? "PASS: CTMC matches the pass-and-swap reference."
      : "MISMATCH: the CTMC walk never left its initial placement (see the note "
        "above).");
```

Running this edition prints

NOTE: the reference selects the recurrent component with `q1.setState([1,2,3,4,5,6]);`
 this port has no PAS setState, and a PAS station's state space is built
 count-shaped rather than as the ordered list the walk reads.

```
SOLVER: CTMC
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
Tput
PASQueue1   Class1      0.39364    0         1.6485     1.6485     0.23878
0.23878
PASQueue1   Class2      0.39364    0         1.6485     1.6485     0.23878
0.23878
PASQueue1   Class3      0.74993    0         5.6857     5.6857     0.1319
0.1319
PASQueue1   Class4      0.76346    0         5.9723     5.9723     0.12783
0.12783
PASQueue1   Class5      0.74993    0         5.6857     5.6857     0.1319
0.1319
PASQueue1   Class6      0.94313    0.94313    12.756     12.756     0.073936
0.073936
PASQueue2   Class1      0.60636    0.36274    2.5394     2.5394     0.23878
0.23878
PASQueue2   Class2      0.60636    0.36274    2.5394     2.5394     0.23878
0.23878
PASQueue2   Class3      0.25007    0         1.896      1.896      0.1319
0.1319
PASQueue2   Class4      0.23654    0         1.8503     1.8503     0.12783
0.12783
```

... (11 captured-output lines omitted; rerun the source example for the full output)

```
PASQueue2 Class2 0.10185 0.60636
PASQueue2 Class3 0.35636 0.25007
PASQueue2 Class4 0.36437 0.23654
PASQueue2 Class5 0.35636 0.25007
PASQueue2 Class6 0.72548 0.05687
```

CTMC vs reference: $\max|dQ| = 6.686e-01$

MISMATCH: the CTMC walk never left its initial placement (see the note above).

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by CTMC.

3.13.2 pas_compatibility_5class

Five classes and the compatibility matrix that says which may swap with which.

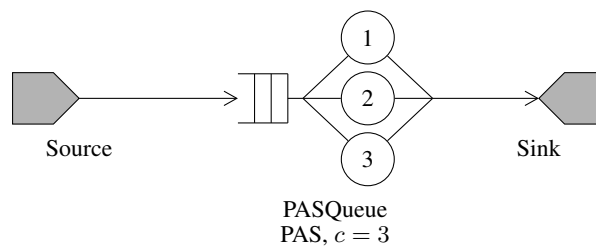


Figure 3.59: Topology of `pas_compatibility_5class`: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	5: Class1 (open), Class2 (open), Class3 (open), Class4 (open), Class5 (open)
Scheduling	PASQueue: PAS ($c = 3$)
Arrivals	Source – Class1: Exp(0.5); Class2: Exp(0.4); Class3: Exp(0.3); Class4: Exp(0.2); Class5: Exp(0.1)
Service	PASQueue – Class1: Exp(1); Class2: Exp(1); Class3: Exp(1); Class4: Exp(2); Class5: Exp(1)
Features	finite buffer 3 at PASQueue
Solvers	CTMC

Table 3.62: Features of `pas_compatibility_5class`.

The example is built and solved as follows.

```
// comp[s][r] = 1 iff server s serves class r (0-based classes).
static const int comp[3][5] = {{1, 0, 0, 1, 0}, {0, 1, 0, 1, 0}, {0, 0, 1, 0, 1}};
const PasRate mu = [] (const std::vector<std::size_t>& c) {
    double n = 0.0;
    for (std::size_t s = 0; s < 3; ++s) {
        bool any = false;
        for (std::size_t i = 0; i < c.size(); ++i)
            if (comp[s][c[i] - 1]) any = true;
        if (any) n += 1.0;
    }
    return n;
};
const std::vector<Edge> edges{{0, 2}, {0, 4}, {1, 3}, {2, 3}, {3, 4}};
Net m = pas_open_model("PAScompatibility", {0.5, 0.4, 0.3, 0.2, 0.1}, mu,
                      swap_bool(5, edges), 3.0, 3.0);
section("CTMC");
ctmc_avg(m.get_struct(), 3.0);
```

Running this edition prints

```
SOLVER: CTMC
NOTE: a PAS station's state space is built count-shaped here, not as the ordered list;
      these numbers do not match MATLAB (see the file header).
```

Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Source	Class1	0	0	0	0	0
0.39099						
Source	Class2	0	0	0	0	0
0.31279						
Source	Class3	0	0	0	0	0
0.23459						
Source	Class4	0	0	0	0	0
0.1564						
Source	Class5	0	0	0	0	0
0.078198						
PASQueue	Class1	0.56149	0.13033	1.4361	1.4361	0.39099
0.39099						
PASQueue	Class2	0.42141	0.10426	1.3472	1.3472	0.31279
0.31279						
PASQueue	Class3	0.30532	0.078198	1.3015	1.3015	0.23459
0.23459						
PASQueue	Class4	0.11235	0.033661	0.71837	0.71837	0.1564
0.1564						
PASQueue	Class5	0.10177	0.026066	1.3015	1.3015	0.078198
0.078198						

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by CTMC.

3.13.3 pas_cyclic_ctmc_vs_bruteforce

A cyclic pass-and-swap network solved by CTMC and by brute-force enumeration, as a cross-check. No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

Solvers	CTMC, LDES
----------------	------------

Table 3.63: Features of `pas_cyclic_ctmc_vs_bruteforce`.

The example is built and solved as follows.

```
const std::size_t R = 2;
const std::vector<std::size_t> K(kCycPop, kCycPop + R);
const std::vector<double> e1(R, 1.0), e2(R, 1.0);
```

```

const double Gbf = cyc_norm_const(K, e1, e2);
std::printf("Brute-force normalizing constant G = %.12g\n\n", Gbf);

std::vector<double> Xbf(R, 0.0);
for (std::size_t r = 0; r < R; ++r) {
    std::vector<std::size_t> Km = K;
    Km[r] -= 1;
    Xbf[r] = cyc_norm_const(Km, e1, e2) / Gbf;
}
std::vector<std::vector<double>> Qbf(2, std::vector<double>(R, 0.0));
const std::vector<std::vector<std::size_t>> splits = enum_splits(K);
for (std::size_t i = 0; i < splits.size(); ++i) {
    std::vector<std::size_t> comp(R, 0);
    for (std::size_t r = 0; r < R; ++r) comp[r] = K[r] - splits[i][r];
    const double w = oi_sum(splits[i], e1, cyc_mul) * oi_sum(comp, e2, cyc_mu2);
    for (std::size_t r = 0; r < R; ++r) Qbf[0][r] += static_cast<double>(splits[i][r]) * w;
}
for (std::size_t r = 0; r < R; ++r) {
    Qbf[0][r] /= Gbf;
    Qbf[1][r] = static_cast<double>(K[r]) - Qbf[0][r];
}

... (68 source lines omitted; full implementation: cpp/examples/advanced/)

    relQ = std::max(relQ, std::fabs(l.QN(i, c) - Qbf[qi][c]) / Qbf[qi][c]);
}
}
std::printf("\nLDES:  max rel|dX| = %.3e  max rel|dQ| = %.3e\n", relX, relQ);
note(relX <= 0.02 && relQ <= 0.02 ? "PASS: LDES matches brute-force product form within 2%."
    : "MISMATCH: LDES differs from brute force by more than 2%."
    );

```

Running this edition prints

```

Brute-force normalizing constant G = 3.694444444444

SOLVER: CTMC
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
Tput
PASQueue1    Class1          1.3534     0.48496    1.3953      1.3953      0.96992
0.96992
PASQueue1    Class2          1.1805     0.40977    1.4404      1.4404      0.81955
0.81955
PASQueue2    Class1          0.64662    0.16165    0.66667     0.66667     0.96992
0.96992
PASQueue2    Class2          0.81955     0.20489      1          1          0.81955
0.81955

--- CTMC vs brute-force ---
metric  class      brute-force      CTMC      abs.err
Tput    Class1    0.9699248120    0.9699248120    1.11e-16
Tput    Class2    0.8195488722    0.8195488722    0.00e+00
QLen q1  Class1    1.3533834586    1.3533834586    0.00e+00
QLen q1  Class2    1.1804511278    1.1804511278    0.00e+00
QLen q2  Class1    0.6466165414    0.6466165414    1.11e-16
QLen q2  Class2    0.8195488722    0.8195488722    3.33e-16

CTMC:  max|dX| = 1.110e-16  max|dQ| = 3.331e-16
PASS: CTMC matches brute-force product form within 1.0e-09.

SOLVER: LDES
N/A: the reference then repeats the comparison with a 2e5-sample simulation at 2% tolerance; this port
    carries no LDES engine

```

The rows repeat for each of the 2 solvers the script runs (CTMC, LDES), which is the point of the example: the same model read by methods of different cost and different exactness.

3.13.4 pas_cyclic_normconst_bruteforce

The same cross-check on the normalizing constant.

No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

The example is built and solved as follows.

```
const std::size_t R = 2;
const std::vector<std::size_t> K(kCycPop, kCycPop + R);
const std::vector<double> e1(R, 1.0), e2(R, 1.0);

const double G = cyc_norm_const(K, e1, e2);
std::printf("Closed cyclic PAS network: R=%zu classes, population K=[", R);
for (std::size_t r = 0; r < R; ++r) std::printf("%s%zu", r ? " " : "", K[r]);
std::printf("]\n");
std::printf("Brute-force normalizing constant G = %.15g\n", G);

std::vector<double> X(R, 0.0);
for (std::size_t r = 0; r < R; ++r) {
    if (K[r] == 0) continue;
    std::vector<std::size_t> Km = K;
    Km[r] -= 1;
    X[r] = cyc_norm_const(Km, e1, e2) / G;
}
std::printf("Per-class throughput X = [");
for (std::size_t r = 0; r < R; ++r) std::printf("%s%.6g", r ? " " : "", X[r]);
std::printf("]\n");

// Single class, two single-server PAS stations: the closed form is the
// geometric sum  $G = \sum_n (1/\mu_1)^n (1/\mu_2)^{N-n}$ .
const std::size_t N = 5;
const std::vector<std::size_t> Kn(1, N);
const std::vector<double> en(1, 1.0);

... (3 source lines omitted; full implementation: cpp/examples/advanced/)

Gcf += std::pow(1.0 / kGnMu1, static_cast<double>(n)) *
        std::pow(1.0 / kGnMu2, static_cast<double>(N - n));
if (std::fabs(Gbf - Gcf) > 1e-12 * Gcf)
    throw NumericError("self-test failed: brute-force " + std::to_string(Gbf) +
        " vs closed-form " + std::to_string(Gcf));
std::printf("Self-test (single-class Gordon-Newell): PASS (G=%.12g)\n", Gbf);
```

Running this edition prints

```
Closed cyclic PAS network: R=2 classes, population K=[2 2]
Brute-force normalizing constant G = 3.694444444444444
Per-class throughput X = [0.969925 0.819549]
Self-test (single-class Gordon-Newell): PASS (G=0.0855195473251)
```

3.13.5 pas_mmk

The pass-and-swap discipline on an M/M/k station.

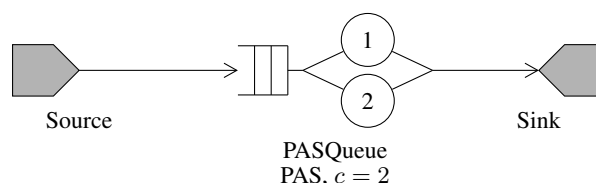


Figure 3.60: Topology of `pas_mmk`: an open network over 1 queueing station.

The example is built and solved as follows.

```
const std::size_t K = 2;
const PasRate mu = [K] (const std::vector<std::size_t>& c) {
```

Nodes	3: Source, Queue, Sink
Classes	2: Class1 (open), Class2 (open)
Scheduling	PASQueue: PAS ($c = 2$)
Arrivals	Source – Class1: Exp(0.7); Class2: Exp(0.5)
Service	PASQueue – Class1: Exp(1); Class2: Exp(1)
Features	finite buffer 4 at PASQueue
Solvers	CTMC

Table 3.64: Features of pas_mmk.

```

    return static_cast<double>(std::min(c.size(), K));
};
Net m = pas_open_model("PASmmk", {0.7, 0.5}, mu, swap_bool(2, {}), 2.0, 4.0);
section("CTMC");
ctmc_avg(m.get_struct(), 4.0);

```

Running this edition prints

```

SOLVER: CTMC
NOTE: a PAS station's state space is built count-shaped here, not as the ordered list;
      these numbers do not match MATLAB (see the file header).

```

Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Source	Class1	0	0	0	0	0
0.64976						
Source	Class2	0	0	0	0	0
0.46411						
PASQueue	Class1	0.80328	0.32488	1.2363	1.2363	0.64976
0.64976						
PASQueue	Class2	0.57377	0.23206	1.2363	1.2363	0.46411
0.46411						

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by CTMC.

3.13.6 pas_nc_sampling

The normalizing constant of a pass-and-swap network, by sampling.

No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

The example is built and solved as follows.

```

const std::size_t R = 6;
const std::vector<std::vector<bool>> G = fig6_swap_graph();

Net m("PASsampling");
Queue q1(m, "PASQueue1", SchedStrategy::PAS);
Queue q2(m, "PASQueue2", SchedStrategy::PAS);
std::vector<std::size_t> cls;
for (std::size_t r = 0; r < R; ++r)
    cls.push_back(m.add_closed_class("Class" + std::to_string(r + 1), 1.0, q1));
declare_pas(m, q1, is_mu1, G);
declare_pas(m, q2, is_mu2, G);
q1.set_number_of_servers(1.0);
q1.set_capacity(static_cast<double>(R));
q2.set_number_of_servers(1.0);
q2.set_capacity(static_cast<double>(R));
Routing P;
for (std::size_t r = 0; r < R; ++r) cyclic(P, cls[r], {q1, q2});
m.link(P);

const Sn& sn = m.get_struct();

std::printf("=== CTMC (exact) ===\n");

```

```

ctmc::CtmcOptions co;
co.cutoff = static_cast<double>(R);
const mva::AvgResult<double> rc = ctmc::solver_ctmc_run_analyzer(sn, co);
print_avg(sn, rc);

... (18 source lines omitted; full implementation: cpp/examples/advanced/)

std::printf("\nNC-samp vs CTMC:  max|dQ| = %.3e (importance-sampling noise)\n", err);
note(err <= 5e-2
    ? "PASS: SolverNC 'sampling' matches the exact CTMC within importance-sampling noise
    : "MISMATCH: the closed PAS CTMC walk starts from a COUNT-shaped initial row and "
      "never leaves it (see the file header), so the exact side of this comparison is "
      "not the reference's.");

```

Running this edition prints

```

=== CTMC (exact) ===
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
Tput
PASQueue1    Class1      0.39364    0          1.6485     1.6485     0.23878
0.23878
PASQueue1    Class2      0.39364    0          1.6485     1.6485     0.23878
0.23878
PASQueue1    Class3      0.74993    0          5.6857     5.6857     0.1319
0.1319
PASQueue1    Class4      0.76346    0          5.9723     5.9723     0.12783
0.12783
PASQueue1    Class5      0.74993    0          5.6857     5.6857     0.1319
0.1319
PASQueue1    Class6      0.94313    0.94313    12.756     12.756     0.073936
0.073936
PASQueue2    Class1      0.60636    0.36274    2.5394     2.5394     0.23878
0.23878
PASQueue2    Class2      0.60636    0.36274    2.5394     2.5394     0.23878
0.23878
PASQueue2    Class3      0.25007    0          1.896      1.896      0.1319
0.1319
PASQueue2    Class4      0.23654    0          1.8503     1.8503     0.12783
0.12783
PASQueue2    Class5      0.25007    0          1.896      1.896      0.1319
0.1319
PASQueue2    Class6      0.056874   0          0.76923    0.76923    0.073936
0.073936
=== SolverNC method='sampling' (importance sampling, 5e5 samples) ===
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
Tput
... (21 captured-output lines omitted; rerun the source example for the full output)

PASQueue2    Class2      0.60636    0.10584
PASQueue2    Class3      0.25007    0.34360
PASQueue2    Class4      0.23654    0.38172
PASQueue2    Class5      0.25007    0.34374
PASQueue2    Class6      0.05687    0.72548

NC-samp vs CTMC:  max|dQ| = 6.686e-01 (importance-sampling noise)
MISMATCH: the closed PAS CTMC walk starts from a COUNT-shaped initial row and never leaves it (see the
file header), so the exact side of this comparison is not the reference's.

```

3.13.7 pas_saturation

The saturated regime, where the swap rule sets the throughput.

No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

Solvers	CTMC, LDES
---------	------------

Table 3.65: Features of `pas_saturation`.

The example is built and solved as follows.

```
// comp[s][r] = 1 iff server s serves class r; cap_s is its capacity.
static const int comp[3][2] = {{1, 0}, {1, 1}, {0, 1}};
static const double cap_s[3] = {2.0, 1.0, 2.0};
const PasRate mu_rank = [] (const std::vector<std::size_t>& c) {
    double s = 0.0;
    for (std::size_t k = 0; k < 3; ++k) {
        bool any = false;
        for (std::size_t i = 0; i < c.size(); ++i)
            if (comp[k][c[i] - 1]) any = true;
        if (any) s += cap_s[k];
    }
    return s;
};

// save_model: mu(c) crosses the wire as a rate table over a box lattice,
// and the table size is constant in the buffer, which is the reference's
// point. It is NOT the reference's SIZE: `lattice_cutoffs` materializes an
// open class over a fixed box of 10, so the cutoffs are [10, 10] and the
// table 120 entries where the reference's saturation cutoffs give [1, 1]
// and 3.
note("the order-independent rate table crosses the wire at a size independent of the buffer:");
const double caps[3] = {8.0, 100.0, 1000.0};
for (std::size_t k = 0; k < 3; ++k) {
    const double cap = caps[k];
    Net w = pas_open_model("PASsaturation", {1.5, 1.0}, mu_rank, swap_bool(2, {}), 3.0, cap);
    const io::detail::json j = io::network_json_envelope(w.get_struct());

    ... (51 source lines omitted; full implementation: cpp/examples/advanced/)

    throw std::logic_error(std::string("expected a finite-buffer error for ") + tags[k]);
} catch (const InputError& e) {
    const std::string msg(e.what());
    std::printf("  %20s -> %s\n", tags[k], msg.substr(0, 88).c_str());
}
}
```

Running this edition prints

```
the order-independent rate table crosses the wire at a size independent of the buffer:
cap=8      table entries = 120  cutoffs = [10,10]
cap=100    table entries = 120  cutoffs = [10,10]
cap=1000   table entries = 120  cutoffs = [10,10]

open compatibility PAS, buffer=8:

SOLVER: CTMC
NOTE: the PAS state space is count-shaped here; see the file header.
Station   JobClass   QLen   Util   RespT   ResidT   ArvR
Tput
Source    Class1      0       0       0       0       0
1.4908
Source    Class2      0       0       0       0       0
0.99389
PASQueue  Class1      1.0552  0.17598  0.70777  0.70777  1.4908
1.4908
PASQueue  Class2      0.57222  0.12393  0.57574  0.57574  0.99389
0.99389
CTMC QLen Class1 = 1.05517
CTMC QLen Class2 = 0.57222

SOLVER: LDES
N/A: the reference matches a 3e5-sample LDES run against CTMC at buffer 8 and then checks the actionable
error an unset (infinite) buffer raises; this port carries no LDES engine, and answering a simulation
block with the CTMC number would be a silent solver substitution
```

The rows repeat for each of the 2 solvers the script runs (CTMC, LDES), which is the point of the example: the same model read by methods of different cost and different exactness.

3.13.8 pas_selfloop

A self-looping class under pass-and-swap.

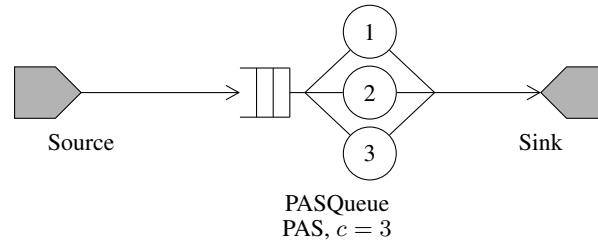


Figure 3.61: Topology of pas_selfloop: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	3: Class1 (open), Class2 (open), Class3 (open)
Scheduling	PASQueue: PAS ($c = 3$)
Arrivals	Source – Class1: Exp(0.6); Class2: Exp(0.4); Class3: Exp(0.3)
Service	PASQueue – Class1: Exp(1); Class2: Exp(1); Class3: Exp(1)
Features	finite buffer 3 at PASQueue
Solvers	CTMC

Table 3.66: Features of pas_selfloop.

The example is built and solved as follows.

```
// Three unit-rate servers, one per class: mu(c) = number of distinct classes.
const PasRate mu = [](const std::vector<std::size_t>& c) {
    bool seen[3] = {false, false, false};
    for (std::size_t i = 0; i < c.size(); ++i) seen[c[i] - 1] = true;
    return static_cast<double>((seen[0] ? 1 : 0) + (seen[1] ? 1 : 0) + (seen[2] ? 1 : 0));
};
BoolGraph g = swap_bool(3, {});
g[0][0] = true; // the self-loop
g[1][2] = true;
g[2][1] = true;
Net m = pas_open_model("PASselfloop", {0.6, 0.4, 0.3}, mu, g, 3.0, 3.0);
section("CTMC");
ctmc_avg(m.get_struct(), 3.0);
```

Running this edition prints

```
SOLVER: CTMC
NOTE: a PAS station's state space is built count-shaped here, not as the ordered list;
      these numbers do not match MATLAB (see the file header).
```

Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Source	Class1	0	0	0	0	0
0.47972						
Source	Class2	0	0	0	0	0
0.31981						
Source	Class3	0	0	0	0	0
0.23986						
PASQueue	Class1	0.72167	0.15991	1.5043	1.5043	0.47972
0.47972						
PASQueue	Class2	0.41993	0.1066	1.313	1.313	0.31981
0.31981						
PASQueue	Class3	0.29409	0.079954	1.2261	1.2261	0.23986
0.23986						

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by CTMC.

3.14 Reward models

A reward model attaches a rate to each state, or a value to each transition, and reports the expected accumulated reward. Cost per waiting job, revenue per completion and penalty per loss are the usual three, and together they turn a performance model into an economic one. Rewards may be declared over aggregate states when the detailed state space is too large to enumerate.

3.14.1 rewardModel_aggregation

Rewards over aggregated states, when the detailed state is too large to enumerate.

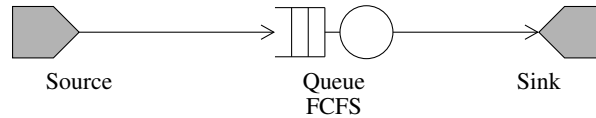


Figure 3.62: Topology of rewardModel_aggregation: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	2: HighPriority (open), LowPriority (open)
Scheduling	Queue: FCFS
Arrivals	Source – HighPriority: Exp(1); LowPriority: Exp(0.8)
Service	Queue – HighPriority: Exp(3); LowPriority: Exp(3)
Features	finite buffer 4 at Queue

Table 3.67: Features of rewardModel_aggregation.

The example is built and solved as follows.

```
std::size_t source = 0, queue = 0, sink = 0;
Net m = reward_model("RewardAggregationExample", SchedStrategy::FCFS, 1.0, 4.0, source, queue
,
    sink);
OpenClass c1(m, "HighPriority");
OpenClass c2(m, "LowPriority");
m.set_arrival(source, c1, Exp(1.0));
m.set_arrival(source, c2, Exp(0.8));
m.set_service(queue, c1, Exp(3.0));
m.set_service(queue, c2, Exp(3.0));
Routing P;
serial(P, c1, {source, queue, sink});
serial(P, c2, {source, queue, sink});
m.link(P);

const std::size_t ist = m.station_index(queue);
const std::size_t K = 2;
m.set_reward("TotalJobs", reward_qlen(ist, K));
m.set_reward("MaxClass", [ist, K](const std::vector<double>& s) {
    return std::max(at(s, ist, K, 1), at(s, ist, K, 2));
});
m.set_reward("ClassCount", [ist, K](const std::vector<double>& s) {
    return (at(s, ist, K, 1) > 0 ? 1.0 : 0.0) + (at(s, ist, K, 2) > 0 ? 1.0 : 0.0);
});
m.set_reward("HP_Jobs", reward_qlen(ist, K, 1));
m.set_reward("LP_Jobs", reward_qlen(ist, K, 2));
m.set_reward("WeightedLoad", [ist, K](const std::vector<double>& s) {
    ... (4 source lines omitted; full implementation: cpp/examples/advanced/)

const ctmc::CtmcOptions opt;
std::vector<std::string> names;
const std::vector<double> R = ctmc::solver_ctmc_avg_reward(sn, opt, &names);
```

```
section("CTMC");
print_rewards(names, R);
```

Running this edition prints

```
SOLVER: CTMC
=== Steady-State Expected Rewards ===
TotalJobs: 1.078418
MaxClass: 0.862690
ClassCount: 0.761440
HP_Jobs: 0.599121
LP_Jobs: 0.479297
WeightedLoad: 1.677539
```

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by CTMC.

3.14.2 rewardModel_mm1k

State rewards on an M/M/1/K queue: cost per waiting job, revenue per completion, penalty per loss.

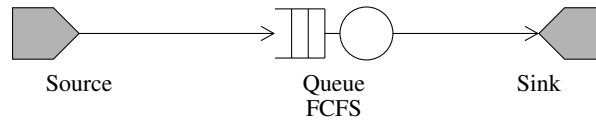


Figure 3.63: Topology of rewardModel_mm1k: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: Class1 (open)
Scheduling	Queue: FCFS
Arrivals	Source – Class1: Exp(2)
Service	Queue – Class1: Exp(3)
Features	finite buffer 3 at Queue
Solvers	CTMC

Table 3.68: Features of rewardModel_mm1k.

The example is built and solved as follows.

```
std::size_t source = 0, queue = 0, sink = 0;
Net m = reward_model("RewardExample", SchedStrategy::FCFS, 1.0, 3.0, source, queue, sink);
OpenClass oclass(m, "Class1");
m.set_arrival(source, oclass, Exp(2.0));
m.set_service(queue, oclass, Exp(3.0));
Routing P;
serial(P, oclass, {source, queue, sink});
m.link(P);

const std::size_t ist = m.station_index(queue);
const std::size_t K = 1;
m.set_reward("QueueLength", reward_qlen(ist, K, 1));
m.set_reward("Utilization", reward_util(ist, K, 1, 1.0));
m.set_reward("BlockingProb", reward_blocking(ist, K, 3.0));
m.set_reward("QueueCost", [ist, K](const std::vector<double>& s) {
    const double n = at(s, ist, K, 1);
    return n * n;
});

const Sn& sn = m.get_struct();
const ctmc::CtmcOptions opt;
std::vector<std::string> names;
const std::vector<double> R = ctmc::solver_ctmc_avg_reward(sn, opt, &names);
```

```

section("CTMC");
print_rewards(names, R);

... (20 source lines omitted; full implementation: cpp/examples/advanced/)

cost += static_cast<double>(n) * static_cast<double>(n) * pi[n];
}
compare("QueueLength", R[0], L);
compare("Utilization", R[1], 1.0 - pi[0]);
compare("BlockingProb", R[2], pi[Kbuf]);
compare("QueueCost", R[3], cost);

```

Running this edition prints

```

SOLVER: CTMC
=== Steady-State Expected Rewards ===
QueueLength: 1.015385
Utilization: 0.584615
BlockingProb: 0.123077
QueueCost: 2.123077

SOLVER: CTMC transient, timespan [0, 5]
Time points: 35
QueueLength      E[r(X(0))] = 0.000000 E[r(X(T))] = 1.014998
Utilization      E[r(X(0))] = 0.000000 E[r(X(T))] = 0.584470
BlockingProb     E[r(X(0))] = 0.000000 E[r(X(T))] = 0.122980
QueueCost        E[r(X(0))] = 0.000000 E[r(X(T))] = 2.122015

SOLVER: M/M/1/K analytical
QueueLength      LINE = 1.015385 Analytical = 1.871341 Error = 8.56e-01
Utilization      LINE = 0.584615 Analytical = 0.662768 Error = 7.82e-02
BlockingProb     LINE = 0.123077 Analytical = 0.005848 Error = 1.17e-01
QueueCost        LINE = 2.123077 Analytical = 8.070120 Error = 5.95e+00

```

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by CTMC.

3.14.3 rewardModel_multiclass

Per-class rewards on a multiclass PS station.

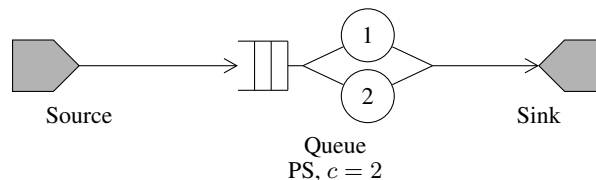


Figure 3.64: Topology of rewardModel_multiclass: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	2: Interactive (open), Batch (open)
Scheduling	Queue: PS ($c = 2$)
Arrivals	Source – Interactive: Exp(2); Batch: Exp(1.5)
Service	Queue – Interactive: Exp(0.5); Batch: Exp(1)
Features	finite buffer 6 at Queue

Table 3.69: Features of rewardModel_multiclass.

The example is built and solved as follows.

```

std::size_t source = 0, queue = 0, sink = 0;
Net m = reward_model("MultiClassRewardExample", SchedStrategy::PS, 2.0, 6.0, source, queue,
sink);

```

```

OpenClass ci(m, "Interactive");
OpenClass cb(m, "Batch");
m.set_arrival(source, ci, Exp(2.0));
m.set_arrival(source, cb, Exp(1.5));
m.set_service(queue, ci, Exp(0.5));
m.set_service(queue, cb, Exp(1.0));
Routing P;
serial(P, ci, {source, queue, sink});
serial(P, cb, {source, queue, sink});
m.link(P);

const std::size_t ist = m.station_index(queue);
const std::size_t K = 2;
m.set_reward("Interactive_QLen", reward_qlen(ist, K, 1));
m.set_reward("Interactive_Util", reward_util(ist, K, 1, 2.0));
m.set_reward("Batch_QLen", reward_qlen(ist, K, 2));
m.set_reward("Batch_Util", reward_util(ist, K, 2, 2.0));
m.set_reward("Total_QLen", reward_qlen(ist, K));
m.set_reward("Total_Util", reward_util(ist, K, 2.0));
m.set_reward("Interactive_Ratio", [ist, K](const std::vector<double>& s) {
    return at(s, ist, K, 1) / std::max(at(s, ist, K, 2), 0.001);
});
m.set_reward("Weighted_Cost", [ist, K](const std::vector<double>& s) {
    ... (20 source lines omitted; full implementation: cpp/examples/advanced/)

if (R[4] > 0) {
    kv("Interactive share (%)", 100.0 * R[0] / R[4]);
    kv("Batch share (%)", 100.0 * R[2] / R[4]);
}
kv("Interactive resp. time", R[0] / lambda_int);
kv("Batch resp. time", R[2] / lambda_batch);

```

Running this edition prints

```

SOLVER: CTMC
=== Steady-State Expected Rewards ===
Interactive_QLen: 3.955248
Interactive_Util: 1.965084
    Batch_QLen: 1.483218
    Batch_Util: 1.272385
    Total_QLen: 5.438466
    Total_Util: 1.994474
Interactive_Ratio: 948.288525
    Weighted_Cost: 13.348962
    Fairness: 0.809949

SOLVER: Analysis
Servers:                2
Total utilization:      1.25
Interactive share (%):   72.7273
Batch share (%):        27.2727
Interactive resp. time:  1.97762
Batch resp. time:       0.988812

```

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by CTMC.

3.14.4 rewardModel_templates

The reward templates that cover the common cases without writing a reward function.

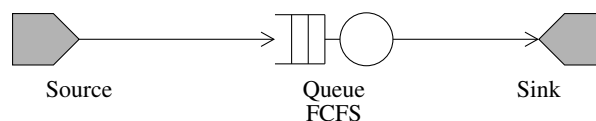


Figure 3.65: Topology of rewardModel_templates: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: Class1 (open)
Scheduling	Queue: FCFS
Arrivals	Source – Class1: Exp(1.5)
Service	Queue – Class1: Exp(2)
Features	finite buffer 10 at Queue

Table 3.70: Features of rewardModel_templates.

The example is built and solved as follows.

```
std::size_t source = 0, queue = 0, sink = 0;
Net m = reward_model("RewardTemplatesExample", SchedStrategy::FCFS, 1.0, 10.0, source, queue,
    sink);
OpenClass oclass(m, "Class1");
m.set_arrival(source, oclass, Exp(1.5));
m.set_service(queue, oclass, Exp(2.0));
Routing P;
serial(P, oclass, {source, queue, sink});
m.link(P);

const std::size_t ist = m.station_index(queue);
const std::size_t K = 1;
m.set_reward("QueueLength", reward_qlen(ist, K));
m.set_reward("QueueLength_Class1", reward_qlen(ist, K, 1));
m.set_reward("Utilization", reward_util(ist, K, 1.0));
m.set_reward("Utilization_Class1", reward_util(ist, K, 1, 1.0));
m.set_reward("BlockingProb", reward_blocking(ist, K, 10.0));

const Sn& sn = m.get_struct();
const ctmc::CtmcOptions opt;
std::vector<std::string> names;
const std::vector<double> R = ctmc::solver_ctmc_avg_reward(sn, opt, &names);

section("CTMC");
print_rewards(names, R);

... (3 source lines omitted; full implementation: cpp/examples/advanced/)

const std::vector<double> pi = mmlk_pi(rho, Kbuf);
double L = 0.0;
for (std::size_t n = 0; n <= Kbuf; ++n) L += static_cast<double>(n) * pi[n];
compare("QueueLength", R[0], L);
compare("Utilization", R[2], 1.0 - pi[0]);
compare("BlockingProb", R[4], pi[Kbuf]);
```

Running this edition prints

```
SOLVER: CTMC
=== Steady-State Expected Rewards ===
QueueLength: 2.514926
QueueLength_Class1: 2.514926
Utilization: 0.738976
Utilization_Class1: 0.738976
BlockingProb: 0.014699

SOLVER: M/M/1/K analytical
QueueLength      LINE =      2.514926  Analytical =      2.514926  Error = 3.11e-15
Utilization      LINE =      0.738976  Analytical =      0.738976  Error = 1.11e-16
BlockingProb     LINE =      0.014699  Analytical =      0.014699  Error = 1.13e-16
```

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by CTMC.

3.15 Agent models

The agent interface builds a network from the point of view of the entities that move through it rather than from the stations. The resulting models are ordinary networks – Jackson networks, closed multiclass networks, G-networks with their negative customers – and are solved by the same solvers.

3.15.1 ag_closed_network

The closed counterpart.

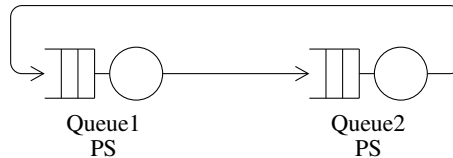


Figure 3.66: Topology of ag_closed_network: a closed network over 2 queueing stations.

Nodes	2: Queue $\times$ 2
Classes	1: Class1 (closed, $N = 10$)
Scheduling	Queue1: PS; Queue2: PS
Service	Queue1 – Class1: Exp(2) Queue2 – Class1: Exp(1)
Solvers	MVA, CTMC

Table 3.71: Features of ag_closed_network.

The example is built and solved as follows.

```
const double N = 10.0, mu1 = 2.0, mu2 = 1.0;

Net m("Closed-2Q");
Queue q1(m, "Queue1", SchedStrategy::PS);
Queue q2(m, "Queue2", SchedStrategy::PS);
ClosedClass c(m, "Class1", N, q1);
q1.set_service(c, Exp(mu1));
q2.set_service(c, Exp(mu2));
Routing P;
cyclic(P, c, {q1, q2});
m.link(P);

note("=== Closed Network ===\n");
const Sn& sn = m.get_struct();
section("AG");
run_ag_inap(sn);
section("MVA");
run_mva(sn);
// The reference guards the exact solve with N <= 20, which holds here.
section("CTMC");
run_ctmc(sn);
```

Running this edition prints

```
=== Closed Network ===

SOLVER: AG
AG (method=inap):
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
  Tput
Queue1       Class1        2.4134     0.72839   1.6567     1.6567     0.99485
1.4568
Queue2       Class1        8.1985     0.99485   8.2409     8.2409     1.4568
0.99485
```

SOLVER: MVA						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Queue1	Class1	0.99463	0.49976	0.99511	0.99511	0.99951
0.99951						
Queue2	Class1	9.0054	0.99951	9.0098	9.0098	0.99951
0.99951						
SOLVER: CTMC						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Queue1	Class1	0.99463	0.49976	0.99511	0.99511	0.99951
0.99951						
Queue2	Class1	9.0054	0.99951	9.0098	9.0098	0.99951
0.99951						

The rows repeat for each of the 2 solvers the script runs (MVA, CTMC), which is the point of the example: the same model read by methods of different cost and different exactness.

3.15.2 ag_gnetwork

A G-network, with the negative customers that remove jobs on arrival.

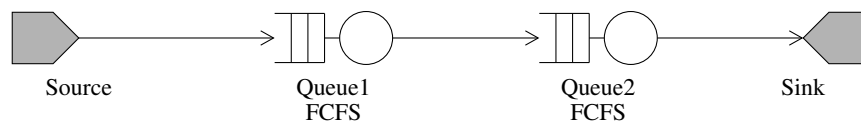


Figure 3.67: Topology of ag_gnetwork: an open network over 2 queueing stations.

Nodes	4: Source, Queue × 2, Sink
Classes	2: Positive (open), Negative (signal)
Scheduling	Queue1: FCFS; Queue2: FCFS
Arrivals	Source – Positive: Exp(1); Negative: Exp(0.3)
Service	Queue1 – Positive: Exp(2); Negative: Exp(2) Queue2 – Positive: Exp(3); Negative: Exp(3)

Table 3.72: Features of ag_gnetwork.

The example is built and solved as follows.

```
const double lam_pos = 1.0, lam_neg = 0.3, mu1 = 2.0, mu2 = 3.0;

Net m("GNetwork-Example");
Source src(m, "Source");
Queue q1(m, "Queue1", SchedStrategy::FCFS);
Queue q2(m, "Queue2", SchedStrategy::FCFS);
Sink snk(m, "Sink");

OpenClass pos(m, "Positive");
src.set_arrival(pos, Exp(lam_pos));
q1.set_service(pos, Exp(mu1));
q2.set_service(pos, Exp(mu2));

OpenClass neg(m, "Negative");
m.set_signal(neg, lang::SignalType::NEGATIVE);
src.set_arrival(neg, Exp(lam_neg));
q1.set_service(neg, Exp(mu1));
q2.set_service(neg, Exp(mu2));

Routing P;
serial(P, pos, {src, q1, q2, snk});
serial(P, neg, {src, q1, q2, snk});
m.link(P);

note("=== G-Network with Negative Customers ===\n");
```

```
section("AG");
run_ag_inap(m.get_struct());
```

Running this edition prints

```
=== G-Network with Negative Customers ===
```

```
SOLVER: AG
AG (method=inap):
```

Station	Tput	JobClass	QLen	Util	RespT	ResidT	ArvR
Source	1	Positive	0	0	0	0	0
Source	0.3	Negative	0	0	0	0	0
Queue1	0.86957	Positive	0.76923	0.43478	0.88462	0.88462	1
Queue1	0	Negative	0	0	0	0	0.3
Queue2	0.86957	Positive	0.40816	0.28986	0.46939	0.46939	0.86957

3.15.3 ag_jackson_network

A Jackson network built agent by agent.

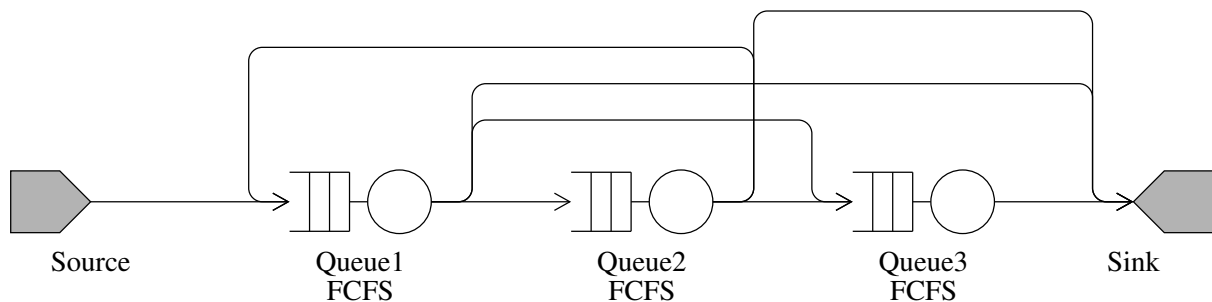


Figure 3.68: Topology of ag_jackson_network: an open network over 3 queueing stations.

Nodes	5: Source, Queue $\times$ 3, Sink
Classes	1: Class1 (open)
Scheduling	Queue1: FCFS; Queue2: FCFS; Queue3: FCFS
Arrivals	Source – Class1: Exp(1)
Service	Queue1 – Class1: Exp(2) Queue2 – Class1: Exp(3) Queue3 – Class1: Exp(2.5)
Solvers	MVA

Table 3.73: Features of ag_jackson_network.

The example is built and solved as follows.

```
const double lam = 1.0;
const double mu[3] = {2.0, 3.0, 2.5};
const double p12 = 0.4, p13 = 0.3, p1s = 0.3;
const double p21 = 0.2, p23 = 0.3, p2s = 0.5;
const double p3s = 1.0;

Net m("Jackson-3Q");
Source src(m, "Source");
Queue q1(m, "Queue1", SchedStrategy::FCFS);
Queue q2(m, "Queue2", SchedStrategy::FCFS);
Queue q3(m, "Queue3", SchedStrategy::FCFS);
Sink snk(m, "Sink");
OpenClass c(m, "Class1");
src.set_arrival(c, Exp(lam));
```



```

q1.set_service(c, Exp(mu[0]));
q2.set_service(c, Exp(mu[1]));
q3.set_service(c, Exp(mu[2]));
Routing P;
P.set(c, c, src, q1, 1.0);
P.set(c, c, q1, q2, p12);
P.set(c, c, q1, q3, p13);
P.set(c, c, q1, snk, p1s);
P.set(c, c, q2, q1, p21);
P.set(c, c, q2, q3, p23);
P.set(c, c, q2, snk, p2s);
P.set(c, c, q3, snk, p3s);
m.link(P);

note("=== Jackson Network ===\n");
const Sn& sn = m.get_struct();
section("AG");
run_ag_inap(sn);
section("MVA");
run_mva(sn);

```

Running this edition prints

```

=== Jackson Network ===

SOLVER: AG
AG (method=inap):
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
  Tput
Source      Class1          0          0          0          0          0
  1
Queue1      Class1      1.1905     0.54349    1.0953     1.1905     1.087
  1.087
Queue2      Class1      0.16949    0.14492    0.38983    0.16949    0.43479
  0.43477
Queue3      Class1      0.22342    0.18262    0.48937    0.22341    0.45653
  0.45655

SOLVER: MVA
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
  Tput
Source      Class1          0          0          0          0          0
  1
Queue1      Class1      1.1905     0.54348    1.0952     1.1905     1.087
  1.087
Queue2      Class1      0.16949    0.14493    0.38983    0.16949    0.43478
  0.43478
Queue3      Class1      0.2234     0.18261    0.48936    0.2234     0.45652
  0.45652

```

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by MVA.

3.15.4 ag_multiclass_closed

The multiclass closed version.

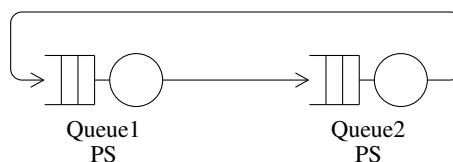


Figure 3.69: Topology of ag_multiclass_closed: a closed network over 2 queueing stations.

The example is built and solved as follows.

```

const double N1 = 5.0, N2 = 3.0;
const double mu1_q1 = 2.0, mu2_q1 = 1.5, mu1_q2 = 1.0, mu2_q2 = 0.8;

```

Nodes	2: Queue $\times$ 2
Classes	2: Class1 (closed, $N = 5$), Class2 (closed, $N = 3$)
Scheduling	Queue1: PS; Queue2: PS
Service	Queue1 – Class1: Exp(2); Class2: Exp(1.5) Queue2 – Class1: Exp(1); Class2: Exp(0.8)
Solvers	MVA

Table 3.74: Features of `ag_multiclass_closed`.

```

Net m("Multiclass-Closed");
Queue q1(m, "Queue1", SchedStrategy::PS);
Queue q2(m, "Queue2", SchedStrategy::PS);
ClosedClass c1(m, "Class1", N1, q1);
ClosedClass c2(m, "Class2", N2, q1);
q1.set_service(c1, Exp(mu1_q1));
q1.set_service(c2, Exp(mu2_q1));
q2.set_service(c1, Exp(mu1_q2));
q2.set_service(c2, Exp(mu2_q2));
Routing P;
cyclic(P, c1, {q1, q2});
cyclic(P, c2, {q1, q2});
m.link(P);

note("=== Multiclass Closed Network ===\n");
const Sn& sn = m.get_struct();
section("AG");
run_ag_inap(sn);
section("MVA");
run_mva(sn);

```

Running this edition prints

```

=== Multiclass Closed Network ===

SOLVER: AG
AG (method=inap):
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
Tput
Queue1       Class1         1.6243     0.67984    1.1946     1.1946     0.95678
1.3597
Queue1       Class2         1.1225     0.62483    1.1977     1.1977     0.70154
0.93725
Queue2       Class1         3.6418     0.95678    3.8062     3.8062     1.3597
0.95678
Queue2       Class2         1.9846     0.87692    2.8289     2.8289     0.93725
0.70154

SOLVER: MVA
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
Tput
Queue1       Class1         0.63167    0.31287    1.0095     1.0095     0.62574
0.62574
Queue1       Class2         0.39711    0.19837    1.3346     1.3346     0.29755
0.29755
Queue2       Class1         4.3683     0.62574    6.981      6.981      0.62574
0.62574
Queue2       Class2         2.6029     0.37194    8.7476     8.7476     0.29755
0.29755

```

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by MVA.

3.15.5 `ag_tandem_open`

An open tandem written through the agent interface.

The example is built and solved as follows.

```
const double lam = 0.5, s1 = 1.0, s2 = 1.5;
```

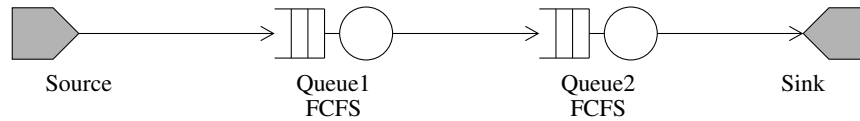


Figure 3.70: Topology of ag_tandem_open: an open network over 2 queueing stations.

Nodes	4: Source, Queue $\times$ 2, Sink
Classes	1: Class1 (open)
Scheduling	Queue1: FCFS; Queue2: FCFS
Arrivals	Source – Class1: Exp(0.5)
Service	Queue1 – Class1: Exp(1) Queue2 – Class1: Exp(1.5)
Solvers	MVA

Table 3.75: Features of ag_tandem_open.

```

Net m("Tandem-MM1");
Source src(m, "Source");
Queue q1(m, "Queue1", SchedStrategy::FCFS);
Queue q2(m, "Queue2", SchedStrategy::FCFS);
Sink snk(m, "Sink");
OpenClass c(m, "Class1");
src.set_arrival(c, Exp(lam));
q1.set_service(c, Exp(s1));
q2.set_service(c, Exp(s2));
Routing P;
serial(P, c, {src, q1, q2, snk});
m.link(P);

const double U1 = lam / s1, U2 = lam / s2;
note("=== Open Tandem Queue (M/M/1 -> M/M/1) ===\n");
note("Analytical (M/M/1):");
std::printf(" Queue1: U=%.4f, Q=%.4f, R=%.4f\n", U1, U1 / (1.0 - U1), 1.0 / (s1 - lam));
std::printf(" Queue2: U=%.4f, Q=%.4f, R=%.4f\n\n", U2, U2 / (1.0 - U2), 1.0 / (s2 - lam));

const Sn& sn = m.get_struct();
section("AG");
run_ag_inap(sn);
// NOT a gap in this port: the reference never calls 'exact', it only notes
// why, so there is no call to record and nothing to refuse. The C++ side
// declines it for the same reason (solver_ag_autocat.h needs an LP/NLP
// solver), which is the reference's situation and not a divergence from it.
note("Note: 'exact' method (AutoCAT) is not yet implemented in native Python");
note("It falls back to INAP with a warning in the JAR version");
section("MVA");
run_mva(sn);

```

Running this edition prints

```

=== Open Tandem Queue (M/M/1 -> M/M/1) ===

Analytical (M/M/1):
Queue1: U=0.5000, Q=1.0000, R=2.0000
Queue2: U=0.3333, Q=0.5000, R=1.0000

SOLVER: AG
AG (method=inap):
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
-----
Source       Class1         0          0          0          0          0
0.5
Queue1       Class1         1          0.5        2          2          0.5
0.5
Queue2       Class1         0.5        0.33333    1          1          0.5
0.5
Note: 'exact' method (AutoCAT) is not yet implemented in native Python
It falls back to INAP with a warning in the JAR version

```

SOLVER: MVA						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Source	Class1	0	0	0	0	0
0.5						
Queue1	Class1	1	0.5	2	2	0.5
0.5						
Queue2	Class1	0.5	0.33333	1	1	0.5
0.5						

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by MVA.

3.15.6 ag_tandem_phasetype

An open tandem whose two queues share a mean service time and differ only in variability (Erlang-2 against a hyperexponential), solved by the RCAT methods, which carry the phase-type law exactly where an M/M/1 reading sees no difference.

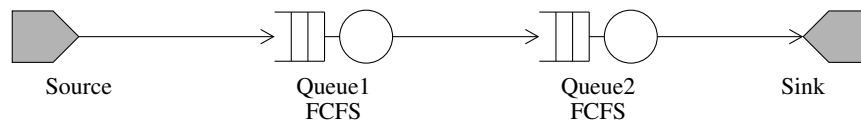


Figure 3.71: Topology of ag_tandem_phasetype: an open network over 2 queueing stations.

Nodes	4: Source, Queue $\times$ 2, Sink
Classes	1: Class1 (open)
Scheduling	Queue1: FCFS; Queue2: FCFS
Arrivals	Source – Class1: Exp(0.5)
Service	Queue1 – Class1: Erlang(2,2) Queue2 – Class1: HyperExp(0.99,0.01;1.14,0.07584)

Table 3.76: Features of ag_tandem_phasetype.

The example is built and solved as follows.

```
const double lam = 0.5, mean_s = 1.0, scv1 = 0.5, scv2 = 4.0;

Net m("Tandem-MPH1");
Source src(m, "Source");
Queue q1(m, "Queue1", SchedStrategy::FCFS);
Queue q2(m, "Queue2", SchedStrategy::FCFS);
Sink snk(m, "Sink");
OpenClass c(m, "Class1");
src.set_arrival(c, Exp(lam));
q1.set_service(c, D::erlang_fit(mean_s, scv1));
q2.set_service(c, lang::hyperexp_fit_mean_scv<double>(mean_s, scv2));
Routing P;
serial(P, c, {src, q1, q2, snk});
m.link(P);

const double rho = lam * mean_s;
const double pk1 = rho + rho * rho * (1.0 + scv1) / (2.0 * (1.0 - rho));
note("=== Open Tandem with Phase-Type Service ===\n");
note("Queue1 is an isolated M/Er2/1, so its exact mean queue length is the");
std::printf("Pollaczek-Khinchine value %.6f. The M/M/1 reading would be %.6f.\n\n",
    pk1, rho / (1.0 - rho));

const Sn& sn = m.get_struct();
section("AG");
run_ag_inap(sn);
// 'inapinf' additionally drops the maxStates truncation, solving each open
// component on its infinite state space through Neuts' rate matrix R. That
```

```
// matters most at Queue2, whose service law has the heavier tail.
section("AG inapinf");
ag::AgOptions oinf;
oinf.method = "inapinf";
print_avg(sn, ag::solver_ag_run_analyzer(sn, oinf));
```

Running this edition prints

```
=== Open Tandem with Phase-Type Service ===

Queue1 is an isolated M/Er2/1, so its exact mean queue length is the
Pollaczek-Khinchine value 0.875000. The M/M/1 reading would be 1.000000.

SOLVER: AG
AG (method=inap):
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
  Tput
Source        Class1          0          0          0          0          0
  0.5
Queue1        Class1      0.875      0.5        1.75       1.75       0.5
  0.5
Queue2        Class1      1.75       0.5        3.4999     3.4999     0.5
  0.5

SOLVER: AG inapinf
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
  Tput
Source        Class1          0          0          0          0          0
  0.5
Queue1        Class1      0.875      0.5        1.75       1.75       0.5
  0.5
Queue2        Class1      1.75       0.5        3.5        3.5        0.5
  0.5
```

3.16 Large-scale models

When the state space cannot be enumerated, the exact solvers are out and the question becomes how much accuracy the approximations retain. The script here is a large closed PS network solved by several approximate methods.

3.16.1 largescale_tbi

A large closed PS network, and the accuracy the approximate solvers retain on it.

No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

The example is built and solved as follows.

```
Net m("tbi_largescale");
std::mt19937 rng(1);
std::uniform_real_distribution<double> u01(0.0, 1.0);

std::vector<std::size_t> Q;
for (std::size_t i = 0; i < kM; ++i)
    Q.push_back(m.add_queue("Q" + std::to_string(i + 1), SchedStrategy::PS));
std::vector<std::vector<std::size_t> > D1(kM, std::vector<std::size_t>(kM, 0));
for (std::size_t i = 0; i < kM; ++i)
    for (std::size_t j = 0; j < kM; ++j)
        if (i != j)
            D1[i][j] = m.add_delay("D" + std::to_string(i + 1) + "_" + std::to_string(j + 1))
;

const std::size_t job = m.add_closed_class("C1", kN, Q[0]);
Matrix<double> lambda(kM, kM, 0.0);
for (std::size_t i = 0; i < kM; ++i) {
    m.set_service(Q[i], job,
```

```

        lang::erlang_fit_mean_order<double>(1.0 / (1.0 + 3.0 * u01(rng)), 4));
    for (std::size_t j = 0; j < kM; ++j)
        if (i != j) {
            m.set_service(Dl[i][j], job,
                lang::erlang_fit_mean_order<double>(0.2 + 2.0 * u01(rng), kKph));
            lambda(i, j) = u01(rng);
        }
}

... (35 source lines omitted; full implementation: cpp/examples/advanced/)

// rows: the transit delays carry the rest of the population and are not
// what the paper's tables report.
std::printf("%-10s %12s %12s %12s\n", "Station", "QLen", "Util", "Tput");
for (std::size_t i = 0; i < kM; ++i)
    std::printf("%-10s %12.5g %12.5g %12.5g\n", sn.stations[i].name.c_str(), sol.QN(i, 0),
        sol.UN(i, 0), sol.TN(i, 0));

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.17 Custom solvers

A solver is a class with an analyzer and a feature set. The feature set is a claim about what the solver can represent, and the dispatcher uses it to decide whether a model may be given to it; the analyzer computes the metrics. These two scripts show the smallest pair that the dispatcher will accept.

3.17.1 solver_custom

A user-defined solver registered with the dispatcher.

No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

The example is built and solved as follows.

```

const std::size_t M = sn.nstations;           // number of stations
const std::size_t K = sn.nclasses;           // number of classes
const std::vector<double> N = sn.njobs();    // job populations
const Matrix<double>& rates = sn.rates;       // arrival and service rates
const std::vector<Matrix<double>>& V = sn.visits; // visits, per chain
(void)N;
(void)rates;
(void)V;

CustomResult r;
r.QN = Matrix<double>(M, K);
r.UN = Matrix<double>(M, K);
r.RN = Matrix<double>(M, K);
r.TN = Matrix<double>(M, K);
r.CN.assign(K, 0.0);
r.XN.assign(K, 0.0);

note("The solution algorithm needs to be implemented in solver_custom: "
    "returning with no result.");
return r;

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

3.17.2 solver_custom_analyzer

Its analyzer, the function that actually computes the metrics.

No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

The example is built and solved as follows.

```
const std::clock_t t0 = std::clock();
note("Any activity prior or after launching the solution algorithm needs to be "
     "implemented in solver_custom_analyzer.");
CustomResult r = solver_custom(sn);

Matrix<double>* mats[4] = {&r.QN, &r.UN, &r.RN, &r.TN};
for (int k = 0; k < 4; ++k)
    for (std::size_t i = 0; i < mats[k]->rows(); ++i)
        for (std::size_t j = 0; j < mats[k]->cols(); ++j)
            if (std::isnan((*mats[k])(i, j))) (*mats[k])(i, j) = 0.0;
std::vector<double>* vecs[2] = {&r.CN, &r.XN};
for (int k = 0; k < 2; ++k)
    for (std::size_t j = 0; j < vecs[k]->size(); ++j)
        if (std::isnan((*vecs[k])[j])) (*vecs[k])[j] = 0.0;

r.runtime = double(std::clock() - t0) / double(CLOCKS_PER_SEC);
return r;
```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

Chapter 4

Model gallery

The gallery is a catalogue of small named models, one function each, returning a model rather than solving it. It exists twice over: as documentation, because a name in Kendall notation is the fastest way to find the model one means, and as the cross-codebase parity suite, because the same 71 functions exist in all four codebases and their metrics are compared numerically.

The models live in `cpp/examples/gallery/`, one factory function per model, registered by name. Calling a gallery function builds and returns the model without solving it, so the caller chooses the solver. Running a Python gallery file as a script prints a short confirmation of the model it built.

```
Net gallery_mml() {
    return one_queue("M/M/1", "mySource", "myQueue", "mySink", SchedStrategy::FCFS,
        D::exp_rate(1.0), D::exp_rate(2.0));
}
```

4.1 Reading the names

The single-station entries are named in Kendall notation, with the distributions abbreviated: `m` exponential, `d` deterministic, `erl` Erlang, `hyp` hyperexponential, `aph` acyclic phase-type, `cox` Coxian, `me` matrix-exponential, `map` Markovian arrival process, `mmap` marked MAP, `gam` Gamma, `par` Pareto, `u` uniform. So `gallery_erlm1` is an Erl/M/1 queue and `gallery_mhypk` an M/HyperExp/k. A trailing `_ps` replaces FCFS by processor sharing, `k` in place of `1` makes the station multiserver, and the suffixes `_tandem`, `_linear`, `_reentrant`, `_feedback`, `_multiclass` and `_prio` describe the topology or the workload around the station.

The remaining entries are named after what they model rather than after a queueing notation: `gallery_cqn` and `gallery_repairmen` for the closed archetypes, `gallery_fj_open` and `gallery_fj_closed` for fork-join, `gallery_fcr` for a finite capacity region, `gallery_cache_lru` and `gallery_cache_routing` for caches, `gallery_renv_breakdown` for a random environment, `gallery_lqn_basic`, `gallery_lqn_workflows` and `gallery_lqn_random` for layered models, `gallery_multitier` and `gallery_multitier_storage` for multitier applications, `gallery_lukumar_reentrant` for the Lu-Kumar re-entrant line, and `gallery_qn_random` for a generated network.

4.2 The catalogue

The complete list, in the order in which the models are registered:

```
gallery_aphm1                                gallery_merl1_linear
```



```

gallery_cache_lru
gallery_cache_routing
gallery_coxml
gallery_cqn
gallery_cqn_multiclass
gallery_detml
gallery_dml
gallery_erldk
gallery_erlerl1
gallery_erlerl1_reentrant
gallery_erlml
gallery_erlml_ps
gallery_erlml_reentrant
gallery_erlmlps
gallery_fcr
gallery_fj_closed
gallery_fj_open
gallery_fj_quorum
gallery_gamml
gallery_hyperl1_feedback
gallery_hyperl1_reentrant
gallery_hyperlk
gallery_hyphyp1_linear
gallery_hyphyp1_reentrant
gallery_hyphyp1_tandem
gallery_hypml
gallery_hypml_reentrant
gallery_lqn_basic
gallery_lqn_random
gallery_lqn_workflows
gallery_lukumar_reentrant
gallery_mapml
gallery_mapmk
gallery_mdk
gallery_merl1
gallery_merl1_reentrant
gallery_merl1_tandem
gallery_merlk
gallery_mhyp1
gallery_mhyp1_linear
gallery_mhyp1_reentrant
gallery_mhyp1_tandem
gallery_mhypk
gallery_mml
gallery_mml_feedback
gallery_mml_linear
gallery_mml_multiclass
gallery_mml_prio
gallery_mml_ps
gallery_mml_ps_feedback
gallery_mml_ps_multiclass
gallery_mml_ps_reentrant
gallery_mml_reentrant
gallery_mml_tandem
gallery_mml_tandem_multiclass
gallery_mmlk
gallery_mmap1
gallery_mmap1_multiclass
gallery_mmapk
gallery_mmk
gallery_mpar1
gallery_multitier
gallery_multitier_storage
gallery_parm1
gallery_qn_random
gallery_renv_breakdown
gallery_repairmen
gallery_replayerml
gallery_um1

```

4.2.1 gallery_aphm1

The APH/M/1 queue: acyclic phase-type interarrival times, exponential service, and a single server.

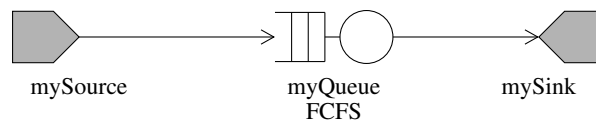


Figure 4.1: Topology of gallery_aphm1: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: myClass (open)
Scheduling	myQueue: FCFS
Arrivals	mySource – myClass: APH
Service	myQueue – myClass: Exp(2)

Table 4.2: Features of gallery_aphm1.

The example is built and solved as follows.

```

Net gallery_aphm1() {
    return one_queue("APH/M/1", "mySource", "myQueue", "mySink", SchedStrategy::FCFS,

```

```
lang::aph_fit_central<double>(1.0, 0.99, 1.999), Exp(2.0));
```

Running this edition prints

```
Model: APH/M/1
```

4.2.2 gallery_cache_lru

A cache node under least-recently-used replacement, read by a closed class.

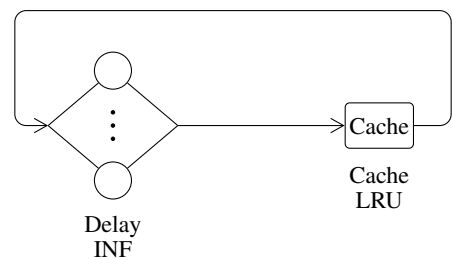


Figure 4.2: Topology of gallery_cache_lru: a closed network over 1 queueing station with a cache node.

Nodes	2: Delay, Cache
Classes	3: JobClass (closed, $N = 1$), HitClass (closed, $N = 0$), MissClass (closed, $N = 0$)
Scheduling	Delay: INF
Service	Delay – JobClass: Exp(1); HitClass: Disabled; MissClass: Disabled
Features	drop rule {'HitClass': 'waitingQueue', 'MissClass': 'waitingQueue'} at Delay; cache: LRU replacement, 5 items, capacity 2; class switching on 2 links

Table 4.3: Features of gallery_cache_lru.

The example is built and solved as follows.

```
const std::size_t n = 5, mslots = 2;
Net m("Cache-LRU");
Delay delay(m, "Delay");
qn::CacheParam<double> ch;
ch.nitems = n;
ch.itemcap = std::vector<int>{static_cast<int>(mslots)};
ch.replacestrat = lang::ReplacementStrategy::LRU;
// JobClass reads uniformly; the hit and miss classes never read.
ch.pread = std::vector<std::vector<double>>{uniform_pread(n), {}, {}};
ch.hitclass = std::vector<std::size_t>{2, 0, 0};
ch.missclass = std::vector<std::size_t>{3, 0, 0};
const std::size_t cache = m.add_cache("Cache", ch);

ClosedClass job(m, "JobClass", 1, delay, 0);
ClosedClass hit(m, "HitClass", 0, delay, 0);
ClosedClass miss(m, "MissClass", 0, delay, 0);
delay.set_service(job, Exp(1.0));

Routing P;
P.set(job, job, delay, cache, 1.0);
P.set(hit, job, cache, delay, 1.0);
P.set(miss, job, cache, delay, 1.0);
m.link(P);
return m;
```

Running this edition prints

```
Model: Cache-LRU
```

4.2.3 gallery_cache_routing

A cache whose hit and miss outcomes route the job to different downstream stations.

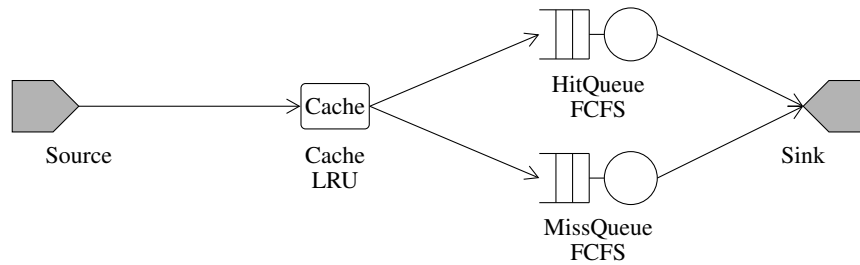


Figure 4.3: Topology of gallery_cache_routing: an open network over 2 queueing stations with a cache node.

Nodes	5: Source, Queue × 2, Cache, Sink
Classes	3: InitClass (open), HitClass (open), MissClass (open)
Scheduling	HitQueue: FCFS; MissQueue: FCFS
Arrivals	Source – InitClass: Exp(1); HitClass: Disabled; MissClass: Disabled
Service	HitQueue – InitClass: Disabled; HitClass: Exp(2); MissClass: Disabled MissQueue – InitClass: Disabled; HitClass: Disabled; MissClass: Exp(1)
Features	cache: LRU replacement, 4 items, capacity 2; drop rule {'InitClass': 'waitingQueue', 'MissClass': 'waitingQueue'} at HitQueue; drop rule {'InitClass': 'waitingQueue', 'HitClass': 'waitingQueue'} at MissQueue; class switching on 4 links

Table 4.4: Features of gallery_cache_routing.

The example is built and solved as follows.

```

const std::size_t n = 4, mslots = 2;
Net m("Cache-Routing");
Source source(m, "Source");
qn::CacheParam<double> ch;
ch.nitems = n;
ch.itemcap = std::vector<int>{static_cast<int>(mslots)};
ch.replacestrat = lang::ReplacementStrategy::LRU;
ch.pread = std::vector<std::vector<double>>{uniform_pread(n), {}, {}};
ch.hitclass = std::vector<std::size_t>{2, 0, 0};
ch.missclass = std::vector<std::size_t>{3, 0, 0};
const std::size_t cache = m.add_cache("Cache", ch);
Queue hq(m, "HitQueue", SchedStrategy::FCFS);
Queue mq(m, "MissQueue", SchedStrategy::FCFS);
Sink sink(m, "Sink");

OpenClass init(m, "InitClass", 0);
OpenClass hit(m, "HitClass", 0);
OpenClass miss(m, "MissClass", 0);
source.set_arrival(init, Exp(1.0));
hq.set_service(hit, Exp(2.0));
mq.set_service(miss, Exp(1.0));

Routing P;
P.set(init, init, source, cache, 1.0);
P.set(hit, hit, cache, hq, 1.0);
P.set(hit, hit, hq, sink, 1.0);
P.set(miss, miss, cache, mq, 1.0);
P.set(miss, miss, mq, sink, 1.0);
m.link(P);
return m;

```

Running this edition prints

Model: Cache-Routing

4.2.4 gallery_coxm1

The Coxian/M/1 queue: Coxian interarrival times, exponential service, and a single server.

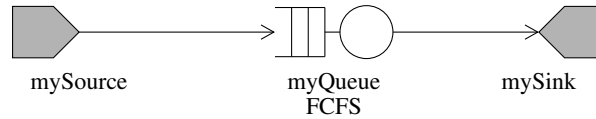


Figure 4.4: Topology of gallery_coxm1: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: myClass (open)
Scheduling	myQueue: FCFS
Arrivals	mySource – myClass: Coxian
Service	myQueue – myClass: Exp(2)

Table 4.5: Features of gallery_coxm1.

The example is built and solved as follows.

```

Net gallery_coxm1() {
    return one_queue("Cox/M/1", "mySource", "myQueue", "mySink", SchedStrategy::FCFS,
        lang::coxian_fit_central<double>(1.0, 0.99, 1.999), Exp(2.0));
}
  
```

Running this edition prints

Model: Cox/M/1

4.2.5 gallery_cqn

The closed queueing network archetype: a fixed job population circulating between a Delay station and a queue.

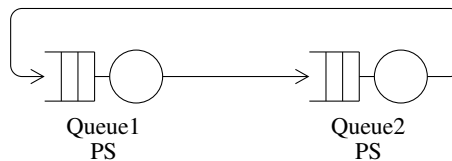


Figure 4.5: Topology of gallery_cqn: a closed network over 2 queueing stations.

The example is built and solved as follows.

```

PyRandom rng(seed);
Net m("Single-class CQN");
std::vector<std::size_t> station;
for (std::size_t i = 0; i < M; ++i)
    station.push_back(m.add_queue("Queue" + std::to_string(i + 1), SchedStrategy::PS));
if (use_delay) station.push_back(m.add_delay("Delay1"));

const double u_pop = rng.random();
ClosedClass cl(m, "Class1", py_round(u_pop * static_cast<double>(M) + 2.0), station[0], 0);
for (std::size_t i = 0; i < M; ++i) {
    const double u = rng.random();
    m.set_service(station[i], cl, D::exp_mean(u + static_cast<double>(i) + 1.0));
}
  
```

Nodes	2: Queue $\times$ 2
Classes	1: Class1 (closed, $N = 3$)
Scheduling	Queue1: PS; Queue2: PS
Service	Queue1 – Class1: Exp(0.8271) Queue2 – Class1: Exp(0.4381)

Table 4.6: Features of gallery_cqn.

```

}
if (use_delay) m.set_service(station.back(), c1, D::exp_mean(2.0));

Routing P;
cyclic(P, c1, station);
m.link(P);
return m;

```

Running this edition prints

```
Model: Single-class CQN
```

4.2.6 gallery_cqn_multiclass

The closed archetype with several classes, each with its own population.

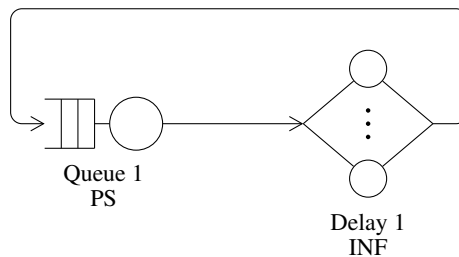


Figure 4.6: Topology of gallery_cqn_multiclass: a closed network over 2 queueing stations.

Nodes	2: Queue, Delay
Classes	2: Class1 (closed, $N = 5$), Class2 (closed, $N = 5$)
Scheduling	Queue 1: PS; Delay 1: INF
Service	Queue 1 – Class1: Exp(0.03226); Class2: Exp(0.07143) Delay 1 – Class1: Exp(0.04762); Class2: Exp(0.08333)

Table 4.7: Features of gallery_cqn_multiclass.

The example is built and solved as follows.

```

unsigned long long seed) {
PyRandom rng(seed);
Net m("Multi-class CQN");
std::vector<std::size_t> node;
for (std::size_t i = 0; i < m_queues; ++i)
    node.push_back(m.add_queue("Queue " + std::to_string(i + 1), SchedStrategy::PS));
if (wantdelay) node.push_back(m.add_delay("Delay 1"));

std::vector<std::size_t> jobclass;
for (std::size_t s = 0; s < r; ++s)
    jobclass.push_back(m.add_closed_class("Class" + std::to_string(s + 1), 5, node[0], 0));

for (std::size_t s = 0; s < r; ++s) {
    for (std::size_t i = 0; i < m_queues; ++i) {
        const double u = rng.random();

```

```
        m.set_service(node[i], jobclass[s], D::exp_mean(py_round(50.0 * u)));
    }
    if (wantdelay) {
        const double u = rng.random();
        m.set_service(node.back(), jobclass[s], D::exp_mean(py_round(100.0 * u)));
    }
}

Routing P;
for (std::size_t s = 0; s < r; ++s) cyclic(P, jobclass[s], node);
m.link(P);
return m;
```

Running this edition prints

```
Model: Multi-class CQN
```

4.2.7 gallery_detm1

The D/M/1 queue: deterministic interarrival times, exponential service, and a single server.

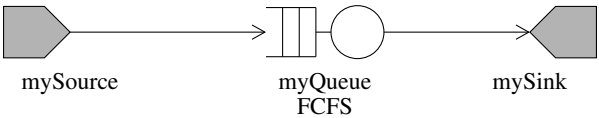


Figure 4.7: Topology of gallery_detm1: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: myClass (open)
Scheduling	myQueue: FCFS
Arrivals	mySource – myClass: Det(1)
Service	myQueue – myClass: Exp(2)

Table 4.8: Features of gallery_detm1.

The example is built and solved as follows.

```
Net gallery_detm1() {
    return one_queue("D/M/1", "mySource", "myQueue", "mySink", SchedStrategy::FCFS, Det(1.0),
        Exp(2.0));
}
```

Running this edition prints

```
Model: D/M/1
```

4.2.8 gallery_dm1

The D/M/1 queue: deterministic interarrival times, exponential service, and a single server.

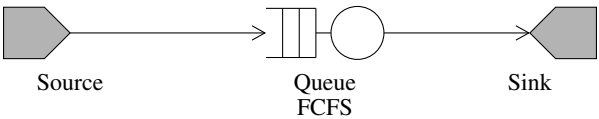


Figure 4.8: Topology of gallery_dm1: an open network over 1 queueing station.

The example is built and solved as follows.

Nodes	3: Source, Queue, Sink
Classes	1: Class1 (open)
Scheduling	Queue: FCFS
Arrivals	Source – Class1: Det(1)
Service	Queue – Class1: Exp(2)

Table 4.9: Features of gallery_dm1.

```
Net gallery_dm1() {
    Net m("D/M/1");
    Source source(m, "Source");
    Queue queue(m, "Queue", SchedStrategy::FCFS);
    Sink sink(m, "Sink");
    OpenClass c1(m, "Class1");
    source.set_arrival(c1, Det(1.0));
    queue.set_service(c1, Exp(2.0));
    Routing P;
    serial(P, {source, queue, sink});
    m.link(P);
    return m;
}
```

Running this edition prints

```
Model: D/M/1
```

4.2.9 gallery_erldk

The Erl/D/k queue: Erlang interarrival times, deterministic service, and *k* parallel servers.

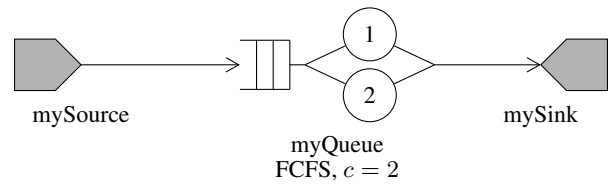


Figure 4.9: Topology of gallery_erldk: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: myClass (open)
Scheduling	myQueue: FCFS (<i>c</i> = 2)
Arrivals	mySource – myClass: Erlang(5,5)
Service	myQueue – myClass: Det(1)

Table 4.10: Features of gallery_erldk.

The example is built and solved as follows.

```
Net gallery_erldk(double k) {
    return one_queue("Erl/D/k", "mySource", "myQueue", "mySink", SchedStrategy::FCFS,
        lang::erlang_fit_mean_order<double>(1.0, 5), Det(2.0 / k), k);
}
```

Running this edition prints

```
Model: Erl/D/k
```

4.2.10 gallery_erlerl1

The Erl/Erl/1 queue: Erlang interarrival times, Erlang service, and a single server.

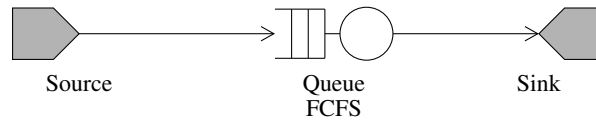


Figure 4.10: Topology of gallery_erlerl1: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	2: Class1 (open), Class2 (open)
Scheduling	Queue: FCFS
Arrivals	Source – Class1: Erlang(5,5); Class2: Disabled
Service	Queue – Class1: Erlang(10,5); Class2: Exp(3)
Features	class switching on 2 links

Table 4.11: Features of gallery_erlerl1.

The example is built and solved as follows.

```
Net m("Erl/Erl/1");
Source source(m, "Source");
Queue queue(m, "Queue", SchedStrategy::FCFS);
Sink sink(m, "Sink");
OpenClass c1(m, "Class1");
OpenClass c2(m, "Class2");
source.set_arrival(c1, lang::erlang_fit_mean_order<double>(1.0, n));
source.set_arrival(c2, Disabled());
queue.set_service(c1, lang::erlang_fit_mean_order<double>(0.5, n));
queue.set_service(c2, Exp(3.0));
Routing P;
P.set(c1, c1, source, queue, 1.0);
P.set(c2, c2, queue, sink, 1.0);
m.link(P);
return m;
```

Running this edition prints

```
Model: Erl/Erl/1
```

4.2.11 gallery_erlerl1_reentrant

An Erl/Erl/1 queue with re-entrant visits.

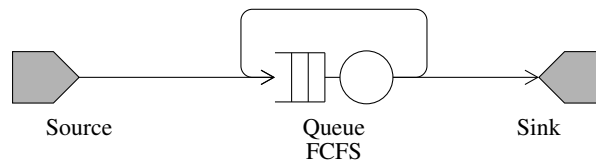


Figure 4.11: Topology of gallery_erlerl1_reentrant: an open network over 1 queueing station.

The example is built and solved as follows.

```
const std::size_t n = 5;
Net m("Erl/Erl/1-Reentrant");
Source source(m, "Source");
```


Nodes	3: Source, Queue, Sink
Classes	2: Class1 (open), Class2 (open)
Scheduling	Queue: FCFS
Arrivals	Source – Class1: Erlang(5,5); Class2: Disabled
Service	Queue – Class1: Erlang(50,5); Class2: Exp(10)
Features	class switching on 1 link

Table 4.12: Features of gallery_erlerl1_reentrant.

```

Queue queue(m, "Queue", SchedStrategy::FCFS);
Sink sink(m, "Sink");
OpenClass c1(m, "Class1");
OpenClass c2(m, "Class2");
source.set_arrival(c1, lang::erlang_fit_mean_order<double>(1.0, n));
source.set_arrival(c2, Disabled());
queue.set_service(c1, lang::erlang_fit_mean_order<double>(0.1, n));
queue.set_service(c2, Exp(10.0));
Routing P;
P.set(c1, c1, source, queue, 1.0);
P.set(c1, c2, queue, queue, 0.5);
P.set(c2, c2, queue, sink, 0.5);
m.link(P);
return m;

```

Running this edition prints

```
Model: Erl/Erl/1-Reentrant
```

4.2.12 gallery_erlm1

The Erl/M/1 queue: Erlang interarrival times, exponential service, and a single server.

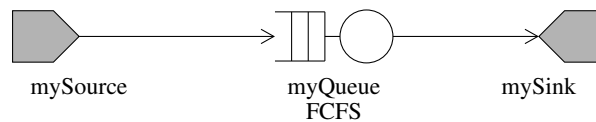


Figure 4.12: Topology of gallery_erlm1: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: myClass (open)
Scheduling	myQueue: FCFS
Arrivals	mySource – myClass: Erlang(5,5)
Service	myQueue – myClass: Exp(2)

Table 4.13: Features of gallery_erlm1.

The example is built and solved as follows.

```

Net gallery_erlm1() {
    return one_queue("Er/M/1", "mySource", "myQueue", "mySink", SchedStrategy::FCFS,
        lang::erlang_fit_mean_order<double>(1.0, 5), Exp(2.0));
}

```

Running this edition prints

```
Model: Er/M/1
```

4.2.13 gallery_erlm1_ps

The Erl/M/1 queue: Erlang interarrival times, exponential service, and a single server. In this variant, the station serves under processor sharing rather than FCFS.

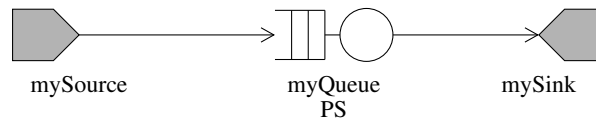


Figure 4.13: Topology of gallery_erlm1_ps: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: myClass (open)
Scheduling	myQueue: PS
Arrivals	mySource – myClass: Erlang(5,5)
Service	myQueue – myClass: Exp(2)

Table 4.14: Features of gallery_erlm1_ps.

The example is built and solved as follows.

```

Net gallery_erlm1_ps() {
    return one_queue("Er/M/1-PS", "mySource", "myQueue", "mySink", SchedStrategy::PS,
        lang::erlang_fit_mean_order<double>(1.0, 5), Exp(2.0));
}
  
```

Running this edition prints

```
Model: Er/M/1-PS
```

4.2.14 gallery_erlm1_reentrant

The Erl/M/1 queue: Erlang interarrival times, exponential service, and a single server. In this variant, jobs re-enter the station, so a visit is counted more than once.

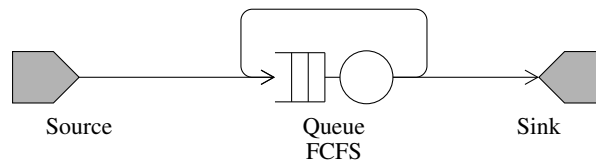


Figure 4.14: Topology of gallery_erlm1_reentrant: an open network over 1 queueing station.

The example is built and solved as follows.

```

Net gallery_erlm1_reentrant() {
    return reentrant("Er/M/1-Reentrant", SchedStrategy::FCFS,
        lang::erlang_fit_mean_order<double>(1.0, 5), Exp(2.0),
        Exp(3.0));
}
  
```

Running this edition prints

```
Model: Er/M/1-Reentrant
```

4.2.15 gallery_erlm1ps

The Erl/M/1 queue: Erlang interarrival times, exponential service, and a single server. In this variant, the station serves under processor sharing rather than FCFS.

Nodes	3: Source, Queue, Sink
Classes	2: Class1 (open), Class2 (open)
Scheduling	Queue: FCFS
Arrivals	Source – Class1: Erlang(5,5); Class2: Disabled
Service	Queue – Class1: Exp(2); Class2: Exp(3)
Features	class switching on 1 link

Table 4.15: Features of gallery_erlml_reentrant.

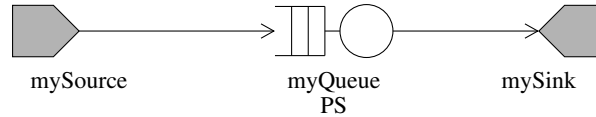


Figure 4.15: Topology of gallery_erlmlps: an open network over 1 queueing station.

The example is built and solved as follows.

```
Net gallery_erlmlps() { return gallery_erlml_ps(); }
```

Running this edition prints

```
Model: Er/M/1-PS
```

4.2.16 gallery_fcr

A finite capacity region, which caps the number of jobs admitted to a set of stations irrespective of their individual buffers.

The example is built and solved as follows.

```
Net m("FCR-Dropping");
Source source(m, "Source");
Queue queue(m, "Queue", SchedStrategy::FCFS);
Sink sink(m, "Sink");
OpenClass c1(m, "Class1", 0);
source.set_arrival(c1, Exp(0.8));
queue.set_service(c1, Exp(1.0));

Routing P;
P.set(c1, c1, source, queue, 1.0);
P.set(c1, c1, queue, sink, 1.0);
m.link(P);

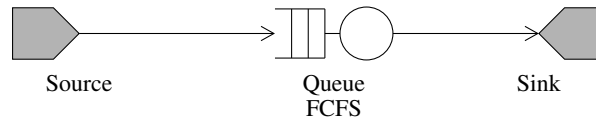
m.add_region({queue}, std::vector<double>{-1.0}, K,
             std::vector<lang::DropStrategy>{lang::DropStrategy::DROP});
return m;
```

Running this edition prints

```
Model: FCR-Dropping
```

Nodes	3: Source, Queue, Sink
Classes	1: myClass (open)
Scheduling	myQueue: PS
Arrivals	mySource – myClass: Erlang(5,5)
Service	myQueue – myClass: Exp(2)

Table 4.16: Features of gallery_erlmlps.

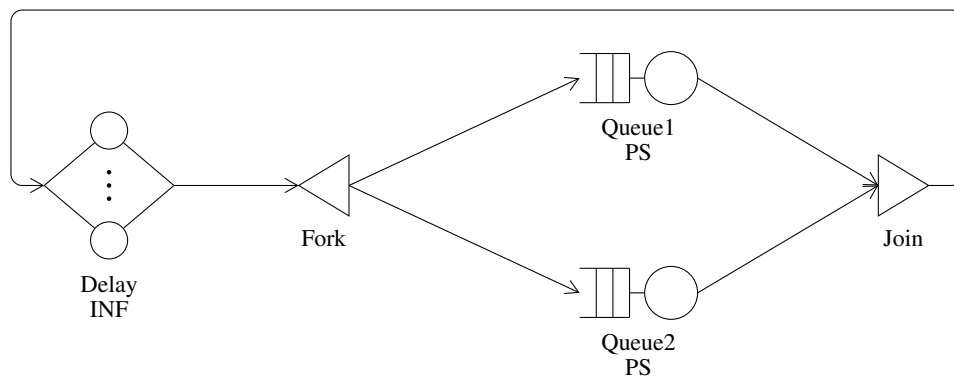
Figure 4.16: Topology of `gallery_fcr`: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: Class1 (open)
Scheduling	Queue: FCFS
Arrivals	Source – Class1: Exp(0.8)
Service	Queue – Class1: Exp(1)

Table 4.17: Features of `gallery_fcr`.

4.2.17 `gallery_fj_closed`

The closed fork-join counterpart, with a fixed population circulating through the block.

Figure 4.17: Topology of `gallery_fj_closed`: a closed network over 3 queueing stations with a fork-join block.

The example is built and solved as follows.

```
Net m("Fork-Join-Closed");
Delay delay(m, "Delay");
Queue q1(m, "Queue1", SchedStrategy::PS);
Queue q2(m, "Queue2", SchedStrategy::PS);
const std::size_t fork = m.add_fork("Fork");
const std::size_t join = m.add_join("Join", fork);

ClosedClass c1(m, "class1", 5, delay, 0);
delay.set_service(c1, Exp(1.0));
q1.set_service(c1, Exp(1.0));
q2.set_service(c1, Exp(1.0));

Routing P;
P.set(c1, c1, delay, fork, 1.0);
P.set(c1, c1, fork, q1, 1.0);
P.set(c1, c1, fork, q2, 1.0);
P.set(c1, c1, q1, join, 1.0);
P.set(c1, c1, q2, join, 1.0);
P.set(c1, c1, join, delay, 1.0);
m.link(P);
return m;
```

Running this edition prints

```
Model: Fork-Join-Closed
```

Nodes	5: Queue $\times$ 2, Delay, Fork, Join
Classes	1: class1 (closed, $N = 5$)
Scheduling	Delay: INF; Queue1: PS; Queue2: PS
Service	Delay – class1: Exp(1) Queue1 – class1: Exp(1) Queue2 – class1: Exp(1)

Table 4.18: Features of gallery_fj_closed.

4.2.18 gallery_fj_open

An open fork-join block: a job forks into parallel tasks and its siblings are joined before departure.

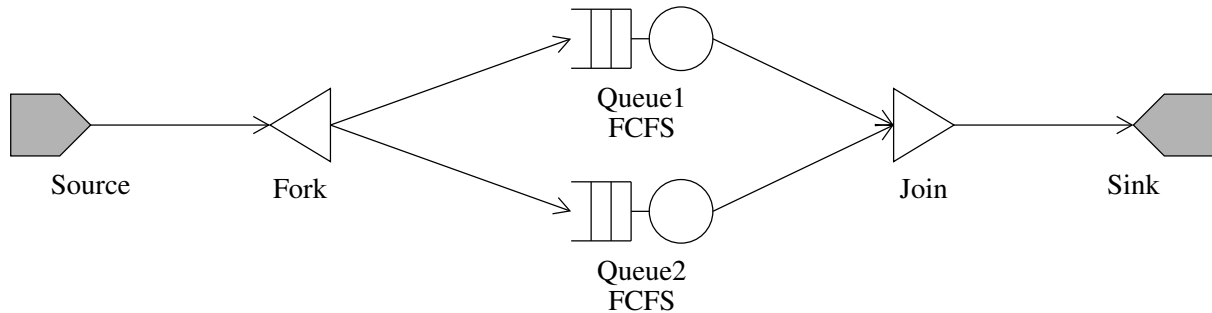


Figure 4.18: Topology of gallery_fj_open: an open network over 2 queueing stations with a fork-join block.

Nodes	6: Source, Queue $\times$ 2, Fork, Join, Sink
Classes	1: class1 (open)
Scheduling	Queue1: FCFS; Queue2: FCFS
Arrivals	Source – class1: Exp(0.05)
Service	Queue1 – class1: Exp(1) Queue2 – class1: Exp(2)

Table 4.19: Features of gallery_fj_open.

The example is built and solved as follows.

```
Net m("Fork-Join-Open");
Source source(m, "Source");
Queue q1(m, "Queue1", SchedStrategy::FCFS);
Queue q2(m, "Queue2", SchedStrategy::FCFS);
const std::size_t fork = m.add_fork("Fork");
const std::size_t join = m.add_join("Join", fork);
Sink sink(m, "Sink");

OpenClass c1(m, "class1", 0);
source.set_arrival(c1, Exp(0.05));
q1.set_service(c1, Exp(1.0));
q2.set_service(c1, Exp(2.0));

Routing P;
P.set(c1, c1, source, fork, 1.0);
P.set(c1, c1, fork, q1, 1.0);
P.set(c1, c1, fork, q2, 1.0);
P.set(c1, c1, q1, join, 1.0);
P.set(c1, c1, q2, join, 1.0);
P.set(c1, c1, join, sink, 1.0);
m.link(P);
return m;
```

Running this edition prints

```
Model: Fork-Join-Open
```

4.2.19 gallery_fj_quorum

A fork-join block whose join fires on a quorum of the siblings, the later ones being discarded when they arrive.

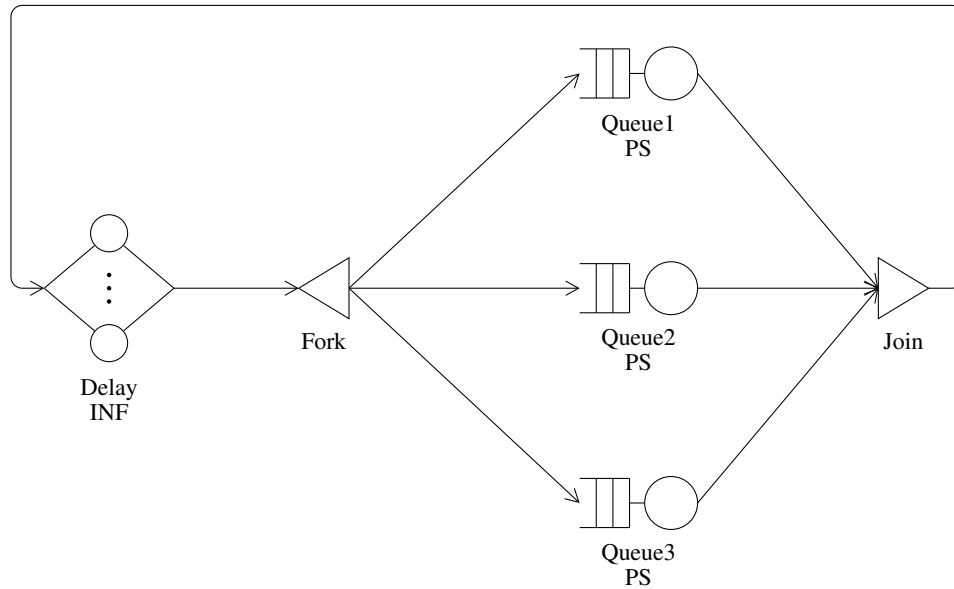


Figure 4.19: Topology of gallery_fj_quorum: a closed network over 4 queueing stations with a fork-join block.

Nodes	6: Queue $\times$ 3, Delay, Fork, Join
Classes	1: class1 (closed, $N = 5$)
Scheduling	Delay: INF; Queue1: PS; Queue2: PS; Queue3: PS
Service	Delay – class1: Exp(1) Queue1 – class1: Exp(2) Queue2 – class1: Exp(2) Queue3 – class1: Exp(2)

Table 4.20: Features of gallery_fj_quorum.

The example is built and solved as follows.

```
Net m("Fork-Join-Quorum");
Delay delay(m, "Delay");
Queue q1(m, "Queue1", SchedStrategy::PS);
Queue q2(m, "Queue2", SchedStrategy::PS);
Queue q3(m, "Queue3", SchedStrategy::PS);
const std::size_t fork = m.add_fork("Fork");
const std::size_t join = m.add_join("Join", fork);

ClosedClass c1(m, "class1", 5, delay, 0);
delay.set_service(c1, Exp(1.0));
q1.set_service(c1, Exp(2.0));
q2.set_service(c1, Exp(2.0));
q3.set_service(c1, Exp(2.0));
m.set_join_strategy(join, lang::JoinStrategy::PARTIAL, 2.0);

Routing P;
P.set(c1, c1, delay, fork, 1.0);
P.set(c1, c1, fork, q1, 1.0);
P.set(c1, c1, fork, q2, 1.0);
P.set(c1, c1, fork, q3, 1.0);
P.set(c1, c1, q1, join, 1.0);
P.set(c1, c1, q2, join, 1.0);
P.set(c1, c1, q3, join, 1.0);
```

```
P.set(c1, c1, join, delay, 1.0);
m.link(P);
return m;
```

Running this edition prints

```
Model: Fork-Join-Quorum
```

4.2.20 gallery_gamm1

The Gamma/M/1 queue: Gamma interarrival times, exponential service, and a single server.

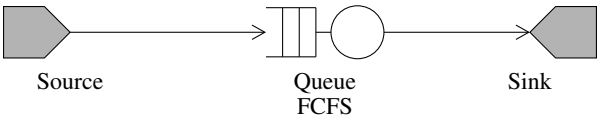


Figure 4.20: Topology of gallery_gamm1: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: Class1 (open)
Scheduling	Queue: FCFS
Arrivals	Source – Class1: Gamma(5,0.2)
Service	Queue – Class1: Exp(2)

Table 4.21: Features of gallery_gamm1.

The example is built and solved as follows.

```
Net gallery_gamm1() {
  Net m("Gam/M/1");
  Source source(m, "Source");
  Queue queue(m, "Queue", SchedStrategy::FCFS);
  Sink sink(m, "Sink");
  OpenClass c1(m, "Class1");
  source.set_arrival(c1, lang::gamma_fit_mean_scv<double>(1.0, 1.0 / 5.0));
  queue.set_service(c1, Exp(2.0));
  Routing P;
  serial(P, {source, queue, sink});
  m.link(P);
  return m;
}
```

Running this edition prints

```
Model: Gam/M/1
```

4.2.21 gallery_hyperl1_feedback

A HyperExp/Erl/1 queue with probabilistic feedback.

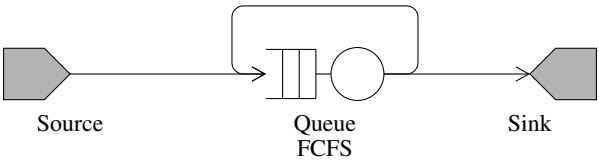


Figure 4.21: Topology of gallery_hyperl1_feedback: an open network over 1 queueing station.

The example is built and solved as follows.

Nodes	3: Source, Queue, Sink
Classes	1: Class1 (open)
Scheduling	Queue: FCFS
Arrivals	Source – Class1: HyperExp(0.99,0.01;2.294,0.01759)
Service	Queue – Class1: Erlang(100,5)

Table 4.22: Features of gallery_hyperl1_feedback.

```

Net m("Hyper/Erl/1-Feedback");
Source source(m, "Source");
Queue queue(m, "Queue", SchedStrategy::FCFS);
Sink sink(m, "Sink");
OpenClass c1(m, "Class1");
source.set_arrival(c1, lang::hyperexp_fit_mean_scv<double>(1.0, 64.0));
queue.set_service(c1, lang::erlang_fit_mean_order<double>(0.05, 5));
Routing P;
P.set(c1, c1, source, queue, 1.0);
P.set(c1, c1, queue, queue, 0.9);
P.set(c1, c1, queue, sink, 0.1);
m.link(P);
return m;

```

Running this edition prints

```
Model: Hyper/Erl/1-Feedback
```

4.2.22 gallery_hyperl1_reentrant

The HyperExp/Erl/1 queue: hyperexponential interarrival times, Erlang service, and a single server. In this variant, jobs re-enter the station, so a visit is counted more than once.

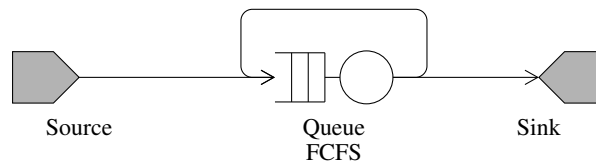


Figure 4.22: Topology of gallery_hyperl1_reentrant: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	2: Class1 (open), Class2 (open)
Scheduling	Queue: FCFS
Arrivals	Source – Class1: HyperExp(0.99,0.01;2.294,0.01759); Class2: Disabled
Service	Queue – Class1: Erlang(10,5); Class2: Exp(3)
Features	class switching on 1 link

Table 4.23: Features of gallery_hyperl1_reentrant.

The example is built and solved as follows.

```

Net gallery_hyperl1_reentrant() {
    return reentrant("Hyper/Erl/1-Reentrant", SchedStrategy::FCFS,
        lang::hyperexp_fit_mean_scv<double>(1.0, 64.0),
        lang::erlang_fit_mean_order<double>(0.5, 5), Exp(3.0));
}

```

Running this edition prints

Model: Hyper/Erl/1-Reentrant

4.2.23 gallery_hyperlk

The HyperExp/Erl/ k queue: hyperexponential interarrival times, Erlang service, and k parallel servers.

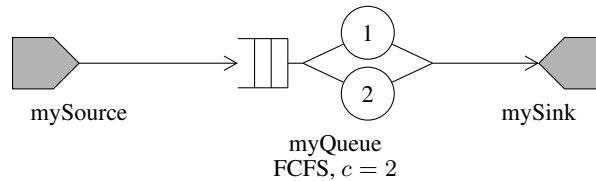


Figure 4.23: Topology of gallery_hyperlk: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: myClass (open)
Scheduling	myQueue: FCFS ($c = 2$)
Arrivals	mySource – myClass: HyperExp(0.1127,0.8873;0.4057,3.194)
Service	myQueue – myClass: Erlang(4,4)

Table 4.24: Features of gallery_hyperlk.

The example is built and solved as follows.

```
Net gallery_hyperlk(double k) {
    return one_queue("Hyper/Erl/k", "mySource", "myQueue", "mySink", SchedStrategy::FCFS,
        lang::hyperexp_fit_mean_scv_balanced<double>(1.0 / 1.8, 4.0),
        D::erlang_fit(1.0, 0.25), k);
}
```

Running this edition prints

Model: Hyper/Erl/k

4.2.24 gallery_hyphyp1_linear

The HyperExp/HyperExp/1 queue: hyperexponential interarrival times, hyperexponential service, and a single server. In this variant, the model is a linear chain of such stations.

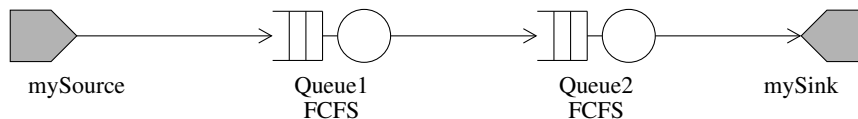


Figure 4.24: Topology of gallery_hyphyp1_linear: an open network over 2 queueing stations.

The example is built and solved as follows.

```
Net gallery_hyphyp1_linear(std::size_t n, double umax) {
    const std::vector<double> means = linear_means(n, umax);
    std::vector<D> svc;
    for (std::size_t i = 0; i < n; ++i)
        svc.push_back(
            lang::hyperexp_fit_mean_scv<double>(means[i], 1.0 + static_cast<double>(i) + 1.0)
        );
    return linear_chain("Hyp/Hyp/1-Linear", n, lang::hyperexp_fit_mean_scv<double>(1.0, 2.0),
        svc);
}
```

Nodes	4: Source, Queue $\times$ 2, Sink
Classes	1: myClass (open)
Scheduling	Queue1: FCFS; Queue2: FCFS
Arrivals	mySource – myClass: HyperExp(0.99,0.01;1.077,0.1244)
Service	Queue1 – myClass: HyperExp(0.99,0.01;1.196,0.1383) Queue2 – myClass: HyperExp(0.99,0.01;1.235,0.1015)

Table 4.25: Features of gallery_hyphyp1_linear.

Running this edition prints

```
Model: Hyp/Hyp/1-Linear
```

4.2.25 gallery_hyphyp1_reentrant

The HyperExp/HyperExp/1 queue: hyperexponential interarrival times, hyperexponential service, and a single server. In this variant, jobs re-enter the station, so a visit is counted more than once.

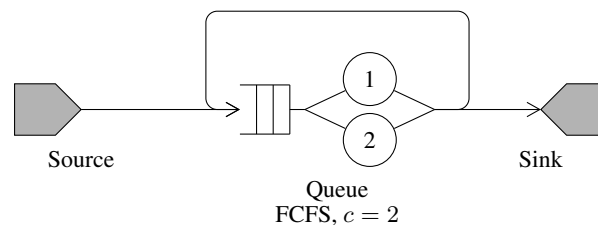


Figure 4.25: Topology of gallery_hyphyp1_reentrant: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	2: Class1 (open), Class2 (open)
Scheduling	Queue: FCFS ($c = 2$)
Arrivals	Source – Class1: HyperExp(0.99,0.01;2.294,0.01759); Class2: Disabled
Service	Queue – Class1: HyperExp(0.99,0.01;2.281,0.1517); Class2: Exp(3)
Features	class switching on 1 link

Table 4.26: Features of gallery_hyphyp1_reentrant.

The example is built and solved as follows.

```
Net gallery_hyphyp1_reentrant() {
    return reentrant("Hyper/Hyper/1-Reentrant", SchedStrategy::FCFS,
        lang::hyperexp_fit_mean_scv<double>(1.0, 64.0),
        lang::hyperexp_fit_mean_scv<double>(0.5, 4.0), Exp(3.0), 2.0);
}
```

Running this edition prints

```
Model: Hyper/Hyper/1-Reentrant
```

4.2.26 gallery_hyphyp1_tandem

Two HyperExp/HyperExp/1 queues in series.

The example is built and solved as follows.

```
Net gallery_hyphyp1_tandem() { return gallery_hyphyp1_linear(2); }
```

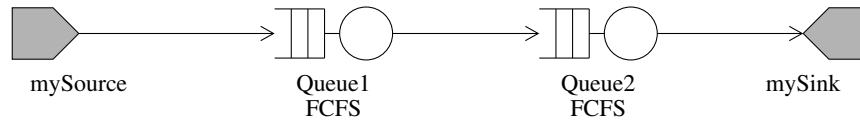


Figure 4.26: Topology of gallery_hyphyp1_tandem: an open network over 2 queueing stations.

Nodes	4: Source, Queue $\times$ 2, Sink
Classes	1: myClass (open)
Scheduling	Queue1: FCFS; Queue2: FCFS
Arrivals	mySource – myClass: HyperExp(0.99,0.01;1.077,0.1244)
Service	Queue1 – myClass: HyperExp(0.99,0.01;1.196,0.1383) Queue2 – myClass: HyperExp(0.99,0.01;1.235,0.1015)

Table 4.27: Features of gallery_hyphyp1_tandem.

Running this edition prints

```
Model: Hyp/Hyp/1-Linear
```

4.2.27 gallery_hypm1

The HyperExp/M/1 queue: hyperexponential interarrival times, exponential service, and a single server.

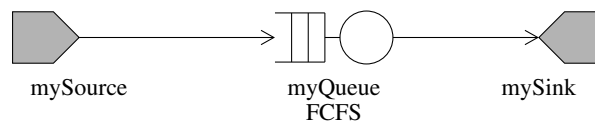


Figure 4.27: Topology of gallery_hypm1: an open network over 1 queueing station.

The example is built and solved as follows.

```
Net gallery_hypm1() {
    return one_queue("H2/M/1", "mySource", "myQueue", "mySink", SchedStrategy::FCFS,
        lang::hyperexp_fit_mean_scv<double>(1.0, 64.0), Exp(2.0));
}
```

Running this edition prints

```
Model: H2/M/1
```

4.2.28 gallery_hypm1_reentrant

The HyperExp/M/1 queue: hyperexponential interarrival times, exponential service, and a single server. In this variant, jobs re-enter the station, so a visit is counted more than once.

The example is built and solved as follows.

```
Net gallery_hypm1_reentrant() {
    return reentrant("Hyper/M/1-Reentrant", SchedStrategy::FCFS,
        lang::hyperexp_fit_mean_scv<double>(1.0, 4.0), Exp(2.0),
        Exp(3.0));
}
```

Running this edition prints

```
Model: Hyper/M/1-Reentrant
```

Nodes	3: Source, Queue, Sink
Classes	1: myClass (open)
Scheduling	myQueue: FCFS
Arrivals	mySource – myClass: HyperExp(0.99,0.01;2.294,0.01759)
Service	myQueue – myClass: Exp(2)

Table 4.28: Features of gallery_hypm1.

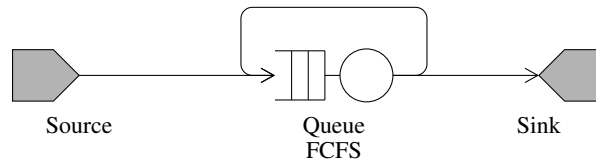


Figure 4.28: Topology of gallery_hypm1_reentrant: an open network over 1 queueing station.

4.2.29 gallery_lqn_basic

The smallest layered queueing network: one reference task calling one server task.
The example is built and solved as follows.

```

Lqn b;
b.processor("P1", 2, SchedStrategy::PS);
b.processor("P2", 3, SchedStrategy::PS);

b.task("T1", 50, SchedStrategy::REF, "P1");
b.think_time("T1", E(1.0 / 2.0));
b.task("T2", 50, SchedStrategy::FCFS, "P1");
b.think_time("T2", E(1.0 / 3.0));
b.task("T3", 25, SchedStrategy::FCFS, "P2");
b.think_time("T3", E(1.0 / 4.0));

b.entry("E1", "T1");
b.entry("E2", "T2");
b.entry("E3", "T3");

b.activity("AS1", E(10.0), "T1");
b.bound_to("AS1", "E1");
b.sync_call("AS1", "E2", 1.0);
b.activity("AS2", E(20.0), "T2");
b.bound_to("AS2", "E2");
b.sync_call("AS2", "E3", 5.0);
b.replies_to("AS2", "E2");
b.activity("AS3", E(50.0), "T3");
b.bound_to("AS3", "E3");
b.replies_to("AS3", "E3");
return b;

```

Running this edition prints

```
Model: LQN-Basic
```

Nodes	3: Source, Queue, Sink
Classes	2: Class1 (open), Class2 (open)
Scheduling	Queue: FCFS
Arrivals	Source – Class1: HyperExp(0.99,0.01;1.14,0.07584); Class2: Disabled
Service	Queue – Class1: Exp(2); Class2: Exp(3)
Features	class switching on 1 link

Table 4.29: Features of gallery_hypm1_reentrant.

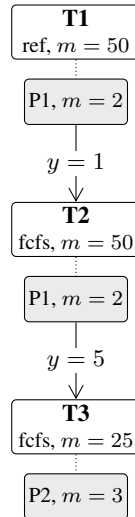


Figure 4.29: Call graph of `gallery_lqn_basic`: 3 tasks over 2 processors, drawn with the reference task on top and a called task below its caller; the shaded box under each task is the processor it runs on.

Processors	2: P1 (ps, $m = 2$), P2 (ps, $m = 3$)
Tasks	3: T1 (ref, $m = 50$), T2 (fcfs, $m = 50$), T3 (fcfs, $m = 25$)
Entries	3: E1, E2, E3
Activities	3, host demands AS1: 0.1, AS2: 0.05, AS3: 0.02
Calls	2 (2 synchronous, 0 asynchronous)
Think times	T1: 2.0

Table 4.30: Features of `gallery_lqn_basic`.

4.2.30 `gallery_lqn_random`

A randomly generated layered model, used to exercise the layered solvers beyond hand-written topologies.

Processors	3: c_processor_1 (inf, $m = 1$), processor_1 (ps, $m = 1$), processor_2 (ps, $m = 1$)
Tasks	5: c_task_1 (ref, $m = 1$), task_1 (fcfs, $m = 1$), task_2 (fcfs, $m = 1$), task_3 (fcfs, $m = 1$), task_4 (fcfs, $m = 1$)
Entries	5: c_entry_1, entry_1, entry_2, entry_3, entry_4
Activities	5, host demands c_activity_1: 1.0, activity_1: 1.0, activity_2: 1.0, activity_3: 1.0, activity_4: 1.0
Calls	4 (4 synchronous, 0 asynchronous)

Table 4.31: Features of `gallery_lqn_random`.

The example is built and solved as follows.

```

Lqn b;
b.processor("c_processor_1", INF_MULT, SchedStrategy::INF);
b.processor("processor_1", 1, SchedStrategy::PS);
b.processor("processor_2", 1, SchedStrategy::PS);

b.task("c_task_1", 1, SchedStrategy::REF, "c_processor_1");
b.task("task_1", 1, SchedStrategy::FCFS, "processor_1");
b.task("task_2", 1, SchedStrategy::FCFS, "processor_1");
b.task("task_3", 1, SchedStrategy::FCFS, "processor_1");
b.task("task_4", 1, SchedStrategy::FCFS, "processor_2");

b.entry("c_entry_1", "c_task_1");
for (std::size_t t = 1; t <= 4; ++t)
    b.entry("entry_" + std::to_string(t), "task_" + std::to_string(t));

```

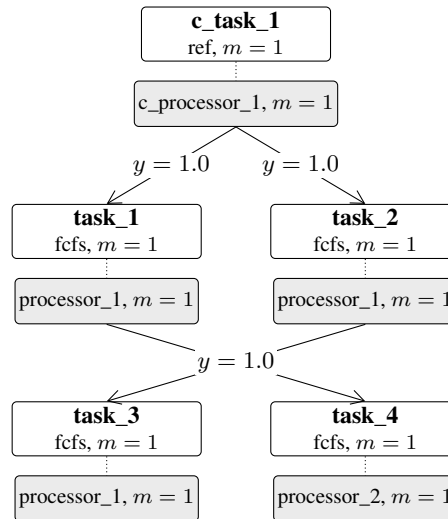


Figure 4.30: Call graph of `gallery_lqn_random`: 5 tasks over 3 processors, drawn with the reference task on top and a called task below its caller; the shaded box under each task is the processor it runs on.

```
b.activity("c_activity_1", E(1.0), "c_task_1");
b.bound_to("c_activity_1", "c_entry_1");
for (std::size_t t = 1; t <= 4; ++t) {
    const std::string a = "activity_" + std::to_string(t);
    const std::string e = "entry_" + std::to_string(t);
    b.activity(a, E(1.0), "task_" + std::to_string(t));
    b.bound_to(a, e);
    b.replies_to(a, e);
}

b.sync_call("c_activity_1", "entry_1", 1.0);
b.sync_call("c_activity_1", "entry_2", 1.0);
b.sync_call("activity_1", "entry_4", 1.0);
b.sync_call("activity_2", "entry_3", 1.0);
return b;
```

Running this edition prints

```
Model: lnw
```

4.2.31 gallery_lqn_workflows

Workflow patterns (sequence, AND-fork and join, loop) as an activity graph in a layered model.

Processors	3: P1 (inf, $m = 1$), P2 (inf, $m = 1$), P3 (ps, $m = 5$)
Tasks	3: T1 (ref, $m = 1$), T2 (inf, $m = 1$), T3 (inf, $m = 1$)
Entries	3: Entry, E2, E3
Activities	14, host demands A1: 1.0, A2: 2.0, A3: 3.0, B1: 0.1, B2: 0.2, B3: 0.3, B4: 0.4, B5: 0.5, ...
Calls	2 (2 synchronous, 0 asynchronous)
Think times	T1: 0.0

Table 4.32: Features of `gallery_lqn_workflows`.

The example is built and solved as follows.

```
Lqn b;
b.processor("P1", INF_MULT, SchedStrategy::INF);
```

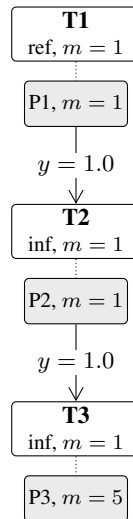


Figure 4.31: Call graph of `gallery_lqn_workflows`: 3 tasks over 3 processors, drawn with the reference task on top and a called task below its caller; the shaded box under each task is the processor it runs on.

```

b.task("T1", 1, SchedStrategy::REF, "P1");
b.think_time("T1", Immediate());
b.entry("Entry", "T1");

b.processor("P2", INF_MULT, SchedStrategy::INF);
b.task("T2", INF_MULT, SchedStrategy::INF, "P2");
b.think_time("T2", Immediate());
b.entry("E2", "T2");

b.processor("P3", 5, SchedStrategy::PS);
b.task("T3", INF_MULT, SchedStrategy::INF, "P3");
b.think_time("T3", D::exp_mean(10.0));
b.entry("E3", "T3");

b.activity("A1", D::exp_mean(1.0), "T1");
b.bound_to("A1", "Entry");
b.activity("A2", D::exp_mean(2.0), "T1");
b.activity("A3", D::exp_mean(3.0), "T1");
b.sync_call("A3", "E2", 1.0);

b.activity("B1", D::exp_mean(0.1), "T2");
b.bound_to("B1", "E2");
b.activity("B2", D::exp_mean(0.2), "T2");
b.activity("B3", D::exp_mean(0.3), "T2");

... (16 source lines omitted; full implementation: cpp/examples/gallery/)

b.serial("B4", "B5");
b.and_fork("B1", {"B2", "B3", "B4"});
b.and_join({"B2", "B3", "B5"}, "B6");
b.or_fork("C1", {"C2", "C3", "C4"}, {0.3, 0.3, 0.4});
b.or_join({"C2", "C3", "C4"}, "C5");
return b;

```

Running this edition prints

```
Model: LQN-Workflows
```

4.2.32 gallery_lukumar_reentrant

The Lu-Kumar re-entrant line, the standard counterexample in which a work-conserving policy destabilizes a network whose stations are individually underloaded.

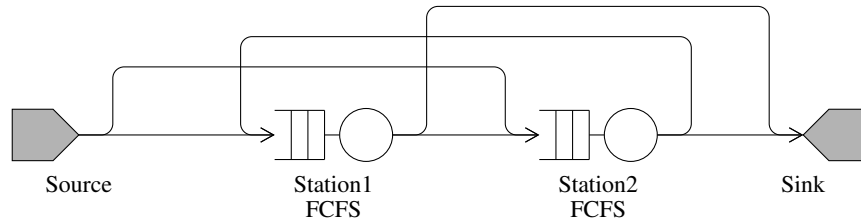


Figure 4.32: Topology of gallery_lukumar_reentrant: an open network over 2 queueing stations.

Nodes	4: Source, Queue $\times$ 2, Sink
Classes	4: Class1 (open), Class2 (open), Class3 (open), Class4 (open)
Scheduling	Station1: FCFS; Station2: FCFS
Arrivals	Source – Class1: Exp(0.08); Class2: Disabled; Class3: Exp(0.08); Class4: Disabled
Service	Station1 – Class1: Exp(0.1); Class2: Disabled; Class3: Disabled; Class4: Exp(1) Station2 – Class1: Disabled; Class2: Exp(1); Class3: Exp(0.1); Class4: Disabled
Features	class switching on 1 link

Table 4.33: Features of gallery_lukumar_reentrant.

The example is built and solved as follows.

```
SchedStrategy s = SchedStrategy::FCFS;
if (sched == "HOL" || sched == "hol") s = SchedStrategy::HOL;
else if (sched == "PS" || sched == "ps") s = SchedStrategy::PS;

Net m("Lu-Kumar-Reentrant");
Source source(m, "Source");
Queue st1(m, "Station1", s);
Queue st2(m, "Station2", s);
Sink sink(m, "Sink");

OpenClass c1(m, "Class1", 1);
OpenClass c2(m, "Class2", 0);
OpenClass c3(m, "Class3", 1);
OpenClass c4(m, "Class4", 0);

const double lambda = 0.08;
source.set_arrival(c1, Exp(lambda));
source.set_arrival(c2, Disabled());
source.set_arrival(c3, Exp(lambda));
source.set_arrival(c4, Disabled());

// Slow first visits (mean 10), fast second visits (mean 1): the asymmetry
// that creates the virtual bottleneck.
st1.set_service(c1, Exp(1.0 / 10.0));
st1.set_service(c2, Disabled());
st1.set_service(c3, Disabled());

... (9 source lines omitted; full implementation: cpp/examples/gallery/)

P.set(c2, c2, st2, sink, 1.0);
P.set(c3, c3, source, st2, 1.0);
P.set(c3, c4, st2, st1, 1.0);
P.set(c4, c4, st1, sink, 1.0);
m.link(P);
return m;
```


Running this edition prints

```
Model: Lu-Kumar-Reentrant
```

4.2.33 gallery_mapm1

The MAP/M/1 queue: a Markovian arrival process, exponential service, and a single server.

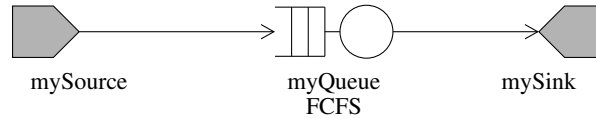


Figure 4.33: Topology of gallery_mapm1: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: myClass (open)
Scheduling	myQueue: FCFS
Arrivals	mySource – myClass: MAP
Service	myQueue – myClass: Exp(2)

Table 4.34: Features of gallery_mapm1.

The example is built and solved as follows.

```
Net gallery_mapm1() {
    return one_queue("MAP/M/1", "mySource", "myQueue", "mySink", SchedStrategy::FCFS,
        map_of(map_rand_23000()), Exp(2.0));
}
```

Running this edition prints

```
Model: MAP/M/1
```

4.2.34 gallery_mapmk

The MAP/M/ k queue: a Markovian arrival process, exponential service, and k parallel servers.

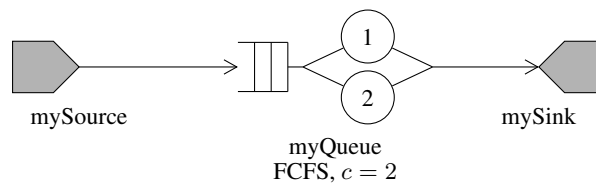


Figure 4.34: Topology of gallery_mapmk: an open network over 1 queueing station.

The example is built and solved as follows.

```
Net gallery_mapmk(double k) {
    return one_queue("MAP/M/k", "mySource", "myQueue", "mySink", SchedStrategy::FCFS,
        map_of(map_rand_23000()), Exp(2.0), k);
}
```

Running this edition prints

```
Model: MAP/M/k
```

Nodes	3: Source, Queue, Sink
Classes	1: myClass (open)
Scheduling	myQueue: FCFS ($c = 2$)
Arrivals	mySource – myClass: MAP
Service	myQueue – myClass: Exp(2)

Table 4.35: Features of gallery_mapmk.

4.2.35 gallery_mdk

The M/D/ k queue: Poisson arrivals, deterministic service, and k parallel servers.

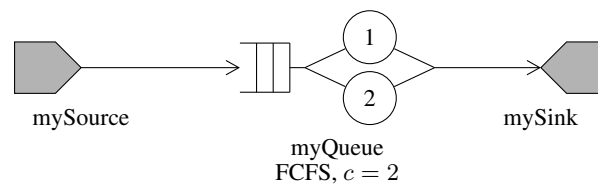


Figure 4.35: Topology of gallery_mdk: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: myClass (open)
Scheduling	myQueue: FCFS ($c = 2$)
Arrivals	mySource – myClass: Exp(1)
Service	myQueue – myClass: Det(1)

Table 4.36: Features of gallery_mdk.

The example is built and solved as follows.

```
Net gallery_mdk(double k) {
    return one_queue("M/D/k", "mySource", "myQueue", "mySink", SchedStrategy::FCFS,
        D::exp_mean(1.0), Det(2.0 / k), k);
}
```

Running this edition prints

```
Model: M/D/k
```

4.2.36 gallery_mer11

The M/Erl/1 queue: Poisson arrivals, Erlang service, and a single server.

The example is built and solved as follows.

```
Net gallery_mer11() {
    return one_queue("M/E/1", "mySource", "myQueue", "mySink", SchedStrategy::FCFS,
        Exp(1.0), lang::erlang_fit_mean_order<double>(0.5, 2));
}
```

Running this edition prints

```
Model: M/E/1
```

4.2.37 gallery_mer11_linear

The M/Erl/1 queue: Poisson arrivals, Erlang service, and a single server. In this variant, the model is a linear chain of such stations.

The example is built and solved as follows.

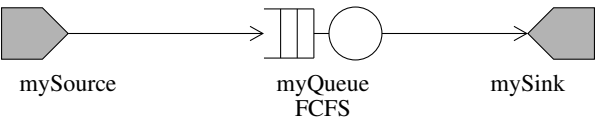


Figure 4.36: Topology of gallery_merl1: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: myClass (open)
Scheduling	myQueue: FCFS
Arrivals	mySource – myClass: Exp(1)
Service	myQueue – myClass: Erlang(4,2)

Table 4.37: Features of gallery_merl1.

```
Net gallery_merl1_linear(std::size_t n, double umax) {
    const std::vector<double> means = linear_means(n, umax);
    std::vector<D> svc;
    for (std::size_t i = 0; i < n; ++i)
        svc.push_back(lang::erlang_fit_mean_order<double>(means[i], i + 1));
    return linear_chain("M/Erl/1-Linear", n, Exp(1.0), svc);
}
```

Running this edition prints

```
Model: M/Erl/1-Linear
```

4.2.38 gallery_merl1_reentrant

The M/Erl/1 queue: Poisson arrivals, Erlang service, and a single server. In this variant, jobs re-enter the station, so a visit is counted more than once.
The example is built and solved as follows.

```
Net gallery_merl1_reentrant() {
    return reentrant("M/Erl/1-Reentrant", SchedStrategy::FCFS, Exp(1.0),
        lang::erlang_fit_mean_order<double>(0.5, 5), Exp(3.0));
}
```

Running this edition prints

```
Model: M/Erl/1-Reentrant
```

4.2.39 gallery_merl1_tandem

Two M/Erl/1 queues in series.
The example is built and solved as follows.

```
Net gallery_merl1_tandem() { return gallery_merl1_linear(2); }
```

Running this edition prints

```
Model: M/Erl/1-Linear
```

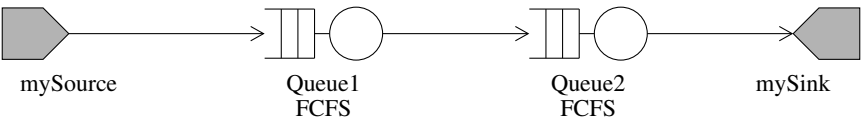


Figure 4.37: Topology of gallery_merl1_linear: an open network over 2 queueing stations.

Nodes	4: Source, Queue $\times$ 2, Sink
Classes	1: myClass (open)
Scheduling	Queue1: FCFS; Queue2: FCFS
Arrivals	mySource – myClass: Exp(1)
Service	Queue1 – myClass: Erlang(1.111,1) Queue2 – myClass: Erlang(2.222,2)

Table 4.38: Features of gallery_mer11_linear.

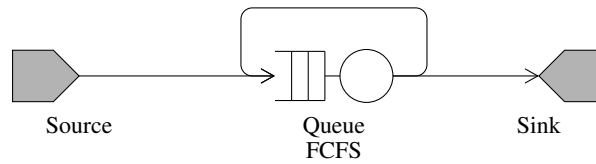


Figure 4.38: Topology of gallery_mer11_reentrant: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	2: Class1 (open), Class2 (open)
Scheduling	Queue: FCFS
Arrivals	Source – Class1: Exp(1); Class2: Disabled
Service	Queue – Class1: Erlang(10,5); Class2: Exp(3)
Features	class switching on 1 link

Table 4.39: Features of gallery_mer11_reentrant.

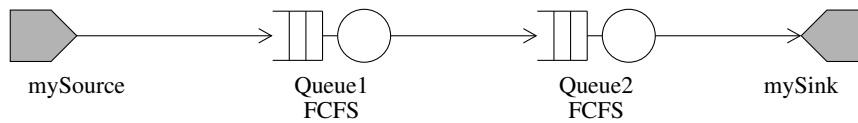


Figure 4.39: Topology of gallery_mer11_tandem: an open network over 2 queueing stations.

Nodes	4: Source, Queue $\times$ 2, Sink
Classes	1: myClass (open)
Scheduling	Queue1: FCFS; Queue2: FCFS
Arrivals	mySource – myClass: Exp(1)
Service	Queue1 – myClass: Erlang(1.111,1) Queue2 – myClass: Erlang(2.222,2)

Table 4.40: Features of gallery_mer11_tandem.

4.2.40 gallery_merlk

The M/Erl/ k queue: Poisson arrivals, Erlang service, and k parallel servers.

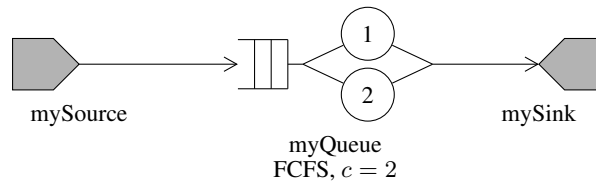


Figure 4.40: Topology of gallery_merlk: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: myClass (open)
Scheduling	myQueue: FCFS ($c = 2$)
Arrivals	mySource – myClass: Exp(1)
Service	myQueue – myClass: Erlang(4,2)

Table 4.41: Features of gallery_merlk.

The example is built and solved as follows.

```
Net gallery_merlk(double k) {
    return one_queue("M/Erl/" + std::to_string(static_cast<int>(k)), "mySource", "myQueue",
        "mySink", SchedStrategy::FCFS, Exp(1.0),
        lang::erlang_fit_mean_order<double>(0.5, 2), k);
}
```

Running this edition prints

```
Model: M/Erl/2
```

4.2.41 gallery_mhyp1

The M/HyperExp/1 queue: Poisson arrivals, hyperexponential service, and a single server.

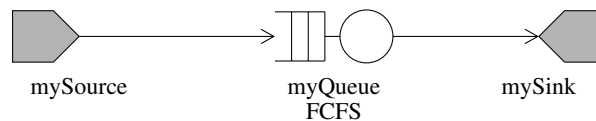


Figure 4.41: Topology of gallery_mhyp1: an open network over 1 queueing station.

The example is built and solved as follows.

```
Net gallery_mhyp1() {
    return one_queue("M/H/1", "mySource", "myQueue", "mySink", SchedStrategy::FCFS,
        Exp(1.0), lang::hyperexp_fit_mean_scv<double>(0.5, 4.0));
}
```

Running this edition prints

```
Model: M/H/1
```

4.2.42 gallery_mhyp1_linear

The M/HyperExp/1 queue: Poisson arrivals, hyperexponential service, and a single server. In this variant, the model is a linear chain of such stations.

The example is built and solved as follows.

Nodes	3: Source, Queue, Sink
Classes	1: myClass (open)
Scheduling	myQueue: FCFS
Arrivals	mySource – myClass: Exp(1)
Service	myQueue – myClass: HyperExp(0.99,0.01;2.281,0.1517)

Table 4.42: Features of gallery_mhyp1.

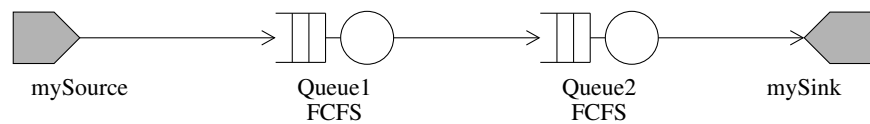


Figure 4.42: Topology of gallery_mhyp1_linear: an open network over 2 queueing stations.

```
Net gallery_mhyp1_linear(std::size_t n, double umax) {
    const std::vector<double> means = linear_means(n, umax);
    std::vector<D> svc;
    for (std::size_t i = 0; i < n; ++i)
        svc.push_back(lang::hyperexp_fit_mean_scv<double>(means[i], static_cast<double>(n)));
    return linear_chain("M/Hyp/1-Linear", n, Exp(1.0), svc);
}
```

Running this edition prints

```
Model: M/Hyp/1-Linear
```

4.2.43 gallery_mhyp1_reentrant

The M/HyperExp/1 queue: Poisson arrivals, hyperexponential service, and a single server. In this variant, jobs re-enter the station, so a visit is counted more than once. The example is built and solved as follows.

```
Net gallery_mhyp1_reentrant() {
    return reentrant("M/Hyper/1-Reentrant", SchedStrategy::FCFS, Exp(1.0),
        lang::coxian_fit_mean_scv<double>(0.5, 4.0), Exp(3.0));
}
```

Running this edition prints

```
Model: M/Hyper/1-Reentrant
```

4.2.44 gallery_mhyp1_tandem

Two M/HyperExp/1 queues in series. The example is built and solved as follows.

```
Net gallery_mhyp1_tandem() { return gallery_mhyp1_linear(2); }
```

Nodes	4: Source, Queue × 2, Sink
Classes	1: myClass (open)
Scheduling	Queue1: FCFS; Queue2: FCFS
Arrivals	mySource – myClass: Exp(1)
Service	Queue1 – myClass: HyperExp(0.99,0.01;1.196,0.1383) Queue2 – myClass: Hyper-Exp(0.99,0.01;1.196,0.1383)

Table 4.43: Features of gallery_mhyp1_linear.

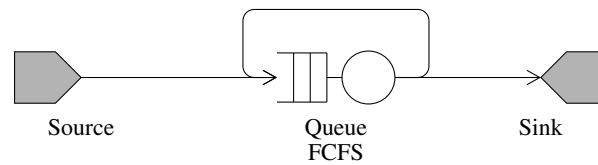


Figure 4.43: Topology of `gallery_mhyp1_reentrant`: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	2: Class1 (open), Class2 (open)
Scheduling	Queue: FCFS
Arrivals	Source – Class1: Exp(1); Class2: Disabled
Service	Queue – Class1: Coxian; Class2: Exp(3)
Features	class switching on 1 link

Table 4.44: Features of `gallery_mhyp1_reentrant`.

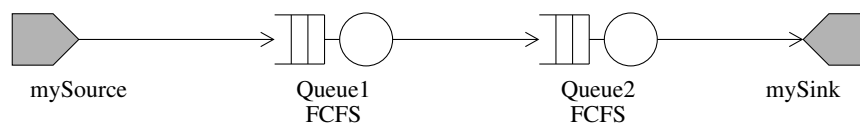


Figure 4.44: Topology of `gallery_mhyp1_tandem`: an open network over 2 queueing stations.

Nodes	4: Source, Queue $\times$ 2, Sink
Classes	1: myClass (open)
Scheduling	Queue1: FCFS; Queue2: FCFS
Arrivals	mySource – myClass: Exp(1)
Service	Queue1 – myClass: HyperExp(0.99,0.01;1.196,0.1383) Queue2 – myClass: HyperExp(0.99,0.01;1.196,0.1383)

Table 4.45: Features of `gallery_mhyp1_tandem`.

Running this edition prints

```
Model: M/Hyp/1-Linear
```

4.2.45 gallery_mhyphk

The M/HyperExp/*k* queue: Poisson arrivals, hyperexponential service, and *k* parallel servers.

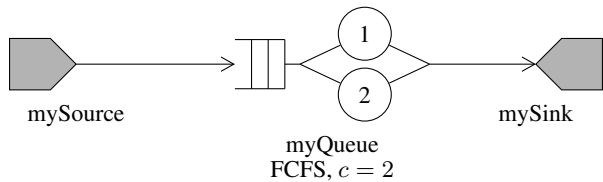


Figure 4.45: Topology of gallery_mhyphk: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: myClass (open)
Scheduling	myQueue: FCFS (<i>c</i> = 2)
Arrivals	mySource – myClass: Exp(1)
Service	myQueue – myClass: Coxian

Table 4.46: Features of gallery_mhyphk.

The example is built and solved as follows.

```
Net gallery_mhyphk(double k) {
    return one_queue("M/Hyper/" + std::to_string(static_cast<int>(k)), "mySource", "myQueue",
        "mySink", SchedStrategy::FCFS, Exp(1.0),
        lang::coxian_fit_mean_scv<double>(0.5, 4.0), k);
}
```

Running this edition prints

```
Model: M/Hyper/2
```

4.2.46 gallery_mm1

The M/M/1 queue: Poisson arrivals, exponential service, and a single server.

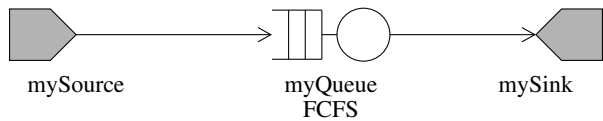


Figure 4.46: Topology of gallery_mm1: an open network over 1 queueing station.

The example is built and solved as follows.

```
Net gallery_mm1() {
    return one_queue("M/M/1", "mySource", "myQueue", "mySink", SchedStrategy::FCFS,
        Exp(1.0), Exp(2.0));
}
```

Running this edition prints

```
Model: M/M/1
```


Nodes	3: Source, Queue, Sink
Classes	1: myClass (open)
Scheduling	myQueue: FCFS
Arrivals	mySource – myClass: Exp(1)
Service	myQueue – myClass: Exp(2)

Table 4.47: Features of gallery_mm1.

4.2.47 gallery_mm1_feedback

The M/M/1 queue: Poisson arrivals, exponential service, and a single server. In this variant, a completing job returns to the station with fixed probability.

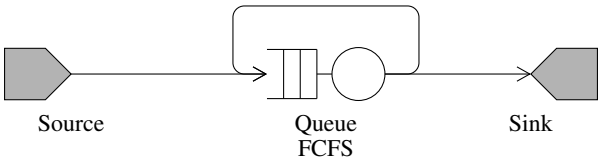


Figure 4.47: Topology of gallery_mm1_feedback: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: Class1 (open)
Scheduling	Queue: FCFS
Arrivals	Source – Class1: Exp(1)
Service	Queue – Class1: Exp(2)

Table 4.48: Features of gallery_mm1_feedback.

The example is built and solved as follows.

```
Net gallery_mm1_feedback() {
    const double p = 1.0 / 3.0;
    Net m("M/M/1-Feedback");
    Source source(m, "Source");
    Queue queue(m, "Queue", SchedStrategy::FCFS);
    Sink sink(m, "Sink");
    OpenClass c1(m, "Class1");
    source.set_arrival(c1, D::exp_mean(1.0));
    queue.set_service(c1, D::exp_mean(0.5));
    Routing P;
    P.set(c1, c1, source, queue, 1.0);
    P.set(c1, c1, queue, queue, p);
    P.set(c1, c1, queue, sink, 1.0 - p);
    m.link(P);
    return m;
}
```

Running this edition prints

```
Model: M/M/1-Feedback
```

4.2.48 gallery_mm1_linear

The M/M/1 queue: Poisson arrivals, exponential service, and a single server. In this variant, the model is a linear chain of such stations.

The example is built and solved as follows.

```
Net gallery_mm1_linear(std::size_t n, double umax) {
```

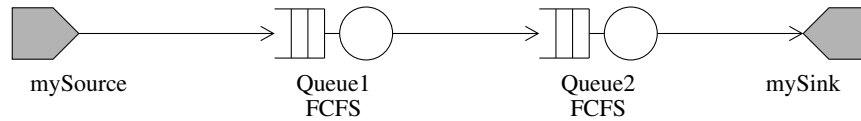


Figure 4.48: Topology of gallery_mm1_linear: an open network over 2 queueing stations.

Nodes	4: Source, Queue $\times$ 2, Sink
Classes	1: myClass (open)
Scheduling	Queue1: FCFS; Queue2: FCFS
Arrivals	mySource – myClass: Exp(1)
Service	Queue1 – myClass: Exp(1.111) Queue2 – myClass: Exp(1.111)

Table 4.49: Features of gallery_mm1_linear.

```
const std::vector<double> means = linear_means(n, umax);
std::vector<D> svc;
for (std::size_t i = 0; i < n; ++i) svc.push_back(D::exp_mean(means[i]));
return linear_chain("M/M/1-Linear", n, Exp(1.0), svc);
```

Running this edition prints

```
Model: M/M/1-Linear
```

4.2.49 gallery_mm1_multiclass

The M/M/1 queue: Poisson arrivals, exponential service, and a single server. In this variant, several open classes share the station.

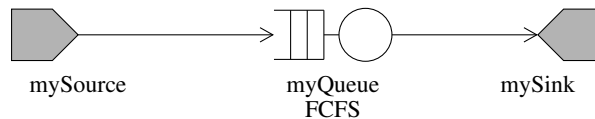


Figure 4.49: Topology of gallery_mm1_multiclass: an open network over 1 queueing station.

The example is built and solved as follows.

```
Net gallery_mm1_multiclass() {
    Net m("M[2]/M[2]/1");
    Source source(m, "mySource");
    Queue queue(m, "myQueue", SchedStrategy::FCFS);
    Sink sink(m, "mySink");
    OpenClass c1(m, "myClass1");
    source.set_arrival(c1, Exp(1.0));
    queue.set_service(c1, Exp(4.0));
    OpenClass c2(m, "myClass2");
    source.set_arrival(c2, Exp(0.5));
    queue.set_service(c2, Exp(4.0));
    Routing P;
    serial(P, c1, {source, queue, sink});
    serial(P, c2, {source, queue, sink});
    m.link(P);
    return m;
}
```

Running this edition prints

```
Model: M[2]/M[2]/1
```

Nodes	3: Source, Queue, Sink
Classes	2: myClass1 (open), myClass2 (open)
Scheduling	myQueue: FCFS
Arrivals	mySource – myClass1: Exp(1); myClass2: Exp(0.5)
Service	myQueue – myClass1: Exp(4); myClass2: Exp(4)

Table 4.50: Features of gallery_mm1_multiclass.

4.2.50 gallery_mm1_prio

The M/M/1 queue: Poisson arrivals, exponential service, and a single server. In this variant, the classes carry priorities, served head-of-line.

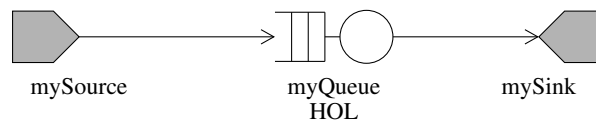


Figure 4.50: Topology of gallery_mm1_prio: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	2: myClass1 (open), myClass2 (open)
Scheduling	myQueue: HOL
Arrivals	mySource – myClass1: Exp(1); myClass2: Exp(0.5)
Service	myQueue – myClass1: Exp(4); myClass2: Exp(4)

Table 4.51: Features of gallery_mm1_prio.

The example is built and solved as follows.

```

Net m("M[2]/M[2]/1");
Source source(m, "mySource");
Queue queue(m, "myQueue", SchedStrategy::HOL);
Sink sink(m, "mySink");
OpenClass c1(m, "myClass1", 1);
source.set_arrival(c1, Exp(1.0));
queue.set_service(c1, Exp(4.0));
OpenClass c2(m, "myClass2", 0);
source.set_arrival(c2, Exp(0.5));
queue.set_service(c2, Exp(4.0));
Routing P;
serial(P, c1, {source, queue, sink});
serial(P, c2, {source, queue, sink});
m.link(P);
return m;

```

Running this edition prints

```
Model: M[2]/M[2]/1
```

4.2.51 gallery_mm1_ps

The M/M/1 queue: Poisson arrivals, exponential service, and a single server. In this variant, the station serves under processor sharing rather than FCFS.

The example is built and solved as follows.

```

Net gallery_mm1_ps() {
    return one_queue("M/M/1-PS", "mySource", "myQueue", "mySink", SchedStrategy::PS,
        Exp(1.0), Exp(2.0));
}

```

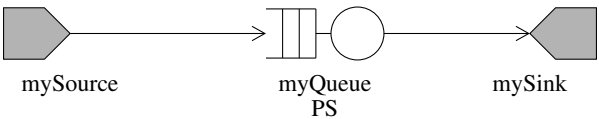


Figure 4.51: Topology of gallery_mm1_ps: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: myClass (open)
Scheduling	myQueue: PS
Arrivals	mySource – myClass: Exp(1)
Service	myQueue – myClass: Exp(2)

Table 4.52: Features of gallery_mm1_ps.

Running this edition prints

```
Model: M/M/1-PS
```

4.2.52 gallery_mm1_ps_feedback

The M/M/1 queue: Poisson arrivals, exponential service, and a single server. In this variant, the station serves under processor sharing rather than FCFS and a completing job returns to the station with fixed probability.

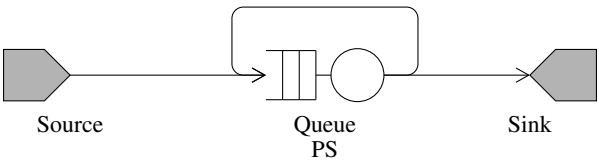


Figure 4.52: Topology of gallery_mm1_ps_feedback: an open network over 1 queueing station.

The example is built and solved as follows.

```
Net gallery_mm1_ps_feedback() {
    const double p = 1.0 / 3.0;
    Net m("M/M/1-PS-Feedback");
    Source source(m, "Source");
    Queue queue(m, "Queue", SchedStrategy::PS);
    Sink sink(m, "Sink");
    OpenClass c1(m, "Class1");
    source.set_arrival(c1, D::exp_mean(1.0));
    queue.set_service(c1, D::exp_mean(0.5));
    Routing P;
    P.set(c1, c1, source, queue, 1.0);
    P.set(c1, c1, queue, queue, p);
    P.set(c1, c1, queue, sink, 1.0 - p);
    m.link(P);
    return m;
}
```

Running this edition prints

```
Model: M/M/1-PS-Feedback
```

Nodes	3: Source, Queue, Sink
Classes	1: Class1 (open)
Scheduling	Queue: PS
Arrivals	Source – Class1: Exp(1)
Service	Queue – Class1: Exp(2)

Table 4.53: Features of gallery_mm1_ps_feedback.

4.2.53 gallery_mm1_ps_multiclass

The M/M/1 queue: Poisson arrivals, exponential service, and a single server. In this variant, the station serves under processor sharing rather than FCFS and several open classes share the station.

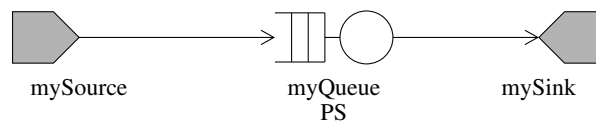


Figure 4.53: Topology of gallery_mm1_ps_multiclass: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	2: myClass1 (open), myClass2 (open)
Scheduling	myQueue: PS
Arrivals	mySource – myClass1: Exp(1); myClass2: Exp(0.5)
Service	myQueue – myClass1: Exp(4); myClass2: Exp(4)

Table 4.54: Features of gallery_mm1_ps_multiclass.

The example is built and solved as follows.

```
Net gallery_mm1_ps_multiclass() {
    Net m("M[2]/M[2]/1-PS");
    Source source(m, "mySource");
    Queue queue(m, "myQueue", SchedStrategy::PS);
    Sink sink(m, "mySink");
    OpenClass c1(m, "myClass1");
    source.set_arrival(c1, Exp(1.0));
    queue.set_service(c1, Exp(4.0));
    OpenClass c2(m, "myClass2");
    source.set_arrival(c2, Exp(0.5));
    queue.set_service(c2, Exp(4.0));
    Routing P;
    serial(P, c1, {source, queue, sink});
    serial(P, c2, {source, queue, sink});
    m.link(P);
    return m;
}
```

Running this edition prints

```
Model: M[2]/M[2]/1-PS
```

4.2.54 gallery_mm1_ps_reentrant

The M/M/1 queue: Poisson arrivals, exponential service, and a single server. In this variant, the station serves under processor sharing rather than FCFS and jobs re-enter the station, so a visit is counted more than once.

The example is built and solved as follows.

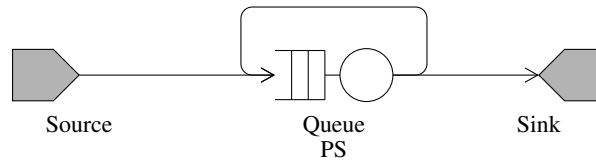


Figure 4.54: Topology of gallery_mm1_ps_reentrant: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	2: Class1 (open), Class2 (open)
Scheduling	Queue: PS
Arrivals	Source – Class1: Exp(1); Class2: Disabled
Service	Queue – Class1: Exp(2); Class2: Exp(3)
Features	class switching on 1 link

Table 4.55: Features of gallery_mm1_ps_reentrant.

```
Net gallery_mm1_ps_reentrant() {
    return reentrant("M/M/1", SchedStrategy::PS, Exp(1.0), Exp(2.0),
        Exp(3.0));
}
```

Running this edition prints

```
Model: M/M/1
```

4.2.55 gallery_mm1_reentrant

The M/M/1 queue: Poisson arrivals, exponential service, and a single server. In this variant, jobs re-enter the station, so a visit is counted more than once.

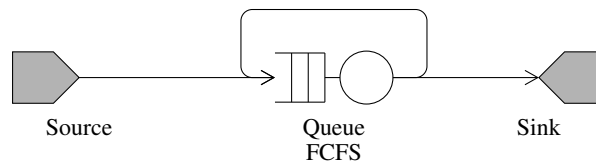


Figure 4.55: Topology of gallery_mm1_reentrant: an open network over 1 queueing station.

The example is built and solved as follows.

```
Net gallery_mm1_reentrant() {
    return reentrant("M/M/1-Reentrant", SchedStrategy::FCFS, Exp(1.0), Exp(2.0),
        Exp(3.0));
}
```

Running this edition prints

```
Model: M/M/1-Reentrant
```

4.2.56 gallery_mm1_tandem

Two M/M/1 queues in series.

The example is built and solved as follows.

```
Net gallery_mm1_tandem() { return gallery_mm1_linear(2); }
```

Running this edition prints

Nodes	3: Source, Queue, Sink
Classes	2: Class1 (open), Class2 (open)
Scheduling	Queue: FCFS
Arrivals	Source – Class1: Exp(1); Class2: Disabled
Service	Queue – Class1: Exp(2); Class2: Exp(3)
Features	class switching on 1 link

Table 4.56: Features of gallery_mm1_reentrant.

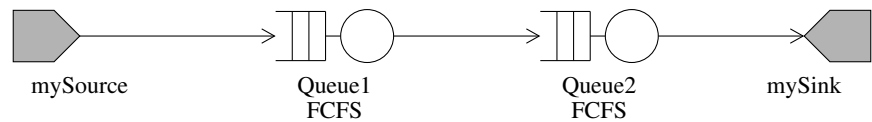


Figure 4.56: Topology of gallery_mm1_tandem: an open network over 2 queueing stations.

Model: M/M/1-Linear

4.2.57 gallery_mm1_tandem_multiclass

Two M/M/1 queues in series under a multiclass workload.
The example is built and solved as follows.

```
Net gallery_mm1_tandem_multiclass() {
    Net m("M[2]/M[2]/1 -> -/M[2]/1");
    Source source(m, "Source");
    Queue q1(m, "Queue1", SchedStrategy::FCFS);
    Queue q2(m, "Queue2", SchedStrategy::FCFS);
    Sink sink(m, "mySink");
    OpenClass c1(m, "myClass1");
    source.set_arrival(c1, Exp(1.0));
    q1.set_service(c1, Exp(4.0));
    q2.set_service(c1, Exp(6.0));
    OpenClass c2(m, "myClass2");
    source.set_arrival(c2, Exp(0.5));
    q1.set_service(c2, Exp(2.0));
    q2.set_service(c2, Exp(6.0));
    Routing P;
    serial(P, c1, {source, q1, q2, sink});
    serial(P, c2, {source, q1, q2, sink});
    m.link(P);
    return m;
}
```

Running this edition prints

Model: M[2]/M[2]/1 -> -/M[2]/1

Nodes	4: Source, Queue × 2, Sink
Classes	1: myClass (open)
Scheduling	Queue1: FCFS; Queue2: FCFS
Arrivals	mySource – myClass: Exp(1)
Service	Queue1 – myClass: Exp(1.111) Queue2 – myClass: Exp(1.111)

Table 4.57: Features of gallery_mm1_tandem.

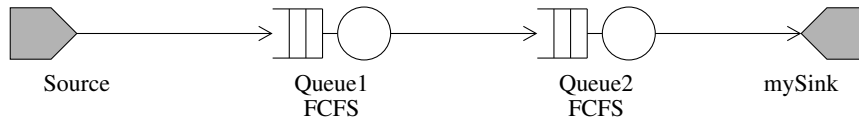


Figure 4.57: Topology of gallery_mm1_tandem_multiclass: an open network over 2 queueing stations.

Nodes	4: Source, Queue $\times$ 2, Sink
Classes	2: myClass1 (open), myClass2 (open)
Scheduling	Queue1: FCFS; Queue2: FCFS
Arrivals	Source – myClass1: Exp(1); myClass2: Exp(0.5)
Service	Queue1 – myClass1: Exp(4); myClass2: Exp(2) Queue2 – myClass1: Exp(6); myClass2: Exp(6)

Table 4.58: Features of gallery_mm1_tandem_multiclass.

4.2.58 gallery_mm1k

The M/M/1/K queue: a single exponential server behind a buffer of K places, so an arrival finding the buffer full is lost.

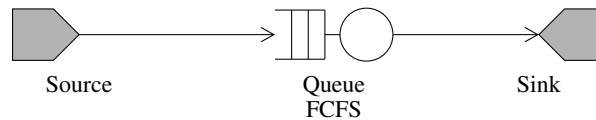


Figure 4.58: Topology of gallery_mm1k: an open network over 1 queueing station.

The example is built and solved as follows.

```
Net gallery_mm1k(double K) {
    Net m("M/M/1/K");
    Source source(m, "Source");
    Queue queue(m, "Queue", SchedStrategy::FCFS);
    Sink sink(m, "Sink");
    queue.set_number_of_servers(1.0);
    queue.set_capacity(K);
    OpenClass oclass(m, "Class1", 0);
    source.set_arrival(oclass, Exp(0.8));
    queue.set_service(oclass, Exp(1.0));
    Routing P;
    P.set(oclass, oclass, source, queue, 1.0);
    P.set(oclass, oclass, queue, sink, 1.0);
    m.link(P);
    return m;
}
```

Running this edition prints

```
Model: M/M/1/K
```

4.2.59 gallery_mmap1

The M/MAP/1 queue: Poisson arrivals, MAP-modulated service, and a single server.

The example is built and solved as follows.

```
Net gallery_mmap1() {
    return one_queue("M/MAP/1", "mySource", "myQueue", "mySink", SchedStrategy::FCFS,
        Exp(1.0), map_of(mam::map_scale(map_rand_23000(), 0.5)));
}
```

Running this edition prints

Nodes	3: Source, Queue, Sink
Classes	1: Class1 (open)
Scheduling	Queue: FCFS
Arrivals	Source – Class1: Exp(0.8)
Service	Queue – Class1: Exp(1)
Features	finite buffer 3 at Queue

Table 4.59: Features of gallery_mm1k.

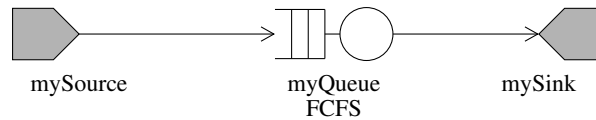


Figure 4.59: Topology of gallery_mm1k: an open network over 1 queueing station.

Model: M/MAP/1

4.2.60 gallery_mm1_multiclass

The M/MAP/1 queue: Poisson arrivals, MAP-modulated service, and a single server. In this variant, several open classes share the station.

The example is built and solved as follows.

```

const mam::Map<double> m1 = mam::map_scale(map_rand_23000(), 0.5);
const mam::Map<double> m2 = mam::map_scale(map_rand_23001_n3(), 0.5);
Net m("M/MAP/1");
Source source(m, "mySource");
Queue queue(m, "myQueue", SchedStrategy::FCFS);
Sink sink(m, "mySink");
OpenClass c1(m, "myClass1");
source.set_arrival(c1, Exp(0.35 / mam::map_mean(m1)));
queue.set_service(c1, map_of(m1));
OpenClass c2(m, "myClass2");
source.set_arrival(c2, Exp(0.15 / mam::map_mean(m2)));
queue.set_service(c2, map_of(m2));
Routing P;
serial(P, c1, {source, queue, sink});
serial(P, c2, {source, queue, sink});
m.link(P);
return m;

```

Running this edition prints

Model: M/MAP/1

4.2.61 gallery_mm1k

The M/MAP/ k queue: Poisson arrivals, MAP-modulated service, and k parallel servers.

Nodes	3: Source, Queue, Sink
Classes	1: myClass (open)
Scheduling	myQueue: FCFS
Arrivals	mySource – myClass: Exp(1)
Service	myQueue – myClass: MAP

Table 4.60: Features of gallery_mm1k.

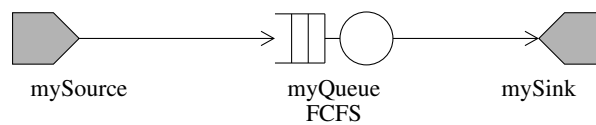


Figure 4.60: Topology of `gallery_mmap1_multiclass`: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	2: myClass1 (open), myClass2 (open)
Scheduling	myQueue: FCFS
Arrivals	mySource – myClass1: Exp(0.7); myClass2: Exp(0.3)
Service	myQueue – myClass1: MAP; myClass2: MAP

Table 4.61: Features of `gallery_mmap1_multiclass`.

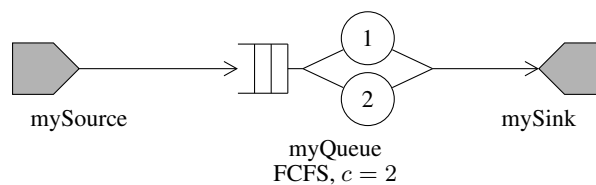


Figure 4.61: Topology of `gallery_mmapk`: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: myClass (open)
Scheduling	myQueue: FCFS ($c = 2$)
Arrivals	mySource – myClass: Exp(1)
Service	myQueue – myClass: MAP

Table 4.62: Features of `gallery_mmapk`.

The example is built and solved as follows.

```
Net gallery_mmapk(double k) {
    return one_queue("M/MAP/k", "mySource", "myQueue", "mySink", SchedStrategy::FCFS,
        Exp(1.0), map_of(map_rand_23000()), k);
}
```

Running this edition prints

```
Model: M/MAP/k
```

4.2.62 gallery_mmk

The M/M/k queue: Poisson arrivals, exponential service, and k parallel servers.

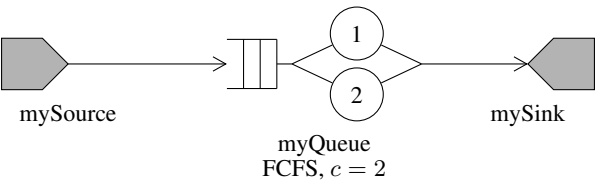


Figure 4.62: Topology of gallery_mmk: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: myClass (open)
Scheduling	myQueue: FCFS ($c = 2$)
Arrivals	mySource – myClass: Exp(1)
Service	myQueue – myClass: Exp(2)

Table 4.63: Features of gallery_mmk.

The example is built and solved as follows.

```
Net gallery_mmk(double k) {
    return one_queue("M/M/k", "mySource", "myQueue", "mySink", SchedStrategy::FCFS,
        Exp(1.0), Exp(2.0), k);
}
```

Running this edition prints

```
Model: M/M/k
```

4.2.63 gallery_mpar1

The M/Pareto/1 queue: Poisson arrivals, Pareto service, and a single server.

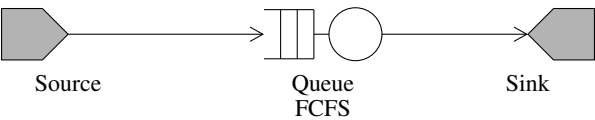


Figure 4.63: Topology of gallery_mpar1: an open network over 1 queueing station.

The example is built and solved as follows.

```
Net gallery_mpar1() {
    Net m("M/Par/1");
    Source source(m, "Source");
    Queue queue(m, "Queue", SchedStrategy::FCFS);
}
```

Nodes	3: Source, Queue, Sink
Classes	1: Class1 (open)
Scheduling	Queue: FCFS
Arrivals	Source – Class1: Exp(1)
Service	Queue – Class1: Pareto(2.008,0.251)

Table 4.64: Features of gallery_mpar1.

```

Sink sink(m, "Sink");
OpenClass c1(m, "Class1");
source.set_arrival(c1, Exp(1.0));
queue.set_service(c1, lang::pareto_fit_mean_scv<double>(0.5, 64.0));
Routing P;
P.set(c1, c1, source, queue, 1.0);
P.set(c1, c1, queue, sink, 1.0);
m.link(P);
return m;

```

Running this edition prints

```
Model: M/Par/1
```

4.2.64 gallery_multitier

A multitier application: presentation, application and database tiers as a closed network.

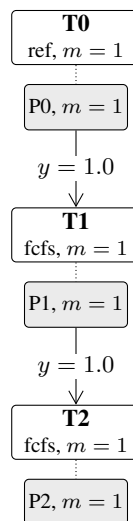


Figure 4.64: Call graph of gallery_multitier: 3 tasks over 3 processors, drawn with the reference task on top and a called task below its caller; the shaded box under each task is the processor it runs on.

Processors	3: P0 (ps, $m = 1$), P1 (ps, $m = 1$), P2 (ps, $m = 1$)
Tasks	3: T0 (ref, $m = 1$), T1 (fcfs, $m = 1$), T2 (fcfs, $m = 1$)
Entries	9: E0, E10, E11, E12, E13, E20, E21, E22, E23
Activities	22, host demands A0: 1.0, A1: 1.0, A2: 1.0, A3: 1.0, B0: 1.0, B1: 1.0, B2: 1.0, B3: 1.0, ...
Calls	8 (8 synchronous, 0 asynchronous)

Table 4.65: Features of gallery_multitier.

The example is built and solved as follows.

```
Lqn b;
multitier_tiers(b);
multitier_client_acts(b);
multitier_app_acts(b);
b.activity("B7a", E(1.0), "T1");
b.activity("B7b", E(1.0), "T1");
b.sync_call("B7b", "E23", 1.0);
b.activity("B8", E(1.0), "T1");
b.replies_to("B8", "E13");

b.activity("C0", E(1.0), "T2");
b.bind_to("C0", "E20");
b.activity("C1", E(1.0), "T2");
b.replies_to("C1", "E20");
b.activity("C2", E(1.0), "T2");
b.bind_to("C2", "E21");
b.replies_to("C2", "E21");
b.activity("C3", E(1.0), "T2");
b.bind_to("C3", "E22");
b.activity("C4", E(1.0), "T2");
b.activity("C5", E(1.0), "T2");
b.replies_to("C5", "E22");
b.activity("C6", E(1.0), "T2");
b.bind_to("C6", "E23");
b.replies_to("C6", "E23");

multitier_shared_precedences(b);
b.or_join({"B7a", "B7b"}, "B8");
return b;
```

Running this edition prints

```
Model: testLQN3
```

4.2.65 gallery_multitier_storage

The multitier application with an explicit storage tier behind the database.

Processors	4: P0 (ps, $m = 1$), P1 (ps, $m = 1$), P2 (ps, $m = 1$), P3 (ps, $m = 1$)
Tasks	4: T0 (ref, $m = 1$), T1 (fcfs, $m = 1$), T2 (fcfs, $m = 1$), T3 (fcfs, $m = 1$)
Entries	10: E0, E10, E11, E12, E13, E20, E21, E22, E23, E3
Activities	24, host demands A0: 1.0, A1: 1.0, A2: 1.0, A3: 1.0, B0: 1.0, B1: 1.0, B2: 1.0, B3: 1.0, ...
Calls	10 (10 synchronous, 0 asynchronous)

Table 4.66: Features of gallery_multitier_storage.

The example is built and solved as follows.

```
const std::size_t totalitems = 10;
const int cachecapacity = 2;

Lqn b;
multitier_tiers(b);
b.processor("P3", 1, SchedStrategy::PS);
b.cache_task("T3", 1, SchedStrategy::FCFS, "P3", totalitems, {cachecapacity},
            lang::ReplacementStrategy::LRU);
b.item_entry("E3", "T3", totalitems, uniform_pmf(totalitems));

multitier_client_acts(b);
multitier_app_acts(b);
b.activity("B7a", E(1.0), "T1");
b.replies_to("B7a", "E13");
b.activity("B7b", E(1.0), "T1");
b.sync_call("B7b", "E23", 1.0);
b.replies_to("B7b", "E13");
```

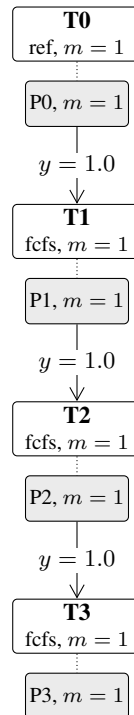


Figure 4.65: Call graph of `gallery_multitier_storage`: 4 tasks over 4 processors, drawn with the reference task on top and a called task below its caller; the shaded box under each task is the processor it runs on.

```

b.activity("C0", E(1.0), "T2");
b.bound_to("C0", "E20");
b.activity("C1", E(1.0), "T2");
b.sync_call("C1", "E3", 1.0);
b.replies_to("C1", "E20");
b.activity("C2", E(1.0), "T2");
b.bound_to("C2", "E21");
b.sync_call("C2", "E3", 1.0);

... (14 source lines omitted; full implementation: cpp/examples/gallery/)

b.activity("D1b", E(0.5), "T3");
b.replies_to("D1b", "E3");

multitier_shared_precedences(b);
b.cache_access("D0", "D1a", "D1b");
return b;

```

Running this edition prints

```
Model: testLQN3_Cache
```

4.2.66 gallery_parm1

The Pareto/M/1 queue: Pareto interarrival times, exponential service, and a single server. The example is built and solved as follows.

```

Net gallery_parm1() {
    Net m("Par/M/1");
    Source source(m, "Source");
    Queue queue(m, "Queue", SchedStrategy::FCFS);
    Sink sink(m, "Sink");
}

```

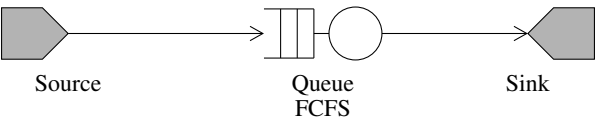


Figure 4.66: Topology of gallery_parm1: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: Class1 (open)
Scheduling	Queue: FCFS
Arrivals	Source – Class1: Pareto(2.008,0.5019)
Service	Queue – Class1: Exp(2)

Table 4.67: Features of gallery_parm1.

```
OpenClass c1(m, "Class1");
source.set_arrival(c1, lang::pareto_fit_mean_scv<double>(1.0, 64.0));
queue.set_service(c1, Exp(2.0));
Routing P;
P.set(c1, c1, source, queue, 1.0);
P.set(c1, c1, queue, sink, 1.0);
m.link(P);
return m;
```

Running this edition prints

```
Model: Par/M/1
```

4.2.67 gallery_qn_random

A randomly generated queueing network, used as a generator-driven parity fixture.

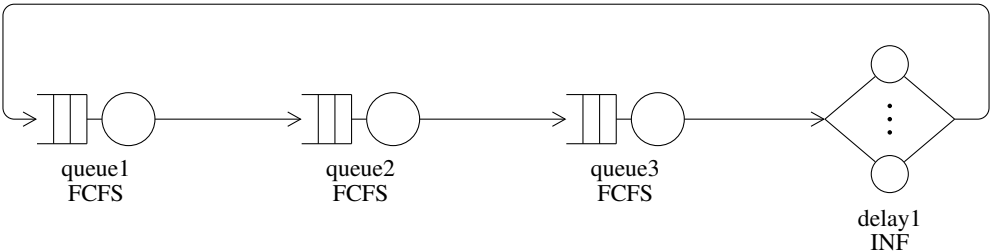


Figure 4.67: Topology of gallery_qn_random: a closed network over 4 queueing stations.

The example is built and solved as follows.

```
Net m("nw");
Queue q1(m, "queue1", SchedStrategy::FCFS);
Queue q2(m, "queue2", SchedStrategy::FCFS);
Queue q3(m, "queue3", SchedStrategy::FCFS);
Delay d1(m, "delay1");

ClosedClass c1(m, "CClass1", 14, q2, 0);
ClosedClass c2(m, "CClass2", 11, q2, 0);
const std::size_t nodes[4] = {q1, q2, q3, d1};
for (std::size_t i = 0; i < 4; ++i) {
    m.set_service(nodes[i], c1, Exp(1.0));
    m.set_service(nodes[i], c2, Exp(1.0));
}

Routing P;
cyclic(P, c1, {q1, q2, q3, d1});
```

Nodes	4: Queue $\times$ 3, Delay
Classes	2: CClass1 (closed, $N = 14$), CClass2 (closed, $N = 11$)
Scheduling	queue1: FCFS; queue2: FCFS; queue3: FCFS; delay1: INF
Service	queue1 – CClass1: Exp(1); CClass2: Exp(1) queue2 – CClass1: Exp(1); CClass2: Exp(1) queue3 – CClass1: Exp(1); CClass2: Exp(1) delay1 – CClass1: Exp(1); CClass2: Exp(1)

Table 4.68: Features of gallery_qn_random.

```
cyclic(P, c2, {q1, q2, q3, dl});
m.link(P);
return m;
```

Running this edition prints

```
Model: nw
```

4.2.68 gallery_renv_breakdown

A station in a random environment that alternates between an operational and a failed stage.

No topology is drawn for this example: the captured run yielded no `Network` or `LayeredNetwork` to draw one from, either because the script builds a model of another kind (an environment or a workflow) or because it builds it inside a helper it does not expose.

The example is built and solved as follows.

```
Net model("ServerWithFailures");
Source source(model, "Arrivals");
Queue queue(model, "Server", SchedStrategy::FCFS);
Sink sink(model, "Departures");
OpenClass jobs(model, "Jobs", 0);
source.set_arrival(jobs, Exp(0.8));
queue.set_service(jobs, Exp(2.0));
queue.set_number_of_servers(1.0);
Routing P;
P.set(jobs, jobs, source, queue, 1.0);
P.set(jobs, jobs, queue, sink, 1.0);
model.link(P);

env::Environment<double> e("ServerEnv", 2);
e.add_node_failure_repair(0, 1, model.get_struct(), "Server", Exp(0.1),
                        Exp(1.0), Exp(0.5));
e.init();
return e;
```

Running this edition prints

```
Model: ServerEnv
```

4.2.69 gallery_repairmen

The machine repairman model: machines think, fail, and queue for one repair station.

Nodes	2: Queue, Delay
Classes	1: Class1 (closed, $N = 8$)
Scheduling	Queue1: PS; Delay1: INF
Service	Queue1 – Class1: Exp(0.7685) Delay1 – Class1: Exp(0.5)

Table 4.69: Features of gallery_repairmen.

The example is built and solved as follows.

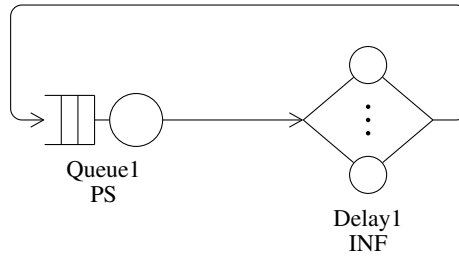


Figure 4.68: Topology of gallery_repairmen: a closed network over 2 queueing stations.

```
const double M = 1.0;
PyRandom rng(seed);
Net m("Finite repairmen CQN");
Queue queue(m, "Queue1", SchedStrategy::PS);
queue.set_number_of_servers(nservers);
Delay delay(m, "Delay1");

const double u_pop = rng.random();
ClosedClass c1(m, "Class1", py_round(u_pop * 10.0 * M + 3.0), queue, 0);
const double u_svc = rng.random();
queue.set_service(c1, D::exp_mean(u_svc + 1.0));
delay.set_service(c1, D::exp_mean(2.0));

Routing P;
cyclic(P, c1, {queue, delay});
m.link(P);
return m;
```

Running this edition prints

```
Model: Finite repairmen CQN
```

4.2.70 gallery_replayerm1

A single queue whose interarrival times are replayed from a measurement trace.

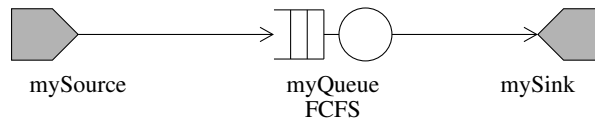


Figure 4.69: Topology of gallery_replayerm1: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: myClass (open)
Scheduling	myQueue: FCFS
Arrivals	mySource – myClass: Replayer(trace)
Service	myQueue – myClass: Exp(29.68)

Table 4.70: Features of gallery_replayerm1.

The example is built and solved as follows.

```
const std::vector<double> trace =
    read_trace(example_data_file("gettingstarted/example_trace.txt"));
const D replayer = D::replayer(trace);
return one_queue("Trace/M/1", "mySource", "myQueue", "mySink", SchedStrategy::FCFS, replayer,
    Exp(3.0 / replayer.mean));
```

Running this edition prints

```
Model: Trace/M/1
```

4.2.71 gallery_um1

The U/M/1 queue: uniform interarrival times, exponential service, and a single server.

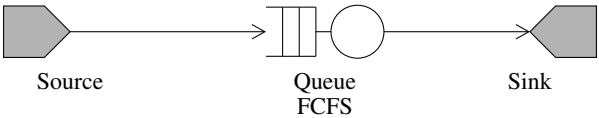


Figure 4.70: Topology of gallery_um1: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: Class1 (open)
Scheduling	Queue: FCFS
Arrivals	Source – Class1: Uniform(1,2)
Service	Queue – Class1: Exp(2)

Table 4.71: Features of gallery_um1.

The example is built and solved as follows.

```
Net gallery_um1() {
    Net m("U/M/1");
    Source source(m, "Source");
    Queue queue(m, "Queue", SchedStrategy::FCFS);
    Sink sink(m, "Sink");
    OpenClass c1(m, "Class1");
    source.set_arrival(c1, Uniform(1.0, 2.0));
    queue.set_service(c1, Exp(2.0));
    Routing P;
    P.set(c1, c1, source, queue, 1.0);
    P.set(c1, c1, queue, sink, 1.0);
    m.link(P);
    return m;
}
```

Running this edition prints

```
Model: U/M/1
```

Each of these is also a parity fixture: the four codebases build the same model and the harness compares the metrics solver by solver, so a divergence in any one of them is caught here before it reaches a larger model.

Chapter 5

Discrete-time models

In a slotted model time advances in slots of fixed length, and every sampled interarrival and service time must fall on the slot lattice. A sample that does not is an error rather than something to round: refusing is what keeps a discrete-time result exact. Within a slot the events are ordered by a fixed phase – completions, internal movement, arrivals, bookkeeping – so that a departure and an arrival in the same slot cannot be confused.

Geometric interarrival and service times give the Geo/Geo/1 queue, whose exact closed form is known, and it is against that form that the solvers are checked. The models live under `cpp/examples/discrete/`, and slotted mode is requested through the solver options rather than by the model.

```
const double a = 0.2, s = 0.5;
Net m("GeoGeo1");
const std::size_t source = m.add_source("Source");
const std::size_t queue = m.add_queue("Queue", SchedStrategy::FCFS);
const std::size_t sink = m.add_sink("Sink");
const std::size_t cl = m.add_open_class("Class1", 0);
m.set_arrival(source, cl, D::geometric(a));
m.set_service(queue, cl, D::geometric(s));
link_serial(m, {source, queue, sink});

run_nc_slotted(m.get_struct());
std::printf("closed form: E[N] = %g, E[T] = %g slots\n",
            a * (1 - a) / (s - a), (1 - a) / (s - a));
```

5.1 The slotted models

5.1.1 dt_bernoulli_loaddep

A load-dependent Bernoulli server, $p(n) = s \min(n, c)$, the discrete-time multiserver approximation.

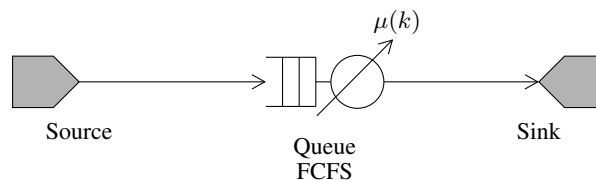


Figure 5.1: Topology of `dt_bernoulli_loaddep`: an open network over 1 queueing station.

The example is built and solved as follows.

```
const double a = 0.6, s = 0.3;
const std::size_t c = 3, L = 20;
```

Nodes	3: Source, Queue, Sink
Classes	1: Class1 (open)
Scheduling	Queue: FCFS
Arrivals	Source – Class1: Geometric(0.6)
Service	Queue – Class1: Geometric(0.3)
Features	finite buffer 20 at Queue; load-dependent rates at Queue
Solvers	NC

Table 5.1: Features of dt_bernoulli_loaddep.

```

Net m("LoadDepBernoulli");
Source source(m, "Source");
Queue queue(m, "Queue", SchedStrategy::FCFS);
Sink sink(m, "Sink");
OpenClass cl(m, "Class1", 0);
source.set_arrival(cl, Geometric(a));
queue.set_service(cl, Geometric(s));
queue.set_capacity(static_cast<double>(L));
std::vector<double> alpha(L);
for (std::size_t n = 1; n <= L; ++n) alpha[n - 1] = static_cast<double>(std::min(n, c));
queue.set_load_dependence(alpha);
link_serial(m, {source, queue, sink});

section("NC (slotted)");
run_nc_slotted(m.get_struct());

std::vector<double> p(L);
for (std::size_t n = 1; n <= L; ++n) p[n - 1] = s * static_cast<double>(std::min(n, c));
const dqsys::BernoulliResult<double> r =
    dqsys::dqsys_bernoulli1(std::vector<double>(1, a), p, L);
print_vec("time-stationary law pi(0..6) :", r.pmf, 7);
print_vec("arrival law pi_1(0..6):", r.arrivalPmf, 7);
std::printf("no PASTA in discrete time: the two rows above are different laws\n");

```

Running this edition prints

```

SOLVER: NC (slotted)
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
Tput
Source       Class1              0          0          0          0          0
0.6
Queue        Class1      2.0644      0.95402      3.4406      3.4406      0.6
0.6
time-stationary law pi(0..6) : 0.045977 0.229885 0.402299 0.268199 0.044700 0.007450 0.001242
arrival law pi_1(0..6): 0.114943 0.402299 0.402299 0.067050 0.011175 0.001862 0.000310
no PASTA in discrete time: the two rows above are different laws

```

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by NC.

5.1.2 dt_cycle

A closed cycle of slotted stations.

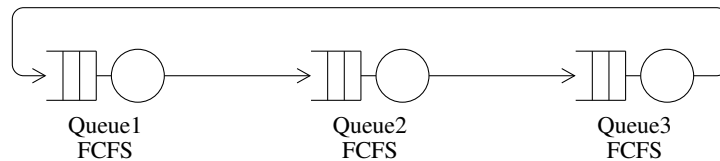


Figure 5.2: Topology of dt_cycle: a closed network over 3 queueing stations.

The example is built and solved as follows.

Nodes	3: Queue $\times$ 3
Classes	1: Jobs (closed, $N = 5$)
Scheduling	Queue1: FCFS; Queue2: FCFS; Queue3: FCFS
Service	Queue1 – Jobs: Geometric(0.5) Queue2 – Jobs: Geometric(0.25) Queue3 – Jobs: Geometric(0.7)
Solvers	NC

Table 5.2: Features of dt_cycle.

```

const std::vector<double> p = cycle_rates();
const std::size_t N = 5;
Net m("BernoulliCycle");
std::vector<std::size_t> station;
for (std::size_t j = 0; j < p.size(); ++j) {
    station.push_back(m.add_queue("Queue" + std::to_string(j + 1), SchedStrategy::FCFS));
}
ClosedClass c1(m, "Jobs", static_cast<double>(N), station[0], 0);
for (std::size_t j = 0; j < p.size(); ++j) m.set_service(station[j], c1, Geometric(p[j]));
link_serial(m, station);

section("NC (slotted)");
run_nc_slotted(m.get_struct());

const dpfqnc::DtNcResult<double> nc = dpfqnc::dpfqnc_nc(p, N);
std::printf("log G(N,J)   = %g\n", nc.lG);
std::printf("throughput   = %g jobs per slot\n", nc.throughput());
std::printf("utilizations =");
for (std::size_t j = 0; j < p.size(); ++j) std::printf(" %.6f", nc.throughput() / p[j]);
std::printf("\n");

```

Running this edition prints

SOLVER: NC (slotted)	Station	JobClass	QLen	Util	RespT	ResidT	ArvR
	Queue1	Jobs	0.72191	0.49656	2.9076	2.9076	0.24828
	Queue2	Jobs	3.8668	0.99313	15.574	15.574	0.24828
	Queue3	Jobs	0.41125	0.35469	1.6564	1.6564	0.24828
log G(N,J) = 6.90882							
throughput = 0.248282 jobs per slot							
utilizations = 0.496564 0.993128 0.354689							

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by NC.

5.1.3 dt_cycle_loaddep

The cycle with a load-dependent slotted server.

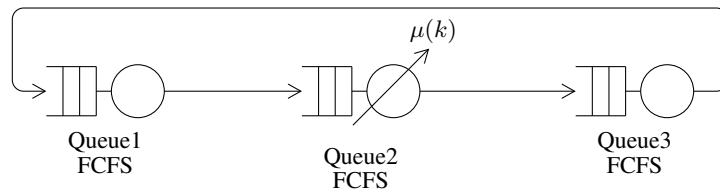


Figure 5.3: Topology of dt_cycle_loaddep: a closed network over 3 queueing stations.

The example is built and solved as follows.

Nodes	3: Queue $\times$ 3
Classes	1: Jobs (closed, $N = 5$)
Scheduling	Queue1: FCFS; Queue2: FCFS; Queue3: FCFS
Service	Queue1 – Jobs: Geometric(0.5) Queue2 – Jobs: Geometric(0.25) Queue3 – Jobs: Geometric(0.7)
Features	load-dependent rates at Queue2
Solvers	NC

Table 5.3: Features of dt_cycle_loaddep.

```

const std::vector<double> p = cycle_rates();
const std::size_t N = 5;
Net m("BernoulliCycleLD");
std::vector<std::size_t> station;
for (std::size_t j = 0; j < p.size(); ++j) {
    station.push_back(m.add_queue("Queue" + std::to_string(j + 1), SchedStrategy::FCFS));
}
ClosedClass c1(m, "Jobs", static_cast<double>(N), station[0], 0);
for (std::size_t j = 0; j < p.size(); ++j) m.set_service(station[j], c1, Geometric(p[j]));
std::vector<double> alpha(N);
for (std::size_t n = 1; n <= N; ++n) alpha[n - 1] = static_cast<double>(std::min<std::size_t>
    >(n, 2));
m.set_load_dependence(station[1], alpha);
link_serial(m, station);

section("NC (slotted)");
run_nc_slotted(m.get_struct());

std::vector<std::vector<double> > P(p.size(), std::vector<double>(N));
for (std::size_t j = 0; j < p.size(); ++j) {
    for (std::size_t n = 1; n <= N; ++n) {
        P[j][n - 1] = (j == 1) ? p[j] * static_cast<double>(std::min<std::size_t>(n, 2)) : p[
        j];
    }
}
const dpfqnc::DtNcLdResult<double> nc = dpfqnc::dpfqnc_ncld(P, N);
print_vec("P(X_2 = 0..N) =", nc.marginal(1), N + 1);

```

Running this edition prints

SOLVER: NC (slotted)	Station	JobClass	QLen	Util	RespT	ResidT	ArvR
	Tput						
	Queue1	Jobs	1.7936	0.84515	4.2445	4.2445	0.42258
	0.42258						
	Queue2	Jobs	2.3912	0.94757	5.6586	5.6586	0.42258
	0.42258						
	Queue3	Jobs	0.81523	0.60368	1.9292	1.9292	0.42258
	0.42258						
P(X_2 = 0..N) = 0.052433 0.204833 0.290108 0.250111 0.156786 0.045729							

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by NC.

5.1.4 dt_cycle_multiclass

The multiclass cycle.

The example is built and solved as follows.

```

const std::vector<double> p = cycle_rates();
const std::size_t N1 = 3, N2 = 2;
Net m("BernoulliCycleMC");
std::vector<std::size_t> station;
for (std::size_t j = 0; j < p.size(); ++j) {
    station.push_back(m.add_queue("Queue" + std::to_string(j + 1), SchedStrategy::FCFS));
}

```

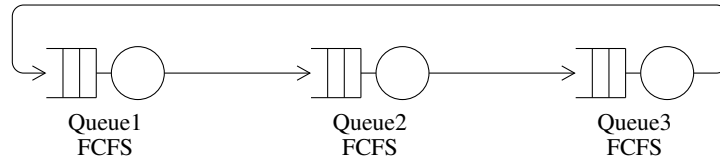


Figure 5.4: Topology of dt_cycle_multiclass: a closed network over 3 queueing stations.

Nodes	3: Queue $\times$ 3
Classes	2: Chain1 (closed, $N = 3$), Chain2 (closed, $N = 2$)
Scheduling	Queue1: FCFS; Queue2: FCFS; Queue3: FCFS
Service	Queue1 – Chain1: Geometric(0.5); Chain2: Geometric(0.5) Queue2 – Chain1: Geometric(0.25); Chain2: Geometric(0.25) Queue3 – Chain1: Geometric(0.7); Chain2: Geometric(0.7)
Solvers	NC

Table 5.4: Features of dt_cycle_multiclass.

```

ClosedClass c1(m, "Chain1", static_cast<double>(N1), station[0], 0);
ClosedClass c2(m, "Chain2", static_cast<double>(N2), station[0], 0);
for (std::size_t j = 0; j < p.size(); ++j) {
    m.set_service(station[j], c1, Geometric(p[j]));
    m.set_service(station[j], c2, Geometric(p[j]));
}
// Both chains follow the same cycle and never switch class.
Routing P;
const std::size_t J = station.size();
for (std::size_t j = 0; j < J; ++j) {
    P.set(c1, c1, station[j], station[(j + 1) % J], 1.0);
    P.set(c2, c2, station[j], station[(j + 1) % J], 1.0);
}
m.link(P);

section("NC (slotted)");
run_nc_slotted(m.get_struct());

const dpfqnc::DtNcResult<double> nc = dpfqnc::dpfqnc_nc(p, N1 + N2);
const double X = nc.throughput();
std::printf("aggregate throughput = %g jobs per slot\n", X);
std::printf("per-chain throughput = %g and %g\n",
    X * N1 / (N1 + N2), X * N2 / (N1 + N2));

```

Running this edition prints

Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Queue1	Chain1	0.43315	0.29794	2.9076	2.9076	0.14897
Queue1	Chain2	0.28876	0.19863	2.9076	2.9076	0.099313
Queue2	Chain1	2.3201	0.59588	15.574	15.574	0.14897
Queue2	Chain2	1.5467	0.39725	15.574	15.574	0.099313
Queue3	Chain1	0.24675	0.21281	1.6564	1.6564	0.14897
Queue3	Chain2	0.1645	0.14188	1.6564	1.6564	0.099313
aggregate throughput = 0.248282 jobs per slot						
per-chain throughput = 0.148969 and 0.0993128						

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by NC.

5.1.5 dt_geogeo1

The Geo/Geo/1 queue: Bernoulli arrivals and Bernoulli service on the slot lattice, against the closed form of Daduna.

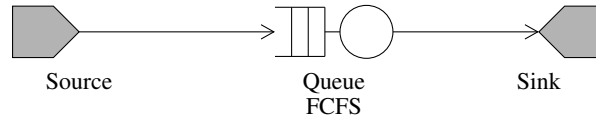


Figure 5.5: Topology of dt_geogeo1: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: Class1 (open)
Scheduling	Queue: FCFS
Arrivals	Source – Class1: Geometric(0.2)
Service	Queue – Class1: Geometric(0.5)
Solvers	NC

Table 5.5: Features of dt_geogeo1.

The example is built and solved as follows.

```

const double a = 0.2, s = 0.5;
Net m("GeoGeo1");
Source source(m, "Source");
Queue queue(m, "Queue", SchedStrategy::FCFS);
Sink sink(m, "Sink");
OpenClass c1(m, "Class1", 0);
source.set_arrival(c1, Geometric(a));
queue.set_service(c1, Geometric(s));
link_serial(m, {source, queue, sink});

section("NC (slotted)");
run_nc_slotted(m.get_struct());
std::printf("closed form: E[N] = %g, E[T] = %g slots\n",
            a * (1 - a) / (s - a), (1 - a) / (s - a));

```

Running this edition prints

```

SOLVER: NC (slotted)
Station  JobClass      QLen      Util      RespT      ResidT      ArrR
Tput
Source   Class1            0          0          0          0          0
0.2
Queue    Class1          0.53333    0.4        2.6667     2.6667     0.2
0.2
closed form: E[N] = 0.533333, E[T] = 2.66667 slots

```

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by NC.

5.1.6 dt_geogeo1_loss

The Geo/Geo/1/L loss system, and the loss probability.

The example is built and solved as follows.

```

const double a = 0.2, s = 0.5;
const std::size_t L = 4;
Net m("GeoGeo1L");
Source source(m, "Source");
Queue queue(m, "Queue", SchedStrategy::FCFS);
Sink sink(m, "Sink");

```

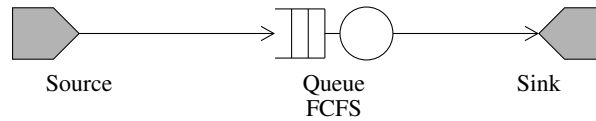



Figure 5.6: Topology of dt_geogeol_loss: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: Class1 (open)
Scheduling	Queue: FCFS
Arrivals	Source – Class1: Geometric(0.2)
Service	Queue – Class1: Geometric(0.5)
Features	finite buffer 4 at Queue
Solvers	NC

Table 5.6: Features of dt_geogeol_loss.

```

OpenClass c1(m, "Class1", 0);
source.set_arrival(c1, Geometric(a));
queue.set_service(c1, Geometric(s));
queue.set_capacity(static_cast<double>(L));
link_serial(m, {source, queue, sink});

section("NC (slotted)");
run_nc_slotted(m.get_struct());

const dqsys::Bernoulli1Result<double> r =
    dqsys::dqsys_bernoulli1(std::vector<double>(1, a), std::vector<double>(1, s), L);
print_vec("queue length law   :", r.pmf, L + 1);
std::printf("loss probability   : %g\n", r.lossProb);
std::printf("carried throughput : %g of the %g offered per slot\n", r.throughput, a);

```

Running this edition prints

SOLVER: NC (slotted)						
Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Source	Class1	0	0	0	0	0
0.2						
Queue	Class1	0.52256	0.3985	2.6226	2.6226	0.2
0.19925						
queue length law : 0.601504 0.300752 0.075188 0.018797 0.003759						
loss probability : 0.0037594						
carried throughput : 0.199248 of the 0.2 offered per slot						

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by NC.

Superposing two slotted streams produces batches, not a slotted stream of the same kind, so a tandem of slotted queues is the first model in which a discrete-time implementation can go wrong while the single queue still looks right. dt_cycle exists for that reason.

Chapter 6

Solver-specific examples

Most examples in this manual are written around a model and let the solver vary. The scripts of this chapter do the opposite: they are written around one solver, and print what it builds rather than only what it concludes. They live under `cpp/examples/solvers/`.

The CTMC solver is the natural subject, because its intermediate objects are the model: the state space, the infinitesimal generator and the stationary distribution are all printable, and the averages are read off them. Printing them is how one checks that a model means what it was intended to mean.

6.0.1 `ctmc_spn_mm1`

The same M/M/1 system written as a queueing model and as a Petri net, and the two generators compared.

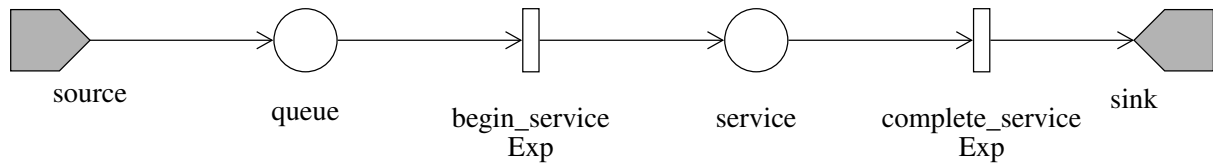


Figure 6.1: Topology of `ctmc_spn_mm1`: a stochastic Petri net over 2 places and 2 transitions; a place is a circle holding its initial marking and a transition a bar, hollow when its firing time is random and solid when it fires at once; the net is open, fed by a Source and drained into a Sink.

Nodes	6: Source, Place $\times$ 2, Transition $\times$ 2, Sink
Classes	1: jobs (open)
Arrivals	source – jobs: Exp(0.8)
Features	2 transitions with 1 firing mode each
Solvers	CTMC

Table 6.1: Features of `ctmc_spn_mm1`.

The example is built and solved as follows.

```
rule('=', 70);
note("M/M/1 Queue as Stochastic Petri Net (SPN)");
rule('=', 70);

Net m("spn_mm1");

// SPN Components: a Source for open arrivals, a Sink for departures
Source source(m, "source");
Sink sink(m, "sink");
```

```
// SPN places represent queue states
Place p_queue(m, "queue"); // Customers waiting
Place p_service(m, "service"); // Customer in service

// Job class
OpenClass jobclass(m, "jobs");

// Define arrival process (lambda = 0.8, mean = 1.25)
source.set_arrival(jobclass, D::exp_mean(1.0 / 0.8));
p_queue.set_service(jobclass, Exp(1.0));
p_service.set_service(jobclass, Exp(1.0));

// "begin_service": 1 customer in queue, service empty; queue -> service.
// "complete_service": 1 customer in service; remove it, and route to the sink.
const std::size_t nnodes = 6;
const std::size_t t_begin =

    ... (63 source lines omitted; full implementation: cpp/examples/solvers/)

rule('=', 70);
std::printf("CTMC analyzes M/M/1 modeled as SPN\n");
std::printf("SPN feature support in CTMC is working\n");
std::printf("  Differences from theory are due to CTMC state space truncation\n");
std::printf("    (open networks with infinite arrivals truncated at cutoff=%zu)\n", cutoff);
rule('=', 70);
```

Running this edition prints

```
=====
M/M/1 Queue as Stochastic Petri Net (SPN)
=====

Solving with CTMC solver...
(Note: open network with CTMC uses state space truncation)

CTMC Results for SPN M/M/1 (441 states, cutoff 10):
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
Tput
source      jobs            0          0          0          0          0
0.77645
queue      jobs          3.2091     3.2091     4.1331     4.1331     0.77645
0.77645
service     jobs          2.9253     2.9253     3.7675     3.7675     0.77645
0.77645

=====
Key Metrics:
=====

... (13 captured-output lines omitted; rerun the source example for the full output)

=====
Summary
=====
CTMC analyzes M/M/1 modeled as SPN
SPN feature support in CTMC is working
  Differences from theory are due to CTMC state space truncation
  (open networks with infinite arrivals truncated at cutoff=10)
=====
```

6.0.2 ctmc_spn_vs_nc_closed_qn

A closed network solved exactly by CTMC over its Petri net form and by the normalizing constant method.

The example is built and solved as follows.

```
rule('=', 72);
note("Closed QN: CTMC (SPN) vs NC Solver Comparison");
rule('=', 72);
```

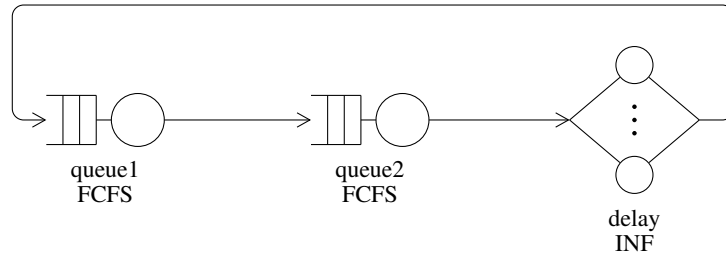


Figure 6.2: Topology of ctmc_spn_vs_nc_closed_qn: a closed network over 3 queueing stations.

Nodes	3: Queue $\times$ 2, Delay
Classes	1: jobs (closed, $N = 3$)
Scheduling	queue1: FCFS; queue2: FCFS; delay: INF
Service	queue1 – jobs: Exp(1) queue2 – jobs: Exp(0.8) delay – jobs: Exp(0.5)

Table 6.2: Features of ctmc_spn_vs_nc_closed_qn.

```
std::printf("\n");
rule('=', 72);
note("Approach 1: Traditional Closed QN (Solved with NC Solver)");
rule('=', 72);

Net model_qn = closed_qn();
const Sn& sn_qn = model_qn.get_struct();
std::printf("\nSolving with NC (Normalizing Constant) solver...\n");
const mva::AvgResult<double> result_nc = nc::solver_nc_run_analyzer(sn_qn, nc::
    NcSolverOptions());
std::printf("NC Results:\n");
print_avg(sn_qn, result_nc);

std::printf("\n");
rule('=', 72);
note("Approach 2: Same Network as SPN (Solved with CTMC)");
rule('=', 72);

Net model_spn = closed_qn_spn();
const Sn& sn_spn = model_spn.get_struct();
std::printf("\nSolving with CTMC solver...\n");
ctmc::CtmcOptions copt;
copt.method = "exact";

... (57 source lines omitted; full implementation: cpp/examples/solvers/)

std::printf(" - Total population conserved (=3 jobs)\n");
std::printf(" - System throughput matches\n");
std::printf(" - Individual station metrics match\n");
std::printf("\nOK CTMC successfully models closed QN as SPN\n");
std::printf("OK Different modeling approaches yield same answers\n");
rule('=', 72);
```

Running this edition prints

```
=====
Closed QN: CTMC (SPN) vs NC Solver Comparison
=====

Approach 1: Traditional Closed QN (Solved with NC Solver)
=====

Solving with NC (Normalizing Constant) solver...
NC Results:
Station      JobClass      QLen      Util      RespT      ResidT      ArrR
  Tput
queue1      jobs          0.80954    0.53644    1.5091     1.5091     0.53644
0.53644
```

```

queue2      jobs      1.1176      0.67055      2.0833      2.0833      0.53644
0.53644
delay       jobs      1.0729      1.0729      2          2          0.53644
0.53644

```

```
=====
```

```
... (49 captured-output lines omitted; rerun the source example for the full output)
```

```

OK Product-form property verified:
- Total population conserved (=3 jobs)
- System throughput matches
- Individual station metrics match

```

```

OK CTMC successfully models closed QN as SPN
OK Different modeling approaches yield same answers
=====

```

6.0.3 ctmc_tandem_mm1

The generator of a two-station tandem, its state space and its stationary distribution, printed in full.

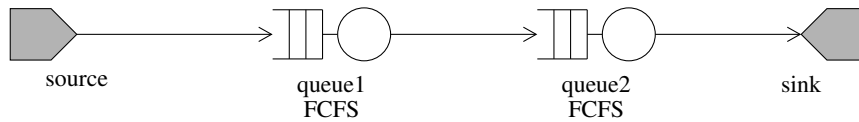


Figure 6.3: Topology of ctmc_tandem_mm1: an open network over 2 queueing stations.

Nodes	4: Source, Queue $\times$ 2, Sink
Classes	1: jobs (open)
Scheduling	queue1: FCFS; queue2: FCFS
Arrivals	source – jobs: Exp(0.6)
Service	queue1 – jobs: Exp(1) queue2 – jobs: Exp(1.2)
Solvers	CTMC

Table 6.3: Features of ctmc_tandem_mm1.

The example is built and solved as follows.

```

rule('=', 70);
note("Tandem M/M/1 Queues (Series of Two Stations)");
rule('=', 70);

Net m("tandem_mm1");
Source source(m, "source");
Queue queue1(m, "queue1", SchedStrategy::FCFS);
Queue queue2(m, "queue2", SchedStrategy::FCFS);
Sink sink(m, "sink");
OpenClass jobclass(m, "jobs");

// External arrivals lambda = 0.6; mu1 = 1.0 and mu2 = 1.2 downstream.
source.set_arrival(jobclass, D::exp_mean(1 / 0.6));
queue1.set_service(jobclass, D::exp_mean(1.0));
queue2.set_service(jobclass, D::exp_mean(1 / 1.2));

Routing P;
serial(P, jobclass, {source, queue1, queue2, sink});
m.link(P);

std::printf("\nSolving tandem M/M/1 system with CTMC...\n");
std::printf("Parameters:\n");
std::printf("  Station 1: lambda=0.6, mu=1.0, rho=0.6\n");
std::printf("  Station 2: lambda=0.6, mu=1.2, rho=0.5\n");

```

```
std::size_t states = 0, cutoff = 0;

... (75 source lines omitted; full implementation: cpp/examples/solvers/)

std::printf("\nCTMC analyzes the tandem M/M/1-M/M/1 system\n");
std::printf("Product-form property verified:\n");
std::printf(" - Each station behaves independently as M/M/1\n");
std::printf(" - System decomposition holds\n");
std::printf("Results match theoretical predictions up to the state-space cutoff\n");
rule('-', 70);
```

Running this edition prints

```
=====
Tandem M/M/1 Queues (Series of Two Stations)
=====

Solving tandem M/M/1 system with CTMC...
Parameters:
  Station 1: lambda=0.6, mu=1.0, rho=0.6
  Station 2: lambda=0.6, mu=1.2, rho=0.5

CTMC Results for Tandem M/M/1 (66 states, cutoff 10):
Station   JobClass      QLen      Util      RespT      ResidT      ArvR
Tput
source      jobs           0           0           0           0           0
0.59619
queue1      jobs          1.4266      0.59619      2.3929      2.3929      0.59619
0.59619
queue2      jobs           0.9728      0.49683      1.6317      1.6317      0.59619
0.59619

=====

... (52 captured-output lines omitted; rerun the source example for the full output)

=====

CTMC analyzes the tandem M/M/1-M/M/1 system
Product-form property verified:
  - Each station behaves independently as M/M/1
  - System decomposition holds
Results match theoretical predictions up to the state-space cutoff
=====
```

Two of the three scripts build the same system twice, once as a queueing model and once as a stochastic Petri net, and compare the generators. That is the strongest available check on either notation: the two descriptions are independent, and if they agree state by state then neither has been misunderstood.

Chapter 7

Model interchange

A model can be written to, and read back from, a JSON document. The format is the interchange between the codebases and the wire protocol of the LDES engine, which is driven entirely by it: the client serializes `model.json`, the engine solves it and returns `result.json`.

The examples under `cpp/examples/json/` are round-trip fixtures: each builds a model that uses one group of wire keys, writes it, reads it back and checks that the reconstructed model has the same structure. What a round trip must preserve is the declared state of the model; what it must not carry is the derived state, which is recomputed on load.

```
Net m = json_hetero_servers(); // a model using one group of wire keys
line::io::write_network_json(m.get_struct(), "model.json");
Net reloaded = line::io::read_network_json<double>("model.json");
```

7.1 The fixtures

7.1.1 json_balking_retrial

Balking thresholds and retrial orbits over the wire.

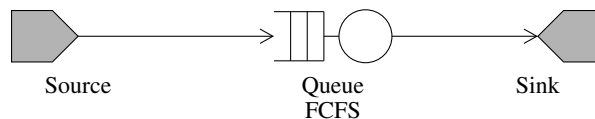


Figure 7.1: Topology of `json_balking_retrial`: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	2: Class1 (open), Class2 (open)
Scheduling	Queue: FCFS
Arrivals	Source – Class1: Exp(1); Class2: Exp(0.5)
Service	Queue – Class1: Exp(2); Class2: Exp(3)
Features	finite buffer 15 at Queue; drop rule {'Class1': 'retrialWithLimit', 'Class2': 'retrialWithLimit'} at Queue

Table 7.1: Features of `json_balking_retrial`.

The example is built and solved as follows.

```
Net m("Balking_Retrial");

Source source(m, "Source");
Queue queue(m, "Queue", SchedStrategy::FCFS);
```

```

Sink sink(m, "Sink");

queue.set_capacity(15.0);

OpenClass c1(m, "Class1");
OpenClass c2(m, "Class2");

source.set_arrival(c1, Exp(1.0));
source.set_arrival(c2, Exp(0.5));
queue.set_service(c1, Exp(2.0));
queue.set_service(c2, Exp(3.0));

// Class1: balking based on queue length
std::vector<qn::Station<double>::BalkingThreshold> thr(2);
thr[0].min_jobs = 5.0;
thr[0].max_jobs = 10.0;
thr[0].probability = 0.3;
thr[1].min_jobs = 11.0;
thr[1].max_jobs = std::numeric_limits<double>::infinity();
thr[1].probability = 1.0;
m.set_balking(queue, c1, lang::BalkingStrategy::QUEUE_LENGTH, thr);

... (9 source lines omitted; full implementation: cpp/examples/json/)

m.link(P);

struct_summary(m);
kv("Balking (Class1)", std::string("QUEUE_LENGTH, [5,10]->0.3, [11,Inf)->1.0"));
kv("Patience (Class1)", std::string("Exp(0.1)"));
kv("Retrial (Class2)", std::string("Exp(0.5), at most 3 attempts"));

```

Running this edition prints

Model:	Balking_Retrial		
Nodes:	3		
Stations:	2		
Classes:	2		
Chains:	2		
Station	Sched	Servers	Capacity
Source	ext	1	Inf
Queue	fcfs	1	15
JobClass	Type	Population	
Class1	open	Inf	
Class2	open	Inf	
Balking (Class1):	QUEUE_LENGTH, [5,10]->0.3, [11,Inf)->1.0		
Patience (Class1):	Exp(0.1)		
Retrial (Class2):	Exp(0.5), at most 3 attempts		

7.1.2 json_classcap_droprule

Per-class capacities and drop rules.

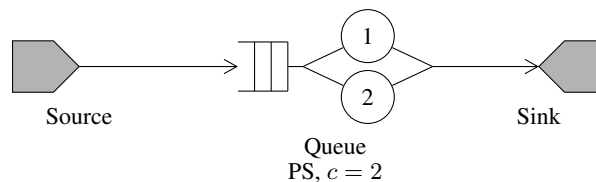


Figure 7.2: Topology of json_classcap_droprule: an open network over 1 queueing station.

The example is built and solved as follows.

```

Net m("ClassCap_DropRule");

Source source(m, "Source");

```


Nodes	3: Source, Queue, Sink
Classes	2: Class1 (open), Class2 (open)
Scheduling	Queue: PS ($c = 2$)
Arrivals	Source – Class1: Exp(1); Class2: Exp(0.5)
Service	Queue – Class1: Exp(3); Class2: Exp(2)
Features	finite buffer 20 at Queue; load-dependent rates at Queue; drop rule {'Class1': 'drop', 'Class2': 'blockingAfterService'} at Queue

Table 7.2: Features of json_classcap_droprule.

```

Queue queue(m, "Queue", SchedStrategy::PS);
Sink sink(m, "Sink");

queue.set_number_of_servers(2.0);
queue.set_capacity(20.0);

OpenClass c1(m, "Class1");
OpenClass c2(m, "Class2");

source.set_arrival(c1, Exp(1.0));
source.set_arrival(c2, Exp(0.5));
queue.set_service(c1, Exp(3.0));
queue.set_service(c2, Exp(2.0));

// Per-class capacity
queue.set_class_capacity(c1, 8.0);
queue.set_class_capacity(c2, 15.0);

// Per-class drop rules. WAITQ is not usable here: no solver honours "wait
// upstream" for an open class at a plain finite capacity, so the capacity
// refresh rejects that combination. BAS is the honoured blocking policy.
queue.set_drop_rule(c1, lang::DropStrategy::DROP);
queue.set_drop_rule(c2, lang::DropStrategy::BAS);

... (7 source lines omitted; full implementation: cpp/examples/json/)

m.link(P);

struct_summary(m);
kv("Class capacity", std::string("Class1=8, Class2=15"));
kv("Drop rule", std::string("Class1=DROP, Class2=BAS"));
kv("Load dependence", std::string("[1, 0.9, 0.8, 0.7]"));

```

Running this edition prints

```

Model:                               ClassCap_DropRule
Nodes:                               3
Stations:                            2
Classes:                             2
Chains:                              2
Station      Sched      Servers      Capacity
Source       ext        1            Inf
Queue        ps         2            20
JobClass     Type       Population
Class1       open       Inf
Class2       open       Inf
Class capacity:      Class1=8, Class2=15
Drop rule:          Class1=DROP, Class2=BAS
Load dependence:     [1, 0.9, 0.8, 0.7]

```

7.1.3 json_fcr_details

The finite capacity region block, with its members, limits and drop rules.
The example is built and solved as follows.

```

Net m("FCR_Details");

```

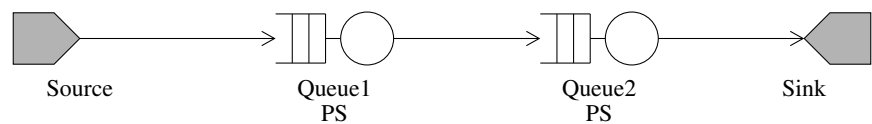


Figure 7.3: Topology of json_fcr_details: an open network over 2 queueing stations.

Nodes	4: Source, Queue × 2, Sink
Classes	2: Class1 (open), Class2 (open)
Scheduling	Queue1: PS; Queue2: PS
Arrivals	Source – Class1: Exp(1); Class2: Exp(0.5)
Service	Queue1 – Class1: Exp(3); Class2: Exp(2) Queue2 – Class1: Exp(4); Class2: Exp(3)

Table 7.3: Features of json_fcr_details.

```
Source source(m, "Source");
Queue q1(m, "Queue1", SchedStrategy::PS);
Queue q2(m, "Queue2", SchedStrategy::PS);
Sink sink(m, "Sink");

OpenClass c1(m, "Class1");
OpenClass c2(m, "Class2");

source.set_arrival(c1, Exp(1.0));
source.set_arrival(c2, Exp(0.5));
q1.set_service(c1, Exp(3.0));
q1.set_service(c2, Exp(2.0));
q2.set_service(c1, Exp(4.0));
q2.set_service(c2, Exp(3.0));

Routing P;
serial(P, c1, {source, q1, q2, sink});
serial(P, c2, {source, q1, q2, sink});
m.link(P);

// Finite Capacity Region: per-class job caps, a global cap, per-class
// weights against that global cap, and a per-job footprint.
const std::size_t fcr =
    m.add_region({q1, q2}, std::vector<double>{8.0, 10.0}, 15.0,

    ... (4 source lines omitted; full implementation: cpp/examples/json/)

struct_summary(m);
kv("Region members", std::string("Queue1, Queue2"));
kv("Region global max jobs", 15.0);
kv("Region class max jobs", std::string("Class1=8, Class2=10"));
kv("Region class weight", std::string("Class1=1, Class2=2"));
kv("Region class size", std::string("Class1=1, Class2=3"));
```

Running this edition prints

Model:	FCR_Details		
Nodes:	4		
Stations:	3		
Classes:	2		
Chains:	2		
Station	Sched	Servers	Capacity
Source	ext	1	Inf
Queue1	ps	1	Inf
Queue2	ps	1	Inf
JobClass	Type	Population	
Class1	open	Inf	
Class2	open	Inf	
Region members:	Queue1, Queue2		
Region global max jobs:	15		
Region class max jobs:	Class1=8, Class2=10		
Region class weight:	Class1=1, Class2=2		

Region class size:	Class1=1, Class2=3
--------------------	--------------------

7.1.4 json_hetero_servers

Heterogeneous server types at one station, their per-type service rates and the policy that orders them.

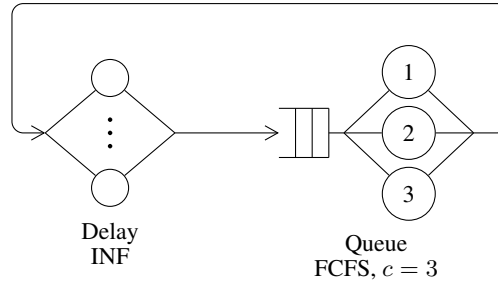


Figure 7.4: Topology of json_hetero_servers: a closed network over 2 queueing stations.

Nodes	2: Queue, Delay
Classes	2: Class1 (closed, $N = 3$), Class2 (closed, $N = 2$)
Scheduling	Delay: INF; Queue: FCFS ($c = 3$)
Service	Delay – Class1: Exp(1); Class2: Exp(2) Queue – Class1: Exp(3); Class2: Exp(2)

Table 7.4: Features of json_hetero_servers.

The example is built and solved as follows.

```
Net m("HeteroServers");

Delay delay(m, "Delay");
Queue queue(m, "Queue", SchedStrategy::FCFS);
queue.set_number_of_servers(3.0);

ClosedClass c1(m, "Class1", 3.0, delay);
ClosedClass c2(m, "Class2", 2.0, delay);

delay.set_service(c1, Exp(1.0));
delay.set_service(c2, Exp(2.0));

// Default service (required before hetero setup)
queue.set_service(c1, Exp(3.0));
queue.set_service(c2, Exp(2.0));

// Heterogeneous server types: Fast serves both classes, Slow only Class1.
qn::Station<double>::ServerType fast;
fast.name = "Fast";
fast.count = 2.0;
fast.compatible = std::vector<bool>{true, true};
fast.service = std::vector<D>{Exp(5.0), Exp(3.0)};
qn::Station<double>::ServerType slow;
slow.name = "Slow";
slow.count = 1.0;
slow.compatible = std::vector<bool>{true, false};

... (9 source lines omitted; full implementation: cpp/examples/json/)

m.link(P);

struct_summary(m);
kv("Server type Fast", std::string("2 servers, Class1 Exp(5), Class2 Exp(3)"));
kv("Server type Slow", std::string("1 server, Class1 Exp(1)"));
kv("Hetero policy", std::string("ORDER"));
```

Running this edition prints

Model:	HeteroServers		
Nodes:	2		
Stations:	2		
Classes:	2		
Chains:	2		
Station	Sched	Servers	Capacity
Delay	inf	Inf	Inf
Queue	fcfs	3	Inf
JobClass	Type	Population	
Class1	closed	3	
Class2	closed	2	
Server type Fast: 2 servers, Class1 Exp(5), Class2 Exp(3)			
Server type Slow: 1 server, Class1 Exp(1)			
Hetero policy: ORDER			

7.1.5 json_join_deadline

Join deadlines and the quorum they imply.

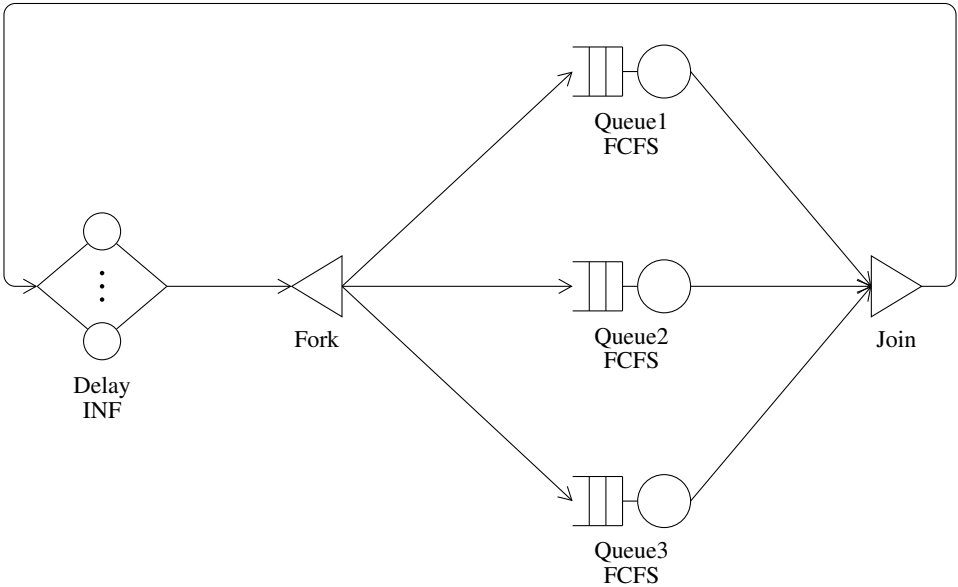


Figure 7.5: Topology of json_join_deadline: a closed network over 4 queueing stations with a fork-join block.

Nodes	6: Queue × 3, Delay, Fork, Join
Classes	1: Class1 (closed, $N = 5$)
Scheduling	Delay: INF; Queue1: FCFS; Queue2: FCFS; Queue3: FCFS
Service	Delay – Class1: Exp(1) Queue1 – Class1: Exp(3) Queue2 – Class1: Exp(4) Queue3 – Class1: Exp(2)

Table 7.5: Features of json_join_deadline.

The example is built and solved as follows.

```
Net m("JoinDeadline");

Delay delay(m, "Delay");
const std::size_t fork = m.add_fork("Fork");
Queue q1(m, "Queue1", SchedStrategy::FCFS);
Queue q2(m, "Queue2", SchedStrategy::FCFS);
Queue q3(m, "Queue3", SchedStrategy::FCFS);
```

```
const std::size_t join = m.add_join("Join", fork);

ClosedClass c1(m, "Class1", 5.0, delay);

delay.set_service(c1, Exp(1.0));
q1.set_service(c1, Exp(3.0));
q2.set_service(c1, Exp(4.0));
q3.set_service(c1, Exp(2.0));

m.set_join_strategy(join, lang::JoinStrategy::PARTIAL, 2.0);

Routing P;
P.set(c1, c1, delay, fork, 1.0);
P.set(c1, c1, fork, q1, 1.0);
P.set(c1, c1, fork, q2, 1.0);
P.set(c1, c1, fork, q3, 1.0);
P.set(c1, c1, q1, join, 1.0);
P.set(c1, c1, q2, join, 1.0);
P.set(c1, c1, q3, join, 1.0);
P.set(c1, c1, join, delay, 1.0);
m.link(P);

struct_summary(m);
kv("Join strategy", std::string("PARTIAL (QUORUM), 2 of 3 siblings"));
na("Class deadline", "ClosedClass.setDeadline(5.0) has no field in the C++ NetworkStruct");
```

Running this edition prints

```
Model:                               JoinDeadline
Nodes:                               6
Stations:                            5
Classes:                             1
Chains:                              1
Station      Sched      Servers      Capacity
Delay        inf        Inf          Inf
Queue1       fcfs        1            Inf
Queue2       fcfs        1            Inf
Queue3       fcfs        1            Inf
Join         inf        Inf          Inf
JobClass     Type       Population
Class1       closed     5
Join strategy:                PARTIAL (QUORUM), 2 of 3 siblings

SOLVER: Class deadline
N/A: ClosedClass.setDeadline(5.0) has no field in the C++ NetworkStruct
```

7.1.6 json_signal_classes

G-network signal classes.

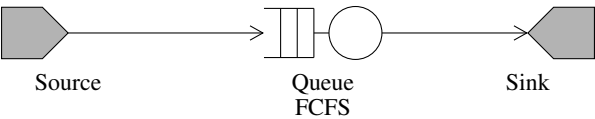


Figure 7.6: Topology of json_signal_classes: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	2: Class1 (open), Signal1 (signal)
Scheduling	Queue: FCFS
Arrivals	Source – Class1: Exp(2); Signal1: Exp(0.5)
Service	Queue – Class1: Exp(5); Signal1: Immediate

Table 7.6: Features of json_signal_classes.

The example is built and solved as follows.

```

Net m("SignalClasses");

Source source(m, "Source");
Queue queue(m, "Queue", SchedStrategy::FCFS);
Sink sink(m, "Sink");

OpenClass c1(m, "Class1");
OpenClass s1(m, "Signal1");
// `OpenSignal(model, 'Signal1', NEGATIVE).forJobClass(Class1)`: the signal is
// an ordinary open class whose target is the class it removes a job from.
m.set_signal(s1, lang::SignalType::NEGATIVE, lang::RemovalPolicy::RANDOM, c1);

source.set_arrival(c1, Exp(2.0));
source.set_arrival(s1, Exp(0.5));
queue.set_service(c1, Exp(5.0));
queue.set_service(s1, Immediate());

Routing P;
serial(P, c1, {source, queue, sink});
serial(P, s1, {source, queue, sink});
m.link(P);

struct_summary(m);
kv("Signal1 type", std::string("NEGATIVE"));
kv("Signal1 target", std::string("Class1"));
kv("Signal1 removal", std::string("RANDOM"));

```

Running this edition prints

Model:	SignalClasses		
Nodes:	3		
Stations:	2		
Classes:	2		
Chains:	2		
Station	Sched	Servers	Capacity
Source	ext	1	Inf
Queue	fcfs	1	Inf
JobClass	Type	Population	
Class1	open	Inf	
Signal1	open	Inf	
Signal1 type:	NEGATIVE		
Signal1 target:	Class1		
Signal1 removal:	RANDOM		

The fixtures are grouped by wire key rather than by model family on purpose: a key that no fixture exercises is a key that no round trip tests, and grouping them this way makes the gap visible.

Chapter 8

Inference and estimation

Inference runs the modeling process backwards: given measurements of a system – arrival rates, utilizations, response times, or a raw trace – it estimates the model parameters that would produce them. The unknown parameters are marked in the model by a distribution with a NaN parameter, and the estimator fills them in.

The estimators divide into three families: likelihood-based (MLE and its quasi-likelihood variant), recursive (the extended Kalman filter and its smoother, which track a parameter that drifts), and Bayesian (sampling and variational, which return a posterior rather than a point estimate). The changepoint scripts test all three against a workload that changes abruptly, which is where a recursive estimator earns its cost. The reference material is in the Inference chapter of the LINE user manual.

The scripts live under `cpp/examples/inference/`, and each one generates a synthetic dataset from known parameters, so the estimate can be compared against the truth.

```
// The unknown demands are declared as NaN and estimated from the data
m.set_service(queue, c1, D::exp_rate(std::nan(" "))); // true demand 0.1
m.set_service(queue, c2, D::exp_rate(std::nan(" "))); // true demand 0.3
```

8.1 The estimators

8.1.1 est_ekf_changepoint

The filter under a changepoint in the workload.

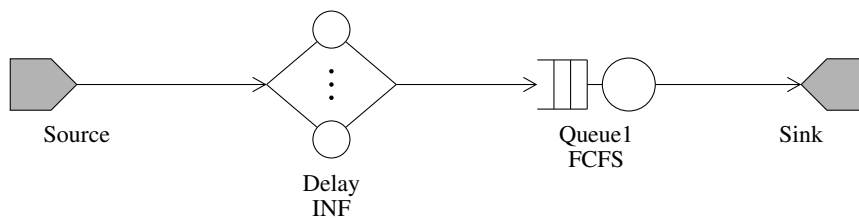


Figure 8.1: Topology of `est_ekf_changepoint`: an open network over 2 queueing stations.

Nodes	4: Source, Queue, Delay, Sink
Classes	1: Class1 (open)
Scheduling	Delay: INF; Queue1: FCFS
Arrivals	Source – Class1: Exp(1)
Service	Delay – Class1: Exp(1) Queue1 – Class1: Exp(3.616)

Table 8.1: Features of `est_ekf_changepoint`.

The example is built and solved as follows.

```
std::size_t delay = 0, queue = 0;
Net m = open4(SchedStrategy::FCFS, {D::exp_mean(1.0)}, {Exp(1.0)}, delay, queue);
model_line(m);

const std::size_t n = 200, changeT = 100, W = 30;
const double D1 = 0.3, D2 = 0.6, lam = 1.0;
// Deterministic part only: the reference adds randn*0.02 from an UNSEEDED stream.
std::vector<double> arvr = filled(n, lam), util(n, 0.0), respt(n, 0.0);
for (std::size_t t = 0; t < n; ++t) {
    const double d = t < changeT ? D1 : D2;
    util[t] = lam * d;
    respt[t] = d / (1.0 - util[t]);
}
sampled("ArvR", "Queue1", "Class1", arvr);
sampled("Respt", "Queue1", "Class1", respt);
sampled("Util", "Queue1", "(aggr)", util);
std::printf(" window W=%zu, warm started from the previous window's estimate\n", W);

std::vector<double> estimates(n - W + 1, 0.0), warm{0.3};
for (std::size_t end = W; end <= n; ++end) {
    const std::vector<double> tw = timestamps(W, end - W);
    const std::vector<double> aw(arvr.begin() + static_cast<std::ptrdiff_t>(end - W),
                                arvr.begin() + static_cast<std::ptrdiff_t>(end));
    const std::vector<double> rw(respt.begin() + static_cast<std::ptrdiff_t>(end - W),
                                respt.begin() + static_cast<std::ptrdiff_t>(end));
    const std::vector<double> uw(util.begin() + static_cast<std::ptrdiff_t>(end - W),
                                util.begin() + static_cast<std::ptrdiff_t>(end));

    ... (8 source lines omitted; full implementation: cpp/examples/inference/)

std::printf("Window size: %zu\n\n", W);
const std::size_t cp[10] = {W, 50, 80, 100, 110, 120, 130, 150, 180, 200};
std::printf("%6s %10s %10s\n", "t", "Estimated", "True");
for (std::size_t i = 0; i < 10; ++i)
    std::printf("%6zu %10.4f %10.4f\n", cp[i], estimates[cp[i] - W],
                cp[i] <= changeT ? D1 : D2);
```

Running this edition prints

```
MODEL: model, Delay(servers=inf) Queue1(servers=1) Source(servers=1) Class1(N=inf)
data ArvR Queue1 Class1 n=200 mean=1
data Respt Queue1 Class1 n=200 mean=0.964286
data Util Queue1 (aggr) n=200 mean=0.45
window W=30, warm started from the previous window's estimate

=== EKF Change-Point Detection (warm-start) ===
True demand: D=0.3 (t<=100), D=0.6 (t>100)
Window size: 30

  t   Estimated      True
  30    0.3000    0.3000
  50    0.3000    0.3000
  80    0.3000    0.3000
 100    0.3000    0.3000
 110    0.6000    0.6000
 120    0.6000    0.6000
 130    0.6000    0.6000
 150    0.6000    0.6000
 180    0.6000    0.6000
 200    0.6000    0.6000
```

8.1.2 est_ekf_closed

The closed-network filter.

The example is built and solved as follows.

```
std::size_t delay = 0, queue = 0;
Net m = closed2({2.0}, delay, queue);
model_line(m);
```

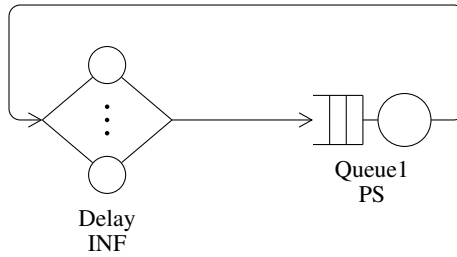



Figure 8.2: Topology of est_ekf_closed: a closed network over 2 queueing stations.

Nodes	2: Queue, Delay
Classes	1: Class1 (closed, $N = 2$)
Scheduling	Delay: INF; Queue1: PS
Service	Delay – Class1: Exp(1) Queue1 – Class1: Exp(2.5)
Solvers	MVA

Table 8.2: Features of est_ekf_closed.

```

const std::size_t n = 100;
// np.random.seed(1): arvr = 1.5 - rand(n)*0.1, so the mean structure is 1.45.
const std::vector<double> arvr = filled(n, 1.45);
std::vector<double> util(n, 0.0), respt(n, 0.0);
for (std::size_t i = 0; i < n; ++i) {
    util[i] = 0.4 * arvr[i];
    respt[i] = 0.4 / (1.0 - util[i]);
}
sampled("ArvR", "Queue1", "Class1", arvr);
sampled("RespT", "Queue1", "Class1", respt);
sampled("Util", "Queue1", "(aggr)", util);

std::printf("\n=== EKF Estimator ===\n");
const Matrix<double> ekf = run_observation_estimator(
    m, "ekf", queue, timestamps(n), {arvr}, {respt}, util, {0.2});
std::printf("Estimated demand: [[%.8f]]\n", ekf(0, 0));
std::printf("\n=== autoMethod selection ===\n");
infer::ParamEstimator automatic(m);
automatic.add_samples(infer::SampledMetric(lang::MetricType::ArvR, timestamps(n), arvr,
    queue, 1));
automatic.add_samples(infer::SampledMetric(lang::MetricType::RespT, timestamps(n), respt,
    queue, 1));
automatic.add_samples(
    infer::SampledMetric(lang::MetricType::Util, timestamps(n), util, queue));
const std::string selected = automatic.auto_method();
std::printf("Selected: %s (%s)\n", selected.c_str(),
    infer::ParamEstimator::get_required_metrics(selected).c_str());
automatic.estimate_at({queue});
solve_after_estimate(m);

```

Running this edition prints

```

MODEL: model, Delay(servers=inf) Queue1(servers=1) Class1(N=2)
data ArvR Queue1 Class1 n=100 mean=1.45
data RespT Queue1 Class1 n=100 mean=0.952381
data Util Queue1 (aggr) n=100 mean=0.58

=== EKF Estimator ===
Estimated demand: [[0.66017254]]

=== autoMethod selection ===
Selected: ubo (ArvR (per-class) + RespT (per-class) + Util (aggregate))

SOLVER: MVA
Station JobClass QLen Util RespT ResidT ArvR
Tput
Delay Class1 1.3208 1.3208 1 1 1.3208

```

1.3208						
Queue1	Class1	0.67925	0.5283	0.51429	0.51429	1.3208
1.3208						

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by MVA.

8.1.3 est_ekf_open

An extended Kalman filter tracking the demands of an open network online.

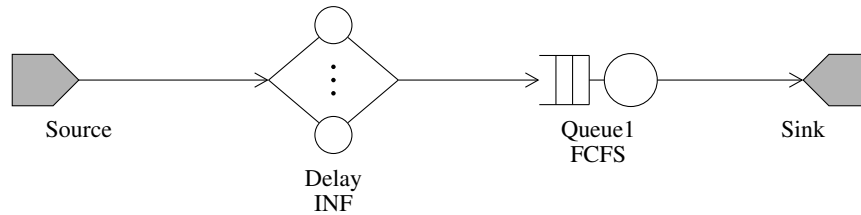


Figure 8.3: Topology of est_ekf_open: an open network over 2 queueing stations.

Nodes	4: Source, Queue, Delay, Sink
Classes	1: Class1 (open)
Scheduling	Delay: INF; Queue1: FCFS
Arrivals	Source – Class1: Exp(1)
Service	Delay – Class1: Exp(1) Queue1 – Class1: Exp(4.123)
Solvers	MVA

Table 8.3: Features of est_ekf_open.

The example is built and solved as follows.

```

std::size_t delay = 0, queue = 0;
Net m = open4(SchedStrategy::FCFS, {D::exp_mean(1.0)}, {Exp(1.0)}, delay, queue);
model_line(m);

const std::size_t n = 100;
// Deterministic part only: the reference adds randn noise from an UNSEEDED stream.
const std::vector<double> arvr = filled(n, 1.0);
const std::vector<double> util = filled(n, 0.4);
std::vector<double> respt(n, 0.0);
for (std::size_t i = 0; i < n; ++i) respt[i] = 0.4 / (1.0 - util[i]);
sampled("ArvR", "Queue1", "Class1", arvr);
sampled("Respt", "Queue1", "Class1", respt);
sampled("Util", "Queue1", "(aggr)", util);

std::printf("\n=== EKF Estimator (Open Network) ===\n");
const Matrix<double> estimate = run_observation_estimator(
    m, "ekf", queue, timestamps(n), {arvr}, {respt}, util, {0.2});
std::printf("Estimated demand: [[%.8f]]\n", estimate(0, 0));
solve_after_estimate(m);
  
```

Running this edition prints

```

MODEL: model, Delay(servers=inf) Queue1(servers=1) Source(servers=1) Class1(N=inf)
data ArvR Queue1 Class1 n=100 mean=1
data Respt Queue1 Class1 n=100 mean=0.666667
data Util Queue1 (aggr) n=100 mean=0.4

=== EKF Estimator (Open Network) ===
Estimated demand: [[0.40000028]]

SOLVER: MVA
  
```

Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Delay	Class1	1	1	1	1	1
1						
Queue1	Class1	0.66667	0.4	0.66667	0.66667	1
1						
Source	Class1	0	0	0	0	0
1						

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by MVA.

8.1.4 est_erps_closed

The closed-network smoother.

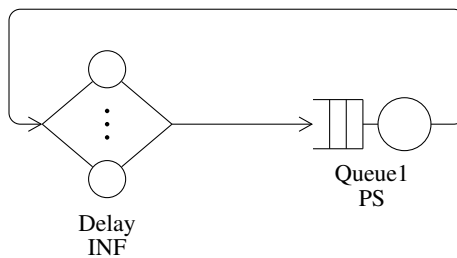


Figure 8.4: Topology of est_erps_closed: a closed network over 2 queueing stations.

Nodes	2: Queue, Delay
Classes	2: Class1 (closed, $N = 1$), Class2 (closed, $N = 3$)
Scheduling	Delay: INF; Queue1: PS
Service	Delay – Class1: Exp(1); Class2: Exp(1) Queue1 – Class1: Exp(10); Class2: Exp(3.333)
Solvers	MVA

Table 8.4: Features of est_erps_closed.

The example is built and solved as follows.

```
std::size_t delay = 0, queue = 0;
Net m = closed2({1.0, 3.0}, delay, queue);
model_line(m);

const std::size_t n = 1000;
std::vector<double> u, r1, r2, q;
erps_data(n, u, r1, r2, q);
sampled("QLen", "Queue1", "arv Cls1", q);
sampled("QLen", "Queue1", "arv Cls2", q);
sampled("RespT", "Queue1", "Class1", r1);
sampled("RespT", "Queue1", "Class2", r2);

const Matrix<double> estimate = run_erps(m, queue, timestamps(n), {r1, r2}, {q, q});
std::printf("Estimated demands: [[%.8f %.8f]]\n", estimate(0, 0), estimate(0, 1));
solve_after_estimate(m);
```

Running this edition prints

```
MODEL: model, Delay(servers=inf) Queue1(servers=1) Class1(N=1) Class2(N=3)
data QLen Queue1 arv Cls1 n=1000 mean=3.0303
data QLen Queue1 arv Cls2 n=1000 mean=3.0303
data RespT Queue1 Class1 n=1000 mean=0.30303
data RespT Queue1 Class2 n=1000 mean=0.909091
Estimated demands: [[0.10000000 0.30000000]]

SOLVER: MVA
```

Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Delay	Class1	0.83698	0.83698	1	1	0.83698
0.83698						
Delay	Class2	1.9821	1.9821	1	1	1.9821
1.9821						
Queue1	Class1	0.16302	0.083698	0.19477	0.19477	0.83698
0.83698						
Queue1	Class2	1.0179	0.59463	0.51353	0.51353	1.9821
1.9821						

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by MVA.

8.1.5 est_erps_open

The Rauch-Tung-Striebel smoother on an open network.

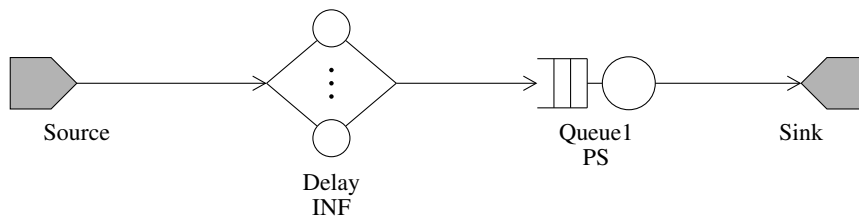


Figure 8.5: Topology of est_erps_open: an open network over 2 queueing stations.

Nodes	4: Source, Queue, Delay, Sink
Classes	2: Class1 (open), Class2 (open)
Scheduling	Delay: INF; Queue1: PS
Arrivals	Source – Class1: Exp(0.1); Class2: Exp(0.05)
Service	Delay – Class1: HyperExp(0.5,0.5;3,10); Class2: HyperExp(0.5,0.5;2,8) Queue1 – Class1: Exp(10); Class2: Exp(3.333)
Solvers	MVA

Table 8.5: Features of est_erps_open.

The example is built and solved as follows.

```
std::size_t delay = 0, queue = 0;
Net m = open4(SchedStrategy::PS, {hyper(0.5, 3.0, 10.0), hyper(0.5, 2.0, 8.0)},
              {Exp(0.1), Exp(0.05)}, delay, queue);
model_line(m);

const std::size_t n = 1000;
std::vector<double> u, r1, r2, q;
erps_data(n, u, r1, r2, q);
sampled("QLen", "Queue1", "arv Cls1", q);
sampled("QLen", "Queue1", "arv Cls2", q);
sampled("RespT", "Queue1", "Class1", r1);
sampled("RespT", "Queue1", "Class2", r2);

const Matrix<double> estimate = run_erps(m, queue, timestamps(n), {r1, r2}, {q, q});
std::printf("Estimated demands: [[%.8f %.8f]]\n", estimate(0, 0), estimate(0, 1));
solve_after_estimate(m);
```

Running this edition prints

```
MODEL: model, Delay(servers=inf) Queue1(servers=1) Source(servers=1) Class1(N=inf) Class2(N=inf)
data QLen Queue1 arv Cls1 n=1000 mean=3.0303
data QLen Queue1 arv Cls2 n=1000 mean=3.0303
data RespT Queue1 Class1 n=1000 mean=0.30303
data RespT Queue1 Class2 n=1000 mean=0.909091
```

Estimated demands: [[0.10000000 0.30000000]]

SOLVER: MVA

Station	JobClass	QLen	Util	RespT	ResidT	ArvR
Tput						
Delay 0.1	Class1	0.021667	0.021667	0.21667	0.21667	0.1
Delay 0.05	Class2	0.015625	0.015625	0.3125	0.3125	0.05
Queue1 0.1	Class1	0.010157	0.01	0.10157	0.10157	0.1
Queue1 0.05	Class2	0.015235	0.015	0.30471	0.30471	0.05
Source 0.1	Class1	0	0	0	0	0
Source 0.05	Class2	0	0	0	0	0

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by MVA.

8.1.6 est_mcmc_closed

Markov chain Monte Carlo sampling of the posterior over the demands.

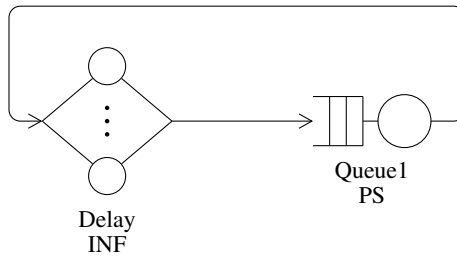


Figure 8.6: Topology of est_mcmc_closed: a closed network over 2 queueing stations.

Nodes	2: Queue, Delay
Classes	2: Class1 (closed, $N = 2$), Class2 (closed, $N = 3$)
Scheduling	Delay: INF; Queue1: PS
Service	Delay – Class1: Exp(1); Class2: Exp(1) Queue1 – Class1: Exp(0.5703); Class2: Exp(0.5703)
Solvers	MVA

Table 8.6: Features of est_mcmc_closed.

The example is built and solved as follows.

```
std::size_t delay = 0, queue = 0;
Net m = closed2({2.0, 3.0}, delay, queue);
model_line(m);

const std::size_t n = 500;
// np.random.seed(1): qlen = 1.5 + rand(n)*0.5, mean structure 1.75.
const std::vector<double> qlen = filled(n, 1.75);
sampled("QLen", "Queue1", "(aggr)", qlen);

std::printf("\n=== MCMC Estimator ===\n");
infer::EstimatorOptions options;
options.method = "mcmc";
options.random_seed = 1;
infer::ParamEstimator estimator(m, options);
estimator.add_samples(
    infer::SampledMetric(lang::MetricType::QLen, timestamps(n), qlen, queue));
const Matrix<double> estimate = estimator.estimate_at({queue});
std::printf("Estimated demands: [[%.8f %.8f]]\n", estimate(0, 0), estimate(0, 1));
```

```
solve_after_estimate(m);
```

Running this edition prints

```
MODEL: model, Delay(servers=inf) Queue1(servers=1) Class1(N=2) Class2(N=3)
data QLen Queue1 (aggr) n=500 mean=1.75

=== MCMC Estimator ===
Estimated demands: [[1.74939951 1.74957108]]

SOLVER: MVA
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
Tput
Delay        Class1        0.22858    0.22858    1           1           0.22858
0.22858
Delay        Class2        0.34284    0.34284    1           1           0.34284
0.34284
Queue1       Class1        1.7714     0.39988    7.7496      7.7496      0.22858
0.22858
Queue1       Class2        2.6572     0.59983    7.7503      7.7503      0.34284
0.34284
```

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by MVA.

8.1.7 est_mle_open

Maximum likelihood estimation of the service demands of an open network from arrival rate, utilization and response time samples.

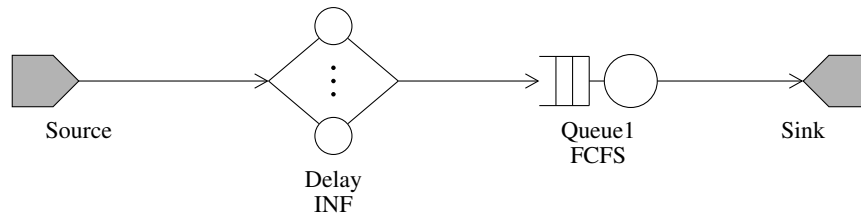


Figure 8.7: Topology of est_mle_open: an open network over 2 queueing stations.

Nodes	4: Source, Queue, Delay, Sink
Classes	2: Class1 (open), Class2 (open)
Scheduling	Delay: INF; Queue1: FCFS
Arrivals	Source – Class1: Exp(1); Class2: Exp(0.5)
Service	Delay – Class1: Exp(1); Class2: Exp(1) Queue1 – Class1: Exp(7.962); Class2: Exp(4.77)
Solvers	MVA

Table 8.7: Features of est_mle_open.

The example is built and solved as follows.

```
std::size_t delay = 0, queue = 0;
Net m = open4(SchedStrategy::FCFS, {D::exp_mean(1.0), D::exp_mean(1.0)},
              {Exp(1.0), Exp(0.5)}, delay, queue);
model_line(m);

const std::size_t n = 100;
// Deterministic part only: the reference adds randn noise from an UNSEEDED stream.
const std::vector<double> a1 = filled(n, 1.0), a2 = filled(n, 0.5);
const std::vector<double> util = filled(n, 0.25);
std::vector<double> r1(n, 0.0), r2(n, 0.0);
for (std::size_t i = 0; i < n; ++i) {
    r1[i] = 0.1 / (1.0 - util[i]);
    r2[i] = 0.3 / (1.0 - util[i]);
}
```

```

}
sampled("ArvR", "Queue1", "Class1", a1);
sampled("ArvR", "Queue1", "Class2", a2);
sampled("RespT", "Queue1", "Class1", r1);
sampled("RespT", "Queue1", "Class2", r2);
sampled("Util", "Queue1", "(aggr)", util);

std::printf("\n=== MLE Estimator (Open Network) ===\n");
const Matrix<double> estimate = run_observation_estimator(
    m, "mle", queue, timestamps(n), {a1, a2}, {r1, r2}, util, {0.1, 0.3});
std::printf("Estimated demands: [[%.8f %.8f]]\n", estimate(0, 0), estimate(0, 1));
solve_after_estimate(m);

```

Running this edition prints

```

MODEL: model, Delay(servers=inf) Queue1(servers=1) Source(servers=1) Class1(N=inf) Class2(N=inf)
data ArvR Queue1 Class1 n=100 mean=1
data ArvR Queue1 Class2 n=100 mean=0.5
data RespT Queue1 Class1 n=100 mean=0.133333
data RespT Queue1 Class2 n=100 mean=0.4
data Util Queue1 (aggr) n=100 mean=0.25

=== MLE Estimator (Open Network) ===
Estimated demands: [[0.07237557 0.32710449]]

SOLVER: MVA
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
  Tput
Delay        Class1          1          1          1          1          1
  1
Delay        Class2          0.5        0.5          1          1          0.5
  0.5
Queue1       Class1      0.14925    0.072376    0.14925    0.14925          1
  1
Queue1       Class2      0.20199    0.16355    0.40398    0.40398          0.5
  0.5
Source       Class1          0          0          0          0          0
  1
Source       Class2          0          0          0          0          0
  0.5

```

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by MVA.

8.1.8 est_qmle_closed

Quasi-likelihood estimation on a closed network.

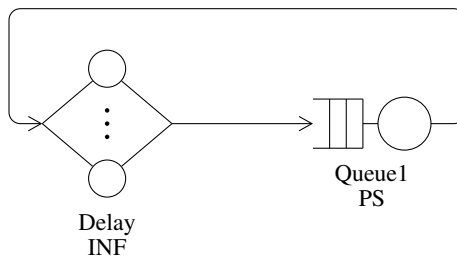


Figure 8.8: Topology of est_qmle_closed: a closed network over 2 queueing stations.

The example is built and solved as follows.

```

std::size_t delay = 0, queue = 0;
Net m = closed2({2.0, 3.0}, delay, queue);

const std::size_t n = 1000;
const double qlen1_mean = 0.35006045994559054; // mean of 0.3 + rand(1000)*0.1, seed 1
const double qlen2_mean = 0.8518112390463691; // mean of 0.8 + rand(1000)*0.1, seed 1
sampled("QLen", "Queue1", "Class1", filled(n, qlen1_mean));

```

Nodes	2: Queue, Delay
Classes	2: Class1 (closed, $N = 2$), Class2 (closed, $N = 3$)
Scheduling	Delay: INF; Queue1: PS
Service	Delay – Class1: Exp(1); Class2: Exp(1) Queue1 – Class1: Exp(9.553); Class2: Exp(4.837)
Solvers	MVA

Table 8.8: Features of `est_qmle_closed`.

```
sampled("QLen", "Queue1", "Class2", filled(n, qlen2_mean));

infer::EstimatorOptions options;
options.method = "qmle";
infer::ParamEstimator estimator(m, options);
estimator.add_samples(infer::SampledMetric(lang::MetricType::QLen, timestamps(n),
                                           filled(n, qlen1_mean), queue, 1));
estimator.add_samples(infer::SampledMetric(lang::MetricType::QLen, timestamps(n),
                                           filled(n, qlen2_mean), queue, 2));
const Matrix<double> est = estimator.estimate_at({queue});
std::printf("Estimated demands: [[%.8f %.8f]]\n", est(0, 0), est(0, 1));

mva::MvaOptions mopt;
Matrix<double> init;
const mva::AvgResult<double> res = mva::solver_mva_run_analyzer(m.get_struct(), mopt, init);
section("MVA");
print_avg(m.get_struct(), res);
```

Running this edition prints

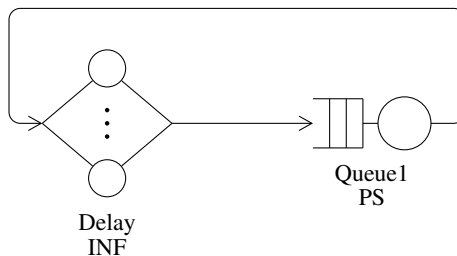
```
data QLen Queue1 Class1 n=1000 mean=0.35006
data QLen Queue1 Class2 n=1000 mean=0.851811
Estimated demands: [[0.10467796 0.20674601]]

SOLVER: MVA
Station JobClass QLen Util RespT ResidT ArvR
Tput
Delay Class1 1.6716 1.6716 1 1 1.6716
1.6716
Delay Class2 2.2072 2.2072 1 1 2.2072
2.2072
Queue1 Class1 0.32836 0.17498 0.19643 0.19643 1.6716
1.6716
Queue1 Class2 0.79283 0.45632 0.3592 0.3592 2.2072
2.2072
```

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by MVA.

8.1.9 `est_trace_closed`

Estimation from a raw arrival and completion trace rather than from aggregate samples.

Figure 8.9: Topology of `est_trace_closed`: a closed network over 2 queueing stations.

The example is built and solved as follows.

Nodes	2: Queue, Delay
Classes	1: Class1 (closed, $N = 5$)
Scheduling	Delay: INF; Queue1: PS
Service	Delay – Class1: Exp(1) Queue1 – Class1: Exp(5.43)
Solvers	MVA, CTMC

Table 8.9: Features of `est_trace_closed`.

```

const double N = 5.0;
std::size_t delay = 0, queue = 0;
Net m = closed2({N}, delay, queue);

std::vector<double> at(kArrivalTimes, kArrivalTimes + 200);
std::vector<double> rt(kResponseTimes, kResponseTimes + 200);
const std::vector<double> tput = filled(200, N / (1.0 + 0.5));
sampled("ArvR", "Queue1", "Class1", at);
sampled("RespT", "Queue1", "Class1", rt);
sampled("Tput", "Queue1", "Class1", tput);

const auto run_trace = [&](const std::string& method) {
    infer::EstimatorOptions options;
    options.method = method;
    options.random_seed = 1;
    infer::ParamEstimator estimator(m, options);
    infer::SampledMetric arrivals(lang::MetricType::ArvR, timestamps(at.size()), at, queue,
                                1);
    arrivals.set_trace();
    infer::SampledMetric response(lang::MetricType::RespT, timestamps(rt.size()), rt, queue,
                                1);
    response.set_trace();
    estimator.add_samples(arrivals);
    estimator.add_samples(response);
    if (method == "gibbs")
        estimator.add_samples(infer::SampledMetric(lang::MetricType::Tput,
                                                    timestamps(tput.size()), tput, queue,
                                                    1));
    ... (18 source lines omitted; full implementation: cpp/examples/inference/)

    std::printf("Gibbs: %.4f\n", gibbs(0, 0));
    mva::MvaOptions mopt;
    Matrix<double> init;
    const mva::AvgResult<double> res = mva::solver_mva_run_analyzer(m.get_struct(), mopt, init);
    section("MVA");
    print_avg(m.get_struct(), res);

```

Running this edition prints

```

data ArvR  Queue1  Class1  n=200 mean=48.5358
data RespT Queue1  Class1  n=200 mean=0.659724
data Tput  Queue1  Class1  n=200 mean=3.33333

=== MLPS Estimator ===
MLPS demand: Class1=0.3813

=== FMLPS Estimator ===
FMLPS demand: Class1=0.4151

=== Gibbs Estimator ===
Gibbs demand: Class1=0.1819

=== Comparison (true demand ~ 0.5) ===
MLPS:  0.3813
FMLPS: 0.4151
Gibbs:  0.1819

SOLVER: MVA
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
Tput
Delay        Class1        3.7174     3.7174     1           1           3.7174
3.7174
Queue1       Class1        1.2826     0.67602    0.34502     0.34502     3.7174
3.7174

```

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by MVA.

8.1.10 est_ubo_changepoint

The variational estimator under a changepoint.

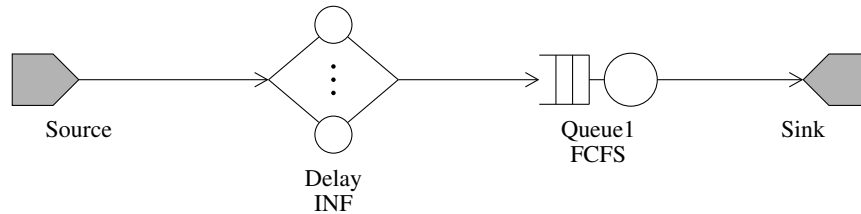


Figure 8.10: Topology of est_ubo_changepoint: an open network over 2 queueing stations.

Nodes	4: Source, Queue, Delay, Sink
Classes	2: Class1 (open), Class2 (open)
Scheduling	Delay: INF; Queue1: FCFS
Arrivals	Source – Class1: Exp(1); Class2: Exp(0.5)
Service	Delay – Class1: Exp(1); Class2: Exp(1) Queue1 – Class1: Exp(3.338); Class2: Exp(5.012)

Table 8.10: Features of est_ubo_changepoint.

The example is built and solved as follows.

```
std::size_t delay = 0, queue = 0;
Net m = open4(SchedStrategy::FCFS, {D::exp_mean(1.0), D::exp_mean(1.0)},
              {Exp(1.0), Exp(0.5)}, delay, queue);
model_line(m);

const std::size_t n = 200, changeT = 100, W = 30;
const double D1_before = 0.1, D1_after = 0.3, D2_val = 0.2;
const double lambda1 = 1.0, lambda2 = 0.5;
// Deterministic part only: the reference adds randn*0.01 from an UNSEEDED stream.
const std::vector<double> a1 = filled(n, lambda1), a2 = filled(n, lambda2);
std::vector<double> util(n, 0.0), r1(n, 0.0), r2(n, 0.0);
for (std::size_t t = 0; t < n; ++t) {
    const double d1 = t < changeT ? D1_before : D1_after;
    util[t] = lambda1 * d1 + lambda2 * D2_val;
    r1[t] = d1 / (1.0 - util[t]);
    r2[t] = D2_val / (1.0 - util[t]);
}
sampled("ArvR", "Queue1", "Class1", a1);
sampled("ArvR", "Queue1", "Class2", a2);
sampled("RespT", "Queue1", "Class1", r1);
sampled("RespT", "Queue1", "Class2", r2);
sampled("Util", "Queue1", "(aggr)", util);

std::vector<double> estimate1(n - W + 1, 0.0), estimate2(n - W + 1, 0.0);
for (std::size_t end = W; end <= n; ++end) {
    const std::vector<double> tw = timestamps(W, end - W);

    ... (19 source lines omitted; full implementation: cpp/examples/inference/)

const std::size_t cp[10] = {W, 50, 80, 100, 110, 120, 130, 150, 180, 200};
std::printf("%6s %8s %8s %8s %8s\n", "t", "Est D1", "True D1", "Est D2", "True D2");
for (std::size_t i = 0; i < 10; ++i)
    std::printf("%6zu %8.4f %8.4f %8.4f %8.4f\n", cp[i],
                estimate1[cp[i] - W], cp[i] <= changeT ? D1_before : D1_after,
```

```
estimate2[cp[i] - W], D2_val);
```

Running this edition prints

```
MODEL: model, Delay(servers=inf) Queue1(servers=1) Source(servers=1) Class1(N=inf) Class2(N=inf)
data ArvR Queue1 Class1 n=200 mean=1
data ArvR Queue1 Class2 n=200 mean=0.5
data RespT Queue1 Class1 n=200 mean=0.3125
data RespT Queue1 Class2 n=200 mean=0.291667
data Util Queue1 (aggr) n=200 mean=0.3
```

```
=== UBO Change-Point Detection (2-class) ===
```

```
Class 1: D=0.1 (t<=100), D=0.3 (t>100)
```

```
Class 2: D=0.2 (constant)
```

```
Window size: 30
```

t	Est D1	True D1	Est D2	True D2
30	0.1000	0.1000	0.2000	0.2000
50	0.1000	0.1000	0.2000	0.2000
80	0.1000	0.1000	0.2000	0.2000
100	0.1000	0.1000	0.2000	0.2000
110	0.1844	0.3000	0.1902	0.2000
120	0.2487	0.3000	0.1926	0.2000
130	0.3000	0.3000	0.2000	0.2000
150	0.3000	0.3000	0.2000	0.2000
180	0.3000	0.3000	0.2000	0.2000
200	0.3000	0.3000	0.2000	0.2000

8.1.11 est_ubo_closed

The closed-network version.

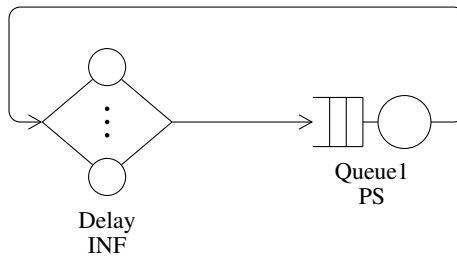


Figure 8.11: Topology of est_ubo_closed: a closed network over 2 queueing stations.

Nodes	2: Queue, Delay
Classes	2: Class1 (closed, $N = 1$), Class2 (closed, $N = 2$)
Scheduling	Delay: INF; Queue1: PS
Service	Delay – Class1: Exp(1); Class2: Exp(1) Queue1 – Class1: Exp(10); Class2: Exp(3.333)
Solvers	MVA

Table 8.11: Features of est_ubo_closed.

The example is built and solved as follows.

```
std::size_t delay = 0, queue = 0;
Net m = closed2({1.0, 2.0}, delay, queue);
model_line(m);

const std::size_t n = 30;
// np.random.seed(1): arvr1 = 2 - rand*0.15 and arvr2 = 1 - rand*0.15.
const std::vector<double> a1 = filled(n, 1.925), a2 = filled(n, 0.925);
std::vector<double> util(n, 0.0), r1(n, 0.0), r2(n, 0.0);
for (std::size_t i = 0; i < n; ++i) {
    util[i] = 0.1 * a1[i] + 0.3 * a2[i];
    r1[i] = 0.1 / (1.0 - util[i]);
}
```

```

    r2[i] = 0.3 / (1.0 - util[i]);
}
sampled("ArvR", "Queue1", "Class1", a1);
sampled("ArvR", "Queue1", "Class2", a2);
sampled("RespT", "Queue1", "Class1", r1);
sampled("RespT", "Queue1", "Class2", r2);
sampled("Util", "Queue1", "(aggr)", util);

const Matrix<double> estimate = run_observation_estimator(
    m, "ubo", queue, timestamps(n), {a1, a2}, {r1, r2}, util);
std::printf("Estimated demands: [[%.8f %.8f]]\n", estimate(0, 0), estimate(0, 1));
solve_after_estimate(m);

```

Running this edition prints

```

MODEL: model, Delay(servers=inf) Queue1(servers=1) Class1(N=1) Class2(N=2)
data ArvR Queue1 Class1 n=30 mean=1.925
data ArvR Queue1 Class2 n=30 mean=0.925
data RespT Queue1 Class1 n=30 mean=0.188679
data RespT Queue1 Class2 n=30 mean=0.566038
data Util Queue1 (aggr) n=30 mean=0.47
Estimated demands: [[0.10000000 0.30000000]]

SOLVER: MVA
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
  Tput
Delay        Class1      0.8666    0.8666      1          1          0.8666
  0.8666
Delay        Class2      1.4216    1.4216      1          1          1.4216
  1.4216
Queue1       Class1      0.1334    0.08666    0.15393    0.15393    0.8666
  0.8666
Queue1       Class2      0.57838   0.42648    0.40685    0.40685    1.4216
  1.4216

```

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by MVA.

8.1.12 est_ubo_open

Variational Bayesian estimation on an open network.

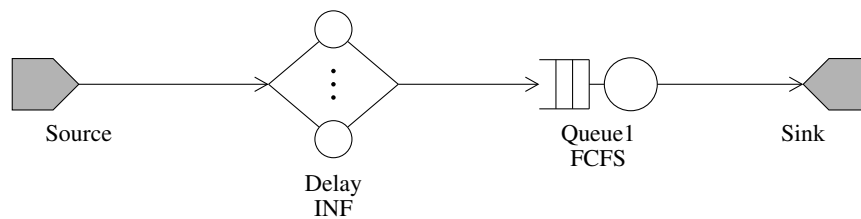


Figure 8.12: Topology of est_ubo_open: an open network over 2 queueing stations.

Nodes	4: Source, Queue, Delay, Sink		
Classes	2: Class1 (open), Class2 (open)		
Scheduling	Delay: INF; Queue1: FCFS		
Arrivals	Source – Class1: Exp(0.1); Class2: Exp(0.05)		
Service	Delay – Class1: HyperExp(0.5,0.5;3,10); Class2: HyperExp(0.5,0.5;2,8)	Queue1 – Class1:	Exp(10); Class2: Exp(3.333)
Solvers	MVA		

Table 8.12: Features of est_ubo_open.

The example is built and solved as follows.

```

std::size_t delay = 0, queue = 0;
Net m = open4(SchedStrategy::FCFS, {hyper(0.5, 3.0, 10.0), hyper(0.5, 2.0, 8.0)},
              {Exp(0.1), Exp(0.05)}, delay, queue);
model_line(m);

const std::size_t n = 1000;
// Deterministic part only: arvr1 = 2 - rand*0.15, arvr2 = 1 - rand*0.10, UNSEED.
const std::vector<double> a1 = filled(n, 1.925), a2 = filled(n, 0.95);
std::vector<double> util(n, 0.0), r1(n, 0.0), r2(n, 0.0);
for (std::size_t i = 0; i < n; ++i) {
    util[i] = 0.1 * a1[i] + 0.3 * a2[i];
    r1[i] = 0.1 / (1.0 - util[i]);
    r2[i] = 0.3 / (1.0 - util[i]);
}
sampled("ArvR", "Queue1", "Class1", a1);
sampled("ArvR", "Queue1", "Class2", a2);
sampled("RespT", "Queue1", "Class1", r1);
sampled("RespT", "Queue1", "Class2", r2);
sampled("Util", "Queue1", "(aggr)", util);

const Matrix<double> estimate = run_observation_estimator(
    m, "ubo", queue, timestamps(n), {a1, a2}, {r1, r2}, util);
std::printf("Estimated demands: [[%.8f %.8f]]\n", estimate(0, 0), estimate(0, 1));
solve_after_estimate(m);

```

Running this edition prints

```

MODEL: model, Delay(servers=inf) Queue1(servers=1) Source(servers=1) Class1(N=inf) Class2(N=inf)
data ArvR Queue1 Class1 n=1000 mean=1.925
data ArvR Queue1 Class2 n=1000 mean=0.95
data RespT Queue1 Class1 n=1000 mean=0.191388
data RespT Queue1 Class2 n=1000 mean=0.574163
data Util Queue1 (aggr) n=1000 mean=0.4775
Estimated demands: [[0.10000000 0.30000000]]

SOLVER: MVA
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
Tput
Delay        Class1      0.021667   0.021667   0.21667    0.21667     0.1
0.1
Delay        Class2      0.015625   0.015625   0.3125     0.3125     0.05
0.05
Queue1       Class1      0.010564   0.01       0.10564    0.10564     0.1
0.1
Queue1       Class2      0.015282   0.015     0.30564    0.30564     0.05
0.05
Source       Class1      0          0          0          0          0
0.1
Source       Class2      0          0          0          0          0
0.05

```

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by MVA.

8.1.13 est_ubo_variants_closed

The variants of the variational estimator, side by side.

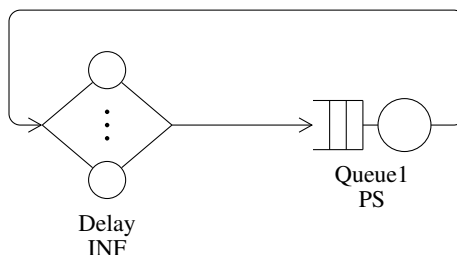


Figure 8.13: Topology of est_ubo_variants_closed: a closed network over 2 queueing stations.

Nodes	2: Queue, Delay
Classes	2: Class1 (closed, $N = 1$), Class2 (closed, $N = 3$)
Scheduling	Delay: INF; Queue1: PS
Service	Delay – Class1: Exp(1); Class2: Exp(1) Queue1 – Class1: Exp(11.21); Class2: Exp(4.547)
Solvers	MVA

Table 8.13: Features of `est_ubo_variants_closed`.

The example is built and solved as follows.

```
std::size_t delay = 0, queue = 0;
Net m = closed2({1.0, 3.0}, delay, queue);
model_line(m);

const std::size_t n = 1000;
// np.random.seed(1): arvr1 = 2 - rand*0.15 and arvr2 = 1 - rand*0.15.
const std::vector<double> a1 = filled(n, 1.925), a2 = filled(n, 0.925);
std::vector<double> util(n, 0.0), r1(n, 0.0), r2(n, 0.0);
for (std::size_t i = 0; i < n; ++i) {
    util[i] = 0.1 * a1[i] + 0.3 * a2[i];
    r1[i] = 0.1 / (1.0 - util[i]);
    r2[i] = 0.3 / (1.0 - util[i]);
}
sampled("ArvR", "Queue1", "Class1", a1);
sampled("ArvR", "Queue1", "Class2", a2);
sampled("RespT", "Queue1", "Class1", r1);
sampled("RespT", "Queue1", "Class2", r2);
sampled("Util", "Queue1", "(aggr)", util);

std::printf("\n=== UBO Estimator ===\n");
const Matrix<double> ubo = run_observation_estimator(
    m, "ubo", queue, timestamps(n), {a1, a2}, {r1, r2}, util);
std::printf("\n=== MLE Estimator ===\n");
const Matrix<double> mle = run_observation_estimator(
    m, "mle", queue, timestamps(n), {a1, a2}, {r1, r2}, util, {0.1, 0.3});
std::printf("\n=== Comparison ===\n");
std::printf("True demands: Class1=0.1000, Class2=0.3000\n");
std::printf("UBO: Class1=%.4f, Class2=%.4f\n", ubo(0, 0), ubo(0, 1));
std::printf("MLE: Class1=%.4f, Class2=%.4f\n", mle(0, 0), mle(0, 1));
solve_after_estimate(m);
```

Running this edition prints

```
MODEL: model, Delay(servers=inf) Queue1(servers=1) Class1(N=1) Class2(N=3)
data ArvR Queue1 Class1 n=1000 mean=1.925
data ArvR Queue1 Class2 n=1000 mean=0.925
data RespT Queue1 Class1 n=1000 mean=0.188679
data RespT Queue1 Class2 n=1000 mean=0.566038
data Util Queue1 (aggr) n=1000 mean=0.47

=== UBO Estimator ===

=== MLE Estimator ===

=== Comparison ===
True demands: Class1=0.1000, Class2=0.3000
UBO: Class1=0.1000, Class2=0.3000
MLE: Class1=0.0892, Class2=0.2205

SOLVER: MVA
Station JobClass QLen Util RespT ResidT ArvR
Tput
Delay Class1 0.86743 0.86743 1 1 0.86743
0.86743
Delay Class2 2.2342 2.2342 1 1 2.2342
2.2342
Queue1 Class1 0.13257 0.077402 0.15283 0.15283 0.86743
0.86743
Queue1 Class2 0.76581 0.49268 0.34277 0.34277 2.2342
2.2342
```

The columns are the mean queue length, utilization, response time, residence time, arrival rate and

throughput of each station-class pair, as returned by MVA.

8.1.14 est_ubr_changepoint

The regression estimator under a changepoint.

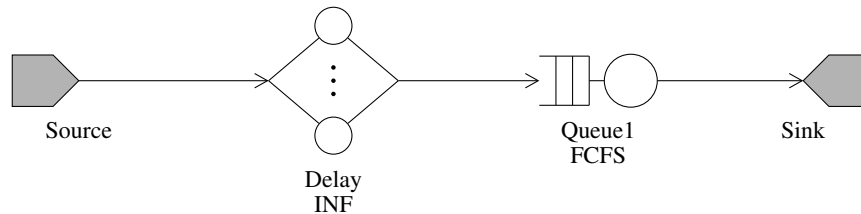


Figure 8.14: Topology of `est_ubr_changepoint`: an open network over 2 queueing stations.

Nodes	4: Source, Queue, Delay, Sink
Classes	1: Class1 (open)
Scheduling	Delay: INF; Queue1: FCFS
Arrivals	Source – Class1: Exp(1)
Service	Delay – Class1: Exp(1) Queue1 – Class1: Exp(1.665)

Table 8.14: Features of `est_ubr_changepoint`.

The example is built and solved as follows.

```
std::size_t delay = 0, queue = 0;
Net m = open4(SchedStrategy::FCFS, {D::exp_mean(1.0)}, {Exp(1.0)}, delay, queue);
model_line(m);

const std::size_t n = 200, changeT = 100, W = 30;
const double D1 = 0.3, D2 = 0.6, lam = 1.0;
// Deterministic part only: the reference adds randn*0.02 from an UNSEEDED stream.
const std::vector<double> arvr = filled(n, lam);
std::vector<double> util(n, 0.0);
for (std::size_t t = 0; t < n; ++t) util[t] = lam * (t < changeT ? D1 : D2);
sampled("ArvR", "Queue1", "Class1", arvr);
sampled("Util", "Queue1", "(aggr)", util);

std::vector<double> estimates(n - W + 1, 0.0);
for (std::size_t end = W; end <= n; ++end) {
    const std::vector<double> tw = timestamps(W, end - W);
    const std::vector<double> aw(arvr.begin() + static_cast<std::ptrdiff_t>(end - W),
                                arvr.begin() + static_cast<std::ptrdiff_t>(end));
    const std::vector<double> uw(util.begin() + static_cast<std::ptrdiff_t>(end - W),
                                util.begin() + static_cast<std::ptrdiff_t>(end));
    estimates[end - W] = run_ubr(m, queue, tw, {aw}, uw)(0, 0);
}
std::printf("\n=== UBR Change-Point Detection ===\n");
std::printf("True demand: D=%g (t<=%zu), D=%g (t>%zu)\n", D1, changeT, D2, changeT);
std::printf("Window size: %zu\n\n", W);
const std::size_t cp[10] = {W, 50, 80, 100, 110, 120, 130, 150, 180, 200};
std::printf("%6s %10s %10s\n", "t", "Estimated", "True");
std::printf("%6s %10s %10s\n", "-----", "-----", "-----");
for (std::size_t i = 0; i < 10; ++i)
    std::printf("%6zu %10.4f %10.4f\n", cp[i], estimates[cp[i] - W],
                cp[i] <= changeT ? D1 : D2);
```

Running this edition prints

```
MODEL: model, Delay(servers=inf) Queue1(servers=1) Source(servers=1) Class1(N=inf)
data ArvR Queue1 Class1 n=200 mean=1
data Util Queue1 (aggr) n=200 mean=0.45
```

=== UBR Change-Point Detection ===		
True demand: D=0.3 (t<=100), D=0.6 (t>100)		
Window size: 30		
t	Estimated	True
---	-----	-----
30	0.3000	0.3000
50	0.3000	0.3000
80	0.3000	0.3000
100	0.3000	0.3000
110	0.4000	0.6000
120	0.5000	0.6000
130	0.6000	0.6000
150	0.6000	0.6000
180	0.6000	0.6000
200	0.6000	0.6000

8.1.15 est_ubr_closed

The closed-network version.

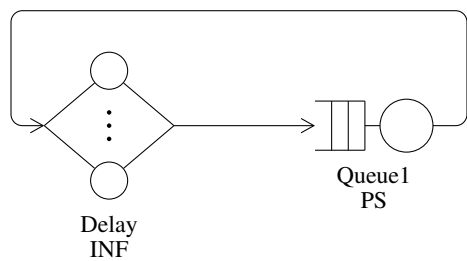


Figure 8.15: Topology of est_ubr_closed: a closed network over 2 queueing stations.

Nodes	2: Queue, Delay
Classes	2: Class1 (closed, $N = 1$), Class2 (closed, $N = 2$)
Scheduling	Delay: INF; Queue1: PS
Service	Delay – Class1: Exp(1); Class2: Exp(1) Queue1 – Class1: Exp(2.502); Class2: Exp(13.93)
Solvers	MVA

Table 8.15: Features of est_ubr_closed.

The example is built and solved as follows.

```
std::size_t delay = 0, queue = 0;
Net m = closed2({1.0, 2.0}, delay, queue);
model_line(m);

const std::size_t n = 1000;
// np.random.seed(1): 2 - rand*0.15, 3 - rand*0.25, 1 - rand*0.05, 0.4*(2 - rand*0.15).
const std::vector<double> a1 = filled(n, 1.925), a2 = filled(n, 2.875);
const std::vector<double> util = filled(n, 0.975), util1 = filled(n, 0.4 * 1.925);
sampled("ArvR", "Queue1", "Class1", a1);
sampled("ArvR", "Queue1", "Class2", a2);
sampled("Util", "Queue1", "(aggr)", util);
sampled("Util", "Queue1", "Class1", util1);

const Matrix<double> estimate = run_ubr(m, queue, timestamps(n), {a1, a2}, util,
                                       {util1, std::vector<double>{}});
std::printf("Estimated demands: [[%.8f %.8f]]\n", estimate(0, 0), estimate(0, 1));
solve_after_estimate(m);
```

Running this edition prints

MODEL: model, Delay(servers=inf) Queue1(servers=1) Class1(N=1) Class2(N=2)
data ArvR Queue1 Class1 n=1000 mean=1.925

data ArvR	Queue1	Class2	n=1000	mean=2.875			
data Util	Queue1	(aggr)	n=1000	mean=0.975			
data Util	Queue1	Class1	n=1000	mean=0.77			
Estimated demands: [[0.40000000 0.07130435]]							
SOLVER: MVA							
Station	JobClass	QLen	Util	RespT	ResidT	ArvR	
Tput							
Delay	Class1	0.68656	0.68656	1	1	0.68656	
0.68656							
Delay	Class2	1.8205	1.8205	1	1	1.8205	
1.8205							
Queue1	Class1	0.31344	0.27462	0.45654	0.45654	0.68656	
0.68656							
Queue1	Class2	0.17953	0.12981	0.098616	0.098616	1.8205	
1.8205							

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by MVA.

8.1.16 est_ubr_open

The regression-based estimator on an open network.

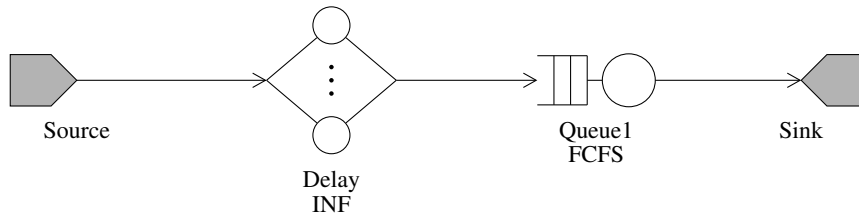


Figure 8.16: Topology of est_ubr_open: an open network over 2 queueing stations.

Nodes	4: Source, Queue, Delay, Sink
Classes	1: Class1 (open)
Scheduling	Delay: INF; Queue1: FCFS
Arrivals	Source – Class1: Exp(0.1)
Service	Delay – Class1: HyperExp(0.5,0.5;3,10) Queue1 – Class1: Exp(1.976)
Solvers	MVA

Table 8.16: Features of est_ubr_open.

The example is built and solved as follows.

```
std::size_t delay = 0, queue = 0;
Net m = open4(SchedStrategy::FCFS, {hyper(0.5, 3.0, 10.0)}, {Exp(0.1)}, delay, queue);
model_line(m);

const std::size_t n = 1000;
// Deterministic part only: 2 - rand*0.15 and 1 - rand*0.05, UNSEEDED.
const std::vector<double> arv = filled(n, 1.925);
const std::vector<double> util = filled(n, 0.975);
std::vector<double> respt(n, 0.0);
for (std::size_t i = 0; i < n; ++i) respt[i] = 0.7 / (1.0 - util[i]);
sampled("ArvR", "Queue1", "Class1", arv);
sampled("Respt", "Queue1", "Class1", respt);
sampled("Util", "Queue1", "(aggr)", util);

const Matrix<double> estimate = run_ubr(m, queue, timestamps(n), {arv}, util);
std::printf("Estimated demand: [[%.8f]]\n", estimate(0, 0));
solve_after_estimate(m);
```

Running this edition prints

```

MODEL: model, Delay(servers=inf) Queue1(servers=1) Source(servers=1) Class1(N=inf)
data ArvR Queue1 Class1 n=1000 mean=1.925
data RespT Queue1 Class1 n=1000 mean=28
data Util Queue1 (aggr) n=1000 mean=0.975
Estimated demand: [[0.50649351]]

```

```

SOLVER: MVA
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
  Tput
Delay        Class1        0.021667   0.021667   0.21667    0.21667    0.1
  0.1
Queue1       Class1        0.053351   0.050649   0.53351    0.53351    0.1
  0.1
Source       Class1         0          0          0          0          0
  0.1

```

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by MVA.

8.1.17 est_vi_closed

Variational inference over the transition counts of a closed loop, from noisy queue-length readings, returning a Gamma posterior per rate rather than a point estimate.

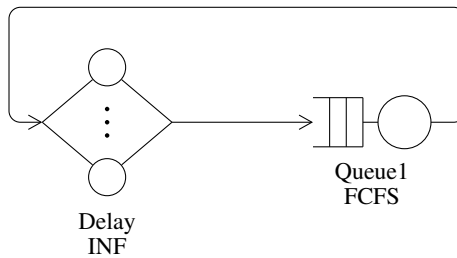


Figure 8.17: Topology of `est_vi_closed`: a closed network over 2 queueing stations.

Nodes	2: Queue, Delay
Classes	1: Class1 (closed, $N = 10$)
Scheduling	Delay: INF; Queue1: FCFS
Service	Delay – Class1: Exp(0.5) Queue1 – Class1: Exp(1.994)
Solvers	MVA

Table 8.17: Features of `est_vi_closed`.

The example is built and solved as follows.

```

const double N = 10.0;
Net m("model");
Delay delay(m, "Delay");
Queue queue(m, "Queue1", SchedStrategy::FCFS);
ClosedClass cl(m, "Class1", N, delay, 0);
m.set_service(delay, cl, D::exp_mean(2.0));
m.set_service(queue, cl, D::exp_mean(0.5)); // starting point of the estimate
Routing P;
P.set(cl, cl, delay, queue, 1.0);
P.set(cl, cl, queue, delay, 1.0);
m.link(P);

// queue-length readings, one per unit time, 10% of them faulty
const std::vector<double> ts = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
const std::vector<double> qlen = {1, 2, 3, 2, 4, 3, 5, 4, 3, 4};
std::vector<double> dlen(qlen.size());
for (std::size_t k = 0; k < qlen.size(); ++k) dlen[k] = N - qlen[k];
sampled("QLen", "Delay", "Class1", dlen);

```

```

sampled("QLen", "Queue1", "Class1", qlen);

infer::EstimatorOptions options;
options.method = "vi";
options.epsilon = 0.1;          // probability that a reading is faulty
options.prior_shape = 2.0;      // Gamma prior shape; the rate comes from the model
options.variational.ngrid = 51;
options.variational.nsamples = 32;

    ... (16 source lines omitted; full implementation: cpp/examples/inference/)

mva::MvaOptions mopt;
Matrix<double> init;
const mva::AvgResult<double> res = mva::solver_mva_run_analyzer(m.get_struct(), mopt, init);
section("MVA");
print_avg(m.get_struct(), res);

```

Running this edition prints

```

data QLen Delay Class1 n=10 mean=6.9
data QLen Queue1 Class1 n=10 mean=3.1
Estimated demand: 0.50152363
posterior service rate ~ Gamma(20.8739, 10.4688), mean 1.9939
evidence lower bound over the iterations: -89.393 -164.046 -163.711 -174.650 -163.234

SOLVER: MVA
Station      JobClass      QLen      Util      RespT      ResidT      ArvR
Tput
Delay        Class1      3.9671     3.9671     2           2           1.9835
1.9835
Queue1       Class1      6.0329     0.99479    3.0415     3.0415     1.9835
1.9835

```

The columns are the mean queue length, utilization, response time, residence time, arrival rate and throughput of each station-class pair, as returned by MVA.

Chapter 9

Optimization

Besides evaluating a given model, LINE can solve design problems on queueing networks. While the solvers answer evaluation questions (given a model, what are its performance metrics?), the optimization layer answers design questions: how many servers, what service rates, which routing probabilities, or what job populations optimize a cost or performance objective subject to service-level constraints. The reference material on decision variables, objectives, constraints and solvers is in the Optimization chapter of the LINE user manual; this chapter collects the worked examples.

The relevant scripts are included under the `cpp/examples/opt/opt.cpp` file and each runs in seconds. The general structure of an optimization script consists of four blocks:

1. Definition of the LINE network model
2. Declaration of the decision variables
3. Specification of the objective and constraints
4. Solution

The C++ implementation is exposed by the public `line::opt` headers. The complete buildable studies in `cpp/examples/opt/opt.cpp` exercise differential evolution, exact monotone bisection, scenarios, Pareto sweeps, decomposition, and layered host-demand optimization. The C++ listings below show the model setup; the source file is authoritative for each corresponding solver invocation and result printer.

9.1 Sizing an M/M/c queue

The M/M/c queue is a classic model of a service center with c parallel servers fed by a common infinite-capacity buffer. In this example, we wish to find the minimum number of servers such that the utilization does not exceed 50%. Jobs arrive at rate $\lambda = 3$ job/s and each server works at rate $\mu = 1$ job/s, hence the utilization with c servers is $\rho = \lambda/(c\mu) = 3/c$. It is known from theory that the optimum is therefore $c^* = 6$; we wish to recover this value. The following script (`opt_server_sizing`) solves the problem with the default differential evolution solver, at a cost of 10 units per server:

```
// Block 1: LINE model (Source -> Queue -> Sink)
Net m("MMc");
const std::size_t source = m.add_source("Arrivals");
const std::size_t queue = m.add_queue("Server", SchedStrategy::FCFS);
const std::size_t sink = m.add_sink("Departures");
const std::size_t jobs = m.add_open_class("Jobs");
m.set_arrival(source, jobs, D::exp_rate(3.0)); // lambda = 3
m.set_service(queue, jobs, D::exp_rate(1.0)); // mu = 1 per server
Routing P;
```

```

serial(P, {source, queue, sink});
m.link(P);
describe(m);

// Blocks 2-4: declared, then refused
variable("ServerAllocation(Server) in [1, 10]");
objective("MinimizeCost, 10.0 per server of Server");
constraint("Utilization(Server) <= 0.5");
note(" solver      : differential evolution, seed 42");
note(" reference answer: 6 servers, cost 60.0");
// The complete LineOptSolver invocation is in cpp/examples/opt/opt.cpp.

```

9.2 Exact sizing by bisection

The sizing problem of Tutorial 1 has a special structure: feasibility is monotone in the decision variable, since adding servers can only decrease utilization. In such cases `BisectionSolver` finds the exact optimum in $O(\log n)$ LINE evaluations. Model and problem definition are identical to Tutorial 1 (`opt_bisection_sizing`); only the solution block changes:

```

note(" solver      : BisectionSolver, monotone feasibility in the server count");
// The complete BisectionSolver invocation is in cpp/examples/opt/opt.cpp.

```

The reference answer is 6 servers at cost 60.0, reached in 4 LINE evaluations against the hundreds explored by differential evolution.

9.3 Continuous service rate tuning

We now choose the cheapest service rate, a continuous decision, meeting a response time SLA on an M/M/1 queue. With $\lambda = 3$, the response time is $R = 1/(\mu - 3)$, so $R \leq 0.5$ requires $\mu \geq 5$; the rate is billed at 20 cost units per unit, hence the optimum is $\mu^* = 5$ at cost 100. In the script (`opt_service_rate`), the model is an M/M/1 queue built as in Tutorial 1 with initial rate $\mu = 4$, and the problem is declared as:

```

Net m = mmc("MM1", 3.0, 4.0);
describe(m);
variable("ServiceRate(Server, Jobs) in [3.5, 8.0]");
objective("MinimizeCost, 20.0 per unit of rate at Server");
constraint("ResponseTime(Server, Jobs) <= 0.5");
note(" solver      : differential evolution, seed 42, 60 iterations");
note(" reference answer: rate 5.0, cost 100.0");
// The complete LineOptSolver invocation is in cpp/examples/opt/opt.cpp.

```

9.4 Load balancing across heterogeneous servers

An arrival stream of rate $\lambda = 2$ must be split between a fast queue ($\mu_1 = 4$) and a slow queue ($\mu_2 = 2.5$) so that the mean end-to-end response time is minimized. Modeling each queue as M/M/1, the mean number of jobs in the system is $N(p) = \frac{\rho_1}{1-\rho_1} + \frac{\rho_2}{1-\rho_2}$ with $\rho_1 = p\lambda/\mu_1$ and $\rho_2 = (1-p)\lambda/\mu_2$, and by Little's law the system response time is $N(p)/\lambda$. Setting $dN/dp = 0$ yields $p^* \approx 0.743$ with response time 0.4264: the optimal policy does not saturate the fast server. The script (`opt_load_balancing`) declares the routing split as the decision variable:

```

Net m("LoadBalance");
const std::size_t source = m.add_source("S");
const std::size_t fast = m.add_queue("Fast", SchedStrategy::PS);
const std::size_t slow = m.add_queue("Slow", SchedStrategy::PS);
const std::size_t sink = m.add_sink("K");

```

```

const std::size_t jobs = m.add_open_class("Jobs");
m.set_arrival(source, jobs, D::exp_rate(2.0));
m.set_service(fast, jobs, D::exp_rate(4.0));
m.set_service(slow, jobs, D::exp_rate(2.5));
Routing P;
// Two outgoing links and no probabilities set: the reference's default RAND.
P.set(jobs, jobs, source, fast, 0.5);
P.set(jobs, jobs, source, slow, 0.5);
P.set(jobs, jobs, fast, sink, 1.0);
P.set(jobs, jobs, slow, sink, 1.0);
m.link(P);

describe(m);
variable("RoutingProbabilities(Jobs, from S, to [Fast, Slow])");
objective("MinimizeSystemResponseTime(Jobs)");
note(" reference answer: 0.743 to Fast, system response time 0.4264");
// The complete LineOptSolver invocation is in cpp/examples/opt/opt.cpp.

```

9.5 Population sizing in a closed network

An interactive system is modeled as a closed network with a Delay station (mean think time 1) and a processor sharing server (rate 2). We seek the largest number of concurrent users the system can sustain while the server response time stays within an SLA of 2 time units. Feasibility is monotone non-increasing in the population, so bisection applies with the `max_feasible` direction (`opt_population_sizing`):

```

Net m("Interactive");
const std::size_t think = m.add_delay("Think");
const std::size_t server = m.add_queue("AppServer", SchedStrategy::PS);
const std::size_t users = m.add_closed_class("Users", 1, think);
m.set_service(think, users, D::exp_rate(1.0));
m.set_service(server, users, D::exp_rate(2.0));
Routing P;
cyclic(P, users, {think, server});
m.link(P);

describe(m);
variable("JobPopulation(Users) in [1, 50]");
objective("MinimizeCost (pure feasibility sizing)");
constraint("ResponseTime(AppServer, Users) <= 2.0");
note(" solver : BisectionSolver, direction max_feasible");
// The complete BisectionSolver invocation is in cpp/examples/opt/opt.cpp.

```

The reference answer is 5 users, reached in 6 LINE evaluations.

9.6 Robust sizing under workload scenarios

Capacity decisions should often hold under load uncertainty. Here the server pool of Tutorial 1 is sized so that the utilization SLA holds both under the average load ($\lambda = 3$) and under a peak-load scenario ($\lambda = 4.5$). Scenario models share the node and class names of the base model and are registered with `add_scenario`; the solver enforces every constraint on all scenarios (`opt_robust_sizing`):

```

Net base = mmc("MMc", 3.0, 1.0); // average load
Net peak = mmc("MMc", 4.5, 1.0); // peak load scenario
note("base scenario (average load)");
describe(base);
note("peak scenario");
describe(peak);

variable("ServerAllocation(Server) in [1, 12]");
objective("MinimizeCost, 10.0 per server of Server");
constraint("Utilization(Server) <= 0.5 on EVERY scenario");

```

```
note("  reference answer: 9 servers, against 6 for the average load alone");
// The complete scenario solve is in cpp/examples/opt/opt.cpp.
```

The average load alone needs 6 servers; the peak scenario pushes the answer to $\lceil 4.5/0.5 \rceil = 9$.

9.7 Cost-performance tradeoff frontier

How much does tightening the utilization SLA cost? `ParetoSweep` re-solves the sizing problem for a sweep of utilization bounds (epsilon-constraint method) and reports the non-dominated cost frontier, here with each point solved exactly by bisection (`opt_pareto_frontier`):

```
Net m = mmc("MMc", 3.0, 1.0);
describe(m);
variable("ServerAllocation(Server) in [1, 12]");
objective("MinimizeCost, 10.0 per server of Server");
constraint("Utilization(Server) <= eps, eps in {0.3, 0.4, 0.5, 0.6, 0.75, 0.9}");
note("  solver      : ParetoSweep, epsilon-constraint method, bisection per point");
// The complete ParetoSweep invocation is in cpp/examples/opt/opt.cpp.
```

9.8 Decomposing a multi-variable problem

Jointly choosing the number of servers and the service rate couples two variable types. The decomposition workflow splits them into blocks (servers first, then rates), solves the blocks in sequence with the other block's values held fixed, and cycles until the full penalized objective stabilizes, i.e., block coordinate descent (`opt_decomposition`):

```
Net m = mmc("MMc", 3.0, 1.0);
describe(m);
variable("ServerAllocation(Server) in [1, 10]");
variable("ServiceRate(Server, Jobs) in [1.0, 4.0]");
objective("MinimizeCost, 10.0 per server and 20.0 per unit of rate at Server");
constraint("ResponseTime(Server, Jobs) <= 0.5");
note("  solver      : DecompositionWorkflow, auto_decompose, 4 cycles, tolerance 1e-3");
// The complete DecompositionWorkflow invocation is in cpp/examples/opt/opt.cpp.
```

As with any coordinate descent scheme on a non-convex problem, the workflow converges to a block-wise optimum that may differ from the joint optimum; a joint solve on this instance finds cost 76. Decomposition remains preferable when the joint dimensionality is intractable.

9.9 LQN host demand tuning

The last tutorial of `cpp/examples/opt/opt.cpp` (`opt_lqn_hostdemand`) tunes the host demands of a three-task client/server layered queueing network, minimizing the system response time of the reference task subject to a processor utilization bound. It builds the LQN through `lqn::LqnBuilder`, compares finite-difference and analytic-sensitivity gradient modes, verifies an explicitly frozen layer, and runs the automatic layered workflow:

```
variable("HostDemand(AS1) in [0.02, 0.2]");
variable("HostDemand(AS2) in [0.01, 0.1]");
objective("MinimizeSystemResponseTime(T1)");
constraint("Utilization(P1) <= 0.95");
note("  solver      : gradient, lqn_gradient in {fd, partial_sens, partial_plus_fd}, seed 1");
note("  then        : frozen_layers=[P2], and the layered workflow with auto_freeze");
// See opt_lqn_hostdemand for all three gradient modes and layered solve.
```

9.10 The models behind the tutorials

The tutorials above are design problems, and each one optimizes a queueing network that the narrative describes but does not draw. This section gives that network for every script of `cpp/examples/opt/opt.cpp`: its topology, its features, and the output the script prints once the optimizer has run.

9.10.1 `opt_bisection_sizing`

The same sizing problem solved exactly by bisection, which is admissible because the constraint is monotone in the number of servers.

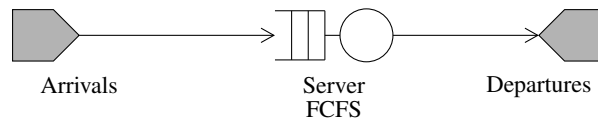


Figure 9.1: Topology of `opt_bisection_sizing`: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: Jobs (open)
Scheduling	Server: FCFS
Arrivals	Arrivals – Jobs: Exp(3)
Service	Server – Jobs: Exp(1)
Features	class switching on 3 links
Solvers	MVA

Table 9.1: Features of `opt_bisection_sizing`.

The example is built and solved as follows.

```
void opt_bisection_sizing() {
    Net m = mmc("MMc", 3.0, 1.0);
    describe(m);
    variable("ServerAllocation(Server) in [1, 10]");
    objective("MinimizeCost, 10.0 per server of Server");
    constraint("Utilization(Server) <= 0.5");
    note(" solver      : BisectionSolver, monotone feasibility in the server count");
    opt::OptimizationProblem problem(m);
    problem.add_variable(std::make_shared<opt::ServerAllocation>("Server", 1, 10))
        .set_objective(std::make_shared<opt::MinimizeCost>(
            std::map<std::string, double>{{"Server", 10.0}}))
        .add_constraint(std::make_shared<opt::UtilizationConstraint>("Server", 0.5));
    const opt::OptimizationResult result = opt::BisectionSolver(problem).solve();
    std::printf(" result      : servers=%g cost=%g feasible=%s evaluations=%zu\n",
        opt::scalar_value(result.variable_values.at("Server_servers")),
        result.objective_value, result.feasible ? "true" : "false",
        result.model_evaluations);
}
```

Running this edition prints

```
MODEL: MMc
station Arrivals    servers=1 Jobs: rate=3
station Server      servers=1 Jobs: rate=1
class Jobs          population=inf
variable : ServerAllocation(Server) in [1, 10]
objective : MinimizeCost, 10.0 per server of Server
constraint: Utilization(Server) <= 0.5
solver    : BisectionSolver, monotone feasibility in the server count
result    : servers=6 cost=60 feasible=true evaluations=4
```


9.10.2 opt_decomposition

Decomposing a multi-variable problem into blocks solved in turn, the coordinate-descent workflow and its blockwise optimum.

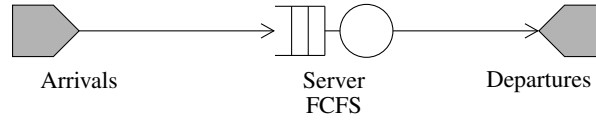


Figure 9.2: Topology of opt_decomposition: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: Jobs (open)
Scheduling	Server: FCFS
Arrivals	Arrivals – Jobs: Exp(3)
Service	Server – Jobs: Exp(1)
Features	class switching on 3 links
Solvers	MVA

Table 9.2: Features of opt_decomposition.

The example is built and solved as follows.

```
void opt_decomposition() {
    Net m = mmc("MMc", 3.0, 1.0);
    describe(m);
    variable("ServerAllocation(Server) in [1, 10]");
    variable("ServiceRate(Server, Jobs) in [1.0, 4.0]");
    objective("MinimizeCost, 10.0 per server and 20.0 per unit of rate at Server");
    constraint("ResponseTime(Server, Jobs) <= 0.5");
    note(" solver : DecompositionWorkflow, auto_decompose, 4 cycles, tolerance 1e-3");
    opt::OptimizationProblem problem(m);
    problem.add_variable(std::make_shared<opt::ServerAllocation>("Server", 1, 10))
        .add_variable(std::make_shared<opt::ServiceRate>("Server", "Jobs", 1.0, 4.0))
        .set_objective(std::make_shared<opt::MinimizeCost>(
            std::map<std::string, double>{{"Server", 10.0}},
            std::map<std::string, double>{{"Server", 20.0}}))
        .add_constraint(std::make_shared<opt::ResponseTimeConstraint>(
            "Server", "Jobs", 0.5));
    opt::LineOptSolverOptions options;
    options.set_seed(42).set_max_iterations(30).set_popsize(10);
    opt::DecompositionWorkflow workflow(problem);
    workflow.auto_decompose().set_solver_options(options);
    const opt::WorkflowResult result = workflow.solve_sequential(4, 1e-3);
    std::printf(" result : converged=%s cycles=%zu objective=%g\n",
        result.converged ? "true" : "false", result.cycles_completed,
        result.final_objective);
}
```

Running this edition prints

```
MODEL: MMc
station Arrivals    servers=1  Jobs: rate=3
station Server      servers=1  Jobs: rate=1
class  Jobs         population=inf
variable : ServerAllocation(Server) in [1, 10]
variable : ServiceRate(Server, Jobs) in [1.0, 4.0]
objective : MinimizeCost, 10.0 per server and 20.0 per unit of rate at Server
constraint: ResponseTime(Server, Jobs) <= 0.5
solver    : DecompositionWorkflow, auto_decompose, 4 cycles, tolerance 1e-3
result    : converged=true cycles=2 objective=100.054
```

9.10.3 opt_load_balancing

Load balancing across heterogeneous servers: the decision variables are the routing probabilities, which must form a simplex.

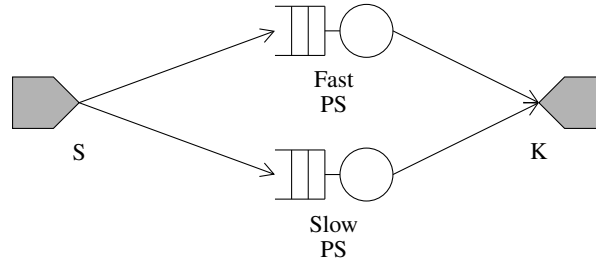


Figure 9.3: Topology of opt_load_balancing: an open network over 2 queueing stations.

Nodes	4: Source, Queue $\times$ 2, Sink
Classes	1: Jobs (open)
Scheduling	Fast: PS; Slow: PS
Arrivals	S – Jobs: Exp(2)
Service	Fast – Jobs: Exp(4) Slow – Jobs: Exp(2.5)
Features	class switching on 4 links
Solvers	MVA

Table 9.3: Features of opt_load_balancing.

The example is built and solved as follows.

```
void opt_load_balancing() {
    Net m("LoadBalance");
    Source source(m, "S");
    Queue fast(m, "Fast", SchedStrategy::PS);
    Queue slow(m, "Slow", SchedStrategy::PS);
    Sink sink(m, "K");
    OpenClass jobs(m, "Jobs");
    source.set_arrival(jobs, Exp(2.0));
    fast.set_service(jobs, Exp(4.0));
    slow.set_service(jobs, Exp(2.5));
    Routing P;
    // Two outgoing links and no probabilities set: the reference's default RAND.
    P.set(jobs, jobs, source, fast, 0.5);
    P.set(jobs, jobs, source, slow, 0.5);
    P.set(jobs, jobs, fast, sink, 1.0);
    P.set(jobs, jobs, slow, sink, 1.0);
    m.link(P);

    describe(m);
    variable("RoutingProbabilities(Jobs, from S, to [Fast, Slow])");
    objective("MinimizeSystemResponseTime(Jobs)");
    note(" solver      : differential evolution, seed 42, 60 iterations");
    note(" reference answer: 0.743 to Fast, system response time 0.4264");
    opt::OptimizationProblem problem(m);
    problem.add_variable(std::make_shared<opt::RoutingProbabilities>(
        "Jobs", "S", std::vector<std::string>{"Fast", "Slow"}));
    .set_objective(std::make_shared<opt::MinimizeSystemResponseTime>("Jobs"));
    opt::LineOptSolverOptions options;
    options.set_seed(42).set_max_iterations(60);
    const opt::OptimizationResult result = opt::LineOptSolver(problem, options).solve();
    const opt::Value& probabilities = result.variable_values.at("Jobs_routing_from_S");
    std::printf(" result      : fast=%g slow=%g response=%g feasible=%s\n",
        probabilities[0], probabilities[1], result.objective_value,
        result.feasible ? "true" : "false");
}
```

Running this edition prints

```
MODEL: LoadBalance
station S          servers=1  Jobs: rate=2
station Fast       servers=1  Jobs: rate=4
station Slow       servers=1  Jobs: rate=2.5
class Jobs         population=inf
variable : RoutingProbabilities(Jobs, from S, to [Fast, Slow])
objective : MinimizeSystemResponseTime(Jobs)
solver : differential evolution, seed 42, 60 iterations
reference answer: 0.743 to Fast, system response time 0.4264
result : fast=0.736233 slow=0.263767 response=0.424791 feasible=true
```

9.10.4 opt_lqn_host_demand

Host demand tuning on a layered model, with the gradient optimizer and layer freezing.

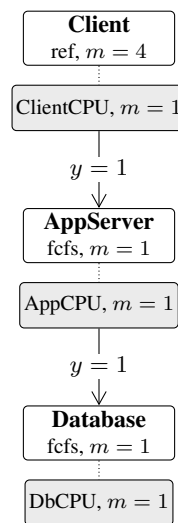


Figure 9.4: Call graph of `opt_lqn_host_demand`: 3 tasks over 3 processors, drawn with the reference task on top and a called task below its caller; the shaded box under each task is the processor it runs on.

Processors	3: ClientCPU (inf, $m = 1$), AppCPU (ps, $m = 1$), DbCPU (ps, $m = 1$)
Tasks	3: Client (ref, $m = 4$), AppServer (fcfs, $m = 1$), Database (fcfs, $m = 1$)
Entries	3: Browse, Render, Query
Activities	3, host demands BrowseAct: 0.5, RenderAct: 0.25, QueryAct: 0.2
Calls	2 (2 synchronous, 0 asynchronous)
Think times	Client: 1.0

Table 9.4: Features of `opt_lqn_host_demand`.

The example is built and solved as follows.

```
const Lqn m = bookstore_model();
const lqn::LqnStruct<double> ln = m.build();
std::printf("MODEL: BookstoreLQN, %zu processors, %zu tasks, %zu entries, %zu activities\n",
    ln.nhosts, ln.ntasks, ln.nentries, ln.nacts);
variable("HostDemand(RenderAct) in [0.05, 0.50]");
variable("HostDemand(QueryAct) in [0.05, 0.50]");
objective("MinimizeSystemResponseTime(Client)");
constraint("Budget: 1.0*RenderAct + 1.0*QueryAct <= 0.45");
note(" solver : gradient, lqn_gradient partial_plus_fd, fd_refresh 3, seed 7");

opt::OptimizationProblem problem(m.model());
```

```

std::map<std::string, double> cost;
cost["RenderAct_hostdemand"] = 1.0;
cost["QueryAct_hostdemand"] = 1.0;
problem.add_variable(std::make_shared<opt::HostDemand>("RenderAct", 0.05, 0.50))
    .add_variable(std::make_shared<opt::HostDemand>("QueryAct", 0.05, 0.50))
    .set_objective(std::make_shared<opt::MinimizeSystemResponseTime>("Client"))
    .add_constraint(std::make_shared<opt::BudgetConstraint>(0.45, cost));

opt::LineOptSolverOptions options;
options.set_seed(7)
    .set_optimizer("gradient")
    .set_lqn_gradient("partial_plus_fd")
    .set_fd_refresh(3)
    .set_max_iterations(8);
const opt::OptimizationResult result = opt::LineOptSolver(problem, options).solve();

... (3 source lines omitted; full implementation: cpp/examples/opt/)

std::printf("Query demand      : %.4f\n",
            opt::scalar_value(result.variable_values.at("QueryAct_hostdemand")));
std::printf("System RespT      : %.4f\n", result.objective_value);
std::printf("Feasible          : %d\n", result.feasible ? 1 : 0);
std::printf("Total violation    : %.4f\n", result.total_violation());
std::printf("LINE evaluations: %zu\n", result.model_evaluations);

```

No execution output has been captured for this language edition. Run the identified implementation to obtain its results.

9.10.5 opt_pareto_frontier

The cost-performance tradeoff frontier, swept rather than solved at one weighting, through ParetoSweep.

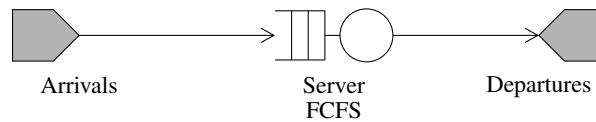


Figure 9.5: Topology of opt_pareto_frontier: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: Jobs (open)
Scheduling	Server: FCFS
Arrivals	Arrivals – Jobs: Exp(3)
Service	Server – Jobs: Exp(1)
Features	class switching on 3 links
Solvers	MVA

Table 9.5: Features of opt_pareto_frontier.

The example is built and solved as follows.

```

void opt_pareto_frontier() {
    Net m = mmc("MMc", 3.0, 1.0);
    describe(m);
    variable("ServerAllocation(Server) in [1, 12]");
    objective("MinimizeCost, 10.0 per server of Server");
    constraint("Utilization(Server) <= eps, eps in {0.3, 0.4, 0.5, 0.6, 0.75, 0.9}");
    note(" solver      : ParetoSweep, epsilon-constraint method, bisection per point");
    opt::OptimizationProblem problem(m);
    problem.add_variable(std::make_shared<opt::ServerAllocation>("Server", 1, 12))
        .set_objective(std::make_shared<opt::MinimizeCost>(
            std::map<std::string, double>{{"Server", 10.0}}));
}

```

```
opt::ParetoSweep sweep(problem,
    [](double epsilon) {
        return std::make_shared<opt::UtilizationConstraint>("Server", epsilon);
    }, {0.3, 0.4, 0.5, 0.6, 0.75, 0.9}, "bisection");
sweep.solve();
for (const opt::ParetoPoint& point : sweep.frontier())
    std::printf("  frontier : epsilon=%g cost=%g servers=%g\n", point.epsilon,
        point.objective_value,
        opt::scalar_value(point.result.variable_values.at("Server_servers")));
```

Running this edition prints

```
MODEL: MMc
station Arrivals      servers=1  Jobs: rate=3
station Server        servers=1  Jobs: rate=1
class  Jobs           population=inf
variable : ServerAllocation(Server) in [1, 12]
objective : MinimizeCost, 10.0 per server of Server
constraint: Utilization(Server) <= eps, eps in {0.3, 0.4, 0.5, 0.6, 0.75, 0.9}
solver    : ParetoSweep, epsilon-constraint method, bisection per point
frontier  : epsilon=0.3 cost=100 servers=10
frontier  : epsilon=0.4 cost=80 servers=8
frontier  : epsilon=0.5 cost=60 servers=6
frontier  : epsilon=0.6 cost=50 servers=5
frontier  : epsilon=0.75 cost=40 servers=4
```

9.10.6 opt_population_sizing

Population sizing in a closed network: how many jobs to admit so that throughput is maximized without breaching a residence-time bound.

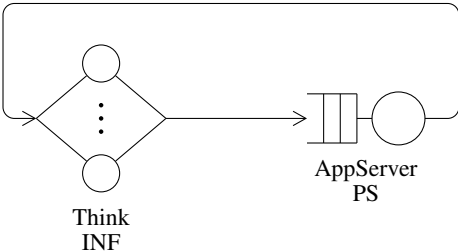


Figure 9.6: Topology of opt_population_sizing: a closed network over 2 queueing stations.

Nodes	2: Queue, Delay
Classes	1: Users (closed, $N = 1$)
Scheduling	Think: INF; AppServer: PS
Service	Think – Users: Exp(1) AppServer – Users: Exp(2)
Features	class switching on 2 links
Solvers	MVA

Table 9.6: Features of opt_population_sizing.

The example is built and solved as follows.

```
void opt_population_sizing() {
    Net m("Interactive");
    Delay think(m, "Think");
    Queue server(m, "AppServer", SchedStrategy::PS);
    ClosedClass users(m, "Users", 1, think);
    think.set_service(users, Exp(1.0));
    server.set_service(users, Exp(2.0));
    Routing P;
    cyclic(P, users, {think, server});
    m.link(P);
}
```

```

describe(m);
variable("JobPopulation(Users) in [1, 50]");
objective("MinimizeCost (pure feasibility sizing)");
constraint("ResponseTime(AppServer, Users) <= 2.0");
note(" solver      : BisectionSolver, direction max_feasible");
opt::OptimizationProblem problem(m);
problem.add_variable(std::make_shared<opt::JobPopulation>("Users", 1, 50))
    .set_objective(std::make_shared<opt::MinimizeCost>())
    .add_constraint(std::make_shared<opt::ResponseTimeConstraint>(
        "AppServer", "Users", 2.0));
const opt::OptimizationResult result =
    opt::BisectionSolver(problem, "max_feasible").solve();
std::printf(" result      : users=%g feasible=%s evaluations=%zu\n",
    opt::scalar_value(result.variable_values.at("Users_population")),
    result.feasible ? "true" : "false", result.model_evaluations);

```

Running this edition prints

```

MODEL: Interactive
station Think      servers=inf Users: rate=1
station AppServer  servers=1  Users: rate=2
class Users        population=1
variable : JobPopulation(Users) in [1, 50]
objective : MinimizeCost (pure feasibility sizing)
constraint: ResponseTime(AppServer, Users) <= 2.0
solver    : BisectionSolver, direction max_feasible
result    : users=5 feasible=true evaluations=6

```

9.10.7 opt_robust_sizing

Robust sizing, in which the constraint must hold across a set of workload scenarios rather than at one nominal load.

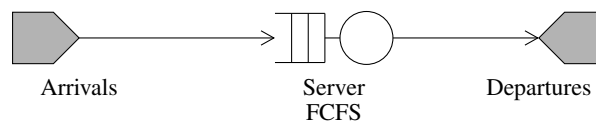


Figure 9.7: Topology of opt_robust_sizing: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: Jobs (open)
Scheduling	Server: FCFS
Arrivals	Arrivals – Jobs: Exp(3)
Service	Server – Jobs: Exp(1)
Features	class switching on 3 links
Solvers	MVA

Table 9.7: Features of opt_robust_sizing.

The example is built and solved as follows.

```

void opt_robust_sizing() {
    Net base = mmc("MMc", 3.0, 1.0);
    Net peak = mmc("MMc", 4.5, 1.0);
    note("base scenario (average load)");
    describe(base);
    note("peak scenario");
    describe(peak);
    variable("ServerAllocation(Server) in [1, 12]");
    objective("MinimizeCost, 10.0 per server of Server");
    constraint("Utilization(Server) <= 0.5 on EVERY scenario");
    note(" solver      : BisectionSolver over the base model plus one scenario");
}

```

```

note(" reference answer: 9 servers, against 6 for the average load alone");
opt::OptimizationProblem problem(base);
problem.add_variable(std::make_shared<opt::ServerAllocation>("Server", 1, 12))
    .set_objective(std::make_shared<opt::MinimizeCost>(
        std::map<std::string, double>{{"Server", 10.0}}))
    .add_constraint(std::make_shared<opt::UtilizationConstraint>("Server", 0.5))
    .add_scenario(peak);
const opt::OptimizationResult result = opt::BisectionSolver(problem).solve();
std::printf(" result      : servers=%g cost=%g feasible=%s\n",
    opt::scalar_value(result.variable_values.at("Server_servers")),
    result.objective_value, result.feasible ? "true" : "false");

```

Running this edition prints

```

base scenario (average load)
MODEL: MMc
  station Arrivals      servers=1  Jobs: rate=3
  station Server        servers=1  Jobs: rate=1
  class Jobs            population=inf
peak scenario
MODEL: MMc
  station Arrivals      servers=1  Jobs: rate=4.5
  station Server        servers=1  Jobs: rate=1
  class Jobs            population=inf
variable : ServerAllocation(Server) in [1, 12]
objective : MinimizeCost, 10.0 per server of Server
constraint: Utilization(Server) <= 0.5 on EVERY scenario
solver    : BisectionSolver over the base model plus one scenario
reference answer: 9 servers, against 6 for the average load alone
result    : servers=9 cost=90 feasible=true

```

9.10.8 opt_sensitivity_report

The sensitivity of the optimum to each parameter, reported alongside the solution.

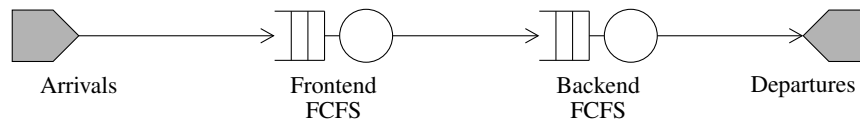


Figure 9.8: Topology of opt_sensitivity_report: an open network over 2 queueing stations.

Nodes	4: Source, Queue × 2, Sink
Classes	1: Jobs (open)
Scheduling	Frontend: FCFS; Backend: FCFS
Arrivals	Arrivals – Jobs: Exp(1)
Service	Frontend – Jobs: Exp(2) Backend – Jobs: Exp(1.5)

Table 9.8: Features of opt_sensitivity_report.

The example is built and solved as follows.

```

void opt_sensitivity_report() {
    Net model("Tandem");
    Source source(model, "Arrivals");
    Queue frontend(model, "Frontend", SchedStrategy::FCFS);
    Queue backend(model, "Backend", SchedStrategy::FCFS);
    Sink sink(model, "Departures");
    OpenClass jobs(model, "Jobs");
    source.set_arrival(jobs, Exp(1.0));
    frontend.set_service(jobs, Exp(2.0));
    backend.set_service(jobs, Exp(1.5));
    Routing routing;
    serial(routing, {source, frontend, backend, sink});
    model.link(routing);
}

```

```

opt::LineEvaluator evaluator(model, {});
const opt::EvaluationResult result = evaluator.evaluate({});
std::printf("Solver used : %s\n", result.solver_used.c_str());
std::printf("Feasible      : %s\n", result.feasible ? "true" : "false");
std::printf("Frontend Q   : %.4f\n", result.queue_length("Frontend"));
std::printf("Backend Q    : %.4f\n", result.queue_length("Backend"));
const auto sensitivity = opt::compute_model_sensitivities(model);
if (!sensitivity) {
    note("No analytic sensitivities available for this model.");
    return;
}
for (const std::string kind : {"QLen", "RespT", "Util", "Tput"}) {
    const opt::SensitivityData::MetricMap* metrics = sensitivity->for_kind(kind);
    if (!metrics) continue;
    std::printf("\n=== d(%s)/d(rate) ===\n", kind.c_str());
    for (const auto& metric : *metrics)
        for (const auto& parameter : metric.second)
            std::printf("  %-24s %-32s % .6f\n", metric.first.c_str(),
                parameter.first.c_str(), parameter.second);
}

```

Running this edition prints

```

Solver used : egflin
Feasible      : true
Frontend Q   : 1.0000
Backend Q    : 2.0000

=== d(QLen)/d(rate) ===
Backend||Jobs      rate||Backend||Jobs      -4.000000
Frontend||Jobs     rate||Frontend||Jobs     -1.000000

=== d(RespT)/d(rate) ===
Backend||Jobs      rate||Backend||Jobs      -4.000000
Frontend||Jobs     rate||Frontend||Jobs     -1.000000

=== d(Util)/d(rate) ===
Backend            rate||Backend||Jobs      -0.444444
Frontend           rate||Frontend||Jobs     -0.250000

=== d(Tput)/d(rate) ===
Backend||Jobs      rate||Backend||Jobs      0.000000
Frontend||Jobs     rate||Frontend||Jobs     0.000000

```

9.10.9 opt_server_sizing

Sizing an M/M/c queue: choose the number of servers that minimizes cost subject to a response-time bound, by differential evolution over an integer decision variable.

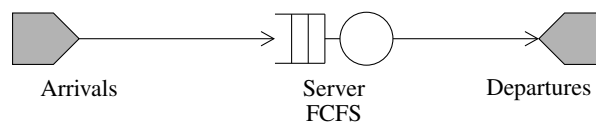


Figure 9.9: Topology of opt_server_sizing: an open network over 1 queueing station.

The example is built and solved as follows.

```

void opt_server_sizing() {
    Net m = mmc("MMC", 3.0, 1.0);
    describe(m);
    variable("ServerAllocation(Server) in [1, 10]");
    objective("MinimizeCost, 10.0 per server of Server");
    constraint("Utilization(Server) <= 0.5");
    note(" solver      : differential evolution, seed 42");
    note(" reference answer: 6 servers, cost 60.0");
    opt::OptimizationProblem problem(m);
}

```


Nodes	3: Source, Queue, Sink
Classes	1: Jobs (open)
Scheduling	Server: FCFS
Arrivals	Arrivals – Jobs: Exp(3)
Service	Server – Jobs: Exp(1)
Features	class switching on 3 links
Solvers	MVA

Table 9.9: Features of `opt_server_sizing`.

```

problem.add_variable(std::make_shared<opt::ServerAllocation>("Server", 1, 10))
    .set_objective(std::make_shared<opt::MinimizeCost>(
        std::map<std::string, double>{{"Server", 10.0}}))
    .add_constraint(std::make_shared<opt::UtilizationConstraint>("Server", 0.5));
opt::LineOptSolverOptions options;
options.set_seed(42).set_max_iterations(60);
const opt::OptimizationResult result = opt::LineOptSolver(problem, options).solve();
std::printf("  result      : servers=%g cost=%g feasible=%s evaluations=%zu\n",
    opt::scalar_value(result.variable_values.at("Server_servers")),
    result.objective_value, result.feasible ? "true" : "false",
    result.model_evaluations);

```

Running this edition prints

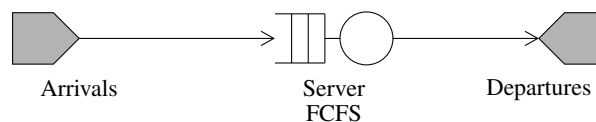
```

MODEL: MMc
station Arrivals      servers=1  Jobs: rate=3
station Server        servers=1  Jobs: rate=1
class  Jobs           population=inf
variable : ServerAllocation(Server) in [1, 10]
objective : MinimizeCost, 10.0 per server of Server
constraint: Utilization(Server) <= 0.5
solver    : differential evolution, seed 42
reference answer: 6 servers, cost 60.0
result     : servers=6 cost=60 feasible=true evaluations=10

```

9.10.10 `opt_service_rate`

Continuous tuning of a service rate, the archetype of a continuous decision variable under a service-level constraint.

Figure 9.10: Topology of `opt_service_rate`: an open network over 1 queueing station.

Nodes	3: Source, Queue, Sink
Classes	1: Jobs (open)
Scheduling	Server: FCFS
Arrivals	Arrivals – Jobs: Exp(3)
Service	Server – Jobs: Exp(4)
Features	class switching on 3 links
Solvers	MVA

Table 9.10: Features of `opt_service_rate`.

The example is built and solved as follows.

```

void opt_service_rate() {
    Net m = mmc("MM1", 3.0, 4.0);
}

```

```

describe(m);
variable("ServiceRate(Server, Jobs) in [3.5, 8.0]");
objective("MinimizeCost, 20.0 per unit of rate at Server");
constraint("ResponseTime(Server, Jobs) <= 0.5");
note(" solver      : differential evolution, seed 42, 60 iterations");
note(" reference answer: rate 5.0, cost 100.0");
opt::OptimizationProblem problem(m);
problem.add_variable(std::make_shared<opt::ServiceRate>(
    "Server", "Jobs", 3.5, 8.0))
    .set_objective(std::make_shared<opt::MinimizeCost>(
        std::map<std::string, double>(),
        std::map<std::string, double>{{"Server", 20.0}}))
    .add_constraint(std::make_shared<opt::ResponseTimeConstraint>(
        "Server", "Jobs", 0.5));
opt::LineOptSolverOptions options;
options.set_seed(42).set_max_iterations(60);
const opt::OptimizationResult result = opt::LineOptSolver(problem, options).solve();
std::printf(" result      : rate=%g cost=%g feasible=%s\n",
    opt::scalar_value(result.variable_values.at("Server_Jobs_rate")),
    result.objective_value, result.feasible ? "true" : "false");

```

Running this edition prints

```

MODEL: MM1
station Arrivals      servers=1  Jobs: rate=3
station Server        servers=1  Jobs: rate=4
class  Jobs           population=inf
variable : ServiceRate(Server, Jobs) in [3.5, 8.0]
objective : MinimizeCost, 20.0 per unit of rate at Server
constraint: ResponseTime(Server, Jobs) <= 0.5
solver    : differential evolution, seed 42, 60 iterations
reference answer: rate 5.0, cost 100.0
result     : rate=5.00078 cost=100.016 feasible=true

```

Index

- ag_closed_network, 269
- ag_gnetwork, 270
- ag_jackson_network, 271
- ag_multiclass_closed, 272
- ag_tandem_open, 273
- ag_tandem_phasetype, 275
- agent models, 269
- analytic gradient, 374

- bisection, 372
- block coordinate descent, 374
- busy period, 234
- busyp_subnetwork, 235

- Cache node, 121
- cache task, 250
- cache_compare_replc, 121
- cache_itemsize_costcap, 122
- cache_mmap_rr_env, 123
- cache_replc_climb, 125
- cache_replc_fifo, 126
- cache_replc_hlru, 127
- cache_replc_lru, 128
- cache_replc_qlru, 129
- cache_replc_routing, 130
- cache_replc_rr, 132
- cache_rmf_transient, 133
- cachenet_tandem, 134
- caching, 121, 279
- capacity planning, 373
- cdf_respt_closed, 227
- cdf_respt_closed_threeclasses, 229
- cdf_respt_distrib, 230
- cdf_respt_open_twoclasses, 232
- cdf_respt_populations, 233
- cl_basic, 145
- cl_closed, 146
- cl_compare, 147
- cl_multiclass, 148
- cl_scheduling, 149
- cl_stations, 150
- cl_sweep, 151

- class priorities, 60
- class switching, 54
- ClassSwitch node, 54
- closed queueing network, 20
- cluster network, 145
- constraint, 371
- cqn_bas_blocking, 21
- cqn_bcmp_theorem, 22
- cqn_lcfs_multiclass, 23
- cqn_mmpp2_service, 24
- cqn_multichain_cs, 26
- cqn_multiserver, 27
- cqn_multiserver_nc, 30
- cqn_online, 31
- cqn_repairmen, 32
- cqn_repairmen_multi, 34
- cqn_scheduling_dps, 35
- cqn_threeclass_hyperl, 37
- cqn_twoclass_erl, 38
- cqn_twoclass_hyperl, 40
- cqn_twoqueues_multi, 41
- cross-codebase parity, 1
- cs_implicit, 54
- cs_multi_diamond, 55
- cs_single_diamond, 56
- cs_transient_class, 58
- CTMC solver, 337
- ctmc_spn_mm1, 337
- ctmc_spn_vs_nc_closed_qn, 338
- ctmc_tandem_mm1, 340
- custom solver, 277
- cyclic polling, 222

- decision variable, 371
- design problem, 371, 375
- differential evolution, 371
- discrete time, 330
- dispatch_closed, 152
- dispatch_mixed, 153
- dispatch_open, 154
- distributions
 - Geometric, 330

[dt_bernoulli_loaddep](#), 330
[dt_cycle](#), 331
[dt_cycle_loaddep](#), 332
[dt_cycle_multiclass](#), 333
[dt_geogeol](#), 335
[dt_geogeol_loss](#), 335

epsilon-constraint method, 374
[est_ekf_changepoint](#), 350
[est_ekf_closed](#), 351
[est_ekf_open](#), 353
[est_erps_closed](#), 354
[est_erps_open](#), 355
[est_mcmc_closed](#), 356
[est_mle_open](#), 357
[est_qmle_closed](#), 358
[est_trace_closed](#), 359
[est_ubo_changepoint](#), 361
[est_ubo_closed](#), 362
[est_ubo_open](#), 363
[est_ubo_variants_closed](#), 364
[est_ubr_changepoint](#), 366
[est_ubr_closed](#), 367
[est_ubr_open](#), 368
[est_vi_closed](#), 369
[example_mapqn2renv](#), 239
Extended Kalman Filter, 350

[fcr_constraints](#), 192
[fcr_lincon](#), 193
[fcr_lossn](#), 194
[fcr_mmlkdrop](#), 195
[fcr_mmlwaitq](#), 196
[fcr_oqndrop](#), 197
[fcr_oqnwaitq](#), 198
feature table, 1
[fes_aggregation](#), 176
[fes_single_class](#), 178
file formats
 JSON, 342
finite capacity region, 192, 279
finite differences, 374
[fj_asymm](#), 67
[fj_basic_closed](#), 68
[fj_basic_nesting](#), 69
[fj_basic_open](#), 70
[fj_complex_serial](#), 72
[fj_cs_multi_visits](#), 73
[fj_cs_postfork](#), 74
[fj_cs_prefork](#), 75

[fj_deep_nesting](#), 76
[fj_delays](#), 78
[fj_mixed_openclosed](#), 79
[fj_nojoin](#), 80
[fj_route_overlap](#), 81
[fj_serialfjs_closed](#), 83
[fj_serialfjs_open](#), 84
[fj_threebranches](#), 85
[fj_tiny_closed](#), 86
[fj_twoclasses_forked](#), 88
[fj_variable_fanout](#), 89
flow-equivalent server, 176
Fork node, 67
fork-join, 67, 279

[gallery_aphm1](#), 280
[gallery_cache_lru](#), 281
[gallery_cache_routing](#), 282
[gallery_coxm1](#), 283
[gallery_cqn](#), 283
[gallery_cqn_multiclass](#), 284
[gallery_detm1](#), 285
[gallery_dm1](#), 285
[gallery_erldk](#), 286
[gallery_erlerl1](#), 287
[gallery_erlerl1_reentrant](#), 287
[gallery_erlm1](#), 288
[gallery_erlm1_ps](#), 289
[gallery_erlm1_reentrant](#), 289
[gallery_erlm1ps](#), 289
[gallery_fcr](#), 290
[gallery_fj_closed](#), 291
[gallery_fj_open](#), 292
[gallery_fj_quorum](#), 293
[gallery_gamm1](#), 294
[gallery_hyperl1_feedback](#), 294
[gallery_hyperl1_reentrant](#), 295
[gallery_hyperlk](#), 296
[gallery_hyphyp1_linear](#), 296
[gallery_hyphyp1_reentrant](#), 297
[gallery_hyphyp1_tandem](#), 297
[gallery_hypm1](#), 298
[gallery_hypm1_reentrant](#), 298
[gallery_lqn_basic](#), 299
[gallery_lqn_random](#), 300
[gallery_lqn_workflows](#), 301
[gallery_lukumar_reentrant](#), 303
[gallery_mapm1](#), 304
[gallery_mapmk](#), 304
[gallery_mdk](#), 305

[gallery_merll](#), [305](#)
[gallery_merll_linear](#), [305](#)
[gallery_merll_reentrant](#), [306](#)
[gallery_merll_tandem](#), [306](#)
[gallery_merlk](#), [308](#)
[gallery_mhyp1](#), [308](#)
[gallery_mhyp1_linear](#), [308](#)
[gallery_mhyp1_reentrant](#), [309](#)
[gallery_mhyp1_tandem](#), [309](#)
[gallery_mhypk](#), [311](#)
[gallery_mm1](#), [311](#)
[gallery_mm1_feedback](#), [312](#)
[gallery_mm1_linear](#), [312](#)
[gallery_mm1_multiclass](#), [313](#)
[gallery_mm1_prio](#), [314](#)
[gallery_mm1_ps](#), [314](#)
[gallery_mm1_ps_feedback](#), [315](#)
[gallery_mm1_ps_multiclass](#), [316](#)
[gallery_mm1_ps_reentrant](#), [316](#)
[gallery_mm1_reentrant](#), [317](#)
[gallery_mm1_tandem](#), [317](#)
[gallery_mm1_tandem_multiclass](#), [318](#)
[gallery_mm1k](#), [319](#)
[gallery_mmap1](#), [319](#)
[gallery_mmap1_multiclass](#), [320](#)
[gallery_mmapk](#), [320](#)
[gallery_mmk](#), [322](#)
[gallery_mpar1](#), [322](#)
[gallery_multitier](#), [323](#)
[gallery_multitier_storage](#), [324](#)
[gallery_parm1](#), [325](#)
[gallery_qn_random](#), [326](#)
[gallery_renv_breakdown](#), [327](#)
[gallery_repairmen](#), [327](#)
[gallery_replayerm1](#), [328](#)
[gallery_um1](#), [329](#)
[Geo/Geo/1 queue](#), [330](#)
[Geometric distribution](#), [330](#)

[HOL scheduling](#), [60](#)

[infinitesimal generator](#), [337](#)
[init_state_fcfs_exp](#), [208](#)
[init_state_fcfs_nonexp](#), [209](#)
[init_state_ps](#), [210](#)

[Join node](#), [67](#)
[JSON format](#), [342](#)
[json_balking_retrial](#), [342](#)
[json_classcap_droprule](#), [343](#)
[json_fcr_details](#), [344](#)

[json_hetero_servers](#), [346](#)
[json_join_deadline](#), [347](#)
[json_signal_classes](#), [348](#)

[Kendall notation](#), [279](#)

[large-scale models](#), [276](#)
[largescale_tbi](#), [276](#)
[layered queueing network](#), [90](#), [279](#)
[lcq_async_cache](#), [250](#)
[lcq_singlehost](#), [252](#)
[lcq_threehosts](#), [253](#)
[ld_class_dependence](#), [179](#)
[ld_fes_multiclass](#), [180](#)
[ld_fes_singleclass](#), [182](#)
[ld_global_dependence](#), [183](#)
[ld_joint_dependence](#), [184](#)
[ld_multiserver_fcfs](#), [186](#)
[ld_multiserver_ps](#), [187](#)
[ld_multiserver_ps_twoclasses](#), [189](#)
[ld_whittle_bandwidth](#), [190](#)
[ldes_warmstart](#), [212](#)
[load balancing](#), [372](#), [375](#)
[load dependence](#), [176](#)
[lqn_basic](#), [90](#)
[lqn_bpmn](#), [91](#)
[lqn_bpmn_trace](#), [93](#)
[lqn_flatph](#), [95](#)
[lqn_fork_open_arrival](#), [97](#)
[lqn_init](#), [98](#)
[lqn_jsq](#), [100](#)
[lqn_moment3](#), [101](#)
[lqn_multi_solvers](#), [102](#)
[lqn_ofbiz](#), [103](#)
[lqn_open_arrival](#), [106](#)
[lqn_paramident](#), [107](#)
[lqn_rrobin](#), [108](#)
[lqn_serial](#), [109](#)
[lqn_server_pools](#), [110](#)
[lqn_setup](#), [111](#)
[lqn_sockshop](#), [112](#)
[lqn_srvnph](#), [115](#)
[lqn_transient](#), [116](#)
[lqn_twotasks](#), [118](#)
[lqn_workflows](#), [119](#)
[Lu-Kumar re-entrant line](#), [279](#)

[mam_transient_mapmap1](#), [2](#)
[maximum likelihood](#), [350](#)
[MCMC](#), [350](#)
[mixed network](#), [43](#)

model features

- caching, [121](#), [279](#)
- class priorities, [60](#)
- class switching, [54](#)
- cyclic polling, [222](#)
- finite capacity region, [192](#), [279](#)
- fork-join, [67](#), [279](#)
- load balancing, [372](#), [375](#)
- load dependence, [176](#)
- random environment, [239](#), [279](#)
- reward models, [264](#)
- slotted time, [330](#)
- state-dependent routing, [213](#)
- switchover times, [226](#)
- warm start, [208](#)

model gallery, [279](#)**model interchange, [342](#)**

- mqn_basic, [43](#)
- mqn_multichain_cs, [44](#)
- mqn_multiserver_fcfs, [47](#)
- mqn_multiserver_ps, [48](#)
- mqn_singleserver_fcfs, [50](#)
- mqn_singleserver_ps, [52](#)

node types

- Cache, [121](#)
- ClassSwitch, [54](#)
- Fork, [67](#)
- Join, [67](#)
- Place, [155](#)
- Transition, [155](#)

objective function, [371](#)**open queueing network, [2](#)**

- opt_bisection_sizing, [375](#)
- opt_decomposition, [376](#)
- opt_load_balancing, [377](#)
- opt_lqn_host_demand, [378](#)
- opt_pareto_frontier, [379](#)
- opt_population_sizing, [380](#)
- opt_robust_sizing, [381](#)
- opt_sensitivity_report, [382](#)
- opt_server_sizing, [383](#)
- opt_service_rate, [384](#)
- oqn_basic, [4](#)
- oqn_bmmapt, [5](#)
- oqn_cs_routing, [6](#)
- oqn_fourqueues, [8](#)
- oqn_mapt, [10](#)
- oqn_mapt_service, [11](#)

- oqn_mmapt, [13](#)

- oqn_multichain_cs, [14](#)

- oqn_nhpp, [15](#)

- oqn_online, [17](#)

- oqn_trace_driven, [18](#)

- oqn_vsinks, [19](#)

parameter inference, [350](#)**Pareto frontier, [374](#), [375](#)**

- pas_closed_tandem_fig6, [255](#)

- pas_compatibility_5class, [256](#)

- pas_cyclic_ctmc_vs_bruteforce, [257](#)

- pas_cyclic_normconst_bruteforce, [258](#)

- pas_mmk, [259](#)

- pas_nc_sampling, [260](#)

- pas_saturation, [261](#)

- pas_selfloop, [263](#)

pass and swap, [254](#)**Place node, [155](#)**

- polling_exhaustive_det, [222](#)

- polling_exhaustive_exp, [223](#)

- polling_gated, [224](#)

- polling_klimited, [225](#)

- prio_hol_closed, [61](#)

- prio_hol_open, [62](#)

- prio_identical, [64](#)

- prio_psprio, [66](#)

QMLE, [350](#)**random environment, [239](#), [279](#)**

- renv_basic, [240](#)

- renv_container_terminal, [241](#)

- renv_fourstages_repairmen, [242](#)

- renv_genqn, [243](#)

- renv_lqn_twostages, [244](#)

- renv_map_fallback, [245](#)

- renv_node_breakdown, [246](#)

- renv_rotterdam_blending, [247](#)

- renv_threestages_repairmen, [247](#)

- renv_twostages_repairmen, [249](#)

response time distribution, [227](#)

- retrieval_chain, [136](#)

- retrieval_default, [138](#)

- retrieval_ps, [139](#)

- retrieval_routing, [142](#)

- retrieval_simple, [143](#)

reward models, [264](#)

- rewardModel_aggregation, [264](#)

- rewardModel_mmlk, [265](#)

- [rewardModel_multiclass](#), 266
- [rewardModel_templates](#), 267
- [round-trip fixture](#), 342
- [routing matrix](#), 1
- [runnable entry](#), 1
- [scheduling strategies](#)
 - [HOL](#), 60
- [script structure](#), 1
- [sdroute_closed](#), 213
- [sdroute_jsq](#), 215
- [sdroute_krzesinski](#), 216
- [sdroute_multibranch](#), 218
- [sdroute_open](#), 219
- [sdroute_twoclasses_closed](#), 220
- [serial routing](#), 1
- [server sizing](#), 371, 375
- [slot](#), 330
- [slotted time](#), 330
- [snc_delay_quantile](#), 236
- [snc_tandem_multiclass](#), 237
- [solution methods](#)
 - [bisection](#), 372
 - [block coordinate descent](#), 374
 - [differential evolution](#), 371
 - [epsilon-constraint method](#), 374
 - [Extended Kalman Filter](#), 350
 - [flow-equivalent server](#), 176
 - [maximum likelihood](#), 350
 - [MCMC](#), 350
 - [QMLE](#), 350
 - [stochastic network calculus](#), 236
 - [variational inference](#), 350
- [solver_custom](#), 277
- [solver_custom_analyzer](#), 278
- [solvers](#)
 - [CTMC](#), 337
- [spn_basic_closed](#), 155
- [spn_basic_open](#), 156
- [spn_closed_fourplaces](#), 157
- [spn_closed_twoplaces](#), 158
- [spn_colored_gspn](#), 159
- [spn_fluid_dae](#), 160
- [spn_fourmodes](#), 161
- [spn_inhibiting](#), 162
- [spn_lpbounds](#), 164
- [spn_open_sevenplaces](#), 165
- [spn_pareto_service](#), 166
- [spn_productform_nc](#), 167
- [spn_queueing_place](#), 168
- [spn_twomodes](#), 169
- [state probability](#), 199
- [state-dependent routing](#), 213
- [statepr_aggr](#), 199
- [statepr_aggr_large](#), 200
- [statepr_allprobs_fcfs](#), 202
- [statepr_allprobs_ps](#), 203
- [statepr_sys_aggr](#), 203
- [statepr_sys_aggr_large](#), 205
- [statepr_sys_marg](#), 206
- [stochastic network calculus](#), 236
- [stochastic Petri net](#), 155, 341
- [switchover times](#), 226
- [switchover_basic](#), 226
- [test_spn_nrm_open](#), 170
- [topology figure](#), 1
- [Transition node](#), 155
- [variational inference](#), 350
- [warm start](#), 208
- [wf_aph](#), 171
- [wf_branch](#), 172
- [wf_complex](#), 172
- [wf_erlang](#), 173
- [wf_loop](#), 174
- [wf_parallel](#), 174
- [wf_serial](#), 175
- [wire key](#), 349
- [worked example](#), 1
- [workflow model](#), 171
- [workload scenario](#), 373